

The Pearson logo, consisting of the word "PEARSON" in a white, sans-serif font inside a blue rectangular box with a thin white border.

PEARSON

A graphic showing a world map with various colored arcs (red, orange, yellow, green, blue) connecting different points across the globe, representing a global network or data flow. Binary code (0s and 1s) is visible in the background.

Introduction to Database Systems

A decorative graphic in the bottom right corner consisting of several overlapping, semi-transparent squares in shades of blue and green, arranged in a cluster.

ITL Education Solutions Limited

Introduction to Database Systems



ITL Education Solutions Ltd.
Research and Development Wing

PEARSON

Delhi • Chennai • Chandigarh

Copyright © 2010 Dorling Kindersley (India) Pvt. Ltd

This book is sold subject to the condition that it shall not, by way of trade or otherwise, be lent, resold, hired out, or otherwise circulated without the publisher's prior written consent in any form of binding or cover other than that in which it is published and without a similar condition including this condition being imposed on the subsequent purchaser and without limiting the rights under copyright reserved above, no part of this publication may be reproduced, stored in or introduced into a retrieval system, or transmitted in any form or by any means (electronic, mechanical, photocopying, recording or otherwise), without the prior written permission of both the copyright owner and the above-mentioned publisher of this book.

ISBN 978-81-317-3192-5

10 9 8 7 6 5 4 3 2 1

Published by Dorling Kindersley (India) Pvt. Ltd., licensees of Pearson Education in South Asia

Head Office: 7th Floor, Knowledge Boulevard, A-8 (A), Sector 62, NOIDA, 201 309, UP, India.

Registered Office: 11 Community Centre, Panchsheel Park, New Delhi 110 017, India

Typeset by AcePro India Pvt. Ltd

Printed in India

Contents

<i>Preface</i>	ix
Chapter 1. DATABASE SYSTEM	1
1.1 Need of Database System	2
1.2 Advantages of Database System	3
1.3 Developments in Database System	5
1.4 Application Areas of Database System	5
1.5 Cost and Risk of Database System	6
1.6 People Who Interact with Databases	6
1.7 DBMS Architecture and Data Independence	8
1.8 Database Models	10
1.9 Database Schema versus Database Instance	14
1.10 DBMS Languages	15
1.11 Component Modules of DBMS	16
1.12 Centralized and Client/Server Database Systems	18
1.13 Classification of DBMSs	19
1.14 Database Design Process	20
<i>Summary</i>	21
<i>Key Terms</i>	23
<i>Exercises</i>	24
Chapter 2. CONCEPTUAL MODELING	27
2.1 Concepts of Entity-Relationship Model	28
2.2 Entity-Relationship Diagram	36
2.3 Enhanced E-R Model	45
2.4 Alternative Notations for E-R Diagrams	51
2.5 Unified Modeling Language	52
<i>Summary</i>	56
<i>Key Terms</i>	57
<i>Exercises</i>	58
Chapter 3. THE RELATIONAL MODEL	63
3.1 Relational Model Concepts	64
3.2 Relational Database Schema	68
3.3 Relational Database Instance	69
3.4 Keys	72
3.5 Data Integrity	76
3.6 Constraint Violation while Updating Data	82
3.7 Mapping E-R Model to Relational Model	84
<i>Summary</i>	94
<i>Key Terms</i>	95
<i>Exercises</i>	95

Chapter 4.	RELATIONAL ALGEBRA AND CALCULUS	99
4.1	Relational Algebra	100
4.2	Relational Calculus	116
4.3	Expressive Power of Relational Algebra and Relational Calculus	124
	<i>Summary</i>	125
	<i>Key Terms</i>	126
	<i>Exercises</i>	127
Chapter 5.	STRUCTURED QUERY LANGUAGE	131
5.1	Basic Features of SQL	132
5.2	Data Definition	132
5.3	Data Manipulation Language	143
5.4	Complex Queries in SQL	151
5.5	Additional Features of SQL	158
5.6	Accessing Databases from Applications	165
	<i>Summary</i>	174
	<i>Key Terms</i>	175
	<i>Exercises</i>	176
Chapter 6.	RELATIONAL DATABASE DESIGN	181
6.1	Features of Good Relational Design	181
6.2	Decomposition	183
6.3	Functional Dependencies	184
6.4	Normal Forms	189
6.5	Insufficiency of Normal Forms	196
6.6	Comparison of BCNF and 3NF	199
6.7	Higher Normal Forms	199
6.8	Denormalization	205
	<i>Summary</i>	206
	<i>Key Terms</i>	207
	<i>Exercises</i>	207
Chapter 7.	DATA STORAGE AND INDEXING	213
7.1	Hierarchy of Storage Devices	213
7.2	Redundant Arrays of Independent Disks	218
7.3	New Storage Systems	222
7.4	Accessing Data from Disk	222
7.5	Placing File Records on Disk Blocks	225
7.6	Organization of Records in Files	229
7.7	Indexing	238
	<i>Summary</i>	247
	<i>Key Terms</i>	248
	<i>Exercises</i>	250

Chapter 8. QUERY PROCESSING AND OPTIMIZATION	253
8.1 Query Processing Steps	253
8.2 External Sort-Merge Algorithm	256
8.3 Algorithms for Relational Algebra Operations	257
8.4 Expressions Containing Multiple Operations	270
8.5 Query Optimization	271
8.6 Query Optimization in Oracle	286
<i>Summary</i>	289
<i>Key Terms</i>	291
<i>Exercises</i>	291
 Chapter 9. INTRODUCTION TO TRANSACTION PROCESSING	 295
9.1 Desirable Properties of a Transaction	296
9.2 States of a Transaction	297
9.3 Concurrent Execution of Transactions	298
9.4 Transaction Schedules	300
9.5 SQL Transaction Statements	308
<i>Summary</i>	310
<i>Key Terms</i>	311
<i>Exercises</i>	311
 Chapter 10. CONCURRENCY CONTROL TECHNIQUES	 315
10.1 Locking	315
10.2 Lock-Based Techniques	318
10.3 Specialized Locking Techniques	324
10.4 Performance of Locking	329
10.5 Timestamp-Based Technique	330
10.6 Optimistic (or Validation) Technique	333
10.7 Multiversion Technique	336
10.8 Dealing with Deadlock	338
<i>Summary</i>	341
<i>Key Terms</i>	342
<i>Exercises</i>	343
 Chapter 11. DATABASE RECOVERY SYSTEM	 347
11.1 Types of Failures	347
11.2 Caching of Disk Pages	348
11.3 Recovery Related Steps During Normal Execution	349
11.4 Recovery Techniques	352
11.5 Recovery for Concurrent Transactions	355
11.6 ARIES Recovery Algorithm	356
11.7 Recovery from Catastrophic Failures	360
<i>Summary</i>	361
<i>Key Terms</i>	362
<i>Exercises</i>	363

Chapter 12. DATABASE SECURITY	367
12.1 Security Issues	367
12.2 Role of DBA in Database Security	369
12.3 Authorization	369
12.4 Authentication	371
12.5 Access Control	372
12.6 Encryption	377
12.7 Statistical Database	380
<i>Summary</i>	381
<i>Key Terms</i>	382
<i>Exercises</i>	383
 Chapter 13. DATABASE SYSTEM ARCHITECTURES	 386
13.1 Overview of Parallel DBMS	386
13.2 Distributed DBMS	388
13.3 Client/Server Systems	401
<i>Summary</i>	403
<i>Key Terms</i>	405
<i>Exercises</i>	405
 Chapter 14. DATA WAREHOUSING, OLAP, AND DATA MINING	 409
14.1 Data Warehousing	410
14.2 Online Analytical Processing (OLAP)	415
14.3 Data Mining Technology	420
<i>Summary</i>	430
<i>Key Terms</i>	432
<i>Exercises</i>	432
 Chapter 15. INFORMATION RETRIEVAL	 436
15.1 Information Retrieval Systems	436
15.2 Indexing of Text Documents	440
15.3 Web Search Engines	442
<i>Summary</i>	444
<i>Key Terms</i>	445
<i>Exercises</i>	446
 Chapter 16. OBJECT-BASED DATABASES	 448
16.1 Need for Object-Based Databases	448
16.2 Object Relational Database Systems	449
16.3 Object-Oriented Database Management Systems	455
16.4 OODBMS versus ORDBMS	463
<i>Summary</i>	464
<i>Key Terms</i>	465
<i>Exercises</i>	466

Chapter 17. XML	469
17.1 Structured, Semi-Structured, and Unstructured Data	470
17.2 Overview of XML	472
17.3 Structure of XML Data	472
17.4 DTD and XML Schema	475
17.5 Querying XML Data	479
17.6 Approaches to Store XML Data	481
17.7 Uses of XML	482
<i>Summary</i>	483
<i>Key Terms</i>	484
<i>Exercises</i>	484
 Chapter 18. LEADING DATABASE SYSTEMS	 487
18.1 PostgreSQL	487
18.2 Oracle	491
18.3 Microsoft SQL Server	493
18.4 IBM DB2 Universal Database	497
<i>Summary</i>	500
<i>Key Terms</i>	501
<i>Exercises</i>	502
 Case Study 1. HOSPITAL MANAGEMENT SYSTEM	 503
Aim	503
Description	503
Tables Description	503
E-R Diagram	505
Relational Database Schema for Case Study	506
Implementation in SQL Server	506
 Case Study 2. RAILWAY RESERVATION	 515
Aim	515
Description	515
List of Assumptions	515
E-R Diagram	516
Description of Tables and Procedures	516
Relational Database Schema for Case Study	517
 Appendix A. MICROSOFT ACCESS AS DATABASE MANAGEMENT SYSTEM	 525
A.1 Components of Access	525
A.2 Starting Access	526
A.3 Working with Tables	527
A.4 Working with Queries	532
A.5 Working with Forms	537
A.6 Working with Reports	539

Appendix B. SQL EXTENSIONS FOR DATA ANALYSIS	543
B.1 Extensions to GROUP BY Clause	543
B.2 New Analytic Functions	546
B.3 Top-N Queries	551
B.4 Windowing in SQL:1999	552
Appendix C. ABBREVIATIONS, ACRONYMS, AND SYMBOLS	555
Appendix D. E. F. CODD'S RULES	559
GLOSSARY	561
INDEX	576

Preface

As organizations strive to excel in modern competitive environment, managing information has acquired greater relevance. Database systems now play a vital role in the effective management of information. Database, being an essential ingredient of modern computing systems, now constitutes an integral part of computer science curricula.

Introduction to Database Systems aims to provide an in-depth knowledge of the most important aspects of database systems, especially the relational database systems. The text includes fundamental concepts like database design, languages, and implementation as well as advanced concepts like distributed databases and query processing and optimization. In addition, some emerging technologies like object-based databases, XML, and data mining are also included. The content of the book is designed to suit the requirements of computer science students at the undergraduate and postgraduate levels as well as for the advanced learners.

Key Features

- Book is written in a clear, concise, and lucid manner, which makes it student-friendly.
- Text is well-structured and illustrated with solved examples and block diagrams.
- Inter-chapter dependencies are kept to a minimum.
- Chapter objectives at the beginning of each chapter describe what lies ahead in the chapter for the reader.
- Features like Notes, Key Points, Learn More, and Things to Remember appear throughout the book which enhance the reader's learning.
- Detailed coverage of the topics makes the book useful to both undergraduate and postgraduate students.
- Two case studies at the end of the book familiarize the students with the implementation of database system concepts in real systems.
- A comprehensive index at the end of the book enables students to access the topics of interest quickly.

Target Audience

- Primary usage of the textbook for the students of BCA, BE, BTech, BIT, BIS, BSc, PGDCA, MCA, MIT, and MSc courses offered by computer science, computer engineering, and information technology departments of various colleges and universities.
- Professionals working in the areas of software development and database designing and administration.
- As a textual resource in training programmes offered by many computer institutes.

Chapter Organization

- *Chapter 1* provides an introduction of database and database management system, its advantages over traditional file system, the DBMS environment, various data models, and the steps involved in database design.
- *Chapter 2* deals with conceptual modeling using E-R and enhanced E-R model. It represents all the concepts with the help of an example of *Online Book* database.

- *Chapter 3* describes the basic concepts of relational model, which is the most commonly used data model. It also covers integrity constraints and deals with constraint violations.
- *Chapter 4* discusses two formal query languages, namely, *relational algebra* and *relational calculus*. The concepts of relational algebra and calculus are explained in a simple manner that helps in easy understanding of these languages.
- *Chapter 5* discusses the Structured Query Language (SQL), which is used as a standard query language in most of the commercial databases.
- *Chapter 6* deals with normalization which is the most important process in database design. It discusses the various issues that are confronted while designing a database schema. It also discusses the formal methodology (normalization) that is used to resolve these issues and design an efficient database system.
- *Chapter 7* covers the concepts of physical database design including database storage and indexing. It describes the primary methods of organizing the files on the disks and various indexing techniques that help in faster retrieval of data.
- *Chapter 8* introduces the basics of query processing and optimization. It provides an understanding of how the queries are evaluated and how the data is retrieved from the database.
- *Chapter 9* introduces the properties of transaction including atomicity, consistency, isolation, and durability. It discusses fundamentals of transaction processing and introduces the concept of serializability.
- *Chapter 10* covers a number of concurrency control techniques including locking, timestamping, and optimistic techniques. It also covers deadlock handling.
- *Chapter 11* describes the database recovery system to ensure the normal execution of transactions regardless of any type of system failure. It discusses various recovery techniques such as system log, checkpoints, etc.
- *Chapter 12* focuses on database security and presents various security issues, the types of threats to the database, and the role of DBA in implementing database security. In addition, it also discusses how the user's access to the database is controlled.
- *Chapter 13* explores the concepts of database storage, query processing, transaction management, concurrency control, and recovery in distributed database systems.
- *Chapter 14* introduces data warehousing, online transaction processing (OLAP), and data mining.
- *Chapter 15* discusses information retrieval techniques to extract information from unstructured textual data along with the retrieval techniques used in web search engines.
- *Chapter 16* focuses on object-relational and object-oriented database features including structured types, type inheritance, table inheritance, persistence, etc. It also discusses object definition language (ODL) and object query language (OQL).
- *Chapter 17* covers XML concepts including XML data, XML schema, and querying XML data. It also discusses various uses of XML.
- *Chapter 18* discusses four leading database systems including PostgreSQL, Oracle, Microsoft SQL Server, and IBM DB2.

Exercises

End-of-chapter exercises include objective questions (multiple choice, and fill in the blanks), descriptive questions and practical questions. These exercises aim to develop wide and varied skill sets among the students.

Acknowledgements

- Our publishers Pearson Education, their editorial team, and panel reviewers for their valuable contribution towards content enrichment.
- Our technical and editorial consultants for devoting their precious time to improve the quality of the book.
- Our entire research and development team who have put in their earnest efforts and relentless perseverance to bring out a high quality book.

Feedback

For any suggestions and comments about this book, please feel free to send your feedback to itlesl@rediffmail.com.

Hope you enjoy reading this book as much as we have enjoyed writing it.

ROHIT KHURANA

Copyright © 2010 Dorling Kindersley (India) Pvt. Ltd

This book is sold subject to the condition that it shall not, by way of trade or otherwise, be lent, resold, hired out, or otherwise circulated without the publisher's prior written consent in any form of binding or cover other than that in which it is published and without a similar condition including this condition being imposed on the subsequent purchaser and without limiting the rights under copyright reserved above, no part of this publication may be reproduced, stored in or introduced into a retrieval system, or transmitted in any form or by any means (electronic, mechanical, photocopying, recording or otherwise), without the prior written permission of both the copyright owner and the above-mentioned publisher of this book.

ISBN 978-81-317-3192-5

10 9 8 7 6 5 4 3 2 1

Published by Dorling Kindersley (India) Pvt. Ltd., licensees of Pearson Education in South Asia

Head Office: 7th Floor, Knowledge Boulevard, A-8 (A), Sector 62, NOIDA, 201 309, UP, India.
Registered Office: 11 Community Centre, Panchsheel Park, New Delhi 110 017, India

Typeset by AcePro India Pvt. Ltd
Printed in India at

DATABASE SYSTEM

After reading this chapter, the reader will understand:

- The basic concept of database and database management system
- The need of database system
- Limitations of traditional file processing system that led to the development of database management system
- Advantages of database system
- Various developments in the field of database system
- Areas where the database system is used
- Cost and risk involved in database system
- Different types of individuals that interact with the database system
- The concept of database schema and database instance
- Three-level DBMS architecture, which provides the abstract view of the database
- The concept of data independence, which helps the user to change the schema at one level of the database system without having to change the schema at the other levels
- Database models, which provide the necessary means to achieve data abstraction
- Different types of DBMS languages
- Component modules of DBMS
- Difference between centralized and client/server database system
- Various types of database management systems
- The steps involved in designing the database system

Storing data and retrieving information has been a necessity in all ages of business. The term **data** can be defined as a set of isolated and unrelated raw facts with an implicit meaning. Data can be anything such as, name of a person, a number, images, sound, etc. For example, 'Monica, 25, student' etc., is a data. When the data is processed and converted into a meaningful and useful form, it is known as **information**. For example, 'Monica is 25 years old and she is a student.' is an information. For a business to be successful, a fast access to information is vital as important decisions are based on the information available at any point of time. Traditionally, the data was stored in voluminous repositories such as files, books, and ledgers. However, storing data and retrieving information from these repositories was a time-consuming task. With the development of computers, the problem of information storage and retrieval was resolved. Computers replaced tons of paper, file folders, and ledgers as the principal media for storing important information.

Since then numerous techniques and devices have been developed to manage and organize the data so that useful information can be accessed easily and efficiently. The eternal quest for data management has led to the development of the database technology. A **database** can be defined as a collection of related data from which users can efficiently retrieve the desired information. A database can be anything from a simple collection of roll numbers, names, addresses, and phone numbers of

students to a complex collection of sound, images, and even video or film clippings. Though databases are generally computerized, instances of non-computerized databases from everyday life can be cited in abundance. A dictionary, a phone book, a collection of recipes, and a TV guide are all common examples of non-computerized databases. The examples of computerized databases include customer files, employee rosters, books catalogue, equipment inventories, and sales transactions.

In addition to the storage and retrieval of data, certain other operations can also be performed on a database. These operations include adding, updating, and deleting data. All these operations on a database are performed using a database management system. A **Database Management System (DBMS)** is an integrated set of programs used to create and maintain a database. The main objective of a DBMS is to provide a convenient and effective method of defining, storing, retrieving, and manipulating the data contained in the database. In addition, the DBMS must ensure the security of the database from unauthorized access and recovery of the data during system failures. It must also provide techniques for data sharing among several users. The database and the DBMS software are collectively known as **database system**.

1.1 NEED OF DATABASE SYSTEM

The need of database systems arose in the early 1960s in response to the traditional file processing system. In the **file processing system**, the data is stored in the form of files, and a number of application programs are written by programmers to add, modify, delete, and retrieve data to and from appropriate files. New application programs are written as and when needed by the organization.

For example, consider a bookstore that uses a file processing system to keep track of all the available books. The system maintains a file named **BOOK** to store information related to books. This information include book title, ISBN, price (in dollars), year of publishing, copyright date, category (such as language books, novels, and textbooks), number of pages, name of the author, name and address of the publisher, etc. In addition, the system has many application programs that allow users to manipulate the information stored in **BOOK** file. For example, the system may contain programs to add information about a new book, modify any existing book information, print the details of books according to their categories, etc.

Now, suppose a need arises to keep additional information about the publishers of the books that includes phone number and email id. To fulfill this need, the system creates a new file say **PUBLISHER**, which includes name, address, phone number, and email id of the publishers. New application programs are written and added to the system to manipulate the information in the **PUBLISHER** file. In this way, as the time goes by, more files and application programs are added to the system.

This system of storing data in the form of files is supported by a conventional operating system. However, the file processing system has a number of disadvantages, which are given here.

- ⇒ Same information may be duplicated in several files. For example, the name and the address of publisher are stored in **BOOK** file as well as in **PUBLISHER** file. This duplication of data is known as **data redundancy**, which leads to wastage of storage space. The other problem with file processing system is that the data may not be updated consistently. Suppose a publisher requests for a change in his address. Since the address of the publisher is stored in the **BOOK** as well as **PUBLISHER** file, both the files must be updated. If the address of the publisher is not modified in any of the two files, then the same publisher will have different address in two different files. This is known as **data inconsistency**.

Learn More

The ISBN is a unique commercial book identifier barcode. It is assigned to each edition of a book. An ISBN consists of several parts such as publisher code, title code, etc. These parts are usually separated by hyphens.

- ⇒ In any application, there are certain data integrity rules that need to be maintained. These rules could be in the form of certain conditions or constraints. In a file processing system, all these rules need to be explicitly programmed in all application programs, which are using that particular data item. For example, the integrity rule that each book should have a book title has to be implemented in all the application programs separately that are using the BOOK file. In addition, when new constraints are to be enforced, all the application programs should be changed accordingly.
- ⇒ The file processing system lacks the insulation between program and data. This is because the file structure is embedded in the application program itself, thus, it is difficult to change the structure of a file as it requires changing all the application programs accessing it. For example, an application program in C++ defines the file structure using the keyword `struct` or `class`. Suppose the data type of the field ISBN is changed from string to number, changes have to be made in all the application programs that are accessing the BOOK file.
- ⇒ Handling new queries is difficult, since it requires change in the existing application programs or requires a new application program. For example, suppose a need arises to print details of all the textbooks published by a particular publisher. One way to handle this request is to use the existing application program that prints details of the books according to their categories, and then manually generate the list of textbooks published by a particular publisher. Obviously, it is unacceptable. Alternatively, the programmer is requested to write a new application program. Suppose such a new application program is written. Now suppose after some time, a request arises to filter the list of textbooks with price greater than \$50. Then again we have to write a new application program.
- ⇒ Since many users are involved in creating files and writing application programs, the various files in the system may have different file structures. Moreover, the programmers may choose different programming languages to write application programs.
- ⇒ Application programs are added in an unplanned manner, due to which details of each file are easily available to every user. Thus, the file processing system lacks security feature.



NOTE *In spite of its limitations, the file processing system is still in use mainly for database backup.*

1.2 ADVANTAGES OF DATABASE SYSTEM

Database approach came into existence due to the bottlenecks of file processing system. In the database approach, the data is stored at a central location and is shared among multiple users. Thus, the main advantage of DBMS is **centralized data management**. The centralized nature of database system provides several advantages, which overcome the limitations of the conventional file processing system. These advantages are listed here.

- ⇒ **Controlled data redundancy:** During database design, various files are integrated and each logical data item is stored at central location. This eliminates replicating the data item in different files, and ensures consistency and saves the storage space. Note that the redundancy in the database systems cannot be eliminated completely as there could be some performance and technical reasons for having some amount of redundancy. However, the DBMS should be capable of controlling this redundancy in order to avoid data inconsistencies.
- ⇒ **Enforcing data integrity:** In database approach, enforcing data integrity is much easier. Various integrity constraints are identified by database designer during database design. Some of these

data integrity constraints can be enforced automatically by the DBMS, and others may have to be checked by the application programs.

- ⇒ **Data sharing:** The data stored in the database can be shared among multiple users or application programs. Moreover, new applications can be developed to use the same stored data. Due to shared data, it is possible to satisfy the data requirements of the new applications without having to create any additional data or with minimal modification.
- ⇒ **Ease of application development:** The application programmer needs to develop the application programs according to the users' needs. The other issues like concurrent access, security, data integrity, etc., are handled by the DBMS itself. This makes the application development an easier task.
- ⇒ **Data security:** Since the data is stored centrally, enforcing security constraints is much easier. The DBMS ensures that the only means of access to the database is through an authorized channel. Hence, data security checks can be carried out whenever access is attempted to sensitive data. To ensure security, a DBMS provides security tools such as user codes and passwords. Different checks can be established for each type of access (addition, modification, deletion, etc.) to each piece of information in the database.
- ⇒ **Multiple user interfaces:** In order to meet the needs of various users having different technical knowledge, DBMS provides different types of interfaces such as query languages, application program interfaces, and graphical user interfaces (GUI) that include forms-style and menu-driven interfaces. A form-style interface displays a form to each user and user interacts using these forms. In menu-driven interface, the user interaction is through lists of options known as menus.
- ⇒ **Backup and recovery:** The DBMS provides backup and recovery subsystem that is responsible for recovery from hardware and software failures. For example, if the failure occurs in between the transaction, the DBMS recovery subsystem either reverts back the database to the state which existed prior to the start of the transaction or resumes the transaction from the point it was interrupted so that its complete effect can be recorded in the database.

In addition to centralized data management, DBMS also has some other advantages, which are discussed here.

- ⇒ **Program-data independence:** The independence between the programs and the data is known as program-data independence (or simply data independence). It is an important characteristic of DBMS as it allows changing the structure of the database without making any changes in the application programs that are using the database. To provide a high degree of data independence, the definition or the description of the database structure (structure of each file, the type and storage format of each data item) and various constraints on the data are stored separately in **DBMS catalog**. The information contained in the catalog is called the **metadata** (data about data). This independence is provided by a three-level DBMS architecture, which is discussed in Section 1.7.
- ⇒ **Data abstraction:** The property of DBMS that allows program-data independence is known as data abstraction. Data abstraction allows the database system to provide an abstract view of the data to its users without giving the physical storage and implementation details.
- ⇒ **Supports multiple views of the data:** A database can be accessed by many users and each of them may have a different perspective or view of the data. A database system provides a facility to define different views of the data for different users. A **view** is a subset of the database that contains virtual data derived from the database files but it does not exist in physical form. That is, no physical file is created for storing the data values of the view; rather, only the definition of the view is stored.

Due to these advantages, the traditional file processing system for the bookstore is replaced by an *Online Book* database to maintain the information about various books, publishers, and authors. The author details include the name, address, URL, and phone number of author. Note that some of the authors also review the books and give ratings and feedbacks about the books. The database also maintains the details about the ratings given to the books by different reviewers.

1.3 DEVELOPMENTS IN DATABASE SYSTEM

The database systems are divided into three generations. The first generation includes the database systems based on hierarchical and network data model. The second generation includes the database systems based on relational data model, and the third generation includes the database systems based on object-oriented and object-relational data model.

In the early 1960s, the first general purpose DBMS, called the Integrated Data Store was designed by Charles Bachman. The development of magnetic tapes led to the automation of various data processing tasks such as payroll and inventory. In the late 1960s, the Information Management System (IMS) DBMS was developed by IBM. Two data models, network data model and hierarchical data model were also developed. During this period, use of hard disks instead of magnetic tapes facilitated direct access to data. In 1970, Edgar Frank "Ted" Codd (E.F. Codd) proposed the relational data model and non-procedural way to query data that led to the development of relational databases. Codd was felicitated with the prestigious Association of Computing Machinery Turing Award for his work.

In the 1980s, relational model became the dominant DBMS paradigm. SQL was standardized and the current standard known as SQL:1999 was adopted by ANSI/ISO. Research on parallel and distributed databases was carried out. During this period, several commercial relational databases such as IBM's DB2, Oracle, Informix UDS extended their systems to store new and complex data types such as images and text.

In the early 1990s, SQL was designed mainly for decision support applications. In this period, vendors introduced products for parallel database. They also began to add object-relational support to their databases. In the late 1990s, with the advent of World Wide Web, the use of DBMS to store data accessed through a Web browser became widespread. Database systems were designed to support very high transaction processing rates and 24×7 availability. Moreover, the database systems became more reliable during this period. In the early 2000s, XML databases and associated query language Xquery were developed to handle large amount of data and high transaction rates.

The emergence of several enterprise resource planning (ERP) and management resource planning (MRP) packages like PeopleSoft, SAP, Siebel, etc., has added a substantial layer of application-oriented features on top of DBMS. These packages provide a general application layer to carry out a common set of tasks such as financial analysis, inventory control, human resource planning, etc. Different companies can customize this layer according to their need, which leads to the reduction in the overall cost of building the application layer.

1.4 APPLICATION AREAS OF DATABASE SYSTEM

Database systems are widely used in different areas because of their numerous advantages. Some of the most common database applications are listed here.

- ⇒ **Airlines and railways:** Airlines and railways use online databases for reservation, and for displaying the schedule information.
- ⇒ **Banking:** Banks use databases for customer inquiry, accounts, loans, and other transactions.
- ⇒ **Education:** Schools and colleges use databases for course registration, result, and other information.

- ⇒ **Telecommunications:** Telecommunication departments use databases to store information about the communication network, telephone numbers, record of calls, for generating monthly bills, etc.
- ⇒ **Credit card transactions:** Databases are used for keeping track of purchases on credit cards in order to generate monthly statements.
- ⇒ **E-commerce:** Integration of heterogeneous information sources (for example, catalogs) for business activity such as online shopping, booking of holiday package, consulting a doctor, etc.
- ⇒ **Health care information systems and electronic patient record:** Databases are used for maintaining the patient health care details.
- ⇒ **Digital libraries and digital publishing:** Databases are used for management and delivery of large bodies of textual and multimedia data.
- ⇒ **Finance:** Databases are used for storing information such as sales, purchases of stocks and bonds or data useful for online trading.
- ⇒ **Sales:** Databases are used to store product, customer and transaction details.
- ⇒ **Human resources:** Organizations use databases for storing information about their employees, salaries, benefits, taxes, and for generating salary checks.

1.5 COST AND RISK OF DATABASE SYSTEM

Database systems have many advantages; however, before implementing database systems over the traditional file processing system, the cost and risk factors of implementing database systems should also be considered. The various cost and risk factors are given here.

- ⇒ **High cost:** Installing new database system may require investment in hardware and software. The DBMS requires more main memory and disk storage. Moreover, DBMS is quite expensive. Therefore, a company needs to consider the overhead cost of implementing a new database system.
- ⇒ **Training new personnel:** When an organization plans to adopt a database system, it may need to recruit or hire a specialized data administration group, which can coordinate with different user groups for designing views, establishing recovery procedures, fine tuning data structures to meet the organization requirements. Hiring such professionals is expensive.
- ⇒ **Explicit backup and recovery:** A shared corporate database must be accurate and available at all times. Therefore, a system using online updation requires explicit backup and recovery procedures.
- ⇒ **System failure:** When a computer system containing the database fails, all users have to wait until the system is functional again. Moreover, if DBMS or application program fails a permanent damage may occur to the database.



NOTE *In spite of cost and risk involved, database systems are widely used.*

1.6 PEOPLE WHO INTERACT WITH DATABASES

Database is an important asset of business. To design, use, and maintain such important asset many people are involved. The people who work with databases include *database users*, *system analysts*, *application programmers*, and *database administrator (DBA)*. **Database users** are those who interact with the database in order to query and update the database, and generate reports. Database users are further classified into the following categories:

- ⇒ **Naive users:** The users who query and update the database by invoking some already written application programs. For example, the owner of the bookstore enters the details of various books in the database by invoking appropriate application program. The naive user interacts with the database using form interface.
- ⇒ **Sophisticated users:** The users, such as business analyst, scientist, etc., who are familiar with the facilities provided by a DBMS interact with the system without writing any application programs. Such users use database query language to retrieve information from the database to meet their complicated requirements.
- ⇒ **Specialized users:** The users who write specialized database programs, which are different from traditional data processing applications, such as banking and payroll management which use simple data types. Specialized users write applications such as computer-aided design systems, knowledge-base and expert systems that store data having complex data types.

Things to Remember

Though DBMS provides several facilities to access a database, but all kind of users need not learn about all these facilities. Naive users need to learn only some simple facilities of DBMS, whereas sophisticated users have to know most of the DBMS facilities in order to fulfill their complex requirements.

System analysts determine the requirements of the database users (especially naive users) to create a solution for their business need, and focus on non-technical and technical aspects. The non-technical aspects involve defining system requirements, facilitating interaction between business users and technical staff, etc. Technical aspects involve developing the specification for user interface (application programs). **Application programmers** are the computer professionals who implement the specifications given by the system analysts, and develop application programs. They can choose tools, such as rapid application development (RAD) to develop the application program with minimal effort. The database application programmer develops application program to facilitate easy data access for the database users.

Database administrator (DBA) is a person who has central control over both data and application programs. The responsibilities of DBA vary depending upon the job description and corporate and organization policies. Some of the responsibilities of DBA are given here.

- ⇒ **Schema definition and modification:** The overall structure of the database is known as **database schema**. It is the responsibility of the DBA to create the database schema by executing a set of data definition statements in DDL. The DBA also carries out the changes to the schema according to the changing needs of the organization.
- ⇒ **New software installation:** It is the responsibility of the DBA to install new DBMS software, application software, and other related software. After installation, the DBA must test the new software.
- ⇒ **Security enforcement and administration:** DBA is responsible for establishing and monitoring the security of the database system. It involves adding and removing users, auditing, and checking for security problems.
- ⇒ **Data analysis:** DBA is responsible for analyzing the data stored in the database, and studying its performance and efficiency in order to effectively use indexes, parallel query execution, etc.
- ⇒ **Preliminary database design:** The DBA works along with the development team during the database design stage due to which many potential problems that can arise later (after installation) can be avoided.

- ⇒ **Physical organization modification:** The DBA is responsible for carrying out the modifications in the physical organization of the database for better performance.
- ⇒ **Routine maintenance checks:** The DBA is responsible for taking the database backup periodically in order to recover from any hardware or software failure (if occurs). Other routine maintenance checks that are carried out by the DBA are checking data storage and ensuring the availability of free disk space for normal operations, upgrading disk space as and when required, etc.

1.7 DBMS ARCHITECTURE AND DATA INDEPENDENCE

The DBMS architecture describes how data in the database is viewed by the users. It is not concerned with how the data is handled and processed by the DBMS. The database users are provided with an abstract view of the data by hiding certain details of how data is physically stored. This enables the users to manipulate the data without worrying about where it is located or how it is actually stored.

In this architecture, the overall database description can be defined at three levels, namely, *internal*, *conceptual*, and *external* levels and thus, named **three-level DBMS architecture**. This architecture is proposed by ANSI/SPARC (American National Standards Institute/Standards Planning and Requirements Committee) and hence, is also known as **ANSI/SPARC architecture**. The three levels are discussed here.

- ⇒ **Internal level:** It is the lowest level of data abstraction that deals with the physical representation of the database on the computer and thus, is also known as **physical level**. It describes *how* the data is physically stored and organized on the storage medium. At this level, various aspects are considered to achieve optimal runtime performance and storage space utilization. These aspects include storage space allocation techniques for data and indexes, access paths such as indexes, data compression and encryption techniques, and record placement.
- ⇒ **Conceptual level:** This level of abstraction deals with the logical structure of the entire database and thus, is also known as **logical level**. It describes *what* data is stored in the database, the relationships among the data and complete view of the user's requirements without any concern for the physical implementation. That is, it hides the complexity of physical storage structures. The conceptual view is the overall view of the database and it includes all the information that is going to be represented in the database.
- ⇒ **External level:** It is the highest level of abstraction that deals with the user's view of the database and thus, is also known as **view level**. In general, most of the users and application programs do not require the entire data stored in the database. The external level describes a part of the database for a particular group of users. It permits users to access data in a way that is customized according to their needs, so that the same data can be seen by different users in different ways, at the same time. In this way, it provides a powerful and flexible security mechanism by hiding the parts of the database from certain users, as the user is not aware of existence of any attributes that are missing from the view.

These three levels are used to describe the schema of the database at various levels. Thus, the three-level architecture is also known as **three-schema architecture**. The internal level has an **internal schema**, which describes the physical storage structure of the database. The conceptual level has a **conceptual schema**, which describes the structure of entire database. The external level has **external schemas** or **user views**, which describe the part of the database according to a particular user's requirements, and hide the rest of the database from that user. The physical level is managed by the operating system under the direction of DBMS. The three-schema architecture is shown in Figure 1.1.

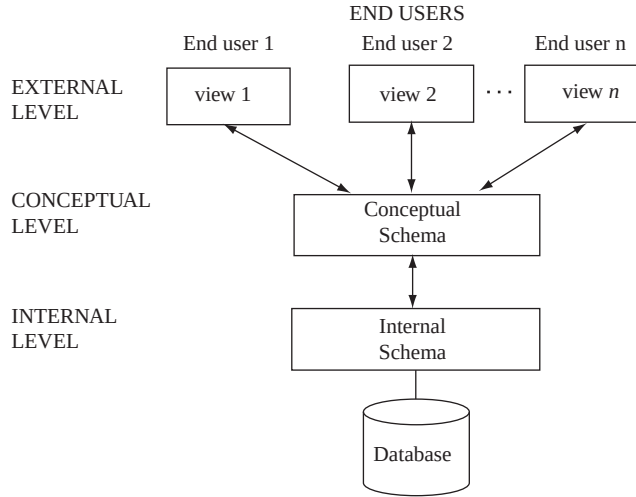


Fig. 1.1 *Three-schema architecture*

To understand the three-schema architecture, consider the three levels of the **BOOK** file in *Online Book* database as shown in Figure 1.2. In this figure, two views (view 1 and view 2) of the **BOOK** file have been defined at the external level. Different database users can see these views. The details of the data types are hidden from the users. At the conceptual level, the **BOOK** records are described by a type definition. The application programmers and the DBA generally work at this level of abstraction. At the internal level, the **BOOK** records are described as a block of consecutive storage locations such as words or bytes. The database users and the application programmers are not aware of these details; however, the DBA may be aware of certain details of the physical organization of the data.

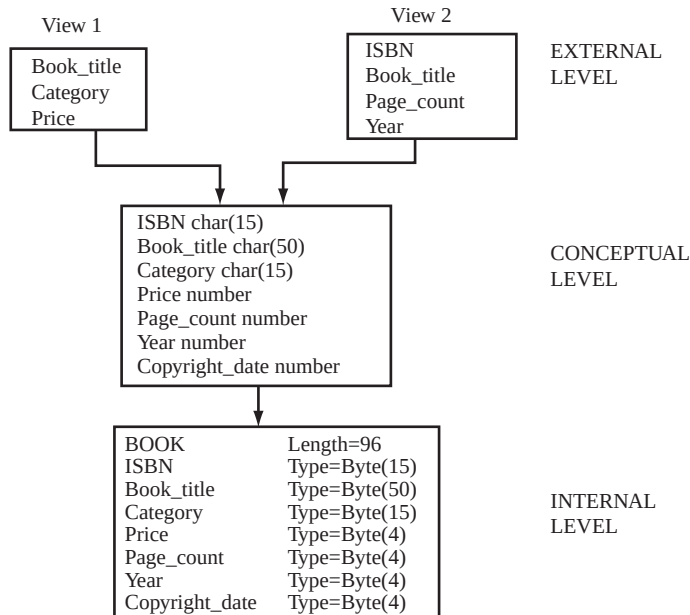


Fig. 1.2 *Three levels of Online Book database (BOOK file)*

In three-schema architecture, each user group refers only to its own external view. Whenever a user specifies a request to generate a new external view, the DBMS must transform the request specified at external level into a request at conceptual level, and then into a request at physical level. If the user requests for data retrieval, the data extracted from the database must be presented according to the need of the user. This process of transforming the requests and results between various levels of DBMS architecture is known as **mapping**.

The main advantage of three-schema architecture is that it provides data independence. **Data independence** is the ability to change the schema at one level of the database system without having to change the schema at the other levels. Data independence is of two types, namely, *logical data independence* and *physical data independence*.

- ⇒ **Logical data independence:** It is the ability to change the conceptual schema without affecting the external schemas or application programs. The conceptual schema may be changed due to change in constraints or addition of new data item or removal of existing data item, etc., from the database. The separation of the external level from the conceptual level enables the users to make changes at the conceptual level without affecting the external level or the application programs. For example, if a new data item, say **Edition** is added to the **BOOK** file, the two views (*view 1* and *view 2* shown in Figure 1.2) are not affected.
- ⇒ **Physical data independence:** It is the ability to change the internal schema without affecting the conceptual or external schema. An internal schema may be changed due to several reasons such as for creating additional access structure, changing the storage structure, etc. The separation of internal schema from the conceptual schema facilitates physical data independence.

Logical data independence is more difficult to achieve than the physical data independence because the application programs are always dependent on the logical structure of the database. Therefore, the change in the logical structure of the database may require change in the application programs.

1.8 DATABASE MODELS

As discussed earlier, the main objective of database system is to highlight only the essential features and to hide the storage and data organization details from the user. This is known as **data abstraction**. A database model provides the necessary means to achieve data abstraction. A **database model** or simply a **data model** is an abstract model that describes how the data is represented and used. A data model consists of a set of data structures and conceptual tools that is used to describe the structure (data types, relationships, and constraints) of a database.

A data model not only describes the structure of the data, it also defines a set of operations that can be performed on the data. A data model generally consists of **data model theory**, which is a formal description of how data may be structured and used, and **data model instance**, which is a practical data model designed for a particular application. The process of applying a data model theory to create a data model instance is known as **data modeling**.

Depending on the concept they use to model the structure of the database, the data models are categorized into three types, namely, *high-level* or *conceptual data models*, *representational* or *implementation data models* and *low-level* or *physical data models*.

1.8.1 Conceptual Data Model

Conceptual data model describes the information used by an organization in a way that is independent of any implementation-level issues and details. The main advantage of conceptual data model is that it is independent of implementation details and hence, can be understood even by the end users having

non-technical background. The most popular conceptual data model, that is, entity-relationship (E-R) model is discussed in detail in Chapter 02.

1.8.2 Representational Data Model

The representational or implementation data models hide some data storage details from the users; however, can be implemented directly on a computer system. Representational data models are used most frequently in all traditional commercial DBMSs. The various representational data models are discussed here.

Hierarchical Data Model

The hierarchical data model is the oldest type of data model, developed by IBM in 1968. This data model organizes the data in a tree-like structure, in which each **child node** (also known as **dependents**) can have only one **parent node**. The database based on the hierarchical data model comprises a set of records connected to one another through links. The link is an association between two or more records. The top of the tree structure consists of a single node that does not have any parent and is called the **root node**.

The root may have any number of dependents; each of these dependents may have any number of lower level dependents. Each child node can have only one parent node and a parent node can have any number of (many) child nodes. It, therefore, represents only one-to-one and one-to-many relationships. The collection of same type of records is known as a **record type**. Figure 1.3 shows the hierarchical model of *Online Book* database. It consists of three record types, namely, **PUBLISHER**, **BOOK**, and **REVIEW**. For simplicity, only few fields of each record type are shown. One complete record of each record type represents a **node**.

The main advantage of the hierarchical data model is that the data access is quite predictable in the structure and, therefore, both the retrieval and updates can be highly optimized by the DBMS.

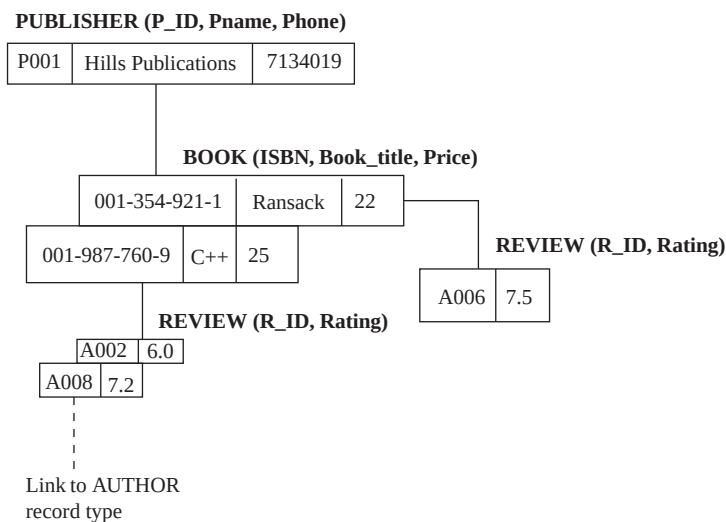


Fig. 1.3 Hierarchical data model for Online Book database

However, the main drawback of this model is that the links are *hard coded* into the data structure, that is, the link is permanently established and cannot be modified. The hard coding makes the hierarchical

model rigid. In addition, the physical links make it difficult to expand or modify the database and the changes require substantial redesigning efforts.

Network Data Model

The first specification of network data model was presented by Conference on Data Systems Languages (CODASYL) in 1969, followed by the second specification in 1971. It is powerful but complicated. In a network model the data is also represented by a collection of records, and relationships among data are represented by links. However, the link in a network data model represents an association between precisely two records. Like hierarchical data model, each record of a particular record type represents a node. However, unlike hierarchical data model, all the nodes are linked to each other without any hierarchy. The main difference between hierarchical and network data model is that in hierarchical data model, the data is organized in the form of trees and in network data model, the data is organized in the form of graphs.

The main advantage of network data model is that a parent node can have many child nodes and a child can also have many parent nodes. Thus, the network model permits the modeling of many-to-many relationships in data. The main limitation of the network data model is that it can be quite complicated to maintain all the links and a single broken link can lead to problems in the database. In addition, since there are no restrictions on the number of relationships, the database design can become complex. Figure 1.4 shows the network model of *Online Bookda* tabase.

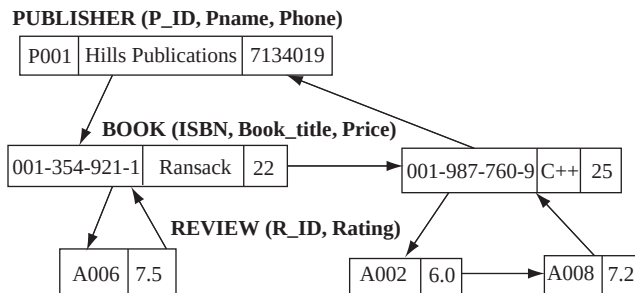


Fig. 1.4 Network data model for *Online Book* database



NOTE Links in hierarchical and network data models are generally implemented through pointers.

Relational Data Model

The relational data model was developed by E. F. Codd in 1970. In the relational data model, unlike the hierarchical and network models, there are no physical links. All data is maintained in the form of tables (generally, known as **relations**) consisting of rows and columns. Each row (record) represents an entity and a column (field) represents an attribute of the entity. The relationship between the two tables is implemented through a common attribute in the tables and not by physical links or pointers. This makes the querying much easier in a relational database system than in the hierarchical or network database systems. Thus, the relational model has become more programmer friendly and much more dominant and popular in both industrial and academic scenarios. Oracle, Sybase, DB2, Ingres, Informix, MS-SQL Server are few of the popular relational DBMSs. Figure 1.5 shows the relational model of *Online Bookda* tabase.

BOOK

ISBN	Book_title	Category	Price	Copyright_date	Year	Page_count	P_ID
001-354-921-1	Ransack	Novel	22	2005	2006	200	P001
001-987-760-9	C++	Textbook	25	2004	2005	800	P001

PUBLISHER

P_ID	Pname	Address	State	Phone	Email_ID
P001	Hills Publications	12, Park street, Atlanta	Georgia	71340 19	H_pub@hills.com

REVIEW

R_ID	ISBN	Rating
A002	001-987-760-9	6.0
A006	001-354-921-1	7.5
A008	001-987-760-9	7.2

Fig. 1.5 Relational data model for Online Book database



NOTE The network, hierarchical, and relational data models are known as **record-based data models** as these models represent data in the form of records.

Object-Based Data Model

In the recent years, the object-oriented paradigm has been applied to database technology, creating two new data models known as object-oriented data model and object-relational data model. The **object-oriented data model** extends the concepts of object-oriented programming language with persistence, versioning, concurrency control, data recovery, security, and other database capabilities. On the other hand, the **object-relational data model** is an extension of relational data model. It combines the features of both the relational data model and object-oriented data model.

Semistructured Data Model

Unlike other data models, where every data item of a particular type must have the same set of attributes, the semistructured data model allows individual data items of the same type to have different set of attributes. In semistructured data model, the information about the description of the data (schema) is contained within the data itself, which is sometimes called *self-describing* data. In such databases there is no clear separation between the data and the schema and thus, allowing data of any type. Semistructured data model has recently emerged as an important topic of study for different reasons given here.

- ⇒ There are data sources such as the Web, which is to be treated as databases; however, they cannot be constrained by a schema.
- ⇒ There is no fixed format for data exchange between heterogeneous databases.
- ⇒ To facilitate browsing of data.

Semistructured data model facilitates data exchange among heterogeneous data sources. It helps to discover new data easily and store it. It also facilitates querying the database without knowing the data types. However, it loses the data type information.

Learn More

XML (Extensible Markup Language) is a means of representing semistructured data. XML enables to define schema using XML schema language, which allows varying levels of schema over data elements.

BOOK

ISBN	Book_title	Category	Price	Copyright_date	Year	Page_count	P_ID
------	------------	----------	-------	----------------	------	------------	------

PUBLISHER

P_ID	Pname	Address	State	Phone	Email_ID
------	-------	---------	-------	-------	----------

AUTHOR

A_ID	Aname	City	State	Zip	Phone	URL
------	-------	------	-------	-----	-------	-----

AUTHOR_BOOK

A_ID	ISBN
------	------

REVIEW

R_ID	ISBN	Rating
------	------	--------

Fig. 1.6 Database schema for Online Book database

1.8.3 Physical Data Model

Physical data model describes the data in terms of files, indices, and other storage structures such as record formats, record ordering, and access paths. This model specifies how the database will be executed in a particular DBMS software such as Oracle, Sybase, etc., by taking into account the facilities and constraints of a given database management system. It also describes how the data is stored on disk and what access methods are available to it.



NOTE An access path (for example, an index) is a structure that helps in efficient searching and retrieval of data from a particular database.

1.9 DATABASE SCHEMA VERSUS DATABASE INSTANCE

While working with any data model, it is necessary to distinguish between the overall design or description of the database (database schema) and the database itself. As discussed earlier, the overall design or description of the database is known as **database schema** or simply **schema**. The database schema is also known as **intension** of the database, and is specified while designing the database. Figure 1.6 shows the database schema for Online Book database.

The data in the database at a particular point of time is known as **database instance** or **database state** or **snapshot**. The database state is also called an **extension** of the schema. The various states of the database are given here.

- ⇒ **Empty state:** When a new database is defined, only its schema is specified. At this point, the database is said to be in empty state as it contains no data.
- ⇒ **Initial state:** When the database is loaded with data for the first time, it is said to be in initial state.
- ⇒ **Current state:** The data in the database is updated frequently. Thus, at any point of time, the database is said to be in the current state.

The DBMS is responsible to check whether the database state is **valid state**. Thus, each time the database is updated, DBMS ensures that the database remains in the valid state. The DBMS refers to DBMS catalog where the metadata is stored in order to check whether the database state satisfies the structure and constraints specified in the schema. Figure 1.7 shows an example of an instance for PUBLISHER schema.

P_ID	Pname	Address	State	Phone	Email_id
P001	Hills Publications	12, Park street, Atlanta	Georgia	7134019	h_pub@hills.com
P002	Sunshine Publishers Ltd.	45, Second street, Newark	New Jersey	6548909	Null
P003	Bright Publications	123, Main street, Honolulu	Hawai	7678985	bright@bp.com
P004	Paramount Publishing House	789, Oak street, New York	New York	9254834	param_house@ param.com
P005	Wesley Publications	456, First street, Las Vegas	Nevada	5683452	Null

Fig.1.7 An example of instance - PUBLISHER instance

The schema and instance can be compared with a program written in a programming language. The database schema is similar to a variable declared along with the type description in a program. The variable contains a value at a given point of time. This value of a variable corresponds to an *instance* of a database schema.



NOTE The database schema changes rarely, whereas the data in the database may change frequently according to the requirements. The process of making any changes in the schema is known as **schema evolution**.

1.10 DBMS LANGUAGES

The main objective of a database management system is to allow its users to perform a number of operations on the database such as insert, delete, and retrieve data in abstract terms without knowing about the physical representations of data. To provide the various facilities to different types of users, a DBMS normally provides one or more specialized programming languages called **Database (or DBMS) Languages**.

The DBMS mainly provides two database languages, namely, *data definition language* and *data manipulation language* to implement the databases. Data definition language (DDL) is used for defining the database schema. The DBMS comprises DDL compiler that identifies and stores the schema description in the DBMS catalog. Data manipulation language (DML) is used to manipulate the database.

Learn More

In addition to DDL and DML, DBMS also provides two more languages, namely, *data control language (DCL)* and *transaction control language (TCL)*. Data control language is used to create user roles, grant permissions, and control access to database by securing it. Transaction control language is used to manage different transactions occurring within a database.

1.10.1 Data Definition Language

In DBMSs where no strict separation between the levels of the database is maintained, the data definition language is used to define the conceptual and internal schemas for the database. On the other hand, in DBMSs, where a clear separation is maintained between the conceptual and internal levels, the DDL is used to specify the conceptual schema only. In such DBMSs, a separate language, namely,

storage definition language (SDL) is used to define the internal schema. Some of the DBMSs that are based on true three-schema architecture use a third language, namely, *view definition language (VDL)* to define the external schema.

The DDL statements are also used to specify the integrity rules (constraints) in order to maintain the integrity of the database. The various integrity constraints are domain constraints, referential integrity, assertions and authorization. These constraints are discussed in detail in subsequent chapters. Like any other programming language, DDL also accepts input in the form of instructions (statements) and generates the description of schema as output. The output is placed in the **data dictionary**, which is a special type of table containing metadata. The DBMS refers the data dictionary before reading or modifying the data. Note that the database users cannot update the data dictionary; instead it is only modified by database system itself.

1.10.2 Data Manipulation Language

Once the database schemas are defined and the initial data is loaded into the database, several operations such as retrieval, insertion, deletion, and modification can be applied to the database. The DBMS provides data manipulation language (DML) that enables users to retrieve and manipulate the data. The statement which is used to retrieve the information is called a **query**. The part of the DML used to retrieve the information is called a **query language**. However, query language and DML are used synonymously though technically incorrect. The DML are of two types, namely, *non-procedural DML* and *procedural DML*.

The **non-procedural** or **high-level** or **declarative DML** enables to specify the complex database operations concisely. It requires a user to specify *what* data is required without specifying *how* to retrieve the required data. For example, SQL (Structured Query Language) is a non-procedural query language as it enables user to easily define the structure or modify the data in the database without specifying the details of *how* to manipulate the database. The high-level DML statements can either be entered interactively or embedded in a general purpose programming language. On the other hand, the **procedural** or **low-level DML** requires user to specify *what* data is required and *how* to access that data by providing step-by-step procedure. For example, relational algebra is procedural query language, which consists of set of operations such as select, project, union, etc., to manipulate the data in the database. Relational algebra and SQL are discussed in Chapters 04 and 05, respectively.

1.11 COMPONENT MODULES OF DBMS

A database system being a complex software system is partitioned into several software components that handle various tasks such as data definition and manipulation, security and data integrity, data recovery and concurrency control, and performance optimization (see Figure 1.8).

- ⇒ **Data definition:** The DBMS provides functions to define the structure of the data. These functions include defining and modifying the record structure, the data type of fields, and the various constraints to be satisfied by the data in each field. It is the responsibility of database administrator to define the database, and make changes to its definition (if required) using the DDL and other privileged commands. The **DDL compiler** component of DBMS processes these schema definitions, and stores the schema descriptions in the DBMS catalog (data dictionary). Other DBMS components then refer to the catalog information as and when required.
- ⇒ **Data manipulation:** Once the data structure is defined, data needs to be manipulated. The manipulation of data includes insertion, deletion, and modification of records. The functions that perform these operations are also part of the DBMS. These functions can handle planned as well as unplanned data manipulation needs.

- The queries that are defined as a part of the application programs are known as **planned queries**. The application programs are submitted to a **precompiler**, which extracts DML commands from the application program and send them to **DML compiler** for compilation. The rest of the program is sent to the **host language compiler**. The object codes of both the DML commands and the rest of the program are linked and sent to the **query evaluation engine** for execution.
 - The sudden queries that are executed as and when the need arises are known as **unplanned queries (interactive queries)**. These queries are compiled by the **query compiler**, and then optimized by the **query optimizer**. The query optimizer consults the data dictionary for statistical and other physical information about the stored data (query optimization is discussed in detail in Chapter 08). The optimized query is finally passed to the query evaluation engine for execution. The naive users of the database can also query and update the database by using some already given application program interfaces. The object code of these queries is also passed to query evaluation engine for processing.
- ⇒ **Data security and integrity:** The DBMS contains functions, which handle the security and integrity of data stored in the database. Since these functions can be easily invoked by the application, the application programmer need not code these functions in the programs.
- ⇒ **Concurrency and data recovery:** The DBMS also contains some functions that deal with the concurrent access of records by multiple users and the recovery of data after a system failure. The concurrency control techniques and database recovery is discussed in Chapters 10 and 11, respectively.
- ⇒ **Performance optimization:** The DBMS has a set of functions that optimize the performance of the queries by evaluating the different execution plans of a query and choosing the best among them.

In this way, the DBMS provides an environment that is both convenient and efficient to use when there is a large volume of data and many transactions need to be processed concurrently.

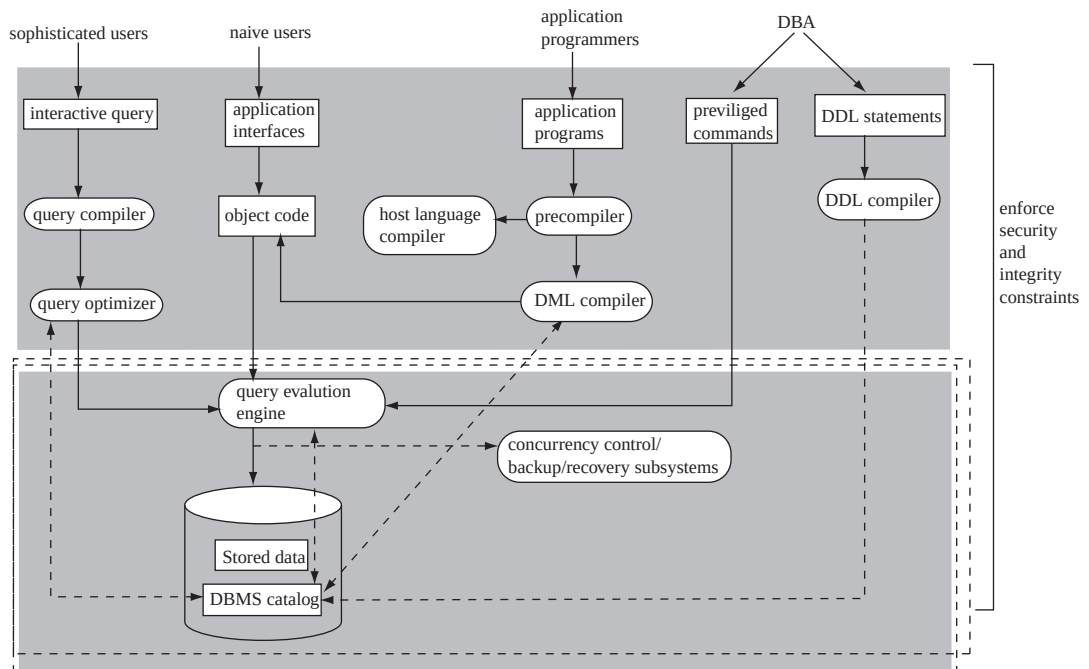


Fig.1.8 Component modules of DBMS

1.12 CENTRALIZED AND CLIENT/SERVER DATABASE SYSTEMS

In **centralized database systems**, the database system, application programs, and user-interface all are executed on a single system and dummy terminals are connected to it. The processing power of single system is utilized and dummy terminals are used only to display the information. As the personal computers became faster, more powerful, and cheaper, the database system started to exploit the available processing power of the system at the user's side, which led to the development of client/server architecture. In **client/server architecture**, the processing power of the computer system at the user's end is utilized by processing the user-interface on that system.

A **client** is a computer system that sends request to the server connected to the network, and a **server** is a computer system that receives the request, processes it, and returns the requested information back to the client. Client and server are usually present at different sites. The end users (remote database users) work on client computer system and database system runs on the server. Servers can be of several types, for example, file servers, printer servers, web servers, database servers, etc. The client machines have user interfaces that help users to utilize the servers. It also provides users the local processing power to run local applications on the client side.

There are two approaches to implement client/server architecture. In the first approach, the user interface and application programs are placed on the client side and the database system on the server side. This architecture is called **two-tier architecture**. The application programs that reside at the client side invoke the DBMS at the server side. The application program interface standards like Open Database Connectivity (ODBC) and Java Database Connectivity (JDBC) are used for interaction between client and server. Figure 1.9 shows two-tier architecture.

The second approach, that is, **three-tier architecture** is primarily used for web-based applications. It adds intermediate layer known as **application server** (or **web server**) between the client and the database server. The client communicates with the application server, which in turn communicates with the database server. The application server stores the business rules (procedures and constraints) used for accessing data from database server. It checks the client's credentials before forwarding a request to database server. Hence, it improves database security.

When a client requests for information, the application server accepts the request, processes it, and sends corresponding database commands to database server. The database server sends the result back to application server which is converted into GUI format and presented to the client. Figure 1.10 shows the three-tier architecture.

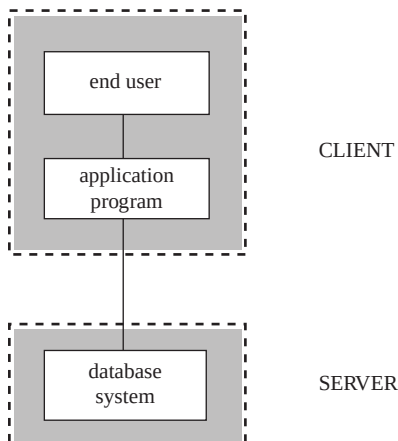


Fig. 1.9 Two-tier architecture

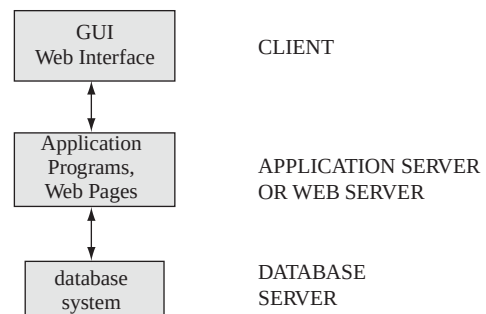


Fig. 1.10 Three-tier architecture

1.13 CLASSIFICATION OF DBMSs

The DBMSs can be classified into different categories on the basis of several criteria such as the data model they are using, number of users they support, number of sites over which the database is distributed, and the purpose they serve.

Based on Data Models

The various data models have already been discussed. Depending on the data model they use, the DBMSs can be classified as *hierarchical*, *network*, *relational*, *object-oriented*, and *object-relational*. Among these, the hierarchical and network data models are the older data models and now known as **legacy data models**. Some of the old applications still run on the database systems based on these models. Most of the popular and current commercial DBMSs are based on relational data model. The object-based data models have been implemented in some DBMSs; however, have not become popular. Due to the popularity of relational databases, the object-oriented concepts have been introduced in these databases that led to the development of a new class of DBMSs called object-relational DBMSs.

Learn More

MYSQL and PostgreSQL are open source (free) DBMS products that are supported by third-party vendors with additional services.

Based on Number of Users

Depending on the number of users the DBMS supports, it is divided into two categories, namely, *single-user system* and *multi-user system*. In **single-user system** the database resides on one computer and is only accessed by one user at a time. The user may design, maintain, and write programs for accessing and manipulating the database according to the requirements, as well as perform all the user roles. The user may also hire database system designers to design a system. In such a case, the single user performs the role of end user only. However, in most enterprises the large amount of data is to be managed and accessed by multiple users and thus, requires **multi-user systems**. In multi-user system, multiple users can access the database simultaneously. In multi-user DBMS, the data is both integrated and shared. For example, the *Online Book* database is a multi-user database system in which the data of books, authors, and publishers are stored centrally and can be accessed by many users.

Based on Number of Sites

Depending on the number of sites over which the database is distributed, it is divided into two types, namely, *centralized* and *distributed database systems*. **Centralized database systems** run on a single computer system. Both the database and DBMS software reside at a single computer site. The user interacts with the centralized system through a dummy terminal connected to it for information retrieval. In **distributed database systems**, the database and DBMS are distributed over several computers located at different sites. The computers communicate with each other through various communication media such as high-speed networks or telephone lines. Distributed databases can be classified as *homogeneous* and *heterogeneous*. In **homogeneous** distributed database system, all sites have identical database management system software, whereas in **heterogeneous** distributed database system, different sites use different database management system software.

Based on the Purpose

Depending on the purpose the DBMS serves, it can be classified as **general purpose** or **specific purpose**. DBMS is a general purpose software system. It can; however, be designed for specific purposes

such as airline or railway reservation. Such systems cannot be used for other applications without major changes. These database systems fall under the category of **online transaction processing (OLTP)** systems. Online transaction processing system is specifically used for data entry and retrieval. It supports large number of concurrent transactions without excessive delays. An automatic teller machine (ATM) for a bank is an example of online commercial transaction processing application. The OLTP technology is used in various industries, such as banking, airlines, supermarkets, manufacturing, etc.

1.14 DATABASE DESIGN PROCESS

The overall database design process has to follow a series of steps. The systematic process of designing a database is known as **design methodology**. Database design involves understanding operational and business needs of an organization, modeling the specified requirements, and realizing the requirements using a database. The goal of designing a database is to produce efficient, high quality, and minimum cost database. In large organizations, database administrator (DBA) is responsible for designing an efficient database system. He is responsible for controlling the database life-cycle process. The overall database design and implementation process consists of several phases.

1. **Requirement collection and analysis:** It is the process of knowing and analyzing the expectations of the users for the new database application in as much detail as possible. A team of analysts or requirement experts are responsible for carrying out the task of requirement analysis. They review the current file processing system or DBMS system, and interact with the users extensively to analyze the nature of business area to be supported and to justify the need for data and databases. The initial requirements may be informal, incomplete, inconsistent, and partially incorrect. The requirement specification techniques such as object-oriented analysis (OOA), data flow diagrams (DFDs), etc., are used to transform these requirements into better structured form. This phase can be quite time-consuming; however, it plays the most crucial and important role in the success of the database system. The result of this phase is the document containing the specification of user requirements.
2. **Conceptual database design:** In this phase, the database designer selects a suitable data model and translates the data requirements resulting from previous phase into a conceptual database schema by applying the concepts of chosen data model. The conceptual schema is independent of any specific DBMS. The main objective of conceptual schema is to provide a detailed overview of the organization. In this phase, a high-level description of the data and constraints are developed. The entity-relationship (E-R) diagram is generally used to represent the conceptual database design. The conceptual schema should be expressive, simple, understandable, minimal, and formal.
3. **Choice of a DBMS:** The choice of a DBMS depends on many factors such as cost, DBMS features and tools, underlying model, portability, and DBMS hardware requirements. The technical factors that affect the choice of a DBMS are the type of DBMS (relational, object, object-relational, etc.), storage structures and access paths that DBMS supports, the interfaces available, the types of high-level query languages, and the architecture it supports (client/server, parallel or distributed). The various types of costs that must be considered while choosing a DBMS are software and hardware acquisition cost, maintenance cost, database creation and conversion cost, personnel cost, training cost, and operating cost.
4. **Logical database design:** Once an appropriate DBMS is chosen, the next step is to map the high-level conceptual schema onto the implementation data model of the selected DBMS. In this phase, the database designer moves from an abstract data model to the implementation of the database. In case of relational model, this phase generally consists of mapping the E-R model into a relational schema.

5. **Physical database design:** In this phase, the physical features such as storage structures, file organization, and access paths for the database files are specified to achieve good performance. The various options for file organization and access paths include various types of indexing, clustering of records, hashing techniques, etc.
6. **Database system implementation:** Once the logical and physical database designs are completed, the database system can be implemented. DDL statements of the selected DBMS are used and compiled to create the database schema and database files, and finally the database is loaded with the data.
7. **Testing and evaluation:** In this phase, the database is tested and fine-tuned for the performance, integrity, concurrent access, and security constraints. This phase is carried out in parallel with application programming. If the testing fails, various actions are taken such as modification of physical design, modification of logical design or upgrade or change DBMS software or hardware.

Note that once the application programs are developed, it is easier to change the physical database design. However, it is difficult to modify the logical database design as it may affect the queries (written using DML commands) embedded in the program code. Thus, it is necessary to carry out the design process effectively before developing the application programs. While designing a database schema, it is necessary to avoid two major issues, namely, *redundancy* and *incompleteness*. These problems may lead to bad database design. The anomalies (an inconsistent, incomplete or contradictory state of the database) that can arise due to these problems are discussed in detail in Chapter 06.



NOTE *The database design phases, namely, conceptual, logical, and physical are discussed in subsequent chapters.*

● SUMMARY ●

1. The term data can be defined as a set of isolated and unrelated raw facts with an implicit meaning.
2. When the data is processed and converted into a meaningful and useful form, it is known as information.
3. A database can be defined as a collection of related data from which users can efficiently retrieve the desired information.
4. A Database Management System (DBMS) is an integrated set of programs used to create and maintain a database. The main objective of a DBMS is to provide a convenient and effective method of defining, storing, retrieving, and manipulating the data contained in the database.
5. The database and the DBMS software are collectively known as database system.
6. Database approach came into existence due to the bottlenecks of file processing system. In the database approach, the data is stored at a central location and is shared among multiple users. Thus, the main advantage of DBMS is centralized data management. Other advantages of DBMS are program-data independence, data abstraction, and support for multiple views of data.
7. Centralized data management in DBMS offers many advantages such as controlled data redundancy, data consistency, enforcing data integrity, data sharing, ease of application development, data security, multiple user interfaces, and backup and recovery.
8. The database systems are divided into three generations. The first generation includes the database systems based on hierarchical and network data model. The second generation includes the database systems based on relational data model and the third generation includes the database systems based on object-oriented and object-relational data model.
9. The most common database application areas are airlines and railways, banking, education, telecommunications, credit card transactions, e-commerce, health care information systems and electronic patient records, digital libraries and digital publishing, finance, sales, and human resources.

10. The people who work with databases include database users, system analysts, application programmers, and database administrator (DBA). Database administrator (DBA) is a person who has central control over both data and application programs.
11. The DBMS architecture describes how data in the database is viewed by the users. It is not concerned with how the data is handled and processed by the DBMS. In this architecture, the overall database description can be defined at three levels, namely, internal, conceptual, and external levels and thus, named three-level DBMS architecture. This architecture is proposed by ANSI/SPARC, and hence is also known ANSI/SPARC architecture.
12. Internal level is the lowest level of data abstraction that deals with the physical representation of the database on the computer. Conceptual level deals with the logical structure of the entire database and thus, is also known as logical level. External level is the highest level of abstraction that deals with the user's view of the database and thus is also known as view level.
13. The process of transforming the requests and results between various levels of DBMS architecture is known as mapping.
14. The main advantage of three-schema architecture is that it provides data independence. Data independence is the ability to change the schema at one level of the database system without having to change the schema at the other levels. Data independence is of two types, namely, logical data independence and physical data independence.
15. The process of highlighting only the essential features of a database system and hiding the storage and data organization details from the user is known as data abstraction.
16. A database model or simply a data model is an abstract model that describes how the data is represented and used. It not only describes the structure of the data but also defines a set of operations that can be performed on the data.
17. Depending on the concept they use to model the structure of the database, the data models are categorized into three types, namely, high-level or conceptual data models, representational or implementation data models, and low-level or physical data models.
18. Conceptual data model describes the information used by an organization in a way that is independent of any implementation-level issues and details.
19. The representational or implementation data models hide some data storage details from the users; however, can be implemented directly on a computer system. The various representational data models are hierarchical, network, relational, object-based, and semistructured data model.
20. Physical data model describes the data in terms of a collection of files, indices, and other storage structures such as record formats, record ordering, and access paths.
21. The overall design or description of the database is known as database schema or simply schema. The database schema is also known as intension of the database, and is specified while designing the database. The data in the database at a particular point of time is known as database instance or database state or snapshot. The database state is also called an extension of the schema.
22. To provide various facilities to different types of users, a DBMS normally provides one or more specialized programming languages often called Database (or DBMS) languages.
23. The DBMS mainly provides two database languages, namely, data definition language and data manipulation language to implement the databases.
24. Data definition language (DDL) is used for defining the database schema. The DBMS comprises DDL compiler that identifies and stores the schema description in the DBMS catalog. Data manipulation language (DML) is used to manipulate the database.
25. A database system being a complex software system is partitioned into several software components that handle various tasks such as data definition and manipulation, security and data integrity, data recovery and concurrency control, and performance optimization.

26. In client/server architecture, the client is the computer system that sends request to the server connected to the network and the server is a computer system that receives the request, processes it, and returns the requested information to the client.
27. There are two approaches of implementing client/server architectures, namely, two-tier architecture and three-tier architecture.
28. The DBMS can be classified into different categories on the basis of several criteria, namely, data models, number of users, number of sites, and purpose.

● KEY TERMS ●

- | | |
|--|---|
| <ul style="list-style-type: none"> ● Database ● Database management system (DBMS) ● File processing system ● Data redundancy ● Data inconsistency ● Centralized data management ● Program-data independence ● DBMS catalog ● Metadata ● Data abstraction ● View ● Database users ● Naive users ● Sophisticated users ● Specialized users ● System analysts ● Application programmers ● Database administrator (DBA) ● Database schema ● Three-level DBMS architecture (ANSI-SPARC architecture) ● Internal level ● Conceptual level ● External level ● Internal schema ● Conceptual schema ● External schema ● Mapping ● Data independence | <ul style="list-style-type: none"> ● Logical data independence ● Physical data independence ● Datamodel ● Conceptual data model ● Representational data model ● Hierarchical data model ● Network data model ● Relational data model ● Object-based data model ● Semistructured data model ● Physical data model ● Database instance ● Schema evolution ● Data Definition language (DDL) ● Storage Definition language (SDL) ● View Definition language (VDL) ● Data dictionary ● Data manipulation language (DML) ● Query ● Query language ● Client/server architecture ● Single-user system ● Multi-user system ● Distributed database system ● General purpose system ● Specific purpose system ● Online transaction processing (OLTP) system ● Design methodology |
|--|---|

● EXERCISES ●

A. Multiple Choice Questions

1. Which of these is not an advantage of database systems?
 - (a) ~~data~~ redundancy
 - (b) ~~Program~~-data independence
 - (c) Centralized data management
 - (d) Data abstraction
2. Which type of users query and update the database by invoking some already written application programs?
 - (a) ~~S~~ sophisticated users
 - (b) ~~N~~ aive users
 - (c) ~~p~~ social users
 - (d) ~~S~~ystem analysts
3. Which of these individuals plays an important role in defining and maintaining a database for an organization?
 - (a) ~~S~~ystem analyst
 - (b) Application programmer
 - (c) Database administrator
 - (d) All of these
4. Which of these levels deals with the physical representation of the database on the computer?
 - (a) ~~Ext~~ernal level
 - (b) ~~c~~onceptual level
 - (c) ~~I~~nternal level
 - (d) ~~d~~ata file
5. The ability to change the conceptual schema without affecting the external schemas or application programs is known as _____.
 - (a) Program-data independence
 - (b) Logical data independence
 - (c) Physical data independence
 - (d) Data abstraction
6. Which of these is not a representational data model?
 - (a) Entity-relationship model
 - (b) Hierarchical data model
 - (c) Relational data model
 - (d) Network data model
7. Which of these is not a feature of Hierarchical model?
 - (a) Organizes the data in tree-like structure
 - (b) Parent node can have any number of child nodes
 - (c) Root node does not have any parent
 - (d) Child node can have any number of parent nodes
8. Which of these data models is an extension of relational data model?
 - (a) Object-oriented data model
 - (b) Object-relational data model
 - (c) Semistructured data model
 - (d) None of these
9. The information about data in a database is called _____.
 - (a) ~~M~~etadata
 - (b) ~~h~~yper text
 - (c) Tera text
 - (d) ~~d~~ata file
10. Which of these DBMS languages is employed by end users and programmers to manipulate data in the database?
 - (a) Data definition language
 - (b) Data presentation language
 - (c) Data manipulation language
 - (d) Data translation language

B. Fill in the Blanks

1. The overall structure of the database is known as _____.
2. A _____ is an integrated set of programs used to create and maintain a database.
3. A _____ is a subset of the database that contains virtual data derived from the database files but it does not exist in physical form.
4. The process of transforming the requests and results between various levels of DBMS architecture is known as _____.
5. Data independence is of two types, namely, _____ and _____.
6. The process of highlighting only the essential features and hiding the storage and data organization details from the user is known as _____.
7. The process of making any changes in the schema is known as _____.
8. A _____ is a computer system that sends request to the server connected to the network, and _____ is a computer system that receives the request, processes it, and returns the requested information back to the client.
9. Depending on the number of users the DBMS supports, it is divided into two categories, namely, _____ and _____.
10. In a _____ distributed database system, all sites have identical database management system software.

C. Answer the Questions

1. Define the following terms:

(a) Database	(b) Database management system	(c) Database system
(d) Database schema	(e) Database instance	(f) Database table
(g) Database languages	(h) DBA	(i) Client/server architecture
(j) View	(k) Mapping	(l) Data abstraction
(m) Application server	(n) Database server	
2. Discuss the disadvantages of file processing system that led to the development of database system.
3. What are the advantages of a database system? What are the various cost and risk factors involved in implementing a database system?
4. What do you understand by the term data abstraction?
5. Discuss the various areas in which database system is used.
6. Who is DBA? List the various responsibilities of DBA.
7. What are the different types of database users who interact with the database system?
8. Explain the three-level architecture of DBMS with the help of an example. Mention its advantages also.
9. What is the difference between logical and physical data independence?
10. Illustrate the differences between hierarchical and network data models. Explain why relational model is preferred over the two.
11. What is the difference between object-based data models and record-based data models?
12. What are the roles of system analyst and application programmer in database system?

13. Write a short note on data definition language and data manipulation language.
14. Explain the various functional components of a DBMS with the help of a suitable diagram.
15. What is the difference between a centralized and a client/server database system?
16. How is a two-tier architecture different from a three-tier architecture?
17. Explain the different criteria on the basis of which DBMS is classified into different categories.
18. What is the aim of designing a database? Explain the overall database design and implementation process.

CONCEPTUAL MODELING

After reading this chapter, the reader will understand:

- Basic concept of entity-relationship model
- Entities and their attributes
- Entity types and entity set
- Different types of attributes
- Key attribute, and the concept of superkey, candidate key and primary key
- Concept of weak and strong entity types
- Relationships among various entity types
- Constraints on relationships
- Binary versus many relationships
- E-R diagram notations
- Features of enhanced E-R model that include specialization, generalization, and aggregation
- Constraints on specialization and generalization
- Alternative notations of E-R diagram
- Unified modeling language (UML), a standard language used to implement object data modeling
- Categories of UML diagrams that include structural diagrams and behavioural diagrams

As discussed in the earlier chapter, the overall database design and implementation process starts with requirement collection and analysis and specifications. Once the user's requirements have been specified, the next step is to construct an abstract or conceptual model of a database based on those requirements. This phase is known as **conceptual modeling** or **semantic modeling**. Conceptual modeling is an important phase in designing a successful database. It represents various pieces of data and their relationships at a very high-level of abstraction. It mainly focuses on what data is required and how it should be organized rather than what operations are to be performed on the data.

The conceptual model is a concise and permanent description of the database requirements that facilitates an easy communication between the users and the developers. It is simple enough to be understood by the end users; however, detailed enough to be used by a database designer to create the database. The model is independent of any hardware (physical storage) and software (DBMS) constraints and hence can be modified easily according to the changing needs of the user. The conceptual model can be represented using two major approaches, namely, *entity-relationship modeling* and the *object modeling*.

This chapter discusses the entity-relationship (E-R) model and its extended features, which are used for designing a conceptual model of any database application. It also discusses the entity-relationship diagram, a basic component of the E-R model used to graphically represent the data objects. An E-R diagram helps in modeling the data representation component of a software system. The other components of the software system such as user interactions with the system, functional modules and their interactions, etc. are specified with the help of Unified Modeling Language (UML), which is a standard methodology used in object modeling.

2.1 CONCEPTS OF ENTITY-RELATIONSHIP MODEL

The entity-relationship (E-R) model is the most popular conceptual model used for designing a database. It was originally proposed by Dr. Peter Chen in 1976 as a way to unify the network and relational database views. The E-R model views the real world as a set of basic objects (known as **entities**), their characteristics (known as **attributes**), and associations among these objects (known as **relationships**). The *entities*, *attributes*, and *relationships* are the basic constructs of an E-R model.

The information gathered from the user forms the basis for designing the E-R model. The *nouns* in the requirements specification document are represented as the entities, the *additional nouns* that describe the nouns corresponding to entities form the attributes, and the *verbs* become the relationships among various entities. The E-R model has several advantages listed as follows.

Things to Remember

The E-R model does not actually gives us a database description; rather it gives us an intermediate step from which we can easily create a database.

- ⇒ It is simple and easy to understand and thus, can be used as an effective communication tool between database designers and end users.
- ⇒ It captures the real-world data requirements in a simple, meaningful, and logical way.
- ⇒ It can be easily mapped onto the relational model. The basic constructs, that is, the entities and attributes of the E-R model can be easily transformed into relations (or tables) and columns (or fields) in a relational model.
- ⇒ It can be used as a design plan and can be implemented in any database management software.

2.1.1 Entity and its Attributes

An **entity** is any distinguishable object that has an independent existence in the real world. It includes all those ‘things’ of an organization about which the data is collected. For example, each book, publisher, and author in the *Online Book* database (discussed in Chapter 01) is an entity. An entity can exist either physically or conceptually. If an entity has a physical existence, it is termed as **tangible** or **concrete entity** (for example, a book, an employee, a place or a part) and if it has a conceptual existence, it is termed as **non-tangible** or **abstract entity** (for example, an event, a job title or a customer account).

Each entity is represented by a set of attributes. The **attributes** are the properties of an entity that characterize and describe it. Figure 2.1 shows the BOOK entity b_1 described by the attributes Book_title, Price (in dollars), ISBN, Year, Page_count and Category. Similarly, the PUBLISHER entity p_1 is described by the attributes P_ID, Name, Address, Phone and Email_ID. Here, b_1 and p_1 are the instances of the entity types. The entity types and instances are discussed later in this section.

Each attribute has a particular **value**. For example, the attributes Book_title, Price, ISBN, P_ID, Year, Page_count and Category can have the values *Ransack*, 22, 001-354-921-1, 2006, 200 and *Novel*, respectively, (as shown in Figure 2.1). Similarly, the attributes P_ID, Name, Address, Phone and Email_ID can have the values *P001*, *Hills Publications*, *12 Park Street Atlanta Georgia 30001*, *7134019*, *h_pub@hills.com*, respectively. Each attribute can accept a value from a set of permitted values, which is called the **domain** or the **value set** of the attribute. For example, the domain of the attribute Page_count is a set of positive integers. On the other hand, the category of a book can be textbook, language book or novel. Thus, the domain of the attribute Category is {*Textbook*, *Language book*, *Novel*}.

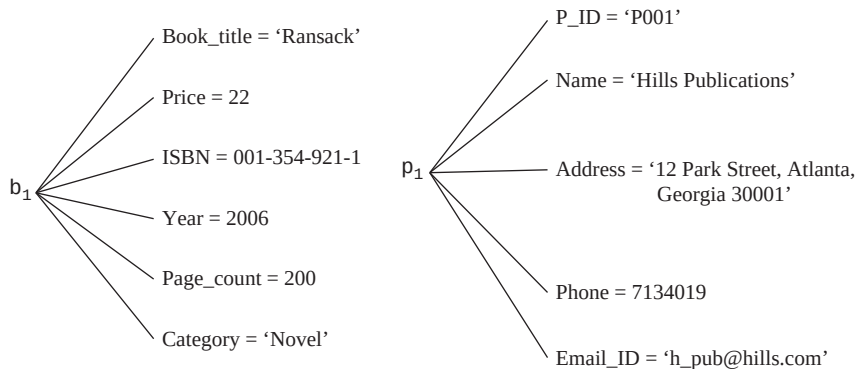


Fig. 2.1 Entity instances b_1 (BOOK) and p_1 (PUBLISHER), and their attributes

Entity Types and Entity Sets

A set or a collection of entities that share the same attributes but different values, is known as an **entity type**. For example, the set of all publishers who publish books can be defined as the entity type PUBLISHER. Similarly, a set of books written by the authors can be defined as the entity type BOOK. A specific occurrence of an entity type is called its **instance**. For example, the novel *Ransack* and the publisher *Hills Publications* are the instances of the entity types BOOK and PUBLISHER, respectively.

The collection of all instances of a particular entity type in the database at any point of time is known as an **entity set**. Each entity set is referred to by its name and attribute values. Note that both the entity set and the entity type are usually referred to by the same name. For example, BOOK refers to both the type of entity and the current set of all book entities. An entity type describes the **schema (intension)** for the entity set that shares the same structure. The entity set, on the other hand, is called the **extension** of the entity type. Figure 2.2 shows two entity types, BOOK and PUBLISHER, and a list of attributes for each of them. Some individual instances of each entity type are also shown along with their attribute values.

Entity Type: BOOK

Attributes: Book_title, Price, ISBN, Year, Page_count, Category

Entity set (extension):

- b_1 {Ransack, 22, 001-354-921-1, 2006, 200, Novel}
- b_2 {C++, 40, 001-987-760-9, 2005, 800, Textbook}
- b_3 {DBMS, 40, 002-678-980-4, 2006, 800, Textbook}

Entity Type: PUBLISHER

Attributes: P_ID, Name, Address, Phone, Email_ID

Entity set (extension):

- p_1 {P001, Hills Publications, 12 Park Street Atlanta Georgia 30001, 7134019, h_pub@hills.com}
- p_2 {P002, Sunshine Publishers Ltd, 45 Second Street Newark New Jersey 07045, 6548901, sun_shine@sunshine.com}
- p_3 {P003, Bright Publications, 123 Main Street Honolulu Hawaii 98702, 7678982, moon@moonlight.com}

Fig. 2.2 Entity types and entity sets

Types of Attributes

The attributes of an entity are classified into the following categories:

- ⇒ **Identifying and descriptive attributes:** The attribute that is used to uniquely identify an instance of an entity is known as **identifying attribute** or simply an **identifier**. For example, the attribute P_ID of the entity type PUBLISHER is identifying attribute, as two publishers cannot have the same publisher ID. Similarly, the attribute ISBN of the entity type BOOK is also an identifier, as two different books cannot have the same ISBN. A **descriptive attribute** or simply a **descriptor**, on the other hand, describes a non-unique characteristic of an entity instance. For example, the attributes Price and Page_count are the descriptive attributes as two books can have the same price and number of pages.
- ⇒ **Simple and composite attributes:** The attributes that are indivisible are known as **simple** (or **atomic**) **attributes**. For example, the attributes Book_title, Price, Year, Page_count, and Category of the entity type BOOK are simple attributes as they cannot be further divided into smaller subparts. On the other hand, **composite attributes** can be divided into smaller subparts. For example, the attribute Address can be further divided into House_number, Street, City, State and Zip_code (as shown in Figure 2.3). Composite attributes help in grouping the related attributes together. They are useful in those situations where the user sometimes wants to refer to an entire attribute and sometimes to only a component of the attribute. In case the user does not want to refer to the individual components, there is no need to subdivide the composite attribute into its component attributes.

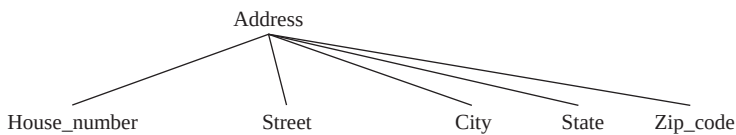


Fig. 2.3 Composite attribute

- ⇒ **Stored and derived attributes:** In some cases, two or more attribute values are related in such a way that the value of one attribute can be determined from the value of other attributes or related entities. For example, if the entity type PUBLISHER has an attribute Books_published, then it can be calculated by counting the book entities associated with that publisher. Thus, the attribute is known as **derived attribute**. Similarly, consider an entity type PERSON and its attributes Date_of_birth and Age. The value of the attribute Age can be determined from the current date and the value of Date_of_birth. Thus, the attribute Age is a derived attribute and the attribute Date_of_birth is known as **stored attribute**.
- ⇒ **Single-valued and multivalued attributes:** The attributes that can have only one value for a given entity are called the **single-valued attributes**. For example, the attribute Book_title is a single-valued attribute as one book can have only one title. However, in some cases, an attribute can have multiple values for a given entity. Such attributes are called **multivalued attributes**. For example, the attributes Email_ID and Phone of the entity type PUBLISHER are multivalued attributes, as a publisher can have zero, one or more e-mail IDs and phone numbers. The multivalued attributes can also have lower and upper bounds to restrict the number of values for each entity. For example, the lower bound of the attribute Email_ID can be specified to zero and upper bound to three. This implies that each publisher can have none or at most three e-mail IDs.
- ⇒ **Complex attributes:** The attributes that are formed by arbitrarily nesting the composite and multivalued attributes are called **complex attributes**. The arbitrary nesting of attributes is

represented by grouping components of a composite attribute between parentheses ‘()’ and by displaying multivalued attributes between braces ‘{}’. These attributes are separated by commas. For example, a publisher can have offices at different places each with different address and each office can have multiple phones. Thus, an attribute **Address_phone** for a publisher can be specified as given in Figure 2.4. Here, **Phone** is a multivalued attribute and **Address** is a composite attribute.

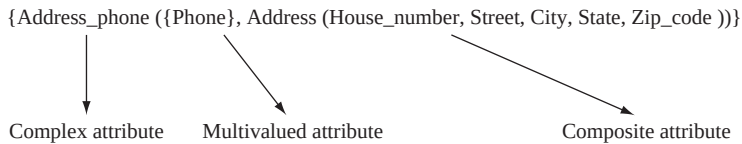


Fig. 2.4 Complex attribute — *Address_phone*

Null Values for the Attributes

Sometimes, there may be a situation where a particular entity may not have an appropriate value for an attribute. In such situations, the attribute takes a special value called **null** value. A null value of an attribute may indicate ‘not applicable’. For example, consider an entity type **PERSON** with three attributes **First_name**, **Middle_name** and **Last_name**. Since, not every person has a middle name, the person without any middle name will have a *null* value for the attribute **Middle_name**. It implies that the attribute **Middle_name** is not applicable to that person. A *null* value can also be used in situations where the value of a particular attribute is unknown. An unknown category can further be classified as *missing* (the value that exists, but is not available) and *not known* (it is not known that the value exists or not). For example, if the attribute **Book_title** has the value *null*, it indicates that the title of the book is missing, as every book must have a title. However, if the attribute **Phone** has the *null* value, it indicates that it is not known whether the value for **Phone** exists or not.

Key Attribute

The attribute (or combination of attributes) whose values are distinct for each individual instance of an entity type is known as a **key attribute**. It is used to uniquely identify each instance of an entity type. That is, no two entities can have the same value for the key attribute. For example, the attribute **ISBN** of the entity type **BOOK** is the key attribute, as two different books cannot have the same ISBN. An entity type may have more than one key attributes. For example, the attributes **P_ID** and **Name** of the entity type **PUBLISHER** are the key attributes, as no two publishers can have the same publisher ID and publisher name.

Superkey, Candidate Key, and Primary Key A set of one or more attributes, when taken together, helps in uniquely identifying each entity is called a **superkey**. For example, the attribute **P_ID** of the entity type **PUBLISHER** can be used to uniquely identify each entity instance. Thus, **P_ID** is a superkey. Similarly, the attribute **Name** of the entity type **PUBLISHER** is also a superkey. The combination of the attributes **P_ID** and **Name** is also a superkey. One can also use the set of all attributes (**P_ID**, **Name**, **Address**, **Phone**, **Email_ID**) to uniquely identify each entity instance of type **PUBLISHER**. Thus, the combination of all attributes is also a superkey.

However, **P_ID** itself can uniquely identify each entity instance, thus, other attributes are not required. Such a minimal superkey that does not contain any superfluous (extra) attributes in it is called a **candidate key**. A candidate key contains a minimized set of attributes that can be used to uniquely identify a single entity instance. For example, the attributes **P_ID** and **Name** are the candidate keys. However, the combination of **P_ID** and **Name** is not a candidate key. The candidate key,

which is chosen by the database designer to uniquely identify entities, is known as the **primary key**. For example, the attribute `P_ID` can be chosen as the primary key. If a primary key is formed by the combination of two or more attributes, it is known as **composite key**. Note that a composite key must be minimal, that is, all attributes that form the composite key must be included in the key attribute to uniquely identify the entities.



An entity type can have only one primary key; however, it can have multiple superkeys and candidate keys.

The attributes whose values are never or rarely changed are chosen as primary key. For example, the attribute `Address` cannot be chosen as a part of primary key as it is likely to be changed. Moreover, the attributes, which can have *null* values, cannot be taken as primary key. For example, the attributes `Phone` and `Email_ID` can have *null* values, as it is not necessary for every publisher to have an email ID or phone number.

Weak and Strong Entity Types

An entity type that does not have any key attribute of its own is called a **weak entity type**. On the other hand, an entity type that has a key attribute is called a **strong entity type**. The weak entity is also called a *dependent entity* as it depends on another entity for its identification. The strong entity is called an *independent entity*, as it does not rely on another entity for its identification. The weak entity has no meaning and relevance in an E-R diagram if shown without the entity on which it depends. For example, consider an entity type `EDITION` with attributes `Edition_no` and `Type` (Indian or International). It depends on another entity type `BOOK` for its existence, as an edition cannot exist without a book. Thus, `EDITION` is a weak entity type and `BOOK` is a strong entity type.

A weak entity type does not have its own identifying attribute that can uniquely identify each entity instance. It has a partial identifying attribute, which is known as **partial key** (or **discriminator**). A partial key is a set of attributes that can uniquely identify the instances of a weak entity type that are related to the same instance of a strong entity type. For example, the attribute `Edition_no` of the entity type `EDITION` can be taken as a partial key.

The primary key of a weak entity type is formed by the combination of its discriminator and the primary key of the strong entity type on which it depends for its existence. Thus, the primary key of the entity type `EDITION` is (`ISBN`, `Edition_no`). Since, the entities belonging to a weak entity type are identified by the attributes of a strong entity type in combination of one of their attribute, the strong entity types are known as **identifying** or **parent** entity type. The weak entity type, on the other hand, is known as **child** or **subordinate** entity type.

2.1.2 Relationships

An association between two or more entities is known as a **relationship**. A relationship describes how two or more entities are related to each other. For example, the relationship `PUBLISHED_BY` (shown in Figure 2.5) associates a book with its publisher. Like entity types and entity sets, relationship types and a relationship sets also exist. Consider n possibly distinct entity types $E_1, E_2, \dots, E_n$ (where, $n \geq 2$). A **relationship type** R among these n entity types defines a set of associations (known as **relationship set**) among instances from these entity types. Like entity type and entity set, the relationship type and relationship set are also referred by the same name, say R . A **relationship instance** represents an association between individual entity instances. For example, an association between the entity types `BOOK` and `PUBLISHER` represents a relationship type; however, an association between the instances `C++` and `P001` represents a relationship instance.



Each relationship instance includes only one instance of each entity type that is participating in the relationship.

Mathematically, a relationship type R can be defined as a subset of the Cartesian product $E_1 \times E_2 \times \dots \times E_n$. Similarly, the relationship set R can be defined as a set of relationship instances r_i , where each r_j associates n individual entity instances $e_1, e_2, \dots, e_n$ of each entity type $E_1, E_2, \dots, E_n$, respectively. Each of the individual entity types $E_1, E_2, \dots, E_n$ is said to **participate** in the relationship type R and each of the individual entity instances $e_1, e_2, \dots, e_n$ is said to participate in the relationship instance r_i . Figure 2.5 shows a relationship type **PUBLISHED_BY** between two entity types **BOOK** and **PUBLISHER**, which associates each book with its publisher. Here, $r_1, r_2, r_3, \dots, r_n$ denote relationship instances.

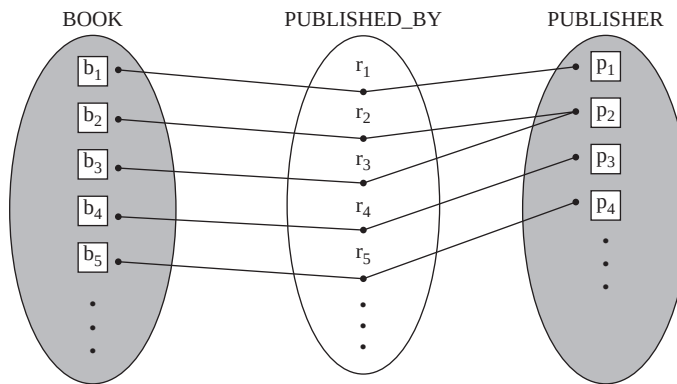


Fig. 2.5 Relationship **PUBLISHED_BY** between two entity types **BOOK** and **PUBLISHER**

A relationship type may also have descriptive attributes. Consider two entity types **AUTHOR** and **BOOK**, which are associated with the relationship type **REVIEWS**. Each author reviews some books and gives rating to that book. An attribute **Rating** could be associated with the relationship to specify the rating of the book given by the author (see Figure 2.6).

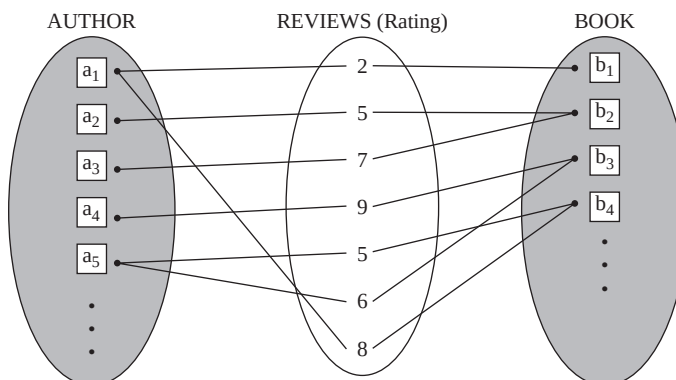


Fig. 2.6 Relationship type **REVIEWS** with attribute **Rating**

Identifying Relationship

The relationship between a weak entity type and its identifying entity type is known as **identifying relationship** of the weak entity type. For example, consider the entity types **EDITION** (weak entity type) and **BOOK** (strong entity type) and a relationship type **HAS** that exists between them. It specifies that each book has an edition. Since, an edition cannot exist without a book, the relationship type **HAS** is an identifying relationship as it associates a weak entity type with its identifying entity type.

Relationship Degree

The number of entity types participating in a relationship type determines the **degree of the relationship type**. The relationship that involves just one entity type is called **unary relationship**. In other words, the relationship that exists between the instances of the same entity type is a unary relationship. The relationship between two entity types is known as **binary relationship** and the relationship between three entity types is known as **ternary relationship**. In general, the relationship between n entity types is known as n -ary relationship. Thus, a unary relationship has a degree 1, a binary relationship has a degree 2, a ternary relationship has a degree 3, and an n -ary relationship has a degree n . The binary relationships are the most common relationships. For example, the relationship type **PUBLISHED_BY** between the entity types **BOOK** and **PUBLISHER** is a binary relationship.

Role Names and Recursive Relationships

Each entity type that participates in a relationship type plays a specific function or a role in the relationship. The role of each participating entity type is represented by the **role name**. If the participating entity types are distinct, their role names are implicit and are not usually specified. For example, in **PUBLISHED_BY** relationship, the entity type **BOOK** plays the role of a *book* and the entity type **PUBLISHER** plays the role of a *publisher*. Thus, the role names in this relationship are implicit.

Now, consider another entity **AUTHOR** which represents the authors who can write books as well as review the books written by other authors and give feedbacks to them. Thus, an author can be a writer as well as a reviewer. Thus, in a relationship type **FEEDBACKS** (given in Figure 2.7), the entity type **AUTHOR** plays two different roles (author and a reviewer). Such a relationship is called a **recursive relationship**. In such relationships, it is necessary to explicitly specify the role names to distinguish the meaning of each participation. Figure 2.7 shows a recursive relationship **FEEDBACKS**, where each relationship instance r_i has two lines each marked by 'r' for the role of a reviewer and 'w' for the role of an author, respectively. Here, the entity instance a_1 plays the role of a reviewer for the instances a_2 , a_3 , and a_4 and the instance a_2 plays the role of a reviewer for the instance a_5 .

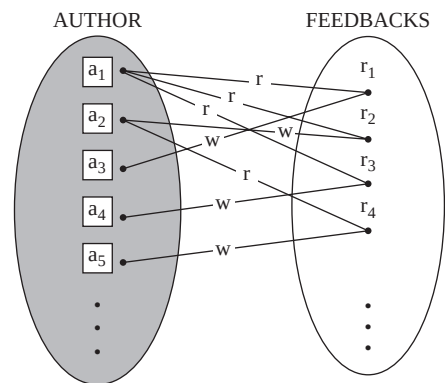


Fig. 2.7 Recursive relationship



A recursive relationship is always a unary relationship as it represents relationship between the instances of the same entity type.

Constraints on Relationship Type

Each relationship type has certain constraints that restrict the possible combination of instances participating in the relationship set. The constraints on the relationship type are of two types, namely, *mapping cardinalities* and *participation constraint*. These two constraints are collectively known as **structural constraints** of a relationship type.

Mapping Cardinalities The maximum number of instances of one entity type to which an instance of another entity type can relate to is expressed by the **mapping cardinalities** or **cardinality ratio**. The mapping cardinalities can be used to describe the relationships between two or more entities. However, they are most commonly used to describe the binary relationships. For a binary relationship between two entity types E_1 and E_2 , the mapping cardinalities can be of four types (see Figure 2.8).

Learn More

Like a binary relationship, a ternary relationship also has four types of mapping cardinalities, namely, one-to-one-to-one ($1 : 1 : 1$), one-to-one-to-many ($1 : 1 : M$) or many-to-one-to-one ($M : 1 : 1$), one-to-many-to-many ($1 : M : N$) or many-to-many-to-one ($M : N : 1$), and many-to-many-to-many ($M : N : P$).

- ⇒ **One-to-one:** In one-to-one mapping, each instance of entity type E_1 is associated with at most one instance of entity type E_2 and vice-versa. It is represented by $1 : 1$ relationship. For example, consider two entities PERSON and DRIVING_LICENSE. Each person can only have one driving license, and one driving license with a particular number can only be issued to a single person. That is, no two persons can have the same driving license. Thus, these two entities have $1 : 1$ relationship [see Figure 2.8(a)]. A one-to-one relationship is generally implemented by combining the attributes of both the entity types into one entity type. In this example, the DRIVING_LICENSE being a weak entity type can be specified as one of the attributes of the entity type PERSON.
- ⇒ **One-to-many:** In one-to-many mapping, each instance of entity type E_1 can be associated with zero or more (any number of) instances of type E_2 . However, an instance of type E_2 can only be associated to at most one instance of E_1 . It is represented by $1 : M$ relationship. For example, one publisher can publish many books; however, one book (with a particular ISBN) can only be published by one publisher. Thus, the entities PUBLISHER and BOOK have $1 : M$ relationship [see Figure 2.8(b)].
- ⇒ **Many-to-one:** In many-to-one mapping, each instance of entity type E_1 can be associated with at most one instance of type E_2 . However, an instance of type E_2 can be associated with zero or more (any number of) instances of type E_1 . It is represented by $M : 1$ relationship. For example, different books can be published by one publisher; however, one book can only be published by one publisher. Thus, the entities BOOK and PUBLISHER have $M : 1$ relationship [see Figure 2.8(c)].



NOTE A many-to-one relationship is same as one-to-many; however, from a different point of view.

- ⇒ **Many-to-many:** In many-to-many mapping, each instance of entity type E_1 can be associated with zero or more (any number of) instances of type E_2 and an instance of type E_2 can also be associated with zero or more (any number of) instances of type E_1 . It is represented by $M : N$ relationship. For example, one author can write many books and one book can be written by more than one authors. Thus, the entities AUTHOR and BOOK have $M : N$ relationship [see Figure 2.8(d)].

Participation Constraint The **participation constraint** specifies whether an entity instance can exist without participating in a relationship with another entity. In some notations, this constraint is also known as **minimum cardinality constraint**. The participation constraints can be of two types,

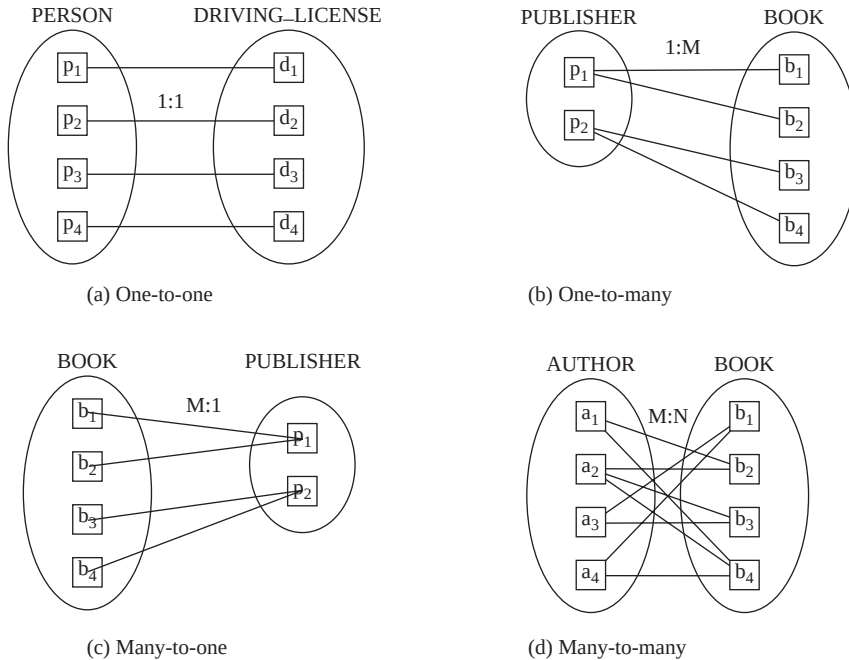


Fig. 2.8 Mapping cardinalities

namely, *total* and *partial*. If every instance of entity type *E* participates in at least one relationship instance of type *R*, the participation is said to be **total**. On the other hand, if only some of the instances of entity type *E* participate in the relationship, the participation is said to be **partial**.

The total participation constraint is also known as **mandatory participation**, which specifies that an entity instance cannot exist without participating in the relationship. It implies that the participation of each entity instance in a relationship type is mandatory. For example, the participation of each instance of the entity type *BOOK* in the relationship type *PUBLISHED_BY* is total, as each book must be published by a publisher. On the other hand, the partial participation constraint, also known as **optional participation**, specifies that an entity instance can exist without participating in the relationship. It implies that the participation of entity instance in a relationship type is optional. For example, the participation of the entity *PUBLISHER* in the relationship type *PUBLISHED_BY* is partial, as not all publishers publish books—some may publish only journals, magazines, and newspapers.



A weak entity type always has a total participation constraint, as each instance of weak entity type must be related to at least one of the instances of its parent entity type.

2.2 ENTITY-RELATIONSHIP DIAGRAM

Once the entity types, relationships types, and their corresponding attributes have been identified, the next step is to graphically represent these components using entity-relationship (E-R) diagram. An **E-R diagram** is a specialized graphical tool that demonstrates the interrelationships among various entities of a database.

Learn More

Entity-relationship models can be classified in two types, namely, *Binary Entity Relationship Model (BERM)* and *General Entity Relationship Model (GERM)*. In a BERM, only binary relationships are allowed; however, in GERM, relationships between three or more entities (*'n-ary'* relationships) are also allowed.

It is used to represent the overall logical structure of the database. While designing E-R diagrams, the emphasis is on the schema of the database and not on the instances. This is because the schema of the database is changed rarely; however, the instances in the entity and relationship sets change frequently. Thus, E-R diagrams are more useful in designing the database. An E-R diagram serves several purposes listed as follows:

- ⇒ It is used to communicate the logical structure of the database to the end users.
- ⇒ It helps the database designer in understanding the information to be contained in the database.
- ⇒ It serves as a documentation tool.

2.2.1 E-RDiagramNotations

There is no standard for representing the data objects in E-R diagrams. Each modeling methodology uses its own notation. The original notation used by Chen is the most commonly used notation. It consists of a set of symbols that are used to represent different types of information. The symbols include *rectangles*, *ellipses*, *diamonds*, *lines*, *double lines*, *double ellipses*, *dashed ellipses*, and *double rectangles*. All these symbols are described here and shown in Figure 2.9.

- ⇒ The entity types such as **BOOK**, **PUBLISHER**, etc. are shown in rectangular boxes labelled with the name of the entity types. Weak entity types such as **EDITION** are shown in double rectangles.
- ⇒ The relationship types such as **PUBLISHED_BY**, **REVIEWS**, **FEEDBACKS**, etc., are shown in diamond-shaped boxes labelled with the name of the relationship types. The identifying relationships such as **HAS** are shown in double diamonds.
- ⇒ The attributes of the entity types such as **Book_title**, **Price**, **Year**, etc., are shown in ellipses. The key attributes are also shown in ellipses; however, with their names underlined. Multivalued attributes such as **Email_ID** and **Phone** are shown in double ellipses and derived attributes are shown in dashed ellipses.
- ⇒ Straight lines are used to link attributes to entity types and entity types to relationship types. Double lines indicate the total participation of an entity type in a relationship type.
- ⇒ The cardinality ratios of each binary relationship type are represented by specifying a 1, M or N on each participating edge.

2.2.2 E-R Diagram Naming Conventions

Before initiating the designing of an E-R model, it is necessary to decide the format of the names to be given to the entity types, relationship types, attributes and roles, which will be used in the model. Some points must always be kept in mind while choosing the proper naming conventions.

- ⇒ The names should be chosen in such a way that the meaning of the context is conveyed as much as possible.
- ⇒ The usage of names should be consistent throughout the E-R diagram.
- ⇒ The usage of hyphens, spaces, digits, and special characters should be avoided.
- ⇒ The use of unpopular abbreviations and acronyms should be avoided.
- ⇒ Singular names should be used for entity types.
- ⇒ The binary relationship names should be chosen in such a way that makes the E-R diagram readable from left to right and from top to bottom. For example, consider the relationship type **PUBLISHED_BY** in Figure 2.12(a). This relationship type is read from left to right as ‘the entity type **BOOK** is *published by* the entity type **PUBLISHER**’. However, if the position of the entities in the E-R diagram is changed, the relationship type **PUBLISHED_BY** becomes **PUBLISHES** and then

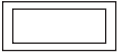
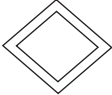
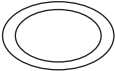
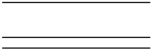
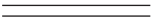
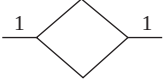
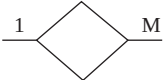
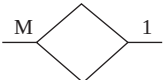
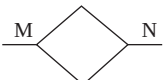
Notation	Purpose
	represents entity types
	represents weak entity types
	represents relationship type
	represents identifying relationship
	represents attributes
	represents multivalued attribute
	represents key attribute
	represents partial key of weak entity type
	represents derived attribute
	connects attributes to entity types and entity types to relationship types
	represents total participation
	represents 1:1 relationship
	represents 1:M relationship
	represents M:1 relationship
	represents M:N relationship

Fig. 2.9 E-R Diagram notations

it will be read as ‘the entity type PUBLISHER publishes the entity type BOOK’.

- ⇒ If a primary key of one entity type is used as a foreign key in another entity type, it should be used unchanged or with some extensions.

Learn More

There are many tools to construct E-R diagrams. Some of them are Avolution, ConceptDraw, ER/Studio, PowerDesigner, Rational Rose, SmartDraw, Microsoft Visio, and Visual Paradigm.

2.2.3 E-R Diagram for *Online BookDatabase*

As discussed in Chapter 01, the *Online Book* database maintains the details such as ISBN, title, price, year of publishing, number of pages, author and publisher of various textbooks, language books and novels. The user can also view the details of various authors and publishers. The author details include the name, address, URL, and phone number. The publisher details include the publisher name, address, e-mail ID, and phone number. The database also maintains the details of the ratings and feedbacks of other authors about a particular book. The ratings of a particular book are displayed along with the other details of the book. However, the detailed feedback about the book can be viewed on the URL of the authors who have reviewed the book.

Based on this specification, the E-R diagram for *Online Book* database can be designed. This section discusses the representation of all the entity types, relationship types, attributes, role names and recursive relationships, weak entity types, and identifying relationship, and cardinality ratios and participation constraints of *Online Book* database in an E-R diagram.

Representing Entity Types and Attributes

The *Online Book* database consists of three entity types, namely, BOOK, PUBLISHER, and AUTHOR. The entity type BOOK consists of the attributes Book_title, Price, ISBN, Year, Page_count, and Category. The entity type PUBLISHER has the attributes P_ID, Name, Address, Phone, and Email_ID. The entity type AUTHOR has the attributes A_ID, Name, Address, URL, and Phone. The attributes ISBN, P_ID, and A_ID are the key attributes. All these entities with their corresponding attributes are shown in Figure 2.10

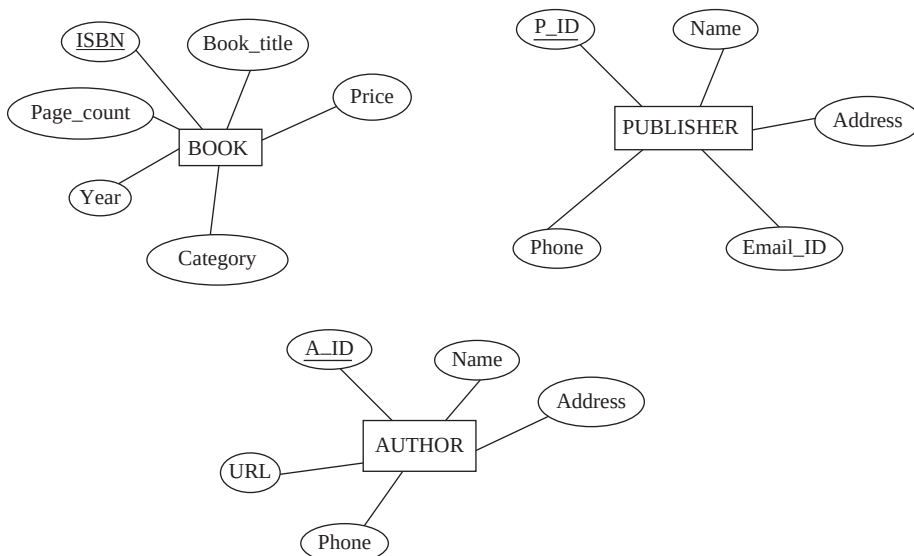


Fig. 2.10 Entity types of *Online Book* database

Representing Multivalued, Composite, and Derived Attributes

The attributes **Phone** and **Email_ID** are the multivalued attributes and **Address** is a composite attribute comprising **House_number**, **Street**, **City**, **State** and **Zip_code** as its component attributes. The **Name** of the author can also be taken as composite attribute with **First_name**, **Middle_name**, and **Last_name** as its component attributes. The composite attributes are shown by attaching ellipses containing the component attributes. Figure 2.11 shows the multivalued and composite attributes. It also shows some attributes in dashed ellipses. These attributes are the derived attributes. For example, the value of the attribute **Books_published** (for the **PUBLISHER**) can be calculated by counting the book entities associated with that publisher. Similarly, the attribute **Books_written** (for the **AUTHOR**) can be calculated by counting the book entities associated with that author.

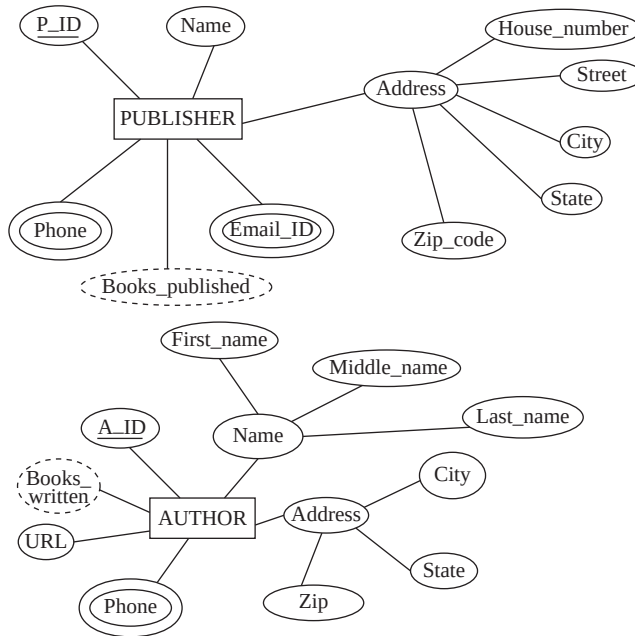


Fig. 2.11 Multivalued, composite, and derived attributes

Representing Relationship Types and Their Attributes

The entity types **BOOK** and **PUBLISHER** are associated with a relationship type **PUBLISHED_BY** by **M : 1** relation. That is, many books can be published by one publisher; however, one book of a particular ISBN can be published by one publisher only. The entity types **AUTHOR** and **BOOK** are associated by the relation type **WRITES** by **M : N** relation. That is, one book can be written by many authors and one author can write many books. The entity types **AUTHOR** and **BOOK** are also associated with the relationship type **REVIEWS** by **M : N** relation, which specifies that one author can review many books and one book can be reviewed by many authors.

The participation (total or partial) of each entity in the relationship type can also be shown in an E-R diagram. For example, the participation of entity type **BOOK** in the relationship type **PUBLISHED_BY** is total, as every book must be published by a publisher. Similarly, the participation of entity types **BOOK** and **AUTHOR** in the relationship type **WRITES** is also total, as every book must be written by at least one author and each author must write at least one book. However, the participation of the entity types **BOOK** and **AUTHOR** in the relationship type **REVIEWS** is partial, as some books may not be reviewed by any author and some authors may not review any book. The E-R diagrams

showing the relationship type **PUBLISHED_BY**, **WRITES** and **REVIEWS**, the participation of each entity in the relationship types, and their descriptive attributes are shown in Figure 2.12.

If a relationship type has some descriptive attributes, then these attributes can also be shown in an E-R diagram. Figure 2.12 shows the relationship types **PUBLISHED_BY** and **REVIEWS** with the descriptive attributes **Copyright_date** and **Rating**, respectively. The attribute **Copyright_date** specifies the date on which the publisher gets the right of publishing the book for the first time. The attribute **Rating** contains a rated value (between 1 and 10) given to the book by other authors after reviewing the book.

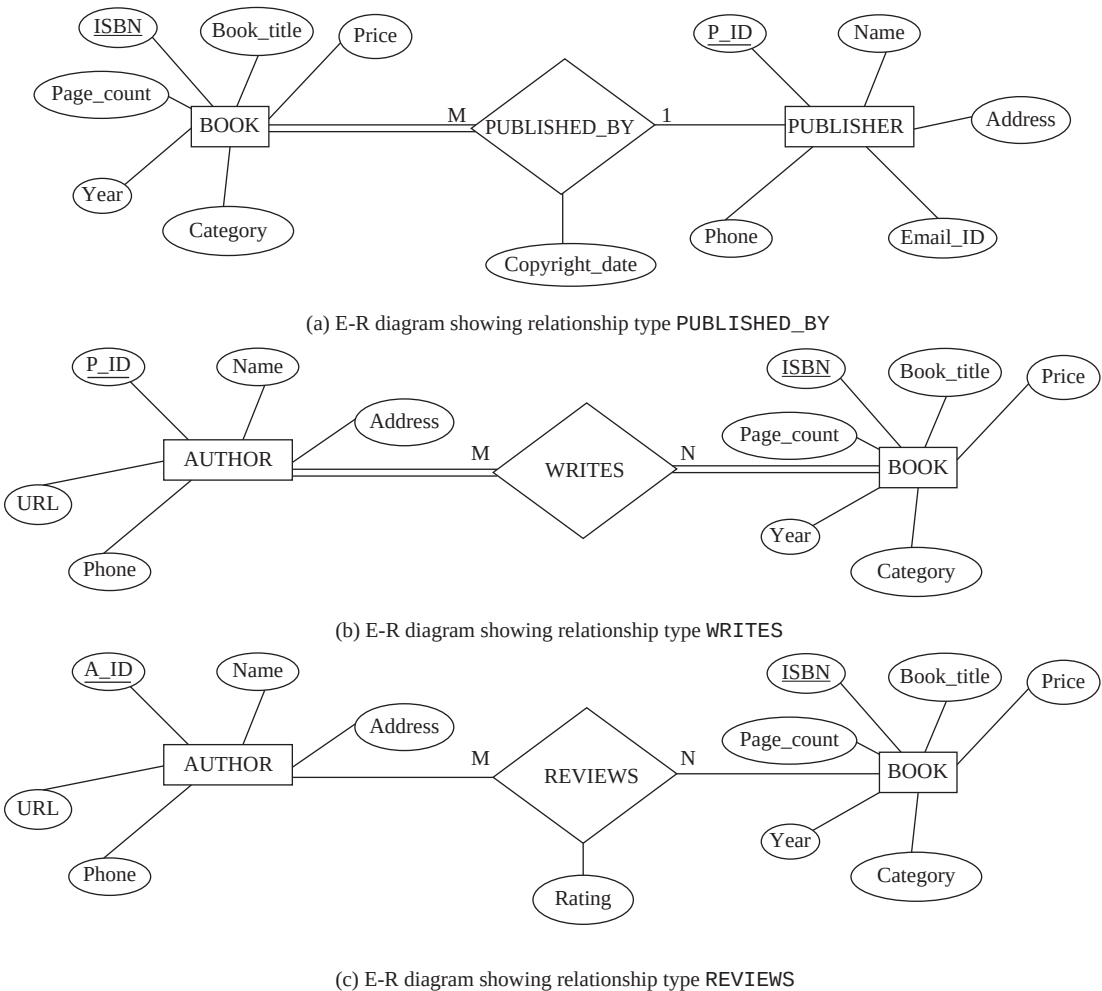


Fig. 2.12 E-R diagrams showing relationship between **BOOK**, **AUTHOR**, and **PUBLISHER**

Representing Role Names and Recursive Relationships

An E-R diagram can also be used to represent the role names and recursive relationships. For example, the author of one book acts as a reviewer for other books. Thus, a recursive relationship exists between the instances of the same entity type **AUTHOR**. This recursive relationship with the role names is shown in Figure 2.13. The role names (*reviewer* and *author*) are indicated by labelling the lines connecting the relationship types to entity types. Since, the relationship type **FEEDBACKS** involves only one entity type, it is a unary relationship.

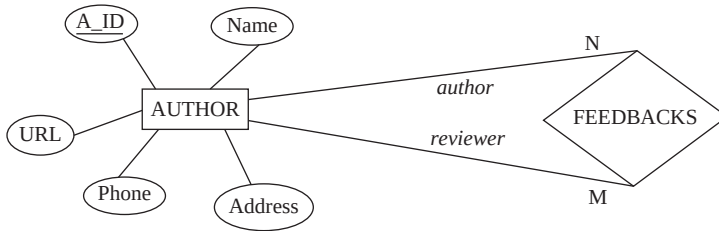


Fig. 2.13 E-R diagram showing role names and recursive relationship

Representing Weak Entity Types and Identifying Relationships

In the *Online Book* database, the entity type **EDITION** is a weak entity type with **Edition_no** as its partial key. It depends on the strong entity type **BOOK**. It has a total participation in the identifying relationship type **HAS**, which specifies that an edition cannot exist without a book (see Figure 2.14).

The complete E-R diagram for the *Online Book* database is shown in Figure 2.15.

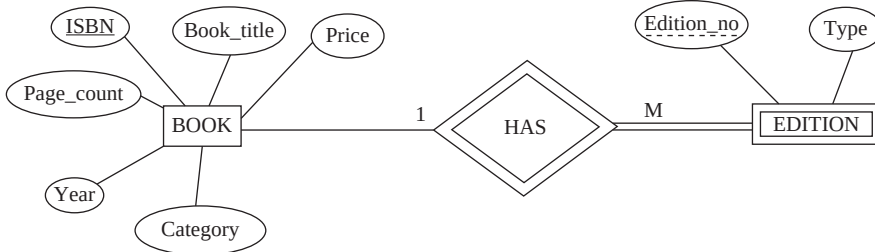


Fig. 2.14 E-R diagram showing weak entity, identifying relationship, and partial key

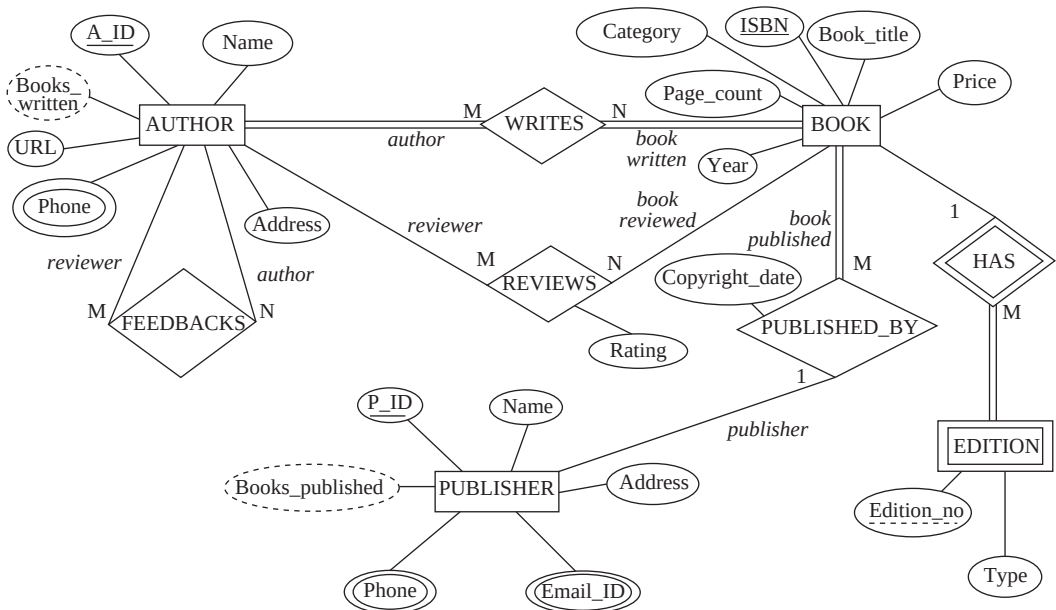


Fig. 2.15 E-R diagram of *Online Book* database

2.2.4 E-R Diagram Representing Ternary Relationships

The E-R diagram shown in Figure 2.15 includes only unary and binary relationships. An E-R diagram can also be used to represent higher degree relationships (relationships of degree greater than two). The most common higher degree relationship is ternary relationship, which describes the relationship between three entity types. For example, consider the entity types **AUTHOR**, **BOOK**, and **SUBJECT**. The entity type **SUBJECT** has the attributes **S_ID** and **Subject_name**. The subject name could be Database Systems, C++, C, Computer Architecture, etc.

Each author writes a book on a particular subject. This relationship is ternary as it associates three entity types. Figure 2.16 shows the relationship type **WRITES** among three entity types **AUTHOR**, **BOOK**, and **SUBJECT**. The mapping cardinality of the relationship type **WRITES** is **M : 1 : 1**, which implies that a group of particular authors (many authors) can write at most one book on a particular subject.

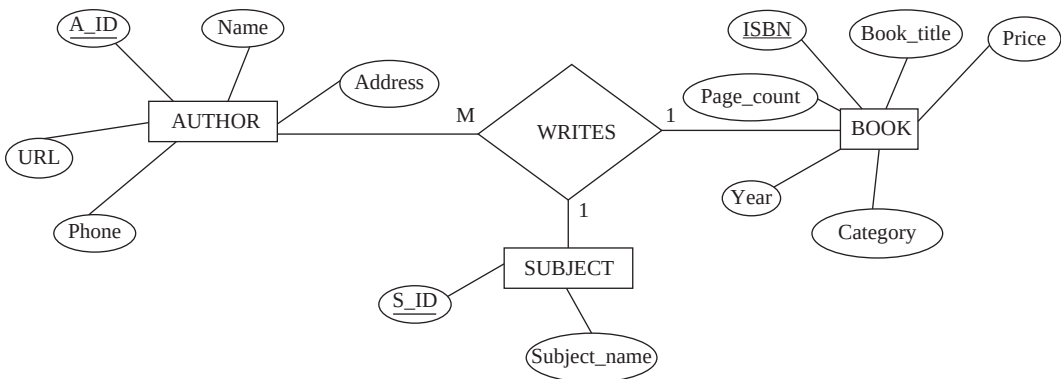


Fig. 2.16 E-R Diagram showing a ternary relationship



NOTE Relationships beyond ternary relationships are not very common; however, can sometimes exist. The discussion of the relationships greater than the degree three is beyond the scope of this book.

2.2.5 E-R Diagram—Design Choices

While designing an E-R diagram, a number of design choices are to be considered. In other words, it is possible to represent a set of entities and their relationships in several ways. This section discusses each of these choices.

Entity Types versus Attributes

When the attributes of a particular entity type are identified, it becomes difficult to determine whether a property of the entity should be represented as an entity type or as an attribute. For example, consider the entity type **AUTHOR** with attributes **A_ID**, **Name**, **Address**, **Phone**, and **URL**. Treating phone as an attribute implies that each author can have at the most one telephone number. In case multiple phone numbers have to be stored for each author, the attribute **Phone** can be taken as a multivalued attribute. However, treating telephone number as a separate entity type better models a situation where the user may want to store some extra information such as location (*home*, *office* or *mobile*) of the telephone. Thus, **Phone** can be taken as a separate entity type with attributes **Phone_number** and **Location**, which implies that an author can have multiple phone numbers at different locations.

After making **Phone** as an entity type, the E-R diagram for the entity type **AUTHOR** can be redesigned as shown in Figure 2.17. In this figure, the entity types **AUTHOR** and **PHONE** are related with the relationship type **HAS_PHONE** with the cardinality ratio 1 : M. It implies that one author can have many phone numbers; however, one phone number can only be associated with one author.

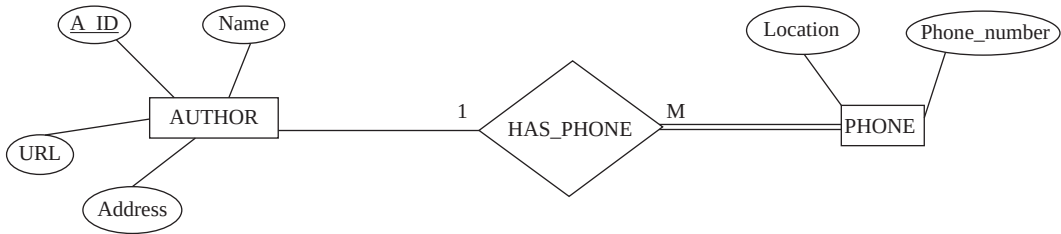


Fig. 2.17 E-R Diagram showing **PHONE** as an entity type

Binary versus Ternary Relationship

Generally, a database contains binary relationships, as it is easy to represent and understand the binary relationships. However, in some cases, ternary relationships need to be created to represent certain situations. For example, consider the ternary relationship **WRITES** in Figure 2.16. Its equivalent E-R diagram comprising three distinct binary relationships, **WRITES_ON**, **WRITES** and **OF** with cardinality ratios $M : N$, $M : N$, and $M : 1$, respectively, is shown in Figure 2.18. By comparing Figures 2.16 and 2.18, it appears that the relationships between three entity types have changed from a single $M : 1 : 1$ to three binary relationships—two of them with $M : N$ mapping cardinality and one with $M : 1$ mapping cardinality.

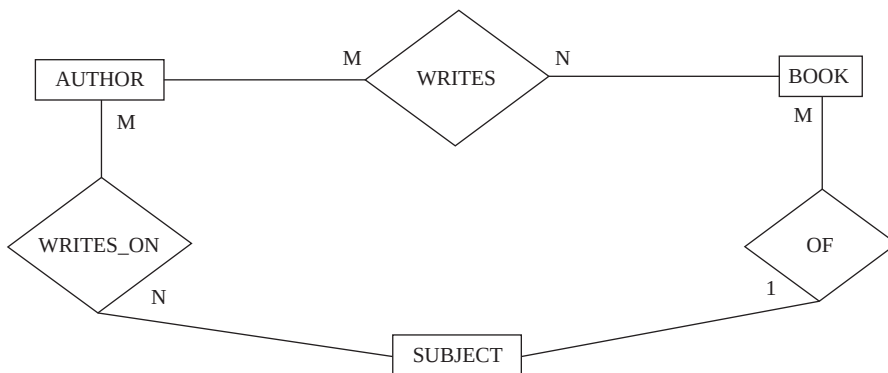


Fig. 2.18 E-R Diagram showing three binary relationships

The E-R diagram shown in Figure 2.16 conveys the meaning that a group of particular authors can write at most one book on a particular subject. That is, same group of authors cannot write many books on the same subject. On the other hand, the E-R diagram shown in Figure 2.18 conveys three different relationships. The relationship **WRITES** shows that one author can write many books and one book can be written by many authors. The relationship **WRITES_ON** shows that one author can write books on many subjects and books on one subject can be written by many authors. Finally, the relationship **OF** shows that many books can be written on a particular subject; however, one book can belong to only one subject. The information conveyed by these three binary relationships is not same

as that of the ternary relationship shown in Figure 2.16. Thus, it is the responsibility of the database designer to make an appropriate decision depending on the situation being represented.

2.3 ENHANCED E-R MODEL

The E-R diagrams discussed so far represents the basic concepts of a database schema. However, some aspects of a database such as inheritance among various entity types cannot be expressed using the basic E-R model. These aspects can be expressed by enhancing the E-R model. The resulting diagrams are known as **enhanced E-R** or **EER diagrams** and the model is called **EER model**.

The basic E-R model can represent the traditional database applications such as typical data processing application of an organization effectively. On the other hand, the EER model is used to represent the new and complex database applications such as telecommunications, Geographical Information Systems (GIS), etc. This section discusses the extended E-R features including *specialization*, *generalization*, and *aggregation* and their representation using EER diagrams.

2.3.1 Specialization and Generalization

In some situations, an entity type may include sub-groupings of its entities in such a way that entities of one subgroup are distinct in some way from the entities of other subgroups. For example, the entity type **BOOK** can be classified further into three types, namely, **TEXTBOOK**, **LANGUAGE_BOOK**, and **NOVEL**. These entity types are described by a set of attributes that includes all the attributes of the entity type **BOOK** and some additional set of attributes that differentiate them from each other. These additional attributes are also known as **local** or **specific** attributes. For example, the entity type **TEXT-BOOK** may have the additional attribute **Subject** (for example, Computer, Maths, Science, etc.), **LANGUAGE_BOOK** may have the attribute **Language** (for example, French, German, Japanese, etc.), and the entity type **NOVEL** may have the attribute **Type** (Fiction, Mystery, Fantasy, etc.). This process of defining the subgroups of a given entity type is called **specialization**.

The entity type containing the common attributes is known as the **superclass** and the entity type, which is a subset of the superclass, is known as its **subclass**. For example, the entity type **BOOK** is a superclass and the entity types **TEXTBOOK**, **LANGUAGE_BOOK**, and **NOVEL** are its subclasses. This process of refining the higher-level entity types (superclass) into lower-level entity types (subclass) by adding some additional features to each of them is a **top-down** design approach.

The design process may also follow a **bottom-up** approach in which multiple lower-level entity types are combined on the basis of common features to form higher-level entity types. For example, the database designer may first identify the entity types **TEXTBOOK**, **LANGUAGE_BOOK**, and **NOVEL** and then combine the common attributes of these entity types to form a higher-level entity type **BOOK**. This process is known as **generalization**.



In simple terms, generalization is the reverse of specialization.

The two approaches are different in terms of their starting and ending point. Specialization starts with a single higher-level entity type and ends with a set of lower-level entity types having some additional attributes that distinguish them from each other. Generalization, on the other hand, starts with the identification of a number of lower-level entity types and ends with the grouping of the common attributes to form a single higher-level entity type. Generalization represents the similarities among lower-level entity types; however, suppresses their differences.

Specialization and generalization can be represented graphically with the help of an EER diagram in which the superclass is connected with a line to a circle, which in turn is connected by a line to each subclass that has been defined (see Figure 2.19). The ‘ $\cup$ ’ shaped symbol on each line connecting a subclass to the circle indicates that the subclass is a subset ‘ $\subset$ ’ of the superclass. The circle can be ‘ ’ (empty) or it can contain a symbol **d** (for disjointness) or **o** (for overlapping), which is explained later in this section.

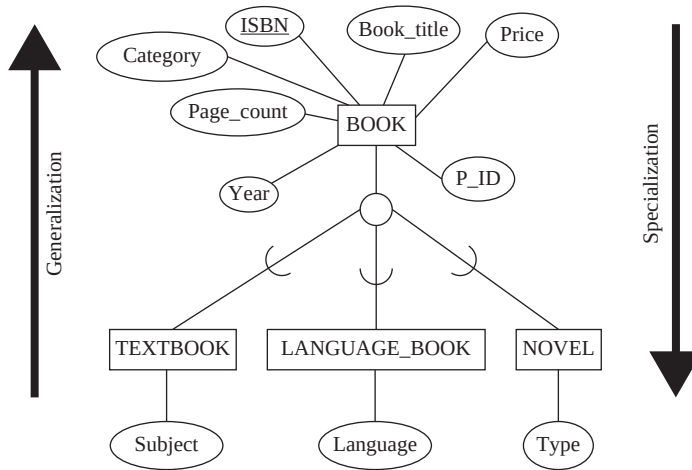


Fig. 2.19 Specialization and generalization

Attribute Inheritance

As discussed earlier, the higher-level and lower-level entity types are created on the basis of their attributes. The higher-level entity type (or superclass) has the attributes that are common to all of its lower-level entity types (or subclasses). These common attributes of the superclass are inherited by all its subclasses. This property is known as **attribute inheritance**. For example, the attributes ISBN, Book_title, Category, Price, Page_count, Year, and P_ID of the entity type BOOK are inherited by the entity types TEXTBOOK, LANGUAGE_BOOK, and NOVEL. This inheritance of entity types represents a hierarchy of classes.

Note that all the lower-level entity types of a particular higher-level entity types also participate in all the relationship types in which the higher-level entity type participates. For example, the entity type BOOK participates in the relationship type PUBLISHED_BY, its subclasses TEXTBOOK, LANGUAGE_BOOK, and NOVEL also participate in this relationship type. That is, all textbooks, language books, and novels are published by a publisher.



All the attributes and relationship types of a higher-level entity type are also applied to its lower-level entity types but not vice-versa.

If a subclass is a subset of only one superclass, that is, it has only one parent, it is known as **single inheritance** and the resulting structure is known as a **specialization (or generalization) hierarchy**. For example, Figure 2.19 shows a single inheritance since the three subclasses are derived from only one superclass. On the other hand, if a subclass is derived from more than one superclass, it is known as **multiple inheritance** and the resulting structure is known as a **specialization (or generalization)**

lattice. In this case, the subclass is known as a **shared subclass**. For example, consider two entity types EMPLOYEE and STUDENT. A student who is working as a part-time employee in an organization as well as pursuing a course from a university can be derived from these two entity types. Figure 2.20 shows a specialization lattice with a shared subclass PARTTIME_EMPLOYEE.

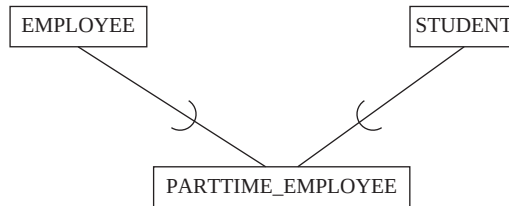


Fig. 2.20 Specialization lattice with shared subclass PARTTIME_EMPLOYEE

Constraints on Specialization and Generalization

To design an accurate and effective model for a particular organization, some constraints can be placed on specialization or generalization. All the constraints can be applied to both generalization and specialization. However, in this section the constraints are discussed in terms of specialization only.

Constraints on Subclass Membership In general, a single higher-level entity type (superclass) may have several lower-level entity types (subclasses). Thus, it is necessary to determine which entities of the superclass can be the members of a given subclass. The membership of the higher-level entities in the lower-level entity types can be either condition-defined or user-defined.

⇒ **Condition-defined:** In condition-defined, the membership of entities is determined by placing a condition on the value of some attribute of the superclass. The subclasses formed on the basis of the specified condition are called **condition-defined** (or **predicate-defined**) **subclasses**. For example, all the instances of type BOOK can be evaluated based on the value of the attribute Category. The entities that satisfy the condition Category = "Textbook" are allowed to belong to the subclass TEXTBOOK only. Similarly, the entities that satisfy the condition Category = "Language Book" are allowed to belong to the subclass LANGUAGE_BOOK only. Lastly, the entities that satisfy the condition Category = "Novel" are allowed to belong to the NOVEL subclass only. Thus, these conditions are the constraints that determine which entities of the superclass can belong to a given subclass.

If the membership condition for all subclasses in specialization is defined on the same attribute (in this case, Category), it is known as an **attribute-defined specialization**, and the attribute is called the **defining attribute** of the specialization. The condition-defined subclass is shown by writing the condition next to the line connecting the subclass to the specialization circle (see Figure 2.21). The symbol **d** denotes the disjointness constraint.

⇒ **User-defined:** If the membership of the superclass entities in a given subclass is determined by the database users and not by any membership condition, it is called **user-defined specialization**, and the subclasses that are formed are known as **user-defined subclasses**. For example, consider an entity CELEBRITY. A celebrity could be a film star, a politician, a newsreader or a player. Thus, the assignment of the entities of type CELEBRITY to a given subclass is specified explicitly by the database users when they apply the operation to add an entity to the subclass (see Figure 2.22). The symbol **o** denotes the overlapping constraint.

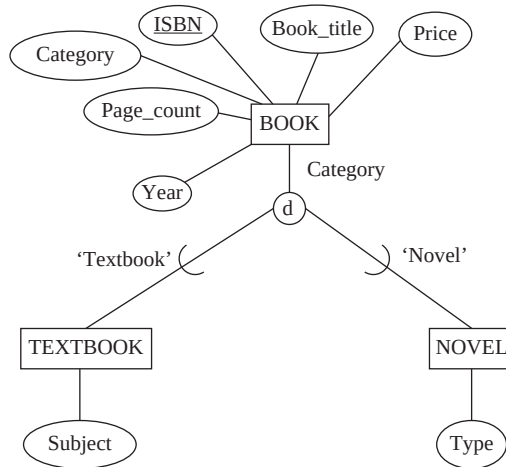


Fig. 2.21 EER diagram for an attribute-defined specialization with disjoint constraint

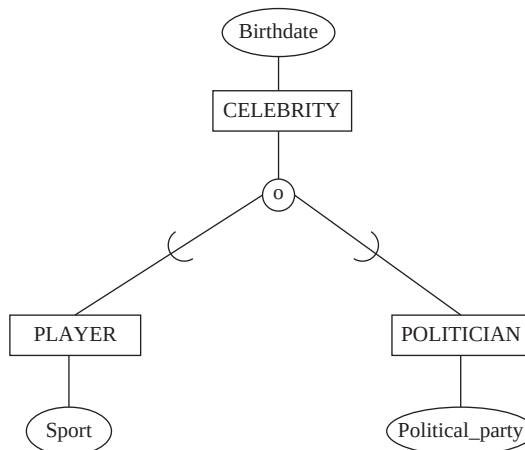


Fig. 2.22 EER diagram for a user-defined specialization with overlapping constraint

Disjoint and Overlapping Constraints In addition to subclass membership constraints, two other types of constraints, namely, *disjointness* and *overlapping*, can also be applied to a specialization. These constraints determine whether a higher-level entity instance can belong to more than one lower-level entity type within a single specialization or not.

- ⇒ **Disjointness constraint:** This constraint specifies that the same higher-level entity instance cannot belong to more than one lower-level entity types. That is, the subclasses of any superclass must be disjoint. For example, an entity of type **BOOK** can belong to either **TEXTBOOK** or **NOVEL** but not both. An attribute-defined specialization in which the defining attribute is single-valued implies a disjointness constraint. The disjoint constraint is represented by a symbol **d** written in a circle in an EER diagram as shown in Figure 2.21.
- ⇒ **Overlapping constraint:** This constraint specifies that the same higher-level entity instance can belong to more than one lower-level entity types. That is, the subclasses of any superclass need not to be disjointed and entities may overlap. In terms of EER diagram, the overlapping constraint

is represented by a symbol **o** written in a circle joining the superclass with its subclasses. For example, the entity types **PLAYER** and **POLITICIAN** show an overlapping constraint, as the celebrity can be a player as well as a politician (see Figure 2.22). Similarly, an entity of type **BOOK** can belong to both **TEXTBOOK** and **LANGUAGE_BOOK** since a language book can also be a prescribed textbook in a university [see Figure 2.23(b)].



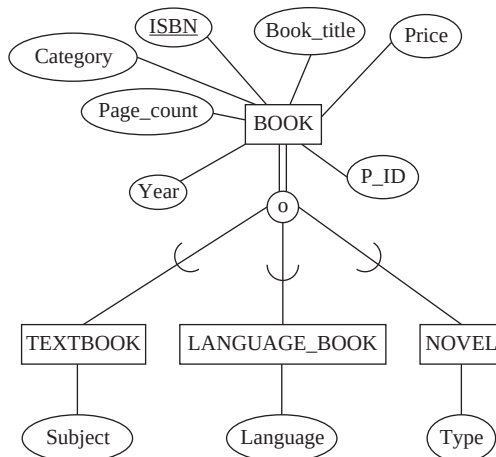
NOTE The lower-level entity types are by default overlapped; however, the disjointness constraint must be explicitly placed on generalization and specialization.

Completeness Constraint The last constraint that can be applied to generalization or specialization is the completeness constraint. It determines whether an entity in higher-level entity set must belong to at least one of the lower-level entity sets or not. The completeness constraint can be total or partial.

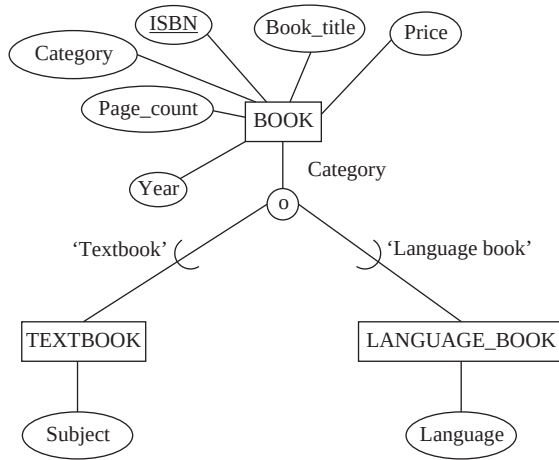
- ⇒ **Total specialization:** It specifies that each higher-level entity must belong to at least one of the lower-level entity types in the specialization. Figure 2.23(a) shows the total specialization of the entity type **BOOK**. Here, each book entity must belong to either textbook or language book or novel category. The total specialization is represented by double lines connecting the superclass with the circle.
- ⇒ **Partial specialization:** It allows some of the instances of higher-level entity type not to belong to any of the lower-level entity types. Figure 2.23(b) shows the partial specialization of the entity type **BOOK**, as all books do not necessarily belong to the category textbooks or language books, some may belong to the category novels also.

2.3.2 Aggregation

The E-R diagrams discussed so far represent the relationships between two or more entities. An E-R diagram cannot represent the relationships among relationships. However, in some situation, it is necessary to represent a relationship among relationships. The best way to represent these types of situations is aggregation. The process through which one can treat the relationships as higher-level entities is known as **aggregation**. For example, in the *Online Book* database, the relationship **WRITES** between the entity types **AUTHOR** and **BOOK** can be treated as a single higher-level entity called



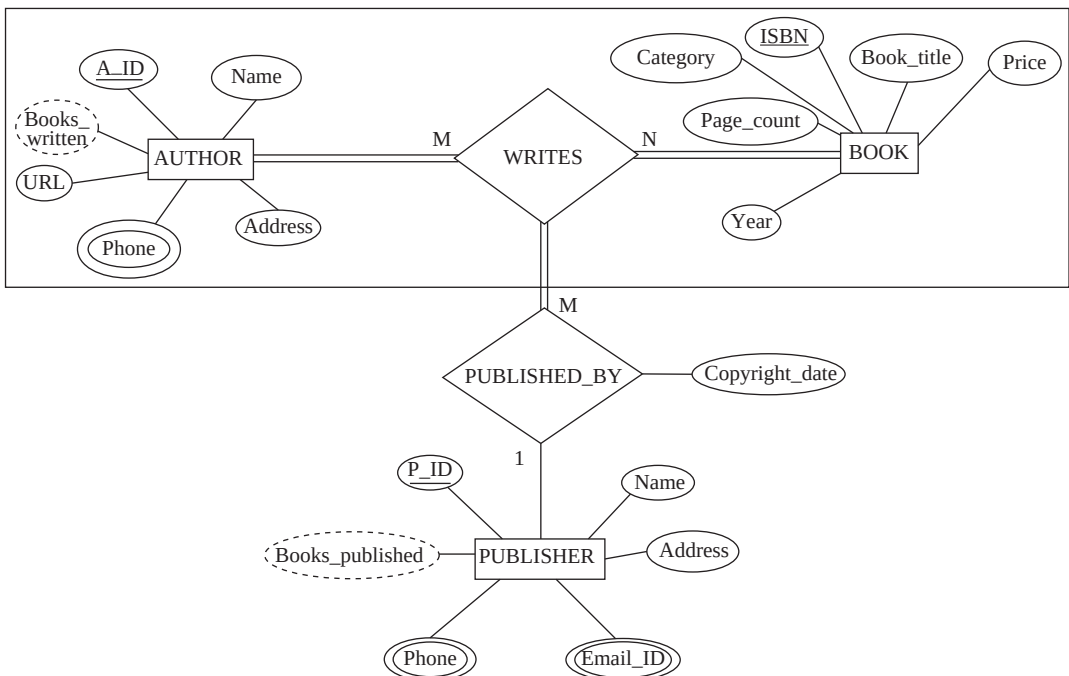
(a) EER diagram showing total participation



(b) EER diagram showing partial participation

Fig. 2.23 EER diagrams showing completeness constraints

WRITES. The relationship **WRITES** and the entity types **AUTHOR** and **BOOK** are aggregated into a single entity type to show the fact that once the author has written a book then only it gets published. The relationship type **PUBLISHED_BY** can be shown between the entity types **PUBLISHER** and **WRITES** as shown in Figure 2.24. The relationship type **PUBLISHED_BY** is a many-to-one relationship. It implies that a book written by a group of authors can be published by only one publisher; however, one publisher can print many books written by different authors.

**Fig. 2.24** EER diagram for Online Book database showing aggregation

Note that the EER diagram shown in Figure 2.24 cannot be expressed as ternary relationship between three entity types **AUTHOR**, **BOOK**, and **PUBLISHER**, as **WRITES** and **PUBLISHED_BY** are two different relationship types and hence, cannot be combined into one.

2.4 ALTERNATIVE NOTATIONS FOR E-R DIAGRAMS

An E-R diagram can also be represented using some other diagrammatic notations. One alternative E-R notation allows specifying the cardinality of the relationship, that is, the number of entities that can participate in a relationship. In this notation, a pair of integers (*min*, *max*) is associated with each participation of an entity type *E* in a relationship type *R*, where $0 \leq \text{min} \leq \text{max}$ and $\text{max} \geq 1$. This implies that each entity instance *e* of type *E* must participate in at least *min* and at most *max* relationship instances in *R*. If *min* is specified as zero, it means partial participation and if *min* > 0, it means total participation.

Figure 2.25 shows the modified E-R diagram of *Online Book* database with (*min*, *max*) notation. For example, the cardinality (1, N) of the **AUTHOR** implies that an author must write at least one book; however, an author can write any number of books. Similarly, the cardinality (1, 4) for the **BOOK** implies that a book must be written by at least one author and maximum by four authors.

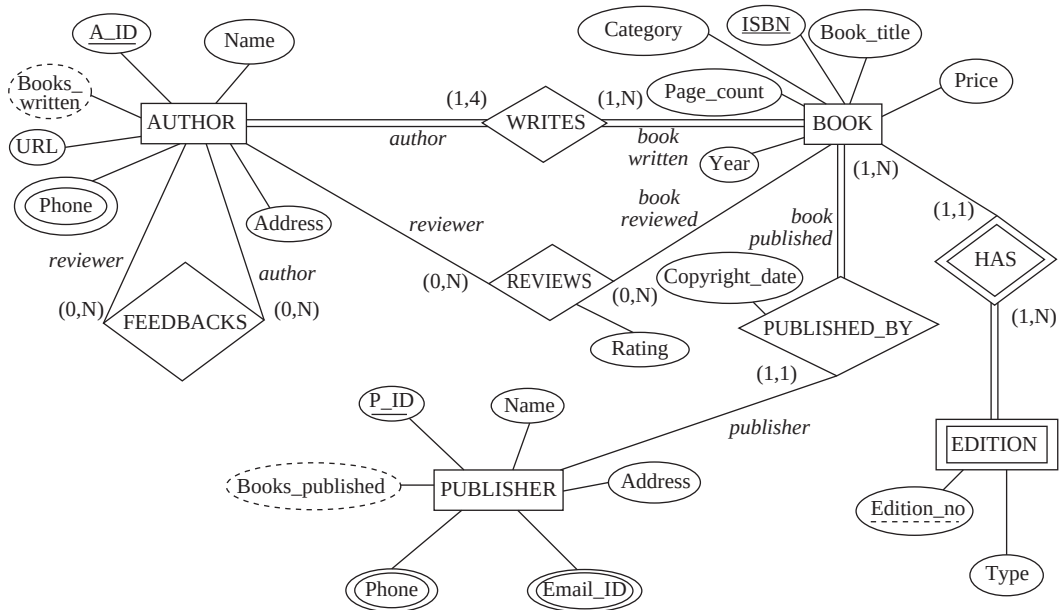


Fig. 2.25 E-R diagram for Online Book database with (*min*, *max*) notation

Another notation is **crow's feet** notation in which the symbol used to represent the many side of the relationship resembles the forward digits of a bird's claw. Figure 2.26 shows the modified E-R diagram of *Online Book* database with crow's feet notation. It actually consists of three symbols, namely, empty circle 'O', vertical bar '|' and crow's feet '<'. These symbols can be used in four combinations listed as follows:

- ⇒ < + O = optional many (either zero, one or more)
- ⇒ | + < = mandatory many (at least one or many)

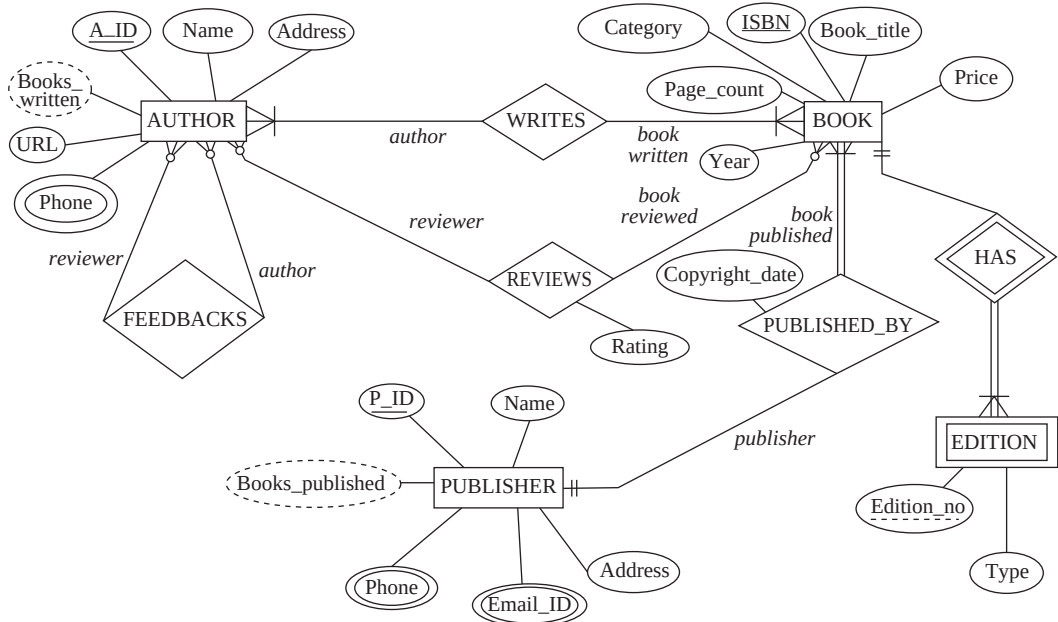


Fig. 2.26 E-R diagram Online Book database with crow's feet notation

⇒ | + | = mandatory one (one and only one)

⇒ | + O = optional one (either zero or one but not many)

2.5 UNIFIED MODELING LANGUAGE

E-R diagrams only represent various entities of an organization and relationships among them. The entities and their relationships form only one part of the overall software design. A software design includes some other components also such as functional modules of the system and interactions among them, user interactions with the system, etc. These components can be specified using a standard called the **Unified Modeling Language (UML)**, which was officially defined by the Object Management Group (OMG).

Unified Modeling Language (UML) is used as one of the techniques to implement object data modeling. The main advantage of object data modeling is that it models the elements of the system as real-world objects and hence, represents the real world very closely. Initially, it was designed for modeling object-oriented software; however, nowadays it is also used for business process modeling, systems engineering modeling, and representing organizational structures.

Learn More

Rational Rose is currently a popular tool used to draw UML diagrams. It helps the software developers in designing clear and easy-to-understand UML models.

2.5.1 UML Diagrams

UML includes a number of diagrams that are used to create an abstract model of a system. This abstract model is usually known as a **UML model**. UML includes ten standard diagrams, which are divided into two main categories, namely, *structural diagrams* and *behavioural diagrams*. Figure 2.27 shows the hierarchy of these UML diagrams.

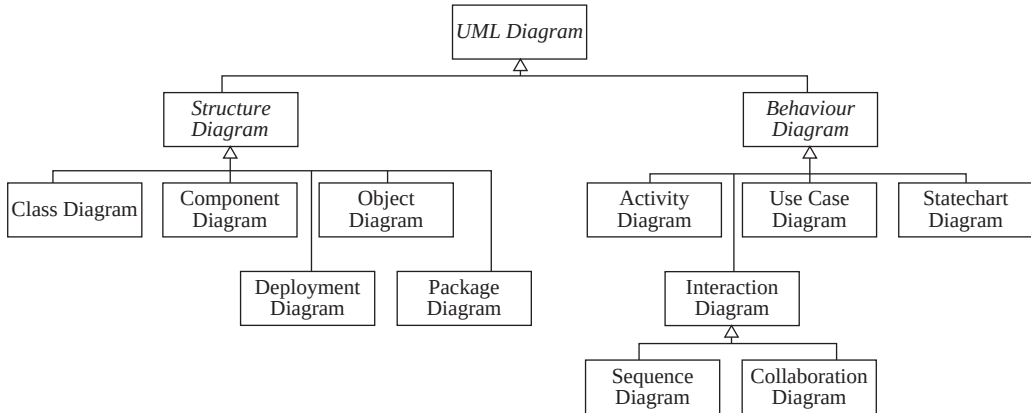


Fig. 2.27 Hierarchy of UML diagrams

Structural Diagrams

The **structural diagrams** describe the static or structural relationships among various components of the system. The structural diagrams are of five types.

- ⇒ **Class diagram:** A class diagram describes the static structure of the entire system by showing the classes of the system, their attributes, and the relationships between them. Like E-R diagrams, a UML class diagram is also used to represent the conceptual database schema. The entity types in an E-R diagram become the classes in the UML class diagram.
- ⇒ **Package diagram:** A package diagram can be treated as a subset of a class diagram. It organizes the elements of a system into related groups such that the dependencies between the various packages is minimized.
- ⇒ **Object diagram:** An object diagram describes a set of objects (instances of a class) and their relationships. It is used to represent the static view of a system at a particular point of time and is typically used to test the class diagrams for accuracy.
- ⇒ **Component diagram:** A component diagram displays the dependencies among various components of the system. The components include source code, run-time (binary) code, and executable code.
- ⇒ **Deployment diagram:** A deployment diagram displays the physical resources in the system such as nodes, components, and connections. The **nodes** are the run-time processing elements.

Behavioural Diagrams

The **behavioural diagrams** describe the dynamic or behavioural relationships among various components of the system. The behavioural diagrams are of four types.

- ⇒ **Use case diagram:** A use case diagram displays the relationship among actors and use cases. An **actor** represents a user or another system that interacts with the system and a **use case** is an external view of the system that represents some action.
- ⇒ **Statechart diagram:** A statechart diagram describes the dynamic behaviour of the system by displaying the sequences of states that an object goes through during its life in response to some external events. It shows the start/initial state of an object before the occurrence of an external event and its stop/final state in response to that external event.
- ⇒ **Activity diagram:** An activity diagram describes the dynamic behaviour of the system by modeling the flow of control from one activity to another. An **activity** is an operation performed on any class in the system that results in a change in the system state.

⇒ **Interaction diagrams:** An interaction diagram models the dynamic characteristics of a system by representing the set of messages exchanged among a set of objects. They are further divided into two categories.

- **Sequence diagram:** It describes the interactions among classes in terms of an exchange of messages over time. It consists of the vertical and horizontal dimensions, where vertical dimension represents the time and horizontal dimension represents different objects participating in the interactions.
- **Collaboration diagram:** It represents the interactions among various objects in terms of a series of sequenced messages. In a collaboration diagrams, the objects are shown as icons and the messages are numbered to show a particular sequence of messages.



In sequence diagrams the emphasis is on the time-ordering of messages; however, in collaboration diagrams the emphasis is on structural organization of objects that interacts with each other by sending messages.

2.5.2 Representing Conceptual Schema Using Class Diagrams

In a class diagram each entity type of an E-R diagram becomes the class and is displayed as a box containing three sections. The top section contains the **class name**, the middle section contains the **attributes** of the class and the last section contains the **operations** that can be performed on the objects of the class. A binary relationship type is represented by just drawing a line connecting the participating classes. The name of the relationship type is written adjacent to the line. The role names can also be specified by writing the role name on the line. If the relationship type has descriptive attributes, they are shown in a box containing the name of the relationship type. This box is connected to the relationship line by a dashed line. The (min, max) notation is represented as $min .. max$ notation. For example, the cardinality $(1, 1)$ is shown as $1 .. 1$, $(1, N)$ is shown as $1 .. *$, and $(0, N)$ is shown as $*$ or $0..*$. Here, the symbol $*$ indicates no maximum limit on participation. Figure 2.28 shows the UML class diagram for the *Online Book* database.

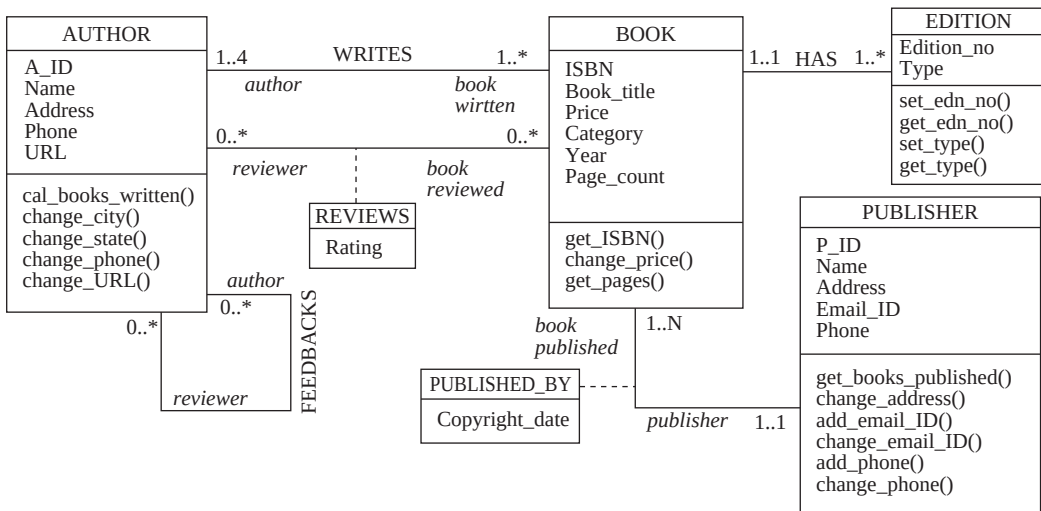
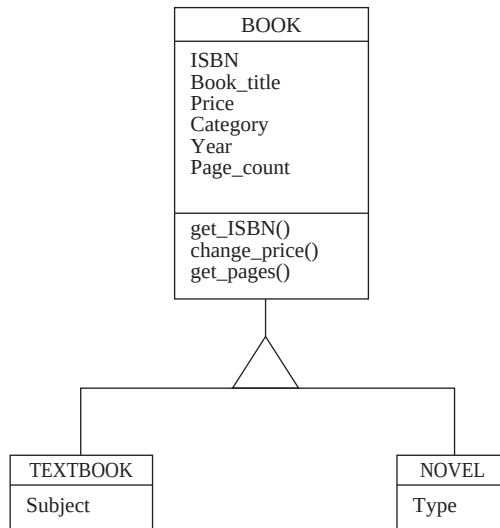


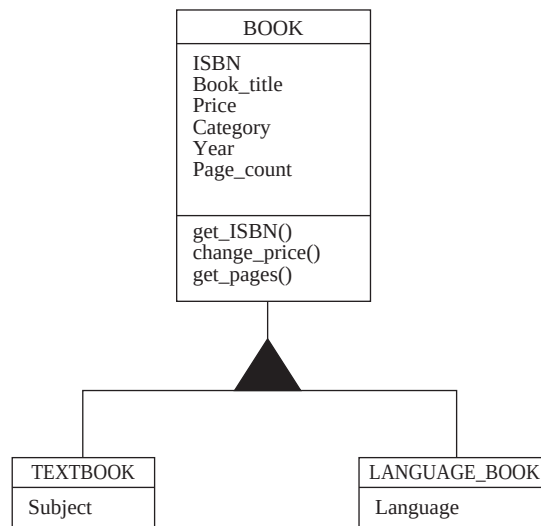
Fig. 2.28 UML class diagram for the Online Book database

Representing Specialization and Generalization

In UML class diagrams, specialization or generalization is represented by connecting the subclasses with the superclass by a blank or filled triangle. A blank triangle indicates specialization with disjoint constraint and a filled triangle indicates specialization with overlapping constraint. Figure 2.29 shows UML class diagrams representing specialization and generalization with disjoint and overlapping constraint. Note that both single and multiple inheritance can be shown using UML class diagram notation. The topmost superclass is known as **base class** and the bottommost subclasses are known as **leaf classes**.



(a) UML class diagram showing specialization with disjoint constraint



(b) UML class diagram showing specialization with overlapping constraint

Fig. 2.29 Specialization and generalization in UML class diagram

SUMMARY

1. The conceptual modeling (or semantic modeling) is an important phase in designing a successful database. It represents various pieces of data and their relationships at a very high-level of abstraction. It mainly focuses on what data is required and how it should be organized rather than what operations are to be performed on the data.
2. The conceptual model can be represented using two major approaches, namely, entity-relationship (E-R) modeling and the object modeling.
3. The E-R model views the real world as a set of basic objects (known as entities), and their characteristics (known as attributes) and associations among these objects (known as relationships).
4. An entity is a distinguishable object that has an independent existence in the real world. If an entity has a physical existence, it is termed as tangible or concrete entity and if it has a conceptual existence, it is termed as non-tangible or abstract entity.
5. Each entity is represented by a set of attributes. The attributes are the properties of an entity that characterize and describe it.
6. Each attribute can accept a value from a set of permitted values, which is called the domain or the value set of the attribute.
7. A set or a collection of entities that share the same attributes but different values is known as an entity type.
8. A specific occurrence of an entity type is called its instance. The collection of all instances of a particular entity type in the database at any point of time is known as an entity set.
9. The attributes of an entity are classified into five categories, namely, identifying and descriptive attributes, simple and composite attributes, stored, and derived attributes, single and multivalued attributes, and complex attributes.
10. Sometimes, there may be a situation where a particular entity may not have an appropriate value for an attribute. In such situations, the attribute takes a special value called *null* value.
11. The attribute (or combination of attributes), whose values are distinct for each individual instance of an entity type is known as a key attribute.
12. A set of one or more attributes, taken together, that helps in uniquely identifying each entity is called a superkey.
13. A minimal superkey that does not contain any superfluous (extra) attribute in it is called a candidate key.
14. The candidate key, which is chosen by the database designer to uniquely identify entities, is known as the primary key.
15. If a primary key is formed by the combination of two or more attributes it is known as a composite key.
16. An entity type that does not have any key attributes of its own is called a weak entity type. On the other hand, an entity type that has a key attribute is called a strong entity type.
17. A weak entity type has a partial identifying attribute known as partial key (or discriminator), which is a set of attributes that can uniquely identify the instances of a weak entity type related to the same instance of a strong entity type.
18. An association between two or more entities is known as a relationship. The relationship between a weak entity type and its identifying entity type is known as an identifying relationship of the weak entity type.
19. The number of entity types participating in a relationship type determines the degree of the relationship type.
20. Each entity type that participates in a relationship type plays a specific function or a role in the relationship. The role of each participating entity type is represented by the role name.

21. Each relationship type has certain constraints that restrict the possible combination of instances participating in the relationship set. The constraints on the relationship type are of two types, namely, mapping cardinalities and participation constraint.
22. An E-R diagram is a specialized graphical tool that is used to represent the overall logical structure of the database.
23. An E-R diagram consists of a set of symbols that are used to represent different types of information. The symbols include rectangles, ellipses, diamonds, lines, double lines, double ellipses, dashed ellipses, and double rectangles.
24. An enhanced E-R diagram is used to represent some other aspects of a database such as inheritance among various entity types.
25. The process of defining the subgroups of a given entity type is called specialization. The entity type containing the common attributes is known as the superclass and the entity type, which is a subset of the superclass, is known as its subclass.
26. The design process in which multiple lower-level entity types are combined on the basis of common features to form higher-level entity types is known as generalization.
27. If a subclass is a subset of only one superclass, that is, it has only one parent, it is known as single inheritance and the resulting structure is known as a specialization (or generalization) hierarchy. On the other hand, if a subclass is derived from more than one superclass, it is known as multiple inheritance and the resulting structure is known as a specialization (or generalization) lattice.
28. The process through which one can treat the relationships as higher-level entities is known as aggregation.
29. Unified modeling language (UML) is used to specify some other components of a software design process including functional modules of the system and interactions among them, user interactions with the system, etc.
30. UML includes ten standard diagrams that are divided into two main categories, namely, structural diagrams and behavioural diagrams.

◀ ● KEY TERMS ● ▶

- | | |
|--|--|
| <ul style="list-style-type: none"> ● Conceptual modeling (Semantic modeling) ● Entity-relationship (E-R) model ● Entity ● Attributes ● Domain ● Entity type ● Entity set ● Identifying and descriptive attributes ● Simple and composite attributes ● Stored and derived attributes ● Single-valued and multivalued attributes ● Complex attributes ● NULL value ● Key attribute ● Superkey | <ul style="list-style-type: none"> ● Candidate key ● Primary key ● Weak entity type ● Strong entity type ● Partial key ● Relationship ● Relationship type ● Relationships et ● Identifying relationship ● Relationship degree ● Role name ● Recursive relationship ● Mapping cardinalities ● One-to-one relationship ● One-to-many relationship |
|--|--|

- Many-to-one relationship
- Many-to-many relationship
- Participation constraint
- Total (or mandatory) participation
- Partial (or optional) participation
- Entity-relationship diagram
- Enhanced E-R model
- Specialization
- Generalization
- Superclass
- Subclass
- Attribute inheritance
- Single and multiple inheritance
- Condition-defined specialization
- User-defined specialization
- Disjoint constraint
- Overlapping constraint
- Completeness constraint
- Total specialization
- Partial specialization
- Aggregation
- Unified modeling language (UML)
- Structural diagrams
- Behavioural diagrams

◀ ● EXERCISES ● ▶

A. Multiple Choice Questions

1. The E-R model is relevant to which of the following phases of database design?
 - (a) Requirement analysis
 - (b) Conceptual database design
 - (c) Logical database design
 - (d) None of these
2. Which of these statements describes an entity correctly?
 - (a) An entity is a distinguishable object that has an independent existence in the real world
 - (b) Each entity is represented by a set of attributes
 - (c) An entity can exist either physically or conceptually
 - (d) All of these
3. The attribute that is used to uniquely identify an instance of an entity is known as _____.
 - (a) ~~unique~~ attribute
 - (b) ~~simple~~ attribute
 - (c) Identifying attribute
 - (d) Multivalued attribute
4. Which of these attributes can be considered as identifying attribute for an entity Student?
 - (a) ~~Roll~~ number
 - (b) ~~marks~~
 - (c) Address
 - (d) Any of these
5. The strong entity type and weak entity type should participate in _____.
 - (a) Many-to-many relationship
 - (b) Many-to-one relationship
 - (c) One-to-many relationship
 - (d) One-to-one relationship
6. In an E-R diagram an entity is represented by a _____.
 - (a) ~~Square~~
 - (b) ~~rectangle~~
 - (c) Diamond-shaped box
 - (d) Ellipse

7. Which of these statements is correct?
 - (a) An entity may be related to only one other entity
 - (b) An entity may be related to itself
 - (c) An entity may be related to many other entities
 - (d) All of these
8. The relationship between a weak entity type and its identifying entity type is known as _____.
 - (a) Identifying relationship
 - (b) Weak relationship
 - (c) Strong relationship
 - (d) Relationship type
9. The relationship between two entity types is known as _____.
 - (a) Binary relationship
 - (b) Two-way relationship
 - (c) Multiple relationship
 - (d) Double relationship
10. A person has a PAN card. This relationship is:
 - (a) ~~1~~0-to-one
 - (b) ~~1~~0-to-many
 - (c) ~~1~~0-to-many
 - (d) ~~1~~0-to-one

B. Fill in the Blanks

1. If an entity has a physical existence, it is termed as _____ entity.
2. A set of entities that share the same attributes but different values, is known as an _____.
3. The attribute (or combination of attributes) whose values are distinct for each individual instance of an entity type is known as a _____.
4. The constraints on the relationship type are of two types, namely, _____ and _____.
5. An _____ is a specialized graphical tool that demonstrates the interrelationships among various entities of a database.
6. The process of defining the subgroups of a given entity type is called _____.
7. In _____, the membership of entities is determined by placing a condition on the value of some attribute of the superclass.
8. _____ constraint specifies that the same higher-level entity instance cannot belong to more than one lower-level entity types.
9. The process through which one can treat the relationships as higher-level entities is known as _____.
10. _____ is a standard language, which is officially defined by the Object Management Group (OMG) as one of the techniques to implement object data modeling.

C. Answer the Questions

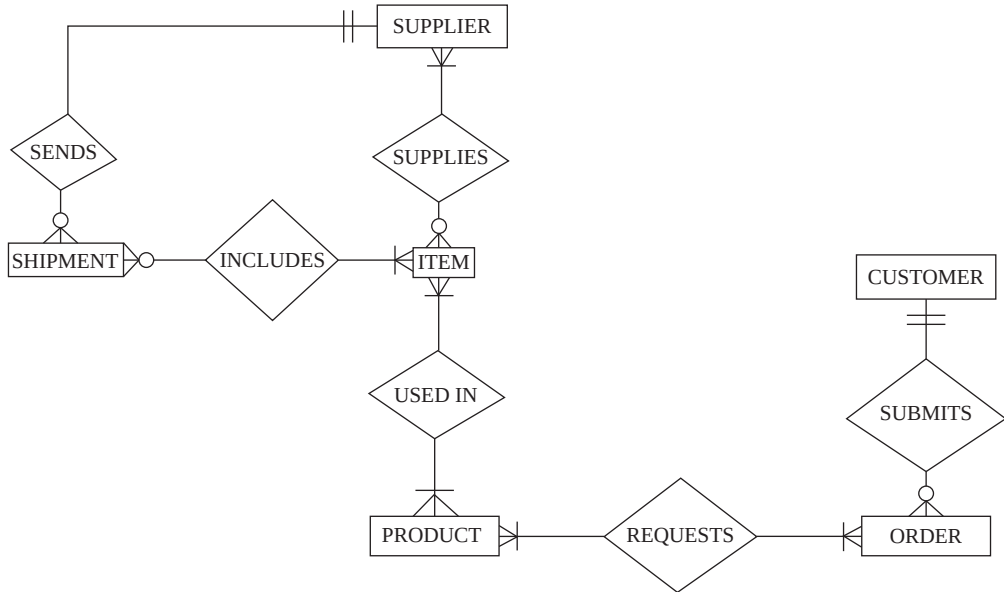
1. Write a short note on conceptual data modeling.
2. What is an E-R model? List some of its advantages and limitations. How do you overcome these limitations?

3. What is an entity? Differentiate between tangible and non-tangible entity.
4. Differentiate between an attribute and a domain.
5. Define the following terms:

(a) Entity type	(b) Entity sets	(c) Instance
(d) NULL value	(e) Attributes	(f) ex attribute
(g) Composite key	(h) domain	(i) Single-valued and multi-valued attributes
(j) erived attributes	(k) ML	(l) Attribute inheritance
(m) Cardinality ratio	(n) Degree of relationship	(o) Weak entity
6. What are the various types of attributes? Discuss the various situations in which an attribute can use a *null* value.
7. Explain the difference between primary key, candidate key, and superkey.
8. Differentiate between strong and weak entity type.
9. What are relationship type and relationship instance? Explain the difference between the two.
10. What is identifying relationship?
11. What is the significance of using role names in the description of relationship types? In what situations are role names necessary?
12. Explain the term 'degree of a relationship' with the help of an example.
13. Discuss the various structural constraints that are applied on relationship types.
14. What is meant by recursive relationship type? Give an example also.
15. Discuss various symbols that are used in the E-R diagrams to represent various types of information.
16. Discuss the E-R diagram naming conventions.
17. Illustrate the issues to be considered while developing an E-R diagram.
18. How are weak entity types represented in an E-R diagram?
19. What is an EER model?
20. Differentiate between specialization and generalization with the help of an example. Can we represent their differences with the help of an E-R diagram? Explain.
21. Differentiate between disjoint and overlapping constraint.
22. List the possible types of relations that may exist between two entities.
23. What is the difference between specialization hierarchy and specialization lattice?
24. What is an aggregation? Explain with the help of an example.
25. What are the various alternative notations to represent E-R diagrams?
26. Define UML. What is UML model? Explain its use and discuss the different categories of UML diagrams.

D. Practical Questions

1. Identify the entity types, relationship types, and mapping cardinalities in the following E-R diagram.



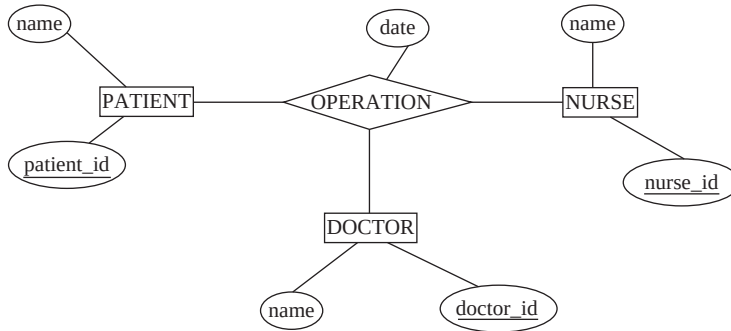
2. Consider the following set of requirements for a **COMPANY** database that is used to keep track of its employees, departments, and projects.

- The company stores the information about the currently working employees. The information includes employee number, name, gender, salary, date of birth, date of joining, address, and phone number. Each employee works for a department on a particular project for a particular number of hours.
- The information about departments includes department number and department name. Each department controls some projects currently running in the company. Also each department is managed by a particular employee who becomes the manager for that department. This employee also supervises all the other employees in that department.
- The project information includes project number, project name and its description.
- An employee can work for only one department; however, a department can have any number of employees. A department is managed by only one manager and a manager can manage only one department. A department can control any number of projects; however, one project can be handled by only one department. Any number of employees can work on any number of projects.

Design an E-R diagram for **COMPANY** database. Specify the key attributes of each entity type, role names, and mapping cardinalities. Make appropriate assumptions to complete the specification.

- Draw the EER diagram for the **COMPANY** database described in question 2 by showing the specialization of employee entity type. The subclasses include *secretary*, *technician*, *engineer*, *manager*, *fulltime employee*, and *part-time employee*. Show all the possible constraints. Make appropriate assumptions, wherever necessary.
- Give an example of multiple relationships between entity types. Draw the E-R diagram for illustration.

5. Consider the following E-R diagram describing surgeries or operations in a HOSPITAL database.



- (a) Represent the relation type **OPERATION** as a weak entity type with some additional constraints that an operation is performed on exactly one patient by one or more doctors and one or more nurses.
 - (b) A doctor can be a consultant or a registrar, but not both and there are some doctors who are neither consultants nor registrars. Represent this information with the help of an extended E-R model.
6. Construct an E-R diagram for a **BANK** database having customer, loan, account, employee, and branch as entity types. A customer has an account in a particular branch of the bank. The customer can also borrow loan from the bank. The bank has number of employees working in different branches of the bank. Add appropriate attributes for each entity type. Represent the key attributes, weak entity types (if any), cardinality ratios, and role names of each entity type. Make appropriate assumptions to complete the specification.
7. Draw the UML diagrams equivalent to the E-R diagrams constructed for Questions 1, 2, and 5.

THE RELATIONAL MODEL

After reading this chapter, the reader will understand:

- The relational model and its three components, namely, structural component, manipulative component and integrity constraints.
- The relation schema and relation instance
- Attribute domain
- Unstructured and structured domain
- Degree and cardinality of a relation
- Characteristics of relations
- The relational database schema and relational database instance
- Keys including superkey, candidate key, primary key, alternate key and foreign key
- Various types of data integrity including domain integrity, entity integrity, referential integrity and semantic integrity
- Different types of integrity constraints
- Legal and illegal relation
- Constraint violation while insert, delete, and update operation
- Mapping of E-R model to relational model
- Representation of entity types to relation
- Representation of weak entity types, relationship types, composite and multivalued attributes, and Extended E-R features

A database is a collection of interrelated data and the way data is related to each other depends upon the model being used. Generally databases are based on one of the three models: the *hierarchical* model, the *network* model or the *relational* model. A database based on the network or hierarchical model is a non-relational database and a database based on the relational model is a relational database. A **relational database** is a collection of relations (or two-dimensional tables) having distinct names. It is a persistent storage mechanism that conforms to the relational model.

At the time when the relational model was proposed, hierarchical and network model were used to structure the data. Both these models are complex since they use pointers to relate the records. However, the relational model is simple as it uses the values of the records instead of pointers to relate the records. For example, in *Online Book* database, the common value from the publisher and book records such as publisher id is used to relate the publisher record to the book record, instead of using an internal pointer. In addition, it gives the freedom to change the way of storing and accessing the data without corresponding change in the way the user perceives the data.

The relational model is based on the branches of mathematics, namely, *set theory* and *predicate logic*. The key assumption is that the data is represented in the form of mathematical n -ary relations, which is a subset of the Cartesian product of n sets. Reasoning about such a data is done in two-valued predicate logic in mathematical model. In two-valued predicate logic, two possible evaluations for each proposition are possible, that is, either true or false. In addition to true or false, third value such as *null* is included in three-value predicate. Some consider only two-valued logic as crucial part of the relational model, whereas others include three-valued logic as crucial part of the relational model.

The relational model includes three components, namely, *structural component* (or data structures), *manipulative component* (or data manipulation), and *integrity constraints*. **Structural component** consists of two components, namely, entity types and their relationships, which act as a framework of the given situation. **Manipulative component** consists of a set of operations that are performed on a table or group of tables. **Integrity constraints** are a set of rules that governs the entity types and their relationships, and thereby, ensures the integrity of database.

3.1 RELATIONAL MODEL CONCEPTS

The relational model represents both data and the relationships among those data using *relation*. A **relation** is used to represent information about any entity (such as book, publisher, author, etc.) and its relationship with other entities in the form of attributes (or columns) and tuples (or rows). A relation comprises of a *relation schema* and a *relation instance*. A relation schema (also termed as relation intension) depicts the attributes of the table, and a relation instance (also termed as relation extension) is a two-dimensional table with a time-varying set of tuples.

Relation Schema

A **relation schema** consists of a relation name R and a set of attributes (or fields) $A_1, A_2, \dots, A_n$. It is represented by $R(A_1, A_2, \dots, A_n)$ and is used to describe a relation R . To understand the concept of relation schema, consider the book, publisher, and author information stored in an *Online Book* database with the schema of the relations given as

```
BOOK(ISBN, Book_title, Category,
Price,
Copyright_date, Year, Page_
count, P_ID)
```

```
AUTHOR(A_ID, Aname, City, State, Zip, Phone, URL)
```

```
PUBLISHER(P_ID, Pname, Address, State, Phone, Email_id)
```

In these statements, ISBN, Book_title, Category, Price, Copyright_date, Year, Page_count, and P_ID are the attributes of relation BOOK. A_ID, Aname, City, State, Zip, Phone, and URL are the attributes of relation AUTHOR. P_ID, Pname, Address, State, Phone, and Email_id are the attributes of relation PUBLISHER.

An attribute can be single-valued as well as multivalued attribute. Single-valued attributes represent single value of data, whereas multivalued attributes represent more than one value of data. Examples of single-valued attributes of relation PUBLISHER are P_ID, Pname, Address, and State, whereas Phone and Email_id are multivalued attributes of relation PUBLISHER.

An attribute consists of an attribute name and a value of one or more bytes from the set of permissible values of same data type. This set of permissible value of same data type for an attribute is termed as the **domain D** of that attribute. The relation schema of book information can be written using the data type of each attribute as

```
BOOK(ISBN: string, Book_title: string, Category: string, Price:
integer, Copyright_date: integer, Year: integer, Page_count:
integer, P_ID: string)
```

Learn More

Codd's model emerged as the dominant model that provides the basis for effectively using both the relational database software, such as Oracle, MS SQL Server, etc., and the personal database systems such as Access, FoxPro, etc. The main basis in developing relational model is that a database is a collection of unordered tables that can be modified through various operations that return a table or group of tables as their result.

In this statement, the permissible value for the attribute **ISBN** consists of a set of character values. Thus, the domain for the attribute **ISBN** is **string**. Note that the data type or format for each domain is the **physical** definition of domains and the value of the domain is the **logical** definition of domains. Here, the data type **string** is the physical definition of domain and a set of character values is the logical definition of domain.

Several attributes may have same domain at the physical level but distinct domains at the logical level. For example, both the attributes **Aname** and **Pname** of relations **AUTHOR** and **PUBLISHER**, respectively, have same data type, that is, **string**. Hence, both the attributes have same domain at the physical level. However, both the attributes **Aname** and **Pname** have different values, say, *Thomas William* and *Hills Publications*, respectively. Hence, both the attributes have distinct domains at the logical level.

Several attributes may have either same or distinct domains at both physical and logical level. For example, the attributes **State** of relations **AUTHOR** and **PUBLISHER** have same domain both at the physical level, that is **string**, and at the logical level, say, *New York*. On the other hand, the attributes **URL** and **Phone** of relations **AUTHOR** and **PUBLISHER**, respectively, ought to have distinct domains both at the physical level that is, **string** for **URL** and **integer** for **Phone**, and also at the logical level.

Note that domain can be unstructured or structured. **Unstructured (or atomic) domain** consists of the non-decomposable or indivisible values that cannot be further divided into sub-values. Atomic domains are also known as *application-independent domains* since it consists of the general sets that are independent of a particular application. Sets of strings, integers, real numbers, etc. are some of the examples of unstructured domain. On the other hand, **structured (or composite) domain** consists of non-atomic values that are decomposable or divisible values. The domain for the attribute **Address** of relation **PUBLISHER** consisting of house number, street name, and city is an example of structured domain. Structured domains are also known as *application-dependent domains* since these are the subset of standard data types and can be defined by specifying the constraints on the values that can be inserted into the application.

RelationInstance

A **relation instance** (or relation) r of the relation schema $R(A_1, A_2, \dots, A_n)$ is a set of n -tuples t . It is also denoted by $r(R)$. A relation instance is an ordered set of attributes values v that belongs to the domain D and it can be denoted as $r = \{t_1, t_2, \dots, t_m\}$ where, $t = \{v_1, \dots, v_n\}$. In other words, it is a set of tuples (or records) having the same number of attributes as the relation schema. It can be specified as $t \in r$, which denotes that tuple t is in relation r . A relation instance can be considered as a *table*, which is a collection of tuples having the same number of attributes.

Note that a relation with n attributes is a subset of a Cartesian product of n domains. In other words, a relation of n attributes is a subset of $D_1 \times D_2 \times D_3 \times \dots \times D_n$. Consider the **BOOK** relation in which domains $D_1, D_2, D_3, \dots, D_8$ denote the set of all **ISBN**, **Book_title**, **Price**, ..., **P_ID**, respectively. Now any tuple of **BOOK** relation must comprise of eight attributes $(A_1, A_2, \dots, A_8)$, where the values of A_1 (**ISBN**) belong to domain D_1 and similarly, the values of remaining attributes $A_2, A_3, \dots, A_8$ belong to domains $D_2, D_3, \dots, D_8$, respectively. Since a **BOOK** relation is a subset of all possible attributes values in a list of domains, it can be written as a subset of $D_1 \times D_2 \times D_3 \times \dots \times D_8$.

Consider the **BOOK** relation of Figure 3.1, the instance **B1** contains nine tuples and eight attributes. If the tuple t denotes the first tuple of the relation, the notation $t[\text{ISBN}]$ refers to the value of t on the **ISBN** attribute. Hence, $t[\text{ISBN}] = "001-354-921-1"$, $t[\text{Book_title}] = "Ransack"$ and so on. Mathematically, it can also be written as $t[1] = "001-354-921-1"$, $t[2] = "Ransack"$ and so on. Here, 1 refers to the first attribute of a **BOOK** relation, 2 refers to the second attribute, and so on.

Attributes (or Fields or Columns)							
ISBN	Book_title	Category	Price	Copyright_date	Year	Page_count	P_ID
001-354-921-1	Ransack	Novel	22	2005	2006	200	P001
001-987-650-5	Differential Calculus	Textbook	30	2003	2003	450	P001
001-987-760-9	C++	Textbook	40	2004	2005	800	P001
002-678-880-2	Call Away	Novel	22	2001	2002	200	P002
002-678-980-4	DBMS	Textbook	40	2004	2006	800	P002
003-456-433-6	Introduction to German Language	Language Book	22	2003	2004	200	P004
003-456-533-8	Learning French Language	Language Book	32	2005	2006	500	P004
004-765-359-3	Coordinate Geometry	Textbook	35	2006	2006	650	P003
004-765-409-5	UNIX	Textbook	26	2006	2007	550	P003

Fig.3.1 Instance B1 of the BOOK relation

The number of attributes in a relation is known as **degree** or **arity**, and the number of tuples in a relation is known as **cardinality**. A relation of degree one is known as **unary relation**, of degree two is known as **binary relation**, of degree three is known as **ternary relation**, and of degree n is known as **n -ary relation**. For example, the degree of the relation in Figure 3.1 is eight and its cardinality is nine.

3.1.1 CharacteristicsofRelations

A relation has certain characteristics, which are given here.

⇒ **Ordering of Tuples in a Relation:** Since a relation is a set of tuples and a set has no particular order among its elements, tuples in a relation do not have any specified order. However, tuples in a relation can be logically ordered by the values of various attributes. In that case, information in a relation remains the same, only the order of tuple varies. Hence, tuple ordering in a relation is irrelevant. For example, if the **BOOK** relation in Figure 3.1 is logically ordered by the values of attribute **Category**,

LearnMore

A relation of zero degree (or with no attributes) is known as nullary relation and it is of the type **RELATION{}**. There are two relations of zero degree, namely, **TABLE_DEE** (or **DEE**) and **TABLE_DUM** (or **DUM**). **DEE** contains only one tuple, namely, 0-tuple and is of the type **RELATION{TUPLE {}}**. On the other hand, **DUM** is empty and contains no tuples. **DEE** and **DUM** are mainly used in the relational algebra and they represent **TRUE** (or **YES**) and **FALSE** (or **NO**), respectively.

the first tuple of **BOOK** relation in Figure 3.1 becomes the third tuple of **BOOK** relation in Figure 3.2. Similarly, second tuple becomes the fifth tuple in Figure 3.2. Thus, every tuple of **BOOK** relation in Figure 3.1 is the tuple of **BOOK** relation in Figure 3.2 and vice versa. Since both the relations contain the same set of tuples, they are the same.

ISBN	Book_title	Category	Price	Copyright_date	Year	Page_count	P_ID
003-456-433-6	Introduction to German Language	Language Book	22	2003	2004	200	P004
003-456-533-8	Learning French Language	Language Book	32	2005	2006	500	P004
001-354-921-1	Ransack	Novel	22	2005	2006	200	P001
002-678-880-2	Call Away	Novel	22	2001	2002	200	P002
001-987-650-5	Differential Calculus	Textbook	30	2003	2003	450	P001
001-987-760-9	C++	Textbook	40	2004	2005	800	P001
002-678-980-4	DBMS	Textbook	40	2004	2006	800	P002
004-765-359-3	Coordinate Geometry	Textbook	35	2006	2006	650	P003
004-765-409-5	UNIX	Textbook	26	2006	2007	550	P003

Fig.3.2 The **BOOK** relation (sort by Category)

⇒ **Ordering of values within a Tuple:** As discussed earlier, n -tuple is an ordered set of attributes values that belongs to the domain D , so, the order in which the values appear in the tuples is significant. However, if a tuple is defined as a set of (**<attribute>: <value>**) pairs, the order in which attributes appear in a tuple is irrelevant. This is due to the reason that there is no preference for one attribute value over another. For example, consider these statements.

$t = \langle \text{ISBN: } 001-987-760-9, \text{ Book_title: } C++, \text{ Category: } \textit{Textbook}, \text{ Price: } 40, \text{ Copyright_date: } 2003, \text{ Year: } 2005, \text{ Page_count: } 800, \text{ P_ID: } P001 \rangle$

$t = \langle \text{Category: } \textit{Textbook}, \text{ ISBN: } 001-987-760-9, \text{ Price: } 40, \text{ P_ID: } P001, \text{ Book_title: } C++, \text{ Page_count: } 800, \text{ Copyright_date: } 2003, \text{ Year: } 2005 \rangle$

In these statements, since the attributes and values are represented as pairs, these two tuples are the same.

⇒ **Values and Nulls in the Tuples:** Relational model is based on the assumption that the tuples in a relation contain only atomic values. In other words, each tuple in a relation contains a single value for each of its attribute. Hence, a relation does not allow composite and multivalued

attributes. Moreover, it allows denoting the value of the attribute as *null*, if the value does not exist for that attribute or the value is unknown. For example, if a publisher does not have an email address, the *null* value can be specified, which signifies that the value does not exist (see Figure 3.3).

P_ID	Pname	Address	State	Phone	Email_id
P001	Hills Publications	12, Park street, Atlanta	Georgia	7134019	h_pub@hills.com
P002	Sunshine Publishers Ltd.	45, Second street, Newark	New Jersey	6548909	null
P003	Bright Publications	123, Main street, Honolulu	Hawaii	7678985	bright@bp.com
P004	Paramount Publishing House	789, Oak street, New York	New York	9254834	param_house@param.com
P005	Wesley Publications	456, First street, Las Vegas	Nevada	5683452	null

Fig.3.3 The PUBLISHER relation



NOTE A relation that assigns a single value to each attribute is said to be in first normal form, which will be discussed in Chapter 06.

- ⇒ **No two tuples are identical in a relation:** Since a relation is a set of tuples and a set does not have identical elements. Therefore, each tuple in a relation must be uniquely identified by its contents. In other words, two tuples with the same value for all the attributes (that is, duplicate tuples) cannot exist in a relation.
- ⇒ **Interpretation of a Relation:** A relation can be used to interpret facts about entities as a type of assertion. For example, the PUBLISHER relation in Figure 3.3 asserts that an entity PUBLISHER has P_ID, Pname, Address, State, Phone, and Email_id. Here, the first tuple asserts the fact that there is a publisher whose publisher id is P001, name is Hills Publications, and so on.

A relation can also be used to interpret facts about relationships. For example, consider the BOOK relation in Figure 3.1, the attribute P_ID relates the BOOK relation with the PUBLISHER relation.

LearnMore

E.F. Codd designed a set of 13 rules (numbered 0 to 12), known as Codd's 12 rules, to define relational database system. These rules are so strict that almost all the popular RDBMSs fail to meet the criteria.

3.2 RELATIONAL DATABASE SCHEMA

A set of relation schemas $\{R_1, R_2, \dots, R_m\}$ together with a set of integrity constraints in the database constitutes **relational database schema**. A relational database schema, say S , is represented as $S = \{R_1, R_2, \dots, R_m\}$.

For example, the relational database schema *Online Book* is a set of relation schemas, namely, BOOK, PUBLISHER, AUTHOR, AUTHOR_BOOK, and REVIEW, which is shown in Figure 3.4.

BOOK

ISBN	Book_title	Category	Price	Copyright_date	Year	Page_count	P_ID
------	------------	----------	-------	----------------	------	------------	------

PUBLISHER

P_ID	Pname	Address	State	Phone	Email_id
------	-------	---------	-------	-------	----------

AUTHOR

A_ID	Aname	City	State	Zip	Phone	URL
------	-------	------	-------	-----	-------	-----

AUTHOR_BOOK

A_ID	ISBN
------	------

REVIEW

R_ID	ISBN	Rating
------	------	--------

Fig.3.4 Relational database schema

Hence, the relational database schema *Online Book* can be represented as

Online Book={BOOK, PUBLISHER, AUTHOR, AUTHOR_BOOK, REVIEW}



NOTE To define a relational database schema, a non-procedural language, SQL, is required to communicate with relational database. SQL will be discussed in Chapter 05.

3.3 RELATIONAL DATABASE INSTANCE

A **relational database instance** S of relational database schema $S = \{R_1, R_2, \dots, R_m\}$ is a set of relation instances $\{r_1, r_2, \dots, r_m\}$ such that each relation instance r_i is a state of corresponding relation schema R_i . In addition, each relation instance must satisfy the integrity constraints specified in the relational database schema. For example, a relational database instance corresponding to the *Online Book* schema is shown in Figure 3.5.

Note that when relational model was developed, some of its versions were developed on the assumption that those attributes which represent same real-world concept would be given same attribute names in all relations. This assumption had a limitation that it would not be possible to represent attributes having same real-world concept with different meanings in the same relation. Nowadays, different names can be given to the attributes representing the same real-world concept with different meanings in the same relation. To elaborate this concept, consider the modified form of *Online Book* database in which the relations, namely, **AUTHOR_BOOK** and **REVIEW** are merged together in a single relation say, **AUTHOR_REVIEWER**. The resultant relation is shown in Figure 3.6.

In the **AUTHOR_REVIEWER** relation, the concept of author id appears twice, once as an author and another as a reviewer who is also an author. The different attribute names, that is, **A_ID** and **R_ID** are given to them to represent different meaning in the same relation.

Attributes representing the same real-world concept may have same or different names in different relations. For example, the attribute **P_ID** in both **PUBLISHER** and **BOOK** relations represents the same concept with same attribute name in different relations. However, the attribute **A_ID** in the **AUTHOR** relation, and the attribute **R_ID** in the **REVIEW** relation represent the

BOOK

ISBN	Book_title	Category	Price	Copyright_date	Year	Page_count	P_ID
001-354-921-1	Ransack	Novel	22	2005	2006	200	P001
001-987-650-5	Differential Calculus	Textbook	30	2003	2003	450	P001
001-987-760-9	C++	Textbook	40	2004	2005	800	P001
002-678-880-2	Call Away	Novel	22	2001	2002	200	P002
002-678-980-4	DBMS	Textbook	40	2004	2006	800	P002
003-456-433-6	Introduction to German Language	Language Book	22	2003	2004	200	P004
003-456-533-8	Learning French Language	Language Book	32	2005	2006	500	P004
004-765-359-3	Coordinate Geometry	Textbook	35	2006	2006	650	P003
004-765-409-5	UNIX	Textbook	26	2006	2007	550	P003

PUBLISHER

P_ID	Pname	Address	State	Phone	Email_id
P001	Hills Publications	12, Park street, Atlanta	Georgia	7134019	h_pub@hills.com
P002	Sunshine Publishers Ltd.	45, Second street, Newark	New Jersey	6548909	null
P003	Bright Publications	123, Main street, Honolulu	Hawaii	7678985	bright@bp.com
P004	Paramount Publishing House	789, Oak street, New York	New York	9254834	param_house@param.com
P005	Wesley Publications	456, First street, Las Vegas	Nevada	5683452	null

AUTHOR

A_ID	Aname	State	City	Zip	Phone	URL
A001	Thomas William	New York	New York	14998	923673	www.thomas.com
A002	James Erin	Georgia	Atlanta	31896	376045	www.ejames.com
A003	Charles Smith	California	Los Angeles	95031	419562	www.csmith.com
A004	Lewis Ian	Washington	Seattle	98012	578932	www.au_lewis.com
A005	Allen Ford	Alaska	Juneau	99502	581231	www.allenford.com
A006	Jones Martin	New York	Albany	14521	218161	www.martin.com
A007	John Stephen	Texas	Austin	75112	316292	www.john_stepen.com
A008	Annie George	Michigan	Detroit	48011	321313	www.annie.com
A009	Suzanne Hedley	Washington	Seattle	98001	785236	www.suzz.com
A010	Richard Flaming	Virginia	Virginia Beach	22111	456163	www.richard.com

REVIEW

AUTHOR_BOOK

A_ID	ISBN
A001	001-987-760-9
A002	001-678-980-4
A003	001-987-760-9
A004	003-456-433-6
A005	001-354-921-1
A006	002-678-880-2
A007	001-987-650-5
A008	002-678-980-4
A008	004-765-409-5
A009	004-765-359-3
A010	003-456-533-8

R_ID	ISBN	Rating
A001	002-678-980-4	2
A002	001-987-760-9	6
A003	002-678-980-4	5
A003	004-765-409-5	4
A004	003-456-533-8	9
A005	002-678-980-4	7
A006	001-354-921-1	7
A006	002-678-880-2	4
A007	004-765-359-3	3
A008	001-987-760-9	7
A009	001-987-650-5	8
A010	003-456-433-6	5

Fig.3.5 Relational database instance

same concept, but with different attribute name in different relations. On the other hand, attributes representing different real-world concept may have same name in different relations. For example, the attribute **Pname** in the **PUBLISHER** relation and the attribute **Aname** in the **AUTHOR** relation represents different real-world concept, that is, publisher name and author name. However, they can be given a same attribute name, say, **Name** in different relations, that is, **PUBLISHER** and **AUTHOR** relations.

A_ID	ISBN	R_ID	Rating
A001	001-987-760-9	A002	6
A002	002-678-980-4	A001	2
A003	001-987-760-9	A002	6
A004	003-456-433-6	A010	5
A005	001-354-921-1	A006	7
A006	002-678-880-2	A005	4
A007	001-987-650-5	A009	8
A008	002-678-980-4	A001	2
A008	004-765-409-5	A003	4
A009	004-765-359-3	A007	3
A010	003-456-533-8	A004	9

Fig.3.6 *The AUTHOR_REVIEWER relation*

3.4 KEYS

Key is one of the important concepts of relational database. It can be *superkey*, *candidate key*, *primary key*, and *foreign key*.

Superkey

A superkey (SK) is a subset of attributes of a relation schema R , such that for any two distinct tuples t_1 and t_2 in the relation state r of R , we have $t_1[SK] \neq t_2[SK]$. It means that the combination of attributes in SK uniquely identify each tuple in r . In every relation schema R , we have at least one default superkey, that is, the set of all the attributes, since no two tuples can be identical in all of its attributes.

For example, consider a relation schema **BOOK** with three attributes **ISBN**, **Book_title**, and **Category**. As we know, the value of **ISBN** is unique for each tuple; hence, $\{\text{ISBN}\}$ is a superkey. In addition, the combination of all the attributes, that is, $\{\text{ISBN}, \text{Book_title}, \text{Category}\}$ is a default superkey for this relation schema.

Candidate Key

Generally, all the attributes of a superkey are not required to identify each tuple uniquely in a relation. Instead, only a subset of attributes of the superkey is sufficient to uniquely identify each tuple. Further, if any attribute is removed from this subset, the remaining set of attributes can no longer serve as a superkey. Such a minimal set of attributes, say K , is a **candidate key** (also known as **irreducible superkey**).

For example, the superkey $\{\text{ISBN}, \text{Book_title}, \text{Category}\}$ is not a candidate key, since its subset $\{\text{ISBN}\}$ is a minimal set of attributes that uniquely identify each tuple of **BOOK** relation. So, **ISBN** is a candidate key.

Note that each relation can have more than one candidate key, and all candidate keys are superkeys whereas all superkeys are not candidate keys. Further, if a superkey includes a single attribute, it is a candidate key. However, if a candidate key has multiple attributes, all these attributes must be needed to uniquely identify each tuple.

Primary Key

Since a relation can have more than one candidate key, one of these keys has to be chosen as the primary key to uniquely identify each tuple. A candidate key that is chosen to uniquely identify a tuple in a relation is known as **primary key** (PK). Other candidate keys that are not chosen as primary key are known as **alternate keys**. For example, in **BOOK** relation, there are two candidate keys, namely, **ISBN** and **Book_title** (considering no two books can have the same title). If the attribute **ISBN** is chosen as a primary key, the attribute **Book_title** becomes the alternate key.



Generally, the primary key comprises one attribute in a relation; however, more than one attribute can form the primary key. In that case, primary key is known as composite key.

The attribute whose value is never or rarely changed should be chosen as primary key. For example, in **PUBLISHER** relation, the attribute **Address** should not be chosen as a primary key since it can be changed often. However, the attribute **P_ID** can be chosen as a primary key since each publisher has a unique publisher id and generally, it does not change.

Foreign Key

Attribute of one relation can be accessed in another relation by enforcing a link between the attributes of two relations. This can be done by defining the attribute of one relation as the foreign key that refers to the primary key of another relation. For example, consider two relations, namely, **PUBLISHER** and **BOOK**. Here, all the books must be associated with a publisher that is already in the **PUBLISHER** relation. In this case, a foreign key will be defined on the **BOOK** relation, which will be related to the primary key of the **PUBLISHER** relation (see Figure 3.7). Thus, all books in the **BOOK** relation would be related to a publisher in the **PUBLISHER** relation.

A relation that references another relation is known as **referencing relation**, whereas a relation that is being referenced is known as **referenced relation**. For example, in Figure 3.7, the **PUBLISHER** relation is a referenced relation and the **BOOK** relation is a referencing relation.

Things to Remember

The candidate key with least number of attributes should be chosen as the primary key of the relation.



A foreign key may refer to its own relation. Such a foreign key is known as **self-referencing** or **recursive** foreign key.

A relation can have zero or more foreign keys and each foreign key can refer to different referenced relations. For example, consider **REVIEW** relation that includes the details of ratings given by the reviewer to a particular book. Here, books and reviewers (or authors) must be associated with book and author that are already in the **BOOK** and **AUTHOR** relations, respectively. In this case, foreign keys will be defined on the **REVIEW** relation that will be related to the primary keys of the **BOOK** and **AUTHOR** relations (see Figure 3.8).

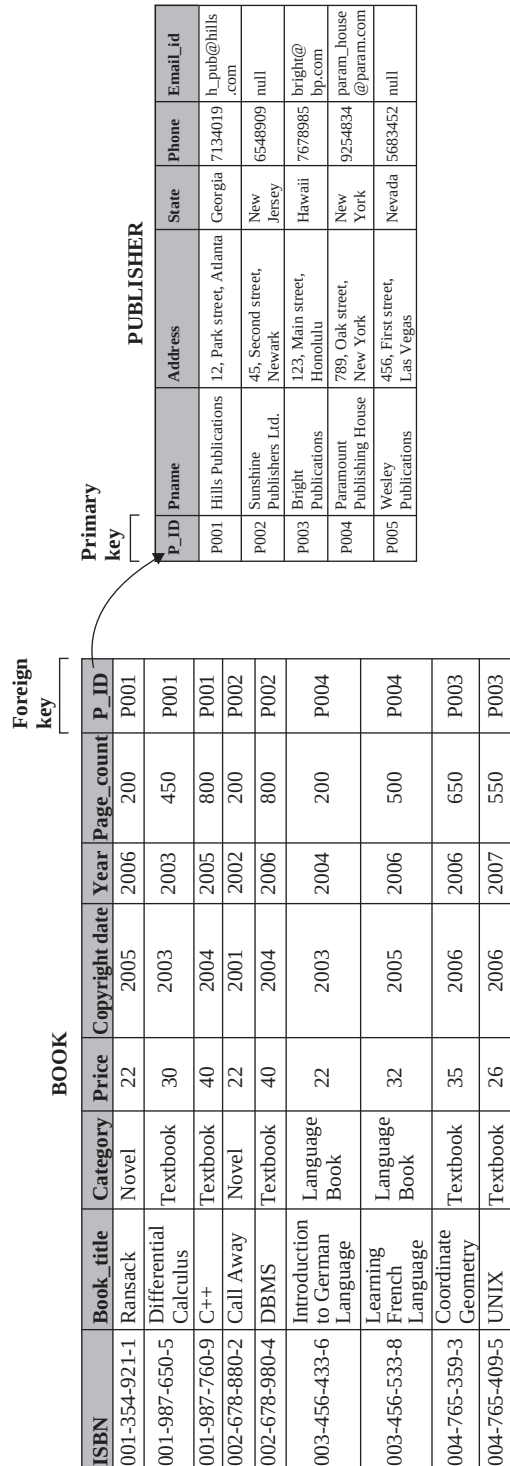


Fig. 3.7 Foreign key

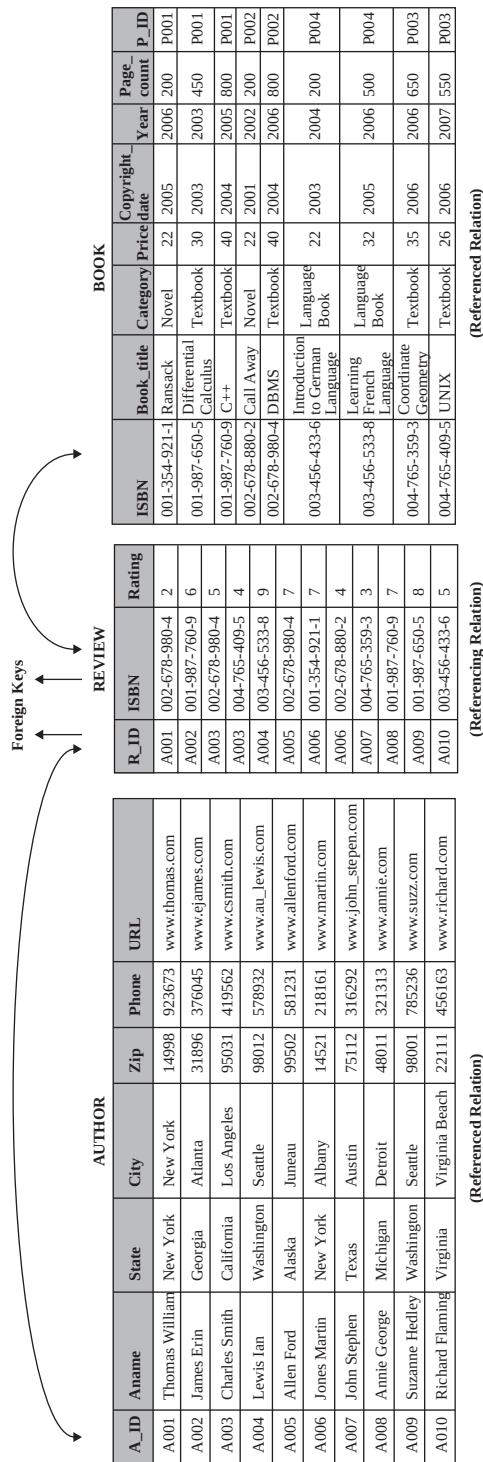


Fig. 3.8 More than one foreign key in one referencing relation

A foreign key attribute in referencing relation may have a different name from that of the primary key attribute of referenced relation. However, the domain of both the attributes must be same. For example, the foreign key attribute `R_ID` of the `REVIEW` relation has a different name from the primary key attribute `A_ID` of the referenced relation `AUTHOR`. However, the data type of both the attributes is same, that is, `string`.



A foreign key in a referencing relation that matches a primary key in a referenced relation represents many-to-one relationship between referencing and referenced relation.

3.5 DATA INTEGRITY

The fundamental function of the DBMS is to maintain the integrity of the data. Data integrity ensures that the data in the database is consistent, accurate, correct, and valid. It ascertains that the data adhere to the set of rules defined by the database administrator and hence, prevents the entry of the invalid information into database.

Data integrity is of four types, namely, *domain integrity*, *entity integrity*, *referential integrity*, and *semantic integrity*. Generally, domain integrity is applied on the attribute; entity integrity is applied on the tuple; referential integrity is applied on the relation; and semantic integrity ensures logical data in the database. This section discusses the data integrity in detail.

Domain Integrity

Domain integrity can be specified for each attribute by defining its specific range or domain. It requires that the attribute value must fall under a particular range in order to be valid. It ensures correct values for an attribute by defining the restrictions on the data type, format or range of possible values. For example, the domain of the attribute `Price` of the `BOOK` relation for the `Category: Textbook` may be between \$20 and \$200. Similarly, the domain of the attribute `Category` may be *Textbook*, *Novel* or *Language Book*.

More formally, domain integrity ensures

$A \text{ in } D$

where, A denotes an attribute, which has the same set of values as that of the domain D . Note that a tuple with n attributes satisfying domain integrity can be written as

$$\{ \langle A_1 : V_1, A_2 : V_2, \dots, A_n : V_n \rangle \mid V_1 \in D_1, V_2 \in D_2, \dots, V_n \in D_n \}$$

where,

$A_1, A_2, \dots, A_n$ are attributes of a tuple

$V_1, V_2, \dots, V_n$ are the value set associated with the domain $D_1, D_2, \dots, D_n$, respectively,

These statements imply that the attribute value must belong to its associated domain values.

For example, the first tuple of `PUBLISHER` relation can be written as

`<P_ID: P001, Pname: Hills Publications, Address: 12, Park Street, Atlanta, State: Georgia, Phone: 7134019, Email_id: h_pub@hills.com>`

Note that the domain integrity tests the value entered by the user into the database by ensuring that the attribute value entered by the user is in its associated domain. The attribute values that do not violate any domain integrity rules are considered as valid values even if they are logically incorrect. For example, if the **Price** attribute of the **BOOK** relation is assigned a value 65 instead of 56, this value is considered as valid since it belongs to the set of values defined in the domain, that is, between \$20 and \$200.

Instead of defining a set of range for the attribute, domain integrity can be defined conditionally on the attribute values that can be validated according to the respective rules applied on them. If the attribute values entered by the user violate the domain integrity rules, the entered values are rejected. For example, the restriction on the domain of the attribute **Price** of **BOOK** relation can be applied as

If **Category** is *Textbook*, **Price** must be between \$20 and \$200

If **Category** is *Language Book*, **Price** must be between \$20 and \$150

If **Category** is *Novel*, **Price** must be between \$20 and \$100

Entity Integrity

Entity integrity assigns restriction on the tuples of a relation and ascertains the accuracy and consistency of the database. It ensures that each tuple in a relation is uniquely identified in order to retrieve each tuple separately. The primary key of a relation is used to identify each tuple of a relation uniquely.

Entity integrity is a rule, which ensures that the set of attribute(s) that participates in the primary key cannot have duplicate value or *null* value. This is because each tuple in a relation is uniquely identified by a primary key attribute and if the value of the primary key attribute is duplicate or null, it becomes difficult to identify such tuple uniquely. For example, if the attribute **ISBN** is declared as a primary key of **BOOK** relation, entity integrity ensures that any two tuples of **BOOK** relation must have unique value for the attribute **ISBN** and must not be *null*.



NOTE If an attribute is declared as a primary key, it adheres to entity integrity rule.

Entity integrity rule requires that various operations like **insert**, **delete**, and **update** on a relation maintains uniqueness and existence of a primary key. In other words, **insert**, **delete**, and **update** operations on a relation results in valid values, that is, unique and non-*null* values in primary key attribute. Any operation that provides either duplicate or *null* value in primary key attribute is rejected.

Note that it is not mandatory for each relation to have a primary key but it is a good practice to create a primary key for each relation

Referential Integrity

Referential integrity condition ensures that the domain for a given set of attributes, say **S1**, in a relation **r1** should be the values that appear in certain set of attributes, say **S2**, in another relation **r2**. Here, the set **S1** is foreign key for the referencing relation **r1** and **S2** is primary key for referenced relation **r2**. In other words, the value of foreign key in the referencing relation must exist in the primary key attribute of the referenced relation, or that the value of foreign key is *null*.

Referential integrity also ensures that the data type of the foreign key in referencing relation must match the data type of the primary key of referenced relation. For example, the data type of both foreign key attribute **P_ID** in **BOOK** relation and primary key attribute **P_ID** in **PUBLISHER** relation is same, that is, **string**.

Note that there may be some tuples in the referenced relation that do not exist in the referencing relation. For example, in Figure 3.9, the tuple with the attribute value **P005** in **PUBLISHER** relation does not exist in the **BOOK** relation.

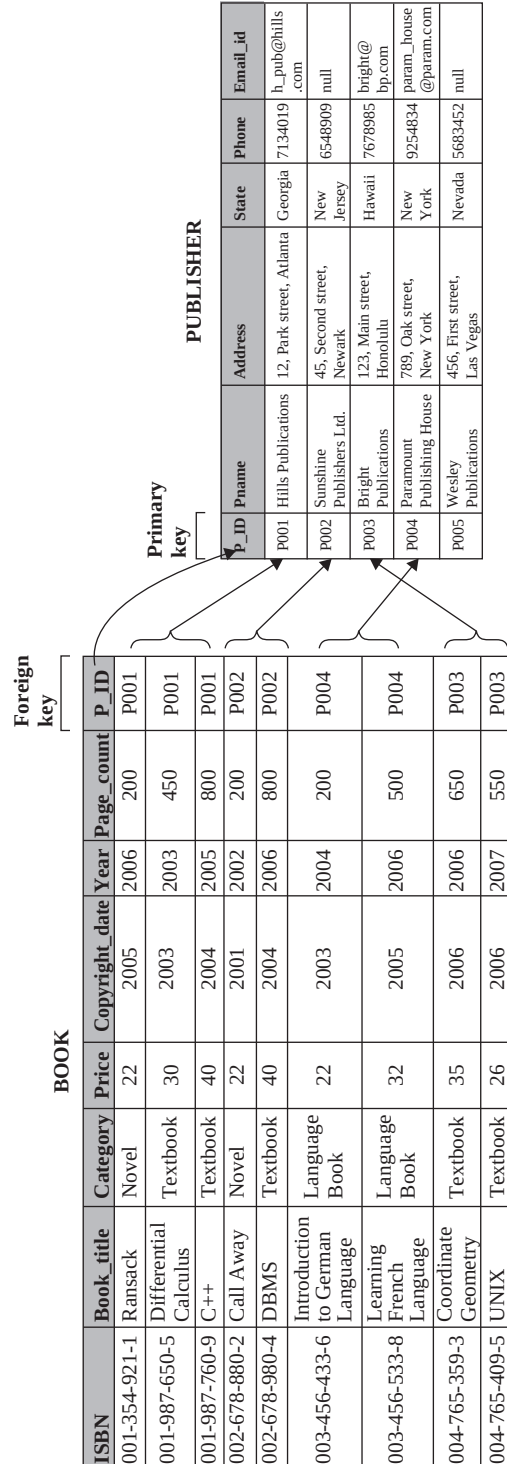


Fig. 3.9 Referential integrity

Semantic Integrity

To represent real world accurately and consistently, the business rules and logical rules (that are derived from our knowledge of the semantics of the real world) must be enforced on database. These rules are known as **semantic integrity**. Semantic integrity ensures that the data in the database is logically consistent and complete with respect to the real world. Examples of semantic integrity are:

- ⇒ Number of pages of a book cannot be zero.
- ⇒ One book can be published by one publisher only.
- ⇒ Copyright date of a book cannot be later than its published date.
- ⇒ An author cannot review his own book.

3.5.1 Enforcing Data Integrity

In order to maintain the accuracy and consistency of data in a database, integrity of the data is enforced through *integrity constraints*. **Integrity constraints** are the rules defined on a relational database schema and satisfied by all the instances of database in order to model the real-world concepts correctly. They ensure the consistency of database while the modifications are made by the users. For example, the integrity constraint ensures that the value of ISBN in BOOK relation cannot be duplicated or null.



A relation that conforms to all the integrity constraints of the schema is known as **legal relation**. On the other hand, a relation that does not conform to all the integrity constraints of the schema is known as **illegal relation**.

Integrity constraints can be categorized into three parts, namely, *table-related constraints*, *domain constraints*, and *assertion constraints*.

- ⇒ **Table-related constraints** are defined within a relation definition. This type of constraint can be specified either as a part of the attribute definition or as an element in the relation definition. When a constraint is specified as a part of the attribute definition, it is known as **column-level** constraint. Column-level constraints are checked when the constrained attribute is modified. On the other hand, when a constraint is specified as an element in the relation definition, it is known as **table-level** constraint. Table-level constraints are checked when any attribute of a relation is modified. Both column-level and table-level constraints hold different types of constraints (see Figure 3.10).

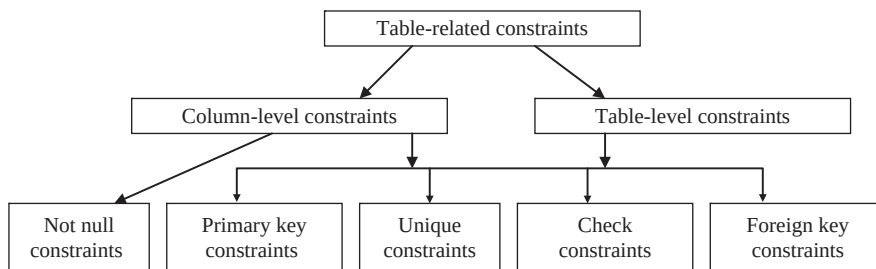


Fig.3.10 Table-related constraints

- ⇒ **Domain constraints** are specified within a domain definition. In other words, it is associated with any attribute that is defined within a particular domain. It supports not null and check constraints.
- ⇒ **Assertion constraints** are specified within an assertion definition, which will be discussed in Chapter 05.



NOTE The syntax for enforcing various constraints is given in Chapter 05.

Primary Key Constraint

The **primary key** constraint ensures that the attributes, which are declared as primary key must be unique and not *null*. Primary key constraint enforces *entity integrity* for the relation and constraints the relation in the following ways:

- ⇒ The primary key attribute must have unique set of values for all the tuples in a relation.
- ⇒ The primary key attribute cannot have *null* value.
- ⇒ A relation can have only one primary key attribute.

Unique Constraint

The **unique** constraint ensures that a particular set of attributes contains unique values and hence, two tuples cannot have duplicate values in specified attributes. Like primary key, it also enforces *entity integrity*. For example, the unique constraint can be specified on **Pname** attribute of **PUBLISHER** relation to prevent tuples with duplicate publisher names.

Unique constraint is preferable over primary key constraint to enforce uniqueness of a non-primary key attribute. This is because

- ⇒ Multiple unique constraints can be applied on a relation, whereas only one primary key constraint can be defined on a relation.
- ⇒ Unlike primary key constraint, unique constraint allows the attribute value to be *null*.



Primary key and unique key constraints act as a parent key when applying a foreign key constraint. Parent key is the key that identifies the tuples of the relation uniquely.

Check Constraint

The **check** constraint can be used to restrict an attribute to have values in a given range, that is, for each tuple in a relation, the values in a certain attribute, must satisfy a given condition. One way to achieve this is by applying check constraint during the declaration of the relation. For example, if the check constraint (**PRICE**>20) is applied on the attribute **Price** of the **BOOK** relation, it ensures that the value entered into the attribute **Price** must be greater than \$20.

Check constraints can be applied on more than one attribute simultaneously. For example, in the attributes **Copyright_date** and **Published_date**, constraints can be applied in such a way that **Copyright_date** must be either less than or equal to the **Published_date**.



NOTE If the inserted value violates the check constraint, the insert or update operations will not be permitted.

The check constraint can also be applied during domain declaration. It allows specifying a condition that must be satisfied by any value assigned to an attribute whose type is the domain. For example, the check constraint can ensure that the domain, say `dom`, allows values between 20 and 200. If domain `dom` is assigned to the attribute `Price` of a `BOOK` relation, it ensures that the attribute `Price` must have values between 20 and 200. As a result, if a value that is not between 20 and 200 is inserted into the constrained attribute, an error occurs.

NotN ullC onstraint

Every attribute has a nullability characteristic that shows whether a particular attribute accepts a *null* value or not. By default, every attribute accepts *null* values but this default nullability characteristic can be overridden by using the **not null** constraint. The not null constraint ensures that the attribute must not accept *null* values. For example, consider a tuple in the `BOOK` relation where the attribute `ISBN` contains a *null* value. Such a tuple provides book information for an unknown ISBN and hence, it does not provide appropriate information. In such case, *null* value must not be allowed and this can be done by constraining the attribute `ISBN` using not null constraint. Here, the not null constraint prevents the *null* value to be inserted into the attribute `ISBN`. Any attempt to change the attribute value of `ISBN` to *null* value results in an error. Note that the *null* values are not allowed in the primary key attribute of a relation. Since `ISBN` is a primary key attribute of `BOOK` relation, it cannot accept *null* values. Hence, it is not necessary to explicitly specify not null constraint for the attribute `ISBN`.

The not null constraint enforces *domain integrity* by ensuring that the attribute of a particular domain is not permitted to take *null* values. For example, a domain, say `dom2`, can be restricted to take non-*null* values. If the domain `dom2` is assigned to the attribute `Category` of a `BOOK` relation, it ensures that the attribute `Category` must have some values. As a result, if *null* value is inserted into the constrained attribute, it will not be accepted as it violates the not null constraint.

ForeignK eyC onstraint

A **foreign key** constraint allows certain attributes in one relation to refer to attributes in another relation. The relation on which foreign key constraint is defined contains the partial information. Its detailed information can be searched from another relation with a matching entry. In this case, a foreign key constraint will not allow the deletion of a tuple from the relation with detailed information if there is a tuple in the relation with partial information exist. In other words, a foreign key constraint not only controls the data that can be stored in the referencing relation but it also controls the changes made to the referenced relation.

For example, if the tuple for a publisher is deleted from the `PUBLISHER` relation, and the publisher's ID is used in the `BOOK` relation, the deleted publisher's ID becomes orphaned in the `BOOK` relation. This situation can be prevented by a foreign key constraint. This constraint ensures that changes made to the data in the `PUBLISHER` relation will not make the data in the `BOOK` relation orphan.

Foreign key constraint enforces *referential integrity* by ensuring that an attribute in a relation `S` whose value in each tuple either matches with the values of the primary key in another relation `R` or is *null*. For example, the foreign key attribute `R_ID` in the `REVIEW` relation must have either the same values as that of the primary key `A_ID` of the `AUTHOR` relation or must be *null*.

LearnMore

It is not necessary that the foreign key constraint can only be related with the primary key constraint in another relation. A foreign key constraint can also be defined to refer to the attributes of a unique constraint in another relation.

3.6 CONSTRAINT VIOLATION WHILE UPDATING DATA

In addition to storing the data, various operations such as add, remove, and modify can also be performed on the relation. These operations must be performed in such a way that various integrity constraints should not be violated. The update operations on the relation are *insert*, *delete*, and *update* (or *modify*).

3.6.1 The Insert Operation

The **insert** operation creates a new tuple or a set of tuples into a relation and provides attribute values to the inserted tuples. These inserted values must satisfy all the integrity constraints to maintain the consistency of the database. For example, the statement `Insert<'003-456-654-3', 'Discrete Mathematics', 'Textbook', 60, 2007, 2007, 650, 'P002'>` satisfies all the integrity constraints and hence, these values can be inserted into the **BOOK** relation.

However, if the attribute values violate one or more integrity constraints, the attribute values are rejected and the insert operation cannot be performed. Consider some cases given here.

- ⇒ If the attribute value is inserted into a tuple of **BOOK** relation, it must belong to the associated domain of possible values otherwise the domain integrity will be violated. For example, in the statement, `Insert <'002-678-999-5', 'LINUX', 'Textbook', 300, 2006, 2007, 750, 'P005'>` into **BOOK**, the price of the book does not belong to the specified domain, that is, between \$20 and \$200. So, this statement violates the domain integrity and hence, the values are rejected.
- ⇒ If the attribute value is inserted into a primary key attribute of **BOOK** relation, it must be unique and non-*null* otherwise the entity integrity will be violated. For example, in the statement, `Insert <'003-456-654-3', 'Software Engineering', 'Textbook', 75, 2005, 2006, 700, 'P003'>` into **BOOK**, the ISBN of the book has the same value as the ISBN of the existing tuple. So, this statement violates the primary key constraint and hence, the values are rejected. Similarly, consider another statement, `Insert <null, 'Software Engineering', 'Textbook', 75, 2005, 2006, 700, 'P004'>` into **BOOK**. In this statement, the ISBN of the book does not have any value, so it violates the entity integrity and hence, the values are rejected.
- ⇒ If the attribute value is inserted into a foreign key attribute of **BOOK** relation, it must exist in the **PUBLISHER** relation otherwise the referential integrity will be violated. For example, in the statement, `Insert <'004-765-558-5', 'Introduction to Japanese Language', 'Language Book', 45, 2007, 2007, 550, 'P007'>` into **BOOK**, the **P_ID** of the **BOOK** relation does not exist in the **P_ID** of the **PUBLISHER** relation. So, this statement violates the referential integrity and hence, the values are rejected.
- ⇒ If the attribute value is inserted into a foreign key attribute of **BOOK** relation, it must be logically correct otherwise the semantic integrity will be violated. For example, in the statement, `Insert<'004-765-558-5', 'Introduction to French Language', 'Language Book', 45, 2007, 2007, 0, 'P003'>` into **BOOK**, the number of pages is zero, which is logically incorrect. So, this statement violates the semantic integrity and hence, the values are rejected.

3.6.2 The Delete Operation

A tuple or a set of tuples can be deleted from a relation using the **delete** operations. Special attention must be given to the tuples that are being referenced by other tuples in the database, since it can violate the referential integrity. If deletion violates the referential integrity then it is either rejected or cascaded. For example, if an attempt is made to delete the tuple with **P_ID** = "P004" in **PUBLISHER** relation, the delete operation is rejected as it violates the referential integrity. On the other hand, if the tuple of **PUBLISHER** relation with **P_ID** = "P005" is deleted, the attribute values of this tuple will

be deleted since this tuple is not referenced by any tuple of **BOOK** relation and does not violate the referential integrity.

Deletion can be cascaded by deleting the tuples in the referencing relation that references the tuple to be deleted in the referenced relation. For example, in order to delete $P_ID = "P004"$ from **PUBLISHER** relation, the tuples with $P_ID = "P004"$ of **BOOK** relation must be first deleted. In addition to rejecting and cascading the deletion, the tuples that reference the tuple to be deleted can be modified either to *null* value or to another existing tuple. Note that assigning the *null* value to the tuple must not violate the entity integrity. If it violates then the *null* value should not be assigned to the tuple.

Note that it is not possible to cascade the deletion in each case. To understand this concept, consider the modified form of *Online Book* database in which the relation **BOOK** is altered by adding an attribute **A_ID**. The resultant relation can be given as shown in Figure 3.11.

ISBN	Book_title	Category	Price	Copyright_date	Year	Page_count	P_ID	A_ID
001-354-921-1	Ransack	Novel	22	2005	2006	200	P001	A005
001-987-650-5	Differential Calculus	Textbook	30	2003	2003	450	P001	A007
001-987-760-9	C++	Textbook	40	2004	2005	800	P001	A001
001-987-760-9	C++	Textbook	40	2004	2005	800	P001	A003
002-678-880-2	Call Away	Novel	22	2001	2002	200	P002	A006
:								
:								

Fig.3.1 1 *The modified BOOK relation*

If an attribute value *A001* has to be deleted from the **AUTHOR** relation, the corresponding tuple with the attribute value *A001* must also be deleted from the modified **BOOK** relation. If this is done, in that case the corresponding book as well as publisher information of that particular author will also be deleted. So in this case, the attribute value of the author (whose tuple has to be deleted) in the modified **BOOK** relation is updated to *null* value instead of deleting that tuple.

3.6.3 TheUpdateOperation

The attribute values in a tuple or a set of tuples can be modified using the **update** operation. Update operation changes the existing value of the attribute in a tuple to the new and valid value. It ensures that the new value satisfies all the integrity constraints. For example, if the value of the attribute **Price** with **ISBN= "001-987-760-9"** is updated from \$25 to \$50, the changes are reflected in a relation since it satisfies all the integrity constraints.

Things to Remember

If a referencing attribute is a part of the primary key and violates the referential integrity, we cannot set it to *null* value, since doing this would result in the violation of entity integrity.

Changing the value of attributes does not create any problem as long as the valid values are entered in a relation. However, changing the value of primary key attribute or foreign key attribute

can be a critical issue. Since the primary key attribute identifies each tuple, changing this attribute value must satisfy all the integrity constraints. For example, if the value of the attribute P_ID in PUBLISHER relation is updated from P001 to P002, the changes are reflected in a relation only if it satisfies all the integrity constraints.

Similarly, the foreign key attribute must be modified in such a manner, which ensures that the new value either refers to the existing value in the referenced relation or is *null*. For example, if the P_ID of the BOOK relation with ISBN= "001-987-760-9" is updated to *null* or any other value that exists in the attribute P_ID of PUBLISHER relation, the value of the foreign key attribute is modified. However, if the new value of P_ID of the BOOK relation does not exist in the P_ID of the PUBLISHER relation, it violates the referential integrity and hence, the values are rejected.

3.7 MAPPING E-R MODEL TO RELATIONAL MODEL

E-R model provides a conceptual model of the real-world concepts, which is represented in a database. To represent the E-R database schema to the relation schema, the relational model is used. In other words, E-R model is mapped to the relational model by representing E-R database schema by a collection of relation schemas.

The mapping from E-R model to relational model is possible due to the reason that both the models represent the real world logically and follow similar design principles. The *Online Book* database is used as an example to illustrate the mapping from E-R database schema to a collection of relation schema. Each entity type and relationship type in the *Online Book* database is represented as a unique relation schema and relation to which the name of the corresponding entity type or relationship type is assigned. The E-R schema of *Online Book* database is shown in Figure 3.12. In this section, the steps to translate an E-R database schema into a collection of relation schema and relations are shown.

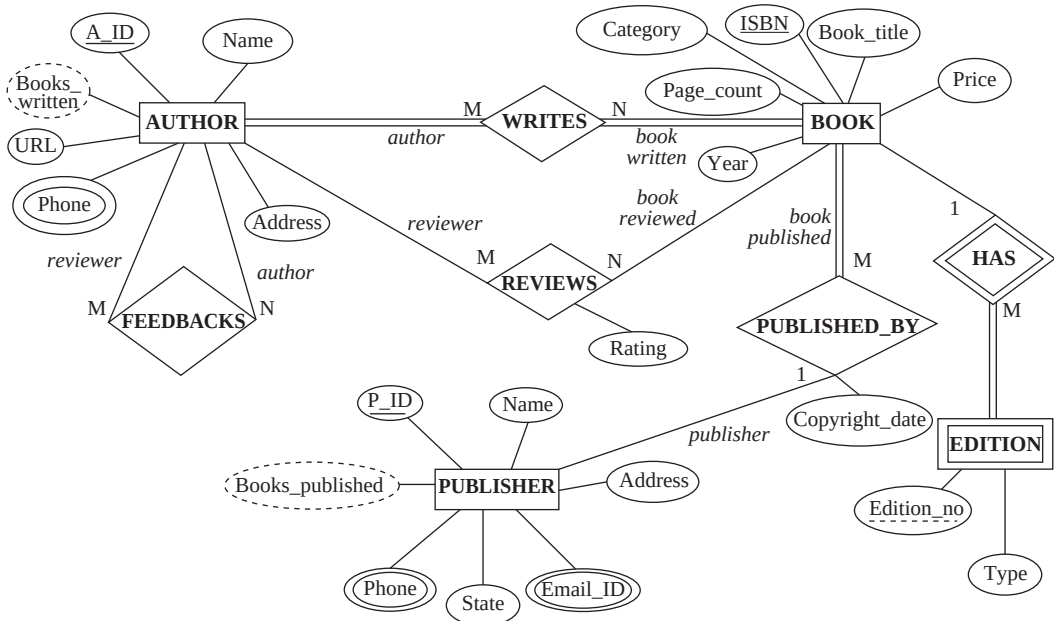


Fig.3.12 E-R schema for the Online Book database

3.7.1 Representation of Entity Types to Relation

An entity type is mapped to a relation by representing different attributes $A_1, A_2, \dots, A_n$ of an entity type E as attributes $A_1, A_2, \dots, A_n$ of a relation R . Each entity instance of the entity type E represents a tuple in the relation R , and the primary key of the entity type E becomes the primary key of the relation R .

Consider the E-R diagram of the entity type **BOOK** in Figure 3.13. The entity type **BOOK** has six attributes, namely, ISBN, Book_title, Category, Price, Year, and Page_count.

The entity type **BOOK** can be represented by a relation schema **BOOK** with six attributes

BOOK(ISBN, Title, Category, Price, Year, Page_count)

As discussed earlier, the mapping from E-R model to relational model results in a relation. The resultant relation is known as **entity relation** since each entity instance of the entity type represents a tuple in a relation. A relation with six attributes of the **BOOK** schema is shown in Figure 3.14.

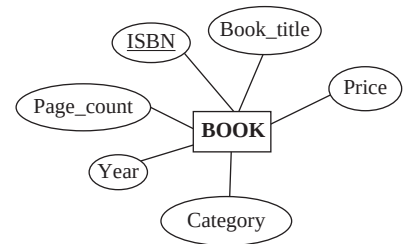


Fig. 3.13 The **BOOK** entity type

ISBN	Book_title	Category	Price	Year	Page_count
001-354-921-1	Ransack	Novel	22	2006	200
001-987-650-5	Differential Calculus	Textbook	30	2003	450
001-987-760-9	C++	Textbook	40	2005	800
002-678-880-2	Call Away	Novel	22	2002	200
002-678-980-4	DBMS	Textbook	40	2006	800
003-456-433-6	Introduction to German Language	Language Book	22	2004	200
003-456-533-8	Learning French Language	Language Book	32	2006	500
004-765-359-3	Coordinate Geometry	Textbook	35	2006	650
004-765-409-5	UNIX	Textbook	26	2007	550

Fig. 3.14 The **BOOK** relation

In this figure, the tuple $\langle '001-354-921-1', 'Ransack', 'Novel', 22, 2006, 200 \rangle$ represents that the book with ISBN '001-354-921-1' has a book title 'Ransack' under the category 'Novel' with 200 pages, its price is \$22, and it is published in the year 2006.



NOTE The attributes *P_ID* and *Copyright_date* will be added into the *BOOK* relation in later steps.

3.7.2 Representation of Weak Entity Types

Representation of weak entity type with attributes $a_1, a_2, \dots, a_m$ depends on the strong entity type with the attributes $A_1, A_2, \dots, A_n$. Weak entity type must have total participation in the relationship type and must participate in one-to-many relationship with the strong entity type that defines it (that is, one strong entity type with many weak entity types). Weak entity type can be represented by the relation schema *R* with one attribute for each member of the set as

$$\{a_1, a_2, \dots, a_m\} \cup \{A_1, A_2, \dots, A_n\}$$

The primary key of the weak entity type consists of the primary key of the strong entity type as foreign key and its own partial key. In addition to the primary key, foreign key constraints are applied on the weak entity type as the primary key of the weak entity type refers to the primary key of the strong entity type.



Whenever a strong entity type is deleted, the related weak entity types must also be deleted. .

For example, consider the E-R diagram of entity type *BOOK* having weak entity type *EDITION* shown in Figure 3.15. The weak entity type *EDITION* has two attributes, namely, *Edition_no* and *Type*.

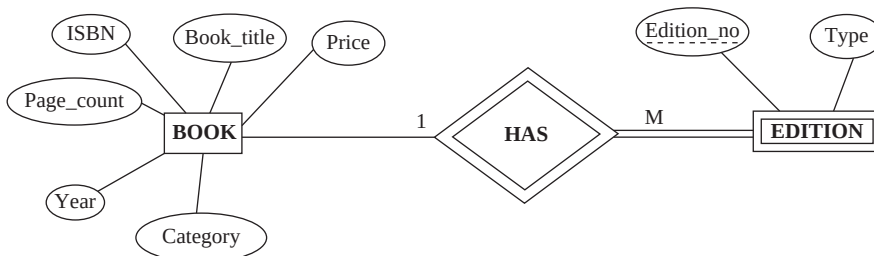


Fig.3.15 The entity type *BOOK* with weak entity type *EDITION*

The primary key of the entity type *BOOK*, on which weak entity type *EDITION* depends, is *ISBN*. Thus, the relation schema of the weak entity type *EDITION* can be represented as

EDITION=(*ISBN*, *Edition_no*, *Type*)

In this statement, the primary key consists of the primary key of *BOOK*, along with the partial key of *Edition*, which is *Edition_no*. The foreign key constraint can be created on the *Edition* schema with the attribute *ISBN* referencing the primary key attribute of the *BOOK* relation and *Edition_no* is its own partial key. The corresponding *EDITION* relation is shown in Figure 3.16.

3.7.3 Representation of Relationship Types

There relationship type *R* is mapped to a relation, which includes

- ⇒ Primary key of each participating entity type as a foreign key
- ⇒ The set of descriptive attributes (if any) of *R*

Generally, primary key attributes of the participating entity types are chosen to identify a specific tuple in a binary relationship types. The primary key is chosen as shown in Table 3.1.

Consider the relationship type *REVIEWS* in the E-R diagram of Figure 3.17. Each author can review one or more books and similarly, one book can be reviewed by one or more authors.

For example, the relationship type *REVIEWS* has two entity types, namely, *AUTHOR* with the primary key *A_ID*, and *BOOK* with the primary key *ISBN*. Since the relationship type has one descriptive attribute, namely, *Rating*, the *REVIEW* schema of the relationship type *REVIEWS* has three attributes that can be shown as

REVIEW(*R_ID*, *ISBN*, *Rating*)

As stated earlier, the same concept of author id can be given a different name in different relation in order to distinguish the role of a reviewer from the author. Hence, the attribute *A_ID* of the *AUTHOR* relation is given a different name, that is, *R_ID* in the *REVIEW* schema.

Since the relationship type is many-to-many, the primary key for the *REVIEW* relation is the union of the primary key of entity type *AUTHOR*, that is, *R_ID* and primary key of entity type *BOOK*, that is, *ISBN*. In addition, *R_ID* and *ISBN* cannot be *null* since these keys are primary keys and hence, not null constraint and unique constraint are implicit for these fields.

In addition to the primary key, foreign key constraints are also created on the *REVIEW* relation with attribute *A_ID* referencing the primary key of *AUTHOR* relation and attribute *ISBN* referencing the primary key of *BOOK* relation.

ISBN	Edition_no	Type
001-354-921-1	First	International
001-987-650-5	Second	Limited
001-987-760-9	Third	Limited
002-678-880-2	First	International
002-678-980-4	Fourth	International
003-456-433-6	First	Limited
003-456-533-8	Second	Limited
004-765-359-3	Second	International
004-765-409-5	Third	International

Fig. 3.16 The *EDITION* relation

Table 3.1 Identifying primary key for binary relationship types

Relationship Type	Primary Key for Relation
one-to-one	Primary key of either entity type participating in the relationship type
one-to-many or many-to-one	Primary key of entity type on the 'many' side of the relationship type
many-to-many	Union of primary key of each participating entity type
<i>n</i> -ary without arrow on its edges	Union of primary key attributes from the participating entity types
<i>n</i> -ary with arrow on one of its edges	Primary key of the entity type not on the 'arrow' side of relationship type

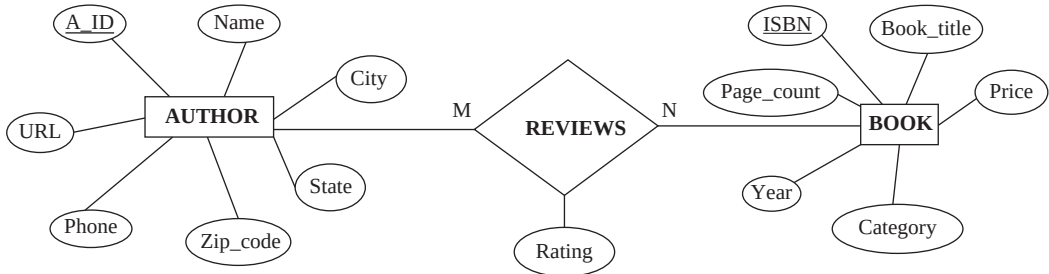


Fig.3.17 A relationship type REVIEWS

The corresponding relation REVIEW of the relation schema is shown in the Figure 3.18.

Note that in the AUTHOR relation, no reference to the BOOK relation is given. Similarly, reference of AUTHOR relation is not given in the BOOK relation. This is because, in case of many-to-many relationship, this is logically implemented by including a separate relation. In this case, a REVIEW relation is included that relates AUTHOR and BOOK relations. The REVIEW relation links the primary key of AUTHOR relation with the primary key of the BOOK relation.



NOTE A relationship type relating a weak entity type to the corresponding strong entity type is not required in a relational database since it creates redundancy.

So far, we map each relationship type R to a different schema. Alternatively, the schema of R can be merged with the schema of one of the entity type that participates in R. Consider a many-to-one relationship type PUBLISHED_BY shown in Figure 3.19.

The double line in the Figure 3.19 indicates that the participation of BOOK in the relationship type PUBLISHED_BY is total. Hence, the book cannot exist without being associated with a particular publisher. Thus, we can combine the schema for PUBLISHED_BY with the schema of BOOK, and we require only two schemas, which are given here.

BOOK(ISBN, Book_title, Category, Price, Copy-right_date, Year, Page_count, P_ID)
 PUBLISHER(P_ID, Pname, Address, State, Phone, Email_id)

R_ID	ISBN	Rating
A001	002-678-980-4	2
A002	001-987-760-9	6
A003	002-678-980-4	5
A003	004-765-409-5	4
A004	003-456-533-8	9
A005	002-678-980-4	7
A006	001-354-921-1	7
A006	002-678-880-2	4
A007	004-765-359-3	3
A008	001-987-760-9	7
A009	001-987-650-5	8
A010	003-456-433-6	5

Fig. 3.18 The REVIEW relation

The corresponding relation BOOK can be given as shown in Figure 3.20.

The primary key of entity type, into whose schema the relationship type schema is combined, becomes the primary key of combined schema. So, the primary key of the combined schema is the primary key of BOOK, that is, ISBN.

The schema representing the relationship type would have foreign key constraints that refer to each of the participating entity type in the relationship type. Here, the foreign key constraint of the entity type, into whose schema the relationship type schema is combined, is dropped and foreign key constraints of the remaining entity types are added to the combined schema. For example, the foreign

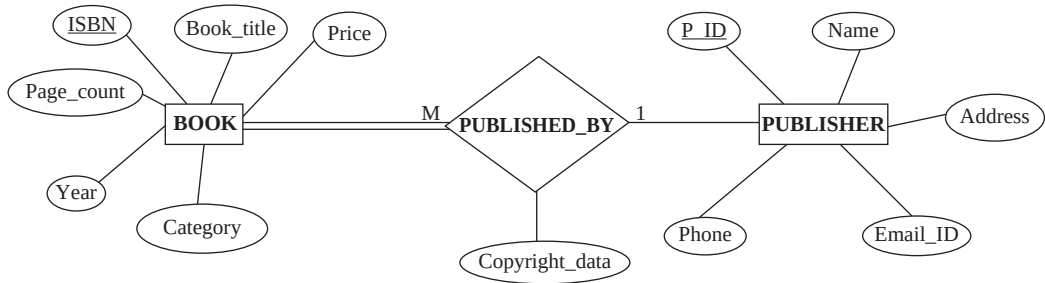


Fig.3.19 E-R diagram of relationship type *PUBLISHED_BY*

ISBN	Book_title	Category	Price	Copyright_date	Year	Page_count	P_ID
001-354-921-1	Ransack	Novel	22	2005	2006	200	P001
001-987-650-5	Differential Calculus	Textbook	30	2003	2003	450	P001
001-987-760-9	C++	Textbook	40	2004	2005	800	P001
002-678-880-2	Call Away	Novel	22	2001	2002	200	P002
002-678-980-4	DBMS	Textbook	40	2004	2006	800	P002
003-456-433-6	Introduction to German Language	Language Book	22	2003	2004	200	P004
003-456-533-8	Learning French Language	Language Book	32	2005	2006	500	P004
004-765-359-3	Coordinate Geometry	Textbook	35	2006	2006	650	P003
004-765-409-5	UNIX	Textbook	26	2006	2007	550	P003

Fig. 3.20 The BOOK relation

key constraint that refers to the **BOOK** is dropped, whereas the foreign key constraint with **P_ID** that refers to **PUBLISHER** is retained as a constraint on the combined schema **BOOK**.



NOTE Even if the participation of the entity type in the relationship type is partial, the schemas can be combined by using null values.

For one-to-one relationship type, the schema of the relationship type can be merged with the relation schema of either of the entity types.

3.7.4 Representation of Composite and Multivalued Attributes

As discussed in Section 3.6.1, different attributes of an entity type become the attributes of a relation. If an entity type has composite attributes, no separate attribute is created for the composite attribute itself. Instead, separate attributes for each of its component attributes are created.

For example, consider the E-R diagram of the entity type **AUTHOR** with the composite attribute **Address** shown in Figure 3.21.

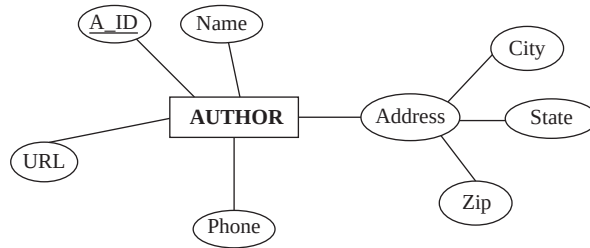


Fig.3.21 E-R diagram of entity type **AUTHOR**

The attribute **Address** of the entity type **AUTHOR** is a composite attribute with the component attributes, namely, **City**, **State**, and **Zip**. The separate attributes are created for these component attributes and the schema of the **AUTHOR** relation is given here.

AUTHOR(A_ID, Aname, State, City, Zip, Phone, URL)

For the multivalued attribute, a separate relation is created with the primary key of entity type and with an attribute corresponding to the multivalued attribute of the entity type. For example, the attribute **Phone** of entity type **AUTHOR** is a multivalued attribute. Consider the E-R diagram of multivalued attribute **Phone** of entity type **AUTHOR** in Figure 3.22.

Multivalued attribute **Phone** of entity type **AUTHOR** can be represented by the relation schema as

AUTHOR_PHONE(A_ID, Phone)

And the corresponding relation, say **AUTHOR_PHONE**, can be given as shown in Figure 3.23.

Note that each value of the multivalued attribute **Phone** is represented in a separate tuple of the **AUTHOR_PHONE** relation. For example, an author with **A_ID** as **A001** and **Phone** as 923673 and 92374 is represented in two tuples, namely, {**A001**, 923673} and {**A001**, 923743} in the **AUTHOR_PHONE** relation.

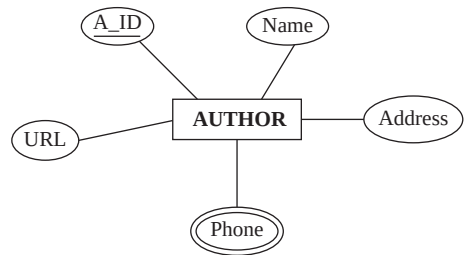


Fig. 3.22 The entity type **AUTHOR** with multivalued attributes

A_ID	Phone
A001	923673
A001	923743
A002	376045
A002	376123
A003	419456
A003	419562
A004	578123
A004	578932

Fig. 3.23 The **AUTHOR_PHONE** relation



If the multivalued attribute is an alternate key of the entity type, the primary key of new relation will be the combination of the primary key and the multivalued attribute of the entity type.

3.7.5 Representation of Extended E-R Features

So far, we have discussed the representation of basic features of E-R model. In this section we discuss the representation of extended E-R features, which include generalization/specialization and aggregation. The representation of generalization involves two different approaches of designing the relation. The first approach creates separate relation for the higher-level entity type as well as for all the lower-level entity types. The relation for higher-level entity type includes its primary key attributes and all other attributes. Each relation for lower-level entity type includes the primary key of higher-level entity type and its own attributes. The primary key attribute of lower-level entity type refers to the primary key attribute of the higher-level entity type and hence, creates the foreign key constraint.

For example, consider the EER diagram of entity type **BOOK** representing generalization in Figure 3.24.

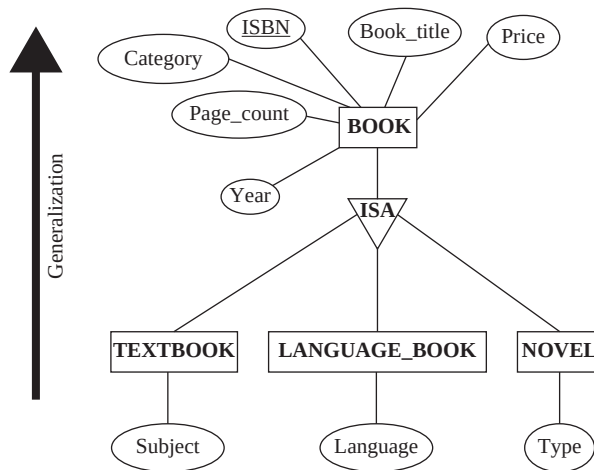


Fig.3.24 EER diagram representing generalization

In this figure, the higher-level entity type is **BOOK** and the lower-level entity types are **TEXT-BOOK**, **LANGUAGE_BOOK**, and **NOVEL**. The relation schemas for the higher-level and lower-level entity types can be given as

```
BOOK(ISBN, Book_title, Category, Price, Year, Page_count)
TEXTBOOK(ISBN, Subject)
LANGUAGE_BOOK(ISBN, Language)
NOVEL(ISBN, Type)
```

The corresponding relation **BOOK** is shown in Figure 3.14 and corresponding relations **TEXT-BOOK**, **LANGUAGE_BOOK**, and **NOVEL** are shown in Figure 3.25.

In this figure, the primary key attribute **ISBN** of relations of lower-level entity types **TEXTBOOK**, **LANGUAGE_BOOK**, and **NOVEL** refers to the primary key attribute of the relation of higher-level entity type **BOOK** and hence, creates the foreign key constraints.

The second approach creates separate relations only for all the lower-level entity types. Each relation of lower-level entity type includes all the attributes of higher-level entity type and its own attributes. This approach is possible when the generalization is disjoint and complete.

TEXTBOOK		LANGUAGE_BOOK		NOVEL	
ISBN	Subject	ISBN	Language	ISBN	Type
001-987-650-5	Maths	003-456-433-6	German	001-354-921-1	Fiction
001-987-760-9	Computer	003-456-533-8	French	002-678-880-2	Non-Fiction
002-678-980-4	Computer				
004-765-359-3	Maths				
004-765-409-5	Computer				

Fig.3.25 *The TEXTBOOK, LANGUAGE_BOOK, and NOVEL relations*

For example, the relation schemas of the lower-level entity types can be given as

TEXTBOOK(ISBN, Book_title, Category, Price, Year, Page_count, Subject)

LANGUAGE_BOOK(ISBN, Book_title, Category, Price, Year, Page_count, Language)

NOVEL(ISBN, Book_title, Category, Price, Year, Page_count, Type)

The second approach has a limitation that the foreign key constraint cannot be created on the relations for lower-level entity type. This is because there is no relation existing to which a foreign key constraint on relations for lower-level entity type can refer. To overcome this limitation, a relation has to be created that contains at least the primary key attribute of the relation for higher-level entity type.

Note that the second approach is not used for overlapping generalization as it may lead to redundancy. For example, if a book is both a textbook as well as a language book, the attributes **Book_title**, **Price**, **Year**, and **Page_count** have to be stored twice. In addition, the second approach is not used for generalization that is not complete. For example, if a book is neither a textbook nor a language book then another relation, that is, **BOOK** has to be used to represent such books.

An EER diagram containing aggregation expresses relationship between the relationships and a collection of entities/entity types. When EER diagram with aggregation is transformed in the relation, the primary key of the relation includes the primary key of the entity type and the primary key of the relationship type. The descriptive attributes of the entity type are also included in the relation.

In addition to the primary key, foreign key constraint can be applied on the relationship types involving aggregation. The primary key of the aggregation refers to the primary keys of the entity type and the relationship type.

For example, consider the EER diagram representing aggregation shown in Figure 3.26.

Since the primary key for the relationship type is the union of the primary keys of the entity types involved, the primary key for the relationship type **WRITES** is the union of the primary keys of the entity type **BOOK** and **AUTHOR**. Hence, the schema for the relationship type **WRITES** can be written as

{ISBN, A_ID}

The primary key of the relation, say **COPYRIGHT**, includes the primary key of the entity type **PUBLISHER** and the primary key of the relationship type **WRITES**. The descriptive attribute **COPYRIGHT_DATE** of the relation **COPYRIGHT** is also included. Now, the schema for the relation **COPYRIGHT** can be represented as {ISBN, A_ID, P_ID, Copyright_date} and the relation is shown in Figure 3.27.

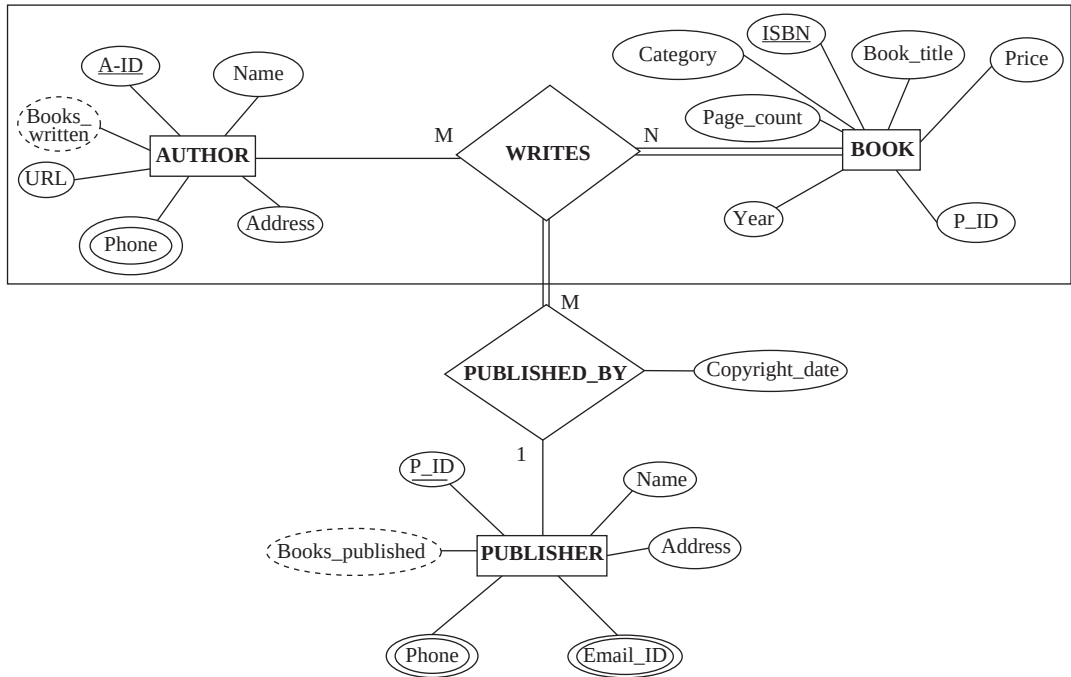


Fig.3.26 EER diagram with aggregation

P_ID	ISBN	A_ID	Copyright_date
P001	001-987-760-9	A001	2004
P001	001-987-760-9	A003	2004
P001	001-354-921-1	A005	2005
P002	002-678-980-4	A002	2005
P002	002-678-980-4	A008	2005
P003	004-765-409-5	A008	2006
P003	004-765-359-3	A009	2006
P004	003-456-433-6	A004	2003
P001	001-987-650-5	A007	2003
P002	002-678-880-2	A006	2001
P004	003-456-533-8	A010	2005

Fig. 3.27 The COPYRIGHT relation

 ● SUMMARY ●

1. A relational database is a collection of relations having distinct names. A relation is used to represent information about an entity and its relationship with other entities in the form of attributes (or columns) and tuples (or rows).
2. The relational model includes three components, namely, structural component (or data structures), manipulative component (or data manipulation), and integrity constraints.
3. Structural component consists of two components, namely, entity types and their relationships, which act as a framework of the given situation.
4. Manipulative component consists of a set of operations that are performed on a table or group of tables.
5. Integrity constraints are a set of rules that governs the entity types and their relationships, and thereby, ensures the integrity of the database.
6. A relation comprises of a relation schema and a relation instance. A relation schema depicts the attributes of the table and a relation instance is a two-dimensional table with a time-varying set of tuples.
7. A relation has certain characteristics. The first is that the tuples in a relation do not have any specified order. The second is that the order in which the values appear in the tuples is relevant. The third is that each tuple in a relation contains a single value for each of its attribute. Finally, each tuple in a relation must be uniquely identified by its contents.
8. The fundamental function of the DBMS is to maintain the integrity of the data. Data integrity ensures that the data in the database is consistent, accurate, correct, and valid.
9. Data integrity is of four types, namely, domain integrity, entity integrity, referential integrity, and semantic integrity.
10. Domain integrity can be specified for each attribute by defining its specific range or domain.
11. Entity integrity assigns restriction on the tuples of a relation and ascertains the accuracy and consistency of the database.
12. Referential integrity ensures that the value of foreign key in the referencing relation must exist in the primary key attribute of the referenced relation, or that the value of the foreign key is null.
13. Semantic integrity ensures that the data in the database is logically consistent and complete with respect to the real world.
14. Integrity constraints are the rules defined on a relational database schema and satisfied by all the instances of a database in order to model the real-world concepts correctly.
15. The primary key constraint ensures that the attributes, which are declared as primary key must be unique and not null.
16. The unique constraint ensures that a particular set of attributes contains unique values and hence, two tuples cannot have duplicate values in specified attributes.
17. The check constraint can be used to restrict an attribute to have values in a given range, that is, for each tuple in a relation, the values in a certain attribute must satisfy a given condition. One way to achieve this is by applying check constraint during the declaration of the relation.
18. The check constraint can also be applied during domain declaration. It allows specifying a condition that must be satisfied by any value assigned to an attribute whose type is the domain.
19. The not null constraint ensures that the attribute must not accept null values.
20. A foreign key constraint allows certain attributes in one relation to refer to attributes in another relation.

● KEY TERMS ●

- | | |
|---|--|
| <ul style="list-style-type: none"> ● Relational model ● Relational database ● Relation ● Relations schema ● Domain ● Unstructured domain ● Structured domain ● Relation instance ● Attribute and Degree ● Tuple and Cardinality ● Unary relation ● Binary relation ● Ternary relation ● Relational databases schema ● Relational database instance ● Superkey ● Candidate key ● Primary key ● Alternate key ● Foreign key | <ul style="list-style-type: none"> ● Referencing relation ● Referenced relation ● Self-referencing foreign key ● Domain integrity ● Entity integrity ● Referential integrity ● Semantic integrity ● Integrity constraints ● Legal and illegal relations ● Table-related constraints ● Column-level constraint ● Table-level constraint ● Domain constraints ● Assertion constraints ● Primary key constraint ● Unique constraint ● Check constraint ● Not null constraint ● Foreign key constraint ● Insert, Delete, and Update operations |
|---|--|

● EXERCISES ●

A. Multiple Choice Questions

1. The relational model is based on the branches of _____.

(a) Science	(b) Engineering
(c) Mathematics	(d) One of the
2. The relational model includes:

(a) Structural components	(b) Manipulative components
(c) Integrity constraints	(d) All of the
3. Which of the following is not a constituent of a relation schema?

(a) Tuples	(b) Set of attributes
(c) Relation name	(d) One or more attributes
4. Candidate key is also known as _____.

(a) Primary key	(b) Irreducible superkey
(c) Superkey	(d) Foreign key

5. In case more than one attribute forms the primary key, the key is known as _____.
 (a) ~~o~~Feign ~~ek~~ (b) Alternate ~~ek~~
 (c) ~~ad~~idate ~~ek~~ (d) ~~om~~posite ~~ek~~
6. Which of the following integrity is applied on the tuple?
 (a) ~~dm~~ain ~~ir~~gity (b) Entity ~~g~~inty
 (c) ~~er~~ferential ~~ir~~gity (d) ~~st~~atic ~~ir~~gity
7. Integrity constraints can be categorized into _____ parts.
 (a) ~~10~~ (b) Two
 (c) Three (d) ~~o~~fr
8. Which of the following can only be defined as column-level constraints?
 (a) ~~o~~N nullbnstraint (b) ~~o~~Feign ~~ek~~ onstraint
 (c) Primary key constraint (d) Check constraint
9. Which of the following is not true with primary key constraint
 (a) Unique key constraint is preferable
 (b) Primary key attribute cannot have *null* value over primary key constraint
 (c) Primary key constraint enforces entity integrity
 (d) Multiple primary key constraints can be defined on a relation
10. While mapping relationship types to a relation, which of the following relationship types allows us to choose primary key of either participating entity type as the primary key of the relation?
 (a) ~~10~~-to-one (b) ~~10~~-to-many
 (c) ~~My~~-to-many (d) All of s~~at~~he

B. Fill in the Blanks

1. The relational model represents both the data and the relationships among data using _____.
2. The number of attributes in a relation is known as _____ or _____.
3. In case, foreign key refers to its own relation, it is known as _____ foreign key.
4. Entity integrity assigns restriction on the _____ of a relation and ascertains the accuracy and consistency of the database.
5. The update operations on the relation are _____, delete, and update (or modify).
6. If deletion violates the _____ integrity, then it is either rejected or _____.
7. The mapping from E-R to relational model results in a _____.
8. Whenever a strong entity type is deleted, the _____ must also be deleted.
9. If an entity type has composite attribute, no _____ is created for the composite attribute itself.
10. The E in EER stands for _____.

C. Answer the Questions

1. Define a relational database, and discuss the various components of a relational model.
2. State the difference between the following terms with the help of suitable examples.

- (a) Relation schema and relation instance
 - (b) Relation database schema and relational database instance
 - (c) Primary key and foreign key
3. Define the following terms:
 - (a) Degree (e) Binary relation
 - (b) ~~relation~~ (f) Ternary ~~relation~~
 - (c) Cardinality (g) Alternate key
 - (d) Unary relation
 4. What are the characteristics of relations?
 5. All candidate keys are superkeys, whereas all superkeys are not candidate keys. Justify this statement with a suitable example.
 6. What do you understand by the term data integrity? What are its four types? Elaborate entity integrity and referential integrity. Mention their differences and discuss the importance of each.
 7. What are integrity constraints? Why are they important? Mention the various categories of integrity constraints with examples. Discuss how you enforce integrity constraints.
 8. What is the difference between primary key constraint and unique constraint? Why is unique constraint preferred over primary key constraint?
 9. Describe check constraint with the help of an example.
 10. What is foreign key constraint? Explain its importance.
 11. How does a not null constraint enforce domain integrity?
 12. Discuss the various update operations that can be performed on a relation.

D. Practical Questions

1. Consider the following update operations performed on the database instance shown in Figure 3.5. Discuss the integrity constraints violated by each operation.
 - (a) Insert<'002-678-999-6', 'Operating System', 'Textbook', 500, 2006, 2007, 700, 'P001'> into BOOK relation.
 - (b) Insert<'P003', 'Express Publications', '100, Second street, Atlanta', 'Georgia', 7134000, 'Express@publications.com'> into PUBLISHER relation.
 - (c) Insert<NULL, 'Richard Martin', 'Washington', 'Seattle', 98012570732, 'www.rmartin.com'> into AUTHOR relation.
 - (d) Insert<'004-765-200-1', 'VC++', 'Textbook', 40, 2007, 2008, 550, 'P010'> into BOOK relation.
 - (e) Insert<'002-876-680-2', 'Sunlight', 'Novel', 25, 2003, 2001, 250, 'P002'> into BOOK relation.
 - (f) Insert<'A001', '001-987-760-9', 8> into REVIEW relation.
 - (g) Delete the tuple with P_ID= "P004" from PUBLISHER relation.
 - (h) Delete the tuple with A_ID= "A003" from AUTHOR relation.
 - (i) Delete the tuple with ISBN= "004-765-409-5" from BOOK relation.
 - (j) Modify the Rating attribute of the REVIEW relation with R_ID= "A001" and ISBN= "002-678-980-4" to "6".
 - (k) Modify the P_ID attribute of the PUBLISHER relation to 'P002'.
2. Consider the relation STUDENT (Enroll#, SemRoll#, Stud_name, Gender, Course, Semester, Address, Phone_no). Specify the candidate keys for this schema and the constraints under which each candidate key would be valid. What other constraints (if any) can be specified on this relation?

3. Consider the following relation schema for a UNIVERSITY database:

PROFESSOR(PID, P_Name, Dept_ID)
COURSE(Code, Dept_ID, C_Name, Syllabus)
PROF_COURSE(PID, Code, Semester)

Identify the primary keys and foreign keys for this schema. Now populate these relations with some tuples, and then give an example of insertion in the PROF_COURSE relation that violates the referential integrity constraints and of another that does not. Identify what you think should be the various candidate keys for this schema. Make any assumption, wherever necessary.

4. Consider the following relation schema for the SALES database:

CUSTOMER(CustNo, Cust_Name, Address)
ORDER(OrderNo, Order_date, Cust_no, Qty, Amount)
PRODUCT(ProdNo, Price, Order_no)

Specify the foreign key constraints for the SALES database. Also insert some tuples in the relations and show some examples of deletion of tuples that violate referential integrity constraints. Make any assumption, wherever necessary.

5. Construct appropriate relation schemas for the E-R diagrams given in Practical questions 1 and 5 in Chapter 02. Make appropriate assumptions, if necessary.
6. Construct appropriate relation schemas for the COMPANY, BANK databases given in Practical questions 2 and 6, respectively, in Chapter 02. Make appropriate assumptions, wherever necessary.

RELATIONAL ALGEBRA AND CALCULUS

After reading this chapter, the reader will understand:

- Different query languages used to extract data from the database
- Difference between relational algebra and relational calculus
- Various unary relational algebra operations like select, project, and rename
- Various binary relational algebra operations like set operations, joins, and division operation
- Various set operations like union, intersection, difference, and cartesian product operation
- Different types of joins like equijoin, natural, and outer joins (left, right, and full outer join)
- The aggregate functions that are used to perform queries, which are mathematical in nature
- Two forms of relational calculus, namely, tuple relational calculus and domain relational calculus
- The use of tuple variables for defining queries in tuple relational calculus
- Transformation of the universal and existential quantifiers
- The concept of safe expressions
- The use of domain variables for defining queries in domain relational calculus
- Expressive power of relational algebra and relational calculus

The two formal languages associated with relational model that are used to specify the basic retrieval requests are *relational algebra* and *relational calculus*. Relational algebra provides a foundation for relational model operations and is used as a basis for applying and optimizing queries in relational database management systems. It consists of basic set of operations, which can be used for carrying out basic retrieval operations. Relational calculus, on the other hand, provides declarative notations based on mathematical logic for specifying relational queries. In relational calculus, query is expressed in terms of variables and formulas on these variables. The formula in relational calculus describes the properties of the required resultant.

Relational algebra and relational calculus are logically equivalent; however, they differ on the basis of the approach they follow to retrieve data from a relation. Relational algebra uses procedural approach by providing a step-by-step procedure for carrying out a query, whereas relational calculus uses non-procedural approach as it describes the information to be retrieved without specifying the method for obtaining that information. DBMS automatically transforms these non-procedural queries into equivalent procedural queries.

This chapter discusses various operations of relational algebra. In addition, it discusses two forms of relational calculus, namely, tuple relational calculus and domain relational calculus. The chapter then concludes with the discussion on the expressive power of relational algebra and relational calculus.

4.1 RELATIONAL ALGEBRA

Relational algebra is a procedural query language that consists of a set of operations that take one or two relations as input and result into a new relation as an output. These operations are divided into two groups. One group consists of operations developed specifically for relational databases such as *select*, *project*, *rename*, *join*, and *division*. The other group includes the set-oriented operations such as *union*, *intersection*, *difference*, and *cartesian product*. Some of the queries, which are mathematical in nature, cannot be expressed using the basic operations of relational algebra. For such queries, additional operations like aggregate functions and grouping are used such as finding sum, average, etc. This section discusses these operations in detail.

Things to Remember

The relational closure property states that the output generated from any type of relational operation is in the form of relations only.

4.1.1 Unary Operations

The operations operating on a single relation are known as **unary operations**. In relational algebra, *select*, *project*, and *rename* are unary operations.

Select Operation

The **select operation** retrieves all those tuples from a relation that satisfy a specific condition. It can also be considered as a filter that displays only those tuples that satisfy a given condition. The select operation can be viewed as the horizontal subset of a relation. The Greek letter sigma (σ) is used as a select operator.

In general, the select operation is specified as

$$\sigma_{\langle \text{selection_condition} \rangle} (R)$$

where,

selection_condition = the condition on the basis of which subset of tuples is selected
R = the name of the relation

For example, to retrieve those tuples from **BOOK** relation whose **Category** is *Novel*, the select operation can be specified as

$$\sigma_{\text{Category}=\text{"Novel"}} (\text{BOOK})$$

The resultant relation of this operation consists of two tuples with **Category** as *Novel* and it is shown in Figure 4.1.

The comparison operators ($=$, $\neq$, $<$, $\leq$, $>$, $\geq$) can be used to specify the conditions for selecting required tuples from a relation. Moreover, more than one condition can be concatenated to one another to give a more specific condition by using any of the logical operators AND ($\wedge$), OR ($\vee$), and NOT ($\neg$). For example, to retrieve those tuples from **BOOK** relation, whose **Category** is *Language Book* and **Page_count** is greater than 400, the select operation can be specified as

$$\sigma_{\text{Category}=\text{"Language Book"} \wedge \text{Page_count} > 400} (\text{BOOK})$$

Consider another example, to retrieve those tuples from **PUBLISHER** relation, whose **State** is either *Georgia* or *Hawaii*, the select operation can be specified as

$$\sigma_{\text{State}=\text{"Georgia"} \vee \text{State}=\text{"Hawaii"}} (\text{PUBLISHER})$$

$\sigma_{\text{Category} = \text{"Novel"}}(\text{BOOK})$

ISBN	Book_title	Category	Price	Copyright_date	Year	Page_count	P_ID
001-354-921-1	Ransack	Novel	22	2005	2006	200	P001
001-987-650-5	Differential Calculus	Textbook	30	2003	2003	450	P001
001-987-760-9	C++	Textbook	40	2004	2005	800	P001
002-678-880-2	Call Away	Novel	22	2001	2002	200	P002
002-678-980-4	DBMS	Textbook	40	2004	2006	800	P002
003-456-433-6	Introduction to German Language	Language Book	22	2003	2004	200	P004
003-456-533-8	Learning French Language	Language Book	32	2005	2006	500	P004
004-765-359-3	Coordinate Geometry	Textbook	35	2006	2006	650	P003
004-765-409-5	UNIX	Textbook	26	2006	2007	550	P003

ISBN	Book_title	Category	Price	Copyright_date	Year	Page_count	P_ID
001-354-921-1	Ransack	Novel	22	2005	2006	200	P001
002-678-880-2	Call Away	Novel	22	2001	2002	200	P002

Fig. 4.1 Select operation



NOTE The comparison operators are used only with the attributes, whose domain is compatible for comparison.

The degree (number of attributes) of the relation resulting from a select operation is same as the degree of relation R. The cardinality (number of tuples) of the resulting relation is always less than or equal to the number of tuples in R.

The sequence of select operations can be applied in any order, as it is commutative in nature, that is,

$$\sigma_{\langle \text{condition}_1 \rangle}(\sigma_{\langle \text{condition}_2 \rangle}R) = \sigma_{\langle \text{condition}_2 \rangle}(\sigma_{\langle \text{condition}_1 \rangle}R)$$

In other words, the order in which conditions are applied is not significant.

Project Operation

A relation might consist of a number of attributes, which are not always required. The **project operation** is used to select some required attributes from a relation while discarding the other attributes. It can be viewed as the vertical subset of a relation. The Greek letter pi (π) is used as project operator.

In general, the project operation is specified as

$$\pi_{\langle \text{attribute_list} \rangle}(R)$$

where,

attribute_list = list of required attributes separated by comma

R = the name of relation

For example, to retrieve only ISBN, Book_title and Price attributes from BOOK relation, the project operation can be specified as

$$\pi_{\text{ISBN, Book_title, Price}}(\text{BOOK})$$

The resultant relation of this operation consists of three attributes of all the tuples as shown in Figure 4.2.

ISBN	Book_title	Category	Price	Copyright_date	Year	Page_count	P_ID
001-354-921-1	Ransack	Novel	22	2005	2006	200	P001
001-987-650-5	Differential Calculus	Textbook	30	2003	2003	450	P001
001-987-760-9	C++	Textbook	40	2004	2005	800	P001
002-678-880-2	Call Away	Novel	22	2001	2002	200	P002
002-678-980-4	DBMS	Textbook	40	2004	2006	800	P002
003-456-433-6	Introduction to German Language	Language Book	22	2003	2004	200	P004
003-456-533-8	Learning French Language	Language Book	32	2005	2006	500	P004
004-765-359-3	Coordinate Geometry	Textbook	35	2006	2006	650	P003
004-765-409-5	UNIX	Textbook	26	2006	2007	550	P003

$\pi_{\text{ISBN, Book_title, Price}}(\text{BOOK})$

ISBN	Book_title	Price
001-354-921-1	Ransack	22
001-987-650-5	Differential Calculus	30
001-987-760-9	C++	40
002-678-880-2	Call Away	22
002-678-980-4	DBMS	40
003-456-433-6	Introduction to German Language	22
003-456-533-8	Learning French Language	32
004-765-359-3	Coordinate Geometry	35
004-765-409-5	UNIX	26

Fig. 4.2 Project operation

Consider another example to retrieve P_ID, State, and Phone attributes from PUBLISHER relation, the project operation can be specified as

$\pi_{\text{P_ID, State, Phone}}(\text{PUBLISHER})$

The resultant relation of project operation consists of only those attributes, which are specified in **<attribute_list>** and are in the same order as they are appearing in the list. Hence, the degree of the resultant relation is equal to the number of attributes in **<attribute_list>**.

If **<attribute_list>** contains only non-key attributes of relation R, duplicate tuples are likely to appear in the resultant relation. The project operation removes duplicate tuples from the resultant relation. Therefore, the result of the project operation is a valid relation containing a set of unique tuples. This feature of project operation is known as **duplicate elimination**. For example, consider the project operation given here.

$\pi_{\text{Category, Page_count}}(\text{BOOK})$

The resultant relation of this operation consists of only two attributes, namely, **Category** and **Page_count**. Since the project operation removes duplicate tuples from the resultant relation, the

tuples (*Textbook*, 800) and (*Novel*, 200) appear only once in the resultant relation, although the combination of these values appear repeatedly in *BOOK* relation as shown in Figure 4.3.

ISBN	Book_title	Category	Price	Copyright_date	Year	Page_count	P_ID
001-354-921-1	Ransack	Novel	22	2005	2006	200	P001
001-987-650-5	Differential Calculus	Textbook	30	2003	2003	450	P001
001-987-760-9	C++	Textbook	40	2004	2005	800	P001
002-678-880-2	Call Away	Novel	22	2001	2002	200	P002
002-678-980-4	DBMS	Textbook	40	2004	2006	800	P002
003-456-433-6	Introduction to German Language	Language Book	22	2003	2004	200	P004
003-456-533-8	Learning French Language	Language Book	32	2005	2006	500	P004
004-765-359-3	Coordinate Geometry	Textbook	35	2006	2006	650	P003
004-765-409-5	UNIX	Textbook	26	2006	2007	550	P003

$\pi_{\text{Category, Page_count}}(\text{BOOK})$

Category	Page_count
Textbook	450
Novel	200
Textbook	800
Novel	200
Textbook	800
Language Book	200
Language Book	500
Textbook	650
Textbook	550

Fig. 4.3 Duplicate elimination in project operation

Unlike select operation, the commutative property does not hold on project operation, that is,

$$\pi_{\langle \text{list}_1 \rangle}(\pi_{\langle \text{list}_2 \rangle}(\mathbf{R})) \neq \pi_{\langle \text{list}_2 \rangle}(\pi_{\langle \text{list}_1 \rangle}(\mathbf{R}))$$

In other words, the order in which the list of attributes is specified in project operation is significant.

Note that, the select and project operations can also be combined together to create more complex queries. For example, to retrieve *P_ID*, *Address*, and *Phone* attributes from *PUBLISHER* relation where *State* is *Georgia*, the combined operation can be specified as

$$\pi_{\text{P_ID, Address, Phone}}(\sigma_{\text{State}=\text{"Georgia"}}(\text{PUBLISHER}))$$

This statement first applies select operation on *PUBLISHER* relation and then applies project operation on the tuples retrieved from select operation.

Rename Operation

When relational algebra operations are performed, the resultant relations are unnamed, hence cannot be used for later reference. In addition, it might be required to apply more operations on the relation

obtained from other operations. In such situations, rename operator proves to be useful. **Rename operation** is used to provide name to the relation obtained after applying any relational algebra operation. The Greek letter rho (ρ) is used as a rename operator. Relation obtained from any operation can be renamed by using rename operator as shown here.

$$\rho(R, E)$$

where,

ρ = rename operator

E = expression representing relational algebra operations

R = name given to relation obtained by applying relational algebra operations specified in expression E

Consider some examples of using rename operator to rename queries discussed earlier.

$$\rho(R_1, \sigma_{\text{Category}=\text{"Novel"}}(\text{BOOK}))$$

$$\rho(R_2, \pi_{P_ID, State, Phone}(\text{PUBLISHER}))$$

$$\rho(R_3, \pi_{P_ID, Address, Phone}(\sigma_{\text{State}=\text{"Georgia"}}(\text{PUBLISHER})))$$

Here, R_1 , R_2 , and R_3 are the names given to the relations obtained from the respective relational algebra expressions. In these examples, the name of the attributes in new relation is same as in their corresponding original relations. However, attributes in a new relation can also be renamed using the rename operator as shown here.

$$\rho(R(A_1, A_2, \dots, A_n), E)$$

where,

$A_1, A_2, \dots, A_n$ are the new names provided to the attributes of the relation R, obtained after executing relational algebra expression E. For example, consider an expression given here.

$$\rho(R_2(P_1, P_2, P_3), \pi_{P_ID, State, Phone}(\text{PUBLISHER}))$$

In this example, P_1, P_2, P_3 are new attribute names in the relation R_2 for the attributes P_ID , $State$, and $Phone$ of the relation PUBLISHER , respectively.

Sometimes the situation may arise where multiple relational algebra operations need to be applied in a sequence. One way to achieve this is by grouping together the sequence of operations to form a single relational algebra expression. The second way is to apply one operation at a time to create many intermediate result relations. In such situations, rename operator can be used to provide name to the intermediate relations so that they can be referred easily in the subsequent operations.

For example, consider a query to retrieve P_ID , $Address$, and $Phone$ of publishers located at *Georgia* state. This query can be expressed in a single relational algebra expression as discussed earlier or can be expressed in two steps with the help of rename operator as shown here.

$$\rho(R_1, \sigma_{\text{State}=\text{"Georgia"}}(\text{PUBLISHER}))$$

$$\rho(R_2, \pi_{P_ID, Address, Phone}(R_1))$$

In this example, select operation is applied to retrieve tuples where $State$ is *Georgia* and resultant relation is named as R_1 . Then, the project operation is applied to retrieve selected attributes,

namely, P_ID, Address, and Phone from the intermediate relation R_1 and the resultant relation is named as R_2 . The relation R_2 can further be used for more operations.



NOTE The rename operator is optional, it is not always necessary to use it. It can be used as per one's convenience.

4.1.2 Binary Operations

The operations operating on two relations are known as **binary operations**. The set operations, join operations, and division operation are all binary operations.

Set Operations

The standard mathematical operations on sets, namely, *union*, *intersection*, *difference*, and *cartesian product* are also available in relational algebra. Of these, the three operations, namely, union, intersection, and difference operations require two relations to be union compatible with each other. The two relations are said to be union compatible, if they satisfy these conditions.

1. The degree of both the operand relations must be same.
2. The domain of n th attribute of relation R_1 must be same as that of the n th attribute of relation R_2 .

However, the cartesian product can be defined on any two relations, that is, they need not be union compatible.

Union Operation The **union operation**, denoted by $\cup$, returns a third relation that contains tuples from both or either of the operand relations. Consider two relations R_1 and R_2 , their union is denoted as $R_1 \cup R_2$, which returns a relation containing tuples from both or either of the given relations. The duplicate tuples are automatically removed from the resultant relation.

For example, consider the two union compatible relations PUBLISHER_1 and PUBLISHER_2. The union of these two relations consists of tuples from both relations without any duplicate tuples as shown in Figure 4.4.

PUBLISHER_1		PUBLISHER_2	
P_ID	Pname	P_ID	Pname
P001	Hills Publications	P001	Hills Publications
P002	Sunshine Publishers Ltd.	P002	Sunshine Publishers Ltd.
P003	Bright Publications	P006	ESL Publications
P004	Paramount Publishing House	P007	Arizon Publishers Ltd.
P005	Wesley Publications		

PUBLISHER_1 U PUBLISHER_2	
P_ID	Pname
P001	Hills Publications
P002	Sunshine Publishers Ltd.
P003	Bright Publications
P004	Paramount Publishing House
P005	Wesley Publications
P006	ESL Publications
P007	Arizon Publishers Ltd.

Fig. 4.4 Union operation

The union operation is, respectively, commutative and associative in nature, that is,

$$R_1 \cup R_2 = R_2 \cup R_1$$

and

$$R_1 \cup (R_2 \cup R_3) = (R_1 \cup R_2) \cup R_3$$

Intersection Operation The **intersection operation**, denoted by $\cap$, returns a third relation that contains tuples common to both the operand relations. The intersection of the relations R_1 and R_2 is denoted by $R_1 \cap R_2$, which returns a relation containing tuples common to both the given relations.

For example, consider two union compatible relations PUBLISHER_1 and PUBLISHER_2. The intersection of these two relations consists of tuples common to both the relations as shown in Figure 4.5.

PUBLISHER_1		PUBLISHER_2	
P_ID	Pname	P_ID	Pname
P001	Hills Publications	P001	Hills Publications
P002	Sunshine Publishers Ltd.	P002	Sunshine Publishers Ltd.
P003	Bright Publications	P006	ESL Publications
P004	Paramount Publishing House	P007	Arizon Publishers Ltd.
P005	Wesley Publications		

PUBLISHER_1 $\cap$ PUBLISHER_2	
P_ID	Pname
P001	Hills Publications
P002	Sunshine Publishers Ltd.

Fig. 4.5 Intersection operation

Like, union operation, intersection operation is, respectively, commutative and associative in nature, that is,

$$R_1 \cap R_2 = R_2 \cap R_1$$

and

$$R_1 \cap (R_2 \cap R_3) = (R_1 \cap R_2) \cap R_3$$

Learn More

Both the union and intersection operations can be performed on any number of relations, as they are associative in nature. Thus, they can be treated as n -ary operations.

Difference Operation The **difference operation**, denoted by $-$ (minus), returns a third relation that contains all tuples present in one relation, which are not present in the second relation. The difference of the relations R_1 and R_2 is denoted by $R_1 - R_2$. The expression $R_1 - R_2$ results in a relation containing all tuples from relation R_1 , which are not present in relation R_2 . Similarly, the expression $R_2 - R_1$ results in a relation containing all tuples from relation R_2 , which are not present in relation R_1 .

For example, consider the two union compatible relations PUBLISHER_1 and PUBLISHER_2, the difference of these two relations is shown in Figure 4.6.

The difference operation is not commutative in nature, that is,

$$R_1 - R_2 \neq R_2 - R_1$$

Note that any relational algebra expression that uses intersection operation can be rewritten by replacing the intersection operation with a pair of difference operation as shown here.

$$R_1 \cap R_2 = R_1 - (R_1 - R_2)$$

PUBLISHER_1		PUBLISHER_2	
P_ID	Pname	P_ID	Pname
P001	Hills Publications	P001	Hills Publications
P002	Sunshine Publishers Ltd.	P002	Sunshine Publishers Ltd.
P003	Bright Publications	P006	ESL Publications
P004	Paramount Publishing House	P007	Arizon Publishers Ltd.
P005	Wesley Publications		

PUBLISHER_1-PUBLISHER_2		PUBLISHER_2-PUBLISHER_1	
P_ID	Pname	P_ID	Pname
P003	Bright Publications	P006	ESL Publications
P004	Paramount Publishing House	P007	Arizon Publishers Ltd.
P005	Wesley Publications		

Fig. 4.6 Difference operation

Thus, intersection operation is not a fundamental operation. It is easier to write $R_1 \cap R_2$ than to write $R_1 - (R_1 - R_2)$. In addition, the intersection operation can also be expressed in terms of difference and union operations as shown here.

$$R_1 \cap R_2 = (R_1 \cup R_2) - ((R_1 - R_2) \cup (R_2 - R_1))$$

Cartesian Product Operation The **cartesian product**, also known as **cross product** or **cross join**, returns a third relation that contains all possible combinations of the tuples from the two operand relations. The cartesian product is denoted by the symbol $\times$. The cartesian product of the relations R_1 and R_2 is denoted by $R_1 \times R_2$.

Each tuple of the first relation is concatenated with all the tuples of the second relation to form the tuples of a new relation. The cartesian product creates tuples with the combined attributes of two relations. Therefore, the degree of a new relation is the sum of the degree of both the operand relations. In addition, the cardinality of the resultant relation is the product of the cardinality of both the operand relations. In other words, if d_1 and d_2 are the degree of the relations R_1 and R_2 , respectively, then the degree of the relation $R_1 \times R_2$ is $d_1 + d_2$, and if c_1 and c_2 are the cardinality of relations R_1 and R_2 , respectively, then the cardinality of the relation $R_1 \times R_2$ is $c_1 * c_2$.

For example, consider two relations **AUTHOR_1** with attributes **A_ID**, **Aname**, and **City**, and **BOOK_1** with attributes **ISBN**, **Book_title**, and **Price**; the cartesian product of these two relations is shown in Figure 4.7.

In this example, each tuple from relation **AUTHOR_1** is concatenated with every tuple of relation **BOOK_1**. The degree of both the operand relations is 3 and hence, the degree of the resultant relation is $6(3+3)$. The cardinality of the resultant relation is 12, which is the product of the cardinality of the relation **AUTHOR_1**, that is, 4 and cardinality of the relation **BOOK_1**, that is, 3.



NOTE The cartesian product can be made more useful and meaningful, if it is followed by select and project operations to retrieve meaningful and valuable information from the database.

Joins

The **join** operation is one of the most useful and commonly used operations to extract information from two or more relations. A join can be defined as a cartesian product followed by select and project operations. The result of a cartesian product is usually much larger than the result of a join.

AUTHOR_1			BOOK_1		
A_ID	Aname	City	ISBN	Book_title	Price
A001	Thomas William	New York	001-987-760-9	C++	40
A002	James Erin	Atlanta	001-354-921-1	Ransack	22
A003	Charles Smith	Los Angeles	002-678-980-4	DBMS	40
A004	Lewis Ian	Seattle			

AUTHOR_1 × BOOK_1					
A_ID	Aname	City	ISBN	Book_title	Price
A001	Thomas William	New York	001-987-760-9	C++	40
A001	Thomas William	New York	001-354-921-1	Ransack	22
A001	Thomas William	New York	002-678-980-4	DBMS	40
A002	James Erin	Atlanta	001-987-760-9	C++	40
A002	James Erin	Atlanta	001-354-921-1	Ransack	22
A002	James Erin	Atlanta	002-678-980-4	DBMS	40
A003	Charles Smith	Los Angeles	001-987-760-9	C++	40
A003	Charles Smith	Los Angeles	001-354-921-1	Ransack	22
A003	Charles Smith	Los Angeles	002-678-980-4	DBMS	40
A004	Lewis Ian	Seattle	001-987-760-9	C++	40
A004	Lewis Ian	Seattle	001-354-921-1	Ransack	22
A004	Lewis Ian	Seattle	002-678-980-4	DBMS	40

Fig. 4.7 Cartesian product

The join operation joins two relations to form a new relation on the basis of one common attribute present in the two operand relations. That is, if both the operand relations have one common attribute defined over some common domain, then they can be joined over this common attribute. The join results in a new wider relation in which each tuple is formed by concatenating the two tuples, one from each of the operand relations, such that two tuples have the same value for that common attribute.

The join is denoted by the symbol $\bowtie$. The most general form of the join operation is based on a condition and a pair of operand relations.

That is, the join operation is specified as

$$R_1 \bowtie_{\text{cond}} R_2 = \sigma_{\text{cond}}(R_1 \times R_2)$$

Note that the condition *cond* refers to the join condition involving attributes of both relations R_1 and R_2 . There are different types of join operations in relational algebra, namely, *equijoin*, *natural join*, *outer join*, etc.

Equijoin A common case of the join operation $R_1 \bowtie R_2$ is one in which the join condition consists only of equality condition. This type of join where the only comparison operator used is, = is known as **equijoin**. The resultant relation of an equijoin always has one or more pairs of attributes that have identical values in every tuple.

For example, the relations **BOOK** and **PUBLISHER** can be joined for the tuples where **BOOK.P_ID** is same as **PUBLISHER.P_ID**. This is an example of equijoin operation and it is specified as shown here.

BOOK ⋈ **PUBLISHER**
 $\text{BOOK.P_ID} = \text{PUBLISHER.P_ID}$

The resultant relation of such a join is shown in Figure 4.8.

ISBN	Book_title	...	BOOK.P_ID	PUBLISHER.P_ID	Pname	...	Email_id
001-987-760-9	C++	...	P001	P001	Hills Publications	...	h_pub@hills.com
001-354-921-1	Ransack	...	P001	P001	Hills Publications	...	h_pub@hills.com
001-987-650-5	Differential Calculus	...	P001	P001	Hills Publications	...	h_pub@hills.com
002-678-980-4	DBMS	...	P002	P002	Sunshine Publishers Ltd.	...	null
002-678-880-2	Call Away	...	P002	P002	Sunshine Publishers Ltd.	...	null
004-765-409-5	UNIX	...	P003	P003	Bright Publications	...	bright@bp.com
004-765-359-3	Coordinate Geometry	...	P003	P003	Bright Publications	...	bright@bp.com
003-456-433-6	Introduction to German Language	...	P004	P004	Paramount Publishing House	...	param_house@param.com
003-456-533-8	Learning French Language	...	P004	P004	Paramount Publishing House	...	param_house@param.com

Fig. 4.8 *Equijoin operation*

In the resultant relation of equijoin operation, the values of the attributes **PUBLISHER.P_ID** and **BOOK.P_ID** are same for every tuple as equality condition is specified on these attributes.



The attribute publisher ID is appearing with the same name **P_ID** in both the relations, thus, in order to differentiate these two, the attribute names are qualified by their corresponding relation name using the dot (.) operator.

The select operation can be applied on the resultant relation to retrieve only selected tuples. For example, if only those tuples are to be displayed where the **Category** of book is *Textbook* and it is published by publishers located in *Georgia*, then the join operation along with the select operation can be specified as

$\sigma_{\text{Category}=\text{"Textbook"} \wedge \text{State}=\text{"Georgia"}}(\text{BOOK} \bowtie_{\text{BOOK.P_ID} = \text{PUBLISHER.P_ID}} \text{PUBLISHER})$

The result of this expression consists of a set of tuples from the resultant relation of join operation satisfying the conditions specified.

Natural Join The joins having equality condition as their join condition result in two attributes in the resulting relation having exactly the same value. However, if one of the two identical attributes is removed from the result of equijoin, it is known as **natural join**.

The natural join of two relations R_1 and R_2 is obtained by applying a project operation to the equijoin of these two relations in a sequence given here.

1. Find the equijoin of two relations R_1 and R_2 .
2. For each attribute A that is common to both relations R_1 and R_2 , project operation is applied to remove the column $R_1.A$ or $R_2.A$. Therefore, if there are m attributes common in both the relations, then m duplicate columns are removed from the resultant relation of equijoin.

The expression to obtain natural join of relations **BOOK** and **PUBLISHER** can be specified as

$\Pi_{\text{ISBN, Book_title, Category, BOOK.P_ID, Pname, Address, Email_ID}} (\text{BOOK} \bowtie_{\text{BOOK.P_ID = PUBLISHER.P_ID}} \text{PUBLISHER})$

The resultant relation of such a natural join is shown in Figure 4.9.

Learn More

Another join similar to natural join is semi-join represented as $R_1 \ltimes R_2$, is a set of all tuples from a relation R_1 having corresponding tuple in the relation R_2 , satisfying the join condition (equality condition). In addition, antijoin represented as $R_1 \not\bowtie R_2$, is a set of tuples from a relation R_1 not having corresponding tuple in the relation R_2 , satisfying the join condition.

ISBN	Book_title	Category	BOOK.P_ID	Pname	Address	Email_id
001-987-760-9	C++	Textbook	P001	Hills Publications	12, Park street, Atlanta	h_pub@hills.com
001-354-921-1	Ransack	Novel	P001	Hills Publications	12, Park street, Atlanta	h_pub@hills.com
002-678-980-4	DBMS	Textbook	P002	Sunshine Publishers Ltd.	45, Second street, Newark	null
004-765-409-5	UNIX	Textbook	P003	Bright Publications	123, Main street, Honolulu	bright@bp.com
004-765-359-3	Coordinate Geometry	Textbook	P003	Bright Publications	123, Main street, Honolulu	bright@bp.com
003-456-433-6	Introduction to German Language	Language Book	P004	Paramount Publishing House	789, Oak street, New York	param_house@param.com
001-987-650-5	Differential Calculus	Textbook	P001	Hills Publishers Ltd.	12, Park street, Atlanta	h_pub@hills.com
002-678-880-2	Call Away	Novel	P002	Sunshine Publishers Ltd.	45, Second street, Newark	null
003-456-533-8	Learning French Language	Language Book	P004	Paramount Publishing House	789, Oak street, New York	param_house@param.com

Fig. 4.9 Natural join operation

Note that, the attribute **P_ID** appears only once in the resultant relation of natural join. The pre-requisite of natural join is that the two attributes involved in the join condition must have the same name. If they have different names then the renaming operation is applied prior to the join operation.

Outer join The joins discussed so far are simple and are known as **inner joins**. An inner join selects only those tuples from both the joining relations that satisfy the joining condition. The **outer join**, on

the other hand, selects all the tuples satisfying the join condition along with the tuples for which no tuples from the other relation satisfy the join condition.

An outer join is a type of equijoin that can be used to display all the tuples from one relation, even if there are no corresponding tuples in the second relation, thus preventing information loss. It is commonly used with relations having one-to-many relationship. The three types of outer joins are

- ⇒ **Left outer join:** The **left outer join** includes all the tuples from both the relations satisfying the join condition along with all the tuples in the left relation that do not have a corresponding tuple in the right relation. The tuples from the left relation not satisfying the join condition are concatenated with the tuples having null values for all the attributes from the right relation. The left outer join of relations R_1 and R_2 is specified as $R_1 \bowtie_{\text{cond}} R_2$.
- ⇒ **Right outer join:** The **right outer join** includes all the tuples from both the relations satisfying the join condition along with all the tuples in the right relation that do not have a corresponding tuple in the left relation. The tuples from the right relation not satisfying the join condition are concatenated with the tuples having null values for all the attributes from the left relation. The right outer join of relations R_1 and R_2 is specified as $R_1 \ltimes_{\text{cond}} R_2$.
- ⇒ **Full outer join:** A **full outer join** combines the results of both the left and the right outer joins. The resultant relation contains all records from both the relations, with null values for the missing corresponding tuples on either side. The full outer join of relations R_1 and R_2 is specified as $R_1 \Join_{\text{cond}} R_2$.

Division

The **division** operation, denoted by $\div$ is useful for queries of the form ‘for all objects having all the specified properties’. To understand the division operation, consider a relation R_1 having exactly two attributes A and B, and a relation R_2 having just one attribute B with the same domain as in R_1 . The division operation $R_1 \div R_2$ is defined as the set of all values of attribute A, such that for every value of attribute B in R_2 , there is a tuple (A, B) in R_1 . In general, for the relations R_1 and R_2 , $R_1 \div R_2$ results in the relation R_3 such that $R_3 \times R_2 \subseteq R_1$.

For example, consider the relations R_1 and R_2 with the sample data as shown in Figure 4.10.

R_1	R_2	$R_1 \div R_2$	R_3	$R_1 \div R_3$	R_4	$R_1 \div R_4$																																							
<table> <tr><th>A</th><th>B</th></tr> <tr><td>a₁</td><td>b₁</td></tr> <tr><td>a₁</td><td>b₂</td></tr> <tr><td>a₁</td><td>b₃</td></tr> <tr><td>a₁</td><td>b₄</td></tr> <tr><td>a₂</td><td>b₁</td></tr> <tr><td>a₂</td><td>b₂</td></tr> <tr><td>a₃</td><td>b₂</td></tr> <tr><td>a₄</td><td>b₂</td></tr> <tr><td>a₄</td><td>b₄</td></tr> </table>	A	B	a ₁	b ₁	a ₁	b ₂	a ₁	b ₃	a ₁	b ₄	a ₂	b ₁	a ₂	b ₂	a ₃	b ₂	a ₄	b ₂	a ₄	b ₄	<table> <tr><th>B</th></tr> <tr><td>b₂</td></tr> </table>	B	b ₂	<table> <tr><th>A</th></tr> <tr><td>a₁</td></tr> <tr><td>a₂</td></tr> <tr><td>a₃</td></tr> <tr><td>a₄</td></tr> </table>	A	a ₁	a ₂	a ₃	a ₄	<table> <tr><th>B</th></tr> <tr><td>b₂</td></tr> <tr><td>b₄</td></tr> </table>	B	b ₂	b ₄	<table> <tr><th>A</th></tr> <tr><td>a₁</td></tr> <tr><td>a₄</td></tr> </table>	A	a ₁	a ₄	<table> <tr><th>B</th></tr> <tr><td>b₁</td></tr> <tr><td>b₂</td></tr> <tr><td>b₃</td></tr> </table>	B	b ₁	b ₂	b ₃	<table> <tr><th>A</th></tr> <tr><td>a₁</td></tr> </table>	A	a ₁
A	B																																												
a ₁	b ₁																																												
a ₁	b ₂																																												
a ₁	b ₃																																												
a ₁	b ₄																																												
a ₂	b ₁																																												
a ₂	b ₂																																												
a ₃	b ₂																																												
a ₄	b ₂																																												
a ₄	b ₄																																												
B																																													
b ₂																																													
A																																													
a ₁																																													
a ₂																																													
a ₃																																													
a ₄																																													
B																																													
b ₂																																													
b ₄																																													
A																																													
a ₁																																													
a ₄																																													
B																																													
b ₁																																													
b ₂																																													
b ₃																																													
A																																													
a ₁																																													
		(a) $R_1 \div R_2$	(b) $R_1 \div R_3$		(c) $R_1 \div R_4$																																								

Fig. 4.10 Division operation

In this example, consider the case of division operation on the relations R_1 and R_2 . The relation $R_1 \div R_2$ is the set of values that forms a tuple with every value in the relation R_2 , appearing in a relation R_1 . To elaborate further, the tuples (a_1, b_2) , (a_2, b_2) , (a_3, b_2) and (a_4, b_2) are appearing in the relation R_1 where, b_2 is a member of the relation R_2 and (a_1, a_2, a_3, a_4) are members of the relation $R_1 \div R_2$. Similarly, the results of operations $R_1 \div R_3$ and $R_1 \div R_4$ are shown in Figure 4.10 (b) and 4.10 (c), respectively.

Now, take another example of *Online Book* database, ‘retrieve the list of all authors writing books for publishers located in *New Jersey* and *Hawaii*’. To achieve this, first the join operation is applied to create a relation $R_1(P_ID, A_ID)$, which contains the list of publisher ID for whom authors have written the books. Now create a relation $R_2(P_ID)$ having the list of publisher ID belonging to *New Jersey* or *Hawaii* state. The relations R_1 and R_2 can be obtained using these algebraic expressions.

$$\rho(R_1, \Pi_{P_ID, A_ID}(\text{BOOK} \bowtie_{\text{BOOK.ISBN}=\text{AUTHOR_BOOK.ISBN}} \text{AUTHOR_BOOK}))$$

$$\rho(R_2, \Pi_{P_ID}(\sigma_{\text{State}=\text{"New Jersey"} \text{ OR } \text{State}=\text{"Hawaii"}}(\text{PUBLISHER})))$$

The resultant relation of division operation $R_1 \div R_2$ is shown in Figure 4.11.

In this example, the relation $R_1 \div R_2$ is the set of values that forms a tuple with every value in relation R_2 , which also appears in relation R_1 . To elaborate further, the tuples (*P002*, *A008*) and (*P003*, *A008*) appear in the relation R_1 where *P002* and *P003* are from relation R_2 .

4.1.3 Aggregate Functions and Grouping

The queries discussed so far are sufficient enough to satisfy most of the requirements of an organization. However, some of the requirements like finding total amount, average price, which are mathematical in nature cannot be expressed by basic relational algebra operations. For such type of queries, aggregate functions are used. Aggregate functions take a collection of values as input, process them, and return a single value as the result. For example, the aggregate function **SUM** returns a sum of the collection of numeric values taken as input. Similarly, aggregate functions, namely, **AVG**, **MAX**, and **MIN** are used to find average, maximum, and minimum of the collection of numeric values, respectively. The **COUNT** function is used for counting tuples or values in a relation.

Sometimes, situation may arise where tuples in a relation are required to be grouped, based on the values of an attribute. For example, the tuples of relation **BOOK** can be divided into groups based on the values of attribute **Category**. In other words, the books belonging to textbook category form one group, books belonging to novel category form another group, and so on.

The aggregate functions can be applied on each group individually, thus, returning a result value for each group separately. For example, for calculating average price for each category of book, **AVG** function is applied on each category of book separately. In other words, if there are three categories of books, then **AVG** function is applied to each of the three categories separately, which returns three average values for each category. The grouping of tuples of a relation can be based on one or more attributes. For example, tuples in relation **BOOK** can be grouped on two attributes, that is, first on the basis of category and then on the basis of publisher ID.

The aggregate function operation can be specified using the symbol $\mathfrak{F}$ (called ‘script F’) as shown here.

$$\langle \text{attribute_list} \rangle \mathfrak{F}_{\langle \text{function(attribute_list)} \rangle} (R)$$

where,

$\langle \text{attribute_list} \rangle$ = list of attributes on the basis of which tuples of a relation are to be grouped

$\langle \text{function(attribute_list)} \rangle$ = list of functions to be applied on different attributes.

R ₁		R ₂	R ₁ ÷ R ₂
P_ID	A_ID	P_ID	A_ID
P001	A001	P002	A008
P002	A002		
P001	A003		
P004	A004	P003	A008
P001	A005		
P002	A006		
P001	A007		
P002	A008		
P003	A008		
P003	A009		
P004	A010		

Fig. 4.11 Example of division operation

For example, consider a relation **BOOK**, where average, maximum, and minimum price is to be calculated for each category of book separately. The expression to represent this query can be specified as

$$\text{Category} \mathcal{F}_{\text{AVG}(\text{Price}), \text{MAX}(\text{Price}), \text{MIN}(\text{Price})}(\text{BOOK})$$

The list of attributes on the basis of which tuples are grouped can be omitted from an expression. If grouping attribute is not specified in the expression, then the function is applied individually on all the tuples in the relation. Consider an example where the number of tuples in a relation has to be calculated. The required expression can be specified as

$$\mathcal{F}_{\text{COUNT}(\text{ISBN})}(\text{BOOK})$$

Whenever a function is applied on an attribute, the duplicate values are also included while performing the calculations. However, concatenating the function name with the string **_DISTINCT** can eliminate duplicate values. For example, consider a relation **BOOK** where the number of categories is to be calculated. If **COUNT** function is applied on attribute **Category**, it counts all the values including duplicate values. However, if **COUNT_DISTINCT** function is applied, it counts only the unique values.

4.1.4 Example Queries in Relational Algebra

Various operations of relational algebra can be combined together to create more advanced queries to extract required data from the database. Some of the queries are discussed here to extract information from *Online Book* database using operations of relational algebra.

Query1 : Retrieve city, phone and URL of the author whose name is *Lewis Ian* .

$$\Pi_{\text{City, Phone, URL}}(\sigma_{\text{Aname}=\text{"Lewis Ian"}}(\text{AUTHOR}))$$

In this query, the select operation is used to retrieve all the tuples where **Aname** is *Lewis Ian*, and then the project operation is used to display the required attributes, namely, **City**, **Phone** and **URL** for the tuples retrieved.

Query2 : Retrieve name, address, and phone of all the publishers located in *New York* state.

$$\Pi_{\text{Pname, Address, Phone}}(\sigma_{\text{State}=\text{"New York"}}(\text{PUBLISHER}))$$

In this query, the select operation is used to retrieve all the tuples where **State** is *New York*, and then the project operation is used to display the required attributes, namely, **Pname**, **Address**, and **Phone** for the tuples retrieved.

Query3 : Retrieve title and price of all the textbooks with page count greater than 600.

$$\Pi_{\text{Book_title, Price}}(\sigma_{\text{Category}=\text{"Textbook"} \wedge \text{Page_count} > 600}(\text{BOOK}))$$

In this query, the select operation is used to retrieve all the tuples where **Category** is *Textbook* and **Page_count** is greater than 600 and then the project operation is used to display the required attributes, namely, **Book_title** and **Price** for the tuples retrieved.

Query4 : Retrieve ISBN, title, and price of the books belonging to either novel or language book category.

$$\Pi_{\text{ISBN, Book_title, Price}}(\sigma_{\text{Category}=\text{"Novel"} \vee \text{Category}=\text{"Language Book"}}(\text{BOOK}))$$

In this query, the select operation is used to retrieve all the tuples where *Category* is either *Textbook* or *Novel* and then the project operation is used to display the required attributes, namely, *ISBN*, *Book_title*, and *Price* for the tuples retrieved.

Query5 : Retrieve ID, name, address, and phone of publishers publishing novels.

$$\Pi_{P_ID, Pname, Address, Phone} (\sigma_{Category="Novel"} (BOOK \bowtie_{BOOK.P_ID=PUBLISHER.P_ID} PUBLISHER))$$

In this query, the join of relations *BOOK* and *PUBLISHER* is created on the basis of common attribute *P_ID*. After creating a join, the select operation is applied on the resultant relation to retrieve all the tuples where *Category* is *Novel*, and then the project operation is used to display the required attributes, namely, *P_ID*, *Pname*, *Address*, and *Phone* for the tuples retrieved.

Query6 : Retrieve title and price of all the books published by *Hills Publications*.

$$\Pi_{Book_title, Price} (\sigma_{Pname="Hills Publications"} (BOOK \bowtie_{BOOK.P_ID=PUBLISHER.P_ID} PUBLISHER))$$

In this query, the join of relations *BOOK* and *PUBLISHER* is created on the basis of common attribute *P_ID*. After creating a join, the select operation is applied on the resultant relation to retrieve all the tuples where *Pname* is *Hills Publications*, and then the project operation is used to display the required attributes, namely, *Book_title* and *Price* for the tuples retrieved.

Query7 : Retrieve book title, reviewers ID, and rating of all the textbooks.

$$\Pi_{Book_title, R_ID, Rating} (\sigma_{Category="Textbook"} (BOOK \bowtie_{BOOK.ISBN=REVIEW.ISBN} REVIEW))$$

In this query, the join of relations *BOOK* and *REVIEW* is created on the basis of common attribute *ISBN*. After creating a join, the select operation is applied on the resultant relation to retrieve all the tuples where *Category* is *Textbook*, and then the project operation is used to display the required attributes, namely, *Book_title*, *R_ID*, and *Rating* for the tuples retrieved.

Query8 : Retrieve title, category, and price of all the books written by *Charles Smith*.

$$\Pi_{Book_title, Category, Price} (\sigma_{Aname="Charles Smith"} ((BOOK \bowtie_{BOOK.ISBN=AUTOR_BOOK.ISBN} AUTOR_BOOK) \bowtie_{AUTOR_BOOK.A_ID=AUTOR.A_ID} AUTOR))$$

In this query, the join of relations *BOOK* and *AUTOR_BOOK* is created on the basis of common attribute *ISBN* and the resultant relation is joined with the relation *AUTOR* on the basis of common attribute *A_ID*. After creating a join, the select operation is applied on the resultant relation to retrieve all the tuples where *Aname* is *Charles Smith*, and then the project operation is used to display the required attributes, namely, *Book_title*, *Category* and *Price* for the tuples retrieved.

Query9 : Retrieve ID, name, URL of author, and category of the book *C++*.

$$\Pi_{A_ID, Aname, URL, Category} (\sigma_{Book_title="C++"} (((AUTOR \bowtie_{AUTOR.A_ID=AUTOR_BOOK.A_ID} AUTOR_BOOK) \bowtie_{AUTOR_BOOK.ISBN=BOOK.ISBN} BOOK)))$$

In this query, the join of relations **AUTHOR** and **AUTHOR_BOOK** is created on the basis of common attribute **A_ID** and the resultant relation is joined with the relation **BOOK** on the basis of common attribute **ISBN**. After creating a join, the select operation is applied on the resultant relation to retrieve all the tuples where **Book_title** is *C++*, and then the project operation is used to display the required attributes, namely, **A_ID**, **Aname**, **URL**, and **Category** for the tuples retrieved.

Query10 : Retrieve book title, price, author name, and URL for the publishers *Bright Publications*.

$$\Pi_{\text{Book_title, Price, Aname, URL}} (\sigma_{\text{Pname} = \text{"Bright Publications"}} ((\text{BOOK} \bowtie_{\text{BOOK.ISBN} = \text{AUTHOR_BOOK.ISBN}} \text{AUTHOR_BOOK}) \bowtie_{\text{AUTHOR_BOOK.A_ID} = \text{AUTHOR.A_ID}} \text{AUTHOR}) \bowtie_{\text{BOOK.P_ID} = \text{PUBLISHER.P_ID}} \text{PUBLISHER}))$$

In this query, four relations are joined on the basis of some common attribute. First the join of the relations **BOOK** and **AUTHOR_BOOK** is created based on the common attribute **ISBN**, then the resultant relation is joined with the relation **AUTHOR** on the basis of a common attribute **A_ID**. After this, the resultant relation is joined with the fourth relation **PUBLISHER** on the basis of a common attribute **P_ID**.

Finally, the select operation is applied on the resultant relation to retrieve all the tuples where **Pname** is *Bright Publications*, and then the project operation is used to display the required attributes, namely, **Book_title**, **Price**, **Aname**, and **URL** for the tuples retrieved.

Query11 : Retrieve the name and address of publishers who have not published any books.

$$\begin{aligned} &\rho(R_1, \Pi_{\text{P_ID}}(\text{PUBLISHER})) \\ &\rho(R_2, \Pi_{\text{P_ID}}(\text{BOOK})) \\ &\rho(R_3, R_1 - R_2) \\ &\rho(R_4, \Pi_{\text{Pname, Address}}(R_3 \bowtie_{R_3.P_ID = \text{PUBLISHER.P_ID}} \text{PUBLISHER})) \end{aligned}$$

This query uses the difference operation to retrieve ID of publishers who have not published any books. In the first step, the list of **P_ID** is retrieved from the relation **PUBLISHER** and stored in the relation R_1 . In the second step, the list of **P_ID** is retrieved from the relation **BOOK** and stored in the relation R_2 . The relation R_3 consists of a list of **P_ID** in the relation R_1 , which is not present in the relation R_2 . In other words, R_3 stores the **P_ID** of the publishers having a record in relation **PUBLISHER** who have not published any book; hence, there is no corresponding entry in relation **BOOK**. Finally, after creating join of relations **PUBLISHER** and R_3 , **Pname** and **Address** for publishers who have not published any books are retrieved in relation R_4 .

Query12 : Retrieve the name of all the publishers and the ISBN and title of books published by them (if any).

$$\Pi_{\text{Pname, ISBN, Book_title}} (\text{BOOK} \bowtie_{\text{BOOK.P_ID} = \text{PUBLISHER.P_ID}} \text{PUBLISHER})$$

In this query, the right outer join of relations **BOOK** and **PUBLISHER** is created to retrieve the name of all the publishers and the **ISBN** and title of the books published by them along with those publishers who have not published any book. In that case, the values for attributes **ISBN** and **Book_title** will have null values in the resultant relation.

■ **Query13 :** Retrieve ID and the number of books written by each author.

$$A_ID \bowtie_{COUNT(ISBN)} (AUTHOR_BOOK)$$

In this query, firstly the tuples of relation **AUTHOR_BOOK** are grouped on the basis of **A_ID** and then, the aggregate function **COUNT** is used to count the number of occurrences of **ISBN** in each group.

■ **Query14 :** Retrieve ISBN and the average rating given to each book.

$$ISBN \bowtie_{AVG(Rating)} (REVIEW)$$

In this query, firstly the tuples of relation **REVIEW** are grouped on the basis of **ISBN** and then the aggregate function **AVG** is used to find the average rating in each group.

■ **Query15 :** Retrieve the name of the publishers who have published all categories of books.

$$\rho(R_1, \Pi_{Pname, Category} (BOOK \bowtie_{BOOK.P_ID = PUBLISHER.P_ID} PUBLISHER))$$

$$\rho(R_2, \Pi_{Category} (BOOK))$$

$$\rho(R_3, R_1 \div R_2)$$

In this query, firstly the join of relations **BOOK** and **PUBLISHER** is created and **Pname** and **Category** are retrieved in a relation R_1 . Then the unique values for **Category** are retrieved from the **BOOK** relation in another relation R_2 . Finally, the relation R_1 is divided by the relation R_2 to retrieve the name of all those publishers who have published the books of all categories in the relation R_3 .

4.2 RELATIONAL CALCULUS

Relational calculus, an alternative to relational algebra, is a non-procedural or declarative query language as it specifies what is to be retrieved rather than how to retrieve it. In relational calculus, queries are expressed in the form of variables and formulas consisting of these variables. The formula specifies the properties of the resultant relation without giving specific procedure for evaluating it. There are two forms of relational calculus, namely, *tuple relational calculus (TRC)* and *domain relational calculus (DRC)*. In tuple relational calculus, the variable ranges over tuples from a specified relation and in domain relational calculus, the variable ranges over attributes from a specified relation. The Structured Query Language (SQL) is influenced by TRC; however, the Query-By-Example (QBE) is influenced by DRC.

4.2.1 Tuple Relational Calculus

The tuple relational calculus is based on **tuple variables**, which takes tuples of a specific relation as its values. A simple tuple relational calculus query is of the form

$$\{T | P(T)\}$$

where, **T** is a tuple variable and **P(T)** is a condition or formula for retrieving required tuples from a relation. The resultant relation of this query is the set of all tuples **T** for which the condition **P(T)**

evaluates to *true*. For example, the query to retrieve all books having price greater than \$30 can be specified in the form of tuple calculus expression as

$$\{T \mid \text{BOOK}(T) \wedge T.\text{Price} > 30\}$$

The condition $\text{BOOK}(T)$ specifies that the tuple variable T is defined for the relation BOOK . The query retrieves the set of all the tuples T satisfying the condition $T.\text{Price} > 30$. Note that $T.\text{Price}$ refers to the attribute Price of tuple variable T . This query extracts all the attributes of the relation BOOK . However, if only selected attributes have to be retrieved, then the attribute list can be specified in the tuple calculus expression as

$$\{T.\text{ISBN}, T.\text{Book_title}, T.\text{Price} \mid \text{BOOK}(T) \wedge T.\text{Price} > 30\}$$

This expression extracts only three attributes, namely, ISBN , Book_title , and Price of the tuple set satisfying the given condition.

The tuple calculus expression consists basically of three components, which are

- ⇒ There relation R for which tuple variable T is defined
- ⇒ A condition $P(T)$ on the basis of which set of tuples is to be retrieved.
- ⇒ An attribute list specifying the required attributes to be retrieved for the tuples satisfying the given condition.

Expressions and Formulas in Tuple Relational Calculus

A general expression of the tuple relational calculus consists of a tuple variable T and a formula $P(T)$. A formula can consist of more than one tuple variable. A formula is made up of atoms and atoms can be of any of the forms given here.

1. $R(T)$, where T is a tuple variable and R is a relation.
2. $T_1.A_1 \text{ opr } T_2.A_2$, where opr is a comparison operator ($=, \neq, <, \leq, >, \geq$), T_1 and T_2 are tuple variables. A_1 is an attribute of the relation on which T_1 ranges and A_2 is an attribute of the relation on which T_2 ranges.
3. $T.A \text{ opr } c$ or $c \text{ opr } T.A$, where opr is any comparison operator, T is a tuple variable, A is an attribute of the relation on which T ranges, and c is a constant in the domain of attribute A .

Each of the atoms evaluates to either *true* or *false* for a specific combination of tuples known as the **truth value** of an atom. A formula is built from one or more atoms concatenated with the help of the logical operators ($\neg, \wedge, \vee$) by using these rules

1. An atom is a formula.
2. If F is a formula, then so is $\neg F$.
3. If F_1 and F_2 are formulae, then so are $F_1 \wedge F_2$, $F_1 \vee F_2$ and $F_1 \Rightarrow F_2$, where $F_1 \Rightarrow F_2$ is also equivalent to $(\neg(F_1) \vee F_2)$.
4. If F is a formula, then so is $\exists T(F)$, where T is a tuple variable.
5. If F is a formula, then so is $\forall T(F)$, where T is a tuple variable.

Note that in the last two clauses, special symbols called **quantifiers**, namely, *existential quantifier* ($\exists$) and *universal quantifier* ($\forall$) are used to quantify the tuple variable T . The expression, $\forall T$, means ‘for every occurrence of T ’ and the expression $\exists T$, means ‘for some occurrence of T ’. A tuple variable T is said to be **bound**, if it is quantified, that is, it appears in $(\exists T)$ or $(\forall T)$ clause, otherwise, it is **free**. A tuple variable T can be said to be free or bound in a formula, on the basis of the rules given here.

- ⇒ A tuple variable T is free in a formula F , which is an atom.
- ⇒ A tuple variable T is free or bound in a formula of the forms $(F_1 \wedge F_2)$, $(F_1 \vee F_2)$ and $\neg F$ depending on whether it is free or bound in F_1 or F_2 (if it occurs in either of the formulas). Note that in a formula of the form $F = (F_1 \vee F_2)$, a tuple variable may be free in F_1 and bound in F_2 , or vice versa. In such cases, one occurrence of the tuple variable is bound and the other is free in F .
- ⇒ All free occurrences of a tuple variable T in F are bound in a formula F' of the form $F' = (\exists T)(F)$ or $F' = (\forall T)(F)$. The tuple variable is bound to the quantifier specified in F' .

A query can be evaluated on the basis of a given instance of the database. The truth value of a formula can be derived as given here.

- (a) An atom (formula) is *true* if tuple variable T is assigned a tuple from a relation R , otherwise it is *false*.
- (b) $\neg F$ is *true* if F is *false*, and it is *false* if F is *true*.
- (c) $F_1 \wedge F_2$ is *true* if both F_1 and F_2 are *true*, otherwise it is *false*.
- (d) $F_1 \vee F_2$ is *false* if both F_1 and F_2 are *false*, otherwise it is *true*.
- (e) In $F_1 \Rightarrow F_2$, F_2 is *true* whenever F_1 is *true*, otherwise it is *false*.
- (f) $(\exists T)(F)$ is *true* if F is *true* for some (at least one) tuple assigned to free occurrences of T in F , otherwise it is *false*.
- (g) $(\forall T)(F)$ is *true* if F is *true* for every tuple (universal) assigned to free occurrences of T in F , otherwise it is *false*.

Transforming the Universal and Existential Quantifiers

An expression containing a universal quantifier can be transformed into an equivalent expression containing an existential quantifier and vice versa. The steps to perform this type of transformation are

1. Replace one type of quantifier into the other and precede the quantifier by NOT ($\neg$) operator.
2. Replace AND($\wedge$) with OR($\vee$) operator and vice versa.
3. Formula is preceded by NOT ($\neg$) operator, as a result negated formula becomes affirmed and vice versa.

Consider some examples of this transformation given here.

$$(\exists T)(F) \equiv \neg(\forall T)(\neg F)$$

$$(\forall T)(F) \equiv \neg(\exists T)(\neg F)$$

$$(\exists T)(F_1 \wedge F_2) \equiv \neg(\forall T)(\neg F_1 \vee \neg F_2)$$

$$(\forall T)(F_1 \vee F_2) \equiv \neg(\exists T)(\neg F_1 \wedge \neg F_2)$$

$$(\exists T)(\neg F_1 \vee \neg F_2) \equiv \neg(\forall T)(F_1 \wedge F_2)$$

$$(\forall T)(\neg F_1 \wedge \neg F_2) \equiv \neg(\exists T)(F_1 \vee F_2)$$

Note that the symbol $\equiv$ represents equivalent to.

Things to Remember

In a formula, when multiple tuple variables are quantified, the evaluation takes place from inside to outside. For example, in the formula $\exists T \forall S (F)$, first $\forall S$ will evaluate and then $\exists T$ will be evaluated. The sequence of identical quantifiers can be altered with no other type of quantifier appearing in between.

Example Queries in Tuple Relational Calculus

The queries expressed in relational algebra can also be expressed in tuple relational calculus. The difference is only in the notation used to express the queries. The queries discussed in relational algebra can be expressed in tuple relational calculus as shown here.

Query1 : Retrieve city, phone, and URL of the author whose name is *Lewis Ian*.

$$\{T.City, T.Phone, T.URL \mid AUTHOR(T) \wedge T.Aname="Lewis Ian"\}$$

In this query, tuple variable *T* for the relation *AUTHOR* and the condition to retrieve required tuples from this relation is specified after the bar ($\mid$). The tuples satisfying the given condition are assigned to *T*, and the attribute values for *City*, *Phone*, and *URL* are displayed for the tuples retrieved.

Query2 : Retrieve name, address, and phone of all the publishers located in *New York* state.

$$\{T.Pname, T.Address, T.Phone \mid PUBLISHER(T) \wedge T.State="New York"\}$$

In this query, the tuple variable *T* for the relation *PUBLISHER* is defined, and the tuples satisfying the given condition are assigned to *T*. Then, the attribute values for *Pname*, *Address*, and *Phone* are displayed for the tuples retrieved.

Query3 : Retrieve title and price of all the textbooks with page count greater than 600.

$$\{T.Book_title, T.Price \mid BOOK(T) \wedge T.Category="Textbook" \wedge T.Page_count > 600\}$$

In this query, the tuple variable *T* for the relation *BOOK* is defined, and the tuples satisfying the given condition are assigned to *T*. Then, the attribute values for *Book_title* and *Price* are displayed for the tuples retrieved.

Query4 : Retrieve ISBN, title and price of the books belonging to either novel or language book category.

$$\{T.ISBN, T.Book_title, T.Price \mid BOOK(T) \wedge (T.Category="Novel" \vee T.Category="Language Book")\}$$

In this query, the tuple variable *T* for the relation *BOOK* is defined, and the tuples satisfying the given condition are assigned to *T*. Then, the attribute values for *ISBN*, *Book_title*, and *Price* are displayed for the tuples retrieved.

Query5 : Retrieve ID, name, address, and phone of publishers publishing novels.

$$\{T.P_ID, T.Pname, T.Address, T.Phone \mid PUBLISHER(T) \wedge (\exists S)(BOOK(S) \wedge S.P_ID=T.P_ID \wedge S.Category="Novel")\}$$

In this query, tuple variables *T* and *S* for the relations *PUBLISHER* and *BOOK*, respectively, are defined. Of these, *T* is a free tuple variable and *S* is bound to the existential quantifier. Note that the condition $S.P_ID=T.P_ID$ is a join condition for *PUBLISHER* and *BOOK* relations. The attribute values of *P_ID*, *Pname*, *Address*, and *Phone* of all the tuples satisfying the specified condition are displayed.

Query6 : Retrieve title and price of all the books published by *Hills Publications*.

$$\{T.Book_title, T.Price \mid BOOK(T) \wedge (\exists S)(PUBLISHERS(S) \wedge S.P_ID = T.P_ID \wedge S.Pname = "Hills Publications")\}$$

In this query, the tuple variables T and S for the relations BOOK and PUBLISHER, respectively, are defined. Here, T is a free tuple variable and S is bound to the existential quantifier. The two relations are joined on the basis of common attribute P_ID. The attribute values of Book_title and Price of all the tuples satisfying the specified condition are displayed.

Query7 : Retrieve book title, reviewers ID, and rating of all the textbooks.

```
{T.Book_title, S.R_ID, S.Rating | BOOK(T) ^ REVIEW(S) ^
T.Category="Textbook" ^ T.ISBN = S.ISBN }
```

In this query, the tuple variables T and S for the relations BOOK and REVIEW, respectively, are defined. The two relations are joined on the basis of common attribute ISBN. The attribute values of Book_title, R_ID, and Rating of all the tuples satisfying the specified condition are displayed.

Query8 : Retrieve title, category, and price of all the books written by *Charles Smith*.

```
{T.Book_title, T.Category, T.Price | BOOK(T) ^ ((∃S)(∃P)(AUTHOR(S) ^ AUTHOR_
BOOK(P) ^ S.Aname="Charles Smith" ^ T.ISBN=P.ISBN ^ P.A_ID=S.A_ID)) }
```

In this query, the tuple variables T, S, and P for the relations BOOK, AUTHOR, and AUTHOR_BOOK, respectively, are defined. Here, T is a free tuple variable, whereas S and P are bound to the existential quantifiers. The relations BOOK and AUTHOR_BOOK are joined on the basis of common attribute ISBN and the resultant relation is joined with the relation AUTHOR on the basis of common attribute A_ID. After this, the attribute values for Book_title, Category, and Price of all the tuples where Aname is *Charles Smith*, are displayed.

Query9 : Retrieve ID, name, URL of author, and category of the book C++.

```
{T.A_ID, T.Aname, T.URL, S.Category | AUTHOR(T) ^ BOOK(S) ^ S.Book_
title ="C++" ^ ((∃P)(AUTHOR_BOOK(P) ^ T.A_ID=P.A_ID ^ P.ISBN=S.ISBN)) }
```

In this query, the tuple variables T, S, and P for the relations AUTHOR, BOOK, and AUTHOR_BOOK, respectively, are defined. Here, T and S are free tuple variables whereas P is bound to the existential quantifier. The relations AUTHOR and AUTHOR_BOOK are joined on the basis of common attribute A_ID, and the resultant relation is joined with the relation BOOK on the basis of common attribute ISBN. After this, the attribute values for A_ID, Aname, URL, and Category of all the tuples where Book_title is C++ are displayed.

Query 10: Retrieve book title, price, author name, and URL for the publishers *Bright Publications*.

```
{T.Book_title, T.Price, S.Aname, S.URL | BOOK(T) ^ AUTHOR(S) ^ ((∃P)
(∃R)(PUBLISHER(P) ^ AUTHOR_BOOK(R) ^ T.ISBN=R.ISBN ^ R.A_ID=S.A_ID
^ T.P_ID=P.P_ID ^ P.Pname="Bright Publications")) }
```

In this query, the tuple variables T, S, P, and R for the relations BOOK, AUTHOR, PUBLISHER, and AUTHOR_BOOK, respectively, are defined. Among these, T and S are free tuple variables, whereas P and R are bound to existential quantifiers. Here, four relations are joined on the basis of some common attribute. First the join of the relations BOOK and AUTHOR_BOOK is created on the basis of the attribute ISBN, then the resultant relation is joined with the relation AUTHOR on the basis of the attribute A_ID. After this, the resultant relation is joined with the fourth relation PUBLISHER on the basis of the

attribute P_ID. Finally, the attribute values for Book_title, Price, Aname, and URL of all tuples where Pname is *Bright Publications*, are displayed.

Query11 : Retrieve name and address of publishers who have not published any books.

$$\{T.Pname, T.Address \mid PUBLISHER(T) \wedge (\neg(\exists S)(BOOK(S) \wedge T.P_ID=S.P_ID)) \}$$

In this query, the tuple variables T and S for the relations PUBLISHER and BOOK, respectively, are defined. Here, T is a free tuple variable, whereas S is bound to existential quantifier. Note that the negation operator ($\neg$) is used to retrieve the tuples, which do not satisfy the join condition, that is, it retrieves the attributes Pname and Address of the publishers who have no corresponding books in relation BOOK.

This query can also be expressed by using universal quantifier instead of existential quantifier by following general transformation rules as shown here.

$$\{T.Pname, T.Address \mid PUBLISHER(T) \wedge ((\forall S)(\neg(BOOK(S)) \vee \neg(T.P_ID=S.P_ID))) \}$$

Safe Expressions

An expression of tuple relational calculus containing universal quantifier, existential quantifier or negation condition may lead to a relation of infinite number of tuples. For example, consider an expression given here.

$$\{T \mid \neg BOOK(T)\}$$

This expression appears to be syntactically correct but it refers to all the tuples in the universe not belonging to BOOK relation, which are infinite in number. These types of expressions are known as **unsafe expressions**. Therefore, care must be taken while using quantifier symbols and negation operator in a relational calculus expression to ensure safe expressions. A **safe expression** is an expression, which results in a relation with finite number of tuples.

Safe expressions can be defined more precisely by introducing the concept of the domain of a tuple relational calculus expression, E. The domain of expression E, denoted by $Dom(E)$, is the set of all values appearing either as constant values in the expression E, or appearing in any of the tuples of relations referred by E. In other words, the domain of E is the set of all values that appear as constant in E, or appear in one or more relations whose names appear in E. Therefore, the domain of the above expression includes all the values appearing in the relation BOOK. Similarly, the domain for the **Query 5** includes all the values appearing in the relations BOOK and PUBLISHER. Hence, an expression E, is said to be safe if all the values appearing in the resultant relation are from the domain of E, $Dom(E)$.

The expression $\{T \mid \neg BOOK(T)\}$ is said to be safe if all the values appearing in the resultant relation are from the domain, $Dom(BOOK)$. However, this expression is unsafe as the resultant relation of this expression includes all the tuples in the universe not appearing in the BOOK relation, hence not belonging to the domain $Dom(BOOK)$. All other expressions discussed in tuple relational calculus are safe expressions.

4.2.2 The Domain Relational Calculus

The **domain relational calculus** is based on domain variables, which unlike tuple relational calculus, range over the values from the domain of an attribute, rather than values for an entire tuple. A simple domain relational calculus query is of the form

$$\{T \mid P(T)\}$$

where, T represents a set of domain variables ($x_1, x_2, \dots, x_n$) that ranges over domains of attributes ($A_1, A_2, \dots, A_n$) and P represents the formula on T . Like, tuple relational calculus, formulae in domain relational calculus are built up from atoms and an atom can be of any of the forms given here.

1. $R(x_1, x_2, \dots, x_n)$, where R is a relation with n attributes and $x_1, x_2, \dots, x_n$ are domain variables or domain constants for the corresponding n attributes.
2. $x_1 \text{ opr } x_2$, where opr is a comparison operator ($=, \neq, <, \leq, >, \geq$) and x_1 and x_2 are domain variables. Note that the domain of attributes x_1 and x_2 are compatible for comparison.
3. $x \text{ opr } c$ or $c \text{ opr } x$, where opr is a comparison operator, x is a domain variable and c is a constant in the domain of the attribute for which x is a domain variable.

Formulas are built from atoms by using these rules

1. An atom is a formula.
2. If F is a formula, then so is $\neg F$.
3. If F_1 and F_2 are formulae, then so are $F_1 \wedge F_2$, $F_1 \vee F_2$ and $F_1 \Rightarrow F_2$.
4. If F is a formula, then so is $\exists x(F)$, where x is a domain variable.
5. If F is a formula, then so is $\forall x(F)$, where x is a domain variable.

Like in tuple relational calculus, atoms evaluate to either *true* or *false* for a specific set of values. In rule 1, an atom (formula) is *true*, if the domain variable is assigned values corresponding to a tuple of the specified relation R . Similarly, in the remaining rules, if the domain variables are assigned values that satisfy the condition, then the atom is *true*, otherwise it is *false*.

Example Queries in Domain Relational Calculus

The queries expressed in relational algebra or tuple relational calculus can also be expressed in domain relational calculus. The difference is only in the notation used to express the queries. In domain relational calculus, the queries based on the relation **AUTHOR** require seven domain variables $a_1, a_2, \dots, a_7$ to range over the domain of each attribute of the relation in order. Similarly, for the relations **BOOK**, **PUBLISHER**, **AUTHOR_BOOK**, and **REVIEW**, the number of domain variables required is eight ($b_1, b_2, \dots, b_8$), six ($p_1, p_2, \dots, p_6$), two (c_1, c_2) and three (r_1, r_2, r_3), respectively.

In domain relational calculus, the domain variables for the attributes to be displayed are specified before the bar ($|$) and the relations and conditions to retrieve the required tuples are specified after the bar ($|$). The queries discussed earlier can be expressed in domain relational calculus as shown here.

Query1 : Retrieve city, phone, and URL of the author whose name is *Lewis Ian* .

$$\{a_4, a_6, a_7 \mid (\exists a_2) (\text{AUTHOR}(a_1, a_2, a_3, a_4, a_5, a_6, a_7) \wedge a_2 = \text{"Lewis Ian"})\}$$

The required attributes *City*, *Phone*, and *URL* to be displayed are specified by free domain variables a_4, a_6 , and a_7 , respectively, for the relation **AUTHOR**. Existential quantifier quantifies the variable a_2 for which condition is specified.

Query2 : Retrieve name, address, and phone of all the publishers located in *New Yorks* tate.

$$\{p_2, p_3, p_5 \mid (\exists p_4) (\text{PUBLISHER}(p_1, p_2, p_3, p_4, p_5, p_6) \wedge p_4 = \text{"New York"})\}$$

The required attributes Pname, Address, and Phone to be displayed are specified by free domain variables p_2 , p_3 , and p_5 , respectively, for the relation PUBLISHER. Existential quantifier quantifies the variable p_4 for which condition is specified.

Query3 : Retrieve title and price of all the textbooks with page count greater than 600.

$$\{b_2, b_4 \mid (\exists b_3) (\exists b_7) (BOOK(b_1, b_2, b_3, b_4, b_5, b_6, b_7, b_8) \wedge b_3 = \text{"Textbook"} \wedge b_7 > 600)\}$$

The required attributes Book_title and Price to be displayed are specified by free domain variables b_2 and b_4 , respectively, for the relation BOOK. Existential quantifiers quantify the variables b_3 and b_7 for which conditions are specified.

Query4 : Retrieve ISBN, title and price of the books belonging to either novel or language book category.

$$\{b_1, b_2, b_4 \mid (\exists b_3) (BOOK(b_1, b_2, b_3, b_4, b_5, b_6, b_7, b_8) \wedge (b_3 = \text{"Novel"} \vee b_3 = \text{"Language Book"}))\}$$

The required attributes ISBN, Book_title, and Price to be displayed are specified by free domain variables b_1 , b_2 , and b_4 , respectively, for the relation BOOK. Existential quantifier quantifies the variable b_3 for which conditions are specified.

Query5 : Retrieve ID, name, address, and phone of publishers publishing novels.

$$\{p_1, p_2, p_3, p_5 \mid (\exists b_3) (\exists b_8) (BOOK(b_1, b_2, b_3, b_4, b_5, b_6, b_7, b_8) \wedge PUBLISHER(p_1, p_2, p_3, p_4, p_5, p_6) \wedge b_8 = p_1 \wedge b_3 = \text{"Novel"})\}$$

The required attributes P_ID, Pname, Address, and Phone to be displayed are specified by free domain variables p_1 , p_2 , p_3 , and p_5 , respectively, for the relation PUBLISHER. Existential quantifiers quantify the variables b_3 and b_8 for which conditions are specified. The variable p_1 appearing in a condition is not quantified as it is appearing as a free domain variable before the bar ($\mid$).

Note that the condition $b_8 = p_1$ is a join condition as it relates two domain variables ranging over common attributes from two different relations. This condition creates join of two relations BOOK and PUBLISHER.

Query6 : Retrieve title and price of all the books published by Hills Publications.

$$\{b_2, b_4 \mid (\exists p_2) (\exists p_1) (\exists b_8) (BOOK(b_1, b_2, b_3, b_4, b_5, b_6, b_7, b_8) \wedge PUBLISHER(p_1, p_2, p_3, p_4, p_5, p_6) \wedge p_1 = b_8 \wedge p_2 = \text{"Hills Publications"})\}$$

The required attributes Book_title and Price to be displayed are specified by free domain variables b_2 and b_4 , respectively, for the relation BOOK. Existential quantifiers quantify the variables p_2 , p_1 , and b_8 for which conditions are specified. Note that the condition $p_1 = b_8$ is a join condition as it creates a join of two relations PUBLISHER and BOOK.

Query7 : Retrieve book title, reviewers ID, and rating of all the textbooks.

$$\{b_2, r_1, r_3 \mid (\exists b_3) (\exists b_1) (\exists r_2) (BOOK(b_1, b_2, b_3, b_4, b_5, b_6, b_7, b_8) \wedge REVIEW(r_1, r_2, r_3) \wedge b_1 = r_2 \wedge b_3 = \text{"Textbook"})\}$$

The required attributes Book_title, R_ID, and Rating to be displayed are specified by free domain variables b_2 for the relation BOOK, r_1 and r_3 for the relation REVIEW, respectively. Existential quantifiers quantify the variables b_3 , b_1 , and r_2 for which conditions are specified. Note that, the condition $b_1 = r_2$ is a join condition as it creates a join of two relations BOOK and REVIEW.

Query8 : Retrieve title, category, and price of all the books written by *Charles Smith*.

$$\{b_2, b_3, b_4 \mid (\exists b_1) (\exists c_2) (\exists c_1) (\exists a_1) (\exists a_2) (\text{BOOK}(b_1, b_2, b_3, b_4, b_5, b_6, b_7, b_8) \wedge \text{AUTHOR}(a_1, a_2, a_3, a_4, a_5, a_6, a_7) \wedge \text{AUTHOR_BOOK}(c_1, c_2) \wedge b_1 = c_2 \wedge c_1 = a_1 \wedge a_2 = \text{"Charles Smith"})\}$$

The required attributes *Book_title*, *Category*, and *Price* to be displayed are specified by free domain variables b_2 , b_3 , and b_4 , respectively, for the relation *BOOK*. Existential quantifiers quantify the variables b_1 , c_2 , c_1 , a_1 , and a_2 for which conditions are specified. The conditions $b_1 = c_2$ and $c_1 = a_1$ creates a join of relations *BOOK*, *AUTHOR_BOOK*, and *AUTHOR*.

Query9 : Retrieve ID, name, URL of author, and category of the book *C++*.

$$\{a_1, a_2, a_7, b_3 \mid (\exists c_1) (\exists b_1) (\exists c_2) (\exists b_2) (\text{BOOK}(b_1, b_2, b_3, b_4, b_5, b_6, b_7, b_8) \wedge \text{AUTHOR}(a_1, a_2, a_3, a_4, a_5, a_6, a_7) \wedge \text{AUTHOR_BOOK}(c_1, c_2) \wedge a_1 = c_1 \wedge c_2 = b_1 \wedge b_2 = \text{"C++"})\}$$

The required attributes *A_ID*, *Aname*, *URL*, and *Category* to be displayed are specified by free domain variables a_1 , a_2 , a_7 for the relation *AUTHOR* and b_3 for the relation *BOOK*, respectively. Existential quantifiers quantify the variables c_1 , b_1 , c_2 , and b_2 for which conditions are specified. Note that variable a_1 appearing in a condition is not quantified as it is appearing as free domain variable before the bar ($\mid$). The conditions $a_1 = c_1$ and $c_2 = b_1$ creates a join of relations *AUTHOR*, *AUTHOR_BOOK*, and *BOOK*.

Query10 : Retrieve book title, price, author name, and URL for the publishers *Bright Publications*.

$$\{b_2, b_4, a_2, a_7 \mid (\exists a_1) (\exists c_1) (\exists c_2) (\exists b_1) (\exists b_8) (\exists p_1) (\exists p_2) (\text{BOOK}(b_1, b_2, b_3, b_4, b_5, b_6, b_7, b_8) \wedge \text{AUTHOR}(a_1, a_2, a_3, a_4, a_5, a_6, a_7) \wedge \text{AUTHOR_BOOK}(c_1, c_2) \wedge \text{PUBLISHER}(p_1, p_2, p_3, p_4, p_5, p_6) \wedge a_1 = c_1 \wedge c_2 = b_1 \wedge b_8 = p_1 \wedge p_2 = \text{"Bright Publications"})\}$$

The required attributes *Book_title*, *Price*, *Aname*, and *URL* to be displayed are specified by free domain variables b_2 , b_4 for the relation *BOOK* and a_2 , a_7 for the relation *AUTHOR*, respectively. Existential quantifiers quantify the variables a_1 , c_1 , c_2 , b_1 , b_8 , p_1 , and p_2 for which conditions are specified. The conditions $a_1 = c_1$, $c_2 = b_1$, and $b_8 = p_1$ creates a join of relations *AUTHOR*, *AUTHOR_BOOK*, *BOOK*, and *PUBLISHER*.

Query11 : Retrieve name and address of publishers who have not published any books.

$$\{p_2, p_3 \mid (\exists p_1) (\text{PUBLISHER}(p_1, p_2, p_3, p_4, p_5, p_6) \wedge (\neg(\exists b_8) (\text{BOOK}(b_1, b_2, b_3, b_4, b_5, b_6, b_7, b_8) \wedge p_1 = b_8)))\}$$

The required attributes *Pname* and *Address* to be displayed are specified by free domain variables p_2 , and p_3 , respectively for the relation *PUBLISHER*. Existential quantifiers quantify the variables p_1 and b_8 for which conditions are specified. The negation operator ($\neg$) is used to retrieve those publisher IDs who have not published any book.

This query can also be expressed by using universal quantifier instead of existential quantifier by following general transformation rules as shown here.

$$\{p_2, p_3 \mid (\exists p_1) (\text{PUBLISHER}(p_1, p_2, p_3, p_4, p_5, p_6) \wedge ((\forall b_7) (\neg(\text{BOOK}(b_1, b_2, b_3, b_4, b_5, b_6, b_7, b_8)) \vee \neg(p_1 = b_7))))\}$$

4.3 EXPRESSIVE POWER OF RELATIONAL ALGEBRA AND RELATIONAL CALCULUS

As discussed earlier, the relational algebra provides a procedure for solving a problem, whereas relational calculus simply describes what the requirement is. However, relational algebra and relational

calculus are logically equivalent. That is, for every safe relational calculus expression there exists equivalent relational algebra expression and vice versa. In other words, there is one-to-one mapping between the two and the difference lies only in the way the query is expressed.

The expressive power of relational algebra is frequently used as a metric to measure the power of any relational database query language. A query language is said to be **relationally complete** if it is at least as powerful as relational algebra, that is, if it can express any query expressed in relational algebra. Relational completeness has become an important basis for comparing the expressive power of various relational database query languages. Most of the relational database query languages are relationally complete. In addition, they have more expressive power than relational algebra or relational calculus with the inclusion of various advanced operations such as aggregate functions, grouping, and ordering.

● SUMMARY ●

1. Relational algebra is a procedural query language that consists of a set of operations that take one or two relations as input, and result into a new relation as an output.
2. Relational algebra operations are divided into two groups. One group consists of operations developed specifically for relational databases such as select, project, rename, join, and division. The other group includes the set-oriented operations such as union, intersection, difference, and cartesian product.
3. The select operation retrieves all those tuples from a relation that satisfy a specific condition. The Greek letter sigma (σ) is used as a select operator.
4. The project operation is used to select some required attributes from a relation while discarding the other attributes. The Greek letter pi (π) is used as project operator.
5. Rename operation is used to provide name to the relation obtained after applying any relational algebra operation. The Greek letter rho (ρ) is used as a rename operator.
6. The union operation, denoted by $\cup$, returns a third relation that contains tuples from both or either of the operand relations.
7. The intersection operation, denoted by $\cap$, returns a third relation that contains tuples common to both the operand relations.
8. The difference operation, denoted by $-$ (minus), returns a third relation that contains all tuples present in one relation, which are not present in the second relation.
9. The cartesian product, also known as cross product or cross join, returns a third relation that contains all possible combinations of the tuples from the two operand relations. The cartesian product is denoted by the symbol $\times$.
10. A join can be defined as a cartesian product followed by select and project operations. The join operation joins two relations to form a new relation on the basis of one common attribute present in the two operand relations.
11. A common case of the join operation in which the join condition consists only of equality condition is known as equijoin.
12. The equijoin operation results in two attributes in the resulting relation having exactly the same value. If one of the two identical attributes is removed from the result of equijoin, it is known as natural join.

13. An inner join selects only those tuples from both the joining relations that satisfy the joining condition. The outer join, on the other hand, selects all the tuples satisfying the join condition along with the tuples for which no tuples from the other relation satisfy the join condition.
14. There are three types of outer joins, namely, left outer join, right outer join, and full outer join.
15. Some of the requirements like finding total amount, average price, which are mathematical in nature, cannot be expressed by basic relational algebra operations. For such type of queries, aggregate functions are used. Aggregate functions take a collection of values as input, process them, and return a single value as the result.
16. Relational calculus, an alternative to relational algebra, is a non-procedural or declarative query language as it specifies what is to be retrieved rather than how to retrieve it. In relational calculus, queries are expressed in the form of variables and formulas consisting of these variables.
17. There are two forms of relational calculus, namely, tuple relational calculus (TRC) and domain relational calculus (DRC).
18. The tuple relational calculus is based on tuple variables, which takes tuples of a specific relation as its values.
19. An expression of tuple relational calculus containing universal quantifier, existential quantifier or negation condition may lead to a relation of infinite number of tuples. These types of expressions are known as unsafe expressions. A safe expression, on the other hand, results in a relation with finite number of tuples.
20. The domain relational calculus is based on domain variables, which unlike tuple relational calculus, range over the values from the domain of an attribute, rather than values for an entire tuple.
21. The expressive power of relational algebra is frequently used as a metric to measure the power of any relational database query language. A query language is said to be relationally complete if it is at least as powerful as relational algebra, that is, if it can express any query expressed in relational algebra.

● KEY TERMS ●

- | | |
|--|--|
| <ul style="list-style-type: none"> ● Select operation ● Project operation ● Rename operation ● Union operation ● Intersection operation ● Difference operation ● Cartesian product ● Join operation ● Equijoin ● Natural join ● Outer join ● Left outer join ● Right outer join | <ul style="list-style-type: none"> ● Full outer join ● Division ● Aggregate functions ● Tuple relational calculus ● Tuple variable ● Atom ● Formula ● Existential quantifier ● Universal quantifier ● Safe expression ● Domain relational calculus ● Domain variable |
|--|--|

● EXERCISES ●

A. Multiple Choice Questions

1. Which of the following is the operation developed specifically for the relational databases?
 - (a) Union
 - (b) Cartesian product
 - (c) Division
 - (d) Difference
2. Which of the following is the set-oriented operation?
 - (a) Select
 - (b) Difference
 - (c) Division
 - (d) Project
3. Which of the following is the binary operation?
 - (a) Union
 - (b) Join
 - (c) Cartesian product
 - (d) All of these
4. Which of the following is true for right outer join?
 - (a) It includes all the tuples from both the relations satisfying the join condition along with all the tuples in the right relation that do not have a corresponding tuple in the left relation.
 - (b) It includes all the tuples from both the relations satisfying the join condition along with all the tuples in the left relation that do not have a corresponding tuple in the right relation.
 - (c) It selects all the tuples satisfying the join condition along with the tuples for which no rows from the other relation satisfy the join condition.
 - (d) None of these
5. Which of the following aggregate functions is used for counting tuples or values in a relation?
 - (a) Avg
 - (b) Count
 - (c) Max
 - (d) Min
6. Which of the following is the component of tuple calculus expression?
 - (a) The relation R for which tuple variable T is defined.
 - (b) A condition P(T) on the basis of which set of tuples is to be retrieved.
 - (c) An attribute list specifying the required attributes to be retrieved for the tuples satisfying the given condition.
 - (d) All of these
7. The truth value of a formula can be derived as
 - (a) $\neg F$ is *true* if F is *false*, and it is *false* if F is *true*.
 - (b) $F_1 \wedge F_2$ is *true* if both F_1 and F_2 are *true*, otherwise it is *false*.
 - (c) $F_1 \vee F_2$ is *false* if both F_1 and F_2 are *false*, otherwise it is *true*.
 - (d) All of these
8. Which of the following is symbol for existential quantifier?
 - (a) $\forall$
 - (b) $\exists$
 - (c) $\neg$
 - (d) $\equiv$
9. Which of the following statements is true?
 - (a) $(\forall T)(F) \equiv \neg(\exists T)(F)$
 - (b) $(\exists T)(F_1 \wedge F_2) \equiv \neg(\forall T)(\neg F_1 \vee \neg F_2)$
 - (c) $(\forall T)(F_1 \vee F_2) \equiv (\exists T)(\neg F_1 \wedge \neg F_2)$
 - (d) $(\forall T)(F_1 \wedge F_2) \equiv \neg(\exists T)(F_1 \vee F_2)$

10. Which of the following is not a rule for forming formula in domain relational calculus?
- (a) An atom is a formula.
 - (b) If F is a formula, then so is $\neg F$.
 - (c) If F_1 and F_2 are formulae, then so are $F_1 \wedge F_2$, $F_1 \vee F_2$ and $F_1 \Rightarrow F_2$.
 - (d) If F is a formula, then so is $\exists T (F)$, where T is a tuple variable.

B. Fill in the Blanks

1. The _____ is used to select some required attributes from a relation while discarding the other attributes.
2. _____ is used to provide name to the relation obtained after applying any relational algebra operation.
3. The _____ can be defined on any two relations, that is, they need not be union compatible.
4. Relational algebra expression that uses intersection operation can be rewritten by replacing the intersection operation with a pair of _____ operation.
5. A common case of the join operation $R_1 \bowtie R_2$ is one in which the join condition consists only of equality condition, also known as _____.
6. The _____ is useful for queries of the form 'for all objects having all the specified properties'.
7. A tuple variable T is said _____, if it is quantified, that is, it appears in $(\exists T)$ or $(\forall T)$ clause, otherwise, it is _____.
8. A _____ is an expression, which results in a relation with a finite number of tuples.
9. The _____ is based on domain variables, which unlike tuple relational calculus, range over the values from the domain of an attribute, rather than values for an entire tuple.
10. A query language is said to be _____ if it is at least as powerful as relational algebra, that is, if it can express any query expressed in relational algebra.

C. Answer the Questions

1. Give similarities and differences between relational algebra and relational calculus.
2. What are the various unary and binary operations in relational algebra? Why are they called so?
3. Discuss various unary operations of relational algebra with example.
4. What do you mean by union compatibility of relations? Explain with example.
5. Discuss various set operations of relational algebra with example.
6. What is the purpose of rename operator in relational algebra?
7. What is the role of join operations in relational algebra? Differentiate between equijoin and natural join.
8. What are outer join operations and how are they different from inner join operations? Discuss various types of outer join operations.
9. What is cartesian product? How is it different from join operation?
10. Discuss division operation in detail with example.
11. What is relational calculus? What are its two forms? Differentiate between them.
12. What information is required to be specified in tuple calculus expression? Explain with example.

13. What are quantifiers and what are its types? How is existential quantifier transformed into universal quantifier and vice versa? Give suitable examples.
14. When is an expression in tuple relational calculus said to be unsafe? How can safe expressions be defined? Explain with example.
15. Differentiate between the following:
 - (a) Tuple variable and domain variable
 - (b) Atom and formula
16. What are the different forms of atom in tuple relational calculus and domain relational calculus? Give rules for forming formulas in tuple relational calculus and domain relational calculus.
17. Convert the following domain calculus expression into corresponding, English statement, relational algebra expression and tuple calculus expression.

$$\{b_2, r_1, r_3 \mid (\exists b_3) (\exists b_1) (\exists r_2) (\text{Rel}_1(b_1, b_2, b_3) \wedge \text{Rel}_2(r_1, r_2, r_3) \wedge b_1 = r_2 \wedge b_3 = \text{"ABC"}) \}$$

18. What do you understand by the expressive power of relational algebra? How are relational algebra and relational calculus logically equivalent? When is a query language said to be relationally complete?

D. Practical Questions

1. Consider the following relational schema for the SALES database:

CUSTOMER(CustNo, CName, City)

ORDER(OrderNo, OrderDate, CustNo, Amount)

ORDER_ITEM(OrderNo, ItemNo, Qty)

ITEM(ItemNo, UnitPrice)

On the basis of this relational schema, write the following queries in relational algebra, tuple calculus, and domain calculus.

- (a) Retrieve number and date of orders placed by customers residing at *Atlanta* city.
- (b) Retrieve number and unit price of items for which order of quantity greater than 50 is placed.
- (c) Retrieve order number, date, and item number for the order of items having unit price greater than 20.
- (d) Retrieve details of customers who have placed order for the item number *I010*.
- (e) Retrieve number and unit price of items for which order is placed by the customer number *C001*.

2. Given the relational schemes:

ENROL(S#, C#, Section)—S# represents student number

TEACH(Prof, C#, Section)—C# represents course number

ADVISE(Prof, S#)—Prof is thesis advisor of S#

PRE_REQ(C#, Pre_C#)—Pre_C# is prerequisite course

GRADES(S#, C#, Grade, Year)

STUDENT(S#, Sname)—Sname is student name

Express the following queries in relational algebra, tuple calculus, and domain calculus.

- (a) List all students taking courses with *Zeba*.
- (b) List all students taking at least one course that their advisor teaches.
- (c) List those professors who teach more than one section of the same course.

3. Consider the relational database:

EMPLOYEE(Empname, Street, City)
 WORKS(Empname, Companyname, Salary)
 COMPANY(Companyname, City)
 MANAGES(Empname, Managername)

Give an expression in the relational algebra for each request.

- Find the names of all employees who work for *First Bank Corporation*.
- Find the names, street addresses, and cities of residence of all employees who work for *First Bank Corporation* and earn more than 200,000 per annum.
- Find the names of all employees in this database who live in the same city as the company for which they work.
- Find the names of all the employees who earn more than every employee of *Small Bank Corporation*.
- Find the number of employees working in each company.
- Find average, maximum, and minimum salary for each company.

4. Consider the following relational schema for the BANK database:

BRANCH(BranchID, Bname, City, Phone)
 ACCOUNT(AccountNo, Aname, AType, BranchID, Balance)
 TRANSACTION(TID, T_Date, T_Type, AccountNo, Amount)

On the basis of this relational schema, write the following queries in relational algebra.

- Retrieve ID and name of all the branches located in *Seattle* city.
- Retrieve ID, type, and amount of all the transactions of withdrawal type.
- List number and type of all accounts opened in the branch having ID *B010*.
- List number and name of account holders withdrawing amount greater than 10,000 on 31st March, 2007.
- List number and name of account holders having savings account in the city *Los Angeles*.
- Calculate average balance of accounts present in the *AX International Bank* in *New York*.
- Calculate maximum and minimum balance of accounts in each city.

5. Express queries (a) to (d) of question 3 in tuple calculus and domain calculus.

6. Express queries (a) to (e) of question 4 in tuple calculus and domain calculus.

7. R_1 and R_2 are two given relations:

R_1

A	B
A1	B1
A2	B2
A3	B3
A4	B4

R_2

X	Y
A1	B1
A7	B7
A2	B2
A4	B4

Find Union, Intersection, and Set difference.

STRUCTURED QUERY LANGUAGE

After reading this chapter, the reader will understand:

- *The basic features of SQL*
- *The concept of schema and catalog in SQL*
- *Different types of data types available in SQL*
- *DDL commands to create relations in SQL*
- *Various constraints that can be applied on the attributes of a relation*
- *DDL commands to alter the structure of a relation*
- *DML commands to manipulate the data stored in the database*
- *Various clauses of SELECT command that can be used to extract data from database*
- *Various set operations like union, intersect, and minus*
- *The use of INSERT, UPDATE, and DELETE commands to insert, update, and delete tuples in a relation*
- *Searching of null values in a relation*
- *The use of various aggregate function in select query*
- *The use of GROUP BY and HAVING clause for the grouping of tuples in a relation*
- *Combining tuples from two relations using join queries*
- *The concept of nested queries*
- *Specifying complex constraints using the concept of assertions*
- *The concept views, whose contents are derived from already existing relations and does not exist in physical form*
- *The concept of stored procedures and functions in SQL*
- *The use of trigger for automatic execution of the stored procedures*
- *The difference between embedded and dynamic SQL*
- *The use of cursor for extracting multiple tuples from a relation*
- *The role of Open Database Connectivity (ODBC) for establishing connection with databases*
- *The use of Java Database Connectivity (JDBC) for establishing connectivity with databases in Java*
- *The use of SQL-Java standard for embedding SQL statements in Java program*

The structured query language (SQL) pronounced as “ess-que-el”, is a language which can be used for retrieval and management of data stored in relational database. It is a non-procedural language as it specifies what is to be retrieved rather than how to retrieve it. It can be used for defining the structure of data, modifying data in the database, and specifying the security constraints. SQL is an interactive query language that helps users to execute complicated queries in minutes, sometimes even in seconds as compared to the number of days taken by a programmer to write a program for those complicated queries. In addition, commands in SQL resemble simple English statements, making it easier to learn and understand. These features make SQL as one of the major reasons of the success of relational

databases. Some common relational database management systems that use SQL are Oracle, Sybase, Microsoft SQL Server, Access, Ingres, etc.

Donald D. Chamberlin and Raymond F. Boyce developed the first version of SQL, also known as **Sequel** as part of the System R project in the early 1970s at IBM. Later on, SQL was adopted as a standard by the ANSI in 1986, and was subsequently adopted as an ISO standard in 1987. This version of SQL was called SQL-86. In 1989, a revised standard commonly known as SQL-89 was published. The demand for more advanced SQL version led to the development of new version of SQL called SQL-92. This version addressed several weaknesses in SQL-89 standard. In 1999, the standard SQL-99 was released by ANSI/ISO. This version addresses some of the advanced areas of modern SQL systems, such as object-relational database concepts, integrity management, etc.

The complete coverage of SQL cannot be provided in a single chapter hence; only the fundamental concepts have been discussed in this chapter. Moreover, the implementation of various SQL commands may differ from version to version.

5.1 BASIC FEATURES OF SQL

SQL has proved to be the standard language as same set of commands are used to create, retrieve, and modify data regardless of the system they are working on. SQL provides different types of commands, which can be used for different purposes. These commands are divided into two major categories.

- ⇒ **Data definition language (DDL):** DDL provides commands that can be used to create, modify, and delete database objects. The database objects include relation schemas, relations, views, sequences, catalogs, indexes, etc.
- ⇒ **Data manipulation language (DML):** DML provides commands that can be used to access and manipulate the data, that is, to retrieve, insert, delete, and update data in a database.

Learn More

The first commercially available implementation of SQL was released in 1979 by Relational Software Inc., which is known as *Oracle Corporation* today. Thus, Oracle is the pioneer RDBMS that started using SQL.

In addition, there are various techniques like embedded SQL, cursors, dynamic SQL, ODBC, JDBC, SQLJ, which can be used for accessing databases from other applications. These techniques will be discussed later in the chapter.

5.2 DATA DEFINITION

Data definition language (DDL) commands are used for defining relation schemas, deleting relations, creating indexes, and modifying relation schemas. In this section, DDL commands are discussed and an overview of the basic data types in SQL is provided. In addition, it discusses how integrity constraints can be specified for a relation.

5.2.1 Concept of Schema and Catalog in SQL

In earlier versions of SQL, there was no concept of relational database schema, thus, all the database objects were considered to be a part of same database schema. It implies that no two database objects can share same name, since they all are part of the same name space. Due to this, all users of the database have to coordinate with each other to ensure that they use different names for database objects. However, the present day database systems provide a three level hierarchy for naming different objects of relational database. This hierarchy comprises **catalogs**, which in turn consists of

schemas, and various database objects are incorporated within the schema. For example, any relation can be uniquely identified as:

`catalog_name.schema_name.relation_name`



NOTE *Schema is identified by a schema name, and its elements include relations, constraints, views, domains, etc.*

Database system can have multiple catalogs, and users working with different catalogs can use the same name for database objects without any clashes. Each user of database system is associated with a default catalog and schema, and whenever the user connects with the database (by providing the unique combination of user name and password), he gets connected with the default catalog and schema.

In SQL, one can create schema by using the **CREATE SCHEMA** command, including definitions of all the elements of schema. Alternatively, the schema can be given a name and authorization identifier to indicate the user who is owner of schema, and the elements of schema can be specified later. For example, to create *Online Book* schema owned by the user identified by authorization identifier *Smith*, the command can be specified as

```
CREATE SCHEMA OnlineBook AUTHORIZATION Smith;
```

The schema can be deleted by using the **DROP SCHEMA** command. The privilege to create or drop schemas, relations, and other components of database is granted to relevant user by the system administrator.



Integrity constraints like referential integrity can be defined between relations only if they belong to schemas within the same catalog.

5.2.2 Data Types

Data type identifies the type of data to be stored in an attribute of a relation and also specifies associated operations for handling the data. Data type defined for an attribute associates a set of properties with that attribute. These properties cause attributes of different data types to have different set of operations that can be performed on these attributes. The common data types supported by standard SQL are

- ⇒ **NUMERIC(p, s)**: used to represent data as floating-point number like 17.312, 27.1204, etc. The number can have **p** significant digits (including sign) and **s** number of the **p** digits can be present on the right of decimal point. For example, data type specified as **NUMERIC(5, 2)** indicates that value of an attribute can be of form 332.32, where as number of the forms 32.332 or 0.332 are not allowed.
- ⇒ **INT** or **INTEGER**: used to represent data as a number without a decimal point. The size of the data is machine dependent.
- ⇒ **SMALLINT**: used to represent data as a number without a decimal point. It is a subset of the **INTEGER** so the default size is usually smaller than **INT**.
- ⇒ **CHAR(n)** or **CHARACTER(n)**: used to represent data as fixed-length string of characters of size **n**. In case of fixed-length strings, a shorter string is padded with blank characters to the right.

For example, if the value *ABC* is to be stored for an attribute with data type **CHAR(8)**, the string is padded with five blanks to the right. Padded blanks are ignored during comparison operation.

- ⇒ **VARCHAR(n)** or **CHARACTER VARYING**: used to represent data as variable length string of characters of maximum size *n*. In case of variable length string, a shorter string is not padded with blank characters.
- ⇒ **DATE** and **TIME**: used to represent data as date or time. The **DATE** data type has three components, namely, *year*, *month*, and *day* in the form **YYYY-MM-DD**. The **TIME** data type also have three components, namely, *hours*, *minutes*, and *seconds* in the form **HH:MM:SS**.
- ⇒ **BOOLEAN**: used to represent the third value *unknown*, in addition to *true* and *false* values, because of the presence of *null* values in SQL.
- ⇒ **TIMESTAMP**: used to represent data consisting of both date and time. The **TIMESTAMP** data type has six components, *year*, *month*, *day*, *hour*, *minute*, and *second* in the form **YYYY-MM-DD-HH:MM:SS[.SF]**, where *F* is the fractional part of the *second* value.

In addition to these data types, there are many more data types supported by SQL like **CLOB** (**CHARACTER LARGE OBJECT**), **BLOB** (**BINARY LARGE OBJECT**), etc. These data types are rarely used and hence, are not discussed here.

In addition to built-in data types, user-defined data types can also be created. The user-defined data type can be created using the **CREATE DOMAIN** command. For example, to create user-defined data type **Vchar**, the command can be specified as

```
CREATE DOMAIN Vchar AS VARCHAR(15);
```

Now, **Vchar** can be used as data type for any attribute for which data type **VARCHAR(15)** is to be defined. After declaring such data type, it becomes easier to make changes in the domain that is being used by numerous attributes in a relation schema.

5.2.3 CREATETABLECommand

The **CREATE TABLE** command is used to define a new relation, its attributes and its data types. In addition, various constraints like key, entity integrity, and referential integrity constraints can also be specified. The syntax for **CREATE TABLE** command is shown here.

```
CREATE TABLE <table_name> (<attribute1> <data_type1> [constraint1],
                           <attribute2> <data_type2> [constraint2],
                           :
                           <attributen> <data_typen> [constraintn],
                           [table_constraint1]
                           :
                           [table_constraintn]
                           );
```

where,

table_name is the name of new relation

attribute_i is the attribute of relation

data_type_i is the data type of values of the attribute

constraint_i is any of the column-level constraints defined on the corresponding attribute

table_constraint_i is any of the table-level constraints

For example, the command to create **BOOK** relation whose schema is **BOOK**(ISBN, Book_title, Category, Price, Copyright_date, Year, Page_count, P_ID) can be specified as

```
CREATE TABLE BOOK (
    ISBN          VARCHAR(15),
    Book_title    VARCHAR(50),
    Category      VARCHAR(20),
    Price         NUMERIC(6,2),
    Copyright_date NUMERIC(4),
    Year          NUMERIC(4),
    Page_count    NUMERIC(4),
    P_ID          VARCHAR(4)
);
```

Once the relation is created, its structure can be viewed by using the **DESCRIBE** (or **DESC**) command. For example, the command to view the structure of **BOOK** relation can be specified as

```
DESCRIBE BOOK
```

```
OR
```

```
DESC BOOK
```

As a result of this command, the structure of the relation **BOOK** is displayed as shown here.

Name	Null?	Type
-----	-----	-----
ISBN		VARCHAR2(15)
BOOK_TITLE		VARCHAR2(50)
CATEGORY		VARCHAR2(20)
PRICE		NUMBER(6,2)
COPYRIGHT_DATE		NUMBER(4)
YEAR		NUMBER(4)
PAGE_COUNT		NUMBER(4)
P_ID		VARCHAR2(4)

In this example, the relation **BOOK** is created without specifying any constraints. Since no constraints are defined, invalid values can be entered in the relation. To avoid such a situation, it is necessary to define constraints within the definition of the relation.

5.2.4 Specifying Constraints

As discussed earlier, constraints are required to maintain the integrity of the data, which ensures that the data in database is consistent, correct, and valid. These constraints can be specified within the

definition of a relation. These include key, integrity, and referential constraints along with the restrictions on attribute domains and *null* values. This section discusses how different types of constraints can be specified at the time of creation of relation.

PRIMARY KEY Constraint

This constraint ensures that attribute declared as primary key cannot have *null* value and no two tuples can have same value for primary key attribute. In other words, the values in the primary key attribute are not *null* and are unique. If a primary key has a single attribute, the **PRIMARY KEY** can be applied as a column-level as well as table-level constraint. The syntax of **PRIMARY KEY** constraint when applied as a column-level constraint is given here.

```
<attribute> <data_type> PRIMARY KEY
```

The syntax of **PRIMARY KEY** constraint when applied as a table-level constraint is

```
PRIMARY KEY (<attribute>)
```

For example, the attribute ISBN of BOOK relation can be declared as a primary key as shown here.

```
ISBN VARCHAR(15) PRIMARY KEY
```

OR

```
PRIMARY KEY (ISBN)
```

If a primary key has more than one attribute, the **PRIMARY KEY** constraint is specified as table-level constraint. The syntax of **PRIMARY KEY** constraint when applied on more than one attribute is

```
PRIMARY KEY (<attribute1>, <attribute2>, ... , <attributen>)
```

For example, attributes ISBN and R_ID of REVIEW relation can be declared as composite primary key as shown here.

```
PRIMARY KEY (ISBN, R_ID)
```

UNIQUE Constraint

The **UNIQUE** constraint ensures that the set of attributes have unique values, that is, no two tuples can have same value in the specified attributes. Like **PRIMARY KEY** constraint, **UNIQUE** constraint can be applied as a column-level as well as table-level constraint. The syntax of **UNIQUE** constraint when applied as a column-level constraint is given here.

```
<attribute> <data_type> UNIQUE
```

The syntax of **UNIQUE** constraint when applied as a table-level constraint is

```
UNIQUE (<attribute>)
```

For example, the **UNIQUE** constraint on attribute Pname of the relation PUBLISHER can be specified as shown here.

```
Pname VARCHAR(50) UNIQUE
```

OR

```
UNIQUE (Pname)
```

When **UNIQUE** constraint is applied on more than one attribute it is specified as table-level constraint. The syntax of **UNIQUE** constraint when applied on more than one attribute is

UNIQUE (<attribute₁>, <attribute₂>, ... , <attribute_n>)

For example, the **UNIQUE** constraint on attributes **Pname** and **Address** of relation **PUBLISHER** can be specified as shown here.

UNIQUE (**Pname**, **Address**)

CHECK Constraint

This constraint ascertains that the value inserted in an attribute must satisfy a given expression. In other words, it is used to specify the valid values for a certain attribute. The syntax of **CHECK** constraint when applied as a column-level constraint is given here.

<attribute> <data_type> **CHECK**(<expression>)

For example, the **CHECK** constraint for ensuring that the value of attribute **Price** of the relation **BOOK** is greater than \$20 can be specified as

Price **NUMERIC**(6,2) **CHECK**(**Price**>20)

The **CHECK** constraint when specified as table-level constraint can be given a separate name that allows referring to the constraint whenever needed. The syntax of **CHECK** constraint when applied as a table-level constraint is

CONSTRAINT <constraint_name> **CHECK** (<expression>)

For example, the constraint on the attribute **Price** of the **BOOK** relation can be given a name as shown here.

CONSTRAINT **Chk_price** **CHECK** (**Price** > 20)

Constraints can also be applied on more than one attribute simultaneously. For example, the constraint that the **Copyright_date** must be either less than or equal to the **Year** (publishing year) can be specified as

CONSTRAINT **Chk_date** **CHECK** (**Copyright_date** <= **Year**)

When **CHECK** constraint is applied to a domain, it allows specifying a condition that must be satisfied by any value assigned to an attribute whose type is the domain. For example, the **CHECK** constraint can ensure that the domain, say **dom**, allows values between 20 and 200 as shown here.

CREATE DOMAIN **dom** **AS** **NUMERIC**(5,2)

CONSTRAINT **chk_dom** **CHECK**(**VALUE** **BETWEEN** 20 and 200)

If the domain **dom** is assigned to the attribute **Price** of a **BOOK** relation, it ensures that the attribute **Price** must have values between 20 and 200. As a result, if a value that is not between 20 and 200 is inserted into the **Price** attribute, an error occurs.

A domain can also be restricted to contain a specified set of values using **IN** clause with **CHECK** constraint. For example, the **CHECK** constraint ensuring that the domain, say **dom1**, allows values within a list of certain values can be specified as

Learn More

SQL allows setting default value for an attribute by adding the **DEFAULT** <default_value> clause to the definition of an attribute in **CREATE TABLE** command.

```
CREATE DOMAIN dom1 CHAR(20)
CONSTRAINT chk_dom1 CHECK (VALUE IN ('Textbook', 'Language
Book', 'Novel'))
```

NOT NULL Constraint

The **NOT NULL** constraint is used to specify that an attribute will not accept *null* values. For example, consider a tuple in the **BOOK** relation where the attribute **Page_count** contains a *null* value. Such a tuple provides incomplete information of that particular book. In such a case, using the **NOT NULL** constraint does not permit *null* value.

Thes yntaxof **NOT NULL** constraint is given here.

```
<attribute> <data_type> NOT NULL
```

For example, the **NOT NULL** constraint for the attribute **Page_count** of **BOOK** relation can be specified as shown here.

```
Page_count Numeric(4) NOT NULL
```

The **NOT NULL** constraint can be specified using **CHECK** constraint. For example, the **NOT NULL** constraint on the attribute **Book_title** can be specified using the **CHECK** constraint as shown here.

```
Book_title VARCHAR(50) CHECK (Book_title IS NOT NULL)
```

The **NOT NULL** constraint ensures that the attribute of a particular domain is not permitted to take *null* values. For example, a domain, say **dom2**, can be restricted to take non-null values as shown here.

```
CREATE DOMAIN dom2 VARCHAR(20) NOT NULL
```

Keypoint: *The **NOT NULL** constraint can be specified only as column-level constraint and not as table-level constraint.*

FOREIGN KEY Constraint

This constraint ensures that the foreign key value in the referencing relation must exist in the primary key attribute of the referenced relation, that is, foreign key references the primary key attribute of referenced relation.

A **FOREIGN KEY** constraint can be applied as a column-level as well as table-level constraint. The syntax of **FOREIGN KEY** constraint when applied as a column-level constraint is given here.

```
<attribute> <data_type> REFERENCES <referenced_relation>
(<key_attribute>)
```

where, **key_attribute** is the primary key of the referenced relation

For example, the attribute **P_ID** of relation **BOOK** can be specified as foreign key, which refers to the primary key **P_ID** of relation **PUBLISHER** as shown here.

```
P_ID VARCHAR(4) REFERENCES PUBLISHER(P_ID)
```

When FOREIGN KEY constraint is applied on more than one attribute it is specified as table-level constraint. The syntax of FOREIGN KEY constraint defined on more than one attribute is given here.

```
FOREIGN KEY (<attribute1>) REFERENCES <referenced_relation>
(<key_attribute_1>)
:
FOREIGN KEY (<attributen>) REFERENCES <referenced_relation>
(<key_attribute_n>)
```

For example, attributes ISBN and R_ID of relation REVIEW can be declared as foreign keys as shown here.

```
FOREIGN KEY (ISBN) REFERENCES BOOK(ISBN)
```

```
FOREIGN KEY (R_ID) REFERENCES AUTHOR(A_ID)
```

All the constraints discussed here can be defined together for a relation. The CREATE TABLE command for the relations of *Online Book* database along with the required constraints is specified here.

```
CREATE TABLE PUBLISHER
```

```
( P_ID      VARCHAR(4),
  Pname     VARCHAR(50) NOT NULL,
  Address   VARCHAR(50),
  State     VARCHAR(15),
  Phone     VARCHAR(20),
  Email_id  VARCHAR(30),
  PRIMARY KEY(P_ID)
);
```

```
CREATE TABLE BOOK
```

```
( ISBN          VARCHAR(15),
  Book_title    VARCHAR(50) NOT NULL,
  Category      VARCHAR(20),
  Price         NUMERIC(6,2),
  Copyright_date NUMERIC(4),
  Year          NUMERIC(4),
  Page_count    NUMERIC(4),
  P_ID          VARCHAR(4)   NOT NULL,
  CONSTRAINT Chk_price CHECK (Price BETWEEN 20 AND 200),
  PRIMARY KEY(ISBN),
  FOREIGN KEY(P_ID) REFERENCES PUBLISHER(P_ID)
);
```


CREATE TABLE AUTHOR

```
( A_ID    VARCHAR(4),
  Aname   VARCHAR(30) NOT NULL,
  State   VARCHAR(15),
  City    VARCHAR(15),
  Zip     VARCHAR(10),
  Phone   VARCHAR(20),
  URL     VARCHAR(30),
  PRIMARY KEY(A_ID)
);
```

CREATE TABLE AUTHOR_BOOK

```
( A_ID    VARCHAR(4)    NOT NULL,
  ISBN    VARCHAR(15)   NOT NULL,
  FOREIGN KEY(A_ID) REFERENCES AUTHOR(A_ID),
  FOREIGN KEY(ISBN) REFERENCES BOOK(ISBN)
);
```

CREATE TABLE REVIEW

```
( R_ID    VARCHAR(4)    NOT NULL,
  ISBN    VARCHAR(15)   NOT NULL,
  Rating  NUMERIC(2),
  PRIMARY KEY(R_ID, ISBN),
  CONSTRAINT Chk_rating CHECK (Rating BETWEEN 1 AND 10),
  FOREIGN KEY(R_ID) REFERENCES AUTHOR(A_ID),
  FOREIGN KEY(ISBN) REFERENCES BOOK(ISBN)
);
```

5.2.5 ALTER TABLE Command and

Once the relation is created, it might be required to make changes in the structure of a relation as per the needs of a user. The changes can be in the form of adding a new attribute, redefining attribute, or dropping attribute from a relation. To meet such requirements, ALTER TABLE command is used. In general, ALTER TABLE command is used for the following requirements.

- ⇒ to add an attribute
- ⇒ to modify the definition of an attribute
- ⇒ to drop an attribute
- ⇒ to add a constraint
- ⇒ to drop a constraint
- ⇒ to rename attribute and relation

Adding an Attribute

The new attribute can be added to the relation by using the ADD clause of ALTER TABLE command. The syntax to add new attribute in an existing relation is given here.

```
ALTER TABLE <table_name> ADD <attribute> <data_type>;
```

For example, the command to add a new attribute Pname in the relation BOOK can be specified as

```
ALTER TABLE BOOK ADD Pname VARCHAR(10);
```

If no default clause is specified, this attribute will have *null* values in all the tuples, hence NOT NULL constraint is not allowed in this case.

Modifying an Attribute

Any attribute of a relation can be modified by using the MODIFY clause of ALTER TABLE command. It can be used to change either its data type or size or both. The attribute to be modified must be empty before modification. The syntax to modify an attribute of a relation is given here.

```
ALTER TABLE <table_name> MODIFY <attribute> <data_type>;
```

For example, the command to change the data type and size of the attribute Pname can be specified as

```
ALTER TABLE BOOK MODIFY Pname VARCHAR(50);
```

Dropping an Attribute

Sometimes, it is required to remove attribute, which is no longer required from a relation. The DROP COLUMN clause of ALTER TABLE command can be used to remove undesirable attributes from a relation. The syntax to remove an attribute from an existing relation is given here.

```
ALTER TABLE <table_name> DROP COLUMN <attribute>;
```

For example, the command to remove the attribute Pname from the relation BOOK can be specified as

```
ALTER TABLE BOOK DROP COLUMN Pname;
```

Whenever a particular attribute is dropped, the data stored in that attribute and its associated constraints are dropped.

Adding a Constraint

Different types of constraints can be added to the definition of an already existing relation. The syntax to add a constraint using ADD clause is given here.

```
ALTER TABLE <table_name> ADD [CONSTRAINT <constraint_name>]  
<Cons>;
```

where,

CONSTRAINT is a keyword
constraint_name is a name given to constraint
Cons can be any of the constraints discussed earlier

Consider the following examples, in which different types of constraints are added for the different attributes of BOOK relation.

```
ALTER TABLE BOOK ADD CHECK(Book_title <> '');
```

```
ALTER TABLE BOOK ADD CONSTRAINT Cons_1 UNIQUE(ISBN);
```

```
ALTER TABLE BOOK ADD FOREIGN KEY(P_ID) REFERENCES PUBLISHER;
```

The NOT NULL constraint is added differently as it cannot be specified as table constraint. For example, the command to apply this constraint on the attribute `Page_count` can be specified as

```
ALTER TABLE BOOK ALTER COLUMN Page_count SET NOT NULL;
```

It is also possible to modify the definition of attribute by defining a new default value for an attribute, as shown here.

```
ALTER TABLE BOOK ALTER COLUMN Category SET DEFAULT 'Novel';
```

Dropping a Constraint

Any of the constraint defined can be dropped, provided it is given a name when specified. For example, the constraint `Cons_1` specified on the attribute `ISBN`, can be dropped as shown here.

```
ALTER TABLE BOOK DROP CONSTRAINT Cons_1;
```

In addition, the DEFAULT and NOT NULL constraint can be dropped, as shown here.

```
ALTER TABLE BOOK ALTER COLUMN Category DROP DEFAULT;
```

```
ALTER TABLE BOOK ALTER COLUMN Page_count DROP NOT NULL;
```

Renaming an Attribute

The name of an attribute of a relation can be modified by using the `RENAME COLUMN ... TO` clause. The syntax to modify the name of an attribute is given here.

```
ALTER TABLE <table_name> RENAME COLUMN <old_name> TO <new_name>;
```

For example, the command to modify the name of an attribute `Page_count` can be specified as

```
ALTER TABLE BOOK RENAME COLUMN Page_count TO P_count;
```

Renaming a Relation

In addition to renaming an attribute, the name of a relation can also be modified using the `RENAME TO` clause. The syntax to rename a relation is given here.

```
ALTER TABLE <old_table_name> RENAME TO <new_table_name>;
```

For example, the command to rename the relation `BOOK` to new name can be specified as

```
ALTER TABLE BOOK RENAME TO Book_detail;
```

5.2.6 DROP TABLE Command

The `DROP TABLE` command is used to remove an already existing relation, which is no more required as a part of a database. The syntax to remove a relation is given here.

```
DROP TABLE <table_name>;
```

For example, the command to remove the relation `Book_detail` can be specified as

```
DROP TABLE Book_detail;
```

As a result of this command, all the tuples stored in `Book_detail` as well as relation itself is permanently removed and cannot be recovered. Hence, the relation `Book_detail` cannot be referred to for any purpose.

The two clauses that can be used with **DROP TABLE** command are **CASCADE** and **RESTRICT**. If **CASCADE** clause is used, all constraints and views that reference the relation to be removed are also dropped automatically. On the other hand, the **RESTRICT** clause, prevents a relation to be dropped if it is referenced by any of the constraints or views. These clauses can be used with **DROP TABLE** command as shown here.

```
DROP TABLE Book_detail CASCADE;
```

```
DROP TABLE Book_detail RESTRICT;
```

5.3 DATA MANIPULATION LANGUAGE

Data manipulation language (DML) commands are used for retrieving and manipulating data stored in the database. Basically it comprises of different actions, which are given here.

- ⇒ retrieval of data stored in the database
- ⇒ insertion of data into the database
- ⇒ modification of data stored in the database
- ⇒ deletion of data stored in the database

The basic retrieval and manipulating commands are **SELECT**, **INSERT**, **UPDATE**, and **DELETE**. For these commands, the relation must already exist.

5.3.1 SELECTCommand

The **SELECT** command is the only data retrieval command in SQL. It is used to retrieve the subset of tuples or attributes from one or more relations. There are different ways and combinations in which **SELECT** command can be used. The syntax of SQL command is given here.

```
SELECT <attribute1>, <attribute2>, ... , <attributen>  
FROM <table_name>;
```

Here, attribute₁ is an attribute of relation table_name. For example, the command to retrieve the attributes **ISBN**, **Book_title**, and **Category** from relation **BOOK** can be specified as

```
SELECT ISBN, Book_title, Category  
FROM BOOK;
```

The order of the attributes appearing in the command can be different from the order of attributes in the relation. For example, the command to retrieve the attributes **ISBN**, **Category**, **Book_title**, **Page_count**, and **Price** can be specified as

```
SELECT ISBN, Category, Book_title, Page_count, Price  
FROM BOOK;
```

In this example, the order of attributes is different from the order in which they are appearing in the relation. When all the attributes of a relation has to be retrieved, then instead of specifying list of all attributes in the **SELECT** command, the symbol asterisk (*) denoting “all attributes” can be used as shown here.

```
SELECT *  
FROM BOOK;
```

The result of the SQL command may consist of duplicate tuples, as SQL does not automatically eliminate duplicate tuples from the resultant relations. For example, the command to retrieve **Category** of books from the relation **BOOK** can be specified as

```
SELECT Category
FROM BOOK;
```

The result of this command consists of tuples having duplicate values for the attribute **Category**. To eliminate duplicate tuples from the resultant relation, the keyword **DISTINCT** is used in **SELECT** command. Hence, the command to display only the unique values for the attribute **Category** can be specified as

```
SELECT DISTINCT Category
FROM BOOK;
```

Instead of **DISTINCT** keyword, if the keyword **ALL** is specified, the result retains the duplicate tuples. The result is same as when neither **DISTINCT** nor **ALL** keywords are specified. The **ALL** keyword can be used as shown here.

```
SELECT ALL Category
FROM BOOK;
```

The **SELECT** command can be used to perform simple numeric computations on the data stored in a relation. SQL allows using scalar expressions and constants along with the attribute list. For example, the command to display the incremented value of price of books by 10% along with the attributes **Book_title**, **Category**, and **Price** can be specified as

```
SELECT Book_title, Category, Price, Price+Price*.10
FROM BOOK;
```

WHERE clause

The real life database might consist of a large number of tuples and it is not required to view all the tuples every time. Moreover, most of the time only subset of tuples is required to be viewed or processed. The criteria to determine the tuples to be retrieved is specified by using the **WHERE** clause. The syntax of **SELECT** command with **WHERE** clause is given here.

```
SELECT <attribute1>, <attribute2>, ... , <attributen>
FROM <table_name>
WHERE <condition>;
```

When a **WHERE** clause is present, the entire relation is scanned, one tuple at a time to determine whether it satisfies the condition or not. A particular tuple is retrieved only if it satisfies the condition specified in **WHERE** clause. For example, the command to retrieve the attributes **Book_title**, **Category**, and **Price** of those books from **BOOK** relation whose **Category** is *Novel* can be specified as

```
SELECT Book_title, Category, Price
FROM BOOK
WHERE Category = 'Novel';
```

The relational operators (**=**, **<>**, **<**, **<=**, **>**, **>=**) can be used to specify the conditions in the **WHERE** clause for selecting tuples from a relation. Moreover, more than one condition can be concatenated to give a more specific condition by using any of the logical operators **AND**, **OR**, and **NOT**. For example, the command to retrieve **Book_title** and **Price** of those books whose

Category is *Language Book* and Page_count is greater than 400 from BOOK relation can be specified as

```
SELECT Book_title, Price
FROM BOOK
WHERE Category = 'Language Book' AND Page_count>400;
```

Consider the queries covered in Chapter 04. These queries can be expressed here using the SELECT command along with the WHERE clause as shown here.

■ **Query 1:** Retrieve city, phone, and url of the author whose name is *Lewis Ian*.

```
SELECT City, Phone, URL
FROM AUTHOR
WHERE Aname = 'Lewis Ian';
```

■ **Query 2:** Retrieve name, address, and phone of all the publishers located in *New York* state.

```
SELECT Pname, Address, Phone
FROM PUBLISHER
WHERE State = 'New York';
```

■ **Query 3:** Retrieve title and price of all the textbooks with page count greater than 600.

```
SELECT Book_title, Price
FROM BOOK
WHERE Category = 'Textbook' AND Page_count>600;
```

■ **Query 4:** Retrieve ISBN, title, and price of the books belonging to either novel or language book category.

```
SELECT ISBN, Book_title, Price
FROM BOOK
WHERE Category = 'Novel' OR Category = 'Language Book';
```

BETWEEN Operator

The BETWEEN comparison operator can be used to simplify the condition in WHERE clause that specifies numeric values based on ranges. For example, the command to retrieve details of all the books with price between 20 and 30 can be specified as

```
SELECT *
FROM BOOK
WHERE Price BETWEEN 20 AND 30;
```

Similarly, the NOT BETWEEN operator can also be used to retrieve tuples that do not belong to the specified range.

IN operator

The IN operator is used to specify a list of values. The IN operator selects values that match any value in a given list of values. For example, the command to retrieve the book details belonging to the categories *Textbook* or *Novel* can be specified as

```
SELECT *
FROM BOOK
WHERE Category IN ('Textbook', 'Novel');
```

Similarly, NOT IN operator can also be used to retrieve tuples that do not belong to the specified list.

Aliasing and Tuple Variables

The attributes of a relation can be given alternate name using AS clause in SELECT command. Note that the alternate name is provided within the query only. For example, consider the command given here.

```
SELECT Book_title AS Title, P_ID AS Publisher_ID
FROM BOOK;
```

In this command, attribute Book_title is renamed as Title and P_ID as Publisher_ID. An AS clause can also be used to define tuple variables. A tuple variable is associated with a particular relation and is defined in the FROM clause. For example, to retrieve details of publishers publishing books of language book category, the command can be specified as

```
SELECT P.P_ID, Pname, Address, Phone
FROM BOOK AS B, PUBLISHER AS P
WHERE P.P_ID = B.P_ID AND Category = 'Language Book';
```

In this command, tuple variables B and P are defined which are associated with the relations BOOK and PUBLISHER, respectively. The attribute publisher ID is present in both the relations with same name P_ID. Thus, to prevent ambiguity, the attribute name is qualified with the corresponding tuple variable separated by the dot (.) operator.

Tuple variable is especially useful for comparing two tuples from the same relation. For example, the command to retrieve title and price of books belonging to the same category as the book titled C++ can be specified as

```
SELECT B1.Book_title, B1.Price
FROM BOOK AS B1, BOOK AS B2
WHERE B1.Category = B2.Category AND B2.Book_title = 'C++';
```



NOTE The relation name itself is the implicitly defined tuple variable, if no other tuple variable is defined for that relation.

String Operations

In SQL, strings are specified by enclosing them in single quotes. Pattern matching is the most commonly used operation on strings. SQL provides LIKE operator, which allows to specify comparison conditions on only parts of the strings. Different patterns are specified by using two special wildcard character, namely, *percent* (%) and *underscore* (_). The % character is used to match substring with any number of characters, whereas, _ is used to match substring with only one character. Pattern matching is case sensitive as uppercase characters are considered different from lowercase characters. That is, the character s is different from the character S. To understand, how pattern matching is handled, consider these examples.

- ⇒ 'A%' matches any string beginning with character A
- ⇒ '%A' matches any string ending with character A
- ⇒ 'A%A' matches any string beginning and ending with character A
- ⇒ '%AO%' matches any string with character AO appearing anywhere in a string as substring
- ⇒ 'A_ _' matches any string beginning with character A and followed by exactly two characters

- ⇒ `'_ _ A'` matches any string ending with character *A* and preceded by exactly two characters
- ⇒ `'A_ _ _ D'` matches any string beginning with character *A*, ending with the character *D* and with exactly three characters in between

Consider the following queries to illustrate the use of **LIKE** operator.

The command to retrieve details of all the authors, whose name begins with the characters *Jo* can be specified as

```
SELECT *
FROM AUTHOR
WHERE Aname LIKE 'Jo%';
```

The command to retrieve details of all the authors, whose name ends with the characters *in* can be specified as

```
SELECT *
FROM AUTHOR
WHERE Aname LIKE '%in';
```

The command to retrieve details of all the authors, whose name begins with the characters *Jo* and ends with the characters *in* can be specified as

```
SELECT *
FROM AUTHOR
WHERE Aname LIKE 'Jo%in';
```

The command to retrieve details of all the authors, whose name begins with the characters *James* followed by exactly five characters, can be specified as

```
SELECT *
FROM AUTHOR
WHERE Aname LIKE 'James_ _ _ _ _';
```

The special pattern characters (%) and (_) can be included as a literal in the string by preceding the character by an escape character. The escape character is specified after the string using the **ESCAPE** keyword. Any character not appearing in a string to be matched can be defined as an escape character. For example, consider the following patterns, in which backslash (\) is used as an escape character.

- ⇒ `LIKE 'A\b%' ESCAPE '\'`, searches the strings beginning with the characters *A%b*
- ⇒ `LIKE 'A\_b%' ESCAPE '\'`, searches the strings beginning with the characters *A_b*
- ⇒ `LIKE 'A\\b%' ESCAPE '\'`, searches the strings beginning with the characters *A\b*

In order to search the strings not matching the patterns defined, **NOT LIKE** operator can be used. For example, the command to retrieve details of all the authors, whose name does not begins with the characters *Jo* can be specified as

```
SELECT *
FROM AUTHOR
WHERE Aname NOT LIKE 'Jo%';
```

Things to Remember

SQL allows to perform various functions on character strings like concatenation of strings (using `'||'` operator), extracting substrings, computing length of a string, converting case of strings uppercase (using `UPPER()` and `LOWER()` function), etc.

Set Operations

As discussed in Chapter 04, there are various set operations that can be performed on relations, like, union, intersection, and difference. In SQL, these operations are implemented using **UNION**, **INTERSECT**, and **MINUS** operations, respectively.

The **UNION** operation is used to retrieve tuples from more than one relation and it also eliminates duplicate records. For example, the command to find the union of all the tuples with **Price** less than 40 and all the tuples with **Price** greater than 30 from the **BOOK** relation can be specified as

```
(SELECT *  
FROM BOOK  
WHERE Price<40)  
UNION  
(SELECT *  
FROM BOOK  
WHERE Price>30);
```

The **INTERSECT** operation is used to retrieve common tuples from more than one relation. For example, the command to find the intersection of all the tuples with **Price** less than 40 and all the tuples with **Price** greater than 30 from the **BOOK** relation can be specified as

```
(SELECT *  
FROM BOOK  
WHERE Price<40)  
INTERSECT  
(SELECT *  
FROM BOOK  
WHERE Price>30);
```

The **MINUS** operation is used to retrieve those tuples present in one relation, which are not present in other relation. For example, the command to find the difference (using **MINUS** operation) of all the tuples with **Price** less than 40 and all the tuples with **Price** greater than 30 from the **BOOK** relation can be specified as

```
(SELECT *  
FROM BOOK  
WHERE Price<40)  
MINUS  
(SELECT *  
FROM BOOK  
WHERE Price>30);
```

The **UNION**, **INTERSECT**, and **MINUS** operations automatically eliminate the duplicate tuples from the result. However, if all the duplicate tuples are to be retained, the **ALL** keyword can be used with these operations. For example, the command to retain duplicate tuples during **UNION** operation can be specified as

```
(SELECT *  
FROM BOOK  
WHERE Price<40)  
UNION ALL  
(SELECT *  
FROM BOOK  
WHERE Price>30);
```

Similarly, ALL keyword can be used along with the INTERSECT and MINUS operations.



NOTE *During all the set operations, the resultant relations of both the participating SELECT commands must be union compatible. That is, both the resultant relations must have same number of attributes having similar data types and order.*

Ordering the Tuples

Sometimes it may be required to display tuples arranged in a sorted order. SQL provides the ORDER BY clause to arrange the tuples of relation in some particular order. The tuples can be arranged on the basis of the values of one or more attributes. For example, the command to display the tuples of the relation BOOK, on the basis of Price attribute can be specified as

```
SELECT *  
FROM BOOK  
ORDER BY Price;
```

By default, the ORDER BY clause arranges the tuples in ascending order on the basis of values of specified attribute. In addition, the ASC keyword can also be used to explicitly mention the order to be ascending. However, the tuples can be sorted in descending order by using the DESC keyword. For example, the command to sort tuples of the relation BOOK on the basis of Price attribute in the descending order can be specified as

```
SELECT *  
FROM BOOK  
ORDER BY Price DESC;
```

Tuples can also be sorted on the basis of more than one attribute. This can be done by specifying more than one attribute in ORDER BY clause. In ORDER BY clause, more than one attributes can also be specified on the basis of which tuples are to be sorted. For example, to arrange the sort of the relation BOOK, on the basis of two attributes Category and Price, the command can be specified as

```
SELECT *  
FROM BOOK  
ORDER BY Category, Price;
```

This command first arranges the tuples in ascending order on the basis of the Category attribute and then the tuples within a particular category are arranged in the ascending order of the Price attribute. The order of two attributes need not be the same. They can be different. For example, to arrange the tuples of relation BOOK in the descending order of Category and then in the ascending order of Price, the command can be specified as

```
SELECT *  
FROM BOOK  
ORDER BY Category DESC, Price ASC;
```

5.3.2 INSERTCom mand

As discussed earlier, the CREATE TABLE command only defines the structure of a relation and tuples can be inserted in the relation using the INSERT command. This command adds a single tuple at a time in a relation. The syntax to add a tuple in a relation is given here.

```
INSERT INTO <table_name> [(<attribute_list>)]  
VALUES (<value_list>);
```

The `value_list` is the list of values to be inserted in the corresponding attributes listed in `attribute_list`. The values should be specified in the same order in which the attributes are listed in `attribute_list`. In addition, the data type of the corresponding values and the attributes must be same. For example, the command to add a tuple in the relation `BOOK` can be specified as

```
INSERT INTO BOOK (ISBN, Book_title, Category, Price, Copyright_
date, Year, Page_count, P_ID)
VALUES ('001-987-760-9', 'C++', 'Textbook', 40, 2004, 2005, 800,
'P001');
```

In this example, the order and the data type of attributes and corresponding values are same. Alternatively, tuple can be inserted into a relation by omitting the attribute list from the command. That is, the tuple can also be inserted as shown here.

```
INSERT INTO BOOK
VALUES ('001-987-760-9', 'C++', 'Textbook', 40, 2004, 2005, 800,
'P001');
```

The `INSERT` command also allows user to insert data only for the selected attributes. That is, in `INSERT` command, values for some of the attributes can be skipped. The attributes skipped in the command are provided with the default values defined for them, otherwise *null* values are inserted. In this case, the attribute list must be provided. For example, the command to add values for the selected attributes of the relation `PUBLISHER` can be specified as

```
INSERT INTO PUBLISHER (P_ID, Pname, Address, Phone)
VALUES ('P002', 'Sunshine Publishers Ltd.', '45, Second street,
Newark', '6548909');
```

In this example, the values for the attributes `State` and `Email_id` are skipped. Hence, these attributes are provided either with the default values or *null* values.

5.3.3 UPDATECom mand

Sometimes, there may be a situation to make changes in the values of attributes of the relations. SQL provides `UPDATE` command for this type of requirement. The `SET` clause in the `UPDATE` command specifies the attributes to be modified and the new values to be assigned to them. More than one attribute can be specified in the `SET` clause separated by comma. The `WHERE` clause is also required to specify the tuples for which the attributes are to be modified, otherwise value for all the tuples in the relation will be modified. While modifying the values of attributes all the constraints must be satisfied, otherwise the update is not done. For example, the command to modify the `City` to *New York* for the author whose name is *Lewis Ian* can be specified as

```
UPDATE AUTHOR
SET City = 'New York'
WHERE Aname = 'Lewis Ian';
```

The command to modify `State` and `Phone` for the publisher *Bright Publications* can be specified as

```
UPDATE PUBLISHER
SET State = 'Georgia', Phone = '27643676'
WHERE Pname = 'Bright Publications';
```

The command to increment the price of all the books by 10 per cent can be specified as

```
UPDATE BOOK  
SET Price = Price + 0.10 * Price;
```

5.3.4 DELETECommand

The tuples, which are no more required as a part of a relation, can be removed from the relation using the DELETE command. Tuples can be deleted from only one relation at a time. The condition in the WHERE clause of DELETE commands is used to specify the tuples to be deleted. If WHERE clause is omitted, all the tuples of a relation are deleted; however, the relation remains in the database as an empty relation. For example, to delete tuples of the BOOK relation, whose Page_Count is less than 50, the command can be specified as

```
DELETE FROM BOOK  
WHERE Page_count <50;
```

Consider another example, to delete those tuples from the BOOK relation, whose category is *Novel*, the command can be specified as

```
DELETE FROM BOOK  
WHERE Category = 'Novel';
```

5.4 COMPLEX QUERIES IN SQL

The queries discussed so far are some of the simple queries in SQL. In addition to these, complex queries can be specified using additional features of SQL. Some of these additional features are discussed in this section.

5.4.1 NullV alues

SQL allows attribute to have *null* values. The *null* value usually represents missing value having one of three interpretations—value is unknown, value is not available, or attribute is not applicable for particular tuple. However, SQL does not distinguish between the different meanings of *null*. The *null* value in an attribute for a relation can be searched using the IS NULL predicate in the WHERE clause. In all cases, *null* value for an attribute is checked with same syntax. For example, the command to retrieve the details of publishers not having email ID can be specified as

```
SELECT *  
FROM PUBLISHER  
WHERE Email_id IS NULL;
```

The predicate IS NOT NULL can be used to check whether an attribute contains non-null value. For example, to retrieve the details of publishers having email ID (Email_id is not *null*), the command can be specified as

```
SELECT *  
FROM PUBLISHER  
WHERE Email_id IS NOT NULL;
```

In addition, the conditions in WHERE clause may involve logical operators AND, OR, and NOT. Hence, when *null* value is involved, the definition of the logical operators is extended with new value *unknown*. In case of NOT operator, the condition, say A, whose value is *unknown*, results in *unknown*

value. Whereas, the **AND** and **OR** operator involves two (or more) conditions. For this, consider a **WHERE** clause involving two conditions A and B, and the result of any one of them, say B, is *unknown*, then the condition evaluates as follows:

⇒ Conditions involving **AND** operator:

- If the value of A is *true* or *unknown*, the result is *unknown*.
- If the value of A is *false*, the result is *false*.

⇒ Conditions involving **OR** operator:

- If the value of A is *false* or *unknown*, the result is *unknown*.
- If the value of A is *true*, the result is *true*.

5.4.2 Aggregate Functions

Aggregate functions process set of values taken as input and return a single value as a result. SQL provides five built-in aggregate functions, namely, **AVG**, **MIN**, **MAX**, **SUM** and **COUNT**.

The **SUM** and **AVG** functions work for numeric values only, whereas other functions can work for numeric as well as non-numeric values, like strings, date, time, etc. For example, the command to find the average price of books in **BOOK** relation can be specified as

```
SELECT AVG(Price)
FROM BOOK;
```

This command calculates the average value of price of all the books. To calculate the average price of only textbooks, the command can be specified as

```
SELECT AVG(Price)
FROM BOOK
WHERE Category='Textbook';
```

Consider another example to find the maximum price of the book belonging to *Novel* category, the command can be specified as

```
SELECT MAX(Price)
FROM BOOK
WHERE Category='Novel';
```

The **COUNT(*)** function is used to count the total number of tuples in the resultant relation, whereas, **COUNT()** is used to count the number of non-null values in a particular attribute. For example, the command to find the total number of tuples in the relation **PUBLISHER** can be specified as

```
SELECT COUNT(*)
FROM PUBLISHER;
```

To find the number of non-null values in the attribute **Email_id**, the command can be specified as

```
SELECT COUNT(Email_id)
FROM PUBLISHER;
```

Consider another example, to find the number of non-null values in the attribute **Category** of **BOOK** relation, the command can be specified as

```
SELECT COUNT(Category)
FROM BOOK;
```

This command counts the total number of non-null values in the attribute **Category**, including duplicate values also. However, if duplicate values are to be eliminated, the **DISTINCT** keyword can be used and the command can be specified as

```
SELECT COUNT(DISTINCT Category)  
FROM BOOK;
```

5.4.3 GROUP BY and HAVING Clause

In many situations, it is required to apply the aggregate function on a group of tuples from a relation rather on the whole relation. The tuples in the relation can be divided on the basis of the values of one or more attribute. The tuples belonging to a particular group have same value for the attribute on the basis of which grouping is done.

The **GROUP BY** clause can be used in **SELECT** command to divide the relation into groups on the basis of values of one or more attributes. After dividing the relation into groups, the aggregate functions can be applied on the individual group independently. The aggregate functions are performed separately for each group and return the corresponding result value separately. For example, the command to calculate average price for each category of book in the **BOOK** relation can be specified as

```
SELECT AVG(Price)  
FROM BOOK  
GROUP BY Category;
```

To calculate maximum, minimum, and average price of the books published by each publisher, the command can be specified as

```
SELECT MAX(Price), MIN(Price), AVG(Price)  
FROM BOOK  
GROUP BY P_ID;
```

Conditions can be placed on the groups using **HAVING** clause. Note that the **HAVING** clause places conditions on groups, whereas, **WHERE** clause is used to place conditions on the individual tuples. Another difference between **WHERE** and **HAVING** clause is that, conditions specified in the **WHERE** clause cannot include aggregate functions, whereas, **HAVING** clause can. For example, the command to retrieve the book categories for which number of books published is less than 5 can be specified as

```
SELECT Category, COUNT(*)  
FROM BOOK  
GROUP BY Category  
HAVING COUNT(*) <5;
```

More than one condition can be specified in the **HAVING** clause by using logical operators. For example, the command to retrieve average price and average page count for each category of books with average price greater than 30 and average page count less than 900 can be specified as

```
SELECT Category, AVG(Price), AVG(Page_count)  
FROM BOOK  
GROUP BY Category  
HAVING AVG(Price)>30 AND AVG(Page_count)<900;
```

Learn More

Using **GROUP BY** clause, one can create groups within groups known as nested grouping. An Up to 10 level of nesting is allowed in a **GROUP BY** expression.

Consider another example, the command to retrieve average price for each category of book with minimum price greater than 30 can be specified as

```
SELECT Category, AVG(Price)
FROM BOOK
GROUP BY Category
HAVING MIN(Price)>30;
```

The IN and the BETWEEN operators can also be used in the HAVING clause. To understand the use of these operators, consider the commands given here.

```
SELECT Category, AVG(Price), MIN(Page_count)
FROM BOOK
GROUP BY Category
HAVING Category IN ('Textbook', 'Novel');

SELECT Category, MIN(Price), MAX(Price)
FROM BOOK
GROUP BY Category
HAVING AVG(Page_count) BETWEEN 300 AND 1000;
```

While using GROUP BY and HAVING clause, some errors may occur. These errors result from the illegal use of group queries. Some of the possible illegal use of group queries is discussed here.

⇒ Combination of non-group and group expressions without using GROUP BY clause.

For example, consider the command given here.

```
SELECT Book_title, AVG(Price)
FROM BOOK;
```

This command includes non-group attribute Book_title and a group function AVG(Price), which is illegal and results in an error.

⇒ Combination of non-group and group expressions with GROUP BY clause.

For example, consider the command given here.

```
SELECT Book_title, Category, AVG(Price)
FROM BOOK
GROUP BY Category;
```

This command includes non-group attribute Book_title along with group attribute Category and group expression AVG(Price). This command generates an error, as Book_title cannot have same value for all books in a particular Category. This query can be made error free by omitting the attribute Book_title.

⇒ Using group function with WHERE clause instead of HAVING clause.

For example, consider the command given here.

```
SELECT Category, AVG(Price)
FROM BOOK
WHERE MIN(Price)>30
GROUP BY Category;
```

The WHERE clause operates on individual tuples, whereas, group function operates on multiple tuples or on the group of tuples. Hence, group functions cannot be used with WHERE clause.

5.4.4 Joins

A query that combines the tuples from two or more relations is known as **join query**. In such type of queries, more than one relation is listed in the FROM clause. The process of combining data from multiple relations is called **joining**. For example, consider the command given here.

```
SELECT *  
FROM BOOK, PUBLISHER;
```

This command returns the cartesian product of the relations **BOOK** and **PUBLISHER**, that is, the resultant relation consists of all possible combinations of all tuples from both the relations. It returns relation with cardinality $C_1 * C_2$, where C_1 is cardinality of first relation and C_2 is cardinality of second relation. Only selected tuples can be included in the resultant relation by specifying condition on the basis of common attribute of both the relations. The condition on the basis of which relations are joined is known as **join condition**. For example, the command to retrieve details of both book and publishers, where **P_ID** attribute in both the relations have identical values can be specified as

```
SELECT *  
FROM BOOK, PUBLISHER  
WHERE BOOK.P_ID=PUBLISHER.P_ID;
```

Using alias names for the relations can make this command simple, as shown here.

```
SELECT *  
FROM BOOK AS B, PUBLISHER AS P  
WHERE B.P_ID=P.P_ID;
```

This type of join query in which tuples are concatenated on the basis of equality condition is known as **equi-join query**. The resultant relation of this query consists of two columns for the attribute **P_ID** having identical values, one from **BOOK** relation and other from **PUBLISHER** relation. This can be avoided by explicitly specifying the name of attributes to be included in the resultant relation. Such type of command can be specified as

```
SELECT ISBN, Book_title, Price, B.P_ID, Pname  
FROM BOOK AS B, PUBLISHER AS P  
WHERE B.P_ID=P.P_ID;
```

As a result of this command only one column for the attribute **P_ID** from relation **BOOK** is displayed in the result. This type of query is known as **natural join query**. In addition, some additional conditions can also be given to make the selection of tuples more specific apart from the join condition. For example, consider Query 5 (Chapter 04) in which the command to retrieve **P_ID**, **Pname**, **Address**, and **Phone** of publishers publishing novels can be specified as

```
SELECT P.P_ID, Pname, Address, Phone  
FROM BOOK AS B, PUBLISHER AS P  
WHERE B.P_ID=P.P_ID AND Category = 'Novel';
```

Sometimes, it is required to extract information from more than two relations. In such a case, more than two relations are required to be joined through join query. This can be achieved by specifying names of all the required relations in **FROM** clause and specifying join conditions in **WHERE** clause using **AND** logical operator. For example, consider Query 8 (Chapter 04) in which the command to retrieve title, category and price of all the books written by author *Charles Smith* can be specified as


```

SELECT Book_title, Category, Price
FROM BOOK AS B, AUTHOR AS A, AUTHOR_BOOK AS AB
WHERE B.ISBN=AB.ISBN AND AB.A_ID=A.A_ID AND Aname = 'Charles
Smith';

```

Consider some other examples of join queries given here.

Query 6: Retrieve title and price of all the books published by *Hills Publications*.

```

SELECT Book_title, Price
FROM BOOK AS B, PUBLISHER AS P
WHERE B.P_ID=P.P_ID AND Pname='Hills Publications';

```

Query 7: Retrieve book title, reviewers ID, and rating of all the textbooks.

```

SELECT Book_title, R_ID, Rating
FROM BOOK AS B, REVIEW AS R
WHERE B.ISBN=R.ISBN AND Category = 'Textbook';

```

Query 9: Retrieve ID, name, url of author, and category of the book *C++*.

```

SELECT AB.A_ID, Aname, URL, Category
FROM BOOK AS B, AUTHOR AS A, AUTHOR_BOOK AS AB
WHERE A.A_ID=AB.A_ID AND AB.ISBN=B.ISBN AND Book_title= 'C++';

```

Query 10: Retrieve book title, price, author name, and url for the publishers *Bright Publications*.

```

SELECT Book_title, Price, Aname, URL
FROM BOOK AS B, AUTHOR_BOOK AS AB, AUTHOR AS A, PUBLISHER AS P
WHERE B.ISBN=AB.ISBN AND AB.A_ID=A.A_ID AND B.P_ID=P.P_ID AND
Pname='Bright Publications';

```

5.4.5 Nested Queries

The query defined in the WHERE clause of another query is known as **nested query** or **subquery**. The query in which another query is nested is known as enclosing query. The result returned by the subquery is used by the enclosing query for specifying the conditions. In general, several levels of nested queries can be defined. That is, query can be defined inside another query number of times.

For example, the command to retrieve ISBN, Book_title, and Category of book with minimum price can be specified as

```

SELECT ISBN, Book_title, Category
FROM BOOK
WHERE PRICE = (SELECT MIN(Price)
               FROM BOOK);

```

In this command, first the nested query returns a minimum Price from the BOOK relation, which is used by the enclosing query to retrieve the required tuples. Consider another example to retrieve ISBN, Book_title, and Category of book published by the publisher residing in the *New York* state.

Things to Remember

The two queries are said to be correlated when condition specified in the WHERE clause of the inner query refers to an attribute of a relation specified in enclosing query.

```
SELECT ISBN, Book_title, Category
FROM BOOK
WHERE P_ID = (SELECT P_ID
               FROM PUBLISHER
               WHERE State = 'New York');
```

SQL provides four useful operators that are generally used with subqueries. These are ANY, ALL, and EXISTS.

- ⇒ ANY operator compares a value with any of the values in a list or returned by the subquery. This operator returns *false* value if the subquery returns no tuple. For example, the command to retrieve details of books with price equal to any of the books belonging to *Novel* category can be specified as

```
SELECT ISBN, Book_title, Price
FROM BOOK
WHERE Price = ANY (SELECT Price
                    FROM BOOK
                    WHERE Category = 'Novel');
```

In addition to ANY operator, the IN operator can also be used to compare a single value to the set of multiple values. For example, the command to retrieve the details of books belonging to category with page count greater than 300 can be specified as

```
SELECT *
FROM BOOK
WHERE Category IN (SELECT Category
                   FROM BOOK
                   WHERE Page_count >300);
```

In case the nested query returns a single attribute and a single tuple, the query result will be a single value. In such a situation, the = can be used instead of IN operator for the comparison.

- ⇒ ALL operator compares a value to every value in a list returned by the subquery. For example, the command to retrieve details of books with price greater than the price of all the books belonging to *Novel* category can be specified as

```
SELECT ISBN, Book_title, Price
FROM BOOK
WHERE Price > ALL (SELECT Price
                   FROM BOOK
                   WHERE Category = 'Novel');
```

- ⇒ EXISTS operator evaluates to *true* if a subquery returns at least one tuple as a result otherwise, it returns *false* value. For example, the command to retrieve the details of publishers having at least one book published can be specified as

```
SELECT P_ID, Pname, Address
FROM PUBLISHER
WHERE EXISTS (SELECT *
              FROM BOOK
              WHERE PUBLISHER.P_ID = BOOK.P_ID);
```

On the other hand, **NOT EXISTS** operator evaluates to *true* if subquery returns no tuple as a result. For example, the command to retrieve the details of publishers having not publishing any book can be specified as

```
SELECT P_ID, Pname, Address
FROM PUBLISHER
WHERE NOT EXISTS (SELECT *
                   FROM BOOK
                   WHERE PUBLISHER.P_ID = BOOK.P_ID);
```

5.5 ADDITIONAL FEATURES OF SQL

Some of the advanced features of SQL that help in specifying complex constraints and queries efficiently are assertions and views. Assertion is a condition that must always be satisfied by the database. Assertions do not fall into any of the categories of constraints discussed earlier. But, view is a virtual relation which derives its data from one or more existing relations. The result of view is not physically stored.

The two standard features of SQL that are related to programming in database are stored procedures and triggers. The stored procedures are program modules that are stored by the DBMS at the database server. These procedures can be invoked and executed whenever required using the single SQL statement from various applications that have access to the database. Whereas, triggers are type of stored procedures which are automatically invoked and are needed to perform complex data verification operations. This section discusses assertions, views, stored procedure, and triggers.

5.5.1 Assertions

Some of the simple data integrities can be specified while defining the structure of the relation with the help of various constraints like **PRIMARY KEY**, **NOT NULL**, **UNIQUE**, **CHECK**, etc. In addition, the referential integrity can be specified using the **FOREIGN KEY** constraint. However, there are some data integrities, which cannot be specified using any of the constraints discussed so far. Such type of constraints can be specified using the assertions. The DBMS enforces the assertion on the database and the constraints specified in assertion must not be violated. Assertions are always checked whenever modifications are done in corresponding relation.

The syntax to create an assertion can be specified as

```
CREATE ASSERTION <assertion_name>
CHECK (<condition>;
```

The assertion name is used to identify the constraints specified by the assertion and can be used for modification and deletion of assertion, whenever required. An assertion is implemented by writing a query that retrieves any tuples that violates the specified condition. Then this query is placed inside a **NOT EXISTS** clause, which indicates that the result of this query must be empty. Hence, the assertion is violated whenever the result of this query is not empty. For example, the price of textbook must not be less than the minimum price of novel, the assertion for this requirement can be specified as

```
CREATE ASSERTION price_constraint
CHECK(NOT EXISTS(
    SELECT *
    FROM BOOK
    WHERE Category = 'Textbook' AND
```

```

        (Price<(SELECT MIN(Price)
        FROM BOOK
        WHERE Category='Novel')
    )
);

```

In this command, query retrieves tuples with category as *Textbook* and price less than the minimum price of book with *Novel* category. The result of this query must be empty; otherwise it will violate the assertion. DBMS tests the assertion for its validity when it is created. After its creation, future modification to the database is allowed only if it does not violate the assertion. Note that, the assertions should be used to specify complex constraints only that cannot be specified by any of the other constraints, as assertions introduce a significant amount of overhead in terms of time and cost.

5.5.2 Views

A view is a virtual relation, whose contents are derived from already existing relations and it does not exist in physical form. The contents of view are determined by executing a query based on any relation and it does not form the part of database schema. Each time a view is referred to, its contents are derived from the relations on which it is based. A view can be used like any other relation, which is, it can be queried, inserted into, deleted from, and joined with other relations or views, though with some limitations on update operations. These are very useful in the situations where only parts of relations are to be accessed frequently instead of complete data.

The **CREATE VIEW** command of SQL can be used for creating views. This command provides the name to the view and specifies the list of attributes and tuples to be included using a subquery. The syntax to create a view is given here.

```

CREATE VIEW <view_name>
AS <subquery>;

```

For example, the command to create a view containing details of books which belong to *Textbook* and *Language Book* categories can be specified as

```

CREATE VIEW BOOK_1
AS SELECT *
    FROM BOOK
    WHERE Category IN ('Textbook', 'Language Book');

```

This command creates a view, named as **BOOK_1**, having details of books satisfying the condition specified in **WHERE** clause. The view created like this consists of all the attributes of **BOOK** relation; however, only selected attributes can also be included in the view and they can be given another name also. For example, consider the command given here.

```

CREATE VIEW BOOK_2(B_Code, B_Title, B_Category, B_Price)
AS SELECT ISBN, Book_title, Category, Price
    FROM BOOK
    WHERE Category IN ('Textbook', 'Language Book');

```

This command creates a view **BOOK_2**, which consists of the attributes, **ISBN**, **Book_title**, **Category**, and **Price** from the relation **BOOK** with new names, namely, **B_Code**, **B_Title**, **B_Category**, and **B_Price**, respectively. Now queries can be performed on these views as they are performed on the other relations. For example, consider the commands given here.

```
SELECT *  
FROM BOOK_1;
```

```
SELECT *  
FROM BOOK_2  
WHERE B_Price > 30;
```

```
SELECT B_Title, B_Category  
FROM BOOK_2  
WHERE B_Price BETWEEN 30 AND 50;
```

Views can be based on more than one relation. For example, the command to create a view consisting of attributes `Book_title`, `Category`, `Price` and `P_ID` of `BOOK` relation, `Pname` and `State` of `PUBLISHER` relation can be specified as

```
CREATE VIEW BOOK_3  
AS SELECT Book_title, Category, Price, BOOK.P_ID, Pname, State  
FROM BOOK, PUBLISHER  
WHERE BOOK.P_ID = PUBLISHER.P_ID;
```

The views that are based on more than one relation are said to be **complex views**. These types of views are inefficient as they are time consuming to execute, especially if multiple queries are involved in the view definition. Since their contents are not physically stored, they are executed each and every time they are referred to. As an alternative to this, certain database systems allow views to be physically stored, also known as **materialized views**. These types of views save the time spent to execute the subqueries each and every time they are referred to. Such views are updated whenever tuples are inserted, deleted, or modified in the underlying relations. The view is stored temporarily, that is, if view is not queried for a certain period of time, it is physically removed and is recomputed when it is again referred in future. While implementing the concept of materialized views, their advantages must be compared with the cost incurred for storing them and their updations.

A view is updateable if it is based on single relation and the update query can be mapped on to the already existing relation successfully under certain conditions. Any view can be made non-updateable, by adding the `WITH READ ONLY` clause. That is, no `INSERT`, `UPDATE`, or `DELETE` command can be carried over that view.

Views can also be created using the queries based on other views. For example, the command to create view based on the view `BOOK_3`, having details, where publishers belong to the state *New York* can be specified as

```
CREATE VIEW BOOK_4  
AS SELECT *  
FROM BOOK_3  
WHERE State = 'New York';
```

Like relation, view can also be deleted from the database by using `DROP VIEW` command. For example, to delete the view `BOOK_1` from the database, the command can be specified as

```
DROP VIEW BOOK_1;
```

5.5.3 Stored Procedures

Stored procedures are procedures or functions that are stored and executed by the DBMS at the database server machine. In SQL standard, stored procedures are termed as **persistent stored modules** (PSM), as these procedures, like data, are persistently stored by the DBMS. Stored procedures improve performance, as these procedures are compiled only at the time of their creation or modifications. Hence, whenever these procedures are called for the execution the time for compilation is saved. Stored procedures are beneficial when a procedure is required by different applications located at remote sites, as it is stored at server site and can be invoked by any of the applications. This eliminates duplication of efforts in writing SQL application logic and makes code maintenance easy.

Most of the DBMSs allow stored procedures to be written in any general-purpose programming language. As an alternative, stored procedures may consist of simple SQL commands like **INSERT**, **SELECT**, **UPDATE**, etc. Stored procedures can be compiled and executed with different parameters and results. They can accept input parameters and pass values to output parameters. A stored procedure can be created as shown here.

```
CREATE PROCEDURE <name> (<parameters1, . . . , parametersn>)
<local_declarations>
<body of procedure>;
```

Parameters and local declarations are optional and if declared must be of valid SQL data types. Parameters declared must have one of the three modes, namely, **IN**, **OUT**, or **INOUT** specified for it. Parameters specified with **IN** mode are arguments passed to the stored procedure and act like a constant, whereas, **OUT** parameters act like an un-initialized variables and they cannot appear on the right hand side of = symbol. Parameters with **INOUT** mode have combined properties of both **IN** and **OUT** mode. They contain values to be passed to the stored procedure and also can be assigned values to be returned from the stored procedure. For example, the procedure to update the value of price of a book with a given ISBN of relation **BOOK** can be specified as

```
CREATE PROCEDURE Update_price (IN B_ISBN VARCHAR(15),
IN New_Price
NUMERIC(6,2))
UPDATE BOOK
SET Price = New_Price
WHERE ISBN = B_ISBN;
```

Functions are required when value is required to be returned to the calling program since procedures cannot return a value. The function can be created as shown here.

```
CREATE FUNCTION <name> (<parameters1, . . . , parametersn>)
RETURNS <return_type>
<local_declarations>
<body of function>;
```

For example, the function returning a rating of book with a given ISBN and author ID can be specified as

```
CREATE FUNCTION Book_rating (IN B_ISBN VARCHAR(15),
IN Au_ID VARCHAR(4))
RETURNS NUMERIC(2)
DECLARE B_rating NUMERIC(2)
```

```

SET B_rating=(SELECT Rating
              FROM REVIEW
              WHERE ISBN=B_ISBN AND R_ID = Au_ID);
RETURN B_rating;

```

SQL/PSM

SQL/PSM is a part of SQL standard that includes programming constructs for writing the coding part of persistent stored modules. It also includes constructs for specifying conditional statements and looping statements. The conditional statements in SQL/PSM can be specified as

```

IF <condition> THEN <statements>
  ELSEIF <condition> THEN <statements>
  :
  ELSEIF <condition> THEN <statements>
  ELSE <statements>
END IF;

```

The while looping construct can be specified as

```

WHILE <condition> DO
  <statements>;
END WHILE;

```

The repeat looping construct can be specified as

```

REPEAT
  <statements>;
UNTIL <condition>
END REPEAT;

```

For example, a function that searches a book with a given ISBN and declares it to be of *High*, *Medium*, or *Low* price can be written as

```

CREATE FUNCTION B_price (IN B_ISBN VARCHAR(15))
RETURNS VARCHAR(7)
DECLARE Book_price NUMERIC(6,2)
SET Book_price=(SELECT Price
                FROM BOOK
                WHERE ISBN=B_ISBN);
IF Book_price>100 THEN RETURN 'High'
  ELSEIF Book_price>50 THEN RETURN 'Medium'
  ELSE RETURN 'Low'
END IF;

```

Similarly, loops can also be used in the code of stored procedure. Moreover, loops can be named in the code. The name of loop can be used for breaking out of the loop based on some condition by using the statement `LEAVE <loop_name>`.

If the procedure or function is written in some general-purpose programming language, it is necessary to specify the language and the file name where program code is stored. In such a case, the procedure can be created as shown here.

```
CREATE PROCEDURE <procedure_name>(<parameters>)  
LANGUAGE <name of programming language>  
EXTERNAL NAME <file path name>;
```

Once the stored procedures or functions are created, they can be called in any application for execution. The calling statement can be specified as

```
CALL <procedure or function name> (<arguments>;
```

For example, the statements for calling procedures and functions can be specified as

```
CALL Update_price('003-456-433-6', 28);
```

```
CALL Book_rating('003-456-533-8', 'A004');
```

```
CALL B_price('001-987-760-9');
```

5.5.4 Triggers

A trigger is a type of stored procedure that is executed automatically when some database related events like, insert, update, delete, etc., occur. In addition, unlike procedures, triggers do not accept any arguments. The main aim of triggers is to maintain data integrity and also one can design a trigger for recording information, which can be used for auditing purposes. The triggers are mainly needed for following purposes.

- ⇒ Implementing and maintaining complex integrity constraints.
- ⇒ Recording the changes for auditing purposes.
- ⇒ Automatically passing signal to other programs that action needs to be taken whenever specific changes are made to a relation.

Triggers are usually defined on relations. However, it can be defined on views and can also be used to execute other triggers, procedures, and functions. Although triggers like constraints are defined to maintain the database integrity, yet they are different from constraints in the following ways.

- ⇒ Triggers affect only those tuples that are inserted after the trigger is enabled, whereas, constraints affect all tuples in the relation.
- ⇒ Triggers allow enforcing the user-defined constraints, which are not possible through standard constraints supported by database.

A database having triggers associated with it is known as **active database**. A trigger consists of three parts.

- ⇒ **Event:** any change in the database that activates the trigger.
- ⇒ **Condition:** when the trigger is activated, the condition is checked.
- ⇒ **Action:** a procedure that is to be executed, whenever the trigger is activated and the corresponding condition is satisfied.

In other words, a trigger is an event-condition-action rule that states that whenever a specified event occurs and the condition is satisfied, the corresponding action must be executed. The trigger can be created as shown here.


```

CREATE TRIGGER <trigger_name>
[BEFORE or AFTER] [INSERT or UPDATE or DELETE] ON <relation_name>
[FOR EACH ROW]
WHEN <condition>
<statements>;

```

For example, when value in **Phone** attribute of new inserted tuple of a relation **AUTHOR** is empty, indicating absence of phone number, the trigger to insert a *null* value in this attribute can be specified as

```

CREATE TRIGGER Setnull_phone BEFORE INSERT ON AUTHOR
REFERENCING NEW ROW AS nr
FOR EACH ROW
WHEN nr.Phone = ' '
SET nr.Phone = NULL;

```

The **FOR EACH ROW** clause makes sure that trigger is executed for every single tuple processed. Such type of trigger is known as **row level trigger**. Whereas, the trigger, which is executed only once for specified statement, regardless of the number of tuples being effected as a result of that statement, is known as **statement level trigger**. The **FOR EACH STATEMENT** clause specifies the trigger as statement level trigger. For example, trigger defined for a **INSERT** command, will be executed only once, regardless of the number of tuples inserted through single **INSERT** statement. The statement level triggers are the default type of triggers, which are identified by omitting the **FOR EACH ROW** clause.

The **REFERENCING NEW ROW AS** clause can be used to create a variable for storing the new values of an updated tuple. Similarly, **REFERENCING OLD ROW AS** clause can be used to create variables for storing old values of an updated tuple. In addition, the clauses **REFERENCING NEW TABLE AS** or **REFERENCING OLD TABLE AS** can be used to refer to temporary relation consisting of all the affected tuples. Such relations can be used with only **AFTER** triggers and not **BEFORE** triggers.

Consider another example, to create the trigger for changing value of **Price** attribute to a default value \$60, if the value entered by the user exceeds the upper limit of \$200 in case of insertion in relation **BOOK**. The trigger can be specified as

```

CREATE TRIGGER New_price BEFORE INSERT ON BOOK
FOR EACH ROW
WHEN (new.Price>200)
BEGIN
    new.Price = 60;
END

```

The keyword **new** and **old** can be used to refer to the values after and before the modifications are made. Triggers can be enabled and disabled by using the **ALTER TRIGGER** command. The triggers which are not required can be removed by using the **DROP TRIGGER** command. For example, consider the following statements.

Learn More

Triggers are provided with a separate namespace from procedures, package, relations (sharing the same namespace), which means that triggers can have same name as of a relation or procedure.

```
ALTER TRIGGER New_price DISABLE;
```

```
ALTER TRIGGER New_price ENABLE;
```

```
DROP TRIGGER New_price;
```

The triggers come with lot of advantages. However, triggers must be used with great care as sometimes action of one trigger can lead to the activation of another trigger. Such triggers are known as **recursive triggers**. In the worst situation, it can lead to an infinite chain of triggering. A database system typically limits the length of such chains of triggers to 16 or 32 and considers longer chains of triggering an error.

5.6 ACCESSING DATABASES FROM APPLICATIONS

So far, we have discussed how queries are executed in SQL. In most of the situations, databases are accessed through software programs implementing database applications. These types of software are usually developed in general-purpose programming languages, such as C, Java, COBOL, Pascal, and FORTRAN. And these general-purpose programming languages are known as **host languages**. Various techniques have been developed for accessing databases from other programs. Some of the techniques for using SQL statements in host language are embedded SQL, cursors, dynamic SQL, ODBC, JDBC, and SQLJ. These techniques are discussed in this section.

5.6.1 EmbeddedSQL

For accessing a database from any application program, the SQL statements are embedded within an application program written in host language. The use of SQL statements within a host language program is known as **embedded SQL**. These statements are also known as **static**, that is, they do not change at runtime. SQL statements included in a host language program must be marked so that the pre-processor can handle them before invoking the compiler for a host language. The embedded SQL statement is prefixed with the keywords **EXEC SQL** to distinguish it from the other statements of host language. Prior to compilation, an embedded SQL program is processed by a special pre-processor or pre-compiler, which identifies the SQL statements during program scan, extracts and pass it to the DBMS for processing. The end of SQL statements is identified by encountering a semicolon (;) or a matching **END-EXEC**. For the discussion of embedded SQL, consider C as a host language.

Variables of host language can be referred in SQL statements. Such variables are also known as **host variables** and are prefixed by a colon (:) when they appear in SQL statements and are declared between the commands **EXEC SQL BEGIN DECLARE SECTION** and **EXEC SQL END DECLARE SECTION**. The declaration of these variables is similar to the declaration of variables in C language and is separated by semicolons. These variables are also known as **shared variables** as they are shared by both C language and SQL statements. For example, the declaration of variables corresponding to the attributes of relation **BOOK** will be like as shown here.

```
EXEC SQL BEGIN DECLARE SECTION;
varchar c_ISBN[15], c_book_title[50], c_category[20];
char c_p_id[4];
int c_copyright_date[10], c_year, c_page_count;
float c_price;
int SQLCODE;
char SQLSTATE[6];
EXEC SQL END DECLARE SECTION;
```

Program variables may or may not have same names as that of attributes in a corresponding relation. The SQL data type **NUMERIC** can be mapped to C data type **int** or **float** (if includes decimal portion). The SQL fixed length and variable length strings can be mapped to arrays of characters of type **char** and **varchar**, respectively. The variables **SQLCODE** and **SQLSTATE** are used to communicate errors and exception conditions between the database system and the host language program. The **SQLCODE** is an integer variable, which returns a positive value (like **SQLCODE=100**) when there is no more data in the resultant relation. The **SQLCODE** returns negative value when an error occurs. **SQLSTATE** is a five character code the 6th character is for *null* character in C language. The value **00000** of the variable **SQLSTATE** indicates error free condition. It associates predefined values with several common error conditions.

SQL statements can appear anywhere in the host language program, where other statements of host language can appear. Before executing any SQL statements, the host program must establish a connection with the database as shown here.

```
EXEC SQL CONNECT TO <server_name> AS <connection_name>
AUTHORIZATION <user_name and password>;
```

In this command, **server_name** identifies the server to which a connection is to be established and the **connection_name** is the name provided to the connection. In addition, **user_name** and **password** identifies the authorized user. While programming, more than one connection can be established with only one connection active at a time. When a connection is no longer needed, it can be ended by using the command as shown here.

```
EXEC SQL DISCONNECT <connection_name>;
```

Note that the commands are terminated by semicolon. Assuming that the required connection is already established, the command to insert a tuple in relation using host variables can be specified as

```
EXEC SQL
INSERT INTO BOOK
VALUES(:c_ISBN, :c_book_title, :c_category, :c_price,
:c_copyright_date, :c_year, :c_page_count, :c_p_id);
```

Consider another example to retrieve the details of book with ISBN value stored in the variable **c_ISBN**, the embedded SQL statement can be specified as

```
EXEC SQL
SELECT Book_title, Category, Price, Year, Page_count
INTO :c_book_title, :c_category, :c_price, :c_year, :c_page_count
FROM BOOK
WHERE ISBN=:c_ISBN;
```

In this command, the details of book satisfying the condition is stored in host variables. As a result, only single tuple is retrieved and hence, can be stored in the host variables. However, while programming, more than one tuple satisfying the condition can also be retrieved. To deal with such a situation, the concept of cursors can be used, which is discussed in the following section.

5.6.2 Cursors

Cursor is a mechanism that provides a way to retrieve multiple tuples from a relation and then process each tuple individually in a host program. The cursor is first opened, then processed, and then closed. Cursor can be declared on any relation or any SQL statement that returns a set of tuples. Once the cursor is declared, it can be opened, moved to n^{th} tuple and closed. When the cursor is

opened, it fetches the query result from the database and it is positioned just before the first tuple. In the query result, any individual tuple of a relation can be fetched by pointing cursor to the desired tuple. Sometimes the cursor is opened implicitly (automatically) by the RDBMS especially for the queries returning single tuple. However, for the queries returning more than one tuple, explicit cursors (defined by user) are declared.

In general, to use the explicit cursor, the following steps are performed.

1. **Declare the cursor:** The cursor is declared using the **DECLARE** command.
2. **Open the cursor:** Before using the cursor, it must be opened after it has been declared.
3. **Fetch tuple from the cursor:** After opening the cursor, the tuples associated with the cursor can be processed individually with the help of **FETCH** command.
4. **Close the cursor:** When cursor is not required, it must be closed by using the **CLOSE** command.

The declaration of cursor takes the following form.

```
DECLARE <cursor_name> [INSENSITIVE][SCROLL] CURSOR
[WITH HOLD] FOR <query>
[ORDER BY <attribute_list>]
[FOR READ ONLY | FOR UPDATE [OF <attribute_list>]];
```

The clause **FOR UPDATE OF** is added in the declaration of cursor, indicating that the tuples associated with cursor will be retrieved for updating and the attribute to be updated is specified with it. The **ORDER BY** clause is used to order the tuples in the resultant relation. A cursor can be declared as read only cursor using the clause **FOR READ ONLY**, indicating the tuples retrieved as a result of SQL query cannot be modified. Other keywords that can be used in the declaration of cursor are **SCROLL**, **INSENSITIVE**, and **WITH HOLD**. When the keyword **SCROLL** is specified, then that cursor is scrollable, this means that the **FETCH** command can be used to position the cursor in a flexible manner instead of default sequential access. The other two keywords **INSENSITIVE** and **WITH HOLD** are used to refer the transaction characteristics of database programs.

For example, the program segment to increment the price of books belonging to a given category from relation **BOOK** by the value entered by the user can be written as shown in Figure 5.1.

When the cursor is opened, it is positioned at the beginning of the first tuple. When **FETCH** command is executed, the attribute values in the corresponding tuple are read and stored in respective host variables in the specified order. To read consecutive tuples, the **FETCH** command is executed repeatedly. If the execution of **FETCH** command results in the moving of cursor to the end of last tuple, a positive value (**SQLCODE**>0) is returned in **SQLCODE**. Positive value in **SQLCODE** indicates presence of no more data in the resultant relation. This state of **SQLCODE** is used to terminate the loop. The **WHERE CURRENT OF Cur1** clause in the embedded **UPDATE** command is used to specify that the tuple referred currently by the cursor is the tuple to be updated. After the complete processing of resultant relation, the cursor is closed using **CLOSE** command.

Things to Remember

Once the cursor is opened it cannot be reopened. That is, cursor must be closed before reopening it. Also, the cursor always moves forward and not backwards.

5.6.3 DynamicSQL

Embedded SQL allows the integration of SQL statements within the host language program. Such statements are of static nature, that is, once these statements are written in the program then they cannot be modified at any time. Hence, to specify a new query, new program must be written to accommodate the new query. Dynamic SQL, unlike embedded SQL statements, are generated at the run-time and it is placed as a string in the host variable. The SQL statements created like this

```

printf("Enter the category of book : ");
scanf("%s", c_category);
EXEC SQL DECLARE Cur1 CURSOR FOR
SELECT Book_title, Price, Year
FROM BOOK
WHERE Category=:c_category
ORDER BY Book_title
FOR UPDATE OF Price;
EXEC SQL OPEN Cur1;
EXEC SQL FETCH FROM Cur1 INTO :c_Book_title, :c_Price, :c_Year;
WHILE(SQLCODE ==0)
{
    printf("Enter the increment amount : ");
    scanf("%f", &inc);
    EXEC SQL
        UPDATE BOOK
        SET Price = Price + :inc
        WHERE CURRENT OF Cur1;
    EXEC SQL FETCH FROM Cur1 INTO :c_Book_title, :c_Price, :c_Year;
}
EXEC SQL CLOSE Cur1;

```

Fig. 5.1 *Retrieving tuples using cursor*

are passed to the DBMS for further processing. Dynamic SQL is slower than embedded SQL as it involves time to generate a query also during the runtime. However, it is more powerful than embedded SQL, as queries can be generated at runtime as per varying user requirements.

For example, consider program segment given in Figure 5.2. This sample program allows user to enter the update SQL query to be executed.

```

EXEC SQL BEGIN DECLARE SECTION;
        char sqlquerystring[200];
EXEC SQL END DECLARE SECTION;
printf("Enter the required update query : \n");
scanf("%s", sqlquerystring);
EXEC SQL PREPARE sqlcommand FROM :sqlquerystring;
EXEC SQL EXECUTE sqlcommand;

```

Fig. 5.2 *Program segment using dynamic SQL*

In this code segment, first the string `sqlquerstring` is declared, which is used to hold the SQL query statement entered by the user. After prompting the required query from the user, it is converted into corresponding SQL command by using the `PREPARE` command. This query is executed by using the `EXECUTE` command. The query entered by the user can be in the form of a single string or it can be created by using concatenation of strings.

5.6.4 Open Database Connectivity (ODBC)

To access the database from general-purpose programming language, the application programs needs to set up a connection with the database server. A standard called **Open Database Connectivity (ODBC)** provides an **application programming interface (API)** that the application programs can use to establish a connection with the database. Once the connection is established, the application program can communicate with the database through queries. Most DBMS vendors provide ODBC drivers for their systems.

An application program from the client-site can access several DBMSs by making ODBC API call. The requests from the client program are then processed at the server-sites and the results are sent back to the client program. Consider an example of C code using ODBC API given in Figure 5.3.

```
void Example()
{
    HENV En1;          //environment
    HDBC Cn;           //to establish connection
    RETCODE Err;
    SQLAllocEnv(&En1);
    SQLAllocConnect(En1,&Cn);
    SQLConnect(Cn, "db.onlinebook.edu", SQL_NTS, "John", SQL_NTS, "Passwd", SQL_NTS);
    int c_Price1, c_Price2;
    int L1, L2;
    HSTMT st;
    char *Query = "SELECT MAX(Price), MIN(Price) FROM BOOK
    GROUP BY Category";
    SQLAllocStmt(Cn, &st);
    Err = SQLExecDirect(st, Query, SQL_NTS);
    if (Err == SQL_SUCCESS)
    {
        SQLBindCol(st, 1, SQL_C_INT, &c_Price1, 0, &L1);
        SQLBindCol(st, 2, SQL_C_INT, &c_Price2, 0, &L2);
        while(SQLFetch(st) == SQL_SUCCESS)
        {
            printf("Maximum Price is %d", c_Price1);
            printf("Minimum Price is %d", c_Price2);
        }
    }
    SQLFreeStmt(st, SQL_DROP);
    SQLDisconnect(Cn);
    SQLFreeConnect(Cn);
    SQLFreeEnv(En1);
}
```

Fig. 5.3 *An example of ODBC code*

In the beginning of the program, the variables `En1`, `Cn`, and `Err` of types `HENV`, `HDBC`, and `RETCODE`, respectively are declared. The variable `En1` and `Cn` are used to allocate an SQL environment and database connection handle, respectively. The variable `Err` is used for error detection. Further, the program establishes a connection with the database by using `SQLConnect()` function, which accepts parameters, database connection handle (`Cn`), server (`db.onlinebook.edu`), user identifier (`John`), and the password (`Passwd`). Notice the use of constant `SQL_NTS`, which denotes that the previous argument is a null-terminated string.

After establishing a connection, the program communicates with the database by sending a query to `SQLExecDirect()` function. The attributes of the query result are bounded to corresponding C variables by using `SQLBindCol()` function. The parameters passed to the `SQLBindCol()` are:

- ⇒ **First parameter (`st`):** stores the result of the query
- ⇒ **Second parameter (1 or 2):** determines the location of an attribute in the result of a query
- ⇒ **Third parameter (`SQL_C_INT`):** specifies the required data type conversion of an attribute from SQL to C.
- ⇒ **Fourth parameter (`&c_Price1` or `&c_Price2`):** specifies the address of the C variable where the attribute value is to be stored. Note that the values of last two parameters passed to `SQLBindCol()` function, depends on the data type of an attribute. For example, in case of fixed-length types, such as float or integer the fifth parameter is ignored and negative value in last parameter indicates *null* value in an attribute.

When the resultant tuple is fetched using `SQLFetch()` function, the attribute values of the query are stored in corresponding C variables. The `SQLFetch()` function executes the statement `st` as long as it returns the value `SQL_SUCCESS`. In each iteration of `while` loop, the attribute values for each category are stored in corresponding C variables and are displayed through `printf` statements. The connection must be closed when it is no more required, using `SQLDisconnect()` function. Also, all the resources that are allocated must be freed.

5.6.5 Java Database Connectivity (JDBC)

Accessing a database in Java requires Java Database Connectivity (JDBC). JDBC provides a standard API that is used to access databases, through Java, regardless of the DBMS. All the direct interactions with specific DBMSs are accomplished by DBMS specific driver. The four main components required for the implementation of JDBC are: application, driver manager, data source specific drivers, and corresponding data sources.

An application establishes and terminates the connection with a data source. The main goal of driver manager is to load JDBC drivers and pass JDBC function calls from the application to the corresponding driver. The driver establishes the connection with data source. The driver performs various basic functions like submitting requests and returning results. In addition, the driver translates data, error formats, and error codes from a form that is specific to data source into the JDBC standard. The data source processes commands from the driver and returns the results.

For understanding the connectivity of Java program with JDBC, consider sample program segment given in Figure 5.4.

In this program segment, following steps are taken when writing a Java application program accessing database through JDBC function calls.

1. Appropriate drivers for the database are loaded by using `Class.forName`.
2. The `getConnection()` function of `DriverManager` class of JDBC is used to create the connection object. The first parameter (URL) specifies the machine name (`db.onlinebook.edu`)

```

public static void Sample(String DB_id, String U_id, String Pword)
{
    String URL = "jdbc:oracle:oci8:@db.onlinebook.edu:100:onbook_db";
    Class.forName("oracle.jdbc.driver.OracleDriver");
    Connection Cn=DriverManager.getConnection(URL,U_id, Pword);
    Statement st = Cn.createStatement();
    try
    {
        st.executeUpdate("INSERT INTO AUTHOR VALUES('A010', 'Smith
        Jones', 'Texas')");
    }
    catch(SQLException se)
    {
        System.out.println("Tuple cannot be inserted."+se);
    }
    ResultSet rs = st.executeQuery("SELECT Aname, State from AUTHOR
    WHERE City = 'Seattle'");
    while(rs.next())
    {
        System.out.println(rs.getString(1) + " " + rs.getString(2));
    }
    st.close();
    Cn.close();
}

```

Fig. 5.4 *An example of JDBC code*

where the server runs, the port number (100) used for communication, schema (onbook_db) to be used and the protocol (jdbc:oracle:oci8) used to communicate with the database.

To connect to the database, username and password are also required which are specified by the strings `U_id` and `Pword`, respectively, the other two parameters of `getConnection()` function.

3. A statement handle is created for the connection established which is used to execute an SQL statement. In this example, `st.executeUpdate` is used to execute an `INSERT` statement of SQL. The `try {..} catch{..}` is used to catch any exceptions that may arise as a result of executing this query statement.
4. Another query is executed using the statement `st.executeQuery`. The result of this may consist of set of tuples, which is assigned to `rs` of type `ResultSet`.
5. The `next()` function is used to fetch one tuple at a time from the result set `rs`. The value of attributes of a tuple is retrieved by using the position of the attribute. The attribute `Aname` is at first (1) position and `State` is at second (2) position. The values for the attributes can also be retrieved by using its name as shown here.


```
System.out.println(rs.getString("Aname")+
" " +rs.getString("State"));
```

6. The connection must be closed after it is no more required at the end of procedure.

The special type of statement also known as **prepared statement**, can be used to specify SQL query, in which unknown values are replaced by '?'. The user provides the unknown values later at the run-time. Some database systems compile the query when it is prepared and whenever the query is executed with new values, database uses this compiled form of this query. The `setString()` function is used to specify the values for the parameters. Consider the following statements, to understand the concept of prepared statement.

```
PreparedStatement ps = Cn.prepareStatement("INSERT INTO BOOK
VALUES(?,?,?)");
ps.setString(1, "A010"); //assigns value to first attribute
ps.setString(2, "Smith Jones"); //assigns value to second
//attribute
ps.setString(3, "Texas"); //assigns value to third attribute
ps.executeUpdate();
```

The values can be assigned to the corresponding attributes through variables also instead of using literal values. Prepared statements are preferred in the situations when query uses the values provided by the user. JDBC also provides a `CallableStatement` interface, which can be used for invoking SQL stored procedures and functions. `CallableStatement` is a subclass of `PreparedStatement`. For example, the command to call the procedure, say `Number_Of_Books`, can be specified as

```
CallableStatement cs1 = Cn.prepareCall("{call Number_Of_
Books}");
ResultSet rs = cs1.executeQuery();
```

A stored procedure may contain multiple SQL statements or a series of SQL statements, thus, resulting into many different `ResultSet` objects. In this example, it is assumed that there is only single `ResultSet`.

5.6.6 SQL-Java(SQLJ)

SQLJ is a standard that has been used for embedding SQL statements in Java programming language, which is an object-oriented language like Java. In SQLJ, a pre-processor called "SQLJ translator" converts SQL statements into Java, which can be executed through JDBC interface. Hence, it is essential to install a JDBC driver while using SQLJ. When writing SQLJ applications, regular Java code is written and SQL statements are embedded into it using a set of rules. SQLJ applications are pre-processed using SQLJ translation program that replaces the embedded SQLJ code with calls to an SQLJ library. Any Java compiler can then compile this modified program code. The SQLJ library calls the corresponding driver, which establishes the connection with the database system.

The use of SQLJ improves the productivity and manageability of JAVA code as code becomes compact and no runtime syntax errors occur as SQL statements are checked at compile time. Moreover, it allows sharing of Java variables with SQL statements.

For understanding the coding in SQLJ, consider a sample SQLJ program segment given in Figure 5.5. This program retrieves the book details belonging to a given category.

```
String Book_title; float Price; String Category;
#sql iterator Bk(String Book_title, float Price);
Bk book = {SELECT Book_title, Price INTO :Book_title, :Price FROM
BOOK WHERE Category = :Category};

while(book.next())
{
    System.out.println(book.Book_title() + ", " + book.Price());
}
book.close();
```

Fig. 5.5 An example of SQLJ code

SQLJ statements always begin with the `#sql` keyword. The result of SQL queries is retrieved through the objects of **iterator**, which are basically a type of cursor. The iterator is associated with the tuples and attributes appearing in the query result. An iterator is declared as an instance of iterator class. The usage of an iterator in SQLJ basically goes through following four steps.

1. **Declaration of iterator class:** In the Figure 5.5, the iterator class Bk is declared by using the statement.
`#sql iterator Bk(String Book_title, float Price);`
2. **Creation and initialization of iterator object:** An object of iterator class is created and initialized with a set of tuples returned as a result of SQL query statement.
3. **Accessing the tuples with the help of iterator object:** Iterator is set to the position just before the first row in the result of query, which becomes the current tuple for the iterator. The `next()` function is then applied on the iterator repeatedly to retrieve the subsequent tuples from the result.
4. **Closing the iterator object:** An object of iterator is closed after all the tuples associated with the result of SQL query are processed.

There are two types of iterator classes, namely, *named* and *positional* iterators. In case of **named iterators** both the variable types and the name of each column of the iterator is specified. This helps in retrieving the individual columns by their name. Like, in the Figure 5.5, the sample program to retrieve `Book_title` from the iterator `book`, the expression `book.Book_title()` is used. While, in case of **positional iterators**, only the variable type for each column of iterator is specified. The individual columns of the iterator are accessed using the `FETCH ... INTO` statement like embedded SQL. Both types of iterators have same performance and can be used according to the user requirement.



NOTE *SQLJ is much easier to read than JDBC code. Thus, SQLJ reduces software development and maintenance cost.*

SUMMARY

1. The structured query language (SQL) pronounced as “ess-que-el”, is a language which can be used for retrieval and management of data stored in relational database. It is a non-procedural language as it specifies what is to be retrieved rather than how to retrieve it.
2. Some common relational database management systems that use SQL are: Oracle, Sybase, Microsoft SQL Server, Access, Ingres, etc.
3. SQL provides different types of commands, which can be used for different purposes. These commands are divided into two major categories, namely, data definition language (DDL) and data manipulation language (DML).
4. Data definition language (DDL) commands are used for defining relation schemas, deleting relations, creating indexes, and modifying relation schemas.
5. Present day database systems provide a three level hierarchy for naming different objects of relational database. This hierarchy comprises catalogs, which in turn consists of schemas, and various database objects are incorporated within the schema.
6. SQL supports various data types such as numeric, integer, character, character varying, date and time, Boolean, and timestamp.
7. In addition to built-in data types, user-defined data types can also be created. The user-defined data type can be created using the **CREATE DOMAIN** command.
8. The **CREATE TABLE** command is used to define a new relation, its attributes, and its data types. In addition, various constraints such as key, integrity, and referential constraints along with the restrictions on attribute domains and null values can also be specified within the definition of a relation.
9. **ALTER TABLE** command is used to change the structure of a relation. The user can add, modify, delete, or rename an attribute. The user can also add or delete a constraint.
10. The **DROP TABLE** command is used to remove an already existing relation, which is no more required as a part of a database.
11. DML commands are used for retrieving and manipulating data stored in the database. The basic retrieval and manipulating commands are **SELECT**, **INSERT**, **UPDATE**, and **DELETE**.
12. The criteria to determine the tuples to be retrieved is specified by using the **WHERE** clause.
13. A tuple variable is associated with a particular relation and is defined in the **FROM** clause.
14. SQL provides **LIKE** operator, which allows to specify comparison conditions on only parts of the strings. Different patterns are specified by using two special wildcard character, namely, per cent (%) and underscore (_).
15. There are various set operations that can be performed on relations, like, union, intersection, and difference. In SQL, these operations are implemented using **UNION**, **INTERSECT**, and **MINUS** operations, respectively.
16. SQL provides the **ORDER BY** clause to arrange the tuples of relation in some particular order. The tuples can be arranged on the basis of the values of one or more attributes.
17. The **INSERT** command is used to add a single tuple at a time in a relation.
18. The **UPDATE** command is used to make changes in the values of the attributes of the relations. The **DELETE** command is used to remove the tuples from the relation, which are no more required as a part of a relation. Tuples can be deleted from only one relation at a time.
19. SQL provides five built-in aggregate functions, namely, **SUM**, **AVG**, **MIN**, **MAX**, and **COUNT**.
20. The **GROUP BY** clause can be used in **SELECT** command to divide the relation into groups on the basis of values of one or more attributes. Conditions can be placed on the groups using **HAVING** clause.
21. A query that combines the tuples from two or more relations is known as join query. In such type of queries, more than one relation is listed in the **FROM** clause.

22. The query defined in the **WHERE** clause of another query is known as nested query or subquery. The query in which another query is nested is known as enclosing query.
23. Some of the advanced features of SQL that help in specifying complex constraints and queries efficiently are assertions and views.
24. Assertion is a condition that must always be satisfied by the database.
25. View is a virtual relation, whose contents are derived from already existing relations and it does not exist in physical form. Certain database systems allow views to be physically stored, also known as materialized views.
26. The two standard features of SQL that are related to programming in database are stored procedures and triggers.
27. Stored procedures are procedures or functions that are stored and executed by the DBMS at the database server machine.
28. A trigger is a type of stored procedure that is executed automatically when some database related events like, insert, update, delete, etc., occur.
29. Various techniques have been developed for accessing databases from other programs. Some of the techniques for using SQL statements in host language are embedded SQL, cursors, dynamic SQL, ODBC, JDBC, and SQLJ.
30. The use of SQL statements within a host language is known as embedded SQL. The embedded SQL statement is prefixed with the keywords **EXEC SQL** to distinguish it from the other statements of host language.
31. Cursor is a mechanism that provides a way to retrieve multiple tuples from a relation and then process each tuple individually in a host program. The cursor is first opened, then processed, and then closed.
32. Dynamic SQL, unlike embedded SQL statements, are generated at the run-time and it is placed as a string in the host variable.
33. A standard called Open Database Connectivity (ODBC) provides an application programming interface (API) that the application programs can use to establish a connection with the database.
34. Accessing a database in Java requires Java Database Connectivity (JDBC). JDBC provides a standard API that is used to access databases, through Java, regardless of the DBMS.
35. SQLJ is a standard that has been used for embedding SQL statements in Java programming language, which is object-oriented language like Java. In SQLJ, a pre-processor called SQLJ translator converts SQL statements into Java, which can be executed through JDBC interface.

◀ ● KEY TERMS ● ▶

- Data definition language (DDL)
- Data manipulation language (DML)
- **CREATE SCHEMA**
- **VARCHAR**
- **NUMERIC**
- **CREATE DOMAIN**
- **CREATE TABLE**
- **DESCRIBE**
- **PRIMARY KEY** constraint
- **UNIQUE** constraint
- **CHECK** constraint
- **CONSTRAINT**
- **NOT NULL** constraint
- **FOREIGN KEY** constraint
- **ALTER TABLE**
- **ADD**
- **MODIFY**
- **DROP COLUMN**
- **DROP CONSTRAINT**
- **RENAME COLUMN**
- **DROP TABLE**
- **SELECT**
- **DISTINCT** keyword
- **WHERE** clause

- BETWEEN operator
- IN operator
- AS clause
- LIKE operator
- UNION operation
- INTERSECT operation
- MINUS operation
- ORDER BY clause
- INSERT
- UPDATE
- DELETE
- IS NULL
- AVG
- MIN
- MAX
- SUM
- COUNT
- GROUP BY clause
- HAVING clause
- Join query
- Equi-join query
- Natural join query
- Nested query
- ANY operator
- ALL operator
- EXISTS operator
- Assertion
- CREATE ASSERTION
- View
- CREATE VIEW
- Materialized views
- Stored procedures
- CREATE PROCEDURE
- CREATE FUNCTION
- SQL/PSM
- Trigger
- CREATE TRIGGER
- Row level trigger
- Statement level trigger
- REFERENCING NEW ROW AS clause
- REFERENCING OLD ROW AS clause
- ALTER TRIGGER
- DROP TRIGGER
- Embedded SQL
- Cursor
- Dynamic SQL
- Open Database Connectivity (ODBC)
- Application programming interface (API)
- Java Database Connectivity (JDBC)
- Prepared statement
- SQL-Java (SQLJ)
- Iterator
- Named iterators
- Positional iterators

● EXERCISES ●

A. Multiple Choice Questions

1. DML provides commands that can be used to
 - (a) Create
 - (b) Modify
 - (c) Delete database objects
 - (d) Delete data from a database
2. Which of these constraints ensures that the foreign key value in the referencing relation must exist in the primary key attribute of the referenced relation?
 - (a) Primary key
 - (b) Reference key
 - (c) Foreign key
 - (d) Unique key

3. Which of these keywords is used to eliminate duplicate tuples from the resultant relation?
 - (a) DELETE
 - (b) DISTINCT
 - (c) EXISTS
 - (d) NON DUPLICATE
4. Which of these operators selects values that match any value in a given list of values?
 - (a) BETWEEN
 - (b) LIKE
 - (c) IN
 - (d) ~~one~~ of ~~these~~
5. Which of these characters is used to match substring with any number of characters?
 - (a) %
 - (b) *
 - (c) ?
 - (d) _
6. Which of these operations is used to retrieve those tuples present in one relation, and not present in other relation?
 - (a) UNION
 - (b) INTERSECT
 - (c) MINUS
 - (d) DIFFERENCE
7. Which of these commands can be used to make changes in the name of publishers in a relation PUBLISHER?
 - (a) UPDATE
 - (b) ALTER
 - (c) MODIFY
 - (d) ~~one~~ of ~~sethe~~
8. Which of the following is true statement?
 - (a) If the value of A is *true* or *unknown*, the result is *unknown*
 - (b) If the value of A is *false*, the result is *false*
 - (c) If the value of A is *false* or *unknown*, the result is *unknown*
 - (d) If the value of A is *true*, the result is *unknown*
9. Which of these clauses is used to restrict groups returned by the Group By clause?
 - (a) DISTINCT
 - (b) WHERE
 - (c) EXISTS
 - (d) HAVING
10. Which of these mechanisms provides a way to retrieve multiple tuples from a relation and then process each tuple individually in a host program?
 - (a) Assertions
 - (b) ~~ursors~~
 - (c) Triggers
 - (d) ~~one~~ of ~~sethe~~

B. Fill in the Blanks

1. The _____ command is used to define a new relation, its attributes and its data types.
2. The _____ constraint ensures that the set of attributes have unique values, that is, no two tuples can have same value in the specified attributes.
3. The _____ clause, prevents a relation to be dropped if it is referenced by any of the constraints or views.
4. A query that combines the tuples from two or more relations is known as _____.
5. _____ operator evaluates to *true* if a subquery returns at least one tuple as a result otherwise, it returns *false* value.

6. _____ are type of stored procedures which are automatically invoked and are needed to perform complex data verification operations.
7. Certain database systems allow views to be physically stored, which are known as _____.
8. _____, unlike embedded SQL statements, are generated at the run-time and it is placed as a string in the host variable.
9. _____ provides a standard API that is used to access databases, through Java, regardless of the DBMS.
10. In case of SQLJ, the result of SQL queries is retrieved through the objects of _____, which are basically a type of cursor.

C. Answer the Questions

1. What is SQL? What are the two major categories of SQL commands? Explain them.
2. What type of information can be specified about a relation in DDL?
3. Discuss the concept of schema and catalog in SQL.
4. Discuss common data types supported by standard SQL.
5. How user-defined data types can be created in SQL? Give example.
6. Give purpose and syntax of **CREATE TABLE** and **DESCRIBE** command.
7. Discuss different types of constraints along with syntax that can be specified while creating a relation in SQL.
8. Give purpose, syntax, and example of the following commands along with their various clauses:
(a) **ALTER TABLE** (b) **DROP TABLE** (c) **SELECT**
(d) **INSERT** (e) **UPDATE** (f) **DELETE**
9. Which operator of SQL is used to specify string patterns in the queries? Explain in detail with examples.
10. Discuss how set operations can be implemented in SQL.
11. Which clause of **SELECT** command can be used to change the order of tuples in a relation? Explain with syntax and example.
12. How *null* values are represented and handled in SQL queries?
13. What is the role of aggregate functions in SQL queries?
14. Explain how the **GROUP BY** clause works. What is the difference between **WHERE** and **HAVING** clauses? Explain them with the help of an example.
15. How queries based on joins are expressed in SQL?
16. What do you understand by nested queries? Explain with example.
17. What are assertions? What is the utility of assertions?
18. What is view in SQL? How are assertions different from views? Compare assertions and views with the help of example.
19. What are the conditions under which views can be updated?
20. What are stored procedures? When are they beneficial? Give syntax and example of creating a procedure and function in SQL.
21. Discuss about triggers. How they are created? How do triggers offer a powerful mechanism for dealing with the changes to database? Explain with example.

22. What do you mean by embedded SQL? How variables are declared in embedded SQL? Explain with examples.
23. What problem is faced while using embedded SQL and how it is addressed by cursors? Explain with example.
24. What is dynamic SQL and how is it different from embedded SQL?
25. What is the purpose of ODBC in SQL? How it is implemented? Explain with example.
26. What is the purpose of JDBC in SQL? How it is implemented? Explain with example.
27. Differentiate between ODBC and JDBC.
28. What is SQLJ? What are the requirements of SQLJ? Briefly describe the working of SQLJ.
29. Give steps involved in the implementation of iterators in SQLJ. Discuss two types of iterators available in SQLJ.

D. Practical Questions

1. Consider the relation schemas given in question 1–4 (Practical questions) of Chapter 04. Write appropriate DDL commands to define these relation schemas along with the required constraints defined. Use INSERT command to add five tuples in each relation.
2. Express queries given in questions 1–4 (Practical questions) of Chapter 04 in SQL using various DML commands.
3. Consider the following relational scheme:

```
BOOKS(Book_Id, B_name, Author, Purchase_date, Cost)
MEMBERS(Member_Id, M_name, Address, Phone, Birth_date)
ISSUE_RETURN(Book_Id, Member_Id, Issue_date, Return_date)
```

Specify the following queries in SQL.

- (a) Find the names of all those books that have not been issued.
 - (b) Display the **Member_Id** and the number of books issued to that member. (Assume that if a book in **ISSUE_RETURN** relation does not have a **Return_date**, then it is issued.)
 - (c) Find the book that has been issued the maximum number of times.
 - (d) Display the names and authors of books that have been issued at any time to a member whose **Member_Id** is *ab*.
 - (e) List the **Book_Id** of those books that have been issued to any member whose date of birth is less than *01-01-1985*, but have not been issued to any member having the birth date equal to or greater than *01-01-1985*.
4. Specify the following queries on *Online Book* database in SQL.
 - (a) In **BOOK** relation, increase the price of all the books belonging to novel category by 10%.
 - (b) In **BOOK** relation, increase the price of all the books published by *Hills Publication* by 5%.
 - (c) In **PUBLISHER** relation, change the phone of *Wesley Publications* to 9256774.
 - (d) In **REVIEW** relation, increase the rating of all the books written by the authors *A003* by one.
 - (e) Calculate and display average, maximum, and minimum price of book from each category.
 - (f) Calculate and display average, maximum, and minimum price of book from each publisher.
 - (g) Delete details of all the books having page count less than 100.
 - (h) Retrieve details of all the books whose name begins with character *D*.

- (i) Retrieve details of all the publishers located in state *Georgia*.
 - (j) Retrieve details of all the authors residing in the city *New York* and whose name begins with character *J*.
 - (k) Retrieve book title, name of publisher, and author name of all language books.
 - (l) Retrieve book details having price equals to average price of all the books.
 - (m) Retrieve author details residing in the same city as *Lewis Ian*.
5. Give commands to create views based on the following queries of *Online Book* database and state whether they are updateable.
- (a) A view containing details of all the books belonging to textbook category.
 - (b) A view containing details of all the books published by the publisher *Bright Publications* and price greater than \$30.
 - (c) A view containing details of all the textbooks written by author *Charles Smith*.
 - (d) A view containing details of all the books having page count 600 and published in the year 2004.
6. Create a procedure to update the value of page count of a book with a given ISBN.
7. Create a procedure to update the value of phone of a publisher with a given publisher ID.
8. Create a function that returns the price of a book with a given ISBN.
9. Create a function that searches a book with a given ISBN and returns the value *Old* if it is published before year 2000 and *New* if it is published in or after year 2000.
10. Create a trigger for changing the value of **Page_count** attribute to a default value 100, if the value entered by the user exceeds the upper limit of 1500 in case of insertion in relation **BOOK**.
11. Specify embedded SQL statement to retrieve the detail of author with **A_ID** value stored in the variable **c_AID**.
12. Write a program segment which uses cursor to update the page count of books belonging to a given publisher ID from relation **BOOK** by the value entered by the user.
13. Write an ODBC code for displaying publisher along with maximum price of books published by them from **BOOK** relation.
14. Write JDBC code for displaying title, price, and page count of books belonging to language book category.
15. Write SQLJ code for retrieving the book details belonging to a given publisher.
16. Write SQLJ code for retrieving the publisher details publishing novels.

RELATIONAL DATABASE DESIGN

After reading this chapter, the reader will understand:

- Features of a relational database to qualify as a good relational database
- Decomposition that resolves the problem of redundancy and null values
- The concept of functional dependencies that plays a key role in developing a good database schema
- Introduction of normal forms and the process of normalization
- Normal forms based on functional dependencies that are 1NF, 2NF, 3NF, BCNF
- Additional properties of decomposition, namely, lossless-join and dependency preservation property.
- Comparison of 3NF and BCNF
- Higher normal forms 4NF and 5NF that are based on multivalued and join dependencies
- The process of denormalization

The primary issue in designing any database is to decide the suitable logical structure for the data, that is, what relations should exist and what attributes should they have. For a smaller application, it is feasible for the designer to decide directly the appropriate logical structure for the data. However, for a large application where the database gets more complex, to determine the quality or goodness of a database, some formal measures or guidelines must be developed.

Normalization theory provides the guidelines to assess the quality of a database in designing process. The normalization is the application of set of simple rules called **normal forms** to the relation schema. In order to determine whether a relation schema is in desirable normal form, information about the real world entity that is modeled in a database is required. Some of this information exists in E-R diagram. The additional information is given by constraints like functional dependencies and other types of data dependencies (multivalued dependencies and join dependencies).

This chapter begins with a discussion of various features of good relational database. It then introduces the concept of functional dependency, which plays a key role in designing the database. Finally, it discusses various normal forms on the basis of functional dependencies and other data dependencies.

6.1 FEATURES OF GOOD RELATIONAL DESIGN

Designing of relation schemas is based on the conceptual design. The relation schemas can be generated directly using conceptual data model such as ER or enhanced ER (EER) or some other conceptual data model. The goodness of the resulting set of schemas depends on the designing of the data model. All the relations that are obtained from data model have basic properties, that is, relations having no duplicate tuples, tuples having no particular ordering associated with them and each element in the relation being atomic. However, to qualify as a good relational database design, it should also have following features.

Minimum Redundancy

A relational database should be designed in such a way that it should have minimum redundancy. Redundancy refers to the storage of same data in more than one location. That is, the same data is repeated in multiple tuples in the same relation or multiple relations. The main disadvantage of redundancy is the wastage of storage space. For example, consider the modified form of *Online Book* database in which information regarding books and publishers is kept in same relation, say, **BOOK_PUBLISHER** which is defined on schema **BOOK_PUBLISHER**(ISBN, Book_title, P_ID, Pname, Phone). The primary key for this relation is ISBN. Some tuples of the relation **BOOK_PUBLISHER** are shown in Figure 6.1.

ISBN	Book_title	P_ID	Pname	Phone
001-987-760-9	C++	P001	Hills Publications	7134019
001-354-921-1	Ransack	P001	Hills Publications	7134019
001-987-650-5	Differential Calculus	P001	Hills Publications	7134019
002-678-980-4	DBMS	P002	Sunshine Publishers Lt d.	6548909
002-678-880-2	Call Away	P002	Sunshine Publishers Ltd.	6548909
004-765-409-5	UNIX	P003	Bright Publications	7678985
004-765-359-3	Coordinate Geometry	P003	Bright Publications	7678985
003-456-433-6	Introduction to German Language	P004	Paramount Publishing House	9254834
003-456-533-8	Learning French Language	P004	Paramount Publishing House	9254834

Fig. 6.1 Instance of relation **BOOK_PUBLISHER**

In this relation, there is repetition of names and phone numbers of publishers with all the books they have published. Now, assume that 100 books are published by any one publisher, and then all the information related to that publisher will be repeated unnecessarily over 100 tuples. In addition, due to redundancy, certain update anomalies can arise, which are as follows:

- ⇒ **Insertion anomaly:** It leads to a situation in which certain information cannot be inserted in a relation unless some other information is stored. For example, in **BOOK_PUBLISHER** relation, information of a new publisher cannot be inserted unless he has not published any book. Similarly, information of a new book cannot be inserted unless information regarding publisher is not known.
- ⇒ **Deletion anomaly:** It leads to a situation in which deletion of data representing certain information results in losing data representing some other information that is associated with it. For example, if the tuple with a given ISBN, say, 003-456-433-6 is deleted, the only information about the publisher *P004* associated with that ISBN is also lost.
- ⇒ **Modification anomaly:** It leads to a situation in which repeated data changed at one place results in inconsistency unless the same data is also changed at other places. For example, change in the name of a publisher, say, *Hills Publications* to *Hills Publishers Pvt. Ltd.* needs modification in multiple tuples. If the name of a publisher is not modified at all places, same **P_ID** will show two different values for publisher name in different tuples.



NOTE *Modification anomaly cannot occur in case of a single relation as a single SQL query updates the entire relation. However, it can be a serious problem, if the data is repeated in multiple relations.*

Fewer Null Values in Tuples

Sometimes, there may be a possibility that some attributes do not apply to all tuples in a relation. That is, values of all the attributes of a tuple are not known (as discussed in insertion anomaly). In that case, *null* value can be stored in those tuples. For example, in relation **BOOK_PUBLISHER**, detail of a new book without having information regarding publisher can be inserted by placing *null* values in

all the attributes of publisher. However, *null* values do not provide complete solution as they cannot be inserted in the primary key attributes and the attributes for which **not null** constraint is specified. For example, detail of a new publisher who has not published any book cannot be inserted, as *null* value cannot be inserted in the attribute ISBN since it is a primary key. Moreover, *null* values waste storage space and may lead to some other problems such as interpreting them, accounting for them while applying aggregate functions such as COUNT or AVG, etc.

In a good database, the attributes whose values may frequently be *null* are avoided. If *null* values are unavoidable, they are applied in exceptional cases only. Thus, all the problems related to *null* values are minimized in a good database.

6.2 DECOMPOSITION

While designing a relational database schema, the problems that are confronted due to redundancy and *null* values can be resolved by decomposition. In decomposition, a relation is replaced with a collection of smaller relations with specific relationship between them. Formally, the **decomposition** of a relation schema R is defined as its replacement by a set of relation schemas such that each relation schema contains a subset of the attributes of R.

For example, the relation schema BOOK_PUBLISHER represents a bad design as it permits redundancy and null values, hence, it is undesirable. Thus, the schema of this relation requires to be modified so that it has less redundancy of information and fewer null values. The undesirable features from the relation schema BOOK_PUBLISHER can be eliminated by decomposing it into two relation schemas as given here.

BOOK_DETAIL (ISBN, Book_title, P_ID) and
PUBLISHER_DETAIL (P_ID, Pname, Phone)

The instances corresponding to the relation schemas BOOK_DETAIL and PUBLISHER_DETAIL are shown in Figure 6.2.

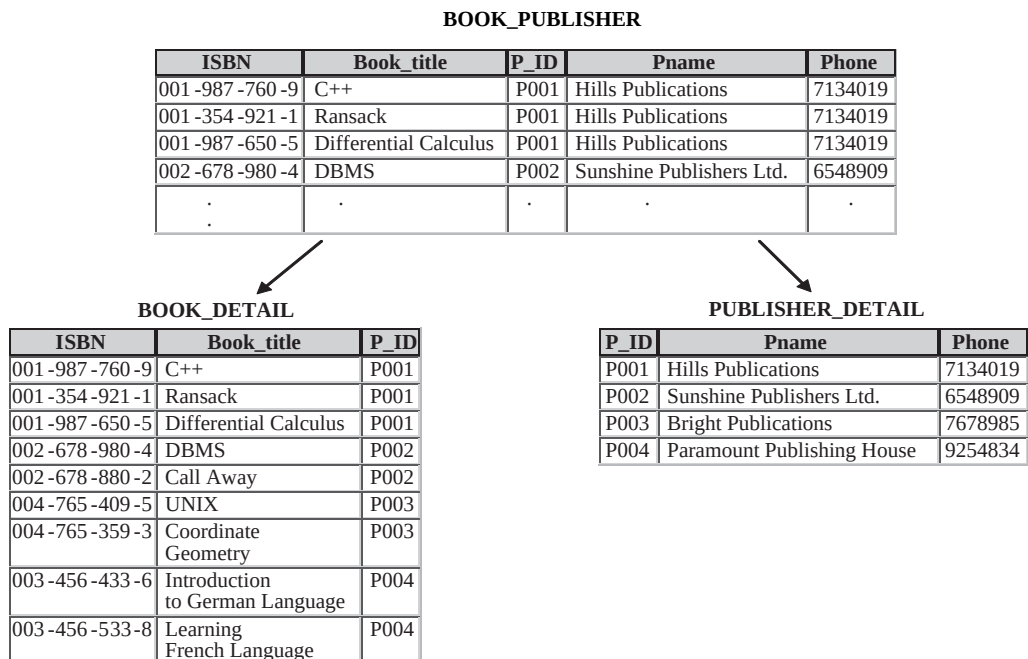


Fig. 6.2 Decomposition

From Figure 6.2, it is apparent that decomposition is actually a process of projection on a relation. Here, both the relations **BOOK_DETAIL** and **PUBLISHER_DETAIL** are the projections of original relation **BOOK_PUBLISHER**.

Now, a tuple for a new publisher can be inserted easily in relation **PUBLISHER_DETAIL**, even if no book is associated with that particular **P_ID** in relation **BOOK_DETAIL**. Similarly, a tuple for a particular **P_ID** in relation **PUBLISHER_DETAIL** can be deleted without losing any information (taking foreign key constraint into account). Moreover, name or phone for a particular **P_ID** can be changed by updating a single tuple in relation **PUBLISHER_DETAIL**. This is more efficient than updating several tuples and the scope for inconsistency is also eliminated.

During decomposition, it must be ensured that each attribute in **R** must appear in at least one relation schema **R_i** so that no attributes are lost. Formally,

$$\cup R_i = R$$

This is called **attribute preservation** property of decomposition.



NOTE There are two additional properties of decomposition, namely, *lossless-join* and *dependency preservation* property which are discussed in Section 6.5.

For smaller applications, it may be feasible for a database designer to decompose a relation schema directly. However, in case of large real-world databases that are usually highly complex, such a direct decomposition process is difficult as there can be different ways in which a relational schema can be decomposed. Further, in some cases, decomposition may not be required. Thus, in order to determine whether a relation schema should be decomposed and which relation schema may be better than another, some guidelines or formal methodology is needed. To provide such guidelines, several normal forms have been defined. These normal forms are based on the notion of *functional dependencies* that are one of the most important concepts in relational schema design.

6.3 FUNCTIONAL DEPENDENCIES

Functional dependencies are the special forms of integrity constraints that generalize the concept of keys. A given set of functional dependencies is used in designing a relational database in which most of the undesirable properties are not present. Thus, functional dependencies play a key role in developing a good database schema.

Definition: Let **X** and **Y** be the arbitrary subsets of set of attributes of a relation schema **R**, then an instance **r** of **R** satisfies functional dependency (FD) **X** → **Y**, if and only if for any two tuples **t₁** and **t₂** in **r** that have **t₁[X] = t₂[X]**, they must also have **t₁[Y] = t₂[Y]**. That means, whenever two tuples of **r** agree on their **X** value, they must also agree on their **Y** value. Less formally, **Y** is functionally dependent on **X**, if and only if each **X** value in **r** is associated with it precisely one **Y** value.

The statement **X** → **Y** is read as **Y** is functionally dependent on **X** or **X** determines **Y**. The left and right sides of a functional dependency are called **determinant** and **dependent**, respectively.

Things to Remember

The definition of FD does not require the set **X** to be minimal; the additional minimality condition must be satisfied for **X** to be a primary key. That is, in an FD **X** → **Y** that holds on **R**, **X** represents a key if **X** is minimal and **Y** represents the set of all other attributes in the relation. Thus, if there exists some subset **Z** of **X** such that **Z** → **Y**, then **X** is a superkey.

Consider the relation schema **BOOK_REVIEW_DETAIL**(ISBN, Price, Page_count, R_ID, City, Rating) that represents review details of the books by the reviewers. The primary key of this relation is the combination of ISBN and R_ID. Note that the same book can be reviewed by different reviewers and same reviewer can review different books. The instance of this relation schema is shown in Figure 6.3.

ISBN	Price	Page_count	R_ID	City	Rating
001-987-760-9	25	800	A002	Atlanta	6
001-987-760-9	25	800	A008	Detroit	7
001-354-921-1	22	200	A006	Albany	7
002-678-980-4	35	860	A003	Los Angeles	5
002-678-980-4	35	860	A001	New York	2
002-678-980-4	35	860	A005	Juneau	7
004-765-409-5	26	550	A003	Los Angeles	4
004-765-359-3	40	650	A007	Austin	3
003-456-433-6	30	500	A010	Virginia Beach	5
001-987-650-5	35	450	A009	Seattle	8
002-678-880-2	25	400	A006	Albany	4
003-456-533-8	30	500	A004	Seattle	9

Fig. 6.3 An instance of **BOOK_REVIEW_DETAIL** relation

In this relation, the attribute **Price** is functionally dependent on the attribute **ISBN** ($\text{ISBN} \rightarrow \text{Price}$), as tuples having same value for the **ISBN** have the same value for the **Price**. For example, the tuples having ISBN 001-987-760-9 have the same value 25 for **Price**. Similarly, the tuples having ISBN 002-678-980-4 have the same value 35 for **Price**. Other FDs such as $\text{ISBN} \rightarrow \text{Page_count}$, $\text{Page_count} \rightarrow \text{Price}$, $\text{R_ID} \rightarrow \text{City}$ are also satisfied. In addition, one more functional dependency $\{\text{ISBN}, \text{R_ID}\} \rightarrow \text{Rating}$ is also satisfied. That is, the attribute **Rating** is functionally dependent on composite attribute $\{\text{ISBN}, \text{R_ID}\}$.

While designing database, we are mainly concerned with the functional dependencies that must hold on relation schema **R**. That is, the set of FDs which are satisfied by all the possible instances that a relation on relation schema **R** may have. For example, in relation **BOOK_REVIEW_DETAIL**, the functional dependency $\text{Page_count} \rightarrow \text{Price}$ is satisfied. However, in real-world, the price of two different books having same number of pages can be different. So, it may be possible that at some time in an instance of relation **BOOK_REVIEW_DETAIL**, FD $\text{Page_count} \rightarrow \text{Price}$ is not satisfied. For this reason, $\text{Page_count} \rightarrow \text{Price}$ cannot be included in the set of FDs that hold on **BOOK_REVIEW_DETAIL** schema. Thus, a particular relation at any point in time may satisfy some FD, but it is not necessary that FD always hold on that relation schema.

The FDs can be represented conveniently by using FD diagram. In FD diagram, the attributes are represented as rectangular box. The arrow out of attribute depicts that on this attribute, the attributes to which arrow points are functionally dependent. The FD diagram for relation schema **BOOK_REVIEW_DETAIL** is shown in Figure 6.4.

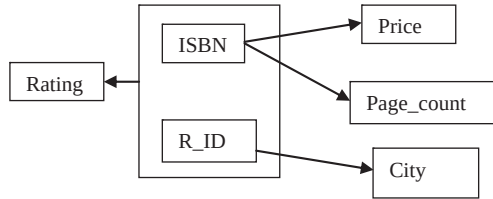


Fig. 6.4 FD diagram for *BOOK_REVIEW_DETAIL*

As stated earlier, while designing a database, the FDs that must hold on the relational schemas are specified. However, for a given relation schema, the complete set of FDs can be very large. For example, once again, consider the relation schema *BOOK_REVIEW_DETAIL*. In addition to the functional dependencies discussed earlier, some other FDs that hold on this relation schema are

$$\begin{aligned} \{R_ID, ISBN\} &\rightarrow City \\ \{R_ID, ISBN\} &\rightarrow \{City, Rating\} \\ \{R_ID, ISBN\} &\rightarrow ISBN \\ \{R_ID, ISBN\} &\rightarrow \{R_ID, ISBN, City, Rating\} \end{aligned}$$

This large set of FDs can reduce the efficiency of database system. As each FD represents certain integrity constraint, the database system must ensure that any of the FDs should not be violated while performing any operation on the database. To reduce the effort spent in checking for violations, there should be some way to reduce the size of that set of FDs. That means for a particular set of FDs, say F_1 , it is required to find some other set, say F_2 , which is smaller than F_1 and every FD in F_1 is implied by FDs in F_2 .

6.3.1 Trivial and Non-Trivial Dependencies

A dependency is said to be a **trivial dependency** if it is satisfied by all relations. For example, $X \rightarrow X$ is satisfied by all relations having attribute X . Similarly, $XY \rightarrow X$ is satisfied by all relations having attribute X . Generally, an FD $A \rightarrow B$ is trivial, if and only if $B \subseteq A$. Consider *BOOK_REVIEW_DETAIL* relation (see Figure 6.3). In this relation, the FDs like $ISBN \rightarrow ISBN$ and $\{R_ID, ISBN\} \rightarrow ISBN$ are trivial dependencies.

The dependencies, which are not trivial, are **non-trivial dependencies**. These are the dependencies that correspond to the essential integrity constraints. Thus, in practice, only non-trivial dependencies are considered. However, while dealing with the formal theory, all dependencies, trivial as well as non-trivial dependencies, have to be accounted.

6.3.2 Closure of a Set of Functional Dependencies

For a given set F of functional dependencies, some other dependencies also hold. That is, some functional dependencies are implied by F . For example, consider a relation schema $R(A, B, C)$, such that both the FDs $A \rightarrow B$ and $B \rightarrow C$ hold for R . Then, FD $A \rightarrow C$ also holds on R as C is dependent on A transitively, via B . Thus, it can be said that FD $A \rightarrow C$ is implied.

The set of all FDs that are implied by a given set F of FDs is called the **closure of F** , denoted as F^+ . For a given F (if it is small), F^+ can be computed directly. However, if F is large, then some rules or algorithms are needed to compute F^+ . Armstrong proposed a set of inference rules or axioms (also known as **Armstrong's axioms**) to find logically implied functional dependencies. These rules are applied repeatedly for a given set F of functional dependencies, so that all of F^+ can be inferred. Let A , B , C , and D be the arbitrary subsets of the set of attributes of a relation schema R and $A \cup B$ is denoted by AB , then these axioms can be stated as

- ⇒ **Reflexivity rule:** If $B \subseteq A$, then $A \rightarrow B$ holds.
- ⇒ **Augmentation rule:** If $A \rightarrow B$ holds, then $AC \rightarrow BC$ holds.
- ⇒ **Transitivity rule:** If both $A \rightarrow B$ and $B \rightarrow C$ hold, then $A \rightarrow C$ holds.

The axioms are **complete** as for a given set F of FDs, all FDs implied by F can be inferred. The axioms are also **sound** as no additional FDs or incorrect FDs can be deduced from F using the rules. All the three axioms are proved for completeness and soundness. Though Armstrong's axioms are complete and sound, computing F^+ using them directly is a complicated task. Thus, to simplify the task of computing F^+ from F , additional rules are derived from Armstrong's axioms. These rules are as follows:

- ⇒ **Decomposition rule:** If $A \rightarrow BC$ then $A \rightarrow B$ holds and $A \rightarrow C$ holds.
- ⇒ **Union rule:** If $A \rightarrow B$ and $A \rightarrow C$ hold, then $A \rightarrow BC$ holds.
- ⇒ **Pseudotransitivity rule:** If $A \rightarrow B$ holds and $BC \rightarrow D$ holds, then $AC \rightarrow D$ holds.

For example, consider a relation schema $R(A, B, C, D, E, G)$ and the set F of functional dependencies $\{A \rightarrow B, BC \rightarrow D, D \rightarrow E, D \rightarrow G\}$. Some of the members of F^+ of the given set F are

- ⇒ $AC \rightarrow D$. Since $A \rightarrow B$ and $BC \rightarrow D$ (Pseudotransitivity rule)
- ⇒ $BC \rightarrow E$. Since $BC \rightarrow D$ and $D \rightarrow E$ (Transitivity rule)
- ⇒ $D \rightarrow EG$. Since $D \rightarrow E$ and $D \rightarrow G$ (Union rule)

6.3.3 Closure of a Set of Attributes

In practice, there is little need to compute the closure of set of FDs. Generally, it is required to compute certain subset of the closure (subset consisting of all FDs with a specified set Z of attributes as determinant). However, to compute subset of closure, computing F^+ is not an efficient process as F^+ can be large. Moreover, computing F^+ for a given set F of FDs involves applying Armstrong's axioms repeatedly until they stop generating new FDs. So, some algorithm must be devised that can compute certain subset of the closure of the given set F of FDs without computing F^+ .

For a given relation schema R , a set Z of attributes of R and the set F of functional dependencies that hold on R , the set of all attributes of R that are functionally dependent on Z (known as **the closure of Z under F** , denoted by Z^+) can be determined using the algorithm as shown in Figure 6.5.

```

Z+ := Z;           //Initialize result to Z
while(change in Z+) do // until change in result
    for each FD X → Y in F do //inner loop
        begin
            if X ⊆ Z+ //check if left hand side is subset
                //of closure(result)
            then Z+ := Z+ ∪ Y; //include the value on right hand
                                // side in result
        end
    end
end

```

Fig. 6.5 Algorithm to Compute Z^+ , the closure of Z under F

For example, consider a relation schema $R(A, B, C, D, E, G, H)$ and the set F of functional dependencies $\{A \rightarrow B, BC \rightarrow DE, AEG \rightarrow H\}$. The steps to compute the closure $\{AC\}^+$ of the set of attributes $\{A, C\}$ under the set F of FDs (using algorithm given in Figure 6.5) are

1. Initialize the result Z^+ to $\{A, C\}$.
2. On the execution of while loop first time, inner loop is repeated three times, once for each FD.
3. On the first iteration of the loop for $A \rightarrow B$, since left hand side A is subset of Z^+ , include B in Z^+ . Thus, $Z^+ = \{A, B, C\}$.
4. On the second iteration of the loop for $BC \rightarrow DE$, include DE in Z^+ . Now $Z^+ = \{A, B, C, D, E\}$.
5. On the third iteration of the loop for $AEG \rightarrow H$, Z^+ remains unchanged, since AEG is not subset of Z^+ .
6. On the execution of while loop second time, inner loop is repeated three times again. In all iterations, no new attribute is added in Z^+ , hence, the algorithm terminates. Thus, $\{AC\}^+ = \{A, B, C, D, E\}$.

Besides, computing subset of closure, the closure algorithm has some other uses which are as follows.

- $\Rightarrow$ To determine if a particular FD, say $X \rightarrow Y$ is in the closure F^+ of F , without computing the closure F^+ . This can be done by simply computing X^+ by using closure algorithm and then checking if $Y \subseteq X^+$.
- $\Rightarrow$ To test if a set of attributes A is a superkey of R . This can be done by computing A^+ , and checking if A^+ contains all the attributes of R .

6.3.4 Equivalence of Sets of Functional Dependencies

Before discussing the equivalence sets of dependencies, one must be familiar with the term cover.

A set F_2 of FDs is covered by another set F_1 or F_1 is **cover** of F_2 if every FD in F_2 can be inferred from F_1 , that is, if every FD in F_2 is also in F_1^+ . This means, if any database system enforces FDs in F_1 , then FDs in F_2 will be enforced automatically.

Two sets F_1 and F_2 of FDs are said to be **equivalent** if $F_1^+ = F_2^+$, that is, every FD in F_1 is implied by F_2 and every FD in F_2 is implied by F_1 . In other words, F_1 is equivalent to F_2 , if both the conditions F_1 covers F_2 and F_2 covers F_1 hold. It is clear from the definition, that if F_1 and F_2 are equivalent and the FDs in F_1 are enforced by any database system, then the FDs in F_2 will be enforced automatically and vice versa.

6.3.5 Canonical Cover

For every set of FDs, if a simplified set that is equivalent to given set of FDs can be generated, then the effort spent in checking the FDs can be reduced on each update. Since, simplified set will have the same closure as that of given set, any database that satisfies the simplified set of FDs will also satisfy the given set and vice versa. For example, consider a relation schema $R(A, B, X, Y)$ with a set F of FDs $AB \rightarrow XY$ and $A \rightarrow Y$. Now, whenever an update operation is applied on the relation, the database verifies both the FDs.

If we carefully examine the above given FDs, then we find that if we remove the attribute Y from the right-hand side of $AB \rightarrow XY$, then it doesn't affect the closure of F , since $AB \rightarrow X$ and $A \rightarrow Y$ implies $AB \rightarrow XY$ (by union rule). Thus, verifying $AB \rightarrow X$ and $A \rightarrow Y$ means same as that of verifying $AB \rightarrow XY$ and $A \rightarrow Y$. However, verifying $AB \rightarrow X$ and $A \rightarrow Y$ requires less effort than checking the original set of FDs. This attribute X is extraneous in the dependency $AB \rightarrow XY$, since the closure of new set is same as the closure of F . Formally, extraneous attribute can be defined as follows.

Consider a functional dependency $X \rightarrow Y$ in a given set F of FDs.

- $\Rightarrow$ Attribute A is extraneous in X if $A \in X$, and F logically implies $(F - \{X \rightarrow Y\}) \cup \{(X - A) \rightarrow Y\}$.
- $\Rightarrow$ Attribute A is extraneous in Y if $A \in Y$, and the set of FDs $(F - \{X \rightarrow Y\}) \cup \{X \rightarrow (Y - A)\}$ logically implies F .

Once all such extraneous attributes are removed from the FDs, a new set say F_c is obtained. In addition, if the determinant of each FD in F_c is unique, that means, there are no two dependencies like $X \rightarrow Y$ and $X \rightarrow Z$ in F_c , then F_c is called **canonical cover** for F .

A canonical cover for a set of functional dependencies can be computed by using the algorithm given in Figure 6.6.

```

 $F_c := F;$ 
while(change in  $F_c$ ) do
    replace any FD in  $F_c$  of the form  $X \rightarrow A, X \rightarrow B$ 
        with  $X \rightarrow AB$  (Use union rule)
    for(each FD  $X \rightarrow Y$  in  $F_c$ )do
        begin
            if(extraneous attribute is found either in  $X$  or in  $Y$ )
                then remove it from  $X \rightarrow Y$ 
        end
end

```

Fig. 6.6 Algorithm to compute canonical cover for F

To illustrate the algorithm given in Figure 6.6, consider a set F of functional dependencies $\{A \rightarrow C, AC \rightarrow D, E \rightarrow AD, E \rightarrow H\}$ on relation schema $R(A, B, C)$. To compute the canonical cover for a set F , follow these steps.

1. Initialize F_c to F .
2. On first iteration of the loop, replace FDs $E \rightarrow AD$ and $E \rightarrow H$ with $E \rightarrow ADH$, as both these FDs have same set of attributes in determinant (left side of FD). Now, $F_c = \{A \rightarrow C, AC \rightarrow D, E \rightarrow ADH\}$. In FD $AC \rightarrow D$, C is extraneous. Thus, $F_c = \{A \rightarrow C, A \rightarrow D, E \rightarrow ADH\}$.
3. On second iteration, replace FDs $A \rightarrow C$ and $A \rightarrow D$ with $A \rightarrow CD$. Now, $F_c = \{A \rightarrow CD, E \rightarrow ADH\}$. In FD $A \rightarrow CD$, no attribute is extraneous. In FD $E \rightarrow ADH$, D is extraneous. Thus, $F_c = \{A \rightarrow CD, E \rightarrow AH\}$.
4. On third iteration, the loop terminates, since determinant of each FD is unique and no extraneous attribute is there. Thus, F_c (canonical cover) is $\{A \rightarrow CD$ and $E \rightarrow AH\}$.

It is obvious that the canonical cover of F , F_c has same closure as F . This implies, if we have determined that F_c is satisfied, then we can say F is also satisfied. Moreover, F_c is minimal as FDs with same left side have been combined and F_c does not contain extraneous attributes. Thus, it is easier to test F_c than to test F itself.

6.4 NORMAL FORMS

Consider a relation R with a designated primary key and a set F of functional dependencies. To decide whether the relation is in desirable form or it should be decomposed into smaller relations, some guidelines or formal methodology is needed (as stated earlier). To provide such guidelines, several normal forms have been defined (see Figure 6.7).

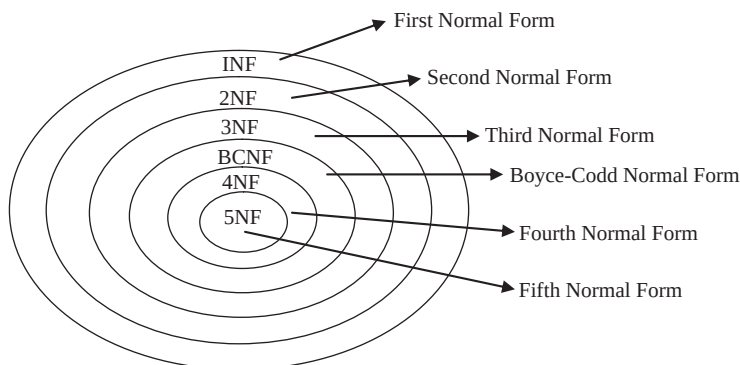


Fig. 6.7 Normal forms

A relation is said to be in a particular normal form if it satisfies certain specified constraints. Each of the normal forms is stricter than its predecessors. The normal forms are used to ensure that various types of anomalies and inconsistencies are removed from the database.

Initially, Codd proposed three normal forms namely, *first normal form (1NF)*, *second normal form (2NF)*, and *third normal form (3NF)*. There were certain inadequacies in the third normal form, so a revised definition was given by Boyce and Codd. This new definition is known as *Boyce-Codd normal form (BCNF)* to distinguish it from the old definition of third normal form. This new definition is stricter than the old definition of third normal form as any relation which is in BCNF is also in third normal form; however, reverse may not be true. All these normal forms are based on the functional dependency. Two other normal forms, namely, *fourth normal form (4NF)* and *fifth normal form (5NF)* have been proposed which are based on the concepts of multivalued dependency and join dependency, respectively.

The process of modifying a relation schema so that it conforms to certain rules (normal forms) is called **normalization**. Normalization is conducted by evaluating a relation schema to check whether it satisfies particular rules and if not, then decomposing schema into set of smaller relation schemas that do not violate those rules. Normalization can be considered as fundamental to the modeling and design of a relational database, the main purpose of which is to eliminate data redundancy and avoid data update anomalies.

Things to Remember

E. F. Codd introduced 1NF in 1970, and 2NF and 3NF in 1971. BCNF was proposed by E. F. Codd and Raymond F. Boyce in 1974. Other normal forms upto DKNF were introduced by Ronald Fagin. He defined 4NF, 5NF, and DKNF in 1977, 1979, and 1981, respectively.



A relation in a lower normal form can always be reduced to a collection of relations in higher normal form. A relation in higher normal form is improvement over its lower normal form.

Generally, it is desirable to normalize the relation schema in a database to the highest possible form. However, normalizing a relation schema to higher forms can introduce complexity to the design and application. As it results in more relations and more relations need more join operations while carrying out queries in database system which can decrease the performance. Thus, while normalization, performance should be taken into account that may result in a decision not to normalize beyond third normal form or BCNF normal form.



NOTE Redundancy cannot be minimized to zero in any database system.

6.4.1 FirstNormalForm

The first normal form states that every tuple must contain exactly one value for each attribute from the domain of that attribute. That is, multivalued attributes, composite attributes, and their combinations are not allowed in 1NF.

Definition: A relation schema R is said to be in **first normal form (1NF)** if and only if the domains of all attributes of R contain atomic (or indivisible) values only.

Consider the relation schema **BOOK_INFO**{ISBN, Price, Page_count, P_ID, R_ID, rating} that represents the information regarding books and reviews. The functional dependencies that hold on relation schema **BOOK_INFO** are {ISBN $\rightarrow$ Price, Page_count, P_ID} and {ISBN, R_ID $\rightarrow$ Rating}. The only key of the relation **BOOK_INFO** is the combination of ISBN and R_ID. For the purpose of explaining normalization, an additional constraint Page_count $\rightarrow$ Price is introduced. It implies that price of book is determined by the number of pages in the book. The FD diagram for **BOOK_INFO** is shown in Figure 6.8 and corresponding instance is shown in Figure 6.9.

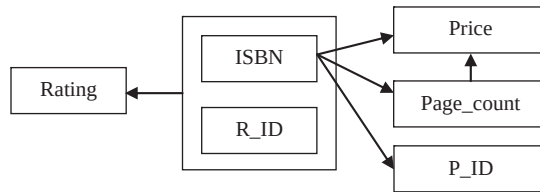


Fig. 6.8 FD diagram for **BOOK_INFO**

ISBN	Price	Page_count	P_ID	R_ID	Rating
001-987-760-9	25	800	P001	A002	6
				A008	7
001-354-921-1	22	200	P001	A006	7
002-678-980-4	35	860	P002	A001	2
				A003	5
				A005	7
004-765-409-5	26	550	P003	A003	4
004-765-359-3	40	650	P003	A007	3
003-456-433-6	30	500	P004	A010	5
001-987-650-5	35	450	P001	A009	8
002-678-880-2	25	400	P002	A006	4
003-456-533-8	30	500	P004	A004	9

Fig. 6.9 Unnormalized relation **BOOK_INFO**

It is clear from Figure 6.9 that tuples contain multiple values for some of the attributes. For example, the tuple for ISBN 001-987-760-9 has two values for the attributes R_ID and Rating. Thus, it is an unnormalized relation. A relation is said to be an **unnormalized relation** if it contains non-atomic values. This relation can be normalized into 1NF by creating one tuple for each value in multivalued attributes as shown in Figure 6.10.

ISBN	Price	Page_count	P_ID	R_ID	Rating
001-987-760-9	25	800	P001	A002	6
001-987-760-9	25	800	P001	A008	7
001-354-921-1	22	200	P001	A006	7
002-678-980-4	35	860	P002	A001	2
002-678-980-4	35	860	P002	A003	5
002-678-980-4	35	860	P002	A005	7
004-765-409-5	26	550	P003	A003	4
004-765-359-3	40	650	P003	A007	3
003-456-433-6	30	500	P004	A010	5
001-987-650-5	35	450	P001	A009	8
002-678-880-2	25	400	P002	A006	4
003-456-533-8	30	500	P004	A004	9

Fig. 6.10 Relation *BOOK_INFO* in 1NF

A good database design always requires that domains of all the attributes of a relation must be atomic, so that the relation schemas are in 1NF. However, depending on the requirement or from the performance point of view, sometimes non-atomic values are useful. For instance, a single attribute *Address* can be used to hold house number, street, city such as *12, Park street, Atlanta* in a relation. Whether the house number, the street name, and city should be separated into distinct attributes depends on how data is to be used. For example, if the address for mailing is needed only, then a single attribute *Address* can be used. However, if the information like all the publishers in a given street is required, then house number, the street name, and city should constitute their own attributes.



NOTE Modern database system supports non-atomic values.

Though the *BOOK_INFO* is in 1NF, it has some shortcomings. The information regarding the same book is repeated a number of times. Further, there are certain anomalies with respect to update operations. For example, the information of a new book cannot be inserted unless some reviewer reviews it. In addition, the change in the price of a book for particular ISBN can lead to inconsistency in the database, if it is not changed in all the tuples where the same ISBN appears. Moreover, the deletion of information of rating for a book having ISBN, say, *003-456-533-8* can cause the loss of detail of the book. This is because there exists only one tuple that contains this particular ISBN. Thus, to resolve these problems, this schema needs further normalization.

6.4.2 Second Normal Form (2NF)

The second normal form is based on the concept of full functional dependency. An attribute *Y* of a relation schema *R* is said to be **fully functional dependent** on attribute *X* ($X \twoheadrightarrow Y$), if there is no *A*, where *A* is proper subset of *X* such that $A \twoheadrightarrow Y$. It implies removal of any attribute from *X* means that the dependency does not hold any more. A dependency $X \twoheadrightarrow Y$ is a **partial dependency** if there is any attribute *A* where *A* is proper subset of *X* such that $A \twoheadrightarrow Y$.

For example, consider the relation schema *BOOK_INFO* in which the attributes *ISBN* and *R_ID* together form the key. In this schema, the attribute *Rating* is fully functional dependent on the key $\{ISBN, R_ID\}$ as *Rating* is not dependent on any subset of key. That is, neither $ISBN \twoheadrightarrow Rating$

nor $R_ID \rightarrow Rating$ holds. However, the attributes *Price*, *Page_count*, *P_ID* are partially dependent on key, since they are dependent on *ISBN*, which is a subset of key $\{ISBN, R_ID\}$.

Definition: A relation schema *R* is said to be in **second normal form (2NF)** if every non-key attribute *A* in *R* is fully functionally dependent on the primary key. An attribute is **non-key** or **non-prime** attribute, if it is not a part of candidate key of *R*.

Consider once again, the relation schema *BOOK_INFO*. This schema is not in 2NF as it involves partial dependencies. This schema can be normalized to 2NF by eliminating all partial dependency between a non-key attribute and the key. This can be achieved by decomposing *BOOK_INFO* into two relation schemas as follows:

BOOK{*ISBN*,*Price*, *Page_count*, *P_ID*}
REVIEW {*ISBN*,*R_ID*, *Rating*}

The instances corresponding to the relation schemas *BOOK* and *REVIEW* are shown in Figure 6.11.

BOOK				REVIEW		
ISBN	Price	Page_count	P_ID	ISBN	R_ID	Rating
001-987-760-9	25	800	P001	001-987-760-9	A002	6
001-354-921-1	22	200	P001	001-987-760-9	A008	7
002-678-980-4	35	860	P002	001-354-921-1	A006	7
004-765-409-5	26	550	P003	002-678-980-4	A001	2
004-765-359-3	40	650	P003	002-678-980-4	A003	5
003-456-433-6	30	500	P004	002-678-980-4	A005	7
001-987-650-5	35	450	P001	004-765-409-5	A003	4
002-678-880-2	25	400	P002	004-765-359-3	A007	3
003-456-533-8	30	500	P004	003-456-433-6	A010	5
				001-987-650-5	A009	8
				002-678-880-2	A006	4
				003-456-533-8	A004	9

Fig. 6.11 Relations in 2NF

The relation schemas *BOOK* and *REVIEW* do not have any partial dependencies. Each of the non-key attributes of relation schemas *BOOK* and *REVIEW* is fully functional dependent on key attribute *ISBN* and (*ISBN*, *R_ID*), respectively. Hence, both the schemas are in 2NF. The FDs corresponding to these relation schemas are shown in Figure 6.12.

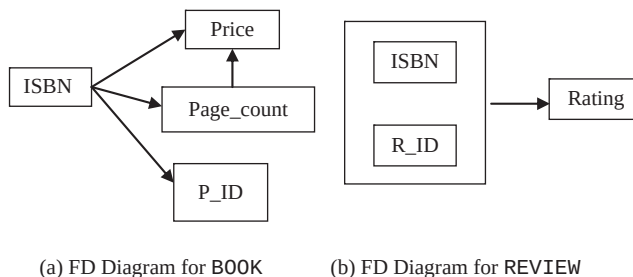


Fig. 6.12 FDs in 2NF

Now, all the problems of **BOOK_INFO** have been overcome by normalizing it into 2NF; however, the decomposed relation schema **BOOK** which is in 2NF still causes problems over update operation. For example, the **Price** for **Page_count** 900 cannot be inserted unless there exists a book having 900 pages. Similarly, the change in the **Price** for particular **Page_count** can lead to inconsistency in the database if all the tuples where the same **Page_count** appears are not changed. Moreover, the deletion of the **Price** for particular **Page_count** say, 200 can cause the loss of information of the book having **ISBN** 001-354-921-1. This is due to the fact that there exists only one tuple that contains the information of book having this particular **ISBN**. These problems can be resolved by normalizing this relation schema further. Since, relation schema **REVIEW** do not have such type of anomalies, there is no need to normalize it further.

Learn More

While normalizing the relation schemas, sometimes, it is required to introduce artificial key (also known as **surrogate key**) in some decomposed relations. It is used when a primary key is not available or inapplicable.



Testing a relation schema for 2NF involves testing FDs where determinant is part of the primary key. If the primary key contains a single attribute, the test need not be applied at all.

6.4.3 Third Normal Form (3NF)

The third normal form is based on the concept of transitive dependency. An attribute **Y** of a relation schema **R** is said to be **transitively dependent** on attribute **X** ($X \rightarrow Y$), if there is set of attributes **A** that is neither a candidate key nor a subset of any key of **R** and both $X \rightarrow A$ and $A \rightarrow Y$ hold. For example, the dependency $\text{ISBN} \rightarrow \text{Price}$ is transitive through **Page_count** in relation schema **BOOK**. This is because both the dependencies $\text{ISBN} \rightarrow \text{Page_count}$ and $\text{Page_count} \rightarrow \text{Price}$ hold and **Page_count** is neither a candidate key nor a subset of candidate key of **BOOK**.

Definition: A relation schema **R** is in **third normal form (3NF)** with a set **F** of functional dependencies if, for every FD $X \rightarrow Y$ in **F**, where **X** be any subset of attributes of **R** and **Y** be any single attribute of **R**, at least one of the following statements hold

- $\Rightarrow X \rightarrow Y$ is a trivial FD, that is, $Y \subseteq X$.
- $\Rightarrow X$ is a superkey for **R**.
- $\Rightarrow Y$ is contained in a candidate key for **R**.

It can also be stated as,

A relation schema **R** is in third normal form (3NF) if and only if it satisfies 2NF and every non-key attribute is non-transitively dependent on the primary key.

For example, the schema **BOOK** can be decomposed into two relation schemas say, **BOOK_PAGE** and **PAGE_PRICE** to eliminate transitive dependency between the attributes **ISBN** and **Price**. Hence, the schemas **BOOK_PAGE** and **PAGE_PRICE** are in 3NF. The instances of these schemas are shown in Figure 6.13, and Figure 6.14 shows FD diagrams of these schemas.

Note that this definition of 3NF can be used to handle relations in which primary key is the only candidate key. Thus, to handle the relations having more than one candidate key, this old definition of 3NF was replaced by new definition by Boyce and Codd normal form which is also known as BCNF.

BOOK_PAGE			PAGE_PRICE	
ISBN	Page_count	P_ID	Page_count	Price
001-987-760-9	800	P001	800	25
001-354-921-1	200	P001	200	22
002-678-980-4	860	P002	860	35
004-765-409-5	550	P003	550	26
004-765-359-3	650	P003	650	40
003-456-433-6	500	P004	500	30
001-987-650-5	450	P001	450	35
002-678-880-2	400	P002	400	25
003-456-533-8	500	P004		

Fig. 6.13 Relations in 3NF

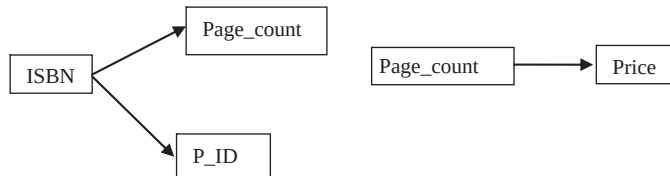


Fig.6.14 FD diagram for BOOK_PAGE and PAGE_PRICE

6.4.4 Boyce-Codd Normal Form (BCNF)

Boyce-Codd normal form was proposed to deal with the relations having two or more candidate keys that are composite and overlap. Two candidate keys **overlap** if they contain two or more attributes and have at least one attribute in common.

Definition: A relation schema R is in **Boyce-Codd normal form (BCNF)** if, for every FD $X \rightarrow A$ in F , where X is the subset of the attributes of R , and A is an attribute of R , one of the following statements holds

- $\Rightarrow X \rightarrow Y$ is a trivial FD, that is, $Y \subseteq X$.
- $\Rightarrow X$ is a superkey.

In simple terms, it can be stated as

A relation schema R is in **BCNF** if and only if every non-trivial FD has a candidate key as its determinant.

For example, consider a relation schema $BOOK(ISBN, Book_title, Price, Page_count)$ with two candidate keys, $ISBN$ and $Book_title$. Further, assume that the functional dependency $Page_count \rightarrow Price$ no longer holds. The FD diagram for this relation schema is shown in Figure 6.15.

As shown in Figure 6.15, there are two determinants $ISBN$ and $Book_title$ and both of them are candidate keys. Thus, this relation schema is in BCNF.

BCNF is the simpler form of 3NF as it makes explicit reference to neither the first and second normal forms, nor to the concept of transitive dependence. Furthermore, it is stricter than 3NF as every relation which is in BCNF is also in 3NF, but vice versa is not necessarily true. A 3NF relation will not be in BCNF if there are multiple candidate keys in the relation such that

- $\Rightarrow$ the candidate keys in the relation are composite keys, and
- $\Rightarrow$ the keys are not disjoint, that is, candidate keys overlap.

To illustrate this point, consider a relation schema **BOOK_RATING**(ISBN, Book_title, R_ID, Rating). The candidate keys are (ISBN, R_ID) and (Book_title, R_ID). This relation schema is not in BCNF since both the candidate keys are composite as well as overlapping. However, it is in 3NF.

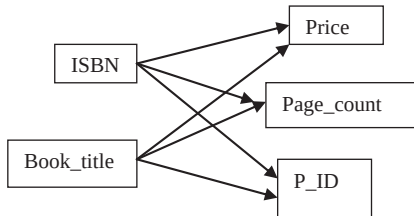


Fig. 6.15 FD diagrams for *BOOK* and with *Book_title* as candidate key

R_ID	ISBN	Book_title	Rating
A002	001-987-760-9	C++	6.0
A008	001-987-760-9	C++	7.2
A006	001-354-921-1	Ransack	7.5
A003	002-678-980-4	DBMS	5.3
A001	002-678-980-4	DBMS	2.1
A003	004-765-409-5	UNIX	4.5
.	.	.	.
.	.	.	.

Fig. 6.16 Instance of relation *BOOK_RATING*

From the instance corresponding to this relation schema as shown in Figure 6.16, it is clear that it suffers from the problem of repetition of information. Thus, changing the title of book can cause logical inconsistency if it is not changed in all the tuples in which it appears. This problem can be resolved by decomposing this relation schema into two relation schemas as shown here.

BOOK_TITLE_INFO(ISBN, Book_title) and
REVIEW(R_ID, ISBN, Rating)

Or

BOOK_TITLE_INFO(ISBN, Book_title) and
REVIEW(R_ID, Book_title, Rating)

Now, all these relation schemas are in BCNF. Note that BCNF is the most desirable normal form as it ensures the elimination of all redundancy that can be detected using functional dependencies.



If there is only one determinant upon which other attributes depend and it is a candidate key, 3NF and BCNF are identical.

6.5 INSUFFICIENCY OF NORMAL FORMS

From the discussion so far, it can be concluded that generally normalization start from a single **universal relation** which includes all attributes of a database. The relation is decomposed repeatedly until a BCNF or 3NF is achieved. However, this condition is not sufficient on its own to guarantee a good

relational database schema design. In addition to this condition, there are two additional properties that must hold on decomposition to qualify it as a good design. These are *lossless-join* property and *dependency preservation* property.

6.5.1 Lossless-JoinProperty

The notion of the lossless-join property is crucial as it ensures that the original relation must be recovered from the smaller relations that result after decomposition. That is, it ascertains that no information is lost after decomposition.

Definition: Let R be a relation schema with a given set F of functional dependencies. Then decomposition $D = \{R_1, R_2, \dots, R_n\}$ on R is said to be **lossless (lossless-join)**, if for every legal relation (that is, relation that satisfies the specified functional dependencies and other constraints), following statement holds.

$$\pi_{R_1}(R) \bowtie \pi_{R_2}(R) \bowtie \dots \bowtie \pi_{R_n}(R) = R$$

In other words, if the original relation can be reconstructed by combining the projections of a relation using natural join (or the process of decomposition is reversible), decomposition is **lossless decomposition**, otherwise, it is a **lossy (lossy-join) decomposition**.

For example, consider again the relation schema $BOOK(ISBN, Price, Page_count)$ with the functional dependencies $ISBN \rightarrow Price$ and $Page_count \rightarrow Price$. Other attributes are ignored to simplify the relation schema. An instance of this relation schema is shown in Figure 6.17.

The relation schema $BOOK(ISBN, Price, Page_count)$ can be decomposed in two different ways, say, decomposition A and decomposition B (see Figure 6.17). The schemas corresponding to decomposition A and decomposition B are given here.

Decomposition A

$BOOK_PRICE(ISBN, Price)$
and $BOOK_PAGES(ISBN, Page_count)$

Decomposition B

$BOOK_PRICE(ISBN, Price)$
and $PRICE_PAGE(Price, Page_count)$

BOOK			
ISBN	Price	Page_count	
001-987-760-9	25	800	
001-354-921-1	22	200	
002-678-745-2	25	810	

Decomposition A			
BOOK_PRICE		BOOK_PAGES	
ISBN	Price	ISBN	Page_count
001-987-760-9	25	001-987-760-9	800
001-354-921-1	22	001-354-921-1	200
002-678-745-2	25	002-678-745-2	810

Decomposition B			
BOOK_PRICE		PRICE_PAGE	
ISBN	Price	Page_count	Price
001-987-760-9	25	800	25
001-354-921-1	22	200	22
002-678-745-2	25	810	25

Fig. 6.17 Instance of relation $BOOK$ and two corresponding decompositions

In Decomposition A (see Figure 6.17), the price and the number of pages for all the books can be determined easily. Since, no information is lost in this decomposition, decomposition A is lossless.

In Decomposition B (see Figure 6.17), it can be determined that price of the books having ISBN 001-987-760-9 and 002-678-980-4 is 25. However, the number of pages in these books cannot be determined as there are two values for `page_count` in relation `PRICE_PAGE` corresponding to the `Price` 25. Thus, this decomposition is lossy.

If an decomposition does not have the lossless-join property, then additional spurious tuples (tuples that are not in original relation) may be obtained on applying join operation on the relations resulted after decomposition. These spurious tuples represent erroneous data. For example, consider the relations `BOOK_PRICE` and `PRICE_PAGE` as shown in decomposition B in Figure 6.17. If these relations are joined, a new relation having additional spurious tuples is obtained as shown in Figure 6.18.

ISBN	Price	Page_count
001-987-760-9	25	800
001-987-760-9	25	810
001-354-921-1	22	200
002-678-980-4	25	800
002-678-980-4	25	810

Spurious tuples

Fig. 6.18 Relation with spurious tuples

However, if the relations as shown in decomposition A in Figure 6.17 are joined, no spurious tuples will be generated. Thus, if lossless-join property holds on decomposition, it is ensured that no spurious tuples are obtained when a natural join operation is applied to the relations resulted after decomposition. So, this property is also known as **non-additive** join property.

A particular decomposition can be tested for having lossless property on the basis of functional dependency by applying the property as given here.

Lossless-join test for binary decompositions: Let R be a relation schema with F a set of FDs over R . The decomposition of R into two relations R_1 and R_2 form a lossless decomposition if at least one of the following dependencies is in F^+ .

$$R_1 \cap R_2 \rightarrow R_1 - R_2$$

$$R_1 \cap R_2 \rightarrow R_2 - R_1$$

This property is limited to **binary decompositions** (decomposition of a relation into two relations) only. In addition, it is a necessary condition only if all constraints are functional dependencies.



NOTE The test for lossless decomposition in case of a decomposition of a relation into multiple relations is beyond this discussion.

6.5.2 Dependency Preservation Property

Another important property that is desired while decomposition is dependency preservation, that is, no FD of the original relation is lost. Consider a relation schema R with set F of FDs $\{X \rightarrow Y, Y \rightarrow Z\}$. One other dependency that is implied from F is $X \rightarrow Z$. Let R is decomposed into two relation schemas, $R_1\{X, Y\}$ with FD $X \rightarrow Y$ and $R_2\{Y, Z\}$ with FD $Y \rightarrow Z$. Here, $X \rightarrow Z$ is enforced automatically since it is implied from FDs of R_1 and R_2 . Hence, this decomposition is dependency preserving. Now, suppose that R is decomposed into two relation schemas, $R_1\{X, Y\}$ with FD $X \rightarrow Y$ and $R_2\{X, Z\}$ with FD $X \rightarrow Z$. Here, FD $Y \rightarrow Z$ is not implied from R_1 and R_2 . To enforce this FD,

join operation is to be applied whenever a new tuple is to be inserted. We know that computing join is a costly process in terms of CPU usage and response time.

Dependency preservation property ensures that each FD represented by original relation is enforced by examining any single relation resulted from decomposition or can be inferred from the FDs in some decomposed relation. Thus, it prevents computation of join. To understand this property, let us first understand the term projection. Given a relation schema R with set F of FDs and $R_1, R_2, \dots, R_n$ be the decomposition of R , the **projection** of F to R_i is the set F_i of all FDs in F^+ (not just F) that include only attributes of R_i . For example, once again consider the decomposition of $R_1\{X, Y\}$ and $R_2\{X, Z\}$ of R . The projection of F to R_1 , denoted by F_{R_1} is $X \rightarrow Z$, since $X \rightarrow Z$ in F^+ , even though it is not in F . This FD can be tested by examining relation R_2 only. Similarly, FDs in each projection can be tested by analyzing only one relation in the decomposed set.

However, testing each projection is not sufficient to determine whether the decomposition is dependency preserving. Since, it is not always necessary that all the dependencies in original relation appear directly in an individual decomposed relation. That is, there may be FDs in F^+ that are not the projections. Thus, we also need to examine the FDs that are inferred from the projections. Formally, the decomposition of relation schema R with a given set F of FDs into relations $R_1, R_2, \dots, R_n$ is **dependency preserving** if $F'^+ = F^+$, where $F' = F_{R_1} \cup F_{R_2} \cup \dots \cup F_{R_n}$.



Lossless-join property is indispensable requirement of decomposition that must be achieved at any cost, whereas the dependency preservation property, though desirable, can be sacrificed sometimes.

6.6 COMPARISON OF BCNF AND 3NF

From the discussion so far, it can be concluded that the goal of a database design with functional dependencies is to obtain schemas that are in BCNF, are lossless and preserves the original set of FDs.

A relation can always be decomposed into 3NF in such a way that the decomposition is lossless and dependency preserving. However, there are some disadvantages to 3NF. There is a possibility that null values are used to represent some of the meaningful relationships among data items. In addition, there is always a scope of repetition of information.

On the other hand, it is always not possible to obtain a BCNF design without sacrificing dependency preservation. For example, consider a relation schema $CITY_ZIP(City, Street, Zip)$ with FDs $\{City, Street\} \rightarrow Zip$ and $Zip \rightarrow City$ is decomposed into two relation schemas $R_1(Street, Zip)$ and $R_2(City, Zip)$. It is observed that both these schemas are in BCNF. In R_1 , there is only trivial dependency and in R_2 , the dependency $Zip \rightarrow City$ holds. It can be seen clearly that the FD $\{City, Street\} \rightarrow Zip$ is not implied from decomposed relation schemas. Thus, this decomposition does not preserve dependency, that is, $(F_1 \cup F_2)^+ \neq F^+$, where F_1 and F_2 are the restrictions of R_1 and R_2 , respectively.

Clearly, it is not always possible to achieve the goal of database design. In that case, we have to be satisfied with relation schemas in 3NF rather than BCNF. This is because, if dependency preservation is not checked efficiently, either system performance can be reduced or it may risk the integrity of the data. Thus, maintaining the integrity of data at the cost of the limited amount of redundant data is a better option.

6.7 HIGHER NORMAL FORMS

As stated earlier, BCNF is the desirable normal form; however, some relation schemas even though they are in BCNF still suffer from the problem of repetition of information. For example, consider

an alternative design for *Online Book* database having a relation schema **BOOK_AUTHOR_DETAIL**(ISBN, A_ID, Phone). An instance of this relation schema is shown in Figure 6.19(a).

In this relation, there are two multivalued attributes, **A_ID** and **Phone** that represents the fact that a book can be written by multiple authors and an author can have multiple phone numbers. That is, for a given **ISBN**, there can be multiple values in attribute **A_ID** and for a given **A_ID**, there can be multiple values for attribute **Phone**. To make the relation consistent, every value of one of the attributes is to be repeated with every value of the other attribute [see Figure 6.19(b)].

ISBN	A_ID	Phone
001-987-760-9	A001	923673
		923743
	A003	419456
		419562
001-354-921-1	A005	678654
		678655
		678657
002-678-980-4	A002	376045
	A008	765490
004-765-409-5	A008	765490
.	.	.
.	.	.
.	.	.
.	.	.

(a)

ISBN	A_ID	Phone
001-987-760-9	A001	923673
001-987-760-9	A001	923743
001-987-760-9	A003	419456
001-987-760-9	A003	419562
001-354-921-1	A005	678654
001-354-921-1	A005	678655
001-354-921-1	A005	678657
002-678-980-4	A002	376045
002-678-980-4	A008	765490
004-765-409-5	A008	765490
.	.	.
.	.	.
.	.	.
.	.	.
.	.	.

(b)

Fig. 6.19 Relation **BOOK_AUTHOR_DETAIL**

From Figure 6.19(b), it is clear that relation **BOOK_AUTHOR_DETAIL** satisfies only trivial dependencies. In addition, it is **all key relation** (a relation in which key is the combination of all the attributes taken together) and any all key relation must necessarily be in BCNF. Thus, this relation is in BCNF. However, it involves lot of redundancies which leads to certain update anomalies. Thus, it is always not possible to decompose a relation on the basis of functional dependencies to avoid redundancy.



NOTE BCNF can eliminate the redundancies that are based on the functional dependencies.

Since, normal forms based on functional dependencies are not sufficient to deal such type of situations, other type of dependencies and normal forms have been defined.

6.7.1 Multivalued Dependencies

Multivalued dependencies (MVDs) are the generalization of functional dependencies. Functional dependencies prevent the existence of certain tuples in a relation. For example, if a FD $X \rightarrow Y$ holds in R , then any $r(R)$ cannot have two tuples having same X value but different Y values. On the contrary, multivalued dependencies do not rule out the presence of those tuples, rather, they require presence of other tuples of a certain form in the relation. Multivalued dependencies arise when a relation having a non-atomic attribute is converted to a normalized form. For each X value in such a relation, there exists a well-defined set of Y values associated with it. This association between the X and Y values does not depend on the values of the other attributes in the relation.

To understand the notion of multivalued dependencies more clearly, consider the relation **BOOK_AUTHOR_DETAIL** as shown in Figure 6.19(a). In this relation, although the **ISBN** does not have a unique **A_ID** (the FD $\text{ISBN} \rightarrow \text{A_ID}$ does not hold), each **ISBN** has a well defined set of corresponding **A_ID**. Similarly for a particular **A_ID**, a well defined set of **Phone** exists. In addition, book is independent of the phone numbers of authors, due to which there is lot of redundancy in this relation. Such situation which cannot be expressed in terms of FD is dealt by specifying a new form of constraint called multivalued dependency or MVD.

Definition: In a relation schema R , an attribute Y is said to be **multi-dependent** on attribute X ($X \twoheadrightarrow Y$) if and only if for a particular value of X , the set of values of Y is completely determined by the value of X alone and is independent of the values of Z where, X , Y , and Z are the subsets of the attributes of R .

$X \twoheadrightarrow Y$ is read as Y is **multi-dependent** on X or X **multi-determines** Y .

From the definition of MVD, it is clear that if $X \rightarrow Y$, then $X \twoheadrightarrow Y$. In other words, every functional dependency is a multivalued dependency. However, the converse is not true.

The multivalued dependencies that hold in the relation **BOOK_AUTHOR_DETAIL**, [see Figure 6.19(b)] can be represented as

$\text{ISBN} \twoheadrightarrow \text{A_ID}$
 $\text{A_ID} \twoheadrightarrow \text{Phone}$

If the MVD $X \twoheadrightarrow Y$ is satisfied by all relations on schema R , then $X \twoheadrightarrow Y$ is a **trivial** dependency on schema R , that is, $X \twoheadrightarrow Y$ is trivial if $Y \subseteq X$ or $Y \cup X = R$.



NOTE Generally, the relations containing non-trivial MVDs are all key relations.

Like functional dependencies (FDs), inference rules for multivalued dependencies (MVDs) have been formulated which are complete as well as sound. These set of rules consist of three Armstrong axioms (discussed earlier) and five additional rules. Three of these rules involve only MVDs which are

- ⇒ **Complementation rule for MVDs:** If $X \twoheadrightarrow Y$, then $X \twoheadrightarrow R - XY$.
- ⇒ **Augmentation rule for MVDs:** If $X \twoheadrightarrow Y$ and $W \supseteq Z$, then $WX \twoheadrightarrow YZ$.
- ⇒ **Transitive rule for MVDs:** If $X \twoheadrightarrow Y$ and $Y \twoheadrightarrow Z$, then $X \twoheadrightarrow (Z - Y)$.

The remaining two inference rules relate FDs and MVDs. These rules are

- ⇒ **Replication rule for FD to MVD:** If $X \rightarrow Y$, then $X \twoheadrightarrow Y$.

⇒ **Coalescence rule for FD to MVD:** If $X \twoheadrightarrow Y$ and there exists W such that $W \cap Y$ is empty, $W \rightarrow Z$ and $Y \supseteq Z$, then $X \rightarrow Z$.



Multivalued dependency is a result of first normal form which disallows an attribute to have multiple values. If a relation has two or more multivalued independent attributes, and is independent of one another, it is specified using multivalued dependency.

6.7.2 FourthNormalForm

Fourth normal form is based on the concept of multivalued dependency. It is required when undesirable multivalued dependencies occur in a relation. This normal form is more restrictive than BCNF. That is, any 4NF is necessarily in BCNF but converse is not true.

Definition: A relation schema R is in **fourth normal form (4NF)** with respect to a set F of functional and multivalued dependencies if, for every non-trivial multivalued dependency $X \twoheadrightarrow Y$ in F^+ , where $X \subseteq R$ and $Y \subseteq R$, X is a superkey of R .

For example, consider, once again relation **BOOK_AUTHOR_DETAIL**. As stated earlier, the problem is due to the fact that book is independent of the phone number of authors. This problem can be resolved by eliminating this undesirable dependency, that is, by decomposing this relation into two relations **BOOK_AUTHOR** and **AUTHOR_PHONE** as shown in Figure 6.20.

BOOK_AUTHOR		AUTHOR_PHONE	
ISBN	A_ID	A_ID	Phone
001-987-760-9	A001	A001	923673
001-987-760-9	A003	A001	923743
001-354-921-1	A005	A003	419456
002-678-980-4	A002	A003	419562
002-678-980-4	A008	A005	678654
004-765-409-5	A008	A005	678655
.	.	A005	678657
.	.	A002	376045
		A008	765490
		.	.
		.	.

Fig. 6.20 Relations in 4NF

Since, the relations **BOOK_AUTHOR** and **AUTHOR_PHONE** do not involve any MVD, they are in 4NF. Hence, this design is an improvement over the original design. Further, relation **BOOK_AUTHOR_DETAIL** can be recovered by joining **BOOK_AUTHOR** and **AUTHOR_PHONE** together.

Note that whenever a relation schema R is decomposed into relation schemas $R_1 = (X \cup Y)$ and $R_2 = (R - Y)$ based on MVD $X \twoheadrightarrow Y$ that holds in R , the decomposition has the lossless-join property. This condition is necessary and sufficient for decomposing a schema into two schemas that have lossless-join property. Formally, this condition can be stated as:

Property for lossless-join decomposition into 4NF: The relation schemas R_1 and R_2 form lossless-join decomposition of R with respect to a set F of functional and multivalued dependencies if and only if

$$\begin{aligned} (R_1 \cap R_2) &\twoheadrightarrow (R_1 - R_2) \text{ or} \\ (R_1 \cap R_2) &\twoheadrightarrow (R_2 - R_1) \end{aligned}$$

6.7.3 JoinDependencies

The property (lossless-join test for binary decompositions) that is discussed earlier, gives the condition for a relation schema to be decomposed in a lossless way by exactly two of its projections. However, in some cases, there exist relations that cannot be lossless decomposed into two projections but can be lossless decomposed into more than two projections. Moreover, it may be in 4NF that means there may neither any functional dependency in R that violates any normal form upto BCNF nor any non-trivial dependency that violates 4NF. However, the relation may have another type of dependency called join dependency which is the most general form of dependency possible.

Consider a relation `AUTHOR_BOOK_PUBLISHER` (see Figure 6.21) which is all key relation (all the attributes form key). Since, in this relation, no non-trivial FDs or MVDs are involved, it is in 4NF. It should be noted that if it is decomposed exactly into any two relations and joined back, a spurious tuple is created (see Figure 6.21); however, if it is decomposed into three relations and joined back as shown in Figure 6.21, original relation is obtained. It is because it specifies another constraint according to which, whenever an author a writes book b , and a publisher p publishes book b , and the author a writes at least one book for publisher p , then author a will also be writing book b for publisher p . For example,

If a tuple having values $\langle A001, C++ \rangle$ exists in `AUTHOR_BOOK`
 and a tuple having values $\langle C++, P001 \rangle$ exists in `BOOK_PUBLISHER`
 and a tuple having values $\langle P001, A001 \rangle$ exists in `PUBLISHER_AUTHOR`
 then a tuple having values $\langle A001, C++, P001 \rangle$ exists in `AUTHOR_BOOK_PUBLISHER`

Note that the converse of this statement is true, that is, if $\langle A001, C++, P001 \rangle$ appears in `AUTHOR_BOOK_PUBLISHER`, then $\langle A001, C++ \rangle$ appears in relation `AUTHOR_BOOK` and so on. Further, $\langle A001, C++ \rangle$ appears in `AUTHOR_BOOK` if and only if $\langle A001, C++, P002 \rangle$ appears in `AUTHOR_BOOK_PUBLISHER` for some $P002$ and similarly, for $\langle C++, P001 \rangle$ and $\langle P001, A001 \rangle$. This statement can be rewritten as a constraint on `AUTHOR_BOOK_PUBLISHER` as “If tuples having values $\langle A001, C++, p002 \rangle$ $\langle A002, C++, P001 \rangle$ and $\langle A001, Unix, P001 \rangle$ exists in `AUTHOR_BOOK_PUBLISHER` then tuple having values $\langle A001, C++, P001 \rangle$ also exists in `AUTHOR_BOOK_PUBLISHER`”.

Since, this constraint is satisfied if and only the relation is joining of certain projections, this constraint is referred as join dependency (JD).

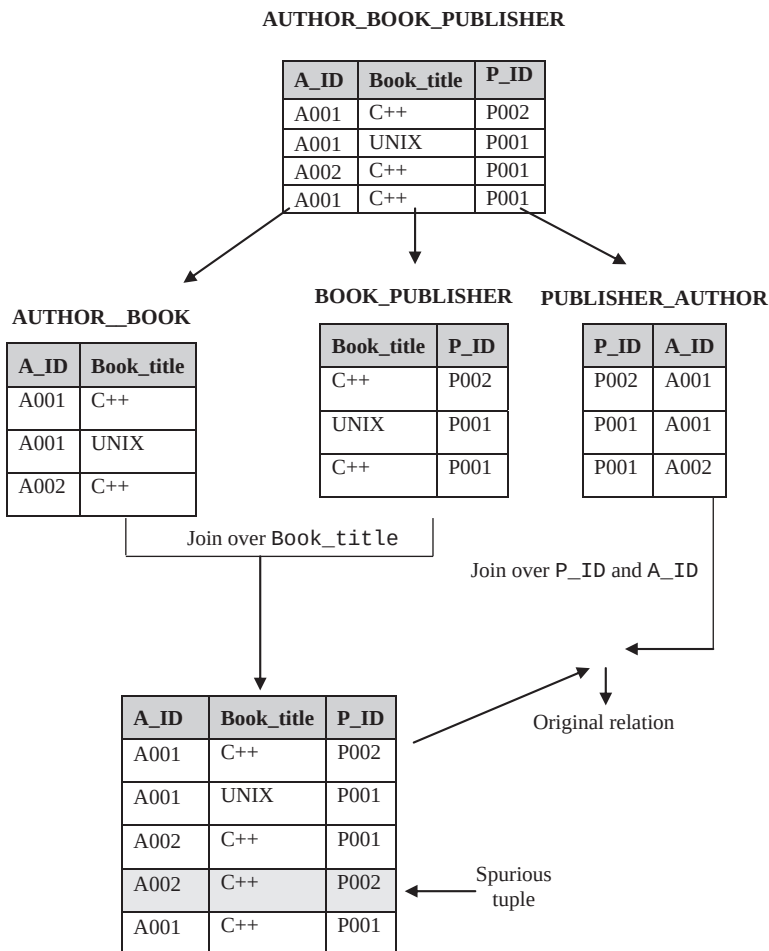


Fig. 6.21 Non-loss decomposition of a relation into three projections

Definition: Let R be a relation schema and $R_1, R_2, \dots, R_n$ be the decomposition of R , R is said to satisfy the **join dependency** $*(R_1, R_2, \dots, R_n)$ (read as “star $R_1, R_2, \dots, R_n$ ”), if and only if

$$\Pi_{R_1}(R) \bowtie \Pi_{R_2}(R) \bowtie \dots \bowtie \Pi_{R_n}(R) = R$$

In other words, relation schema R satisfies the JD $*(R_1, R_2, \dots, R_n)$ if and only if every legal instance $r(R)$ is equal to join of its projections on $R_1, R_2, \dots, R_n$.

From this definition, it is clear that MVD is a special case of a JD, where $n = 2$ (or join dependency is generalization of MVD). A join dependency $*(R_1, R_2, \dots, R_n)$ specified on a relation schema R is a trivial JD, if at least one relation schema R_i in $JD *(R_1, R_2, \dots, R_n)$ is the set of all attributes of R (that is, one of the relation schemas R_i in $JD *(R_1, R_2, \dots, R_n)$ is equal to R). Such a dependency is called trivial because it has the lossless-join property for any instance of R and hence, does not specify any constraint on R .

6.7.4 FifthNormalForm

Fifth Normal Form also called **project-join normal form (PJ/NF)** is based on the concept of join dependencies.

Definition: A relation schema R is in **fifth normal form (5NF)** with respect to a set F of functional, multivalued, and join dependencies if, for every non-trivial join dependency $*(R_1, R_2, \dots, R_n)$ in F^+ , every R_i is a superkey of R .

For example, consider once again the `AUTHOR_BOOK_PUBLISHER` relation (see Figure 6.21) which satisfies join dependency that is not implied by its sole superkey (that key being the combination of all its attributes). Thus, it is not in 5NF. In addition, this relation suffers from a number of update anomalies. These anomalies can be eliminated if it is decomposed into three relations `AUTHOR_BOOK`, `BOOK_PUBLISHER`, and `PUBLISHER_AUTHOR`. Each of these relations is in 5NF since they do not involve any join dependency. Further, if join operation is applied on any two of these relations, spurious tuples are generated; however, it is not so on applying join operation on all three relations. Note that any relation in 5NF is automatically in 4NF.

Learn More

There exists another normal form called **domain-key normal form (DK/NF)**, which is based on domain, key, and general constraints. It is considered as the highest form of normalization. However, it is not always possible to generate schemas in this form. Thus, it is rarely used.



Unlike FDs and MVDs, it is difficult to detect join dependencies as there is no set of complete and sound inference rules for this constraint. Thus, 5NF is rarely achievable.

6.8 DENORMALIZATION

From the discussion so far, it is clear that when the process of normalization is applied to a relation, the redundancy is reduced. However, it results in more relations which increase the CPU overhead. Hence, it fails to provide adequate response time, that is, makes the database system slow which often counteracts the redundant data. Thus, sometimes, in order to speed up database access, the relations are required to be taken from higher normal form to lower normal form. This process of taking a schema from higher normal form to lower normal form is known as **denormalization**. It is mainly used to improve system performance to support time-critical operations. There are several other factors which require denormalizing relations are given here.

- ⇒ Many critical queries that require data to be retrieved from more than one relation.
- ⇒ Many computations need to be applied to one or many attributes before answering a query.
- ⇒ The values in multivalued attributes need to be processed in a group instead of individually.
- ⇒ Relations need to be accessed in different ways by different users during same duration.

Like normalization, there is no formal guidance that outlines the approach to denormalizing a database. Once the process of denormalization is started, it is not clear where should it be stopped. Further, the relations that are denormalized suffer from a number of other problems like redundancy and update problems. It is obvious as the relations, which are to be dealt are once again the relations that are less than fully normalized. Moreover, denormalization makes certain queries harder to express. Thus, the main aim of denormalization process is to balance the need for good system response time and need to maintain data while avoiding the various problems associated with denormalized relations.

SUMMARY

1. Designing of relation schemas is based on the conceptual design. The relation schemas can be generated directly using conceptual data model such as ER or enhanced ER (EER) or some other conceptual data model. The goodness of the resulting set of schemas depends on the designing of the data model.
2. A relational database is considered as good if it has minimum redundancy and null values.
3. While designing relational database schema, the problems that are confronted due to redundancy and null values can be resolved by decomposition.
4. In decomposition, a relation is replaced with a collection of smaller relations with specific relationship between them.
5. During decomposition, it must be ensured that each attribute in R must appear in at least one relation schema R_i so that no attributes are lost. This is called attribute preservation property of decomposition.
6. Functional dependencies are one of the most important concepts in relational schema design. They are the special forms of integrity constraints that generalize the concept of keys. A given set of functional dependencies is used in designing a relational database in which most of the undesirable properties are not present.
7. For a given set F of functional dependencies, some other dependencies also hold. That is, some functional dependencies are logically implied by F. The set of all FDs that are implied by a given set of FDs is called the closure of F, denoted as F^+ .
8. Armstrong proposed a set of inference rules or axioms (also known as Armstrong's axioms) to find logically implied functional dependencies. The axioms are complete and sound.
9. Two sets F_1 and F_2 of FDs are said to be equivalent if $F_1^+ = F_2^+$, that is, every FD in F_1 is implied by F_2 and every FD in F_2 is implied by F_1 .
10. For every set of FDs, canonical cover (a simplified set) that is equivalent to given set of FDs can be generated.
11. To determine the quality or goodness of a database, some formal measures or guidelines are required. Normalization theory provides the guidelines to assess the quality of a database in designing process. The normalization is the application of set of simple rules called normal forms to the relation schema.
12. A relation is said to be in a particular normal form if it satisfies certain specified constraints.
13. A relation is said to be an unnormalized relation if it contains non-atomic values.
14. Four normal forms, namely, first normal form (1NF), second normal form (2NF), third normal form (3NF), and Boyce-Codd normal form (BCNF) are based on the functional dependency.
15. A relation is said to be in first normal form (1NF) if the domains of all its attributes contain atomic (or indivisible) values only.
16. The second normal form is based on the concept of full functional dependency.
17. The third normal form is based on the concept of transitive dependency.
18. Boyce-Codd normal form was proposed to deal with the relations having two or more candidate keys that are composite and overlap.
19. There are two additional properties that must hold on decomposition to qualify a relational schema as a good design. These are lossless-join property and dependency preservation property.
20. Lossless-join property ensures that the original relation must be recovered from the smaller relations that result after decomposition.
21. Dependency preservation property ensures that each FD represented by original relation is enforced by examining any single relation resulted from decomposition or can be inferred from the FDs in some decomposed relation.
22. Lossless-join property is indispensable requirement of decomposition that must be achieved at any cost, whereas the dependency preservation property, though desirable, can be sacrificed sometimes.

23. Normal forms based on functional dependencies are always not sufficient to avoid redundancy. So, other types of dependencies and normal forms have been defined.
24. Multivalued dependencies (MVDs) are the generalization of functional dependencies.
25. Fourth normal form is based on the concept of multivalued dependency. It is required when undesirable multivalued dependencies occur in a relation.
26. A relation may have another type of dependency called join dependency which is the most general form of dependency possible.
27. Fifth normal form (5NF), also called project–join normal form (PJ/NF) is based on the concept of join dependencies.
28. The process of normalization is applied to a relation to reduce redundancy. However, it results in more relations which increase the CPU overhead.
29. In order to speed up database access, the relations are required to be taken from higher normal form to lower normal form. This process of taking a schema from higher normal form to lower normal form is known as denormalization.

● KEY TERMS ●

- | | |
|---|---|
| <ul style="list-style-type: none"> ● Decomposition ● Attribute preservation property ● Functional dependency ● Determinant dependent ● Trivial dependency ● Non-trivial dependencies ● Closure of a set of functional dependencies ● Armstrong's axioms ● Closure of attribute sets ● Equivalence of sets of functional dependencies ● Cover ● Canonical cover ● Normalization ● Unnormalized relation ● Atomic (or indivisible) values ● First normal form (1NF) ● Fully functional dependent ● Partial dependency ● Non-key or non-prime attribute | <ul style="list-style-type: none"> ● Second normal form (2NF) ● Transitive dependency ● Third normal form (3NF) ● Boyce-Codd normal form (BCNF) ● Universal relation ● Lossless-join property ● Lossy (lossy-join) decomposition ● Projection of F to R_i ● Dependency preservation property ● Non-additive property ● Binary decompositions ● All key relation ● Multivalued dependencies (MVDs) ● Multi-dependent ● Fourth normal form ● Join dependency ● Fifth normal form (project–join normal form (PJ/NF)) ● Denormalization |
|---|---|

● EXERCISES ●

A. Multiple Choice Questions

1. A relation which is in a higher normal form
 - (a) May not satisfy the conditions of lower normal form
 - (b) Is also be in lower normal form
 - (c) Is independent of lower normal form
 - (d) None of these

2. In a relation R , a functional dependency $X \rightarrow Y$ holds, then for X
 - (a) There is only one Y value
 - (b) There is none or one Y value
 - (c) There are many Y values
 - (d) Both (a) and (b)
3. An attribute A may be functionally dependent on
 - (a) No attribute
 - (b) A single attribute B
 - (c) A composite attribute (A, B)
 - (d) Both (b) and (c)
4. A 3NF relation will not be in BCNF if there are multiple candidate key in the relation such that
 - (a) The candidate keys in the relation are composite keys
 - (b) The keys are not disjoint, that is, candidate keys overlap
 - (c) Both (a) and (b)
 - (d) None of these
5. A relation in which every non-key attribute is dependent on primary key, but has transitive dependencies is in normal form
 - (a) Second
 - (b) Third
 - (c) BCNF
 - (d) None of these
6. If $A \rightarrow B$ holds, then $AC \rightarrow BC$ holds, which axiom is this?
 - (a) Augmentation rule
 - (b) Reflexivity rule
 - (c) Union rule
 - (d) None of these
7. The use of closure algorithm is to
 - (a) Determine if a particular FD, say $X \rightarrow Y$ is in the closure F^+ of F
 - (b) Test if a set of attributes A is a superkey of R
 - (c) Both of these
 - (d) None of these
8. 3NF and BCNF are identical if
 - (a) There is only one determinant upon which other attributes depend
 - (b) There is only one determinant upon which other attributes depend and it is a candidate key
 - (c) There are more than one candidate key and they overlap
 - (d) None of these
9. The set of inference rules for multivalued dependencies (MVDs) consists of three Armstrong axioms and _____ additional rules.
 - (a) Three
 - (b) Four
 - (c) Five
 - (d) Six
10. The decomposition of a relation $R(A, B, C, D)$ having FDs $A \rightarrow B$ and $C \rightarrow D$ into $R_1(A, B)$ and $R_2(C, D)$ is
 - (a) Only lossless-join
 - (b) Only dependency preserving
 - (c) Both of these
 - (d) None of these
11. Which of the following is a key for relation $R(A, B, C, D, E, F)$ having FDs $A \rightarrow B$, $C \rightarrow F$, $E \rightarrow A$ and $EC \rightarrow D$?
 - (a) AD
 - (b) AE
 - (c) EC
 - (d) AB

B. Fill in the Blanks

1. _____ provides the guidelines to assess the quality of a database in designing process.
2. While designing a relational database schema, the problems that are confronted due to _____ and _____ values can be resolved by decomposition.
3. _____ are the special forms of integrity constraints that generalize the concept of keys.
4. The set of all FDs that are implied by a given set F of FDs is called the _____.
5. Fifth normal form is based on the concept of _____.
6. An attribute is known as _____, if it is not a part of candidate key of R .
7. Two additional properties that must hold on decomposition to qualify a design as a good design are _____ and _____.
8. A relation can always be decomposed into _____ in such a way that the decomposition is lossless and dependency preserving.
9. Any all key relation must necessarily be in _____.
10. The process of taking a schema from higher normal form to lower normal form is known as _____.

C. Answer the Questions

1. Explain insertion, deletion, and modification anomalies with example.
2. A relation having many null values is considered as bad. Explain.
3. Define functional dependency. Explain its role in the process of normalization.
4. Why some functional dependencies are called trivial?
5. When are two sets of functional dependencies equivalent? How can we determine their equivalence?
6. Define the closure of a set of FDs and the closure of a set of attributes under a set of FDs.
7. What does it mean to say that Armstrong's inference rules are complete and sound?
8. Are normal forms alone sufficient as a condition for a good schema design? Explain.
9. Why is it necessary to decompose a relation?
10. What is meant by unnormalized relation?
11. What undesirable dependencies are avoided when a relation is in 2NF?
12. An instance of a relation schema $R(A, B, C)$ has distinct values for attribute A . Can A be considered as a candidate key of R ?
13. Define Boyce-Codd normal form. Why is it considered a stronger form of 3NF?
14. "If a relation is broken into BCNF, it will be lossless and dependency preserving". Prove or disprove this statement with the help of an example. Compare BCNF with fourth normal form.
15. Given a relation schema $R(A, B)$ without any information about FDs involved. Can you determine its normal form? Justify your answer.
[Hint: A binary relation is in BCNF]
16. What are the two important properties of decomposition? Out of these two, which one must always be satisfied?

17. What are the design goals of a good relational database design? Is it always possible to achieve these goals? If some of these goals are not achievable, what alternate goals should you aim for and why?
18. What is multivalued dependency? How is 4NF related to multivalued dependency? Is 4NF dependency preserving in nature? Justify your answer.
19. Define fourth normal form. Explain why it is more desirable than BCNF.
20. Discuss join dependencies and fifth normal form with example.
21. What is meant by denormalization? Why is it required?

D. Practical Questions

1. A relation $R(A, B, C, D, E, F, G)$ satisfies the following FDs.

$A \rightarrow B$

$BC \rightarrow DE$

$AEF \rightarrow G$

Find the closure $\{A, C\}^+$ under this set of FDs. Is FD $ACF \rightarrow DG$ implied by this set?

2. Consider a relation $R(A, B, C, D, E)$ with the following functional dependencies $AB \rightarrow C$, $CD \rightarrow E$ and $DE \rightarrow B$. Is AE a candidate key of this relation? If not, which is the candidate key? Explain.
3. Consider two sets of functional dependencies $F_1 = \{A \rightarrow C, AC \rightarrow D, E \rightarrow AD, E \rightarrow H\}$ and $F_2 = \{A \rightarrow CD, E \rightarrow AH\}$. Are they equivalent?
4. A relation $R(A, B, C, D, E, F)$ satisfies the following FDs.

$AB \rightarrow C$

$C \rightarrow A$

$BC \rightarrow D$

$ACD \rightarrow B$

$BE \rightarrow C$

$CE \rightarrow FA$

$CF \rightarrow BD$

$D \rightarrow EF$

- (i) Find the closure of the given set of FDs.
 - (ii) List the candidate keys for R .
 - (iii) Find the canonical cover for this set of FDs.
5. Consider a relation $R(A, B, C, D, E, F, G, H, I, J, K)$ with following FDs.

$AI \rightarrow BFG$

$I \rightarrow K$

$IC \rightarrow ADE$

$BIG \rightarrow CJ$

$K \rightarrow HA$

- (i) Find a canonical cover of this set of FDs.
 - (ii) Find a dependency-preserving and lossless-join 3NF decomposition of R .
 - (iii) Is there a BCNF decomposition of R that is both dependency-preserving and also lossless? If so, find one such decomposition.

6. Examine the table shown here.

Employee ID	Branch No.	Branch Address	Name	Position	Hrs/Week
E101	B02	Sun Plaza, Atlanta, 46227	Steve	Assistant	15
E101	B04	2/3 UT, New York, 46238	Steve	Assistant	10
E122	B02	Sun Plaza, Atlanta, 46227	John	Assistant	14
E122	B04	2/3 UT, New York, 46238	John	Assistant	9

(i) Is this table in 2NF? If not, Why?

(ii) Describe the process of normalizing the data shown in the table to third normal form (3NF).

(iii) Identify the primary and foreign keys in 3NF relations.

7. A relation $R(A, B, C, D, E, F)$ have the following set of functional dependency.

$A \rightarrow CD$

$B \rightarrow C$

$F \rightarrow DE$

$F \rightarrow A$

Is the decomposition of R in $R_1(A, B, C)$, $R_2(A, F, D)$, and $R_3(E, F)$ dependency preserving and losses decomposition?

8. Consider a relation $R(A, B, C, D, E, F, G, H)$ having the following set of dependency.

$A \rightarrow BCDEFGH$

$BCD \rightarrow AEFGH$

$BCE \rightarrow ADEFGH$

$CE \rightarrow H$

$CD \rightarrow H$

Find BCNF decomposition of R . Is it dependency preserving?

9. Give an example of a relation schema R and a set of functional dependencies such that R is in BCNF, but not in 4NF.

10. Consider the relation R as shown here.

A	B	C
a	b	c
d	b	c
e	c	c
e	c	d

Which of the following functional and multivalued dependencies does not hold over relation R?

- $A \rightarrow B$
- $A \twoheadrightarrow B$
- $B \rightarrow C$
- $B \twoheadrightarrow C$
- $BC \rightarrow A$
- $BC \twoheadrightarrow A$

11. Give a set of FDs for the relation schema $R(A, B, C, D)$ with primary key AB under which R is in 2NF but not in 3NF.
12. A database is designed as a single relation consisting of the attributes ($Stud_id, S_name, S_add, Course_no, Course_title, Teacher_name, marks_obt$). Apply normalization to improve the design.
13. Consider the following relation for an order-processing system.
 $Order(CustomerNo, Customer_name, Customer_city, Order_no, Order_date, Total_amount, Item_no, Quantity, Item_price)$
 - (i) What normal form is the relation in? Explain your answer.
 - (ii) Apply normalization until you cannot decompose the relation further. (State assumptions, if you make any.)
14. Consider the universal relation $R(A, B, C, D, E, F, G, H, I, J)$ having the following set of functional dependencies.

- $AB \rightarrow C$
- $A \rightarrow DE$
- $B \rightarrow F$
- $F \rightarrow GH$
- $D \rightarrow IJ$

What is the key for R? Decompose R into 2NF, and then 3NF relations.

15. Consider a database is required to store following information regarding sales representatives, sales areas, and products.
 - $\Rightarrow$ Each representative is responsible for sales in one or more areas.
 - $\Rightarrow$ Each area has one or more responsible representatives.
 - $\Rightarrow$ Each representative is responsible for sales of one or more products and each product has one or more responsible representatives.
 - $\Rightarrow$ Every product is sold in every area; No two representatives sell the same product in the same area.
 - $\Rightarrow$ Every representative sells the same set of products in every area for which that a representative is responsible.

Design the relation schemas for the database that are each in 3NF or BCNF.

DATA STORAGE AND INDEXING

After reading this chapter, the reader will understand:

- Different types of storage devices including primary storage, secondary storage, and tertiary storage
- Important characteristics of storage media
- Magnetic disk and its organization
- Seek time, rotational delay, and data transfer time
- Redundant array of independent disk, that is, RAID
- Performance and reliability improvement of disk with RAID
- All the RAID levels from level 0 to 6
- New storage systems
- The concept of files and pages
- Responsibilities of buffer manager
- Management of buffer space
- Page-replacement policies
- Fixed-length and variable-length records
- Spanned and unspanned organization of records in disk blocks
- Various file organizations including heap file organization, sequential file organization, and hash file organization
- Hashing (both static and dynamic), hash function, and hash table
- Collision and collision resolution
- The concept of bucket
- Primary index, clustering index, and secondary index
- Multilevel indexes including B-tree and B⁺-tree
- Indexes on multiple keys

In the previous chapters, we have viewed the database as a collection of tables (relational model) at the conceptual or logical level. The database users are largely concerned with the logical level of the database as they have hardly to do anything with the physical details of the implementation of the database system. The database is actually stored on a storage medium in the form of *pages*, which is a collection of records consisting of related data items such as name, address, phone, email-id, and so on.

In this chapter, we start our discussion with the characteristics of the underlying storage medium. There are various media available to store data; however, database is typically stored on the magnetic disk because of its higher capacity and persistent storage. This chapter also discusses a number of methods used to organize *pages* such as heap, sequential, index sequential, direct, etc., and the issues involved in the choice of a method. Further, the goal of a database system is to simplify and facilitate efficient access to data. One such technique is indexing, which is also discussed in this chapter.

7.1 HIERARCHY OF STORAGE DEVICES

In a computer system, several types of storage media such as cache memory, main memory, magnetic disk, etc. exist. On the basis of their characteristics such as cost per unit of data and speed with which data can be accessed, they can be arranged in a hierarchical order (see Figure 7.1). In addition to the

speed and cost of the various storage media, another issue is volatility. **Volatile storage** loses its contents when power supply is switched off, whereas **non-volatile storage** does not.

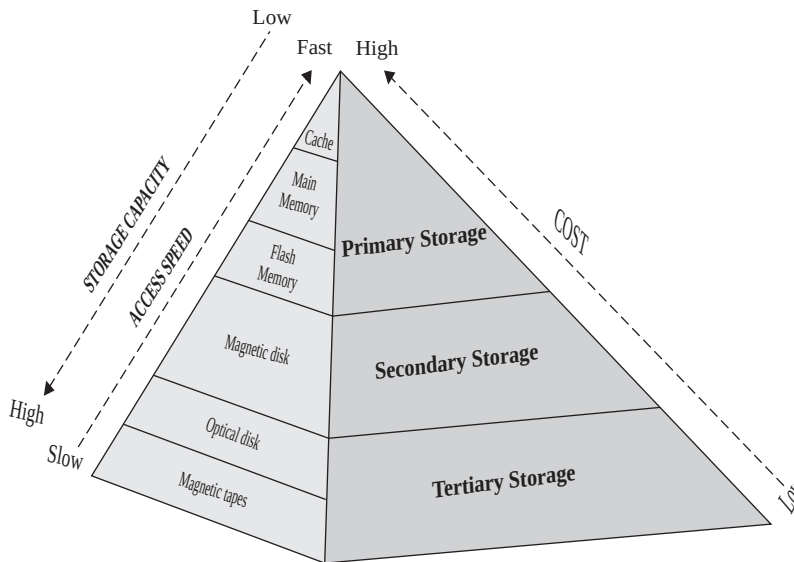


Fig. 7.1 Storage device hierarchy

The cost per unit of data as well as the speed of accessing data decreases while moving down in the memory hierarchy. The storage media at the top such as cache and main memory is the highest speed memory and is referred to as **primary storage**. The storage media in the next level consists of slower devices, such as magnetic disk, and is referred to as **secondary storage**. The storage media in the lowest level is the slowest storage devices, such as optical disk and magnetic tape, and are referred to as **tertiary storage**. Let us discuss these storage devices in detail.

Primary Storage

Primary storage generally offers limited storage capacity due to its high cost but provides very fast access to data. Primary storage is usually made up of semiconductor chips. Semiconductor memory may be volatile or non-volatile in nature. The two basic forms of semiconductor memory are static RAM or SRAM (used for cache memory) and dynamic RAM or DRAM (used for main memory).

- ⇒ **Cache memory:** Cache memory is a static RAM and is very small in size. The cache memory is the fastest; however, most expensive among all the storage media available. It is directly accessed by the CPU and is basically used to speed up the execution. However, major limitation of cache is high cost and its volatility.
- ⇒ **Main memory:** Main memory is a dynamic RAM (DRAM) and is used to keep the data that is currently being accessed by the CPU. Its storage capacity typically ranges from 64 MB to 512 MB or sometimes it is extended up to hundreds of gigabytes. It offers fast access to data as compared to other storage media except static RAM. However, it is expensive and offers limited storage capacity. Thus, entire database cannot be placed in main memory at once. Moreover, it is volatile in nature.

Another form of semiconductor memory is **flash memory**. Flash memory uses an electrical erasing technology (like EEPROM). Reading data from flash memory is as fast as reading it from

main memory. However, it requires an entire block to be erased and written at once. This makes the writing process little complicated.

Flash memory is non-volatile in nature and the data survives system crash and power failure. This type of memory is commonly used for storing data in various small appliances such as digital cameras, MP3 players, USB (Universal Serial Bus) storage accessories, etc.

Secondary Storage

Secondary storage devices usually provide bulk of storage capacity for storing data but are slower than primary storage devices. Secondary storage is non-volatile in nature, that is, the data is permanently stored and survives power failure and system crashes. Data on the secondary storage are not directly accessed by the CPU. Instead, data to be accessed must move to main memory so that it can be accessed. **Magnetic disk** (generally called disk) is the primary form of secondary storage that enables storage of enormous amount of data. It is used to hold on-line data for a long term. Generally, the entire database is placed on magnetic disk. Magnetic disk is discussed in detail in Section 7.1.1.

Tertiary Storage

Removable media, such as optical discs and magnetic tapes, are considered as tertiary storage. Larger capacity and least cost are major advantage of tertiary storage. Data access speed; however, is much slower than primary storage. Since these devices can be removed from the drive, they are also termed as **off-line storage**.

- ⇒ **Optical disc:** Optical disc comes in various sizes and capacities. A compact disc (CD-ROM) with a capacity of 600–700 MB of data having 12 cm diameter is the most popular means of optical disc. Another most common type of optical disc with high-storage capacity is the DVD-ROM, which stands for digital video disc (read only memory). It can store up to 8.5 GB of data per side and DVD allows for double-sided discs. As the name suggests, both the CD-ROM and DVD-ROM come pre-recorded with data, which cannot be altered. However, both use laser beams for performing read operations.
- ⇒ **Tape storage:** Tape storage is cheap and reliable storage medium for organizing archives and taking backup. However, tape storage is not suitable for data files that need to be revised or updated often because it stores data in a sequential manner and offers only **sequential-access** to the stored data. Tape now has a limited role because disk has proved to be a superior storage medium. Furthermore, disk data allows **direct-access** to the stored data. Table 7.1 lists some of the important characteristics of various storage media available in computer system.

Table 7.1 Important characteristics of storage media

Device Name	Capacity	Nature	Cost	Access Method	Storage
Cache memory	Very small	Volatile	High	Random	On-line data
Main memory	Small	Volatile	High	Random	On-line data
Flash memory	Medium	Non-volatile	Moderate	Random	On-line data
Magnetic disk	Very large	Non-volatile	Low	Direct	On-line data
Optical disc	Large	Non-volatile	Low	Direct	Off-line data
Magnetic tapes	Large	Non-volatile	Low	Sequential	Off-line data

7.1.1 Magnetic Disks

A magnetic disk is the most commonly used secondary storage medium. It offers high storage capacity and reliability. It can easily accommodate entire database at once. However, if the data stored on the disk needs to be accessed by CPU it is first moved to the main memory and then the required operation is performed. Once the operation is performed, the modified data must be copied back to the disk for consistency of the database. The system is responsible for transferring the data between disk and the main memory as and when required. Data on the disk survives power failures and system crash. There is a chance that disk may sometimes fail itself and destroy the data; however, such failures occur rarely.

Data is represented as magnetized spots on a disk. A magnetized spot represents a 1 (bit) and the absence of a magnetized spot represents a 0 (bit). To read the data, the magnetized data on the disk is converted into electrical impulses, which is transferred to the processor. Writing data onto the disk is accomplished by converting the electrical impulses from the processor into magnetic spots on the disk. The data in a magnetic disk can be erased and reused virtually infinitely. The disk is designed to reside in a protective case or cartridge to shield it from the dust and other external interference.

Organization of Magnetic Disks

A magnetic disk consists of **plate/platter**, which is made up of metal or glass material, and its surface is covered with magnetic material to store data on its surface. If the data can be stored on only one side of the platter, the disk is **single-sided disk**, and if both sides are used to hold the data, the disk is **double-sided disk**. When the disk is in use, the spindle motor rotates the platters at a constant high speed. Usually the speed at which they rotate is 60, 90, or 120 revolutions per second.

Disk surface of a platter is divided into imaginary tracks and sectors. **Tracks** are concentric circles where the data is stored, and are numbered from the outermost to the innermost ring, starting with zero. There are about 50,000 to 100,000 tracks per platter and a disk generally has 1 to 5 platters. Tracks are further subdivided into sectors (or track sector). A **sector** is just like an arc that forms an angle at the center. It is the smallest unit of information that can be transferred to/from the disk. There are about hundreds of sectors per track and the sector size is typically 512 bytes. The inner tracks are of smaller length than the outer tracks, thus, there are about 500 sectors per track in the inner tracks, and about 1000 sectors per track towards the boundary. In general, the disk containing large number of tracks on each surface of platter and more sectors per track has higher storage capacity.

A disk contains one **read-write head** for each surface of a platter, which is used to store and retrieve data from the surface of platter. Information is magnetically stored on a sector by the read-write head. The head moves across the surface of platter to access different tracks. All the heads are attached to a single assembly called a **disk arm**. Thus, all the heads of different platters move together. The disk platters mounted on a **spindle** together with the heads mounted on a disk arm is known as **head-disk assemblies**. All the read-write heads are on the equal diameter track on different platters at one time. The tracks of equal diameter on different platters form a **cylinder**. Accessing data of one cylinder is much faster than accessing data that is distributed among different cylinders. A close look at the internal structure of magnetic disk is shown in Figure 7.2.

Learn More

Some disks have one read-write head for each track of platter. These disks are termed as **fixed-head** disks, since one head is fixed on each track and is not moveable. On the other hand, disks in which the head moves along the platter surface are termed as **moveable-head** disks.

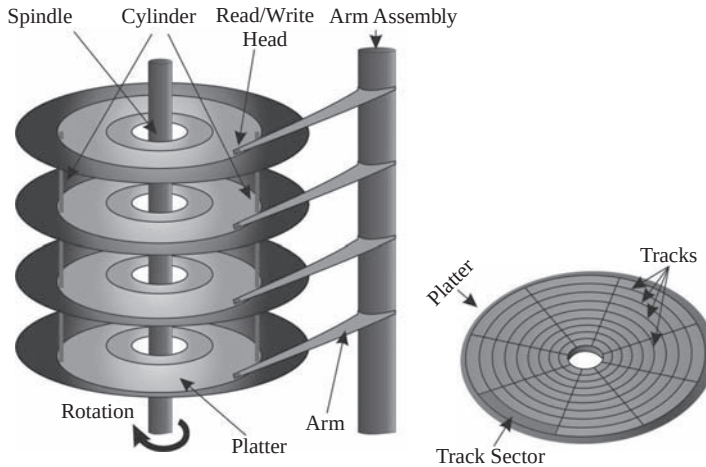


Fig. 7.2 Moving head disk mechanism

Accessing Data from Magnetic Disk

Data in a magnetic disk is recorded on the surface of the circular tracks with the help of read/write head, which is mounted on the arm assembly. These heads can be multiple in numbers to access the adjacent tracks simultaneously and thus access to a disk is faster. The transfer of data between memory and disk drive is handled by a **disk controller**, which interfaces the disk drive to the computer system.

Some common interfaces used for disk drives on personal computers and workstations are **small-computer-system interconnect (SCSI)** (pronounced 'scuzzy'), **AT attachment (ATA)**, and **serial ATA (SATA)**. In the latest technology, disk controller is implemented within the disk drive. The controller accepts high level I/O commands (to read or write sector) and start positioning the disk arm over the right track in order to read or write the data. Disk controller computes the **checksums** for the data to be written on the sector and attach it with the sector. When the sector is to be read, the controller again computes the checksum from the sector data and compares it with stored checksum. If there is any difference between them, the controller will retry reading data several times. However, if the difference between computed and stored checksum continues to occur, the controller signals a read failure.

Remapping of bad sectors is another major task performed by the disk controllers. During the initial formatting of disk, if the controller detects a bad (or damaged) sector, it is logically mapped to another physical location by the disk controller. Disk is notified for the remapping and any further operation is carried out on the new location.

The process of accessing data comprises three steps:

1. **Seek:** As soon as the disk unit receives the read/write command, the read/write heads are positioned on specific track on the disk platter. The time taken in doing so is known as **seek time**. It is average time required to move the heads from one track to some other desired track on the disk. Seek times of modern disk may range between 6 to 15 milliseconds.
2. **Rotate:** Once the heads are positioned on the desired track, the head of the specific platter is activated. Since the disk is rotated constantly, the head has to wait for the required sector or cluster (desired data) to come under it. This delay is known as **rotational delay time** or **latency** of the disk. The average rotational latencies range from 4.2 to 6.7 ms.

3. **Data transfer:** After waiting for the desired data location, the read/write head transfers the data to or from the disk to primary memory. The rate at which the data is read from or written to the disk is known as **data transfer rate**. It is measured in kilobits per second (kbps). Some of the latest hard disks have a data transfer rate of 66 MB/second. The data transfer rate depends upon the rotational speed of the disk. If the disk has a rotational speed of 6000 rpm (rotations per minute), having 125 sectors and 512 bytes/sector, the data transfer rate per revolution will be $125 \times 512 = 64000$ bytes. Hence, the total transfer rate per second will be $64000 \times 6000/60 = 6,400,000$ bytes/second or 6.4 MB/second.

The combined time (seek time, latency time, and data transfer time) is known as the **access time**. Specifically, it can be described as the period of time that elapses between a request for information from disk or memory, and the information arriving at the requesting device. **Memory access time** refers to the time it takes to transfer a character from memory to or from the processor, while **disk access time** refers to the time it takes to place the read/write heads over the given sector and transfer the data to or from the requested device. RAM may have an access time of 80 nanoseconds or less, while disk access time could be 12–19 milliseconds.

The **reliability** of the disk is measured in terms of **mean time to failure (MTTF)**. It is the amount of time for which the system can run continuously without any failure. Manufacturers claim that the mean time to failure of disks ranges between 1,000,000 hours (about 116 years) to 1,500,000 hours (about 174 years). Although, various research studies conclude that failure rates are, in some cases, 13 times greater than what manufacturer claims. Generally, expected life span of most disks is about 4 to 5 years. However, disks have a high rate of failure when they become a few years old.

7.2 REDUNDANT ARRAYS OF INDEPENDENT DISKS

The technology of semiconductor memory has been advancing at a much higher rate than the technology of secondary storage. The performance and capacity of semiconductor memory is much superior to secondary storage. To match this growth in semiconductor memory, a significant development is required in the technology of secondary storage. A major advancement in secondary storage technology is represented by the development of **Redundant Arrays of Independent Disks (RAID)**. The basic idea behind RAID is to have a large array of small independent disks. Presence of multiple disks in the system improves the overall transfer rates, if the disks are operated in parallel. Parallelizing the operation of multiple disks allow multiple I/O to be serviced in parallel. This setup also offers opportunities for improving the reliability of data storage, because data can be stored redundantly on multiple disks. Thus, failure of one disk does not lead to loss of data. In other words, this large array of independent disks acts as a single logical disk with improved performance and reliability.

Improving Performance and Reliability with RAID

In order to improve the performance of disk, a concept called **data striping** is used which utilizes parallelism. Data striping distributes the data transparently among N disks, which make them appear as a single large, fast disk. Striping of data across multiple disks improves the transfer rate as well, since operations are carried out in parallel. Data striping also balances the load among multiple disks.

In the simplest form, data striping splits each byte of data into bits and stores them across different disks. This splitting of each byte into bits is known as **bit-level data striping**. Having 8-bits per byte, an array of eight disks (or either a factor or multiple of eight) is treated as one large logical disk.

Learn More

Originally RAID stands for Redundant Array of Inexpensive Disk, since array of cheap smaller capacity disk was used as an alternative to large expensive disk. Those days the cost per bit of data of smaller disk was less than that of larger disk.

In general, bit i of each byte is written i^{th} disk. However, if an array of only two disks is used, all odd numbered bits go to first disk and even numbered bits to second disk. Since each I/O request is accomplished with the use of all disks in the array, transfer rate of I/O requests goes to N times, where N represents the number of disks in the array.

Alternatively, blocks of a file can be striped across multiple disks. This is known as **block-level striping**. Logical blocks of a file are assigned to multiple disks. Large requests for accessing multiple blocks can be carried out in parallel, thus improving data transfer rate. However, transfer rate for the request of a single block is same as it is in case of one disk. Note that the disks that are not participating in the request are free to carry out other operations.

Having an array of N disks in a system improves the system performance; however, lowers the overall storage system reliability. The chance of failure of at least one disk out of total N disks is much higher than that of a specific single disk. Assume that the mean time to failure (MTTF) of a disk is about 150,000 hours (slightly over 16 years). Then, for an array of 100 disks, the MTTF of some disk is only $150,000/100 = 1500$ hours (about 62 days). With such short MTTF of a disk, maintaining one copy of data in an array of N disks might result in loss of significant information. Thus, some solution must be employed to increase the reliability of such storage system. The most acceptable solution is to have **redundant** information. Normally, the redundant information is not needed; however, in case of disk failure it can be used to restore the lost information of failed disk.

One simple technique to keep redundant information is **mirroring** (also termed as **shadowing**). In this technique, the data is redundantly stored on two physical disks. In this way every disk is duplicated, and all the data has two copies. Thus, every write operation is carried on both the disks. During read operation, the data can be retrieved from any disk. In case of failure of one disk, second disk can be used until the first disk gets repaired. If the second disk also fails before the repairing of first disk is completed, the data is lost. However, occurrence of such event is very rare. The mean time to failure of mirrored disk depends on two factors.

1. **Mean time to failure** of independent disks and
2. **Mean time to repair** a disk. It is the time taken, on an average, to restore the failed disk or to replace it, if required.

Suppose failure of two disks is independent of each other. Further, assume that the mean time to repair of a disk is 15 hours and mean time to failure of single disk is 150,000 hours, then mean time to data loss in mirrored disk system is $(150,000)^2/(2 * 15) = 7.5 * 10^8$ hours or about 85616 years.

An alternative solution to increase the reliability is storing error-correcting codes such as parity bits and hamming codes (discussed in next section). Such additional information is needed only in case of recovering the data of failed disk. Error-correcting codes are maintained in a separate disk called **check disk**. The parity bits corresponding to each bit of N disks are stored in check disk.

7.2.1 RAID Levels

Data striping and mirroring techniques improve the performance and reliability of a disk. However, mirroring is expensive and striping does not improve reliability. Thus, several RAID organizations referred to as **RAID levels** have been proposed which aim at providing redundancy at lower cost by using the combination of these two techniques. These levels have different cost-performance trade-offs. The RAID levels are classified into seven levels (from level 0 to level 6), as shown in Figure 7.3, and are discussed here. To understand all the RAID levels consider a disk array consisting of 4 disks.

⇒ **RAID level 0:** RAID level 0 uses block-level data striping but does not maintain any redundant information. Thus, the write operation has the best performance with level 0, as only one copy of data is maintained and no redundant information needs to be updated. However, RAID level 0

does not have the best read performance among all the RAID levels, since systems with redundant information can schedule disk access based on shortest expected seek time and rotational delay. In addition, RAID level 0 is not fault-tolerant because failure of just one drive will result in loss of data. However, absence of redundant information ensures 100 per cent space utilization for RAID level 0 systems.

- ⇒ **RAID level 1:** RAID level 1 is the most expensive system as this level maintains duplicate or redundant copy of data using mirroring. Thus, two identical copies of the data are maintained on two different disks. Every write operation needs to update both the disks, thus, the performance of RAID level 1 system degrades while writing. However, performance while read operation is improved by scheduling request to the disk with the shortest expected access time and rotational delay. With two identical copies of data, RAID level 1 system ensures only 50 per cent space utilization.
- ⇒ **RAID level 2:** RAID level 2 is known as **error-correcting code (ECC)** organization. Two most popular error detecting and correcting codes are parity bits and hamming codes. In memory system, each byte is associated with a parity bit. The parity bit is set to 0 if the number of bits in the byte that are set to 1 is even; otherwise the parity bit is set to 1. If any one bit in the byte gets changed, then parity of that byte will not match with the stored parity bit. In this way, use of parity bit detects all 1-bit errors in the memory system. Hamming code has the ability to detect the damaged bit. It stores two or more extra bits to find the damaged bit and by complementing the value of damaged bit, the original data can be recovered. RAID level 2 requires three redundant disks to store error detecting and correcting information for four original disks. Thus, effective space utilization is about 57 per cent in this case. However, space utilization increases with the number of data disks because check disks grow logarithmically with the number of data disks.
- ⇒ **RAID level 3:** As discussed, RAID level 2 uses check disks to hold information to detect the failed disk. However, disk controllers can easily detect the failed disk and hence, check disks need not to contain information to detect the failed disk. RAID level 3 maintains only single check disk with parity bit information for error correction as well as for detection. This level is also named as **bit-interleaved parity organization**.

Performance of RAID level 2 and RAID level 3 are very similar but RAID level 3 has lowest possible overheads for reliability. So, in practice, level 2 is not used. In addition, level 3 has two benefits over level 1. Level 1 maintains one mirror disk for every disk, whereas level 3 requires only one parity disk for multiple disks, thus, increasing effective space utilization. In addition, level 3 distributes the data over multiple disks, with N-way striping of data, which makes the transfer rate for reading or writing a single block by N times faster than level 1. Since every disk has to participate in every I/O operation, RAID level 3 supports lower number of I/O operations per second than RAID level 1.

- ⇒ **RAID level 4:** Like RAID level 0, RAID level 4 uses block-level striping. It maintains parity block on a separate disk for each corresponding block from N other disks. This level is also named as **block-interleaved parity organization**. To restore the block of the failed disk, blocks from other disks and corresponding parity block is used. Requests to retrieve data from one block are processed with only one disk, leaving remaining disks free to handle other requests. Writing a single block involves one data disk and check disk. The parity block is required to be updated with each write operation, thus, only one write operation can be processed at a particular point of time.

With four data disks, RAID level 4 requires just one check disk. Effective space utilization for our example of four data disk is 80 per cent. However, as always one check disk is required to hold parity information, effective space utilization increases with the number of data disks.

- ⇒ **RAID level 5:** Instead of placing data across N disks and parity information in one separate disk, this level distributes the **block-interleaved** parity and data among all the N+1 disks. Such distribution has advantage in processing read/write requests. All disks can participate in processing

read request, unlike RAID level 4, where dedicated check disks never participates in read request. So level 5 can satisfy more number of read requests in given amount of time. Since bottleneck of single check disk has been eliminated, several write request could also be processed in parallel. RAID level 5 has the best performance among all the RAID levels with redundancy. In our example of 4 actual disks, RAID level 5 has 5 disks overall, thus, effective space utilization for level 5 is same as in level 3 and level 4.

⇒ **RAID level 6:** RAID level 6 is an extension of RAID level 5 and applies $P + Q$ redundancy scheme using Reed-Solomon codes. **Reed-Solomon codes** enable RAID level 6 to recover from up to two simultaneous disk failures. RAID level 6 requires two check disks; however, like RAID level 5, redundant information is distributed across all disks using block-level striping.

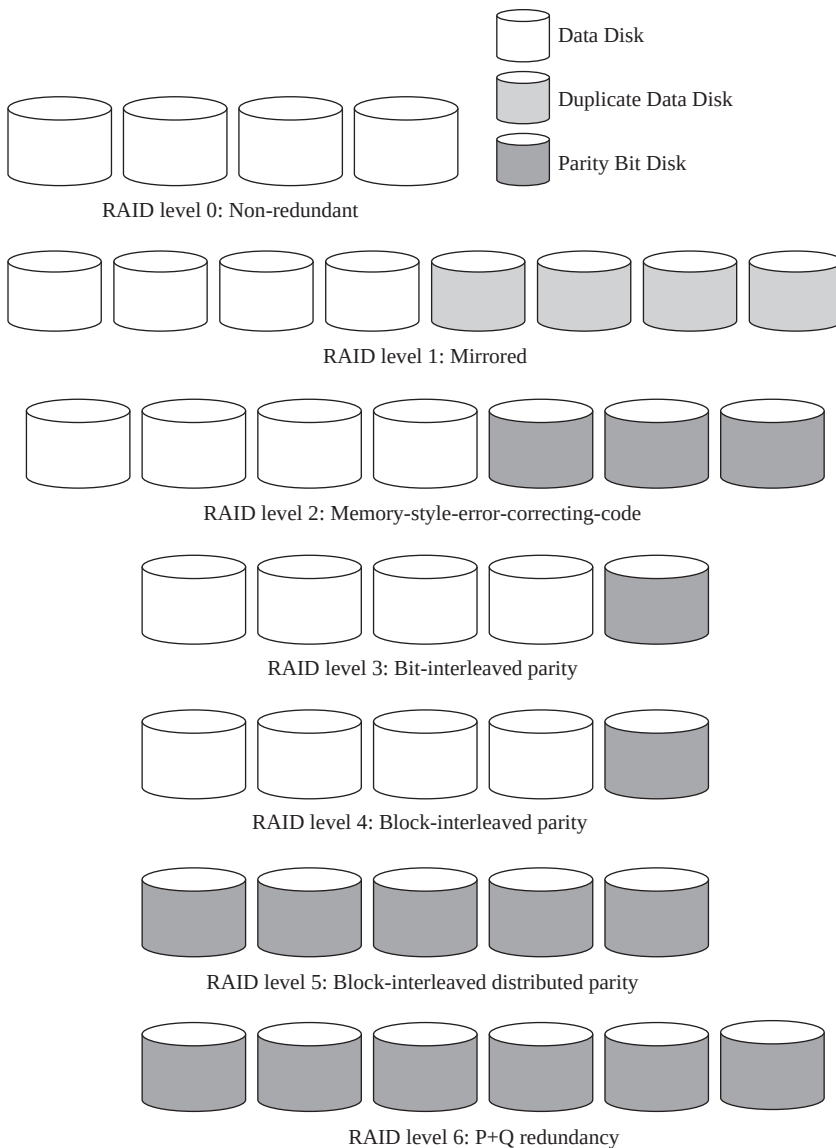


Fig. 7.3 Representing RAID levels

7.3 NEW STORAGE SYSTEMS

Demand for storage of data is increasing across the organizations with the advent of Internet-driven applications such as e-commerce. Enterprise Resource Planning (ERP) systems and data warehouses need enough space to keep bulk data or information across the organization. In addition, the organizations have different branches across the world and providing data to different users across different branches in 24×7 environment is a challenging task. As a result, managing all the data becomes costly with an increase in simultaneous requests. In some cases, cost of managing server-attached storage exceeds the cost of the server itself. Therefore, various organizations choose the concept called **storage area network (SAN)** to manage their storage.

In SAN architecture, many server computers are connected with a large number of disks on a high-speed network. Storage disks are placed at a central server room, and are monitored and maintained by system administrators. The storage subsystem and the computer communicate with each other by using SCSI or fiber channel interfaces (FCI). Several SAN providers came up with their own topologies, thus, providing different performance and connectivity options allowing storage subsystem to be placed far apart from the server computers. Storage subsystem is shared among multiple computers that could process different parts of an application in parallel. Servers are connected to multiple RAID systems, tape libraries, and other storage systems in different configurations by using fiber-channel switches and fiber-channel hubs. Thus, the main advantage is many-to-many connectivity among server computers and storage system. Furthermore, isolation capability of SAN allows easy addition of new peripherals and servers.

An alternative to SAN is **network-attached storage (NAS)**. A NAS device is a server that allows file sharing. NAS does not provide any of the services that a typical server provides. NAS devices are being used for high performance storage solutions at low cost. NAS devices allow enormous amount of hard disk storage space to be made available to multiple servers without shutting them down for maintenance and upgrades. The NAS devices do not need to be located within the server, but can be placed anywhere in a local area network (LAN) and may be combined in different configurations. A single hardware device (called the **NAS box** or **NAS head**) acts as the interface between the NAS system and network clients. NAS units usually have a web interface and do not require a monitor, keyboard, or mouse. In addition, NAS system usually contains one or more hard disks or tape drives, often arranged into logical, redundant storage containers or RAID, as traditional file servers do. Further, NAS removes the file serving responsibility from other servers on the network.

NAS provides both storage and file system. It is often contrasted with SAN, which provides only block-based storage and leaves file system concern on the client side. The file based protocols used by NAS are NFS or SMB, whereas SAN protocols are SCSI or fiber channel. NAS increases the performance as the file serving is done by the NAS and not by the server, which is responsible for doing other processing. The performance of NAS devices depends heavily on the speed of and traffic on the network, and also on the amount of cache memory on the NAS devices.

7.4 ACCESSING DATA FROM DISK

The database is mapped into a number of different files, which are physically stored on disk for persistency and the underlying operating system is responsible for maintaining these files. Each file is decomposed into equal size **pages**, which is the unit of exchange between the disk and the main memory. The size of the page chosen is equal to the size of disk block and a page can be stored in any disk block. Hence, reading or writing a page can be done in one disk I/O. Generally, main memory is too small to hold all the pages of a file at one time. Thus, the pages of a file need to be transferred into main memory as and when requested by the CPU. If there is no free space in main memory, some

existing page must be replaced to make space for the new page. The policy that is used to choose a page to be replaced with the new page is called the **replacement policy**.

To improve the performance of a database, it is required to keep as many pages in the main memory as possible. Since it is not possible to keep all the pages in main memory at a time, the available space of main memory should be managed efficiently. The goal is to minimize the number of disk accesses by transferring the page in the main memory before a request for that page occurs in the system. The main memory space available for storing the data is called **buffer pool** and the subsystem that manages the allocation of buffer space is called **buffer manager**. The available main memory is partitioned into page size **frames**. Each frame can hold a page of file at any point of time.

7.4.1 BufferManager

When a request for a page is generated, the buffer manager handles it by passing the memory address of the frame that contains the requested page to its requester, if the page is already in the buffer pool. However, if the page is not available in buffer pool, buffer manager allocates space for that page. If no free frame is available in buffer pool, allocation of space requires de-allocation of some other pages that are no longer needed by the system. After allocating the space in buffer pool, buffer manager transfers the requested page from disk to main memory. While de-allocating a page, the buffer manager writes back the updated information of that page on the disk, if the page has been modified since its last access from the disk (see Figure 7.4).

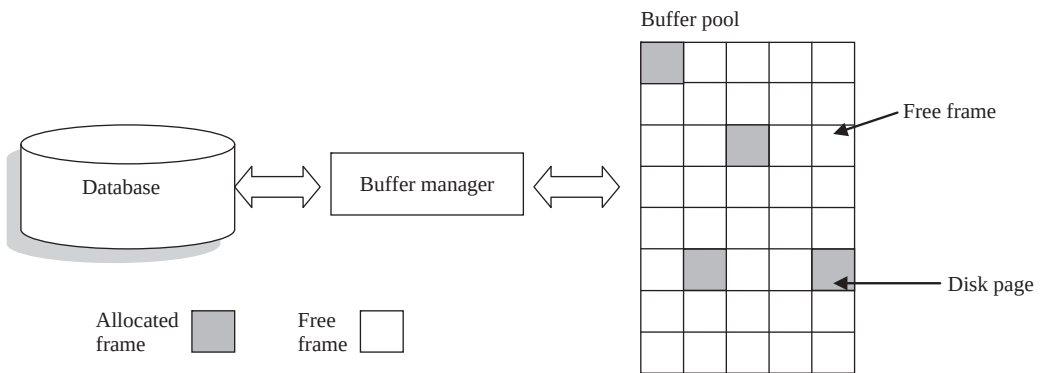


Fig. 7.4 Buffer management

In addition to the buffer pool, the buffer manager maintains two variables for each frame in the buffer pool.

- ⇒ **pin_count**: This variable keeps track of the number of times the page currently in a given frame has been requested but not released. In this way, it records the number of current users of the page.
- ⇒ **dirty**: It is a Boolean variable that keeps track whether the current page in the frame has been modified or not, since it was brought into the frame from disk.

Initially, the value of **pin_count** is set to 0 and **dirty** bit is turned off for every frame. On receiving a request for a page, the buffer manager first searches the buffer pool for that page. If the requested page is available in any frame, its **pin_count** is incremented by one. If no frame in the buffer pool is holding the requested page, buffer manager chooses a frame for replacement using replacement policy. If the **dirty** bit is on for the replacement frame, the page in that frame is written

back to the disk and then, the requested page is transferred from disk to that frame in the buffer pool. After having the requested page in buffer pool, buffer manager passes the memory address of frame that contains the requested page to its requestor.

The process of incrementing the `pin_count` is generally called **pinning** the page. When the requested page is subsequently released by the process, `pin_count` of that frame is decremented. It is called **unpinning** the page. At the time of unpinning the page, the `dirty` bit needs to be set by the buffer manager, if the page has been modified by its requestor. Note that the buffer manager will not replace the page in the frame until it is unpinned by all its requestor, that is, until its `pin_count` becomes 0.

For each new page from disk, buffer manager always choose a frame with `pin_count` 0 to be replaced, if no free frame is available. In case two or more such frames are found, then buffer manager chooses a frame according to its replacement policy.

7.4.2 Page-Replacement Policies

Buffer manager uses replacement policy to choose a page for replacement from the list of unpinned pages. The main goal of replacement policies is to minimize the number of disk accesses for the requested page. For general-purpose programs, it is not possible to anticipate accurately which page will be requested in near future and hence, operating system uses the pattern of past references to predict the future references.

Commonly used replacement policies include **least recently used (LRU)**, **most recently used (MRU)**, and **clock replacement**. LRU replacement policy assumes that the pages that have been referenced recently have greater chance to be referenced again in the near future. Thus, the least recently referenced page should be chosen for replacement. However, the pattern of future references is more accurately predicted by the database system than an operating system. A user-request to the database system involves several steps and by carefully looking at these steps, database system can apply this policy more efficiently than operating system.

In some cases, the optimal policy to choose a page for replacement is MRU policy. As the name indicates, MRU chooses the most recently accessed page for replacement, as and when required. Note that the page currently being used by the system must be pinned and hence, they are not eligible for replacement. After completing the processing with the page, it must be unpinned by the system so that it becomes available for replacement.

Another replacement policy is **clock replacement** policy, which is a variant of LRU. In this replacement policy, an additional **reference** bit is associated with each frame, which is set on as soon as `pin_count` of that frame becomes 0. All the frames are considered as arranged in a circular manner. A variable `current` moves around the logical circle of frames, starting with the value 1 through N (total number of frames in the buffer pool).

The clock replacement policy considers the `current` frame for replacement. If the considered frame is not chosen for replacement, value of `current` is incremented by one and the next frame is considered. The policy continues to consider the frame until some frame is chosen for replacement. If the `pin_count` of `current` frame is greater than 0, then the `current` frame is not a candidate for replacement. If the `current` frame has `pin_count` 0 and `reference` bit on, the algorithm turns it off—this helps to ignore the recently referenced pages to be chosen for replacement. Only the frame having `pin_count` 0 and `reference` bit off can be chosen for replacement. If in some sweep of clock, all the frames have `pin_count` greater than 0, it means no frame in the buffer pool is a candidate for replacement.

Learn More

Buffer manager should not attempt to choose the pages of indexes (discussed in Section 7.7) for replacement, until or unless necessary, because pages of indexes are generally accessed much frequently than the pages of the file itself.

Although, no single policy is suitable for all types of applications, a large number of database systems use LRU. The policy to be used by buffer manager is influenced by various factors other than time at which the page will be referenced again. If the database system is handling requests from several users concurrently, the system may need to delay certain requests by using some concurrency-control mechanism to maintain database consistency. Buffer manager needs to alter its replacement policy using information from concurrency control subsystem, indicating which requests are being delayed.

7.5 PLACING FILE RECORDS ON DISK BLOCKS

A database consists of a number of relations where each relation is typically stored as a file of records. Each record of a file represents an entity instance and its attributes. For example, each record of PUBLISHER relation of *Online Book* database consists of P_ID, Pname, Address, State, Phone, and Email_id fields. These file records are mapped onto disk blocks. The size of blocks are fixed and is determined by the physical properties of disk and by the operating system; however, the size of records may vary. Generally in the relational database, different relations have tuple of different sizes. Thus, before discussing how file records are mapped onto disk blocks, we need to discuss how database is represented in terms of files.

There are two approaches to organize the database in the form of files. In the first approach, all the records of a file are of fixed-length. However, in the second approach, the records of a file vary in size. A file of fixed-length records is simple to implement as all the records are of fixed-length. However, deletion of record results in fragmented memory. On the other hand, in case of files of variable-length records, memory space is efficiently utilized; however, locating the start and end of record is not simple.

7.5.1 Fixed-Length Records

All the records in a file of fixed-length record are of same length. Consider a relation PUBLISHER whose record is defined here.

```
create PUBLISHER as
    (P_ID          char(5)          PRIMARY KEY NOT NULL,
     Pname         char(20)         NOT NULL,
     Address       char(50),
     State         char(20),
     Phone         numeric(10),
     Email_id      char(50))
```

It is clear from the given definition that each record consists of P_ID, Pname, Address, State, Phone, and Email_id fields, resulting in records of fixed-length, but of varying length fields. Assume that each character requires 1 byte and numeric(10) requires 5 bytes, then a PUBLISHER record is 150 bytes long. Figure 7.5 represents a record of PUBLISHER relation with starting of each field within the record.

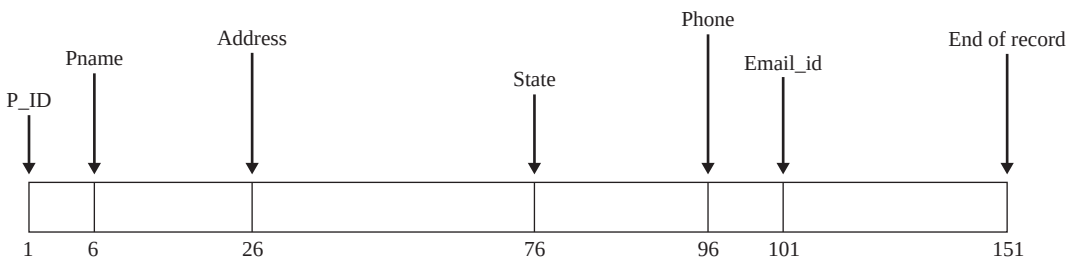


Fig. 7.5 Fixed-length record

In a file of fixed-length records, every record consists of same number of fields and size of each field is fixed for every record. It ensures easy location of field values as their positions are pre-determined. Since each record occupies equal memory, as shown in Figure 7.6, identifying start and end of record is relatively simple.

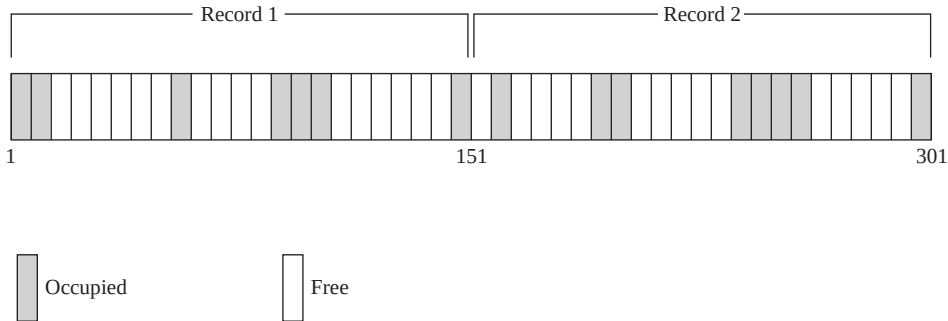


Fig. 7.6 Location of fixed-length records

A major drawback of fixed-length records is that a lot of memory space is wasted. Since a record may contain some optional fields and space is reserved for optional fields as well, it stores *null* value if no value is supplied by the user for that field. Thus, if certain records do not have values for all the fields, memory space is wasted. In addition, it is difficult to delete a record as deletion of a record leaves blank space in between the two records. To fill up that blank space, all the records following the deleted record need to be shifted.

It is undesirable to shift a large number of records to fill up the space made available by a deleted record, since it requires additional disk access. Alternatively, the space can be reused by placing a new record at the time of insertion of new records, since insertions tend to be more frequent. However, there must be some way to mark the deleted records so that they can be ignored during the file scan. In addition to simple marker on deleted record, some additional structure is needed to keep track of free space created by deleted or marked records. Thus, certain number of bytes is reserved in the beginning of the file for a **file header**. The file header stores the address of first marked record, which further points to second marked record and so on. As a result, a linked list of marked slot is formed, which is commonly termed as **free list**. Figure 7.7 shows the record of a file with file header pointing to first marked record and so on. New record is placed at the address pointed by file header and header pointer is changed to point next available marked record. In case, no marked record is available, new record is appended at the end of the file.

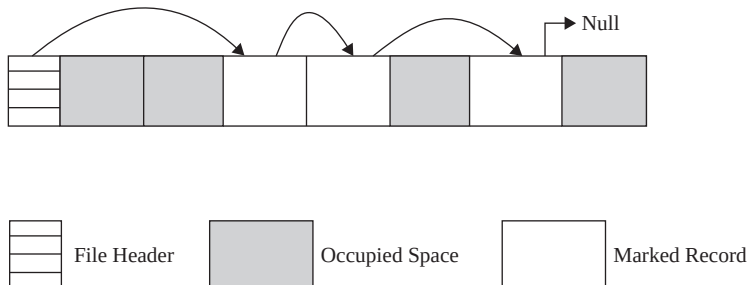


Fig. 7.7 Fixed-length records with free list of marked records

7.5.2 Variable-Length Records

Variable-length records may be used to utilize memory more efficiently. In this approach, the exact length of field is not fixed in advance. Thus, to determine the start and end of each field within the record, special separator characters, which do not appear anywhere within the field value, are required (see Figure 7.8). Locating any field within the record requires scan of record until the field is found.



Fig. 7.8 Organization of variable-length records with '%' delimiter

Alternatively, an array of integer offset could be used to indicate the starting address of fields within a record. The i^{th} element of this array is the starting address of the i^{th} field value relative to the start of the record. An offset to the end of record is also stored in this array, which is used to recognize the end of last field. The organization is shown in Figure 7.9. For null value, the pointer to starting and end of field is set same. That is, no space is used to represent a null value. This technique is more efficient way to organize the variable-length records. Handling such an offset array is an extra overhead; however, it facilitates direct access to any field of the record.

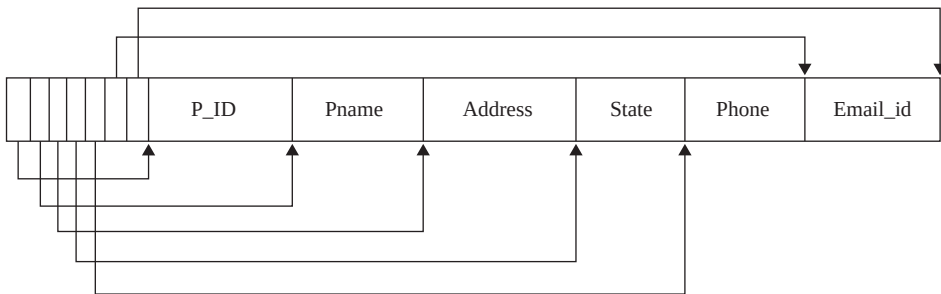


Fig. 7.9 Variable-length record organization using an array of field offsets

Sometimes there may be possibility that the values for a large number of fields are not available or are null. In that case, we can store the sequence of pair <field name, field value> instead of just field values in each record (see Figure 7.10). In this figure, three separator characters are used – one for separating two fields, second one for separating field value from field name, and third one for separating two records.

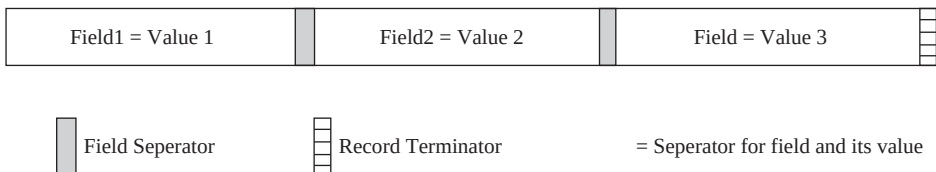


Fig. 7.10 Organization for variable-length record

It is clear from the discussion that the memory is utilized efficiently but processing the variable-length records require complicated program. Moreover, modifying the value of any field might require shifting of all the following fields, since new value may occupy more or less space than the space occupied by the existing value.

7.5.3 Mapping Records

After discussing how the database is mapped to files, we will discuss how these file records can be mapped on to disk blocks. There are two ways to organize or place them on to disk blocks. Generally, multiple records are stored in one disk block; however, some files may have large records that cannot fit in one block. Block size is not always multiple of record size, therefore, each disk block might have some unused space. This unused space can be utilized by storing part of a record on one block and the rest on another. In case, consecutive blocks are not allocated to the file, pointer at the end of first block points to the block that contains the remaining part of its last record [see Figure 7.11(a)].

Such type of organization in which records can span more than one disk block is called **spanned organization**. Spanned organization can be used with variable-length records as well as with fixed-length records. On the other hand, in **unspanned organization** the records are not allowed to cross block boundaries, thus, part of each block is wasted [see Figure 7.11(b)]. Unspanned organization is generally used with fixed-length records. This organization ensures starting of records at known position within the block, which makes processing of records simple. It is advantageous to use spanned organization to reduce the lost space in each block if average record is larger than a block.

Learn More

In order to simplify the management of free space, most RDBMS impose an upper limit on the record size. However, now-a-days, database often needs to store image, audio, or video objects which are much larger than the size of disk block. In these situations, large objects are stored in separate file(s) and a pointer to them is stored in the record.



NOTE Generally records are stored contiguously in the disk block that means next record starts where the previous record ends.

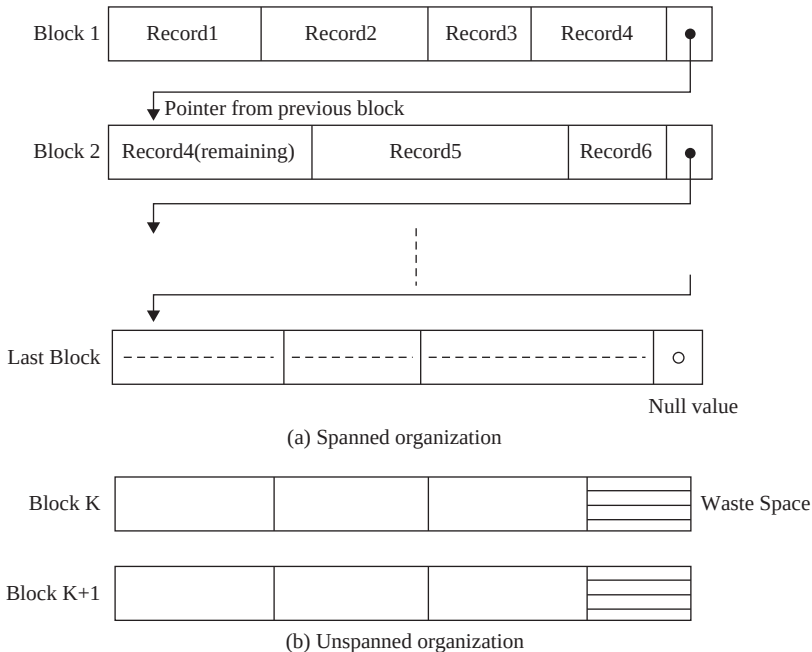


Fig. 7.11 Types of record organization

In case of fixed length record, equal number of records is stored in each block, thus, determining the start and end of record is relatively simple. However, in case of variable-length record, different number of records is stored in each block. Thus, some technique is needed to indicate the start and end of the record within each block. One common technique to implement variable-length records is **slotted page structure**, which is shown in Figure 7.12. Under this technique, some space is reserved for header in the beginning of each block that contains the following information.

- ⇒ total number of records in the block
- ⇒ end of free space in the block
- ⇒ location and size of each record within the block

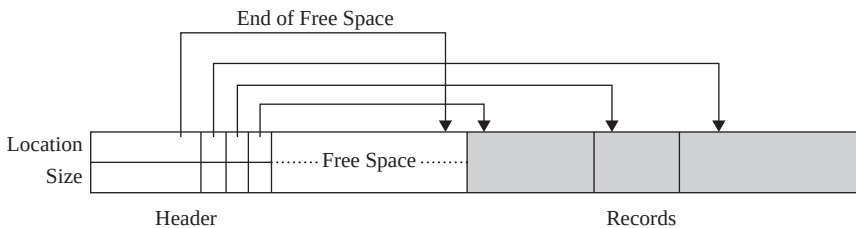


Fig. 7.12 *Slotted page structure*

In a slotted page structure starting from the end of the block, the actual records are stored continuously and the free space is contiguous between the final entry in the header array and the first record. The space for new record is allocated at the end of free space and corresponding entry of record location and record size is included in the header. Record is deleted by removing its entry from the header and freeing the space occupied by the record. To make the free space contiguous within the block again, all the records before deleted records are shifted to their right. In addition, header entry pointing to the end of free space is updated as well.

Here, no outside pointers point directly to actual records, rather they must point to the entry in the header, which contains the location and size of each record. This support of indirect pointers to records allows records to be moved within the block to prevent fragmented free space inside a block.

7.6 ORGANIZATION OF RECORDS IN FILES

Arrangement of the records in a file plays a significant role in accessing them. Moreover, proper organization of files on disk helps in accessing the file records efficiently. There are various methods (known as **file organization**) of organizing the records in a file while storing a file on disk. Some popular methods are *heap file organization*, *sequential file organization*, and *hash file organization*.

7.6.1 Heap File Organization

It is the simplest file organization technique in which no particular order of record is maintained in the file. Instead, records can be placed anywhere in the file, where there is enough space for that record. Such an organization is generally termed as **heap file** or **pile file**. Heap file is divided into pages of equal size and every record is identified by its unique id or record id.

Operations that can be carried out on a heap file include *creation* and *deletion* of files, *insertion* and *deletion* of record, *searching* a particular record, and *scanning* of all records in the file. Heap file supports efficient insertion of new record; however, scanning, searching, and deletion of records is an expensive process.

Generally, new record is appended at the end of B_{le}. The last page of B_{le} is copied to the buffer, new record is added and then the page is written back to the disk. The address of last page of B_{le} is kept in the B_{le} header.

Scanning requires retrieving all the pages of heap B_{le} and processing each record on the page one after the other. Let a B_{le} has R records on each page and every record takes S time to process. Further, if the B_{le} contains P pages and retrieving one page requires T time, then total cost to scan the complete B_{le} is $P(T + RS)$.

Assuming that exactly one record satisfies the equality selection, it means, the selection is specified on a candidate key whose values are uniformly distributed. On an average, half the B_{le} needs to be scanned using linear search technique. If no record satisfies the given condition then, all the records of the B_{le} need to be scanned to verify that the searched record is not found.

For deleting a particular record, it needs to be located in the B_{le} using its record-id. Record-id helps in identifying the page that contains the record. The page is then transferred in the buffer and after having located the record it is removed from the page and the updated page is written back.

Implementing Heap Files

As stated earlier, heap B_{les} support various operations. To implement deletion, searching, and scanning operation, there is a need to keep track of all the pages allocated to that B_{le}. However, for insertion operation, identification of the pages that contain free space is also required. A simple way to keep this information is to maintain two linked lists of pages: one for those having no free space and another for those having some free space. The system only needs to keep track of first page of the B_{le} called header page. One such representation of heap B_{le} is shown in Figure 7.13.

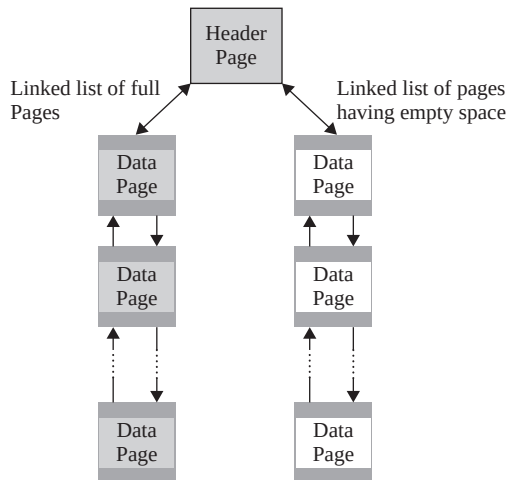


Fig. 7.13 *Linked list of heap file organization*

When a new record is to be inserted in the B_{le}, the page with enough free space is located and the record is inserted in that page. If there is no page with enough space to accommodate that new record, a new page is allocated to the B_{le} and record is inserted in the new page.

The main disadvantage of this technique is that we sometimes may require examining several pages for inserting a new record. Moreover, in case of variable length records, each page might always have some free space in it. Thus, all the pages allocated to the B_{le} will be on the list of free pages.

This disadvantage of implementing heap B_{le} can be addressed by another technique in which a directory of pages is maintained. In this directory, each entry has a pointer to a page in the B_{le} and the amount of free space in that page. When directory itself contains several pages, they are linked together to form a linked list. Organization of heap B_{le} using directory is shown in Figure 7.14.

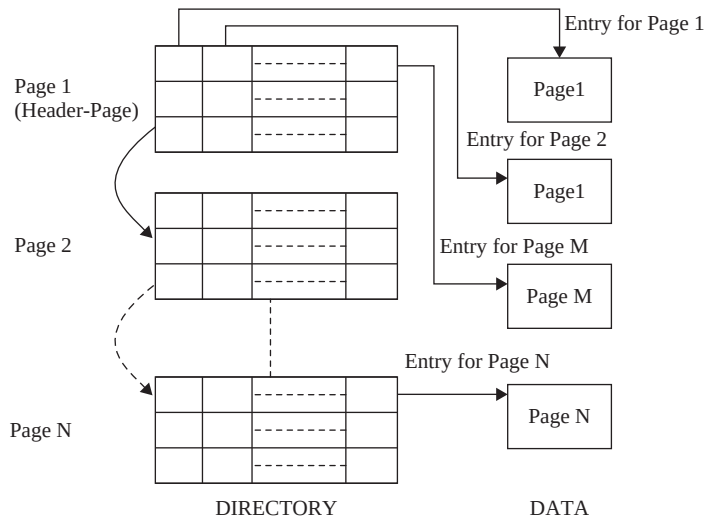


Fig. 7.14 Directory-based heap file organization

7.6.2 Sequential File Organization

Often, it is required to process the records of a file in the sorted order based on the value of one of its field. If the records of the file are not physically placed in the required order, it consumes time to fulfill this request. However, if the records of that file are placed in the sorted order based on that field, we would be able to efficiently fulfill this request. A file organization in which records are sorted based on the value of one of its field is called **sequential file organization**, and such a file is called **sequential file**. In a sequential file, the field on which the records are sorted is called **ordered field**. This field may or may not be the key field. In case, the file is ordered on the basis of key, then the field is called the **ordering key**.

Searching of records is more efficient in a sequential file if search condition is specified on the ordering field because then binary search is applicable instead of linear search. Moreover, retrieval of records with the range condition specified on the ordering field is also very efficient. In this operation, all the records starting with the first record satisfying the range selection condition till the first record that does not satisfy the condition are retrieved.



NOTE Sequential file does not make any improvement in processing the records in random order.

However, handling deletion and insertion operations are complicated. Deletion of a record leaves a blank space in between the two records. This situation can be handled by using deletion marker in the same way as already discussed in the Section 7.5.1. When a new record is to be inserted in a sequential file, there are two possibilities. First, we can insert the record at its actual position in the file. Obviously, this needs locating the first record that has to come after the new record and making space for the new record. Making space for a record may require shifting a large number of records and this is very costly in terms of disk access. Second, we can insert that record in an overflow area allocated to the file instead of its correct position in the original file. Note that the records in the overflow area are unordered. Periodically, the records in the overflow area are sorted and merged with the records in the original file.

The second approach of insertion makes the insertion process efficient; however, it may affect the search operation. This is because the required record needs to be searched in the overflow area using linear search if it is not found in the original file using binary search.

7.6.3 Hash File Organization

In this organization, records are organized using a technique called **hashing**, which enables very fast access to records on the basis of certain search conditions. This organization is called **hash file** or **direct file**. In hashing, a **hash function** h is applied to a single field, called **hash field**, to determine the page to which the record belongs. The page is then searched to locate the record. In this way, only one page is accessed to locate a record. Note that the search condition must be an equality condition on the hash field. The hash field is known as **hash key** in case the hash field is the key field of the file.

Hashing is typically implemented as a **hash table** through the use of an array of records. Given the hash field value, the hash function tells us where to look in the array for that record. Suppose that there are N locations for records in the array, it means the index of array ranges from 0 to $N-1$. The hash function h converts the hash field value between 0 and $N-1$. Note that non-integer hash field value can be converted into integer before applying hash function. For example, the numeric (ASCII) code associated with characters can be used in converting character values into integers. Some common hash functions are discussed here.

Things to Remember

Locating a record for equality search conditions is very effective using hashing technique. However, with range conditions, it is almost as worse as scanning all the records of the file.

- ⇒ **Cut key hashing:** This hash function cuts some digits from the hash field value (or simply key) and uses it as hash address. For example, if there are 100 locations in the array, a 2-digit address, say 37, may be obtained from the key 1324**37** by picking the highlighted digits. It is not a good algorithm because most of the key is ignored and only a part of the key is used to compute the address.
- ⇒ **Folded key:** This hash function involves applying some arithmetic functions, such as *addition/subtraction* or some logical functions, such as *and/or* to the key value. The result obtained is used as the record address. For example, the key is broken down into a group of 2-digit numbers from the left most digits and they are added like $13 + 24 + 37 = 74$. The sum is then used as record address. It is better than cut key hashing as the entire key is used but not good enough as records are not distributed evenly among pages.
- ⇒ **Division-remainder hashing:** This hash function is commonly used and in this the value of hash field is divided by N and remainder is used as record address. $\text{Record address} = (\text{Key value}) \bmod (N)$.

For example, $(132437) \bmod (101) = 26$

This technique works very well provided that N is either a prime number or does not have a small divisor.

The main problem associated with most hashing functions is that they do not yield distinct addresses for distinct hash field values, because the number of possible values a hash field can take is much larger than the number of available addresses to store records. Thus, sometimes a problem, called **collision**, occurs when hash field value of a record to be inserted hashes to an address which is already being occupied by another record. It should be resolved by finding some other location to place the new record. This process of finding another location is called **collision resolution**. Some popular methods for collision resolution are given here.

- ⇒ **Open addressing:** In this method, the program keeps checking the subsequent locations starting from the occupied location specified by hash function until an unused or empty location is found.
- ⇒ **Multiple hashing:** In this method, the program can use another hash function if the first hash function results in a collision. If another collision occurs, the program may apply another hash function and so on, or it can use open addressing if necessary. This technique of applying multiple hash function is called **double hashing** or **multiple hashing**.
- ⇒ **Chained overflow:** In this method, all the records whose addresses are already occupied are stored in an overflow space. In addition, a pointer field is associated with each record location. A collision is resolved by placing the new record in overflow space and the pointer field of occupied hash address location is set to that location in overflow space. In this way, a linked list of records, which hash to same address, is maintained, as shown in Figure 7.15.

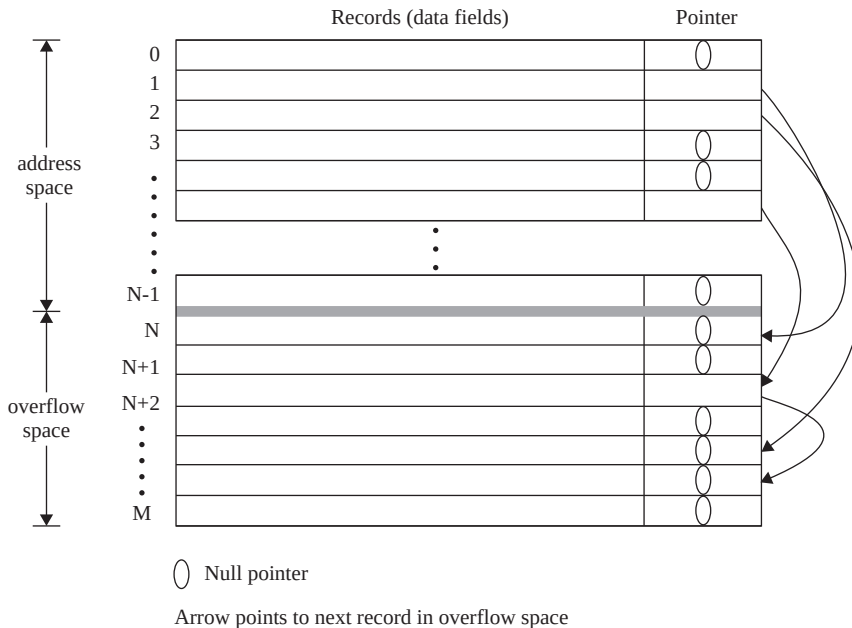


Fig. 7.15 Collision resolution using chained overflow

All these methods work well; however, each of them requires its own algorithms for insertion, retrieval, and deletion of records. All the algorithms for chained overflow are simple, whereas open addressing requires tricky algorithm for deletion of records.

A good hash function must distribute records uniformly over the available addresses so as to minimize collisions. In addition, it should always return same value for same input. Generally, hashing works best when division remainder hash function is used in which N is chosen as a prime number, as it distributes the records better over the available addresses.

Further, the collision problem is less severe by using the concept of **buckets**, which represents the storage unit that can store one or more records. There are two types of hashing schemes namely, *static hashing* and *dynamic hashing*, depending on the number of buckets allocated to a file.

Static Hashing

The hashing scheme in which a fixed number of buckets, say N , are allocated to a file to store records is called **static hashing**. The pages of a file can be viewed as a collection of buckets, with one **primary** page and additional **overflow** pages. The file consists of 0 to $N-1$ buckets, with one primary page per

bucket initially. Further, let V denote the set of all values hash field can take then the hash function h maps V to N .

To insert a record, hash function is applied on the hash field to identify the bucket to which the record belongs and then it is placed there. If the bucket is already full, a new overflow page is allocated and the record is placed in the new overflow page and then the page is added to the **overflow chain** of bucket.

To search a record, the hash function transforms the hash field value to the bucket to which the required record belongs. Hash field value of all the records in the bucket is then verified to search the required record. Note that to speed up the searching process within the bucket, all the records are maintained in a sorted order by hash field value.

To delete a record, hash function is applied to identify the bucket to which it belongs. The bucket is then searched to locate the corresponding record and after locating the record, it is removed from the bucket. Note that the overflow page is removed from the overflow chain of the bucket if it is the last record in the overflow page.

Figure 7.16 illustrates the static hashing which represents the records of **BOOK** relation with **Price** as hash field. In this example, the hash function h takes the remainder of **Price** after dividing it by three to determine the bucket for a record. Function h is defined as

$h(\text{Price}) = 0$, record for bucket 1

$h(\text{Price}) = 1$, record for bucket 2

$h(\text{Price}) = 2$, record for bucket 3

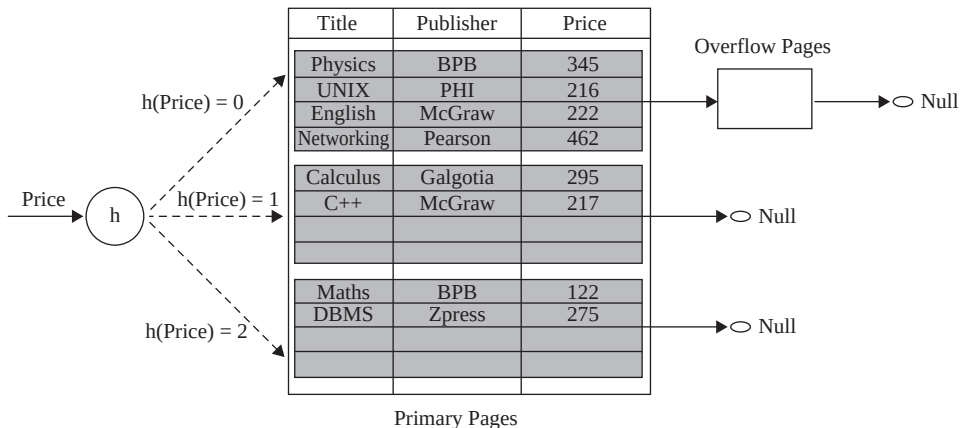


Fig. 7.16 Static hashing with *Price* as hash field

In static hashing, since the number of buckets allocated to a file are fixed when the file is created, the primary pages can be stored on consecutive disk pages. Hence, searching a record requires just one disk I/O, and other operations, like insertion and deletion, require two I/Os (read and write the page). However, as the file grows, long overflow chains are developed which degrade the performance of the file as searching a bucket requires searching all the pages in its overflow chain.

It is undesirable to use static hashing with files that grow or shrink a lot dynamically, since number of buckets is fixed in advance. It is a major drawback for dynamic files. Suppose a file grows substantially more than the allocated space, a long overflow chain is developed which results in poor performance. Similarly, if a file shrinks greatly, a lot of space is left unused. In either case, the number of buckets allocated to the file needs to be changed dynamically and new hash function based on new value of N should be used for distribution of records. However, such reorganization consumes a lot of time for large files. Another alternative is to use dynamic hashing, which allows number of buckets to vary dynamically with only minor (internal) reorganization.

Dynamic Hashing

As discussed earlier, the number of available addresses for records is fixed in advance in static hashing, which makes it unsuitable for the files that grow or shrink dynamically. Thus, **dynamic hashing** technique is needed which allocates new buckets dynamically as needed. In addition, it allows the hash function to be modified dynamically to accommodate the growth or shrinkage of database files. Dynamic hashing is of two forms, namely, *extendible hashing* and *linear hashing*. In our discussion of dynamic hashing, we consider the result of hash function as binary representation of hash field value.

Extendible Hashing Extendible hashing uses a **directory** of pointers to buckets. A directory is just an array of 2^d size, where d (called **global depth**) is the number of bits of hash value used to locate the directory element. Each element of the directory is a pointer to a bucket in which the corresponding records are stored.

In this technique, the hash function is applied on the hash field value and the last d bits of hash value are used to locate the directory element. The pointer in this array position points to the bucket to which the corresponding record belongs. To understand this technique, consider a directory consisting of an array of 2^2 size (see Figure 7.17), which means an array of size 4. Here d is 2, thus, last 2 bits of hash value are used to locate a directory element. Further, assume that each bucket can hold three records. The search for a key, say 21, proceeds as follows. Last 2 bits of hash value 21 (binary 10101) hashes into the directory element 01, which points to bucket 2.

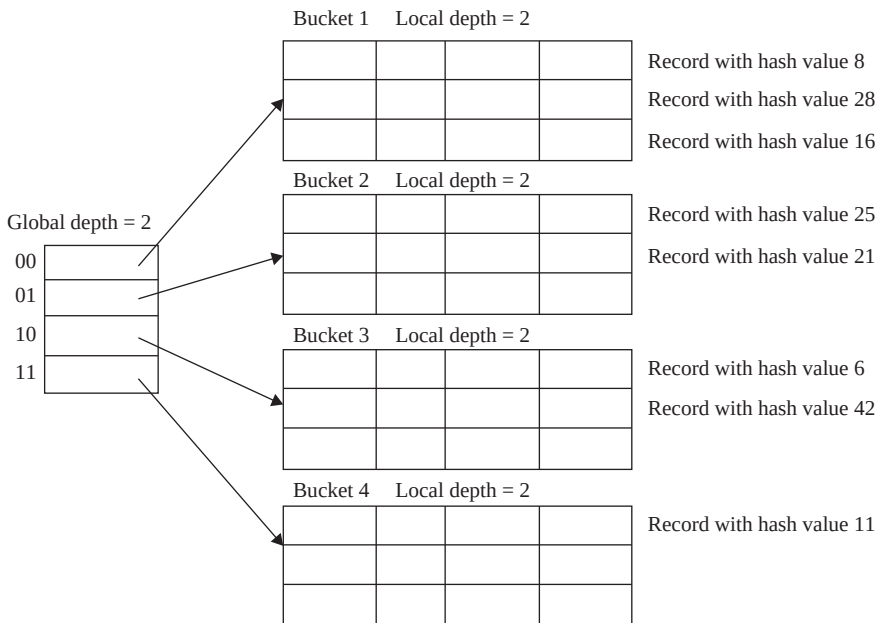


Fig. 7.17 Extendible hashed file

Consider the insertion of a new record with hash field value 9 (binary 1001). It hashes to the directory element 01, which further points to the bucket 2. Since bucket 2 has space for a new record, it is inserted there. Next, consider the insertion of a record with hash field value 24 (binary 11000). It hashes to the directory element 00 which points to bucket 1, which is already full. In this situation, extendible hashing splits this full bucket by allocating a new bucket and redistributes the records across the old bucket and its split image. To redistribute the records among these two buckets, the last three bits of hash value are considered. The last two bits (00 in this case) indicate the directory element and the third bit differentiate between these two buckets. Splitting of bucket with redistributing of records is shown in Figure 7.18. Now, the contents

of bucket 1 and bucket 1a are identified by 3 bits of hash value, whereas the contents of all other buckets can be identified by 2 bits. To identify the number of bits on which the bucket contents are based, a **local depth** is maintained with each bucket, which specifies the number of bits required to identify its contents.

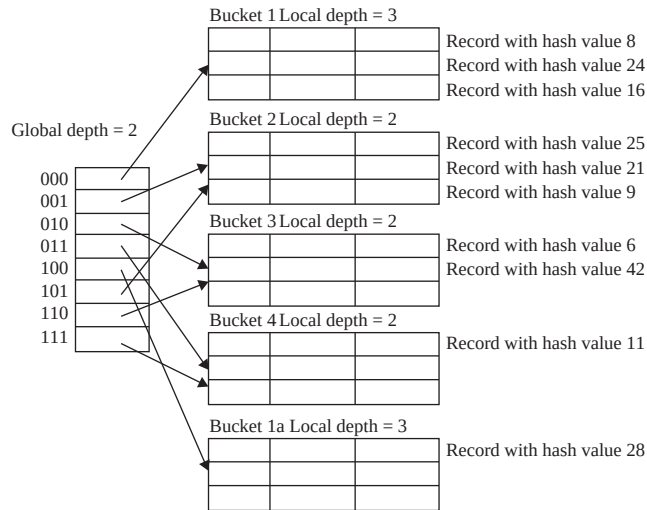


Fig. 7.18 *Extendible hashing with splitting bucket*

Note that if the local depth of overflowed bucket is equal to the global depth there is a need to double the number of entries in the directory and global depth is incremented by one. However, if local depth of overflowed bucket is less than the global depth there is no need to double the directory entries. For example, consider the insertion of a record with hash value 5 (binary 101). It hashes to the directory element 01 which points to bucket 2, which is already full. This situation is handled by splitting the bucket 2 and using the directory element 001 and 101 to point to the bucket 2 and its split image, respectively, (see Figure 7.19).

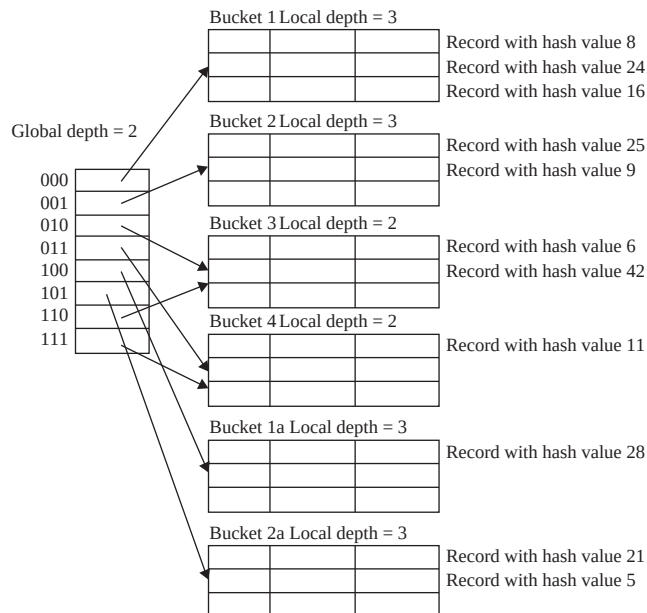


Fig. 7.19 *After inserting record with hash value 5*

To delete a record, first the record is located and then removed from the bucket. In case, the deletion leaves the bucket empty the bucket can be merged with its split bucket image. The local depth is decreased if the buckets are merged. After merging, if the local depth of all the buckets becomes less than the global depth, the number of entries in the directory can be halved and global depth is reduced by one.

The main advantage of extendible hashing over static hashing is that the performance of the B+ tree does not degrade as the B+ tree grows dynamically. In addition, there is no space allocated in advance for future growth of the B+ tree. However, buckets can be allocated dynamically as needed. The size of directory is likely to be much smaller than the B+ tree itself, since each directory element is just a page-id. Thus, the space overhead for the directory table is negligible. The directory can be extended up to 2^n , where n is the number of bits in the hash value. Another advantage of extendible hashing is that bucket splitting requires minor reorganization in most cases. The reorganization is expensive when directory needs to be doubled or halved. One disadvantage of extendible hashing is that the directory must be accessed and searched before accessing the record in the bucket. Thus, most record retrievals require two block accesses, one for the directory and the other for the bucket.

Linear Hashing Linear hashing also allows a hash B+ tree to grow or shrink dynamically by allocating new buckets. In addition, there is no need to maintain a directory of pointers to buckets. Like static hashing, collision can be handled by maintaining overflow chains of pages; however, linear hashing solves the problem of long overflow chains. In linear hashing, overflow of any bucket in the B+ tree leads to a bucket split. Note that the bucket to be split is not necessarily the same as the overflow bucket, instead buckets are chosen to be split in the linear order 0, 1, 2, ...

To understand the linear hashing scheme, consider a B+ tree with T buckets, initially numbered 0 through $T-1$. Suppose that the B+ tree uses a mod hash function h_i , denoted by $h(V) = V \bmod T$. Linear hashing uses a value j to determine the bucket to be split. Initially j is set to 0 and is incremented by 1 each time a split occurs. Thus, whenever an insert triggers an overflow record in any bucket, the j^{th} bucket in the B+ tree is split into two buckets, namely, the bucket j and a new bucket $T + j$ and j is incremented by 1. In addition, all the records of original bucket j are redistributed among the bucket j and the bucket $T + j$ using a new hash function h_{i+1} , denoted by $h_{i+1}(V) = V \bmod 2T$. Thus, when all the original T buckets have been split, all buckets use the hash function h_{i+1} .



A key property of hash function h_{i+1} is that any record that is hashed to bucket j by h_i will hash either to bucket j or bucket $T + j$. The range of hash function h_{i+1} is twice as that of h_i , that is, if h_i hashes a record into one of T buckets, h_{i+1} hash that record in one of $2T$ buckets.

In order to search a record with hash key value V , first the hash function h_i is applied to V and if $h_i(V) < j$, it means that the hashed bucket is already split. Then, the hash function h_{i+1} is applied to V to determine which of the two split buckets contain the record.

Since j is incremented with each split, so, when $j = T$, it indicates that all the original buckets have been split. At this point, j is reset to 0 and any overflow leads to the use of new hash function h_{i+2} , denoted by $h_{i+2}(V) = V \bmod 4T$. In general, linear hashing scheme uses a family of hash functions $h_{i+k}(V) = V \bmod (2^k T)$, where $k = 0, 1, 2, \dots$. Each time when all the original buckets 0 through $(2^k T) - 1$ have been split, k is incremented by 1 and new hashing function h_{i+k} is needed.

7.7 INDEXING

Once the records of a file are placed on the disk using some file organization method, the main issue is to provide quick response to different queries. For this, additional structure called **index** on the file can be created. Creating an index on a file does not affect the physical placement of the records but still provides efficient access to the file records. Index can be created on any field of the file, and that field is called **indexing field** or **indexing attribute**. Further, more than one index can be created on a file.

Accessing records of a file using index requires accessing the index, which helps us to locate the record efficiently. This is because the values in the index are stored in the sorted order and the size of the index file is small as compared to the original file. Thus, applying binary search on index is efficient as compared to applying binary search on the original file. Different types of indexes are possible, which we discuss in this section.

7.7.1 Single-Level Indexes

Indexes can be created on the field based on which the file is ordered as well as on the field based on which the file is not ordered. Consider an index created on the ordered attribute **ISBN** of the **BOOK** relation (see Figure 7.20). This index is called **primary index**, since it is created on the primary key of the relation.

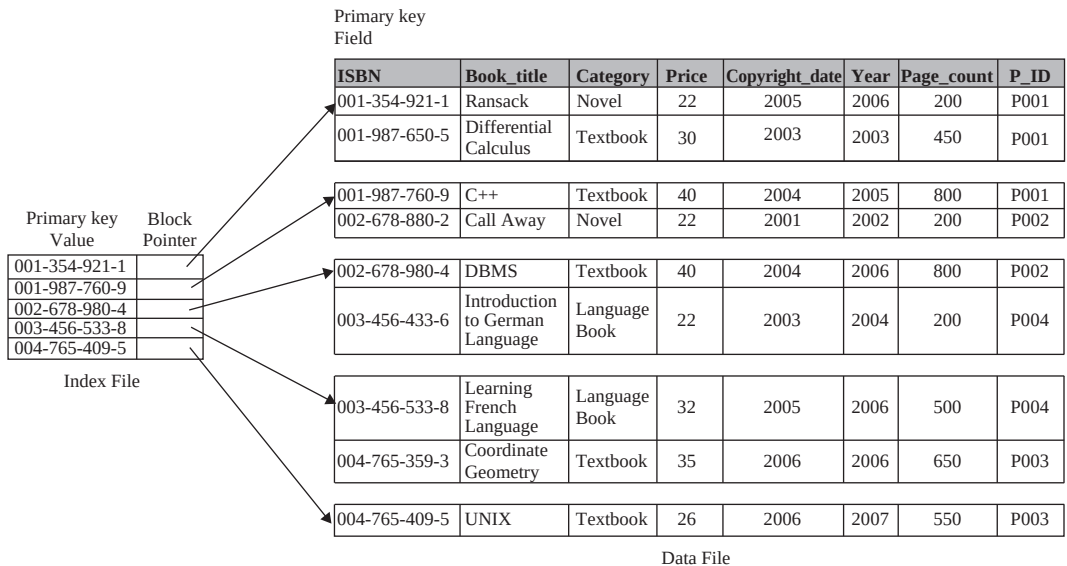


Fig. 7.20 Index on the attribute **ISBN** of **BOOK** relation

This index file contains two attributes. First attribute stores the value of the attribute on which the index is created and second attribute contains a pointer to the record in the original file. Note that it contains an entry for the first record in each disk block instead of every value of the indexing attribute. Such types of indexes which do not include an entry for each value of the indexing attribute are called **sparse indexes** (or **non-dense indexes**). On the other hand, the indexes which include an entry for each value of the indexing attribute are called **dense indexes**.

Given the search-key value for accessing any record using primary index, we locate the largest value in the index B_{le} which is less than or equal to the search-key value. The pointer corresponding to this value directs us to the 1st record of the disk block in which the required record is placed (if available). If the 1st record is not the required record, we sequentially scan the entire disk block in order to find that record. Insertion and deletion operation on the ordered B_{le} is handled in the same way as discussed in the sequential B_{le}. Here; however, it may also require modifying the pointers in several entries of the index B_{le}.

It is not always the case that index is created on the primary key attribute. If an index is created on the ordered non-key attribute, then it is called **clustering index** and that attribute is called **clustering attribute**. Figure 7.21 shows a clustering index created on the non-key attribute P_ID of the BOOK relation.

Learn More

Though dense index provides faster access to the file records than sparse index, having a sparse index with one entry per block is advantageous in terms of disk space and maintenance overhead while update operations. Moreover, once the disk block is identified and its records are brought into the memory, scanning them is almost negligible.

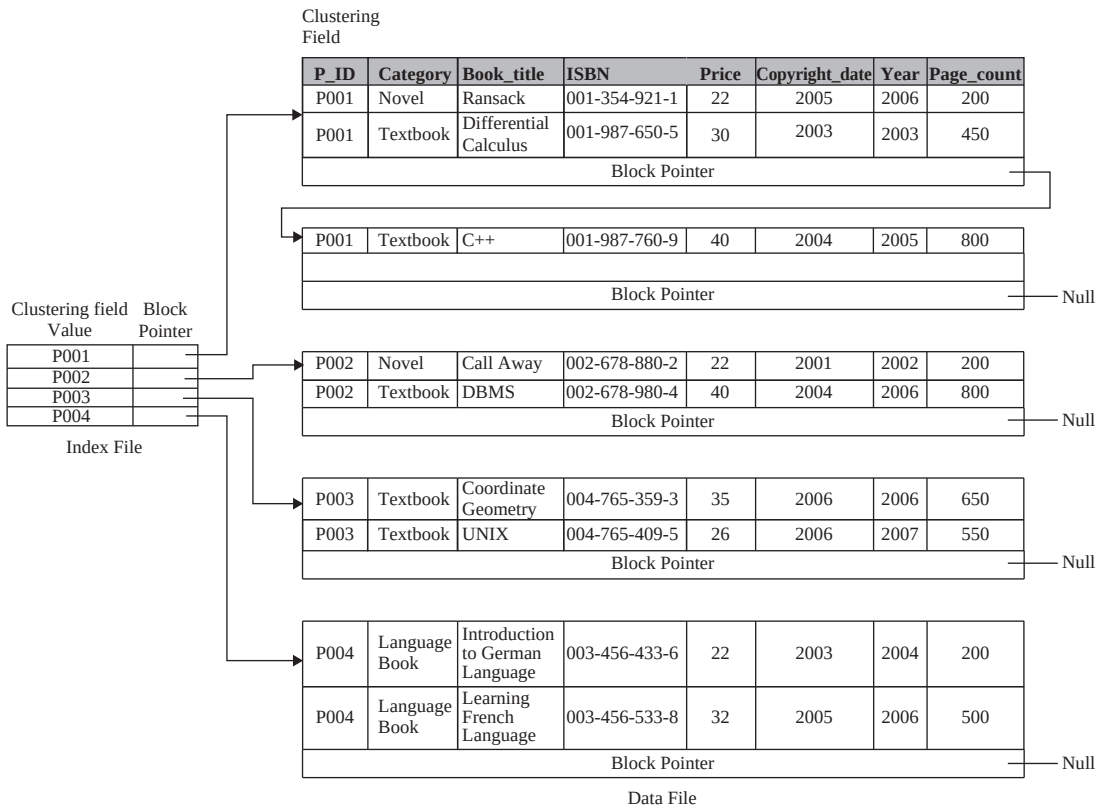


Fig. 7.21 Clustering index on P_ID field of BOOK relation

This index contains an entry for each distinct value of the clustering attribute and the corresponding pointer points to the 1st record with that clustering attribute value. Other records with the same

clustering attribute value are stored sequentially after that record, since the B+ tree is ordered on that attribute.

Inserting a new record in the B+ tree is easy if space is available in the block in which it has to be placed. Otherwise, a new block is allocated to the B+ tree and the new record is placed in that block. In addition, the pointer of the block to which the record actually belongs is made to point to the new block.

So far, we have discussed the case in which the index is created on the ordered attribute of the B+ tree. Indexes can also be created on the non-ordered attribute of the B+ tree. Such indexes are called **secondary indexes**. The indexing attribute may be a candidate key or a non-key attribute with duplicate values. In either case, secondary index must contain an entry for each value of the indexing attribute. It means the secondary index must be a dense index. This is because the B+ tree is not ordered on the indexing attribute and if some of the values are stored in the index, it is not possible to find a record whose entry does not exist in the index. Figure 7.22 shows a secondary index created on the non-ordered attribute `Book_title` of `Book` relation.

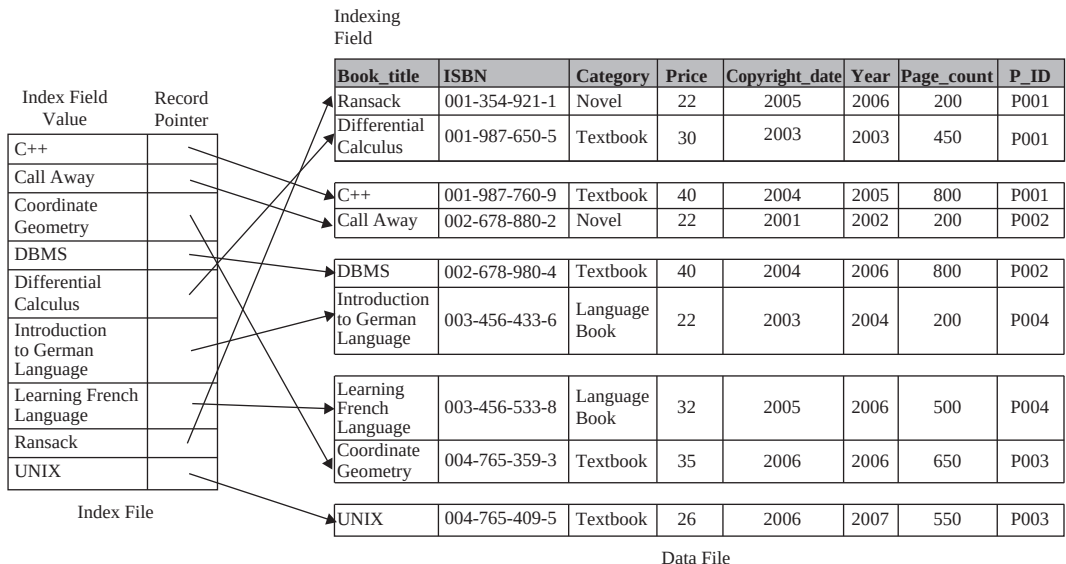


Fig. 7.22 Secondary index with record pointers on a non-ordering key field of `BOOK` relation

This index contains an entry for each value of the indexing attribute and the corresponding pointer points to the record in the B+ tree. Note, that the index B+ tree itself is ordered on the indexing attribute. Inserting and deleting a record is straightforward with a little modification in the index B+ tree.

If the indexing attribute of the secondary index is a non-key attribute with duplicate values, then we may use an extra level of indirection to handle multiple pointers. In this case, the pointer in the index points to a bucket of pointers instead of the actual records in the B+ tree. Each pointer in the bucket either points to the records in the B+ tree or to the disk block that contains the record. This structure of secondary index is illustrated in Figure 7.23.

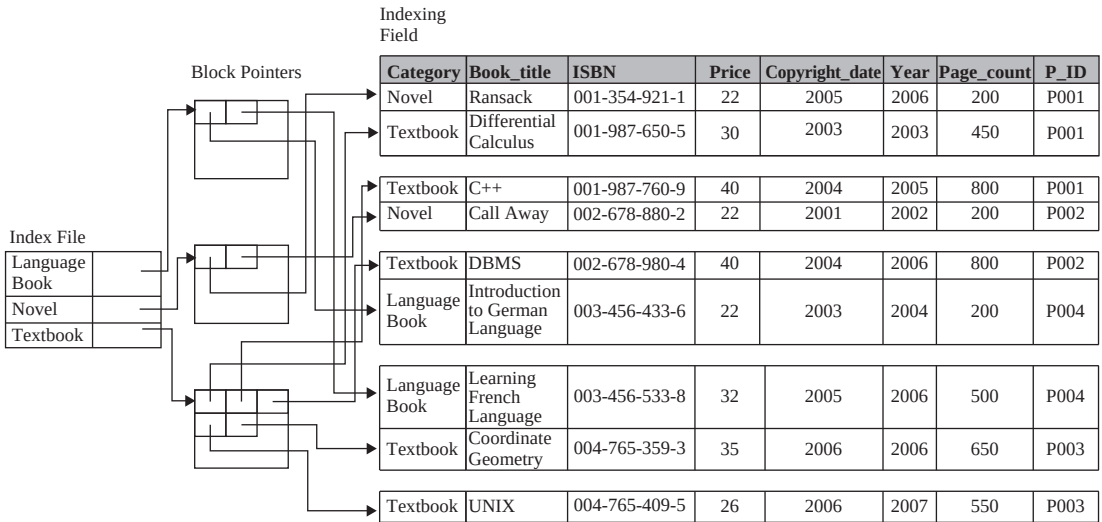


Fig. 7.23 Secondary index on a non-key field using an extra level of indirection

In general, secondary index occupies more space and consumes more time in searching, since it contains more entries than the primary index. However, having a secondary index is advantageous because in the absence of secondary index, accessing any random record requires a linear scan of the file. On the other hand, if primary index does not exist, binary search is still applicable on the file, since records are ordered.

7.7.2 Multilevel Indexes

It is common to have several thousands (or even lacs) of records in the database of medium or large-scale organizations. As the size of the file grows, the size of the index (even sparse index) grows as well. If the size of the index becomes too large to fit in the main memory at once, it becomes inefficient for processing, since it must be kept sequentially on disk just like an ordinary file. It implies that searching large indexes require several disk-block accesses.

One solution to this problem is to view the index file, which we refer to as the **first** or **base level** of a multilevel index, just as any other sequential file and construct a primary index on the index file. This index to the first level is called the **second level** of the multilevel index. In order to search a record, we first apply binary search on the second level index to find the largest value which is less than or equal to the search-key. The pointer corresponding to this value points to the block of the first level index that contains an entry for the searched record. This block of first level index is searched to locate the largest value which is less than or equal to the search-key. The pointer corresponding to this value directs us to the block of file that contains the required record.

If second level index is too small to fit in main memory at once, fewer blocks of index file needs to be accessed from disk to locate a particular record. However, if the second level index file is too large to fit in main memory at once, another level of index can be created. In fact, this process of creating the index on index can be repeated until the size of index becomes small enough to fit in main memory. This type of index with multiple levels (two or more levels) of index is called **multilevel index**. Figure 7.24 illustrates multilevel indexing.

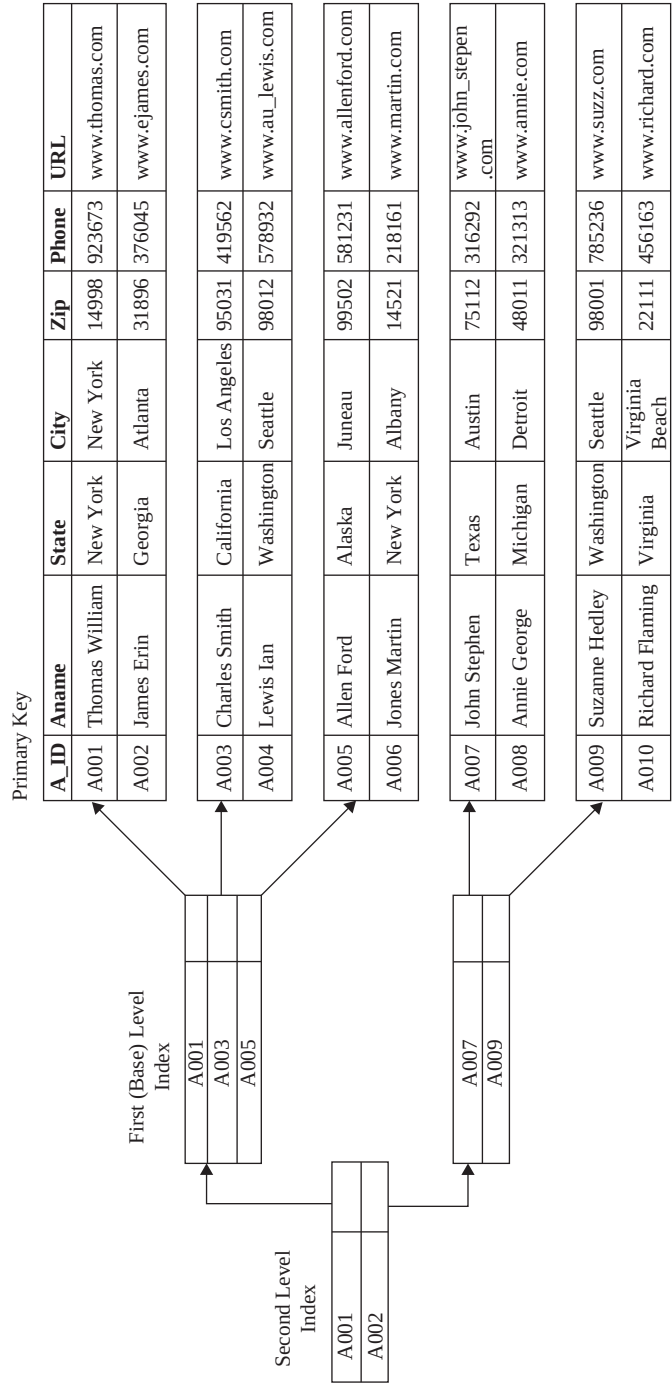


Fig. 7.24 Multilevel indexing (two-level index)

Multilevel indexing reduces the number of disk accesses while searching for a record; however, insertion and deletion are complicated because all the index B-trees at different levels are ordered B-trees. Any insertion or deletion may cause several index B-trees at different levels to be modified. One solution to this problem is to keep some space reserved in each block for new entries. This is often implemented using B-trees and B⁺-trees.

B-trees

It is a kind of tree satisfying some additional properties or constraints. Each node of a B-tree consists of $n-1$ values and n pointers. Pointers can be of two types, namely, *tree pointers* and *data pointers*. The tree pointer points to any other node of the tree and the data pointer points either to the block which contains the record with that index value or to the record itself. The order of information within each node is $\langle P_1, \langle V_1, R_1 \rangle, P_2, \langle V_2, R_2 \rangle, P_3, \langle V_3, R_3 \rangle, \dots, P_{n-1}, \langle V_{n-1}, R_{n-1} \rangle, P_n \rangle$ where P_i is a tree pointer, V_i is any value of indexing attribute and R_i is a data pointer (see Figure 7.25).

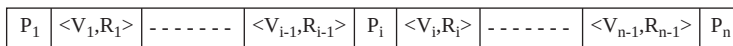


Fig. 7.25 A node of B-tree



NOTE The structure of leaf node and internal node are same except the difference that all the tree pointers in a leaf node are null.

All the values in a node should be in the sorted order, it means $V_1 < V_2 < \dots < V_{n-1}$ within each node of the tree. In addition, the node pointed to by any tree pointer P_i should contain all the values larger than the value V_{i-1} and less than the value V_i . Another constraint that the B-tree must satisfy is that it must be balanced; it means all the leaf nodes should be at the same level. B-tree also ensures that each node, except root and leaf nodes, should be at least half full, that means it should have at least $n/2$ tree pointers.



The number of tree pointers in each node (called **fan-out**) identifies the order of B-tree.

Maintaining all the constraints of a B-tree while insertion and deletion operation requires complex algorithms. First, consider the insertion of a value in a B-tree. Initially, the B-tree has only one node (root node) which is a leaf node at level 0. Suppose that the order of the B-tree is n . Thus, when $n-1$ values are inserted into the root node, it becomes full. Next insertion splits the root node into two nodes at level 1. The middle value, say m , is kept in the root node and the values from 1 to $m-1$ are shifted in the left child node and values from $m+1$ to $n-1$ are shifted in the right child node. However, when such situation occurs with a non-root node, it is split into two nodes at the same level and middle value is transferred to its parent node. Parent node then points to both the nodes as its left child and right child. If the parent node is also full, it is also split and splitting may propagate to other tree levels. If splitting gets propagated all the way to the root node and if root is also split, it increases the number of levels of the B-tree. Although, insertion in a node that is not full is quite simple and efficient. A typical B-tree of order $n = 3$ is shown in Figure 7.26.

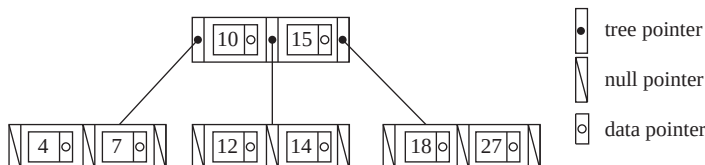


Fig. 7.26 A B-tree of order $n=3$

Now, consider the deletion of a value from a B-tree. If deleting a value from a node leaves the node more than half empty, its values are merged with its neighbouring nodes. This may also propagate all the way to the root of the B-tree. Thus, deletion of a value may result in a tree with lesser number of levels than the number of levels of the tree before deletion. However, deletion from a node that is full or nearly full is a simple process.

B⁺-trees

B⁺-tree, a variation of B-tree, is the most widely used structure for multilevel indexes. Unlike B-tree, B⁺-tree includes all the values of indexing attribute in the leaf nodes, and internal nodes contain values just for directing the search operation. All the pointers in the internal nodes are tree pointers and there is no data pointer in them. All the data pointers are included in the leaf nodes and there is one tree pointer in each leaf node that points to the next leaf node. Structure of leaf node and internal nodes of a B⁺-tree is shown in Figure 7.27.

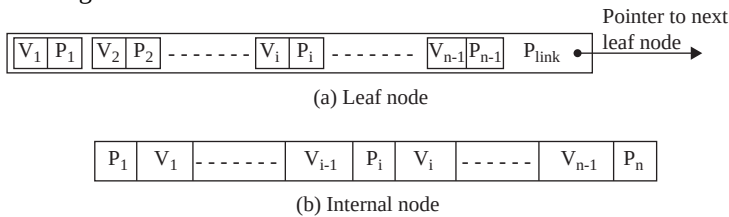


Fig. 7.27 Nodes of a B⁺-tree

Each leaf node has $n-1$ values along with a data pointer for each value. If the indexing attribute is the key attribute, the data pointer points either to the record in the file or to the disk block containing the record with that indexing attribute value. On the other hand, if the indexing attribute is not the key attribute, the data pointer points to the bucket of pointers, each of which points to the record in the file.



The leaf nodes and internal nodes of a B⁺-tree corresponds to the first level and other levels of multilevel index, respectively.

Since data pointers are not included in the internal nodes of a B⁺-tree, more entries can be fit into an internal node of a B⁺-tree than corresponding B-tree. As a result, we have larger order for B⁺-tree than B-tree for the same size of disk block. A typical B⁺-tree of order $n=3$ is shown in Figure 7.28.

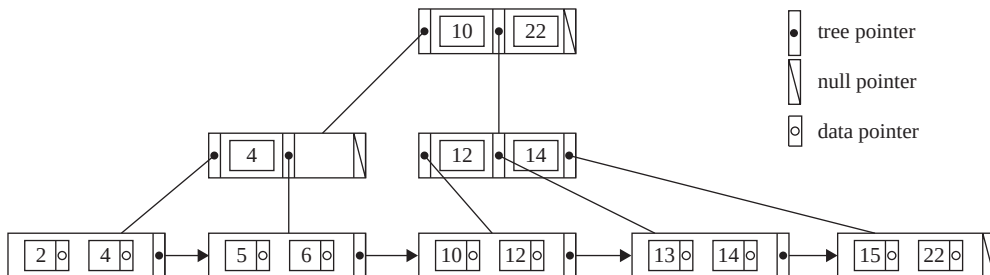


Fig. 7.28 AB⁺-tree of order $n=3$



In B-tree and B⁺-tree data structure, each node corresponds to a disk block. In addition, each node is kept between half-full or completely full.

Indexed Sequential Access Method

A variant of B⁺-tree is **indexed sequential access method (ISAM)**. In ISAM, the leaf nodes and the overflow nodes (chained to some leaf nodes) contain all the values of indexing attribute and data pointers, and the internal nodes contain values and tree pointers just for directing the search operation. Unlike B⁺-tree, the ISAM index structure is static, it means the number of leaf nodes never change; if need arises overflow node is allocated and chained to the leaf node. The leaf nodes are assumed to be allocated sequentially, thus, no pointer is needed in the leaf node to point to the next leaf node.

The algorithms for insertion, deletion, and search are simple. All searches start at root node and the value in the node helps in determining the next node to search. To insert a value, first the appropriate node is determined and if there is space in the node, the value is inserted there. Otherwise, an overflow node is allocated and chained to the node, and then the value is inserted in the overflow node. To delete a value, the node containing the value is determined and the value is deleted from that node. In case, it is the only value in the overflow node, the overflow node is also removed. In case, it is the only value in the leaf node, the node is left empty to handle future insertions (data from overflow nodes, if any, linked to that leaf node is not moved to the leaf node).

The ISAM structure provides the benefits of sequentially allocated leaf nodes and fast searches as in the case of B⁺-trees. However, the structure is unsuitable for situations where file grows or shrinks much. This is because if too much insertions are made to the same leaf node, a long overflow chain can develop, which degrades the performance while searching as overflow nodes need to be searched if the search reaches to that leaf node. A sample ISAM tree is shown in Figure 7.29.

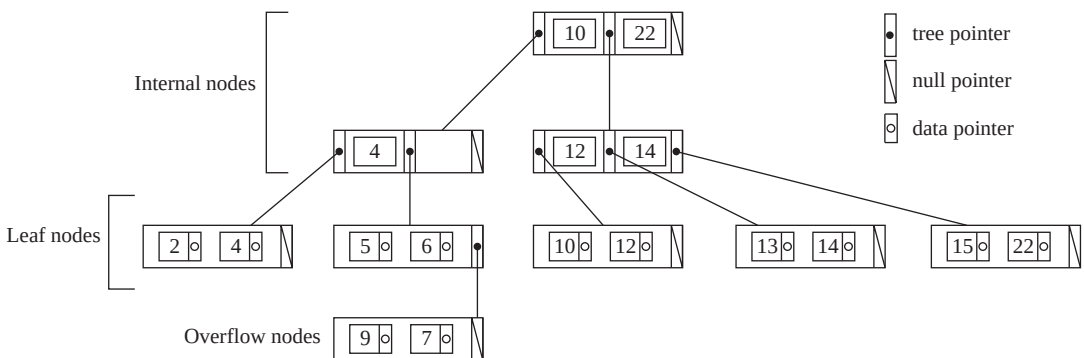


Fig. 7.29 Sample ISAM tree

7.7.3 Indexes on Multiple Keys

So far, we have discussed that the index is created on primary or secondary keys, which consists of single attributes. However, many query requests are based on multiple attributes. For such queries, it is advantageous to create an index on multiple attributes that are involved in most of the queries.

To understand the concept, consider this query: List all the books from BOOK relation where Category = "Textbook" and P_ID = "P001". This query involves two non-key attributes, namely, Category and P_ID, thus, multiple records may satisfy either condition. There are three possible strategies to process this query, which are given here.

- ⇒ Assume that we have an index on Category field. Using this index we can find all the books having Category = "Textbook". From these records, we can search all those books whose P_ID = "P001".

- ⇒ Assume that we have an index on **P_ID** field. Using this index we can find all the records having **P_ID** = "P001". From these records, we can search all those books whose **Category** = "Textbook".
- ⇒ Assume that we have an index on both the attributes, namely, **Category** and **P_ID**. Using index on **Category** field, we can find the set of all the books whose **Category** = "Textbook". Similarly, using index on **P_ID** field we can find all the books whose **P_ID** = "P001". By performing the intersection of both the set, we can find all the books satisfying both the given conditions.

All of the strategies yield a correct set of records; however, if many records satisfy individual condition and only a few records satisfy both the conditions, then none of the above alternative is a good choice in terms of efficiency. Thus, some efficient technique is needed.

There are a number of techniques that would treat the combination of **Category** and **P_ID** as a search-key. One solution to this problem is an ordered index on multiple attributes. The idea behind **ordered index on multiple attribute** is to create an index on combination of more than one attribute. It means search-key consists of multiple attributes. Such a search-key containing multiple attribute is termed as **composite search-key**. In this example, search-key consists of both the attributes, namely, **Category** and **P_ID**. In this type of index, search-key represents a tuple with values $\langle V_1, V_2, V_3, \dots, V_n \rangle$ where index attributes are $\langle A_1, A_2, A_3, \dots, A_n \rangle$. The values of search-key are **lexicographic ordered**. Lexicographic ordering for above case states that all the entries of books published by *P001* precedes the books published by *P002* and so on. This type of index differs from the other index only in terms of search-key. However, basic structure is similar as that of any other index having search-key on single attribute. Thus, working of index on composite search-key is similar as any other index discussed so far.

Another solution to this problem is **partitioned hashing**, an extension of static hashing, which allows access on multiple keys. Partitioned hashing does not support range queries; however, it is suitable for equality comparisons. In partitioned hashing, for a composite search-key consisting of n attributes, the chosen hash function produces n separate hash addresses. These n addresses are concatenated to obtain the bucket address. All the buckets, which match the part of the address, are then searched for the combined search-key.

The main advantage of partitioned hashing is that it can be easily extended to any number of attributes in the search-key. In addition, there is no need to maintain separate access structure for individual attributes. On the other hand, the main disadvantage of partitioned hashing is that it does not support range queries on any of the component attributes.

Another alternative to such type of a problem is grid files. In the **grid files**, a grid array with one linear scale (or dimension) for each of the search attribute is created. Linear scales are made in order to provide uniform distribution of that attribute values in the grid array. Linear scale groups the values of one search-key attribute to each dimension. Figure 7.30 shows a grid array along with two linear scales, one is for attribute **Category** and the other is for attribute **P_ID**. Each grid cell points to some bucket address where the corresponding records are stored. The number of attributes in search-key determines the dimension of grid array. Thus, for search-key having n attributes, the grid array would have n dimensions. In our example, two attributes are taken as search-key, thus, the grid array is of two dimensions. Linear scale is looked up to map into the corresponding grid cell. Thus, the query for **Category** = "Textbook" and **P_ID** = "P001" maps into the cell (2, 0), which further points to the bucket address where the required records are stored.

The main advantage of this method is that it can be applied for range queries. In addition, it is extended to any number of attributes in the search-key. Grid file method is good for multiple-key search. On the other hand, the main disadvantage is that it represents space and management overhead in terms of grid array because it requires frequent reorganization of grid files with dynamic files.

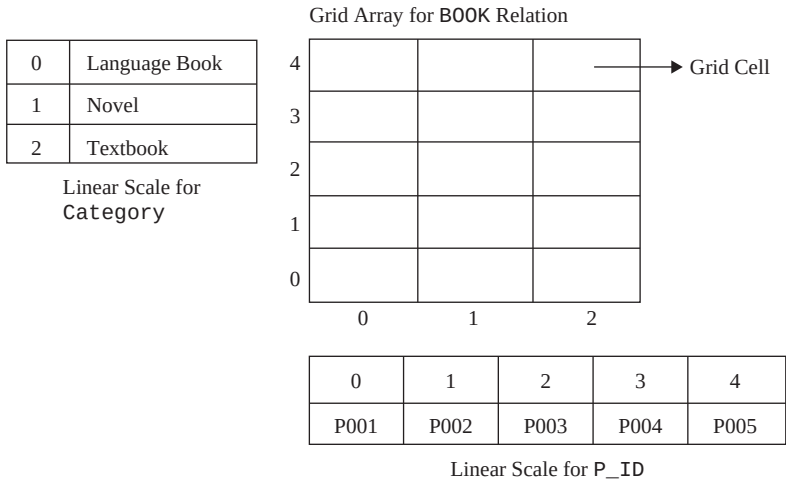


Fig. 7.30 Grid array along with two linear scales

● SUMMARY ●

1. The database is stored on storage medium in the form of Bles, which is a collection of records consisting of related data items, such as name, address, phone, email-id, and so on.
2. Several types of storage media exist in computer systems. They are classified on the basis of speed with which they can access data. Among the media available are cache memory, main memory, flash memory, magnetic disks, optical disks, and magnetic tapes. Database is typically stored on the magnetic disk because of its higher capacity and persistent storage.
3. The storage media that consist of high-speed devices are referred to as primary storage. The storage media that consist of slower devices are referred to as secondary storage. Magnetic disk (generally called disk) is the primary form of secondary storage that enables storage of enormous amount of data. The slowest media comes under the category of tertiary storage. Removable media, such as optical discs and magnetic tapes, are considered as tertiary storage.
4. The CPU does not directly access data on the secondary storage and tertiary storage. Instead, data to be accessed must move to main memory so that it can be accessed.
5. A major advancement in secondary storage technology is represented by the development of Redundant Arrays of Independent Disks (RAID). The idea behind RAID is to have a large array of small independent disks. Data striping is used to improve the performance of disk, which utilizes parallelism. Mirroring (also termed as shadowing) is a technique used to improve the reliability of the disk. In this technique, the data is redundantly stored on two physical disks.
6. Several different RAID organizations are possible, each with different cost, performance, and reliability characteristics. Raid level 0 to 6 exists. RAID level 1 (mirroring) and RAID level 5 are the most commonly used.
7. In Storage Area Network (SAN) architecture, many server computers are connected with a large number of disks on a high-speed network. Storage disks are placed at a central server room, and are monitored and maintained by system administrators. An alternative to SAN is Network-Attached Storage (NAS). NAS device is a server that allows Bles sharing.
8. The database is mapped into a number of different Bles, which are physically stored on disk for persistency. Each Bles is decomposed into equal size pages, which is the unit of exchange between the disk and main memory.

9. It is not possible to keep all the pages of a file in main memory at once; thus, the available space of main memory should be managed efficiently.
10. The main memory space available for storing the data is called buffer pool and the subsystem that manages the allocation of buffer space is called buffer manager.
11. Buffer manager uses replacement policy to choose a page for replacement if there is no space available for a new page in the main memory. The commonly used policies include LRU, MRU, and clock replacement policy.
12. There are two approaches to organize the database in the form of files. In the first approach, all the records of a file are of fixed-length. However, in the second approach, the records of a file vary in size.
13. Once the database is mapped to files, the records of these files can be mapped on to disk blocks. The two ways to organize them on disk blocks are spanned organization and unspanned organization.
14. There are various methods of organizing the records in a file while storing a file on disk. Some popular methods are heap file organization, sequential file organization, and hash file organization.
15. Heap file organization is the simplest file organization technique in which no particular order of record is maintained in the file. In sequential file, the records are physically placed in the sorted order based on the value of one or more fields, called the ordering field. The ordering field may or may not be the key field of the file, whereas records are organized using a technique called hashing in a hash file organization.
16. Hashing provides very fast access to an arbitrary record of a file, on the basis of certain search conditions. The hashing scheme in which a fixed number of buckets, say N, are allocated to a file to store records is called static hashing. Dynamic hashing technique allocates new buckets dynamically as needed. It allows the hash function to be modified dynamically to accommodate the growth or shrinkage of database files. Two hashing techniques for files that grow and shrink in number of records dynamically namely, extendible and linear hashing.
17. To speed up the retrieval of records, additional access structures called indexes are used. Index can be created or defined on any field termed as indexing field or indexing attribute of the file. Creating single-level indexes, multilevel indexes, and indexes on multiple keys are possible.
18. Different types of single-level index are primary index, clustering index, and secondary index.
19. A primary index is an ordered file that contains an index on the primary key of the file. An index defined on a sequentially ordered non-key attribute (containing duplicate values) of a file is called clustering index. The attribute on which the clustering index is defined is known as a clustering attribute.
20. A secondary index is an index defined on a non-ordered attribute of the file. The indexing attribute may be a candidate key or a non-key attribute of the file.
21. The type of index with multiple levels (two or more levels) of index is called multilevel index. Multilevel indexes can be implemented using B-trees and B⁺-trees, which are dynamic structures that allow an index to expand and shrink dynamically. B⁺-trees can generally hold more entries in their internal nodes than B-trees.
22. Partitioned hashing is an extension of static hashing, which allows access on multiple keys. It does not support range queries; however, it is suitable for equality comparisons.

● KEY TERMS ●

- | | |
|---|--|
| <ul style="list-style-type: none"> ● Storage devices ● Volatile storage ● Non-volatile storage | <ul style="list-style-type: none"> ● Primary storage ● Cache memory ● Main memory |
|---|--|

- Flash memory
- Secondary storage
- Tertiary storage
- Optical disc
- CD-ROM
- DVD-ROM
- Tapes storage
- Sequential access
- Direct access
- Magnetic disk
- Platter
- Single-sided disk
- Double-sided disk
- Tracks and sectors
- Read/write head
- Disk arm
- Spindle
- Head-disk assemblies
- Cylinder
- Disk controller
- SCSI, ATA, and SATA interfaces
- Checksums
- Seek time
- Rotational delay
- Data transfer time
- Access time
- Disk reliability and MTTF
- RAID
- Data striping
- Bit-level data striping
- Block-level data striping
- Redundant information
- Mirroring (or shadowing)
- Mean time to failure
- Mean time to repair
- Error correcting codes and check disk
- RAID levels
- Storage area network (SAN)
- Network-attached storage (NAS)
- Pages
- Bufferpool
- Buffer manager
- Frames
- Pinning and unpinning pages
- Page-replacement policies
- Least recently used
- Most recently used
- Clock replacement
- Fixed-length records
- File header
- Free list
- Variable-length records
- Mapping records
- Spanned organization
- Unspanned organization
- Slotted page structure
- File organization
- Heap file organization
- Heap file or pile file
- Sequential file organization
- Sequential file
- Ordered file
- Ordering key
- Hash file organization
- Hashing
- Hash file or direct file
- Hash function
- Hash table
- Cut key hashing
- Folded key hashing
- Division-remainder hashing
- Collision
- Collision resolution
- Open addressing
- Multiple addressing
- Chained overflow
- Bucket
- Static hashing
- Primary and overflow pages
- Overflow chain of bucket
- Dynamic hashing
- Extendible hashing

- Directory of pointers
- Global de pth
- Local de pth
- Linear hashing
- Indexing
- Index
- Indexing based on indexing attribute
- Single-level index
- Primary index
- Sparse index
- Dense index
- Clustering index
- Clustering attribute
- Secondary index
- Multilevel index
- First or base level
- Second level
- B-tree
- Tree pointers
- Data pointers
- B⁺-tree
- Indexes on multiple keys
- Composite search-key
- Lexicographic order
- Partitioned hashing
- Grid files

● EXERCISES ●

A. Multiple Choice Questions

1. Which of the following is not classified as primary storage?
 - (a) Flash memory
 - (b) Main memory
 - (c) Magnetic disk
 - (d) Cache
2. Which of the following term is not related to magnetic disk?
 - (a) Arm
 - (b) Cylinder
 - (c) Platter
 - (d) None of these
3. The time taken to position read/write heads on specific track is known as _____.
 - (a) Rotational delay time
 - (b) Seek time
 - (c) Data transfer time
 - (d) Access time
4. Which of the following RAID level uses block-level striping?
 - (a) Level 0
 - (b) Level 1
 - (c) Level 4
 - (d) Level 6
5. _____ actually handles the request to access a page of a file.
 - (a) Disk controller
 - (b) Database
 - (c) Buffer manager
 - (d) Main memory
6. Which replacement policy requires the reference bit to be associated with each frame?
 - (a) Least recently used
 - (b) Most recently used
 - (c) Clock replacement
 - (d) Both (a) and (b)
7. While mapping fixed-length records to disk, which of the following organization is generally used?
 - (a) Spanned organization
 - (b) Unspanned organization
 - (c) Both (a) and (b)
 - (d) None of these

8. Which of the following is not a file organization method?
 - (a) Heap file organization
 - (b) Sequential file organization
 - (c) Hash file organization
 - (d) None of these
9. Which of the following hash functions never result in collision?
 - (a) Cut key
 - (b) Folded key
 - (c) Division-remainder
 - (d) None of these
10. The type of indexes that include an entry for each value of indexing attribute is known as _____.
 - (a) Clustering index
 - (b) Sparse index
 - (c) Dense index
 - (d) Secondary index

B. Fill in the Blanks

1. _____ is a static RAM and is very small in size.
2. Disk surface of a platter is divided into imaginary _____ and _____.
3. The reliability of the disk is measured in terms of _____.
4. In order to improve the performance of disk, a concept called _____ is used which utilizes parallelism.
5. In practice, the RAID level _____ is not used.
6. The process of incrementing the `pin_count` is generally called _____ the page.
7. A large number of database systems use _____ page replacement policy.
8. One common technique to implement variable-length record is _____.
9. A file organization in which records are sorted based on the value of one of its field is called _____.
10. In static hashing, the pages of a file can be viewed as a collection of buckets, with one _____ page and additional _____ pages.

C. Answer the Questions

1. List the different types of storage media available in computer system. Also explain how they are classified into different categories.
2. Give hardware description and various features of magnetic disk. How do you measure its performance?
3. Define these terms: buffer manager, buffer pool, mirroring, file header, free list, ordering field, ordering key.
4. Define RAID. What is the need of having RAID technology?
5. How can the reliability and performance of disk be improved using RAID? Explain different RAID levels.
6. Define the role of buffer manager. Explain the policies used by buffer manager to replace a page.
7. Compare and contrast fixed-length and variable-length records. Discuss various techniques that the system uses to delete a record from a file.
8. Why do variable-length records require separator characters? Is there any alternative of using separator character? Justify your answer.

9. Compare and contrast spanned and unspanned organization of records. What is slotted page structure, and why is it needed?
10. Discuss the importance of file organization in databases. Define various types of file organizations available.
11. What is page replacement policy? Compare and contrast LRU and MRU replacement policies with the help of example.
12. Discuss the two techniques of implementing heap file with diagram.
13. Discuss the commonly used hash functions. What is the main problem associated with most of them, and how can it be resolved?
14. How does hashing allow a file to expand and shrink dynamically?
15. What are the advantages of having an index on a file? List the different types of single-level indexes available. Discuss them in detail.
16. What are dense and sparse indexes?
17. What is multilevel index? Also give reasons for having multilevel index.
18. What does the order of a B-tree and B⁺-tree indicate? Also explain the structure of both internal and external nodes of a B-tree and B⁺-tree.
19. Why is B⁺-tree preferable over B-tree?
20. Explain the significance of index defined on multiple attributes. Also explain partitioned hashing and grid files with example.
21. Write a short note on the following.
 - (a) Clock replacement policy
 - (b) SAN and NAS
 - (c) Partitioned hashing

D. Practical Questions

1. Consider an empty B⁺-tree with order $p=3$, and perform the following.
 - (a) Show the tree after inserting these values: 70, 15, 20, 35, 18, 55, 43. Assume that the values are inserted in the order given
 - (b) Show the tree after deleting the node with value 18 from the tree
 - (c) Show the steps involved in finding the value 35
2. Perform (a), (b), and (c) part of previous question for a B-tree with order $p=3$.
3. Create a linear hash file on a file that contains records with these key values: 70, 15, 20, 35, 18, 55, 43. Suppose the hash function used is $h(k) = k \bmod 5$, and each bucket can hold three records.
4. Show the hash file created in question 3 when following steps are performed on the database file.
 - (a) Insertion of a record with key value 12
 - (b) Deletion of record with key value 18
5. Consider a fixed-length record file having 1500 records of PUBLISHER relation (defined in Section 7.5.1). Assume that block size is 100 bytes. How many blocks are needed to store the file if records are organized using
 - (a) spanned organization, assuming each block has a 5-byte pointer to the next block
 - (b) unspanned organization

Calculate the number of bytes wasted in each block due to unspanned organization

QUERY PROCESSING AND OPTIMIZATION

After reading this chapter, the reader will understand:

- The steps involved in query processing
- How SQL queries are translated into relational algebra expressions
- The role of sort operation in database systems and the external sort–merge algorithm used for sorting
- Various algorithms for implementing the relational algebra operations such as select operation, project operation, join operation, set operations, and aggregate operations
- Two approaches of evaluating the expressions containing multiple operations, namely, materialized evaluation and pipelined evaluation
- Different types of query optimization techniques, namely, cost-based query optimization, heuristics-based query optimization, and semantic query optimization
- A set of equivalence rules, which are used to generate a set of expressions that are logically equivalent to the given expression
- How an optimal join ordering helps in reducing the size of intermediate results
- The use of statistics stored in DBMS catalog in estimating the size of intermediate result
- How histograms help in increasing the accuracy of cost estimates
- The steps involved in heuristic-query optimization that transform an initial query tree into an optimal query tree
- How semantic query optimization is different from other two approaches of query optimization
- Query optimization in ORACLE

A database management system manages a large volume of data which can be retrieved by specifying a number of queries expressed in a high-level query language such as SQL. Whenever a query is submitted to the database system, a number of activities are performed to process that query. **Query processing** includes translation of high-level queries into low-level expressions that can be used at the physical level of the file system, query optimization, and actual execution of the query to get the result. **Query optimization** is a process in which multiple query-execution plans for satisfying a query are examined and a most efficient query plan is identified for execution.

This chapter discusses the steps that are performed while processing a high-level query, the algorithms to implement various relational algebra operations and different techniques to optimize a query. It also discusses the operation called external sorting which is one of the primary and relatively expensive operations used in query processing.

8.1 QUERY PROCESSING STEPS

Query processing is a three-step process that consists of parsing and translation, optimization, and execution of the query submitted by the user. These steps are shown in Figure 8.1.

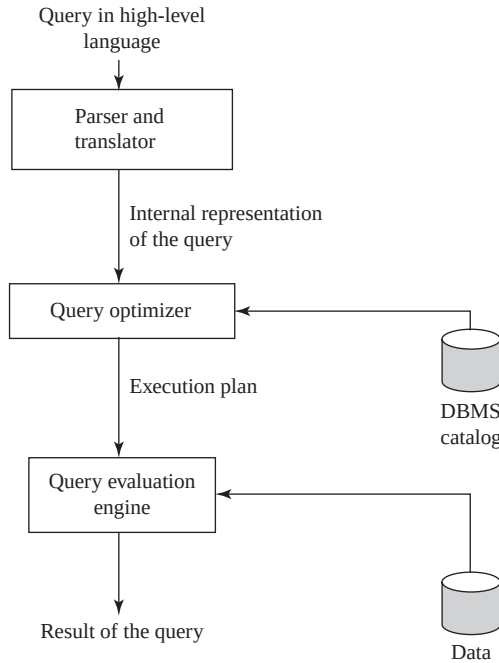


Fig. 8.1 Query processing steps

Whenever a user submits a query in high-level language (such as SQL) for execution, it is first translated into its internal representation suitable to the system. The internal representation of the query is based on the extended relational algebra. Thus, an SQL query is first translated into an equivalent extended relational algebra expression. During translation, the parser (portion of the query processor) checks the syntax of the user's query according to the rules of the query language. It also verifies that all the attributes and relation names specified in the query are the valid names in the schema of the database being queried.

Just as there are a number of ways to express a query in SQL, there are a number of ways to translate a query into a number of relational algebra expressions. For example, consider a simple query to retrieve the title and price of all books with price greater than \$30 from the *Online Book* database. This query can be written in SQL as given here.

```

SELECT Book_title, Price
FROM BOOK
WHERE Price>30;

```

This SQL query can be translated into any of these two relational algebra expressions.

Expression 1: $\sigma_{\text{Price}>30}(\pi_{\text{Book_title, Price}}(\text{BOOK}))$

Expression 2: $\pi_{\text{Book_title, Price}}(\sigma_{\text{Price}>30}(\text{BOOK}))$

The relational algebra expressions are typically represented as query trees or parse trees. A **query tree** is a tree data structure in which the input relations are represented as the leaf nodes and the relational algebra operations as the internal nodes. When the query tree is executed, first the internal node operation is executed whenever its operands are available. Then the internal node is replaced by the relation resulted after the execution of the operation. The execution terminates when the root node

is executed and the result of the query is produced. The query tree representations of the preceding relational algebra expressions are given in Figure 8.2(a).

Each operation in the query (such as select, project, join, etc.) can be evaluated using one of the several different algorithms, which are discussed in Section 8.3. The query tree representation does not specify how to evaluate each operation in the query. To fully specify how to evaluate a query, each operation in the query tree is annotated with the instructions which specify the algorithm or the index to be used to evaluate that operation. The resultant tree structure is known as **query-evaluation plan** or **query-execution plan** or simply **plan**. One possible query-evaluation plan for our example query is given in Figure 8.2(b). In this figure, the select operator σ is annotated with the instruction **Linear Scan** that specifies a complete scan of the relation.

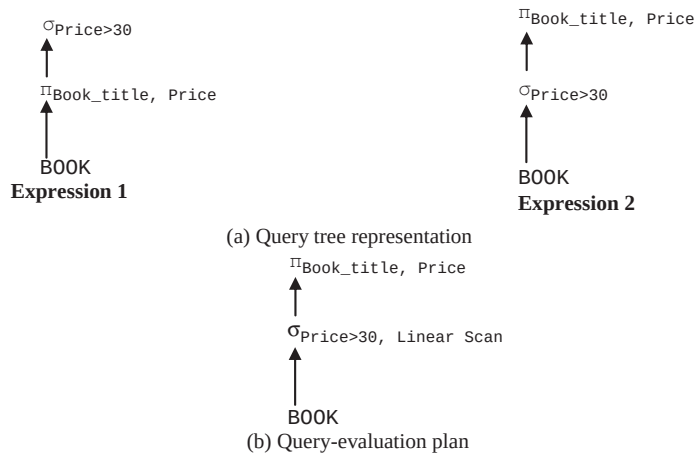


Fig. 8.2 Query trees and query-evaluation plan

Different evaluation plans for a given query can have different costs. It is the responsibility of the **query optimizer**, a component of DBMS, to generate a least-costly plan. The metadata stored in the special tables called **DBMS catalog** is used to find the best way of evaluating a query. Query optimization is discussed in detail in Section 8.5. The best query-evaluation plan chosen by the query optimizer is finally submitted to the **query-evaluation engine** for actual execution of the query to get the desired result.

Query Blocks

Whenever a query is submitted to the database system, it is decomposed into query blocks. A **query block** forms a basic unit that can be translated into relational algebra expression and optimized. It consists of a single **SELECT-FROM-WHERE** expression. It also contains the **GROUP BY** and **HAVING** clauses if these are parts of the block. If the query contains the aggregate operators such as **AVG**, **MAX**, **MIN**, **SUM**, and **COUNT**, these operators are also included in the extended relational algebra.

In case of nested queries, each query within a query is identified as a separate query block. For example, consider an SQL query on **BOOK** relation.

```
SELECT Book_title
FROM BOOK
WHERE Price > (SELECT MIN(Price)
                FROM BOOK
                WHERE Category='Textbook');
```

This query contains a nested subquery and hence, is decomposed into two query blocks. Each query block is then translated into relational algebra expression and optimized. The optimization of nested queries is beyond the discussion of this chapter.

8.2 EXTERNAL SORT-MERGE ALGORITHM

Sorting of tuples plays an important role in database systems. It is discussed before other relational algebra operations because it is required in a variety of situations. Some of these situations are discussed here.

- ⇒ Whenever an **ORDER BY** clause is specified in an SQL query, the output of the query needs to be sorted.
- ⇒ Sorting is useful for eliminating duplicate values in the collection of records.
- ⇒ Sorting can be applied on the input relations before applying join, union, or intersection operations on them to make these operations efficient.

If an appropriate index is available on a sort key, then there is no need to physically sort the relation as the index allows ordered access to the records. However, this type of ordering leads to a disk access (disk seek plus block transfer) for each record, which can be very expensive if the number of records is large. Thus, if the number of records is large, then it is desirable to order the records physically.

If a relation entirely fits in the main memory, in-memory sorting (called **internal sorting**) can be performed using any standard sorting techniques such as quick sort. If the relation does not fit entirely in the main memory, **external sorting** is required. An external sort requires the use of external memory such as disks or tapes during sorting. In external sorting, some part of the relation is read in the main memory, sorted, and written back to the disk. This process continues until the entire relation is sorted. Since, in general, the relations are large enough not to fit in the memory; this section discusses only the external sorting. The most commonly used external sorting algorithm is the external sort-merge algorithm (given in Figure 8.3). As the name suggests, the external sort-merge algorithm is divided into two phases, namely, *sort phase*, and *merge phase*.

- ⇒ **Sort phase:** In this phase, the records in the file to be sorted are divided into several groups, called **runs** such that each run can fit in the available buffer space. An internal sort is applied to each run and the resulting sorted runs are written back to the disk as temporarily sorted runs. The **number of initial runs** (n_1) are computed as $\lceil b_R/M \rceil$, where b_R is the number of blocks containing records of relation R and M is the size of available buffer in blocks. If $M = 5$ blocks and $b_R = 20$ blocks, then $n_1 = 4$ each of size 5 blocks. Thus, after the sorting phase 4 runs are stored on the disk as temporarily sorted runs.
- ⇒ **Merge phase:** In this phase, the sorted runs created during the sort phase are merged into larger runs of sorted records. The merge continues until all records are in one large run. The output of merge phase is the sorted relation.

If $n_1 > M$, it is not possible to allocate a buffer for each sorted run in one pass. In this case, the merge operation is carried out in multiple passes. In each pass, $M - 1$ buffer blocks are used as input buffers which are allocated to $M - 1$ input runs and one buffer block is kept for holding the merge result. The number of runs that can be merged together in each pass is known as **degree of merging** (d_m). Thus, the value of d_m is the smaller of $(M - 1)$ or n_1 . If two runs are merged in each pass, it is known as **two-way merging**. In general, if N runs are merged in each pass, it is known as **N-way merging**.

```
//Sort phase
  Load in M consecutive blocks of the relation.
  //M is number of blocks that can fit in main memory
  Sort the tuples in those M blocks using some internal
  sorting technique.
  Write the sorted run to disk.
  Continue with the next M blocks, until the end of the
  relation.
//Merge phase (assuming that  $n_i < M$ ) //  $n_i$  is the number of
//initial runs
  Load the first block of each run into the buffer page of
  memory.
  Choose the first tuple (with the lowest value) among all
  the runs.
  Write the tuple to an output buffer page and remove it
  from the run.
  When the buffer page of any run is empty, load the next
  block of that run.
  When the output buffer page is full, write it to disk and
  start a new one.
  Continue until all buffer pages are empty.
```

Fig. 8.3 External sort-merge algorithm

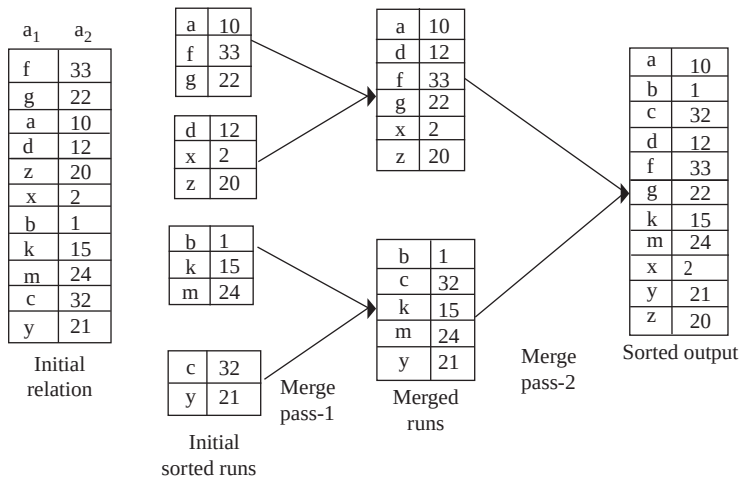


Fig. 8.4 An example of external sort-merge algorithm

An example of external sort-merge algorithm is given in Figure 8.4. We have assumed that the main memory holds at most three page frames out of which two are used as input and one for output during merge phase.

8.3 ALGORITHMS FOR RELATIONAL ALGEBRA OPERATIONS

The system provides a number of different algorithms for implementing several relational algebra operations. Several factors that influence which algorithm performs best include the sizes of the relations involved, existing indexes and sort orders, the size of available buffer pool, and the buffer replacement policy. These algorithms can be developed using any of these simple techniques.

- ⇒ **Indexing:** In this technique, an index can be used to examine the tuples satisfying the condition specified in a select and join operation.
- ⇒ **Iteration:** In this technique, all the tuples of a relation are examined one after the other. However, if the user wants only those attributes from the relation on which an index is defined, then instead of scanning all the tuples in the relation, the index data entries can be scanned.
- ⇒ **Partitioning:** In this technique, the tuples are partitioned on the basis of a sort key. The two commonly used partitioning techniques are *sorting* and *hashing*.

Note that whichever technique is followed to develop an algorithm, the main aim is to minimize the cost of query evaluation. The cost of query evaluation depends on a number of factors such as seek time, number of block transfers between disk and memory, CPU time, storage cost, etc. For simplicity, we have given the cost functions in terms of number of block transfers only, and ignored all the other factors. In distributed or parallel database systems, the cost of communication is also included which is discussed in Chapter 13.

8.3.1 Implementation of Select Operations

The select operation is a simple retrieval of tuples from a relation. There are several algorithms for executing a select operation depending on the selection condition and the file being accessed. These algorithms are also known as **file scans** as they scan the records of the file to locate and retrieve the records satisfying the selection condition. In relational databases, if a relation is stored in a single, dedicated file, a file scan allows an entire relation to be read. If the search algorithm involves the use of indexes, it is known as **index scan**. This section discusses various algorithms for simple and complex select operations. The relational algebra expressions based on the *Online Book* database are given to illustrate the algorithms.

Implementing Simple Select Operations

If the selection condition does not involve any logical operator such as **OR**, **AND**, or **NOT**, it is termed as simple select operation. The algorithms to implement simple select operations are explained here.

- ⇒ **S1 (linear search):** In linear search, every file block is scanned and all the records are tested to see whether their attribute values satisfy the selection condition. The linear search algorithm is also known as **brute force** algorithm. It is the simplest algorithm and can be applied to any file, regardless of the ordering of the file, or the availability of indexes. However, it may be slower than the other algorithms that are used for implementing selection. Since linear search algorithm searches all the blocks of the file, it requires b_R block transfers, where b_R is the total number of blocks of relation R .

If the selection condition involves key attribute, the scan is terminated when the required record is found, without looking further at other records of the relation. The average cost in this case is equal to $b_R/2$ block transfers if the record is found. In the worst case, the select operations involving key attribute may also require the scanning of all the records (b_R block transfers) if no record satisfies the condition or the record is at the last location.

- ⇒ **S2 (binary search):** If the file is ordered on an attribute and the selection condition involves an equality comparison on that attribute, binary search can be used to retrieve the desired tuples. Binary search is more efficient than the linear search as it requires scanning of lesser number of file blocks. For example, for the relational algebra operation $\sigma_{P_ID="P002"}(PUBLISHER)$, binary search algorithm can be applied if the **PUBLISHER** relation is ordered on the basis of the attribute **P_ID**.

The cost of this algorithm is equal to $\lceil \log_2(b_R) \rceil$ block transfers. If the selection condition involves a non-key attribute, it is possible that more than one block contains the required records. Thus, the cost of reading the extra blocks has to be taken into account while estimating the cost.

Linear search and binary search are the basic algorithms for implementing select operation. If the tuples of a relation are stored together in one file, these algorithms can be used to implement the select operation. However, if indexes such as primary, clustering, or secondary indexes are available on the attributes involved in the selection condition, the index can be directly used to retrieve the desired record(s).

An index is also referred to as **access path** as it provides a path through which the data can be found and retrieved. Though indexes provide a fast, direct, and ordered access, they also add an extra overhead of accessing those blocks containing the index. The algorithms that use an index are described here.

Things to Remember

In general, binary search algorithm is not suited for searching purposes in the database, since sorted files are not used unless they have corresponding primary index file also.

- ⇒ **S3 (use of primary index, equality on key attribute):** If the selection condition involves an equality comparison on a key attribute with a primary index, the index can be used to retrieve the record satisfying the equality condition. For example, for the relational algebra operation $\sigma_{P_ID = \text{"P002"}}(\text{PUBLISHER})$, the primary index on the key attribute P_ID (if available) can be used to directly retrieve the desired record. If B⁺-tree is used, the cost of this algorithm is equal to height of the tree plus one block transfer to retrieve the desired record.



The select operation involving the equality comparison on the key attribute retrieves at most a single record, as a relation cannot contain two records with the same key value.

- ⇒ **S4 (use of clustering index, equality on non-key attribute):** If the selection condition involves an equality comparison on the non-key attribute with a clustering index, the index can be used to retrieve all the records satisfying the selection condition. For example, for the relational algebra expression $\sigma_{P_ID = \text{"P002"}}(\text{BOOK})$, the clustering index on the non-key attribute P_ID of the BOOK relation (if available) can be used to retrieve the desired records. The cost of this algorithm is equal to the height of the tree plus the number of blocks containing the desired records, say *b*.
- ⇒ **S5 (use of secondary index, equality on key or non-key attribute):** If the selection condition involves an equality condition on an attribute with a secondary index, the index can be used to retrieve the desired records. This search method can retrieve a single record if the indexing field is a key (candidate key) and multiple records if the indexing field is a non-key attribute. In the first case, only one record is retrieved, thus the cost is equal to height of the tree plus one block transfer to retrieve the desired record. In the second case, multiple records may be retrieved and each record may reside on a different disk blocks, thus, the cost is equal to the height of the tree plus *n* block transfers, where *n* is the number of records fetched.

For example, for the relational algebra expression $\sigma_{Pname = \text{"Hills Publications"}}(\text{PUBLISHER})$, secondary index on the candidate key Pname (if available) can be used to retrieve the desired record.

The selection conditions discussed so far are equality conditions. The selection conditions involving comparisons can be implemented either by using a linear or binary search or by using indexes in one of these ways.

- ⇒ **S6 (use of primary index):** If the selection condition involves comparisons of the form $A > v$, $A \geq v$, $A < v$, or $A \leq v$, where *A* is an attribute with a primary index and *v* is the attribute value, the primary ordered index can be used to retrieve the desired records. The cost of this algorithm is equal to the height of the tree plus $b_r/2$ block transfers.

- For comparison conditions of the form $A \geq v$, the record satisfying the corresponding equality condition $A = v$ is first searched using the primary index. A file scan starting from that record up to the end of file returns all the records satisfying the selection condition. For $A > v$, the file scan starts from the first tuple that satisfy the condition $A > v$ —the record satisfying the condition $A = v$ is not included.
 - For the conditions of the form $A < v$, the index is not searched, rather the file is simply scanned starting from the first record until the record satisfying the condition $A = v$ is encountered. For $A \leq v$, the file is scanned until the first record satisfying the condition $A > v$ is encountered. In both the cases, the index is not useful.
- ⇒ **S7 (use of secondary index):** The secondary ordered index can also be used for the retrievals based on comparison conditions such as $<$, $\leq$, $>$, or $\geq$. For the conditions of the form $A < v$ or $A \leq v$, the lowest-level index blocks are scanned from the smallest value up to v and for the conditions of the form $A > v$ or $A \geq v$, the index blocks are scanned from v up to the end of the file. The secondary index provides the pointers to the records and not the actual records. The actual records are fetched using the pointers. Thus, each record fetch may require a disk access, since records may be on different blocks. This implies that using the secondary index may even be more expensive than using the linear search if the number of fetched records is large.

Implementing Complex Select Operations

The selection conditions discussed so far are assumed to be simple conditions of the form $A \text{ op } B$, where op can be $=$, $<$, $\leq$, $>$, or $\geq$. If the select operation involves complex conditions, made up of several simple conditions connected with logical operators **AND** (conjunction) or **OR** (disjunction), some additional algorithms are used.

- ⇒ **S8 (conjunctive selection using one index):** If an attribute involved in one of the simple conditions specified in the conjunctive condition has an access path (such as indexes), any one of the selection algorithms S2 to S7 can be used to retrieve the records satisfying that condition. Each retrieved record is then checked to determine whether it satisfies the remaining simple conditions. The cost of this algorithm depends on the cost of the chosen algorithm.

For example, for the relational algebra operation $\sigma_{P_ID="P001" \wedge \text{Category}="Textbook"}(\text{BOOK})$, if a clustering index is available on the attribute P_ID (algorithm S4), that index can be used to retrieve the records with $P_ID="P001"$. These records can then be checked for the other condition. The records that do not satisfy the condition $\text{Category}="Textbook"$ are discarded.

- ⇒ **S9 (conjunctive selection using composite index):** If a select operation specifies an equality condition on two or more attributes, the composite index (if available) on these combined attributes can be used to directly retrieve the desired records. For example, consider a relational algebra operation $\sigma_{R_ID="A001" \wedge \text{ISBN}="002-678-980-4"}(\text{REVIEW})$. If an index is available on the composite key (ISBN , R_ID) of the **REVIEW** relation, it can be used to directly retrieve the desired records.
- ⇒ **S10 (conjunctive selection by intersection of record pointers):** If indexes or other access paths with record pointers (and not block pointers) are available on the attributes involved in the individual conditions, then each index can be used to retrieve the set of record pointers that satisfy an individual condition. The intersection of all the retrieved record pointers gives the pointers to the tuples that satisfy the conjunctive condition.

For example, consider the relational algebra expression $\sigma_{P_ID="P001" \wedge \text{Category}="Textbook"}(\text{BOOK})$. If secondary indexes are available on the attributes P_ID and Category of **BOOK** relation, then these indexes can be used to retrieve the pointers to the tuples satisfying the individual

Resultant relation R1: $\sigma_{P_ID="P001" \wedge \text{Category}="Textbook"}(\text{BOOK})$

Record pointers	ISBN	Book_title	Category	Price	Copy right -date	Year	Page_count	P_ID
r ₁	001-354-921-1	Ransack	Novel	22	2005	2006	200	P001
r ₂	001-987-650-5	Differential Calculus	Textbook	30	2003	2003	450	P001
r ₃	001-987-760-9	C++	Textbook	40	2004	2005	800	P001

Resultant relation R2: $\sigma_{P_ID="P001" \vee \text{Category}="Textbook"}(\text{BOOK})$

	ISBN	Book_title	Category	Price	Copy right -date	Year	Page_count	P_ID
r ₂	001-987-650-5	Differential Calculus	Textbook	30	2003	2003	450	P001
r ₃	001-987-760-9	C++	Textbook	40	2004	2005	800	P001
r ₄	002-678-980-4	DBMS	Textbook	40	2004	2006	800	P002
r ₅	004-765-359-3	Coordinate Geometry	Textbook	35	2006	2006	650	P003
r ₆	004-765-409-5	UNIX	Textbook	26	2006	2007	550	P003

$R1 \cap R2$

	ISBN	Book_title	Category	Price	Copy right -date	Year	Page_count	P_ID
r ₂	001-987-650-5	Differential Calculus	Textbook	30	2003	2003	450	P001
r ₃	001-987-760-9	C++	Textbook	40	2004	2005	800	P001

$R1 \cup R2$

	ISBN	Book_title	Category	Price	Copy right -date	Year	Page_count	P_ID
r ₁	001-354-921-1	Ransack	Novel	22	2005	2006	200	P001
r ₂	001-987-650-5	Differential Calculus	Textbook	30	2003	2003	450	P001
r ₃	001-987-760-9	C++	Textbook	40	2004	2005	800	P001
r ₄	002-678-980-4	DBMS	Textbook	40	2004	2006	800	P002
r ₅	004-765-359-3	Coordinate Geometry	Textbook	35	2006	2006	650	P003
r ₆	004-765-409-5	UNIX	Textbook	26	2006	2007	550	P003

Fig. 8.5 An example of search algorithms S10 and S11

condition. The intersection of these record pointers yields the record pointers of the desired tuples. These records pointers are then used to retrieve the actual tuples (see Figure 8.5).

If indexes are not available for all the individual conditions, the retrieved records are further checked to determine whether they satisfy the remaining simple conditions. The cost of this algorithm is the cost of individual index scan plus the cost of retrieving the actual tuples. If the list of pointers is

sorted before actually retrieving the actual tuples, this cost can be further reduced as it allows retrieval of the records in sorted order.

⇒ **S11 (disjunctive selection by union of record pointers):** If indexes or other access paths are available on the attributes involved in the individual conditions, each index can be used to retrieve the set of record pointers that satisfy an individual condition. The union of all retrieved pointers gives the pointers to the tuples that satisfy the disjunctive condition.

For example, consider the relational algebra expression $\sigma_{P_ID="P001" \vee Category="Textbook"}(BOOK)$. If secondary indexes are available on the attributes **P_ID** and **Category** of **BOOK** relation, then these indexes can be used to retrieve the pointers to the tuples satisfying an individual condition. The union of these record pointers yields the pointers to the desired tuples. These records pointers are then used to retrieve the actual tuples (see Figure 8.5). If an index is not available on even one of the simple conditions, a linear search has to be performed on the relation to find the tuples that satisfy the disjunctive condition.

8.3.2 Implementation of Join Operations

The most time-consuming and complex operation in query processing is the join operation. If join operation is performed on two relations, it is termed as **two-way join**, and if more than two relations are involved in join, it is termed as **multi way join**. For simplicity, only two-way joins are discussed in this section. We will discuss various algorithms for join operation of the form $R \bowtie_{R.A=S.B} S$, where **R** and **S** are the two relations on which join operation is to be applied, and **A** and **B** are the domain-compatible attributes of **R** and **S**, respectively.

To understand the algorithms for join operations, consider the following operation on **PUBLISHER** relation of the *Online Book* database.

$BOOK \bowtie_{BOOK.P_ID = PUBLISHER.P_ID} PUBLISHER$

Further, assume the following information about these two relations. This information will be used to estimate the cost of each algorithm.

number of tuples of **PUBLISHER** = $n_{PUBLISHER} = 2000$
 number of blocks of **PUBLISHER** = $b_{PUBLISHER} = 100$
 number of tuples of **BOOK** = $n_{BOOK} = 5000$
 number of blocks of **BOOK** = $b_{BOOK} = 500$

The various algorithms for implementing join operations are discussed here. For simplicity, the join condition **BOOK.P_ID = PUBLISHER.P_ID** is removed from all the expressions.

J1 (Nested-Loop Join)

The nested-loop join algorithm consists of a pair of nested **for** loops, one for each relation say, **R** and **S**. One of these relations, say **R** is chosen as the **outer relation** and the other relation **S** as the **inner relation**. The outer relation is scanned row by row and for each row in the outer relation the inner relation is scanned and looked for the matching rows. In other words, for each tuple **r** in **R**, every tuple **s** in **S** is retrieved and then checked whether the two tuples satisfy the join condition $r[A]=s[B]$. If the join condition is satisfied, then the values of these two tuples are concatenated, which is then added to the result. The tuple constructed by concatenating the values of the tuples **r** and **s** is denoted by **r . s**. Since this search scans the entire relation, it is also called **naive nested loop join**.

For natural join, this algorithm can be extended to eliminate the repeated attributes using a project operation. The nested-loop algorithms for equijoin and natural join operations are given in Figure 8.6. This

```

for each tuple r in R do begin
    for each tuple s in S do begin
        if r[A]=s[B] then add r.s to the result.
    end
end

```

(a) Nested-loop join for equijoin operation

```

for each tuple r in R do begin
    for each tuple s in S do begin
        if r[A]=s[B] then
            begin
                delete one of the repeated attributes from
                the tuple r.s.
                add r.s to the result.
            end
        end
    end
end

```

(b) Nested-loop join for natural join operation

Fig. 8.6 *Nested-loop join*

algorithm is similar to the linear search algorithm for the select operation in a way that it does not require any indexes and it can be used with any join condition. Thus, it is also called **brute force** algorithm.

An extra buffer block is required to hold the tuples resulted from the join operation. When this block is filled, the tuples are appended to the disk file containing the join result (known as **result file**). This buffer block can then be reused to hold additional result records.

The nested-loop join algorithm is expensive, as it requires scanning of every tuple in S for each tuple in R . The pairs of tuples that need to be scanned are $n_R * n_S$, where n_R and n_S denote the number of tuples in R and S , respectively. For example, if **PUBLISHER** is taken as outer relation and **BOOK** as inner relation, $2000 * 5000 = 10^7$ pairs of tuples need to be scanned.

In terms of number of block transfers, the cost of this algorithm is equal to $b_R + (n_R * b_S)$, where b_R and b_S denote the number of blocks of R and S , respectively. For example, if **PUBLISHER** is taken as outer relation and **BOOK** is taken as inner relation, $100 + (2000 * 500) = 1,000,100$ block transfers are required, which is very high. On the other hand, if **BOOK** is taken as outer relation and **PUBLISHER** as inner relation, then $500 + (5000 * 100) = 5,00,500$ block transfers are required. Thus, by making the smaller relation as the inner relation, the overall costs of the join operation can be reduced.

If both or one of the relations fit entirely in the main memory, the cost of join operation will be $b_R + b_S$ block transfers. This is the best-case scenario. For example, if either of the relations **PUBLISHER** and **BOOK** (or both) fits entirely in the memory, $100 + 500 = 600$ block transfers are required, which is optimal.

J2 (Block Nested-Loop Join)

If the buffer is too small to hold either of the two relations entirely, the block accesses can still be reduced by processing the relations on a per-block basis rather than on a per-tuple basis. Thus, each block in the inner relation S is read only once for each block in the outer relation R (instead of once for each tuple in R). In the worst case, when neither of the relations fit entirely in the memory, this algorithm requires $b_R + (b_R * b_S)$ block transfers. It is efficient to use the smaller relation as the outer relation. For example, if **PUBLISHER** is taken as outer relation and **BOOK** as inner relation, $100 + (100 * 500) = 50,100$ block transfers are required (which is much lesser than 1,000,100 or 5,00,500 block transfers as in the nested-loop join).

In the best case, where one of the relations fits entirely in the main memory, the algorithm requires $b_R + b_S$ block transfers, which is same as that of nested-loop join. In this case, the smaller relation is chosen as the inner relation. The algorithm for block nested-loop join operation is given in Figure 8.7.

```

for each block  $B_R$  of  $R$  do begin
    for each block  $B_S$  of  $S$  do begin
        for each tuple  $r$  in  $B_R$  do begin
            for each tuple  $s$  in  $B_S$  do begin
                if  $r[A]=s[B]$  then
                    add  $r.s$  to the result.
            end
        end
    end
end
end

```

Fig. 8.7 Block nested-loop join

J3 (Indexed Nested-Loop Join)

If an index is available on the join attribute of one relation say S , it is taken as the inner relation and the index scan (instead of file scan) is used to retrieve the matching tuples. In this algorithm, each tuple r in R is retrieved, and the index available on the join attribute of relation S is used to directly retrieve all the matching tuples. Indexed nested-loop join can be used with the existing as well as with a **temporary index**—an index created by the query optimizer and destroyed when the query is completed. If a temporary index is used to retrieve the matching records, the algorithm is called **temporary indexed nested-loop join**. If indexes are available on both the relations, the relation with fewer tuples can be used as the outer relation for better efficiency.

For example, if an index is available on the attribute P_ID of the **BOOK** relation, then the **BOOK** relation can be used as the inner relation. If a tuple with $P_ID="P001"$ exists in the **PUBLISHER** relation, then the matching tuples in the **BOOK** relation are those that satisfy the condition $P_ID="P001"$. Since an index is available on P_ID , the matching tuples can be directly retrieved. The cost of this algorithm can be computed as $b_R + n_R * C$, where C is the cost of traversing the index and retrieving all the matching tuples of S . The algorithm for indexed nested-loop join operation is given in Figure 8.8.

```

for each tuple  $r$  of  $R$  do begin
    probe the index on  $S$  to locate all tuples  $s$  such that
     $s[B]=r[A]$ .
    for each matching tuple  $s$  in  $S$  do
        add  $r.s$  to the result.
    end
end

```

Fig. 8.8 Indexed nested-loop join

J4 (Sort-Merge Join)

If the tuples of both the relations are physically sorted by the value of their respective join attributes, the sort-merge join algorithm can be used to compute equijoins and natural joins. Both the relations are scanned concurrently in the order of the join attributes to match the tuples that have the same values for the join attributes. Since the relations are in sorted order, each tuple needs to be scanned only once and hence, each block is also read only once. Thus, if both the relations are in sorted order, the sort-merge join requires only a single pass through both the relations.

The number of block transfers is equal to the sum of number of blocks in both the relations R and S , that is, $b_R + b_S$. Thus, the join operation $BOOK \bowtie PUBLISHER$ requires $100 + 500 = 600$ block transfers. If the relations are not sorted, they can be first sorted using external sorting. The sort-merge join algorithm is given in Figure 8.9(a) and an example of sort-merge join algorithm is given in Figure 8.9(b). In this figure, the relations R and S are sorted on the join attribute a_1 .

A variation of sort-merge algorithm can also be performed on unsorted relations if secondary indexes are available on the join attributes of both the relations. The records are scanned through indexes

if R is unsorted then sort the tuples in R on the attribute A.
 if S is unsorted then sort the tuples in S on the attribute B.

```

i:=1;
j:=1;
t:=1;
while(i<=nR)do begin
    while (R(i)[A]==S( t)[B])do begin
        add R(i).S(t) to the result.
        t := t+1;
    end
    i :=i+1;
    if (R( i )[A]==R( i - 1 )[ A]) then
        t=j;
    else
        j := t ;
end
    
```

(a) Sort-merge join

a ₁	a ₂
a	2
b	13
d	7
e	9
e	11
f	1
g	10

Relation R

a ₁	a ₃
b	2.5
b	7.1
c	8.2
d	9.9
e	11.0
e	2.1

Relation S

a ₁	a ₁	a ₂	a ₃
b	b	13	25
b	b	13	7.1
d	d	7	9.9
e	e	9	11.0
e	e	9	2.1
e	e	11	11.0
e	e	11	2.1

Equijoin using
sort-merge join

a ₁	a ₂	a ₃
b	13	2.5
b	13	7.1
d	7	9.9
e	9	11.0
e	9	2.1
e	11	11.0
e	11	2.1

Natural join using
sort-merge join

(b) An example of sort-merge join

Fig. 8.9 Implementing sort-merge join

which allow the retrieval of records in sorted order. Since the records are physically scattered all over the file blocks, accessing each tuple may require accessing a disk block, which is very expensive.

This cost can be reduced by using a **hybrid merge-join technique** which combines sort-merge join with indexes. To understand the hybrid merge-join algorithm, consider a sorted relation R and an unsorted relation S having a secondary B⁺-tree index on the join attribute. The tuples of sorted relation are merged with the leaf entries of the secondary B⁺-tree index. The result file in this case contains the tuples from the sorted relation and the addresses of the tuples of the unsorted relation. This result file is then sorted on the basis of the addresses of tuples of the unsorted relation. The corresponding tuples can then be retrieved in physical storage order to complete the join. If both the relations are unsorted, the addresses of the tuples of both the relations are merged and sorted, and the corresponding tuples are retrieved from both the relations in physical storage order.

J5 (Hash Join)

In this algorithm, a hash function h is used to partition the tuples of each relation into sets of tuples with each set containing tuples that have the same hash value on the join attributes A of relation R and B of relation S. The algorithm is divided into two phases, namely, *partitioning phase* and *probing*

phase. In **partitioning** (also called **building**) phase, the tuples of relations R and S are partitioned into the hash file buckets using the same hash function h . Let $P_{r0}, P_{r1}, \dots, P_{rn}$ be the partitions of tuples in relation R and $P_{s0}, P_{s1}, \dots, P_{sn}$ be the partitions of tuples in relation S . Here, n is the number of partitions of relations R and S . The main property of hash join is that the tuples in the partition P_{ri} need to be joined with tuples in the partition P_{si} . This is because of the fact that if tuples of relation R with some hash value hash to i , then the tuples of relation S with same hash value also hash to i .

During partitioning phase, a single in-memory buffer with size one disk block is allocated for each partition to store the tuples that hash to this partition. Whenever the in-memory buffer gets filled, its contents are appended to a disk sub file that stores this partition. For a simple two-way join, the partitioning phase has two iterations. In the first iteration, the relation R is partitioned into n partitions, and in second iteration, the relation S is partitioned in n partitions. In **probing** (also called **matching** or **joining**) phase, n iterations are required. In i^{th} iteration, the tuples in the partition P_{ri} are joined with the tuples in the corresponding partition P_{si} .

The hash join algorithm requires $3(b_R + b_S) + 4n$ block transfers. The first term $3(b_R + b_S)$ represents that the block transfer occurs three times—once when the relations are read into the main memory for partitioning, second when they are written back to the disk and third, when each partition is read again during the joining phase. The second term $4n$ represents an extra overhead ($2n$ for each relation) of reading and writing of each partition to and from the disk—since the number of blocks occupied by each partition is slightly more than $b_R + b_S$ because of partially filled blocks. The extra overhead of $4n$ is generally very small as compared to $b_R + b_S$ and hence, can be ignored.

The main difficulty of the algorithm is to ensure that the partitioning hash function is uniform, that is, it creates partitions of almost same size. If the partitioning hash function is non-uniform, then some partitions may have more tuples than the average, and they might not fit in the available main memory. This type of partitioning is known as **skewed partitioning**. This problem can be handled by further partitioning the partition into smaller partitions using a different hash function.

Recursive Partitioning and Hybrid Hash Join If the value of n is less than the value $M-1$ (M is the number of page frames in the main memory), the hash join algorithm is very simple and efficient to execute as both the relations can be partitioned in one pass. However, if the value n is greater than or equal to M , then the relations cannot be partitioned in a single pass, rather partitions have to be done in repeated passes.

In the first pass, the input relation is partitioned into $M-1$ partitions (one page frame is kept to be used as input buffer) using a hash function h . In the next pass, each partition created in the previous pass is partitioned again to create smaller partitions using a different hash function. This process is repeated until each partition of the input relation can fit entirely in the memory. This technique is called **recursive partitioning**. Note that a different hashing function is used in each pass. A relation S does not require recursive partitioning if $M > \sqrt{b_S}$ or equivalently $M > n+1$.

If memory size is relatively large, a variation of hash join called **hybrid hash join** can be used for better performance. In hybrid hash join technique, the probing phase for one of the partitions is combined with the partitioning phase. The main objective of this technique is to join as many tuples during the partitioning phase as possible in order to save the cost of storing these tuples back to the disk and accessing them again during the joining (probing) phase.

To better understand the hybrid hash join algorithm, consider a hash function $h(K) = K \bmod n$ that creates n partitions of the relation, where $n < M$ and consider the join operation $\text{BOOK} \bowtie \text{PUBLISHER}$.

In the first iteration, the algorithm divides the buffer space among n partitions such that all the blocks of first partition of the smaller relation **PUBLISHER** completely reside in the main memory. A single input buffer (with size one disk block) is kept for each of the other partitions, and they are written back to the disk as in the regular hash join. Thus, at the end of the first iteration of the

partitioning phase, the first partition of the relation PUBLISHER resides completely in the main memory and other partitions reside in a disk sub file.

In the second iteration of the partitioning phase, the tuples of the second relation BOOK are being partitioned. If a tuple hashes to the first partition, it is immediately joined with the matching tuple of PUBLISHER relation. If the tuple does not hash to the first partition, it is partitioned normally. Thus, at the end of the second iteration, all the tuples of BOOK relation that hash to the first partition have been joined with the tuples of PUBLISHER relation. Now, there are $n-1$ pairs of partitions on the disk. Therefore, during the joining phase, $n-1$ iterations are needed instead of n .

8.3.3 Implementation of Project Operations

A project operation of the form $\pi_{\langle \text{attribute_list} \rangle}(R)$ is easy to implement if the $\langle \text{attribute_list} \rangle$ includes the key attribute of the relation R . This is because in this case duplicate elimination is not required, and the result will contain the same number of tuples as R but with only the values of attributes listed in the $\langle \text{attribute_list} \rangle$. However, if the $\langle \text{attribute_list} \rangle$ does not contain the key attribute, duplicate tuples may exist and must be eliminated. Duplicate tuples can be eliminated by using either sorting or hashing.

- ⇒ **Sorting:** The result of the operation is first sorted, and then the duplicate tuples that appear adjacent to each other are removed. If external sort-merge technique is used, the duplicate tuples can be found while creating runs, and they can be removed before writing the runs on the disk. The remaining duplicate tuples can be removed during merging. Thus, the final sorted run contains no duplicates.
- ⇒ **Hashing:** As soon as a tuple is hashed and inserted into in-memory hash file bucket, it is first checked against those tuples that are already present in the bucket. The tuple is inserted in the bucket if it is not already present, otherwise it is discarded. All the tuples are processed in the same way.

Note that the cost of duplicate elimination is very high. Thus, in SQL the user has to give an explicit request to remove duplicates using the **DISTINCT** keyword. The algorithm to implement project operation by eliminating duplicates using sorting is given in Figure 8.10.

```

for each tuple r in R do
    add a tuple r[<attribute_list>] in T'.
    //T' may contain duplicate tuples
if <attribute_list> includes the key attribute of R
then
    T:=T';      // T contains the projection result
                // without duplicates
else
begin
    sort the tuples in T'.
    i:=1;
    j:=2;
    while(i<=nR) do begin
        if (T'(i)[A]==T'(j)[A]) then
            j:=j+1;
        else begin
            output the tuple T'[i] to T.
            i:=j;
        end
    end
end
end

```

Fig. 8.10 Algorithm for project operation

8.3.4 Implementation of Set Operations

There are four set operations, namely, *union*, *intersection*, *set difference*, and *Cartesian product*. Generally, the Cartesian product operation $R \times S$ is quite expensive as it results in a large resultant relation with $n_R * n_S$ tuples and $j+k$ attributes (j and k are the number of attributes in R and S , respectively). Thus, it is better to avoid Cartesian product operation.

The other three set of operations, namely, *union*, *intersection*, and *set difference* can be implemented by first sorting the relations on the basis of same attribute and then scanning each sorted relation once to produce the result. For example, the operation $R \cup S$ can be implemented by concurrently scanning and merging both the sorted relations—whenever same tuple exists in both the relations, only one of them is kept in the merged result. The operation $R \cap S$ can be implemented by adding only those tuples to the merged result that exist in both the relations. Finally, the operation $R - S$ is implemented by retaining only those tuples that exist in R but are not present in S .

If the relations are initially sorted in the same order, all these operations require $b_R + b_S$ block transfers. If the relations are not sorted initially, the cost of sorting the relations has to be included. The algorithms for the operations $R \cup S$, $R \cap S$, and $R - S$ using sorting are given in Figure 8.11.

Each of these operations can also be implemented using hashing as shown below.

Implementing $R \cup S$

1. Partition the tuples of relation R into hash file buckets.
2. Probe (hash) each tuple of relation S , and insert the tuple in the bucket if no identical tuple from R is found in the bucket.
3. Add the tuples in the hash file bucket to the result.

Implementing $R \cap S$

1. Partition the tuples of relation R into hash file buckets.
2. Probe (hash) each tuple of relation S , and if an identical tuple from R is already present in the bucket, then add that tuple to the result file.

Implementing $R - S$

1. Partition the tuples of relation R into hash file buckets.
2. Probe (hash) each tuple of relation S , and if an identical tuple from R is already present in the bucket, then remove that tuple from the bucket.
3. Add the tuples remaining in the bucket to the result file.

8.3.5 Implementation of Aggregate Operations

If the aggregate operators **MIN**, **MAX**, **SUM**, **AVG**, and **COUNT** are applied to an entire relation (that is, without using a **GROUP BY** clause), they can be computed either by scanning the entire relation or by using an index, if available. For example, consider the following operation.

$\mathcal{F}_{\text{Max(Price), MIN(Price)}}(\text{BOOK})$

If an index on the **Price** attribute of the **BOOK** relation exists, it can be used to search for the largest and the smallest price value. In case, the **GROUP BY** clause is included in the query, the aggregate operation can be implemented either by using sorting or hashing as done for duplicate elimination. The only difference is that instead of removing the tuples with the same value for the grouping attribute, these tuples are grouped together and the aggregate operations are applied on each group to

```

sort the tuples of R and S on the basis of same unique sort
attributes.
i:=1;
j:=1;
while(i<=nR) and (j<=nS) do begin
    if(R(i) < S(j)) then begin
        add R(i) to the result.
        i:=i+1;
    end
    elseif (R(i) > S(j)) then begin
        add S(j) to the result.
        j:=j+1;
    end
    else begin
        add R(i) to the result.
        i:=i+1;
        j:=j+1;    //R(i)=S(j), add only one tuple
    end
end
while(i<=nR) do begin
    add R(i) to the result.    //add remaining tuples
                                //of R, if any
    i:=i+1;
end
while(j<=nS) do begin
    add S(j) to the result.    //add remaining tuples
                                //of S, if any
    j:=j+1;
end
end
(a) Implementing UNION operation

```

```

sort the tuples of R and S on the basis of same unique sort
attributes.
i:=1;
j:=1;
while(i<=nR) and (j<=nS) do begin
    if(R(i) < S(j)) then
        i:=i+1;
    elseif(R(i) > S(j)) then
        j:=j+1;
    else begin
        //R(i)=S(j), add one of the tuples to the result
        add R(i) to the result.
        i:=i+1;
        j:=j+1;
    end
end
(b) Implementing INTERSECTION operation

```

```

sort the tuples of R and S on the basis of same unique sort
attributes.
i:=1;
j:=1;
while(i<=nR) and (j<=nS)do begin
    if(R(i) < S(j)) thenbegin
        add R(i) to the result.
        //R(i)has no matchingS(j)
        //so add R(i) to the result
        i:=i+1;
    end
    if(R(i) > S(j)) then
        j:=j+1;
    else begin
        //R(i)=S(j) so skip both the tuples
        i:=i+1;
        j:=j+1;
    end
end
while(i<=nR) do begin
    add R(i) to the result.
    //add remaining tuples of R, if any
    i:=i+1;
end
(c) Implementing SET DIFFERENCE operation

```

Fig. 8.11 *Implementing set operations*

get the desired result. The cost of implementing the aggregate operation is same as that of duplicate elimination.

The aggregate operations can also be implemented during the creation of the groups. For example, as soon as two tuples in the same group are found, these two tuples can be replaced with a single tuple containing the result of SUM, MIN, or MAX in the columns being aggregated. In case of COUNT operation, the system maintains a counter for each group and as soon as a tuple belonging to a particular group is found, the counter for that group is incremented by one. The AVG operation is implemented by dividing the sum and count values calculated using the method described earlier.

Learn More

If there exists a clustering index for a grouping attribute, the tuples are already grouped on the basis of grouping attribute and can be directly used for the required computation.

8.4 EXPRESSIONS CONTAINING MULTIPLE OPERATIONS

So far we have discussed how individual relational algebra operations are evaluated. This section discusses how the expressions containing multiple operations are evaluated. Such expressions are evaluated either using materialization approach or pipelining approach.

8.4.1 Materialized Evaluation

In materialized evaluation, each operation in the expression is evaluated one by one in an appropriate order and the result of each operation is **materialized** (created) in a temporary relation which becomes input for the subsequent operations. For example, consider this relation algebra expression.

$$\pi_{\text{Pname, Email_id}}(\sigma_{\text{Category}=\text{"Novel"}}(\text{BOOK}) \bowtie \text{PUBLISHER})$$

This expression consists of three relational operations, join, select, and project. The query tree (also known as **operator tree** or **expression tree**) of this expression is given in Figure 8.12.

In materialization approach, the lowest-level operations (operations at the bottom of operator tree) are evaluated first. The inputs to the lowest-level operations are the relations in the database. For example, in Figure 8.12, the operation $\sigma_{\text{Category}=\text{"Novel"}}(\text{BOOK})$ is evaluated first and the result is stored in a temporary relation. The temporary relation and the relation PUBLISHER are then given as inputs to the next-level operation, that is, the join operation. The join operation then performs a join on these two relations and stores the result in another temporary relation which is given as input to the project operation which is at the root of the tree.

A single buffer is used to hold the intermediate result and when it gets filled it is written back to the disk. If **double buffering** is used, the expression evaluation becomes fast. In double buffering, two buffers are used—one of them continues with the execution of the algorithm while the other being written to the disk. This allows CPU activity to perform in parallel with I/O activity. The cost of materialized evaluation is the sum of the costs of the individual operations involved plus the cost of writing the intermediate results to the disk.

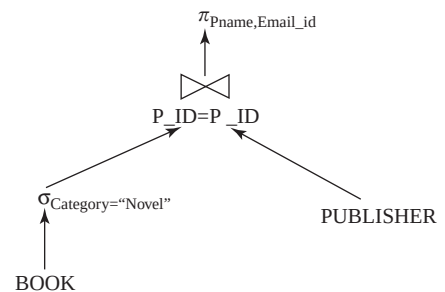


Fig. 8.12 Operator tree for the expression

8.4.2 Pipelined Evaluation

The main limitation of materialized evaluation is that it results in a number of temporary relations which affects the query-evaluation efficiency. To improve the efficiency of query evaluation, we need to reduce the number of temporary relations that are produced during query execution. This can be achieved by combining several operations into a **pipeline** or a **stream** of operations. This type of evaluation of expressions is called **pipelined evaluation** or **stream-based processing**. With pipelined evaluation, operations form a queue and results are passed from one operation to another as they are calculated. This technique thus avoids write-outs of the temporary relations.

For example, the operations given in Figure 8.12 can be placed in a pipeline. As soon as a tuple is generated from the select operation, it is immediately passed to the join operation for processing. Similarly, a tuple generated from the join operation is passed immediately to the project operation for processing. Thus, the evaluation of these three operations in a pipelined manner directly generates the final result without creating temporary relations. The system maintains one buffer for each pair of adjacent operations to hold the tuples being passed from one operation to the next. Since the results of the operations are not stored for a long time, lesser amount of memory is required in pipelining approach. However, the inputs are not available to the operations for processing all at once in the pipelining.

There are two ways of executing pipelines, namely, *demand-driven pipeline* and *producer-driven pipeline*.

- ⇒ **Demand-driven pipeline:** In this approach, the parent operation requests next tuple from its child operations as required, in order to output its next tuple. Whenever an operation receives a request for the next set of tuples, it computes those tuples and then returns them. Since, the tuples are generated *lazily*, on demand; this technique is also known as **lazy evaluation**.
- ⇒ **Producer-driven pipeline:** In this approach, each operation at the bottom of the pipeline produces tuples without waiting for the request from the next higher-level operations. Each operation then puts the output tuples in the output buffer associated with it. This output buffer is used as input buffer by the operation at next higher level. The operation at any other level consumes the tuples from its input buffer to produce the output tuples, and puts them in its output buffer. In any case, if the output buffer is full, the operation has to wait until tuples are consumed by the operation at the next higher-level. As soon as the buffer has some space for more tuples, the operation again starts producing the tuples. This process continues until all the tuples are generated. Since, the tuples are generated *eagerly*; this technique is also known as **eager pipelining**.

8.5 QUERY OPTIMIZATION

The success of relational database technology is largely due to the systems' ability to automatically find evaluation plans for declaratively specified queries. The query submitted by the user can be processed in a number of ways especially if the query is complex. Thus, it is the responsibility of the query optimizer to construct a most efficient query-evaluation plan having the minimum cost.

To find the least-costly plan, the query optimizer generates alternative plans that produce the same result as that of the given expression and then chooses the one with the minimum cost. There are two main techniques of implementing query optimization, namely, *cost-based optimization* and *heuristics-based optimization*. Practical query optimizers incorporate elements of both these approaches. In addition to these approaches,

Learn More

The languages used in legacy database systems such as network DML and hierarchical DML do not provide the query optimization techniques. Thus, the programmer has to choose the query-execution plan while writing a database program. On the other hand, the query optimization is necessary in languages used for relational DBMSs such as SQL as they only specify what data is required rather than how it should be obtained.

another approach called **semantic query optimization** has also been introduced which is knowledge-based optimization. It makes use of problem-specific knowledge, represented as integrity constraints, to answer the queries efficiently.

8.5.1 Cost-Based Query Optimization

A cost-based query optimizer generates a number of query-evaluation plans from a given query using a number of equivalence rules, and then chooses the one with the minimum cost. The cost of executing a query include:

- ⇒ cost of accessing secondary storage—cost of searching, reading, and writing disk blocks.
- ⇒ storage cost—cost of storing intermediate results.
- ⇒ computation cost—cost of performing in-memory operations.
- ⇒ memory usage cost—cost related to the number of occupied memory buffers.
- ⇒ communication cost—cost of communicating the query result from one site to another.

The cost of each evaluation plan can be estimated by collecting the statistical information about the relations such as relation sizes and available primary and secondary indexes from the DBMS catalog. This section discusses the various equivalence rules which are used to generate a set of expressions that are logically equivalent to the given expression, and the various techniques to estimate the statistics.

Equivalence Rules

An expression is said to be **equivalent** to a given relational algebra expression if, on every legal database instance, it generates the same set of tuples as that of original expression irrespective of the ordering of the tuples. For example, consider a relational algebra expression for the query “Retrieve the publisher name and category of the book C++”.

$$\pi_{Pname, Category}(\sigma_{Book_title="C++"}(BOOK \bowtie PUBLISHER))$$

The initial operator tree of this expression is given in Figure 8.13(a). It is clear from the initial operator tree of this expression that first the join operation is applied on the relations **BOOK** and **PUBLISHER**, which results in a large intermediate relation. However, the user is only interested in those tuples having **Book_title="C++"**. Thus, another way to execute this query is to first select only those tuples from the **BOOK** relation satisfying the condition **Book_title="C++"**, and then join it with the **PUBLISHER** relation. This reduces the size of the intermediate relation. The query can now be represented by the following expression.

$$\pi_{Pname, Category}(\sigma_{Book_title="C++"}(BOOK) \bowtie PUBLISHER)$$

This expression is equivalent to the original expression; however, with less number of tuples in the intermediate relation and thus, it is more efficient. The transformed operator tree of the equivalent expression is given in Figure 8.13(b). Therefore, the main aim of generating logically equivalent expressions is to minimize the size of intermediate results.

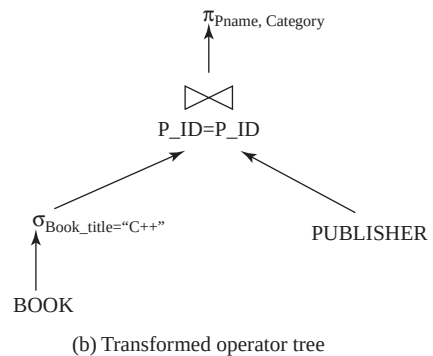
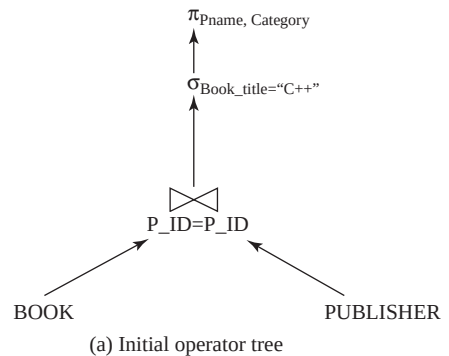


Fig. 8.13 Initial and transformed operator trees

Equivalence rules specify how to transform an expression into a logically equivalent one. An **equivalence rule** states that expressions of two forms are equivalent if an expression of one form can be replaced by expression of the other form, and vice versa. While discussing the equivalence rules it is assumed that $C, C_1, C_2, \dots, C_n$ are the predicates, $A, A_1, A_2, \dots, A_n$ are the set of attributes and $R, R_1, R_2, \dots, R_n$ are the relation names.

Rule 1: The selection condition involving conjunction of two or more predicates can be deconstructed into a sequence of individual select operations. That is,

$$\sigma_{c_1 \wedge c_2}(R) = \sigma_{c_1}(\sigma_{c_2}(R))$$

This transformation is called **cascading of select operator**.

Rule 2: Select operations are **commutative**. That is,

$$\sigma_{c_1}(\sigma_{c_2}(R)) = \sigma_{c_2}(\sigma_{c_1}(R))$$

Rule 3: If a query contains a sequence of project operations, only the final operation is needed, the others can be omitted. That is,

$$\Pi_{A_1}(\Pi_{A_2}(\dots(\Pi_{A_n}(R))\dots)) = \Pi_{A_1}(R)$$

This transformation is called **cascading of project operator**.

Rule 4: If the selection condition C involves only the attributes $A_1, A_2, \dots, A_n$ that are present in the projection list, the two operations can be commuted. That is,

$$\Pi_{A_1, A_2, \dots, A_n}(\sigma_C(R)) = \sigma_C(\Pi_{A_1, A_2, \dots, A_n}(R))$$

Rule 5: Selections can be combined with Cartesian products and joins. That is,

$$(a) \sigma_C(R_1 \times R_2) = R_1 \bowtie_C R_2$$

$$(b) \sigma_{c_1}(R_1 \bowtie_{c_2} R_2) = R_1 \bowtie_{c_1 \wedge c_2} R_2$$

Rule 6: Cartesian product and join operations are **commutative**. That is,

$$(a) R_1 \times R_2 = R_2 \times R_1$$

$$(b) R_1 \bowtie_C R_2 = R_2 \bowtie_C R_1$$

Note that the ordering of the attributes in the results of left-hand side and right-hand side expressions will be different. Thus, if the ordering is to be taken into account, the equivalence does not hold. In order to appropriately reorder the attributes, a project operator can be added to either of the side of equivalence. For simplicity, project operator is omitted and the attribute order is ignored in most of our examples.

Rule 7: Cartesian product and join operations are **associative**. That is,

$$(a) (R_1 \times R_2) \times R_3 = R_1 \times (R_2 \times R_3)$$

$$(b) (R_1 \bowtie R_2) \bowtie R_3 = R_1 \bowtie (R_2 \bowtie R_3)$$

Rule 8: The select operation distributes over the join operation in these two conditions.

(a) If all the attributes in the selection condition C_1 involve only the attributes of one of the relations (say, R_1), then the select operation distributes. That is,

$$\sigma_{c_1}(R_1 \bowtie_C R_2) = (\sigma_{c_1}(R_1)) \bowtie_C R_2$$

- (b) If the selection condition C_1 involves only the attributes of R_1 and C_2 involves the attributes of R_2 , then the select operation distributes. That is,

$$\sigma_{C_1 \wedge C_2}(R_1 \bowtie_c R_2) = (\sigma_{C_1}(R_1)) \bowtie_c (\sigma_{C_2}(R_2))$$

Rule 9: The project operation distributes over the join operation in these two conditions.

- (a) If the join condition C involves only the attributes in $A_1 \cup A_2$, where A_1 and A_2 are the attributes of R_1 and R_2 , respectively, the project operation distributes. That is,

$$\pi_{A_1 \cup A_2}(R_1 \bowtie_c R_2) = (\pi_{A_1}(R_1)) \bowtie_c (\pi_{A_2}(R_2))$$

- (b) If the attributes A_3 and A_4 of R_1 and R_2 , respectively, are involved in the join condition C , but not in $A_1 \cup A_2$, then,

$$\pi_{A_1 \cup A_2}(R_1 \bowtie_c R_2) = \pi_{A_1 \cup A_2}((\pi_{A_1 \cup A_3}(R_1)) \bowtie_c (\pi_{A_2 \cup A_4}(R_2)))$$

Rule 10: The set operations union and intersection are commutative and associative. That is,

- (a) Commutative

$$\begin{aligned} R_1 \cup R_2 &= R_2 \cup R_1 \\ R_1 \cap R_2 &= R_2 \cap R_1 \end{aligned}$$

- (b) Associative

$$\begin{aligned} (R_1 \cup R_2) \cup R_3 &= R_1 \cup (R_2 \cup R_3) \\ (R_1 \cap R_2) \cap R_3 &= R_1 \cap (R_2 \cap R_3) \end{aligned}$$



The set difference operation is neither commutative nor associative.

Rule 11: The select operation distributes over the union, intersection, and set difference operations. That is,

$$\begin{aligned} \sigma_c(R_1 \cup R_2) &= \sigma_c(R_1) \cup \sigma_c(R_2) \\ \sigma_c(R_1 \cap R_2) &= \sigma_c(R_1) \cap \sigma_c(R_2) \\ \sigma_c(R_1 - R_2) &= \sigma_c(R_1) - \sigma_c(R_2) \end{aligned}$$

Rule 12: The project operation distributes over the union operation. That is,

$$\pi_A(R_1 \cup R_2) = \pi_A(R_1) \cup \pi_A(R_2)$$



The equivalence rules only state that one expression is equivalent to another, and not that one expression is better than the other.

Note that some of the equivalence rules given earlier can be derived from the combination of other rules. For example, rule 8(b) can be derived from rule 1 and rule 8(a). A set of equivalence rules is said to be **minimal** if no rule can be derived from the others. Thus, this set of equivalence rules is not a minimal set. If a non-minimal set of rules is used for finding an equivalent expression, the number

of different ways of generating an expression increases. A query optimizer, therefore, uses a minimal set of equivalence rules.

To better understand these equivalence rules, consider some expressions and their equivalence expressions based on *Online Book* database.

Expression 1: $\sigma_{\text{BOOK.P_ID}=\text{PUBLISHER.P_ID}}(\text{BOOK} \bowtie \text{PUBLISHER})$

Its equivalent expression is $\text{BOOK} \bowtie_{\text{BOOK.P_ID} = \text{PUBLISHER.P_ID}} \text{PUBLISHER}$ (rule 5(a))

Expression 2: $\pi_{\text{Aname, URL, Category}}(\sigma_{\text{Book_title}=\text{"C++"}}(\text{BOOK} \bowtie (\text{AUTHOR_BOOK} \bowtie \text{AUTHOR})))$

Its equivalent expression is $\pi_{\text{Aname, URL, Category}}((\sigma_{\text{Book_title}=\text{"C++"}}(\text{BOOK})) \bowtie (\text{AUTHOR_BOOK} \bowtie \text{AUTHOR}))$ (rule 8(a))

Expression 3: $\pi_{\text{Book_title, Aname, Rating}}(\sigma_{\text{Category}=\text{"Textbook"} \wedge \text{Rating}>7}(\text{BOOK} \bowtie (\text{REVIEW} \bowtie \text{AUTHOR})))$

To find the equivalent expression of this expression, multiple equivalence rules have to be applied one after the other on the query or its subparts. The initial expression tree for this expression is given in Figure 8.14(a).

Step 1: Apply rule 7 (associativity of joins) to generate the following expression.

$\pi_{\text{Book_title, Aname, Rating}}(\sigma_{\text{Category}=\text{"Textbook"} \wedge \text{Rating}>7}((\text{BOOK} \bowtie \text{REVIEW}) \bowtie \text{AUTHOR}))$

Step 2: Since the selection condition involves attributes of both **BOOK** and **REVIEW** relation, rule 8(a) can be applied on the expression obtained after Step 1. The expression now becomes

$\pi_{\text{Book_title, Aname, Rating}}((\sigma_{\text{Category}=\text{"Textbook"} \wedge \text{Rating}>7}(\text{BOOK} \bowtie \text{REVIEW})) \bowtie \text{AUTHOR})$

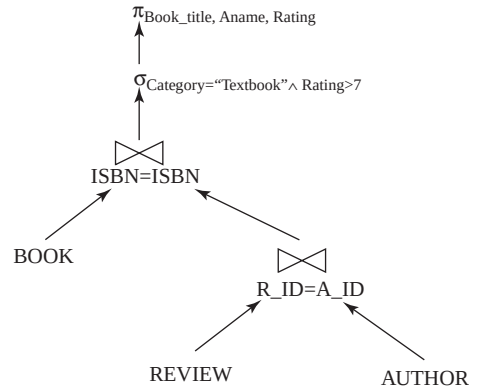
Step 3: Since, the selection condition 1 involves the attribute of relation **BOOK** and selection condition 2 involves the attribute of relation **REVIEW**, the select operation can be distributed over the join operation using rule 8(b). The expression now becomes

$\pi_{\text{Book_title, Aname, Rating}}((\sigma_{\text{Category}=\text{"Textbook"}}(\text{BOOK})) \bowtie_{\sigma_{\text{Rating}>7}} (\text{REVIEW})) \bowtie \text{AUTHOR})$

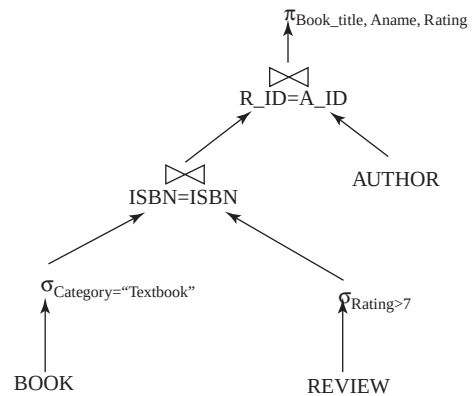
The final query tree of the equivalent expression is given in Figure 8.14(b).

Expression 4: $\pi_{\text{Book_title, Pname}}(\text{BOOK} \bowtie \text{PUBLISHER})$

In this expression, the attributes involved in the project operation are **Book_title** and **Pname** of relation **BOOK** and **PUBLISHER**, respectively.



(a) Initial expression tree



(b) Final expression tree

Fig. 8.14 An example of performing multiple transformations

Therefore, the project operation can be distributed over the join using rule 9(a). Moreover, the only attribute involved in join operation is P_ID of both the relations. Thus, rule 9(b) can also be applied on this expression in order to further reduce the size of the intermediate relation. The expression now becomes

$$\pi_{Book_title, Pname}((\pi_{Book_title, P_ID}(BOOK)) \bowtie (\pi_{P_ID, Pname}(PUBLISHER)))$$

It is clear from this example that the extraneous attributes can be eliminated before applying join or select operations to reduce the size of intermediate relation.

Join Ordering

Another way to reduce the size of intermediate results is to choose an optimal ordering of join operations. Thus, most of the query optimizers pay a lot of attention to the proper ordering of join operations. As discussed earlier, the join operation is commutative and associative in nature. That is,

$$R_1 \bowtie R_2 = R_2 \bowtie R_1$$

$$(R_1 \bowtie R_2) \bowtie R_3 = R_1 \bowtie (R_2 \bowtie R_3)$$

Though the expressions on the left-hand side and right-hand side in the second equation are equivalent, the cost of evaluating these two expressions may be different. Thus, it is the responsibility of the query optimizer to choose an optimal join order with the minimal cost. For example, consider a join operation on three relations R_1 , R_2 , and R_3 .

$$R_1 \bowtie R_2 \bowtie R_3$$

Since the join operation is associative and commutative in nature, these three relations can be joined in 12 different ways as given here.

$$\begin{array}{l} R_1 \bowtie (R_2 \bowtie R_3) \quad R_1 \bowtie (R_3 \bowtie R_2) \quad (R_2 \bowtie R_3) \bowtie R_1 \quad (R_3 \bowtie R_2) \bowtie R_1 \\ R_2 \bowtie (R_1 \bowtie R_3) \quad R_2 \bowtie (R_3 \bowtie R_1) \quad (R_1 \bowtie R_3) \bowtie R_2 \quad (R_3 \bowtie R_1) \bowtie R_2 \\ R_3 \bowtie (R_1 \bowtie R_2) \quad R_3 \bowtie (R_2 \bowtie R_1) \quad (R_1 \bowtie R_2) \bowtie R_3 \quad (R_2 \bowtie R_1) \bowtie R_3 \end{array}$$

In general, for n relations, there are $(2(n-1))! / (n-1)!$ different join orderings. For example, for $n=5$, the join orderings are 1680, for $n=6$, the join orderings are 30240. As n increases, the number of join orderings also increases manifolds. However, generating such a large number of equivalent expressions for a given expression is not feasible.

The query optimizer, therefore, generates only a subset of equivalent expressions. For example, for the join operation $(R_1 \bowtie R_2 \bowtie R_3) \bowtie R_4 \bowtie R_5$, it is not necessary to generate 1680 expressions. First the different join orders of $R_1 \bowtie R_2 \bowtie R_3$ are generated and the best join order with the least cost can be found. Then, the resultant intermediate relation say, R_1 generated from the join of relations $\{R_1, R_2, R_3\}$ is joined with the remaining two relations, that is, $R_1 \bowtie R_4 \bowtie R_5$. Thus, instead of 1680 expressions, only $12 + 12 = 24$ expressions need to be examined.

In addition to finding the optimal join ordering, it is also necessary to find the order in which the tuples are generated by the join of several relations as it may affect the cost of further joins. If any particular sorted order of the tuples is useful for some operation later on, it is known as **interesting sort order**. For example, generating the tuples of $R_1 \bowtie R_2 \bowtie R_3$ sorted on the attribute common with R_4 or R_5 is more useful than generating tuples sorted on the attributes common to only R_1 and R_2 . Thus, the goal is to find the best join order for each subset, for each interesting sort order of the join result for that subset.

To understand the need of proper join ordering, consider the query “Retrieve the publisher name, rating, and the name of the authors who have rated the C++ book”. The corresponding relation algebra expression for this query is

$$\Pi_{Pname, Rating, Aname} ((\sigma_{Book_title="C++"}(BOOK)) \bowtie PUBLISHER \bowtie REVIEW \bowtie AUTHOR))$$

In this query, first the selection is performed on the **BOOK** relation. This will generate a set of tuples with book title **C++**. If the operation **PUBLISHER** $\bowtie$ **REVIEW** is performed first, it results in the Cartesian product of these two relations as no common attributes exist between **PUBLISHER** and **REVIEW**, which results in a large intermediate relation. As a result, this strategy is rejected. Similarly, the operation **PUBLISHER** $\bowtie$ **AUTHOR** is also rejected for the same reason.

If the operation **REVIEW** $\bowtie$ **AUTHOR** is performed, it also results in a large intermediate relation but the user is only interested in the authors who have reviewed the **C++** book. Thus, this order of evaluating join operation is also rejected. However, if the result obtained after the select operation on **BOOK** relation is joined with the **PUBLISHER**, which in turn joined with **REVIEW** relation, and finally with the **AUTHOR** relation, the number of tuples is reduced as shown in Figure 8.15. Thus, the equivalent final expression is

$$\Pi_{Pname, Rating, Aname} (((\sigma_{Book_title="C++"}(BOOK)) \bowtie PUBLISHER) \bowtie REVIEW) \bowtie AUTHOR))$$

$$\sigma_{Book_title="C++"}(Book)$$

ISBN	Book_title	Category	Price	Copyright_date	Year	Page_count	P_ID
001-987-760-9	C++	Textbook	25	2004	2005	800	P001

$$(\sigma_{Book_title="C++"}(Book)) \bowtie PUBLISHER$$

ISBN	Book_title	...	P_ID	Pname	...	Email_ID
001-987-760-9	C++	...	P001	Hills publications	...	h_pub@hills.com

$$((\sigma_{Book_title="C++"}(Book)) \bowtie PUBLISHER) \bowtie REVIEW$$

ISBN	Book_title	...	P_ID	Pname	...	R_ID	Rating
001-987-760-9	C++	...	P001	Hills Publications	...	A002	6.0
001-987-760-9	C++	...	P001	Hills Publications	...	A008	7.2

$$(((\sigma_{Book_title="C++"}(Book)) \bowtie PUBLISHER) \bowtie REVIEW) \bowtie AUTHOR$$

ISBN	Book_title	...	P_ID	Pname	...	R_ID	Rating	Aname	...
001-987-760-9	C++	...	P001	Hills Publications	...	A002	6.0	James Erin	...
001-987-760-9	C++	...	P001	Hills Publications	...	A008	7.2	Annie George	...

$$\Pi_{Pname, Rating, Aname} (((\sigma_{Book_title="C++"}(Book)) \bowtie PUBLISHER) \bowtie REVIEW) \bowtie AUTHOR))$$

Final
Result

Pname	Rating	Aname
Hills publications	60	James Erin
Hills publications	72	Annie George

Fig. 8.15 An example of join ordering

Converting Query Trees into Query-Evaluation Plans

Generating only a set of logically equivalent expressions is not enough. A query-evaluation plan is also needed that defines exactly which algorithm should be used for each operation in the query. A query-evaluation plan can be generated by annotating each operation in the query tree with the instructions that specify the algorithm to be used to evaluate the operation. Since each relational algebra operation can be implemented with different algorithms (discussed in Section 8.3), a number of query-evaluation plans can be generated. One such possible query-evaluation plan for the expression given in Figure 8.14(b) is given in Figure 8.16.

Estimating Statistics

The statistics are used to estimate the size of intermediate results. Some of the statistics about database relations is stored in DBMS catalog, which include:

- ⇒ the number of tuples in relation R , denoted by n_R .
- ⇒ the number of blocks containing the tuples of relation R , denoted by b_R .
- ⇒ the blocking factor of relation R , denoted by f_R is the number of tuples that fit in one block.
- ⇒ the size of each tuple of relation R in bytes, denoted by s_R .
- ⇒ the number of distinct values appearing in relation R for the attribute A , denoted by $V(A, R)$. This value is same as the size of $\pi_A(R)$. If A is a key attribute, then $V(A, R) = n_R$.
- ⇒ height of index i , that is, number of levels in i , denoted by H_i .

Assuming that the tuples of relation R are stored together physically in a file, then

$$b_R = \lceil n_R / f_R \rceil$$

Every time a relation is modified, the statistics must also be updated. Updating the statistics incurs a substantial amount of overhead, thus, most of the systems update the statistics in the period of light system load—rather than updating it on every update.

In addition to the relation sizes and indexes height, real-world query optimizers also maintain some other statistical information to increase the accuracy of cost estimates. For example, most databases maintain **histograms** representing the distribution of values for each attribute. The values for the attribute are divided into a number of ranges, and the number of tuples whose attribute value falls in a particular range is associated with that range in the histogram. For example, the histograms for the attributes **Price** and **Page_count** of the **BOOK** relation are shown in Figure 8.17.

A histogram takes very little space so histograms on various attributes can be maintained in the system catalog. A histogram can be of two types, namely, *equi-width histogram* and *equi-depth histogram*. In **equi-width histograms**, the range of values is divided into equal-sized subranges. On the other hand, in **equi-depth histograms**, the sub ranges are chosen in such a way that the number of tuples within each sub range is equal.

Size Estimation for Select Operations The estimated size (number of tuples returned) of the result of a select operation depends on the selection condition.

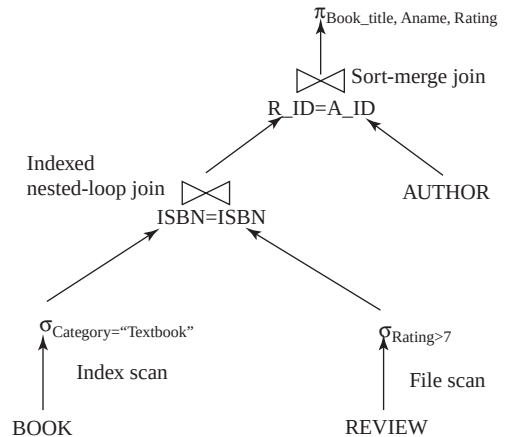


Fig. 8.16 Query-evaluation plan

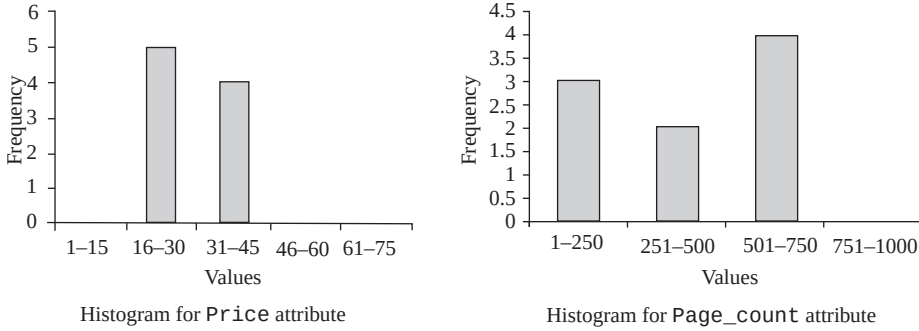


Fig. 8.17 Histograms on *Price* and *Page_count*

- ⇒ For the select operation of the form $\sigma_{A=a}(R)$, the result can be estimated to have $n_R/V(A, R)$ tuples, assuming that the values are distributed uniformly (that is, each value appears with equal probability). For example, consider a relational algebra expression $\sigma_{\text{Page_count}=200}(\text{BOOK})$. The **BOOK** relation contains 9 tuples (n_R) and number of distinct values for the attribute **Page_count** [$V(A, R)$] is 6 (that is, 200, 450, 800, 500, 650, 550). Therefore, the result of this select operation is estimated to have $9/6 = 1.5$ tuples.

Note that the assumption that the values are distributed uniformly is not always true as it is unrealistic to assume that each value appears with equal probability. However, despite the fact that the uniform-distribution assumption is generally not correct, this method provides good approximation in many cases and keeps our presentation relatively simple.

If a histogram is available for the attribute *A*, the number of tuples can be estimated with more accuracy. The range in which the value *a* belongs is first located in the histogram. The variable n_R is replaced with the frequency count and $V(A, R)$ is replaced with the number of distinct values appearing in that range. For example, the histogram for the attribute **Page_count** shows that the select operation $\sigma_{\text{Page_count}=200}(\text{BOOK})$ is estimated to have $3/1 = 3$ tuples because the frequency for the range 1–200 is 3 and the number of distinct values appearing in this range is only 1 (that is, 200).

- ⇒ For the select operation of the form $\sigma_{A \leq a}(R)$, the estimated number of tuples that will satisfy the condition $A \leq a$ is 0 if $a < \min(A, R)$, is n_R if $a \geq \max(A, R)$ and

$$n_R \cdot \frac{a - \min(A, R)}{\max(A, R) - \min(A, R)}$$

otherwise. Here, $\min(A, R)$ and $\max(A, R)$ is the minimum and maximum values for the attribute *A*. These values can be stored in the catalog. In this case also, if a histogram is available, more accurate estimate can be obtained.

- ⇒ For the select operation of the form $\sigma_{A > a}(R)$ the result can be estimated to have

$$n_R \cdot \frac{\max(A, R) - a}{\max(A, R) - \min(A, R)}$$

tuples.

- ⇒ For the conjunctive selection of the form $\sigma_{c_1 \wedge c_2 \wedge \dots \wedge c_n}(R)$, the size s_i of each selection condition c_i can be estimated as described earlier. Thus, the probability that a tuple satisfies selection condition c_i is s_i/n_R . This probability is called the **selectivity** of the selection $\sigma_{c_i}(R)$. Therefore, the number of tuples in the complete conjunctive selection is

$$n_R * \frac{s_1 * s_2 * \dots * s_n}{(n_R)^n}$$

where, the term

$$\frac{s_1 * s_2 * \dots * s_n}{(n_R)^n}$$

represents the probability that a tuple satisfies all the conditions. It is simply the product of all the probabilities (assuming that the conditions are independent of each other).

- ⇒ For the disjunctive selection of the form $\sigma_{c_1 \vee c_2 \vee \dots \vee c_n}(R)$, the probability that a tuple satisfies all the disjunction conditions is 1 minus the probability that it will satisfy none of the conditions. That is,

$$1 - \left(1 - \frac{s_1}{n_R}\right) * \left(1 - \frac{s_2}{n_R}\right) * \dots * \left(1 - \frac{s_n}{n_R}\right)$$

The number of tuples in the complete disjunctive selection is estimated by using the following equation.

$$n_R * \left[1 - \left(1 - \frac{s_1}{n_R}\right) * \left(1 - \frac{s_2}{n_R}\right) * \dots * \left(1 - \frac{s_n}{n_R}\right)\right]$$

- ⇒ For the negation selection of the form $\sigma_{\neg c}(R)$, the estimated number of tuples in the absence of *null* values is

$$n_R - \text{size}(\sigma_c(R))$$

If *null* values are accounted, then the estimated number of tuples can be calculated by first estimating the number of tuples for which the condition C would evaluate to *unknown*. This number is then subtracted from the estimate calculated earlier that ignore *null* values.

Size Estimation for Join Operations Estimating the size of a natural join operation is more complicated than that of the selection operation. Let R_A and S_A denote all the attributes of relation R and S , respectively.

- ⇒ If $R_A \cap S_A = \phi$, that is, the relations have no common attributes, then the size estimate of the operation $R \bowtie S$ is same as that of $R \times S$.
- ⇒ If $R_A \cap S_A$ forms the key of R and foreign key of relation S , then the number of tuples returned by the operation $R \bowtie S$ is exactly the same as the number of tuples in S , that is, n_S . However, if $R_A \cap S_A$ forms the key of S and foreign key of relation R , then the number of tuples returned by the operation $R \bowtie S$ is exactly the same as the number of tuples in R , that is, n_R .
- ⇒ If $R_A \cap S_A$ neither forms the key of R nor S , the size estimate is most difficult to find. Assuming that each value appears with equal probability, the estimated number of tuples produced by a tuple t of R in $R \bowtie S$ is

$$\frac{n_s}{V(A, S)}$$

Therefore, the estimated number of tuples produced by all the tuples of R in $R \bowtie S$ is

$$\frac{n_R * n_S}{V(A, S)}$$

If the roles of both the relations are reversed in the preceding estimate, the size estimate becomes

$$\frac{n_R * n_S}{V(A, R)}$$

If $V(A, R) \neq V(A, S)$, then the cost estimates of both the equations differ. Then the lower value of the two estimates obtained is probably the more accurate one. If histograms are available on the join attributes, more accurate estimate can be obtained.

For example, consider a join operation $BOOK \bowtie PUBLISHER$. Since the attribute P_ID of $PUBLISHER$ relation is the foreign key in the $BOOK$ relation, then the number of tuples obtained is exactly n_{BOOK} , which is 5000.

Size Estimation for Project Operations As discussed in Chapter 04, the project operation gives result after removing duplicates. Thus, the estimated size of the project operation of the form $\pi_A(R)$ is $V(A, R)$, where $V(A, R)$ is the number of distinct values appearing in relation R for the attribute A . For example, the operation $\pi_{Category, Page_count}(BOOK)$ returns seven tuples for the relation $BOOK$ as shown in Figure 8.18. Since the project operation removes duplicate tuples from the resultant relation, the tuples (*Textbook*, 800) and (*Novel*, 200) appear only once in the resultant relation, though the combination of these values appears repeatedly in $BOOK$ relation.

Size Estimation for Set Operations If the inputs to the set operations union, intersection, and set difference are the selections on the same relation say R , these set operations can be rewritten as disjunction, conjunction, and negation, respectively. For example, the set operation $\sigma_{c1}(R) \cup \sigma_{c2}(R)$ can be rewritten as $\sigma_{c1 \vee c2}(R)$ and $\sigma_{c1}(R) \cap \sigma_{c2}(R)$ can be rewritten as $\sigma_{c1 \wedge c2}(R)$. Since, these operations are now disjunctive and conjunctive selections, respectively, their sizes can be estimated as described earlier.

However, if the two inputs are not the selections on the same relation, the estimated size of $R \cup S$ is simply the sum of sizes of R and S , provided both the relations do not contain any common tuples. If some of the tuples are common then this estimate only gives the upper bounds on the size. Similarly, for the operation $R \cap S$, the estimated size is the minimum of the sizes of R and S , provided that the smaller relation say, S contains all the tuples appearing in R . If some of the tuples of S do not appear in R then this estimate also gives the upper bound on the size. For the operation $R - S$, the estimated size is same as the size of R , provided no tuple of R appears in S . If some of tuples of R appear in S , then this estimate gives only the upper bound on the size.

Category	Page_count
Novel	200
Textbook	450
Textbook	800
Language Book	200
Language Book	500
Textbook	650
Textbook	550

Fig. 8.18 Result of the operation $\pi_{Category, Page_count}(BOOK)$

Size Estimation for Aggregate Operations. The estimated size of the aggregate operation of the form $\mathcal{F}_A(R)$ is $V(A, R)$ because for each distinct value of A , there is only one tuple returned by the aggregate operation. Here, A is the attribute of relation R on which the aggregate function F is to be applied.



NOTE *The estimated statistics coupled with the cost estimates for various algorithms discussed in Section 8.3 give the estimated cost of a query-evaluation plan. The least-costly query-evaluation plan is finally chosen for the execution.*

8.5.2 Heuristic Query Optimization

In cost-based optimization, generating a large number of query-evaluation plans for a given query, and finding the cost of each plan can be very time-consuming and expensive. Thus, finding the optimal plan from such a large number of plans requires a lot of computational effort. To reduce this cost and computational effort, optimizers apply some heuristics (rules) to find the optimized query-evaluation plan. These rules are known as **heuristics** as they usually (but not always) help in reducing the cost of execution of a query. The heuristic query optimizers simply apply these rules without finding out whether the cost is reduced by this transformation.

Generally, many different relational algebra expressions and hence, many different query trees can be created for the same query. The query parser; however, usually generates a standard initial query tree that corresponds to an SQL query without any optimization. The heuristic query optimizer then transforms this initial (canonical) query tree into the final, optimized query tree, which is more efficient to execute. However, the heuristic optimizer must make sure that the transformation steps always result in an equivalent query tree. For this, the query optimizer must be aware of the equivalence rules that preserve this equivalence. In general, the heuristic query optimizer follows these steps to transform initial query tree into an optimal query tree.

1. Conjunctive selection operations are transformed into cascading selection operations (equivalence rule 1).
2. Since the select operation is commutative with other operations according to the equivalence rules 2, 4, 8, and 11, it is moved as far down the query tree as permitted by the attributes involved in the selection condition. That is, select operation is performed as early as possible to reduce the number of tuples returned.
3. The select and join operations that are more restrictive (result in relations with the fewest tuples) are executed first. That is, the select and join operation are rearranged so that they are accomplished with the least amount of system overhead. This is done by reordering the leaf nodes of the tree among themselves by applying rule 6, 7, and 10 (commutativity and associativity of binary operations).
4. Any Cartesian product operation followed by a select operation is combined into a single join operation according to the equivalence rule 5(a).
5. The cascaded project operation is broken down and the attributes in the projection list are moved down the query tree as far as possible. New project operation can also be created as required. The attributes that are required in the query result and in the subsequent operations should only be kept after each project operation. This will eliminate the return of unnecessary attributes (rules 3, 4, 9, and 12).
6. The sub trees that represent groups of operations that can be executed by a single algorithm are identified.

To understand the use of heuristics in query optimization, consider these relational algebra expressions.

Expression 5: $\Pi_{Pname, Address} (\sigma_{Category="Novel"} (BOOK \bowtie PUBLISHER))$

The initial query tree of this expression is given in Figure 8.19(a). After applying the select operation on the **BOOK** relation early, the number of tuples can be reduced. Similarly, after applying the project operation to extract the desired attributes **P_ID**, **Pname** and **Address** from the **PUBLISHER** relation, and **P_ID** from the result of σ on **BOOK** relation (**P_ID** is the join attribute), the size of the intermediate result can be reduced further. The final expression now becomes

$$\pi_{Pname, Address} \left(\left(\pi_{P_ID} \left(\sigma_{Category="Novel"} (BOOK) \right) \right) \bowtie \left(\pi_{P_ID, Pname, Address} (PUBLISHER) \right) \right)$$

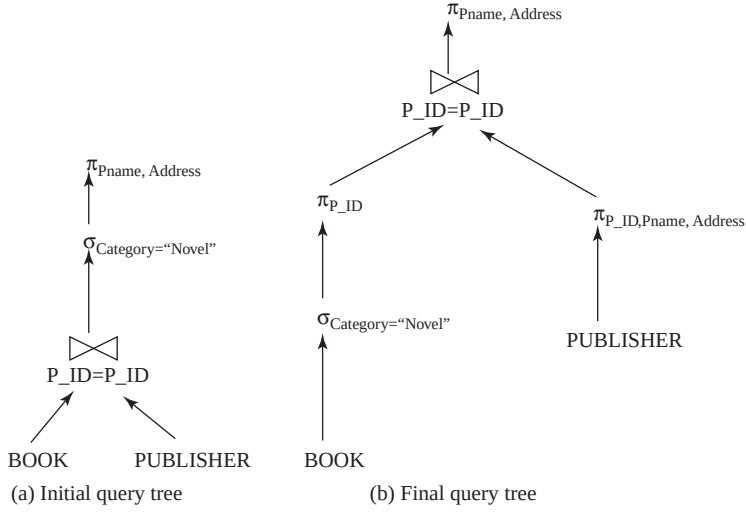


Fig. 8.19 Query trees for expression 5

The transformed query tree of the final expression after applying the select and project operations earlier is given in Figure 8.19(b).

Expression 6: $\pi_{Book_title, Rating} \left(\sigma_{Rating > 7 \wedge P_ID = "P002"} (REVIEW \bowtie BOOK \bowtie PUBLISHER) \right)$

The initial query tree of this expression is given in Figure 8.20(a). Since the selection condition involves conjunctive condition, it can be transformed into a cascade of select operations (heuristic rule 1). This gives a greater degree of freedom in moving the select operations down the different branches of the query tree as shown in Figure 8.20(b). In addition, after interchanging the position of PUBLISHER and REVIEW relation, the number of tuples can be further reduced as the select operation on PUBLISHER relation involves primary key which returns only one tuple (heuristic rule 3). The final expression now becomes

$$\pi_{Book_title, Rating} \left(\left(\sigma_{P_ID = "P002"} (PUBLISHER) \right) \bowtie (BOOK) \bowtie \left(\sigma_{Rating > 7} (REVIEW) \right) \right)$$

The final query tree for this expression is given in Figure 8.20(c).

Expression 7: $\pi_{Book_title, Pname} \left(\sigma_{BOOK.P_ID = PUBLISHER.P_ID} (BOOK \times PUBLISHER) \right)$

The initial query tree of this expression is given in Figure 8.21(a). Since the condition involved in select operation is the join condition, the Cartesian product can be replaced by the join operation. The final expression now becomes

$$\pi_{Book_title, Pname} \left((BOOK \bowtie_{BOOK.P_ID = PUBLISHER.P_ID} PUBLISHER) \right)$$

The final query tree after replacing the Cartesian product with the join operation is given in Figure 8.21(b).

Many real-life query optimizers have additional heuristics such as restricting the search to some particular kinds of join orders, rather than considering all join orders to further reduce the cost of optimization. For example, System R optimizer considers only those join orders in which the right operand of each join is one of the base relations $R_1, R_2, \dots, R_n$ defined in DBMS catalog. Such join

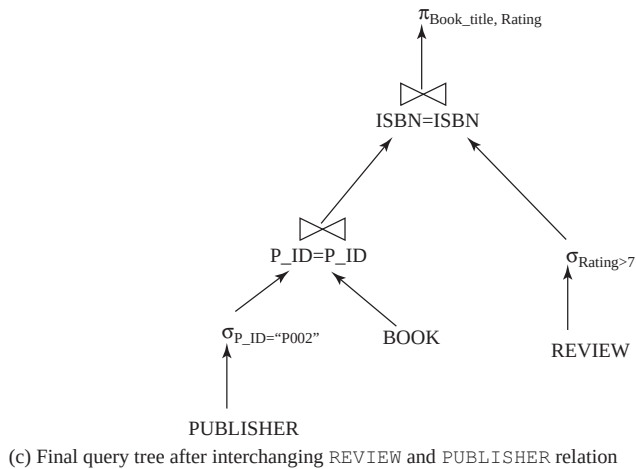
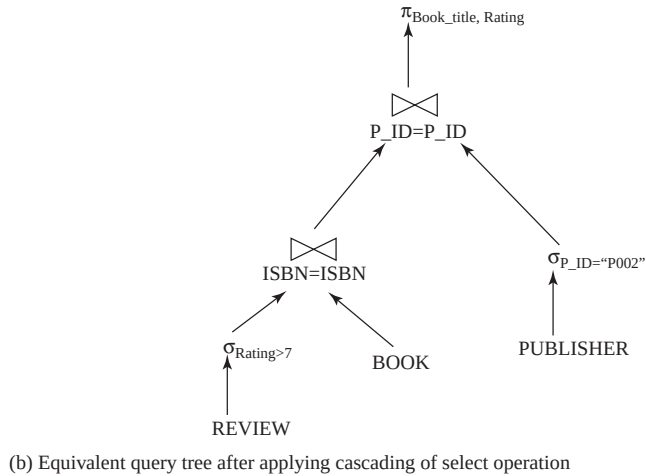
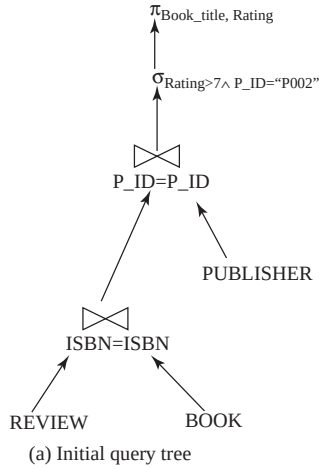


Fig. 8.20 Query trees for expression 6

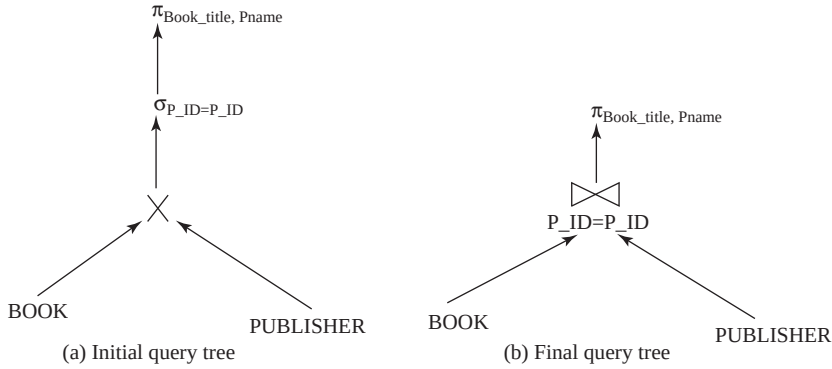


Fig. 8.21 Query trees for expression 7

orders are called **left-deep join orders** and the corresponding query trees are called **left-deep join trees**. An example of left-deep and non-left-deep join tree is given in Figure 8.22.

The right child in left-deep trees is considered to be the inner relation when executing a nested-loop join. The main advantage of left-deep join tree is that since one of the inputs of each join is the base relation, it allows the optimizer to utilize any access path on that relation. This may speed up the

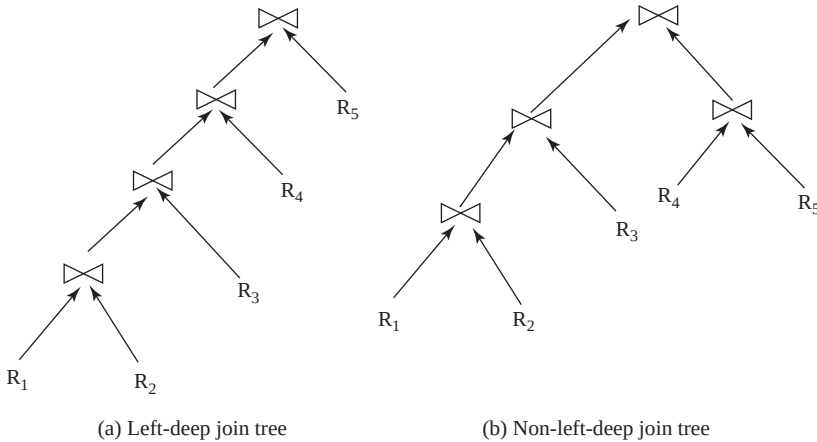


Fig. 8.22 Left and non-left deep join trees

execution of the join operation. Moreover, since, the right operand is always the base relation, only one input to each join needs to be pipelined. Thus, they are more convenient for pipelined evaluation.

8.5.3 Semantic Query Optimization

Conventional query optimization techniques attempt to determine a most efficient query-evaluation plan, based on the syntax of a particular query and the access methods available. On the other hand, semantic optimization uses available semantic knowledge to transform a query into a new query which produces the same answer as the original query in any legal database state; however, requires less system resources to execute.

The semantic query optimization is a different approach that is currently the focus of considerable research. This technique operates on the key idea that the optimizer has a basic understanding of the constraints such as unique, check, and other complex constraints specified on the database schema. When a query is submitted, the optimizer uses its knowledge of system constraints to modify the given query into another query that is more efficient to execute or to simply ignore a particular query if it is guaranteed to return an empty result set. For example, consider this SQL query.

```
SELECT Book_title, Page_count, Price
FROM BOOK
WHERE Copyright_date>Year;
```

This query retrieves the title, price, and page count of the books whose copyright-date is greater than the publishing year. As discussed in Chapter 03, the constraint that the copyright-date of a book is always less than its publishing year is specified on the **BOOK** relation. Thus, if the semantic query optimizer checks for the existence of this constraint, the query need not be executed as the query optimizer now has the knowledge that the result of the query will be empty. Hence, this technique may save a considerable time if the database catalog is checked efficiently for the constraint. However, searching the catalog for the constraints that are applicable to the given query and that may semantically optimize it can be quite time-consuming.

8.6 QUERY OPTIMIZATION IN ORACLE

In the early versions of Oracle, only rule-based query optimization was used. However, in later versions of Oracle7 and all versions of Oracle8, both rule-based and cost-based query optimizers are available. This section first discusses both types of query optimizers and then it discusses some guidelines for writing a query in Oracle.

8.6.1 Rule-based Query Optimizer

The rule-based query optimizer uses a set of 15 heuristically ranked access paths to determine the best execution plan for a query. These ranked access paths are maintained in a table, where a lower ranking implies more efficient approach and higher ranking implies lesser efficient approach. The access paths are listed in the order of their ranks in Table 8.1.

The main advantage of the rule-based optimizer is that the rule chosen is based strictly on the query structure and not on the number of tuples, partitioning or other statistics. Thus, for a given query, the execution plan is stable irrespective of the size of the relation and its related indexes. However, its main disadvantage is that the execution plan never changes despite of increase in the size of relation and its indexes.

8.6.2 Cost-based Query Optimizer

The cost-based query optimizer first generates the logically equivalent query-evaluation plans for a given query, and then determines the statistics of the relations involved in the query. These statistics are used to determine the cost of each step involved in the query-evaluation plan. Finally, the query-evaluation plan with the minimal cost is chosen for the execution of the given query. The cost-based algorithm depends entirely on the accuracy of table statistics for determining the accurate cost information. The main advantage of the cost-based optimizer is that depending on the

Learn More

While calculating the cost of query-evaluation plan, the ORACLE query optimizer takes into consideration the estimated usage of different resources such as, CPU, I/O, memory, etc.

Table 8.1 *List of heuristically ranked access paths*

Rank	Access Path
1	single row by ROWID
2	single row by cluster join
3	single row by hash cluster key with unique or primary key
4	single row by unique or primary key
5	cluster join
6	hash cluster key
7	indexed cluster key
8	composite Index
9	single column index
10	bounded range search on indexed columns
11	unbounded range search on indexed columns
12	sort-merge join
13	MAX or MIN on indexed column
14	ORDER BY indexed columns
15	complete table scan

relation and index sizes the optimizer can re-evaluate the execution plan to find the optimal execution plan.

The ANALYZE command is used for creating and maintaining the table statistics. The syntax of ANALYZE command is

```
ANALYZE [TABLE <table_name> | INDEX <table_name>]
COMPUTE STATISTICS |
ESTIMATE STATISTICS [SAMPLE <integer> ROWS | PERCENT] |
DELETE STATISTICS;
```

where,

COMPUTE STATISTICS examines all the tuples in the table to compute the exact value of statistics

ESTIMATE STATISTICS examines specified number of tuples (by default 1064 tuples) to compute the estimated size

SAMPLE is the number or percentage of rows to be used for sampling

DELETE STATISTICS deletes the statistics of the table

To understand the use of ANALYZE command, consider these examples.

- ⇒ ANALYZE TABLE BOOK ESTIMATE STATISTICS SAMPLE 50 PERCENT;
- ⇒ ANALYZE TABLE REVIEW COMPUTE STATISTICS;
- ⇒ ANALYZE TABLE PUBLISHER ESTIMATE STATISTICS SAMPLE 1000 ROWS;
- ⇒ ANALYZE TABLE BOOK DELETE STATISTICS;



NOTE The Oracle9i and later versions prefer the package **DBMS_STATS** instead of **ANALYZE** command to collect and maintain the statistics.

Once the query optimizer chooses the optimal query-execution plan, the `EXPLAIN PLAN` command is used to generate and store information about the plan chosen by the query optimizer. This information about the query-execution plan is stored in a table called `PLAN_TABLE`. Note that the `EXPLAIN PLAN` command gives information for `SELECT`, `UPDATE`, `DELETE`, and `INSERT` statements only.

The syntax of the `EXPLAIN PLAN` command is

```
EXPLAIN PLAN SET STATEMENT_ID = '<name>'
```

```
FOR <SQL statement>
```

where,

`STATEMENT_ID` is a unique ID given to every execution plan—it is just a simple string
`SQL statement` is just a normal select statement

For example, to generate an execution plan for the query “Retrieve the title and price of all the books published by Hills Publications”, the following command can be given.

```
EXPLAIN PLAN SET STATEMENT_ID = 'bookplan'
FOR SELECT Book_title, Price
FROM BOOK, PUBLISHER
WHERE BOOK.P_ID=PUBLISHER.P_ID
AND Pname='Hills Publications';
```

This `EXPLAIN PLAN` statement causes the query optimizer to insert data about the execution plan for the query into `PLAN_TABLE`. Note that before creating a new plan with the same ID, all the entries from the `PLAN_TABLE` need to be deleted using the following command.

```
DELETE FROM PLAN_TABLE;
```

8.6.3 Guidelines for Writing a Query in ORACLE

Since many different SQL statements can be used to achieve the same result, it is often the case that only one statement will be the most efficient choice in a given situation. Some guidelines are given here that give information about whether one form of the statement is more efficient than the other.

- ⇒ Always include a `WHERE` clause in the `SELECT` statement to narrow the number of tuples returned until all the tuples are required. This reduces the network traffic and increases the overall performance of the query.
- ⇒ Try to limit the size of the queries result set by projecting only the specific attributes (not all) from the relation.
- ⇒ Try using stored procedures and views in place of large queries.
- ⇒ Try using constraints in place of triggers, wherever possible.
- ⇒ Try to avoid performing operations on database objects referenced in the `WHERE` clause. For example, consider this SQL statement.

```
SELECT Book_title, Price
FROM BOOK
WHERE Page_count+300 > 900;
```

This SQL statement can be written as

```
SELECT Book_title, Price
FROM BOOK
WHERE Page_count>600;
```

- ⇒ Try to use the = operator as much as possible and <> operator as least as possible. This is because the <> operator slows down the execution process.
- ⇒ Try to avoid the text functions such as LOWER or UPPER in the SELECT statement as it affects the overall performance. If the DBMS is case-sensitive then also avoid these text functions. For example, consider this SQL statement.

```
SELECT Category
FROM BOOK
WHERE UPPER(Book_title) = 'UNIX';
```

This statement can be written as

```
SELECT Category
FROM BOOK
Book_title = 'UNIX' OR Book_title = 'unix';
```

- ⇒ Try to avoid the HAVING clause, whenever possible as it filters the selected tuples only after all the tuples have been fetched. For example, consider this SQL statement.

```
SELECT AVG(Page_count)
FROM BOOK
GROUP BY Category
HAVING Category='Textbook';
```

This statement can be written as

```
SELECT AVG(Page_count)
FROM BOOK
WHERE Category='Textbook'
GROUP BY Category;
```

- ⇒ Try to avoid using the DISTINCT clause, whenever possible as it results in performance degradation. It should be used only when absolutely necessary.

◀ ● SUMMARY ● ▶

1. Query processing includes translation of high-level queries into low-level expressions that can be used at the physical level of the file system, query optimization, and actual execution of the query to get the result.
2. Query optimization is a process in which multiple query-execution plans for satisfying a query are examined and a most efficient query plan is identified for execution.
3. Query processing is a three-step process that consists of parsing and translation, optimization, and execution of the query submitted by the user.
4. Whenever a user submits an SQL query for execution, it is first translated into an equivalent extended relational algebra expression. During translation, the parser checks the syntax of the user's query according to the rules of the query language. It also verifies that all the attributes and relation names specified in the query are the valid names in the schema of the database being queried.
5. The relational algebra expressions are typically represented as query trees or parse trees. A query tree is a tree data structure in which the input relations are represented as the leaf nodes and the relational algebra operations as the internal nodes.

6. To fully specify how to evaluate a query, each operation in the query tree is annotated with the instructions which specify the algorithm or the index to be used to evaluate that operation. The resultant tree structure is known as query-evaluation plan or query-execution plan or simply plan.
7. Different evaluation plans for a given query can have different costs. It is the responsibility of the query optimizer, a component of DBMS, to generate a least-costly plan. The best query-evaluation plan chosen by the query optimizer is finally submitted to the query-evaluation engine for actual execution of the query to get the desired result.
8. Whenever, a query is submitted to the database system, it is decomposed into query blocks. A query block forms a basic unit that can be translated into relational algebra expression and optimized.
9. Sorting of tuples plays an important role in database systems. If a relation entirely fits in the main memory, in-memory sorting (called internal sorting) can be performed using any standard sorting technique such as quick sort. If the relation does not fit entirely in the main memory, external sorting is required.
10. The external sort–merge algorithm is divided into two phases, namely, sort phase and merge phase.
11. The system provides a number of different algorithms for implementing several relational algebra operations such as select, join, project, set, and aggregate operations. These algorithms can be developed using indexing, iteration, or partitioning.
12. The expressions containing multiple operations are evaluated either using materialization approach or pipelining approach.
13. In materialized evaluation, each operation in the expression is evaluated one by one in an appropriate order and the result of each operation is materialized (created) in a temporary relation which becomes input for the subsequent operations.
14. With pipelined evaluation, operations form a queue and results are passed from one operation to another as they are calculated. There are two ways of executing pipelines, namely, demand-driven pipeline and producer-driven pipeline.
15. The query submitted by the user can be processed in a number of ways especially if the query is complex. It is the responsibility of the query optimizer to construct a most efficient query-evaluation plan having the minimum cost.
16. There are two main techniques of implementing query optimization, namely, cost-based optimization and heuristics-based optimization.
17. A cost-based query optimizer generates a number of query-evaluation plans from a given query using a number of equivalence rules that specify how to transform an expression into a logically equivalent one.
18. An equivalence rule states that expressions of two forms are equivalent if an expression of one form can be replaced by expression of the other form, and vice versa.
19. The cost of each evaluation plan can be estimated by collecting the statistical information about the relations such as relation sizes and available primary and secondary indexes from the DBMS catalog.
20. In addition to the relation sizes and indexes height, real-world query optimizers also maintain histograms representing the distribution of values for each attribute to increase the accuracy of cost estimates.
21. A histogram can be of two types, namely equi-width histogram and equi-depth histogram. In equi-width histograms, the range of values is divided into equal-sized sub ranges. On the other hand, in equi-depth histograms, the sub ranges are chosen in such a way that the number of tuples within each sub range is equal.
22. In cost-based optimization, generating a large number of query-evaluation plans for a given query, and finding the cost of each plan can be very time-consuming and expensive. Therefore, the optimizers use heuristic rules to reduce the cost of optimization.
23. The semantic query optimization operates on the key idea that the optimizer has a basic understanding of the constraints such as unique, check, and other complex constraints specified on the database schema.

KEY TERMS

- Query processing
- Query optimization
- Query tree (parse tree)
- Query-evaluation plan (query-execution plan)
- Query optimizer
- Query-evaluation engine
- Query block
- Internal sorting
- External sorting
- External sort-merge algorithm
- Runs
- Degree of merging
- Two-way merging
- N-way merging
- File scan
- Index scan
- Linear search (brute force)
- Binary search
- Access path
- Select operation using indexes
- Conjunctive selection
- Disjunctive selection
- Composite index
- Intersection of record pointers
- Union of record pointers
- Two-way join
- Multiway join
- Nested-loop join
- Outer relation
- Inner relation
- Result file
- Block nested-loop join
- Indexed nested-loop join
- Temporary index
- Sort-merge join
- Hybrid merge-join
- Hash join
- Partitioning phase
- Probing phase
- Skewed partitioning
- Recursive partitioning
- Hybrid hash join
- Materialized evaluation
- Operator tree
- Double buffering
- Pipelined evaluation
- Demand-driven pipeline (lazy evaluation)
- Producer-driven pipeline (eager pipelining)
- Cost-based query optimization
- Equivalence rule
- Join ordering
- Interesting order
- Histograms
- Equi-width histograms
- Equi-depth histograms
- Heuristic query optimization
- Left-deep join orders
- Left-deep join trees
- Semantic query optimization

EXERCISES

A. Multiple Choice Questions

1. Which of the following DBMS components is responsible for generating a least-costly plan?
 - (a) Query-evaluation engine
 - (b) Translator
 - (c) Query optimizer
 - (d) Parser

2. If there are two relations R and S, with n and m number of tuples, respectively, the nested-loop join requires _____ pairs of tuples to be scanned.
 - (a) $n + m$
 - (b) $n * m$
 - (c) m
 - (d) n
3. If there are two relations R and S containing 700 and 900 tuples, respectively, then which relation should be taken as outer relation in block nested-loop join algorithm?
 - (a) R
 - (b) S
 - (c) Any of these
 - (d) None of these
4. An index created by the query optimizer and destroyed when the query is completed is known as _____.
 - (a) Optimized index
 - (b) Permanent index
 - (c) Temporary index
 - (d) Secondary index
5. Which of these is a phase in external sort–merge algorithm?
 - (a) Sort phase
 - (b) Merge phase
 - (c) Both (a) and (b)
 - (d) None of these
6. Which of these algorithms combines sort–merge join with indexes?
 - (a) Hybrid merge-join
 - (b) Indexed merge-join
 - (c) External merge-join
 - (d) Nested merge-join
7. Which of these evaluations is also known as lazy evaluation?
 - (a) Producer-driven pipeline
 - (b) Demand-driven pipeline
 - (c) Materialized evaluation
 - (d) Both (a) and (b)
8. Which of these equivalence rules is known as cascading of project operator?
 - (a) $\sigma_{c_1 \wedge c_2}(R) = \sigma_{c_1}(\sigma_{c_2}(R))$
 - (b) $\sigma_{c_1}(\sigma_{c_2}(R)) = \sigma_{c_2}(\sigma_{c_1}(R))$
 - (c) $\pi_{A_1}(\pi_{A_2}(\dots(\pi_{A_n}(R))\dots)) = \pi_{A_1}(R)$
 - (d) $\pi_{A_1, A_2, \dots, A_n}(\sigma_c(R)) = \sigma_c(\pi_{A_1, A_2, \dots, A_n}(R))$
9. Which of these operations is not commutative?
 - (a) Select
 - (b) Union
 - (c) Intersection
 - (d) Set difference
10. Given four relations P, Q, R, and S, in how many ways these relations can be joined?
 - (a) 4
 - (b) 20
 - (c) 16
 - (d) 24

B. Fill in the Blanks

1. Query processing is a three-step process that consists of parsing and translation, _____ and execution of the query submitted by the user.
2. A _____ is a tree data structure in which the input relations are represented as the leaf nodes and the relational algebra operations as the internal nodes.
3. A _____ forms a basic unit that can be translated into relational algebra expression and optimized.
4. The external sort–merge algorithm is divided into two phases, namely, sort phase and _____.
5. The linear search algorithm is also known as _____ algorithm.

6. If join operation is performed on two relations, it is termed as _____.
7. There are two main techniques of implementing query optimization, namely, _____ optimization and _____ optimization.
8. In _____ histograms, the range of values is divided into equal-sized subranges.
9. A set of equivalence rules is said to be _____ if no rule can be derived from the others.
10. In _____ pipeline, each operation at the bottom of the pipeline produces tuples without waiting for the request from the next higher-level operations.

C. Answer the Questions

1. Discuss the various steps involved in query processing with the help of a block diagram. What is the goal of query optimization? Why is it important?
2. What is a query-evaluation plan? Explain with the help of an example.
3. How does sorting of tuples play an important role in database system? Discuss the external sort-merge algorithm for select operation.
4. What are the various techniques used to develop the algorithms for relational algebra operations? Discuss the two commonly used partitioning techniques.
5. Discuss the various factors on which the cost of query optimization depends.
6. Describe the index nested loop join. How does it differ from block nested loop join?
7. Compare the brute force and binary search algorithms for select operation. Which one is better?
8. Discuss the various algorithms for implementing the select operations involving complex conditions.
9. Discuss the external sort-merge algorithm for join operation.
10. Discuss the hybrid merge-join technique. How can the cost be reduced by using this technique?
11. Describe the hash join algorithm. What is the additional optimization in hybrid hash join?
12. What is recursive partitioning?
13. Discuss the algorithms for implementing various set operations.
14. Describe pipelined evaluation. Discuss its advantages over materialized evaluation. Differentiate between demand-driven and producer-driven pipelining.
15. Discuss the two techniques of optimization.
16. What is an equivalence rule? Discuss all equivalence rules.
17. Discuss the two techniques used to eliminate duplicates during project operation.
18. How does an optimal join ordering play an important role in query optimization? Explain with the help of an example. What is interesting sort order?
19. Describe the role of statistics gathered from the database in query optimization.
20. How does the available buffer size affect the result of query optimization?
21. What is a left-deep join tree? What is the significance of such a tree with respect to join pipelining?
22. How do histograms help in increasing the accuracy of cost estimates? Discuss the two types of histograms.
23. How do heuristics help in reducing the cost of execution of a query?
24. Why do query optimizers try to apply selections early?
25. A common heuristic is to avoid the enumeration of cross products. Show a query in which this heuristic significantly reduces the optimization complexity.
26. What is meant by semantic query optimization? How does it differ from other query optimization techniques?

D. Practical Questions

- Consider a file with 10,000 pages and three available buffer pages. Assume that the external sort-merge algorithm is used to sort the file. Calculate the initial number of runs produced in the first pass. How many passes will it take to sort the file completely? What is the total I/O cost of sorting the file? How many buffer pages are required to sort the file completely in two passes?
- Consider three relations $R(\underline{A}, B, C)$, $S(\underline{C}, D, E)$ and $T(\underline{E}, F)$. If R has 500 tuples, S has 1000 tuples, and T has 900 tuples. Compute the size of $R \bowtie S \bowtie T$, and give an efficient way for computing the join.
- Consider the following SQL queries. Which of these queries is faster to execute and why? Draw the query tree also.
 - SELECT** Category, **COUNT**(*) **FROM** BOOK
WHERE Category != 'Novel'
GROUP BY Category;
 - SELECT** Category, **COUNT**(*) **FROM** BOOK
GROUP BY Category
HAVING Category != 'Novel';
- Consider the following relational algebra queries on *Online Book* database. Draw the initial query trees for each of these queries.
 - $\Pi_{P_ID, Address, Phone}(\sigma_{State="Georgia"}(PUBLISHER))$
 - $\sigma_{Category="Textbook" \wedge State="Georgia"}(BOOK \bowtie_{BOOK.P_ID = PUBLISHER.P_ID} PUBLISHER)$
 - $\Pi_{Book_title, Category, Price}(\sigma_{Aname="Charles Smith"}((BOOK \bowtie_{BOOK.ISBN=AUTHOR_BOOK.ISBN} AUTHOR_BOOK) \bowtie_{AUTHOR_BOOK.A_ID=AUTHOR.A_ID} AUTHOR))$
- Consider the following SQL queries on *Online Book* database.
 - SELECT** P.P_ID, Pname, Address, Phone
FROM BOOK B, PUBLISHER P
WHERE P.P_ID = B.P_ID **AND** Category = 'Language Book';
 - SELECT** ISBN, Book_title, Category, Price
FROM BOOK B, PUBLISHER P
WHERE P.P_ID = B.P_ID **AND** Pname='Bright Publications'
AND Price>30;
 - SELECT** ISBN, Book_title, Year, Page_count, Price
FROM BOOK B, AUTHOR A, AUTHOR_BOOK AB
WHERE B.ISBN = AB.ISBN **AND** AB.A_ID = A.A_ID
AND Aname = 'Charles Smith';
 - SELECT** Book_title, Pname, Aname
FROM BOOK B, PUBLISHER P, AUTHOR A, AUTHOR_BOOK AB
WHERE P.P_ID = B.P_ID **AND** AB.A_ID=A.A_ID
AND B.ISBN=AB.ISBN **AND** Category='Language book';

Transform these SQL queries into relational algebra expressions. Draw the initial query trees for each of these expressions, and then derive their optimized query trees after applying heuristics on them.

INTRODUCTION TO TRANSACTION PROCESSING

After reading this chapter, the reader will understand:

- The four desirable properties of a transaction, which include atomicity, concurrency, isolation, and durability
- The various states that a transaction passes through during its execution
- Why the transactions are executed concurrently
- Anomalies due to interleaved execution of transactions
- The various possible schedules of executing transactions
- The concept of serializable schedules
- Different ways of defining schedule equivalence, which include result equivalence, conflict equivalence, and view equivalence
- Use of precedence graph in testing schedules for conflict serializability
- The recoverable and cascadeless schedules
- SQL transaction statements and transaction support in SQL

A collection of operations that form a single logical unit of work is called a **transaction**. The operations that make up a transaction typically consist of requests to access existing data, modify existing data, add new data, or any combination of these requests. The statements of a transaction are enclosed within the **begin transaction** and **end transaction** statements.

For successful completion of a transaction and database changes to be permanent, the transaction must be completed in its entirety. That is, each step (or operation) in the transaction must succeed for the transaction to be successful. If any part of the transaction fails, then the entire transaction fails. An example of a transaction is paying of the utility bill from the customer's bank account which involves several operations such as debiting the bill amount from customer's account and crediting the amount to the utility provider's account. All the operations of fund transfer from customer's account to utility provider's account must succeed or fail as a group. It would be unacceptable if the customer's account is debited but the utility provider's account is not credited.

This chapter first covers the basic concepts of transactions that include the desirable properties and various states of a transaction. It then discusses how concurrent execution of multiple transactions may lead to several anomalies. The chapter also discusses the various types of transaction schedules including serial, serializable, recoverable, and cascadeless schedules. The chapter concludes by giving an overview of transaction concept in SQL.

9.1 DESIRABLE PROPERTIES OF A TRANSACTION

To ensure the integrity of the data, the database system must maintain some desirable properties of the transaction. These properties are known as **ACID properties**, the acronym derived from the first letter of the terms **a**tomicity, **c**onsistency, **i**solation, and **d**urability. **Atomicity** implies that either all of the operations that make up a transaction should execute or none of them should occur. It is the responsibility of the transaction management component of a DBMS to ensure atomicity. **Consistency** implies that if all the operations of a transaction are executed completely, the database is transformed from one consistent state to another. It is the responsibility of the application programmers who code the transactions to ensure consistency of the database. Note that during the execution of the transaction the state of the database becomes inconsistent. Such an inconsistent state of the database should not be visible to the users or other concurrently running transactions.

Learn More

If the operations in a transaction do not modify the database but only retrieve the data, the transaction is known as **read-only transaction**.

Isolation implies that each transaction appears to run in isolation with other concurrently running transactions. That is, the execution of a transaction should not be interfered by any other concurrently running transaction. It is the responsibility of the concurrency control component (discussed in Chapter 10) of the database system to allow concurrent execution of transactions without any interference from each other. **Durability** (also known as **permanence**) implies that once a transaction is completed successfully, the changes made by the transaction persist in the database, even if the system fails. The durability property is ensured by the recovery management component of the database system which is discussed in Chapter 11.

Let us explain these ACID properties with the help of an example. Consider a bookshop that maintains the *Online Book* database to give details about the available books to its customers. The customers can order the books online and can make payments through credit card or debit card. Suppose that the customer pays the bill amount through debit card. The customer has to enter the debit card number and the pin number (secret number) for his or her verification. After the verification, the bill amount is immediately transferred from the customer's account to the shopkeeper's account. Consider the customer's account as account *A* and shopkeeper's account as account *B*. For the rest of the discussion, we will consider account *A* and account *B* instead of customer's account and shopkeeper's account, respectively.

Consider a transaction T_1 that transfers \$100 from account *A* to account *B*. Let the initial values of account *A* be \$2000 and account *B* is \$1500. The sum of the values of account *A* and *B* is \$3500 before the execution of transaction T_1 . Since T_1 is a fund transferring transaction, the sum of the values of account *A* and *B* should be \$3500 even after its execution. The transaction T_1 can be written as follows:

```
 $T_1$ : read(A);
      A:=A-100;
      write(A);
      read(B);
      B:= B+100;
      write(B);
```

If transaction T_1 is executed, either \$100 should be transferred from account *A* to *B* or neither of the accounts should be affected. If T_1 fails after debiting \$100 from account *A*, but before crediting \$100 to account *B*, the effects of this failed transaction on account *A* must be undone. This is the atomicity property of transaction T_1 . The execution of transaction T_1 should also preserve the consistency of the database, that is, the sum of the values of account *A* and *B* should be \$3500 even after the execution of T_1 .

Now, let us consider the isolation property of T_1 . Suppose that during the execution of transaction T_1 when \$100 is debited from account *A* and not yet credited to account *B*, another concurrently

running transaction, say T_2 reads the values of account A and B . Since T_1 has not yet completed, T_2 will read inconsistent values. The isolation property ensures that the effects of the transaction T_1 are not visible to other transaction T_2 until T_1 is completed. If the transaction T_1 is successfully completed and the user who initiated the transaction T_1 has been notified about successful transfer of funds, then the data regarding the transfer of funds should not be lost even if the system fails. This is the durability property of T_1 . The durability can be guaranteed by ensuring that either

- ⇒ all the changes made by the transaction are written to the disk before the transaction completes, or
- ⇒ the information about all the changes made by the transaction is written to the disk and is sufficient to enable the database system to reconstruct the changes when the database system is restarted after the failure.

9.2 STATES OF A TRANSACTION

Whenever a transaction is submitted to a DBMS for execution, either it executes successfully or fails due to some reasons. During its execution, a transaction passes through various states that are *active*, *partially committed*, *committed*, *failed*, and *aborted*. Figure 9.1 shows the state transition diagram that describes how a transaction passes through various states during its execution.

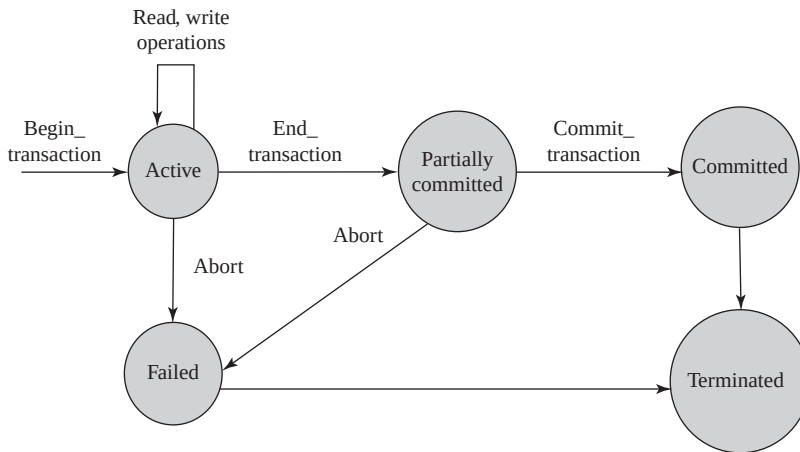


Fig. 9.1 State transition diagram showing various states of a transaction

A transaction enters into the **active** state with its commencement. At this point, the system marks **BEGIN_TRANSACTION** operation to specify the beginning of the transaction execution. During its execution, the transaction stays in the active state and executes several **READ** and **WRITE** operations on the database. The **READ** operation transfers a data item from the database to a local buffer of the transaction that has executed the read operation. The **WRITE** operation transfers the data item from the local buffer of the transaction back to the database.

Once the transaction executes its final operation, the system marks **END_TRANSACTION** operation to specify the end of the transaction execution. At this point, the transaction enters into **partially committed** state. The actual output at this point may still be residing in the main memory and thus, any kind of hardware failure might prevent its successful completion. In such a case, the transaction may have to be aborted.

Before actually updating the database on the disk, the system first writes the details of updates performed by the transaction in the log file (discussed in Chapter 11). The log file is then written to the

disk so that, even in case of failure, the system can re-construct the updates performed by the transaction when the system restarts after the failure. When this information is successfully written out in the log file, the system marks **COMMIT_TRANSACTION** operation to indicate the successful end of the transaction. Now, the transaction is said to be **committed** and all its changes must be reflected permanently in the database.

If the transaction is aborted during its active state or the system fails to write the changes in log file, the transaction enters into the **failed** state. The failed transaction must be rolled back to undo its effects on the database to maintain the consistency of the database. When the transaction leaves the system, it enters into the **terminated** state. At this point, the transaction information maintained in the log file during its execution is removed.

An aborted transaction can be **restarted** either automatically or by the user. A transaction can only be restarted automatically by the system if it was aborted due to some hardware or software error and not due to some internal logical error of the transaction. If the transaction was aborted due to some internal logic error or because the desired data was not found in the database, then it must be **killed** by the database system. In this case, the user must first rectify the error in the transaction and then submit it again to the system as a new transaction for execution.

Note that once a transaction has been committed, its effects cannot be undone by aborting it. The only way to undo the effects of a committed transaction is to execute a **compensating transaction**, which reverses the effect of a committed transaction. For example, a transaction that has added \$100 in an account can be reversed back by executing a compensating transaction that would subtract \$100 from the account. However, it is not always possible to create a compensating transaction. Therefore, it is the responsibility of the user, and not of the database system, to write and execute a compensating transaction.

Learn More

There are some situations when a transaction needs to perform write operations on a terminal or a printer. These writes are known as **observable external writes**. Since the content once written on the printer or the terminal cannot be erased, the database system allows such writes to take place only after the transaction has entered the committed state.

9.3 CONCURRENT EXECUTION OF TRANSACTIONS

The transaction-processing system allows concurrent execution of multiple transactions to improve the system performance. In concurrent execution, the database management system controls the execution of two or more transactions in parallel; however, allows only one operation of any transaction to occur at any given time within the system. This is also known as **interleaved execution** of multiple transactions. The database system allows concurrent execution of transactions due to two reasons.

First, a transaction performing read or write operation using I/O devices may not be using the CPU at a particular point of time. Thus, while one transaction is performing I/O operations, the CPU can process another transaction. This is possible because CPU and I/O system in the computer system are capable of operating in parallel. This overlapping of I/O and CPU activities reduces the amount of time for which the disks and processors are idle and, thus, increases the **throughput** of the system (the number of transactions executed in a given amount of time).

Second, interleaved execution of a short and a long transaction allows short transaction to complete quickly. If a short transaction is executed after the execution of a long transaction in a serial order, the short transaction may have to wait for a long time, leading to unpredictable delays in executing a transaction. On the other hand, if these transactions are executed concurrently, it reduces the unpredictable delays, and thereby reduces the average response time.

Note that despite the correctness of each individual transaction, undesirable interleaving of multiple transactions may lead to database inconsistency. Thus, interleaving should be done carefully to

ensure that the result of a concurrent execution of transactions is equivalent to some serial execution (one after the other) of the same set of transactions. The database system must control the interaction among concurrently executing transactions. The database system does so with the help of several **concurrency-control techniques** which are discussed in Chapter 10.

9.3.1 Anomalies Due to Interleaved Execution

If the transactions are interleaved in an undesirable manner, it may lead to several anomalies such as lost update, dirty read, and unrepeatable read. To understand the effect of these anomalies on the database, consider two transactions T_1 and T_2 , where T_1 transfers \$100 from account A to account B, and T_2 adds two percent interest to account A. Suppose that the initial values of account A and B are \$2000 and \$1500, respectively. Then, after the serial execution of transactions T_1 and T_2 (T_1 followed by T_2), the value of account A should be \$1938 and that of account B should be \$1600.

Suppose that the operations of T_1 and T_2 are interleaved in such a way that T_2 reads the value of account A before T_1 updates its value in the database. Now, when T_2 updates the value of account A in the database, the value of account A updated by the transaction T_1 is overwritten and hence, is lost. This is known as **lost update problem**. The value of account A at the end of both the transactions is \$2040 instead of \$1938 which leads to data inconsistency. The interleaved execution of transactions T_1 and T_2 that leads to lost update problem is shown in Figure 9.2.

The second problem occurs when a transaction fails after updating a data item, and before this data item is changed back to its original value, another transaction reads this updated value. For example, assume that T_1 fails after debiting \$100 from account A, but before crediting this amount to account B. This will leave the database in an inconsistent state. The value of account A is now \$1900, which must be changed back to original one, that is, \$2000. However, before the transaction T_1 is rolled back, let another transaction T_2 reads the incorrect value of account A. This incorrect value of account A that is read by transaction T_2 is called **dirty data**, and the problem is called **dirty read** problem. The interleaved execution of transactions T_1 and T_2 that leads to dirty read problem is shown in Figure 9.3.

The third problem occurs when a transaction tries to read the value of the data item twice, and another transaction updates the same data item in between the two read operations of the first transaction. As a result, the first transaction reads varied values of same data item during its execution. This is known as **unrepeatable read**. For example, consider

T_1	T_2
read(A) A:=A - 100	
	read(A) X:=A * 0.02 A:=A + X
write(A) read(B)	
	write(A)
B:=B + 100 write(B)	

Fig. 9.2 Lost update problem

T_1	T_2
read(A) A:=A - 100 write(A)	
	read(A) X:=A * 0.02 A:=A + X write(A)
read(B)	
T_1 fails	

Fig. 9.3 Dirty read problem

Learn More

The three anomalies, namely, *lost update*, *dirty read*, and *unrepeatable read* can also be described in terms of when the actions of two transactions, say T_1 and T_2 , conflict with each other. In this case, the lost update problem is known as **write-write (WW) conflict**, the dirty read problem is known as **write-read (WR) conflict**, and unrepeatable problem is known as **read-write (RW) conflict**.

a transaction T_3 that reads the value of account A. At this point, let another transaction T_4 updates the value of account A. Now if T_3 again tries to read the value of account A, it will get a different value. As a result, the transaction T_3 receives different values for two reads of account A. The interleaved schedule of transactions T_3 and T_4 that leads to a problem of unrepeatable read is shown in Figure 9.4.

T_3	T_4
read(A)	
	read(A)
	A:=A + 100
	write(A)
read(A)	
sum:=sum + A	
write(sum)	

Fig. 9.4 Unrepeatable read

9.4 TRANSACTION SCHEDULES

A list of operations (such as reading, writing, aborting or committing) from a set of transactions is known as a **schedule** (or **history**). A schedule should comprise all the instructions of the participating transactions and also preserve the order in which the instructions appear in each individual transaction.

Consider two transactions T_1 and T_5 , where T_1 transfers \$100 from account A to account B and T_5 transfers \$50 from account B to account C. Here, account C is another account of the shopkeeper. Suppose that the values of account A, B, and C are \$2000, \$1500, and \$500, respectively. The sum $A + B + C$ is \$4000. Also suppose that the two transactions are executed one after the other in the order T_1 followed by T_5 . Figure 9.5 shows the serial execution of transactions T_1 and T_5 . This sequence of execution of the transactions is called a **serial schedule**. A serial schedule consists of a sequence of instructions from various transactions, where the operations of one single transaction appear together in that schedule. Hence, for a set of n transactions, $n!$ different valid serial schedules are possible.

In Figure 9.5, the sequence of instructions is in chronological order from top to bottom with the instructions of transaction T_1 appearing in the left column followed by the instructions of T_5 appearing in the right column. The final values of accounts A, B, and C after the execution of both the transactions are \$1900, \$1550, and \$550, respectively. The sum $A + B + C$ is preserved after the execution of both the transactions.

Similarly, if the transaction T_1 is executed after the execution of transaction T_5 , then also the sum $A + B + C$ is preserved. The serial schedule of transactions T_1 and T_5 in the order T_5 followed by T_1 is shown in Figure 9.6.

A schedule can also be described with the help of shorthand notation that uses the symbols r, w, c, and a, for the operations read, write, commit, and abort,

T_1	T_5
read(A)	
A:=A - 100	
write(A)	
read(B)	
B:=B + 100	
write(B)	
	read(B)
	B:= B - 50
	write(B)
	read(C)
	C:= C + 50
	write(C)

Fig. 9.5 Schedule 1—A serial schedule in the order T_1 followed by T_5

T_1	T_5
	read(B)
	B:= B - 50
	write(B)
	read(C)
	C:= C + 50
	write(C)
read(A)	
A:=A - 100	
write(A)	
read(B)	
B:=B + 100	
write(B)	

Fig. 9.6 Schedule 2—A serial schedule in the order T_5 followed by T_1

respectively. Note that for the purpose of recovery and concurrency control, we are only interested in these four operations. The transaction id (transaction number) is also appended as a subscript to each operation in the schedule. For example, *Schedule 1* of Figure 9.5, which may be named S_1 , can be expressed as given here.

$S_1: r_1(A); w_1(A); r_1(B); w_1(B); r_5(B); w_5(B); r_5(C); w_5(C);$

Each transaction in a serial schedule is executed independently without any interference from the operations of other transactions. As long as every transaction is executed from beginning to end without any interference from other transactions, it gives a correct end result on the database. Therefore, every serial schedule is considered to be correct. However, in serial schedules only one transaction is active at a time. The **commit** or **abort** operations of the active transaction initiates the next transaction. The transactions are not interleaved in a serial schedule. Thus, if the active transaction is waiting for an I/O operation, CPU cannot be used for another transaction, which results in CPU idle time. Moreover, if one transaction is quite long, the other transactions have to wait for long time for their turn. Hence, serial schedules are unacceptable.

9.4.1 SerializableSchedules

In a multi-user database system, several transactions are executed concurrently for efficient use of system resources. When two transactions are executed concurrently, the operating system may execute one transaction for some time, then perform a context switch and execute the second transaction for some time, and then switch back to the first transaction, and so on. Thus, when multiple transactions are executed concurrently, the CPU time is shared among all the transactions. The schedule resulted from this type of interleaving of operations from various transactions is known as **non-serial schedule**.

In general, there is no way to predict exactly how many operations of a transaction will be executed before the CPU switches to another transaction. Thus, for a set of n transactions, the number of possible non-serial schedules is much larger than $n!$. However, not all non-serial schedules result in the correct state of the database. For example, the schedule for the transactions T_1 and T_5 shown in Figure 9.7 is not a correct schedule as it leaves the database in an inconsistent state. After the execution of this schedule, the final values of accounts A, B, and C are \$1900, \$1600, and \$550, respectively. Thus, the sum $A + B + C$ is not preserved.

The consistency of the database under concurrent execution can be ensured by interleaving the operations of transactions in such a way that the final output is same as that of some serial schedule of those transactions. Such a schedule is referred to as **serializable schedule**. Thus, a schedule S of n transactions $T_1, T_2, T_3, \dots, T_n$ is serializable if it is equivalent to some serial schedule of the same n transactions.

Learn More

If the operating system is given the entire responsibility of executing the transactions concurrently, then it can even generate the schedules that can leave the database in inconsistent state. Therefore, it is the responsibility of concurrency-control component of database system to ensure that only those schedules should be executed that will leave the database in a consistent state.

T_1	T_5
read(A)	
A:=A - 100	
write(A)	
read(B)	
B:=B + 100	
	read(B)
	B:= B - 50
	write(B)
	read(C)
write(B)	C:= C + 50
	write(C)

Fig. 9.7 *Schedule 3—A concurrent schedule resulting in an inconsistent state of database*

Figure 9.8 shows a non-serial schedule of transactions T_1 and T_5 which is equivalent to *Schedule 1*. After the execution of this schedule, the final values of accounts A , B , and C are \$1900, \$1550, and \$550. Thus, the sum $A + B + C$ is preserved and hence, it is a serializable schedule. The shorthand notation for this schedule is given here.

$S_4: r_1(A); w_1(A); r_5(B); w_5(B);$
 $r_1(B); w_1(B); r_5(C); w_5(C);$

9.4.2 Schedules Equivalence

Two different schedules may produce the same final state of the database. Such schedules are known as **result equivalent**, since they have same effect on the database. However, in some cases, two different schedules may accidentally produce the same final state of database. For example, consider two schedules S_i and S_j that are updating the data item Q , as shown in the Figure 9.9.

Suppose that value of data item Q is \$100 then the final state of database produced by schedules S_i and S_j is same, that is, they produce the same value of Q (\$200). However, if the initial value of data item Q is other than \$100, the final state may not be the same. For example, if the initial value of Q is \$200, then these schedules will not produce the same final state. Thus, with this initial value, the schedules S_i and S_j are not result equivalent. Moreover, these two schedules execute two different transactions and hence, cannot be considered as equivalent. Thus, result equivalence alone is not the sufficient condition for defining the equivalence of the schedules. Two other most commonly used forms of schedule equivalence are **conflict equivalence** and **view equivalence** that lead to the notions of **conflict serializability** and **view serializability**.

Conflict Equivalence and Conflict Serializability

Two operations in a schedule are said to **conflict** if they belong to different transactions, access the same data item, and at least one of them is the write operation. On the other hand, two operations belonging to different transactions in a schedule do not conflict if both of them are read operations or both of them are accessing different data items. To understand the concept of conflicting operations in a schedule, consider two transactions T_6 and T_7 that are updating the data items Q and R in the database. A non-serial schedule of these transactions is shown in Figure 9.10. Here, we have taken only **read** and **write** operations as these are the only significant operations from the scheduling point of view.

In *Schedule 5*, the **write(Q)** operation of T_6 conflicts with the **read(Q)** operation of T_7 , because both the operations are accessing the same data item Q , and one of these operations is the **write** operation. On the other hand,

T_1	T_5
read(A)	
A:=A - 100	
write(A)	
	read(B)
	B:= B - 50
	write(B)
read(B)	
B:=B + 100	
write(B)	
	read(C)
	C:= C + 50
	write(C)

Fig. 9.8 *Schedule 4—A concurrent schedule resulting in consistent state of the database*

S_i	S_j
read(Q)	read(Q)
Q:= Q + 100	Q:= Q * 2
write(Q)	write(Q)

Fig. 9.9 *Schedules S_i and S_j*

T_6	T_7
read(Q)	
write(Q)	
	read(Q)
	write(Q)
read(R)	
write(R)	
	read(R)
	write(R)

Fig. 9.10 *Schedule 5—showing only read and write operations*

the `write(Q)` operation of T_7 is not conflicting with `read(R)` operation of T_6 , because both are accessing different data items Q and R.

The order of execution of two conflicting operations is important because if they are applied in different order, they can have different effect on the database or on the other transactions in the schedule. For example, if `write(Q)` operation of T_6 is swapped with `read(Q)` operation of T_7 , the value read by the `read(Q)` operation can be different. However, the order of execution of non-conflicting operations does not matter. It means that if `write(Q)` operation of T_7 is swapped with `read(Q)` operation of T_6 , then a new schedule is produced, which will have the same effect on the database as *Schedule 5*. The equivalent schedule (*Schedule 6*) in which all the operations appear in the same order as that of the *Schedule 5* except for the `write(Q)` and `read(R)` operations of T_7 and T_6 , respectively is shown in Figure 9.11. Note that the schedules shown in Figures 9.10 and 9.11 produce the same final state of the database regardless of initial system state.

Similarly, we can reorder the following non-conflicting operations in the *Schedule 5*, without affecting its impact on the database.

- ⇒ `read(R)` operation of T_6 is swapped with `read(Q)` operation of T_7
- ⇒ `write(R)` operation of T_6 is swapped with `write(Q)` operation of T_7
- ⇒ `write(R)` operation of T_6 is swapped with `read(Q)` operation of T_7

The resultant schedule generated from the sequence of these steps is shown in Figure 9.12. Note that this schedule is a serial schedule. Thus, we have shown that *Schedule 5* is equivalent to a serial schedule (*Schedule 7*). This equivalence means that regardless of the initial system state, *Schedule 5* will produce the same final state as some serial schedule.

If a schedule S can be transformed into a schedule S' by performing a series of swaps of non-conflicting operations, then S and S' are **conflict equivalent**. In other words, two schedules are said to be conflict equivalent if the order of any two conflicting operations is same in both the schedules. For example, *Schedule 5* and *Schedule 6* are conflict equivalent. The concept of conflict equivalence leads to the concept of conflict serializability. A schedule, which is conflict equivalent to some serial schedule, is known as **conflict serializable**. Hence, *Schedule 5* is a conflict serializable schedule as it is conflict equivalent to a serial schedule (*Schedule 7*).

To better understand the concept of conflict equivalent and conflict serializable schedules, consider *Schedule 8* and *Schedule 9* of transactions T_8 and T_9 shown in Figure 9.13. The `write(Q)` operation of T_8 and `read(Q)` operation of T_9 are conflicting operations in both the schedules and the order of these operations in both the schedules is also same. Thus, *Schedule 8* and *Schedule 9* are conflict equivalent. Moreover, *Schedule 9* is a serial schedule; hence, *Schedule 8* is conflict serializable schedule.

T_6	T_7
read(Q)	
write(Q)	
	read(Q)
read(R)	write(Q)
write(R)	read(R)
	write(R)

Fig. 9.11 *Schedule 6 — after swapping non-conflicting operations of Schedule 5*

T_6	T_7
read(Q)	
write(Q)	
read(R)	
write(R)	
	read(Q)
	write(Q)
	read(R)
	write(R)

Fig. 9.12 *Schedule 7—serial schedule equivalent to Schedule 5*

T_8	T_9	T_8	T_9
write(Q)		write(Q)	
	read(Q)	read(R)	
read(R)			read(Q)
	write(Q)		write(Q)

(a) Schedule 8

(b) Schedule 9

Fig. 9.13 Conflict serializable schedules

Testing Schedules for Conflict Serializability The conflict serializability of a schedule can be tested using a simple algorithm that considers only `read` and `write` operations in a schedule to construct a **precedence graph** (or **serialization graph**). A precedence graph is a directed graph $G = (N, E)$ where $N = \{T_1, T_2, \dots, T_n\}$ is a set of nodes and $E = \{e_1, e_2, \dots, e_n\}$ is a set of directed edges. For each transaction T_i in a schedule, there exists a node in the graph. An edge e_i in the graph is shown as $(T_i \rightarrow T_j)$, where T_i is the **starting node** and T_j is the **ending node** of the edge e_i . The edge e_i is created if one of the operations in T_j appears in the schedule before some conflicting operation in T_i .

The algorithm to test the conflict serializability of a schedule S is given here.

1. For each transaction T_i participating in the schedule S , create a node labelled T_i .
2. If T_j executes a `read` operation after T_i executes a `write` operation on any data item say Q , create an edge $(T_i \rightarrow T_j)$ in the precedence graph.
3. If T_j executes a `write` operation after T_i executes a `read` operation on any data item say Q , create an edge $(T_i \rightarrow T_j)$ in the precedence graph.
4. If T_j executes a `write` operation after T_i executes a `write` operation on any data item say Q , create an edge $(T_i \rightarrow T_j)$ in the precedence graph.
5. If the resultant precedence graph is acyclic, that is, it does not contain any cycle, then the schedule S is considered to be conflict serializable. If a precedence graph contains a cycle, the schedule S is not conflict serializable.

Some examples of cyclic and acyclic precedence graphs are shown in Figure 9.14. Note that the edges $(T_i \rightarrow T_j)$ in a precedence graph can also be labelled by the name of the data item that led to the creation of the edge (see Figure 9.15).

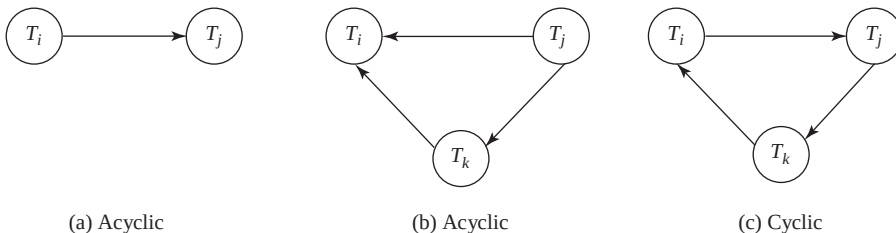


Fig. 9.14 Acyclic and cyclic precedence graphs

An edge from T_i to T_j in the precedence graph implies that the transaction T_i must appear before T_j in any serial schedule S' that is equivalent to S . Thus, if there are no cycles in the precedence graph, a serial schedule S' equivalent to S can be created by ordering the transactions in such a way that whenever an edge $(T_i \rightarrow T_j)$ exists in the precedence graph, T_i must appear before T_j in the equivalent serial

schedule S' . The process of ordering the nodes of an acyclic precedence graph is known as **topological sorting**. The topological ordering produces equivalent serial schedule.

For example, consider *Schedule 5* shown in Figure 9.10. The precedence graph of this schedule is shown in Figure 9.15(a). The precedence graph contains two nodes, one for each transaction— T_6 and T_7 . An edge from T_6 to T_7 is created which is labelled with the data items Q and R that led to the creation of this edge. Since there are no cycles in the precedence graph, *Schedule 5* is conflict serializable. The serial schedule equivalent to *Schedule 5* is shown in Figure 9.15(b).

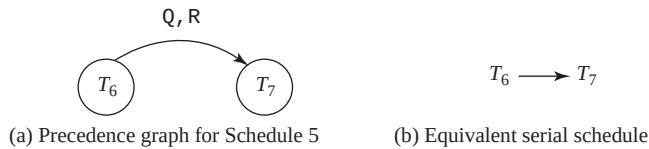


Fig. 9.15 An example of testing conflict serializability

Though only one serial schedule exists for *Schedule 5*, in general, there can be more than one equivalent serial schedule for a conflict serializable schedule. For example, consider a precedence graph for a schedule of three transactions T_1 , T_2 , and T_3 shown in Figure 9.16(a). It has two equivalent serial schedules shown in Figure 9.16(b).

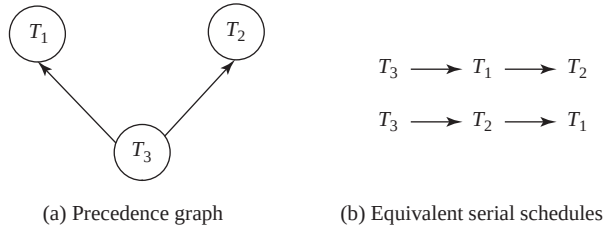


Fig. 9.16 Precedence graph with two equivalent schedules

To better understand the concept of testing a conflict serializability of a schedule, consider *Schedule 10* of three transactions T_{10} , T_{11} , and T_{12} shown in Figure 9.17(a). In this schedule, the $\text{read}(Q)$ operation of T_{10} and $\text{write}(Q)$ operation of T_{12} are the conflicting operations. Similarly, the $\text{write}(Q)$ operation of T_{12} and $\text{read}(Q)$ operation of T_{11} are the conflicting operations. The $\text{write}(R)$ operation of T_{11} and $\text{read}(R)$ operation of T_{10} are also conflicting operations. Thus, a precedence graph with edges from T_{10} to T_{12} , T_{12} to T_{11} , and T_{11} to T_{10} is created [see Figure 9.17(b)]. Since this graph contains cycle ($T_{10} \rightarrow T_{12} \rightarrow T_{11} \rightarrow T_{10}$), the *Schedule 10* is not conflict serializable. Therefore, no serial schedule equivalent to this schedule will exist.

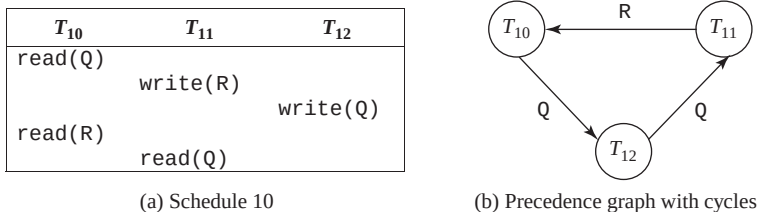


Fig. 9.17 Schedule 10 and its precedence graph

View Equivalence and View Serializability

Equivalence of schedules can also be determined by another less restrictive definition of equivalence called view equivalence. Two schedules S and S' are said to be **view equivalent** if the schedules satisfy these conditions.

1. The same set of transactions participates in S and S' , and S and S' include the same set of operations of those transactions.
2. If the transaction T_i reads the initial value of a data item say Q in schedule S , then T_i must read the initial value of same data item Q in schedule S' also.
3. For each data item Q , if T_i executes $\text{read}(Q)$ operation after the $\text{write}(Q)$ operation of transaction T_j in schedule S , then T_i must execute the $\text{read}(Q)$ operation after the $\text{write}(Q)$ operation of T_j in schedule S' also.
4. If the transaction T_i performs the final write operation on any data item say Q in schedule S , then it must perform final write operation on the same data item Q in schedule S' also.

Conditions 2 and 3 ensure that as long as each transaction reads the same values in both the schedules, they perform the same computation. Conditions 4 (along with conditions 2 and 3) ensure that the final write operation on each data item is same in both the schedules and hence, produces the same final state of the database. The concept of view equivalence leads to the concept of view serializability. A schedule S is said to be **view serializable** if it is view equivalent to some serial schedule.

To better understand the concept of view equivalence and view serializability, consider *Schedule 11* and *Schedule 12* of three transactions T_{13} , T_{14} , and T_{15} shown in Figure 9.18. In both these schedules, the transaction T_{14} reads the initial value of data item Q (condition 2). Moreover, in both the schedules, the transaction T_{13} performs the $\text{read}(R)$ operation after the $\text{write}(R)$ operation of T_{14} (condition 3). Similarly, the transaction T_{15} performs $\text{read}(R)$ operation after the $\text{write}(R)$ operation of T_{14} in both the schedules. Finally, in both the schedules, the transaction T_{15} performs the final $\text{write}(Q)$ operation (condition 4). Thus, *Schedule 11* and *Schedule 12* are view equivalent. Moreover, *Schedule 12* is a serial schedule hence, *Schedule 11* is view serializable also.

Now, consider *Schedule 13* given in Figure 9.19. In this schedule, the transactions T_{17} and T_{18} perform $\text{write}(Q)$ operation without performing the $\text{read}(Q)$ operation. Thus, the value written by the $\text{write}(Q)$ operation is independent of the old value of Q in the database. These types of write operations are known as **blind writes**. This schedule is view equivalent to some serial schedule of transactions T_{16} , T_{17} , and T_{18} . Hence, it is view serializable. However, it is not conflict serializable since its precedence graph is cyclic (see Figure 9.20). Thus, blind writes appear in a view serializable schedule that is not conflict serializable.

T_{13}	T_{14}	T_{15}
	$\text{read}(Q)$	
$\text{read}(R)$	$\text{write}(R)$	
		$\text{read}(R)$
$\text{write}(Q)$	$\text{write}(Q)$	$\text{write}(Q)$

(a) Schedule 11

T_{13}	T_{14}	T_{15}
	$\text{read}(Q)$	
$\text{read}(R)$	$\text{write}(R)$	
$\text{write}(Q)$	$\text{write}(Q)$	
		$\text{read}(R)$
		$\text{write}(Q)$

(b) Schedule 12

Fig. 9.18 View serializable schedules

Learn More

The definitions of conflict serializability and view serializability are similar if a condition known as the **constrained write assumption** holds on all transactions in a given schedule. It states that if any write operation performed on a data item P in transaction T_i is preceded by a read operation on the same data item P in T_i , the value of P written by the write operation in T_i depends only on the value of P read by that $\text{read}(P)$ operation.



Every conflict serializable schedule is also view serializable but not vice-versa.

T_{16}	T_{17}	T_{18}
read(Q)		
write(Q)		write(Q)
	write(Q)	

Fig. 9.19 Schedule 13—showing an example of blind write

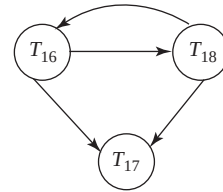


Fig. 9.20 Precedence graph of Schedule 13

9.4.3 Recoverable and Cascadeless Schedules

The computer systems are prone to failures. Hence, it is important to consider the effects of transaction failure during concurrent execution. As discussed earlier, if a transaction T_i fails due to any reason; its effects need to be undone to ensure atomicity property of the transaction. During concurrent execution of transactions, it must be ensured that any transaction T_j that depends on T_i (that is, T_j has read the value of data item written by T_i) must also be rolled back. Thus, some restrictions must be placed on the type of schedules permitted in the system.

Two types of schedules, namely, *recoverable* and *cascadeless* schedules are acceptable from the viewpoint of recovery from transaction failure. If, in a schedule S , a transaction T_j reads a data item written by T_i , then the **commit** operation of T_i should appear before the **commit** operation of T_j . Such a schedule is known as **recoverable schedule**.

Consider *Schedule 14* of transactions T_{19} and T_{20} as shown in the Figure 9.21. The transaction T_{20} performs **read(Q)** operation after the **write(Q)** operation of T_{19} . Suppose that immediately after **read(Q)** operation, transaction T_{20} commits. Now if transaction T_{19} fails, its effects need to be undone in order to ensure atomicity of the transaction. Since, T_{20} has read the value of Q written by T_{19} , it also has to be rolled back. However, T_{20} has already been committed, thus, cannot be rolled back. In such situations, it becomes impossible to recover from the failure of transaction T_{19} . Such schedules are known as **non-recoverable schedules**. The database system must ensure that the schedules are recoverable. If the **commit** operation of transaction T_{20} in Figure 9.21 is performed after the **commit** operation of T_{19} , the schedule becomes recoverable.

T_{19}	T_{20}
read(Q)	
write(Q)	
	read(Q)
	commit
read(R)	

Fig. 9.21 Schedule 14

Sometimes, even if a schedule is recoverable, several transactions may have to be rolled back in order to recover completely from the failure of a transaction T_i . It happens if transactions have read the value of a data item written by T_i . For example, consider *Schedule 15* shown in Figure 9.22. In this schedule, the transaction T_{22} reads the value of data item Q written by transaction T_{21} , and transaction T_{23} reads the value of data item Q written by transaction T_{22} .

T_{21}	T_{22}	T_{23}
read(Q)		
write(Q)		
	read(Q)	
	write(Q)	
		read(Q)

Fig. 9.22 Schedule 15

Now suppose that the transaction T_{21} fails due to any reason, it has to be rolled back. Since transaction T_{22} has read the data written by T_{21} , it also needs to be rolled back. The transaction T_{23} also needs to be rolled back because it has in turn read the data

written by T_{22} . Such a situation, in which failure of a single transaction leads to a series of transaction rollbacks, is called **cascading rollback**.

Cascading rollback requires undoing of significant amount of work and thus, is undesirable. Therefore, schedules should be restricted to avoid cascading rollbacks. The schedules that avoid cascading rollbacks are known as **cascadeless schedule**. In cascadeless schedules, transactions T_i and T_j are interleaved in such a way that T_j reads a data item previously written by T_i ; however, **commit** operation of T_i appears before the **read** operation of T_j .



Every cascadeless schedule is also recoverable.

9.5 SQL TRANSACTION STATEMENTS

As discussed earlier that the statements of a transaction are enclosed within the **begin transaction** and **end transaction** statements. However, in SQL:92 standard, there is no explicit **BEGIN TRANSACTION** statement. A transaction initiates implicitly when SQL statements are encountered. However, an explicit end statement is required which can be **COMMIT** or **ROLLBACK** statement. Execution of a **COMMIT** statement or **ROLLBACK** statement completes the current transaction.

The **COMMIT** statement terminates the current transaction and makes all the changes made by the transaction permanent in the database. That is, it *commits* the changes to the database. The **COMMIT** statement has the following general format.

```
COMMIT [WORK]
```

where,

WORK is an optional keyword that does not change the semantics of **COMMIT**

The **ROLLBACK** statement, on the other hand, terminates the current transaction and undoes all the changes made by the transaction. That is, it *rolls back* the changes to the database. The **ROLLBACK** statement has the following general format.

```
ROLLBACK [WORK]
```

In this case also, **WORK** is an optional keyword that does not change the semantics of **ROLLBACK**.

9.5.1 Transaction Characteristics in SQL

Every transaction in SQL has certain characteristics associated with it such as *access mode*, *diagnostic area size*, and *isolation level*. The **SET TRANSACTION** statement in SQL:92 is used to specify these characteristics. The **access mode** can be specified as **READ ONLY** or **READ WRITE**. The **READ ONLY** mode allows only retrieval of data from the database. The **READ WRITE** mode, on the other hand, allows **UPDATE**, **INSERT**, **DELETE**, and **CREATE** commands to be executed.

The **diagnostic area size** option, specified as **DIAGNOSTIC SIZE n**, determines the number of error or exception conditions that can be recorded in the diagnostic area for the current transaction. The diagnostic area is populated for each SQL statement that is executed. If the statement raised more conditions than the specified **DIAGNOSTIC SIZE**, only most severe conditions will be reported. For example, if we specify **DIAGNOSTIC SIZE 5** but one of the SQL statements raised 10 conditions, it is most likely that only “most severe” 5 conditions will be reported.

SQL:92 also supports four levels of transaction isolation. The **isolation levels** reduce the isolation between concurrently executing transactions. The level providing the greatest isolation from other transactions is **SERIALIZABLE**. It is the default level of isolation. At this level, a transaction is fully isolated from the changes made by other transactions. The other isolation levels are **READ UNCOMMITTED**, **READ COMMITTED**, and **REPEATABLE READ**. **READ UNCOMMITTED** is the lowest level of isolation. At this level, a transaction can read subsequent changes made by other transactions, either committed or uncommitted. With read uncommitted isolation level, one or more of the three problems, namely, *dirty read*, *unrepeatable read*, and *phantom read* may occur.

We have already discussed the dirty read and unrepeatable read problems. Phantom read problem occurs when a transaction T_i reads a set of rows from a table based on some condition specified in **WHERE** clause, meanwhile another transaction T_j inserts a new row into that table that also satisfies the condition and commits. Now, if T_i is again executed, then it will see a tuple (or phantom tuple) that previously did not exist.

The next level above **READ UNCOMMITTED** is **READ COMMITTED**, and above that is **REPEATABLE READ**. In **READ COMMITTED** isolation level, dirty reads are not possible since a transaction is not allowed to read the updates made by uncommitted transactions. However, unrepeatable reads and phantoms are possible. In **REPEATABLE READ** isolation level, dirty reads and unrepeatable reads are not possible but phantoms are possible. In **SERIALIZABLE** isolation level, dirty reads, unrepeatable reads, and phantoms are not possible. The possible violations for different transaction isolation levels are summarized in Table 9.1.

Table 9.1 Possible violations for different isolation levels

	Dirty Reads	Unrepeatable Reads	Phantoms
Read uncommitted	Possible	Possible	Possible
Read committed	Not possible	Possible	Possible
Repeatable read	Not possible	Not possible	Possible
Serializable	Not possible	Not possible	Not possible

An example of an SQL transaction consisting of multiple SQL statements is given in Figure 9.23. In this transaction, two SQL statements are given that update the **BOOK** and **PUBLISHER** relations of *Online Book* database. The first statement updates the price of all the books published by publisher *P001*. The next statement updates the email-id of the publisher with **P_ID="P002"**. If an error occurs on any SQL statement, the entire transaction is rolled back. That is, any updated value would be restored to its original value.

```

EXEC      SQL WHENEVER SQLERROR GOTO mylabel;
EXEC      SQL SET TRANSACTION
          READ WRITE,
          ISOLATION LEVEL SERIALIZABLE,
          DIAGNOSTIC SIZE 10;
EXEC      SQL UPDATE BOOK
          SET Price=Price*1.2
          WHERE P_ID="P001";
EXEC      SQL UPDATE PUBLISHER
          SET Email_id="sunsh@sunshinepub.com"
          WHERE P_ID="P002";
EXEC      SQL COMMIT;
GOTO      LAST;
mylabel:  EXEC SQL ROLLBACK;
LAST:...
```

Fig. 9.23 An example of SQL transaction

SUMMARY

1. A collection of operations that form a single logical unit of work is called a transaction. The statements of a transaction are enclosed within the begin transaction and end transaction statements.
2. For a transaction to be completed and database changes to be made permanent, a transaction has to be completed in its entirety.
3. To ensure the integrity of the data, the database system must maintain some desirable properties of the transaction. These properties are known as ACID properties, the acronym derived from the first letter of the terms atomicity, consistency, isolation, and durability.
4. Atomicity implies that either all of the operations that make up a transaction should execute or none of them should occur.
5. Consistency implies that if all the operations of a transaction are executed completely without the interference of other transactions, the consistency of the database is preserved.
6. Isolation implies that each transaction appears to run in isolation with other concurrently running transactions.
7. Durability (also known as permanence) implies that once a transaction is completed successfully, the changes made by the transaction persist, even if the system fails.
8. Whenever a transaction is submitted to a DBMS for execution, either it executes successfully or fails due to some reasons. During its execution, a transaction passes through various states such as active, partially completed, committed, failed, and aborted.
9. The transaction-processing system allows concurrent execution of multiple transactions to improve the system performance.
10. In concurrent execution, the database management system controls the execution of two or more transactions in parallel; however, allows only one operation of any transaction to occur at any given time within the system. This is also known as interleaved execution of multiple transactions.
11. The undesirable interleaved execution of transactions may lead to certain problems such as lost update, dirty read, and unrepeatable read.
12. A list of operations (such as reading, writing, aborting, or committing) from a set of transaction is known as a schedule (or history). A schedule should comprise all the instructions of the participating transactions and also preserve the order in which the instructions appear in each individual transaction.
13. A serial schedule consists of a sequence of instructions from various transactions, where the operations of one single transaction appear together in that schedule.
14. The consistency of the database under concurrent execution can be ensured by interleaving the operations of transactions in such a way that the final output is same as that of some serial schedule of those transactions. Such a schedule is referred to as serializable schedule.
15. There are several ways to define schedule equivalence. The simplest way of defining schedule equivalence is to compare the result of the schedules on the database. Two schedules are known as result equivalent if they produce same final state of the database.
16. Two other most commonly used forms of schedule equivalence are conflict equivalence and view equivalence that lead to the notions of conflict serializability and view serializability.
17. Two operations in a schedule are said to conflict if they belong to different transactions, access the same data item and at least one of them is write operation.
18. Two schedules are said to be conflict equivalent if the order of any two conflicting operations is same in both the schedules. A schedule, which is conflict equivalent to some serial schedule, is known as conflict serializable.
19. The conflict serializability of a schedule can be tested using a simple algorithm that considers only **read** and **write** operations in a schedule to construct a precedence graph. A precedence graph is a directed graph consists of a set of nodes and a set of directed edges.

20. If the precedence graph is acyclic then the schedule is considered to be conflict serializable. If a precedence graph contains a cycle, the schedule is not conflict serializable.
21. The process of ordering the nodes of an acyclic preceding graph is known as topological sorting.
22. Equivalence of schedules can also be determined by another less restrictive definition of equivalence called view equivalence. A schedule S is said to be view serializable if it is view equivalent to some serial schedule.
23. The computer systems are prone to failure. Hence, it is important to consider the effects of transaction failure during concurrent execution.
24. A situation, in which failure of single transaction leads to a series of transaction rollbacks, is called cascading rollback. The database system must ensure that the schedules are recoverable and cascadeless.

● KEY TERMS ●

- | | |
|---|--|
| <ul style="list-style-type: none"> ● ACID properties ● Atomicity ● Consistency ● Isolation ● Durability ● Active state ● Read-only transaction ● Partially committed state ● Committed transaction ● Failed state ● Terminated state ● Compensating transaction ● Interleaved execution ● Throughput ● Lost update ● Dirty read ● Unrepeatable read ● Schedule ● Serial schedule ● Non-serial schedule ● Serializable schedule | <ul style="list-style-type: none"> ● Result equivalent schedules ● Conflict serializability ● View serializability ● Conflict ● Conflict equivalent schedules ● Conflict serializable schedules ● Precedence graph (or serialization graph) ● Topological sorting ● View equivalent schedules ● View serializable schedules ● Blind writes ● Recoverable schedule ● Non-recoverable schedule ● Cascading rollback ● Cascadeless schedule ● Begin transactions statement ● End transactions statement ● Access mode ● Diagnostic release ● Isolation levels ● Phantom read problem |
|---|--|

● EXERCISES ●

A. Multiple Choice Questions

1. What does C stand for in transaction ACID properties?

(a) correctness	(b) consistency
(c) committed	(d) completeness

2. Once the transaction executes its final operation, it enters into _____ state.
(a) ~~Committed~~ (b) Terminated
(c) Partially committed (d) Failed
3. The only way to undo the effects of a committed transaction is to execute a _____.
(a) Undo transaction (b) Compensating transaction
(c) Redo transaction (d) None of these
4. Why is concurrent execution of transactions preferred over serial execution?
(a) To increase system throughput (b) To decrease average response time
(c) Both (a) and (b) (d) None of these
5. Which of these anomalies is also known as WW conflict?
(a) Lost update (b) Dirty read
(c) Unrepeatable read (d) None of these
6. Which of these operations is considered for the purpose of recovery and concurrency control?
(a) Write (b) Commit
(c) Abort (d) All of these
7. How many valid serial schedules are possible for four transactions?
(a) 24 (b) 16
(c) 4 (d) 8
8. Two schedules are known as _____ if they produce same final state of the database.
(a) Serializable (b) Conflict equivalent
(c) Result equivalent (d) View equivalent
9. The schedules that avoid cascading rollbacks are known as _____.
(a) Cascadeless schedules (b) Recoverable schedules
(c) Serial schedules (d) Non-recoverable schedules
10. How many levels of transaction isolation does SQL:92 support?
(a) Two (b) Three
(c) Four (d) Five

B. Fill in the Blanks

1. _____ implies that once a transaction is completed successfully, the changes made by the transaction persist in the database, even if the system fails.
2. If the transaction is aborted during its active state or the system fails to write the changes in log file, the transaction enters into the _____ state.
3. If the transactions are interleaved in an undesirable manner, it may lead to several anomalies such as _____, _____, and _____.
4. The interleaving of the operations of transactions in such a way that the final output is same as that of some serial schedule of those transactions is known as _____.
5. Two schedules are known as _____ if they produce same final state of the database.

6. Two operations in a schedule are said to _____ if they belong to different transactions, access the same data item and at least one of them is write operation.
7. A schedule, which is conflict equivalent to some serial schedule, is known as _____.
8. The process of ordering the nodes of an acyclic preceding graph is known as _____.
9. A schedule S is said to be _____ if it is view equivalent to some serial schedule.
10. A situation, in which failure of single transaction leads to a series of transaction rollbacks, is called _____.

C. Answer the Questions

1. What is a transaction? What are ACID properties of a transaction? Violation of which property causes lost update problem? Justify with the help of an example.
2. What are the two main operations that a transaction uses to access data from the database?
3. What do you mean by state transition diagram? When a transaction is said to be failed, explain with the help of an example.
4. Why is it important to interleave multiple transactions? Discuss the various anomalies that can occur due to undesirable interleaving of transactions. Give suitable examples also.
5. Define the following terms:

(a) Schedule	(b) Non-serial schedule	(c) Topological sorting
(d) Average response time	(e) Observable external writes	(f) Cascading rollback
(g) Blind writes		
6. What is a serial schedule? Why is it always considered to be correct?
7. What is a serializable schedule? Give an example also.
8. When two schedules are result equivalent? Explain with example.
9. Discuss the two different forms of schedule equivalence. Give the steps to test the conflict serializability of a schedule S .
10. What is a precedence graph?
11. Every cascadeless schedule is also recoverable. Justify this statement by taking suitable example.
12. Discuss the four levels of isolation in SQL. What are the possible violations for these levels?

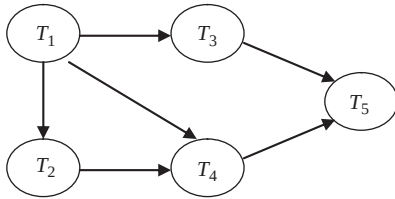
D. Practical Questions

1. Consider two transactions T_1 and T_2 given below:

T_1	T_2
read(X)	read(X)
$X := X - 10$	$X := X * 1.02$
write(X)	$Y := Y * 1.02$
read(Y)	write(X)
$Y := Y + 10$	write(Y)
write(Y)	

Give a serial schedule for these transactions and an equivalent non-serial schedule for these transactions.

2. Consider the following precedence graph. Is the corresponding schedule is conflict serializable? Give reasons also.



3. Consider three transactions T_1 , T_2 , and T_3 and two schedules S_1 and S_2 given below. Draw the precedence graph for schedules S_1 and S_2 and test whether they are conflict serializable or not.

T_1 : $r_1(P)$; $r_1(R)$; $w_1(P)$;

T_2 : $r_2(R)$; $r_2(Q)$; $w_2(R)$; $w_2(Q)$;

T_3 : $r_3(P)$; $r_3(Q)$; $w_3(Q)$;

S_1 : $r_1(P)$; $r_2(R)$; $r_1(R)$; $r_3(P)$; $r_3(Q)$; $w_1(P)$; $w_3(Q)$; $r_2(Q)$; $w_2(R)$; $w_2(Q)$;

S_2 : $r_1(P)$; $r_2(R)$; $r_3(P)$; $r_1(R)$; $r_2(Q)$; $r_3(Q)$; $w_1(P)$; $w_2(R)$; $w_3(Q)$; $w_2(Q)$;

4. Consider schedule S_3 given below. Determine whether it is cascadeless, recoverable, or non-recoverable.

S_3 : $r_1(P)$; $r_2(R)$; $r_1(R)$; $r_3(P)$; $r_3(Q)$; $w_1(P)$; $w_3(Q)$; $r_2(Q)$; $w_3(R)$; $w_2(Q)$; c_1 ; c_2 ; c_3

5. Give an example of SQL transaction.

CONCURRENCY CONTROL TECHNIQUES

After reading this chapter, the reader will understand:

- The need of concurrency control techniques
- The basic concept of locking, types of locks and their implementation
- Lock based techniques for concurrency control
- Two-phase locking and its different variants
- Specialized locking techniques, such as multiple-granularity locking, tree structured indexes, etc.
- Factors affecting the performance of locking
- Timestamp-based techniques for concurrency control
- Variants of timestamp ordering techniques
- Optimistic techniques for concurrency control
- Multiversion technique based on timestamp ordering
- Multiversion technique based on locking
- Handling deadlock
- Different deadlock prevention techniques
- Deadlock detection and recovery

Transactions execute either serially or concurrently. If all the transactions are restricted to execute serially, isolation property of transaction is maintained and the database remains in a consistent state. However, executing transactions serially unnecessarily reduce the resource utilization. On the other hand, execution of several transactions concurrently may result in interleaved operations and isolation property of transaction may no longer be preserved. Thus, it may leave the database in an inconsistent state. Consider a situation in which two transactions concurrently access the same data item. One transaction modifies a tuple, and another transaction makes a decision on the basis of that modification. Now, suppose that the first transaction rolls back. At this point, the decision of second transaction becomes invalid. Thus, **concurrency control techniques** are required to control the interaction among concurrent transactions. These techniques ensure that the concurrent transactions maintain the integrity of a database by avoiding the interference among them.

This chapter discusses the concurrency control techniques, which ensure serializability order in the schedule. These techniques are locking, timestamp-based, optimistic, and the multiversion.

10.1 LOCKING

Whenever a data item is being accessed by a transaction, it must not be modified by any other transaction. In order to ensure this, a transaction needs to acquire a *lock* on the required data items. A **lock** is a variable associated with each data item that indicates whether a read or write operation can be applied to the data item. In addition, it synchronizes the concurrent access of the data item. Acquiring the lock

by modifying its value is called **locking**. It controls the concurrent access and manipulation of the locked data item by other transactions and hence, maintains the consistency and integrity of the database. Database systems mainly use two modes of locking, namely, *exclusive locks* and *shared locks*.

Exclusive lock (denoted by X) is the commonly used locking strategy that provides a transaction an exclusive control on the data item. A transaction that wants to read as well as write a data item must acquire exclusive lock on the data item. Hence, an exclusive lock is also known as an *update lock* or a *write lock*. If a transaction T_i has acquired an exclusive lock on a data item, say Q, then T_i can both read and write Q. Further, if another transaction, say T_j , requests to access Q, then T_j has to wait for T_i to release its lock on Q. It means that no transaction is allowed to access data item when it is exclusively locked by another transaction. By prohibiting other transactions to modify the locked data item, exclusive lock prevents concurrent transactions from corrupting one another.

Shared lock (denoted by S) can be acquired on a data item when a transaction wants to only read a data item and not modify it. Hence, it is also known as **read lock**. If a transaction T_i has acquired a shared lock on data item Q, T_i can read but cannot write on Q. Moreover, any number of transactions can acquire shared locks on the same data item simultaneously without any risk of interference between transactions. However, if a transaction has already acquired a shared lock on the data item, no other transaction can acquire an exclusive lock on that data item.

10.1.1 LockCompatibility

Depending on the type of operations a transaction needs to perform, it should request a lock in an appropriate mode on that data item. If the requested data item is not locked by another transaction, the lock request is granted immediately by the concurrency control manager. However, if the requested data item is already locked, the lock request may or may not be granted depending on the *compatibility* of the locks. Lock compatibility determines whether locks can be acquired on a data item by multiple transactions at the same time.

Suppose a transaction T_i requests a lock of mode m_1 on a data item Q on which another transaction T_j currently holds a lock of mode m_2 . If mode m_2 is compatible with mode m_1 , the request is immediately granted, otherwise rejected. The lock compatibility can be represented by a matrix called the **compatibility matrix** (see Figure 10.1). The term “YES” indicates that the request can be granted and “NO” indicates that the request cannot be granted.

Requested mode	Shared	Exclusive
Shared	YES	NO
Exclusive	NO	NO

Fig. 10.1 Compatibility matrix

If the mode m_1 is shared, the lock request of transaction T_j is granted immediately, if and only if m_2 is also shared. Otherwise the lock request is not granted and the transaction T_j has to wait. On the other hand, if mode m_1 is exclusive, the lock request (either shared or exclusive) by transaction T_j is not granted and T_j has to wait.

Hence, it is clear that multiple shared-mode locks can be acquired on a data item by different transactions simultaneously. However, an exclusive-mode lock cannot be acquired on a data item until all other locks on that data item have been released.

Whenever a lock on a data item is released, the *lock status* of the data item changes. If the exclusive-mode lock on the data item is released by the transaction, the lock status of that data item becomes unlocked. Now, the lock request of the waiting transaction is considered. If more than one transactions are waiting, the lock request of one of them is granted.

If the shared-mode lock is released by the transaction holding the lock on the data item, the lock status of that data item may not always be unlocked. This is due to the reason that more than one shared-mode lock may be held on the data item. The lock status of the data item becomes unlock

only when the transaction releasing the lock is the only transaction holding the shared-mode lock on that data item. In order to count the number of transactions holding a shared lock on a data item, the number of transaction is stored in an appropriate data structure along with the data item. The number is increased by one when another transaction is granted a shared lock and is decreased by one when a transaction releases the shared lock. Note that when this number becomes zero, the lock status of the data item becomes unlocked.

A data item Q can be locked in the shared mode by executing the statement $\text{lock-S}(Q)$. Similarly, Q can be locked in the exclusive mode by executing the statement $\text{lock-X}(Q)$. A lock on the data item Q can be released by executing the statement $\text{unlock}(Q)$. If a transaction already holds a lock on any data item, we assume that it will not request a lock on that data item again. In addition, if a transaction does not hold a lock on a data item, we assume that it will not unlock a data item.

10.1.2 Implementation of Locking

A lock can be considered as a control block, which has the information about the nature of the locked data item and the identity of the transaction that acquires the lock. The locking or unlocking of the data items is implemented by a subsystem of the database system known as **lock manager**. It receives the lock requests from transactions and replies them with lock grant message or rollback message (in case of deadlock). In response to an unlock request, the lock manager only replies with an acknowledgment. In addition, it may also result in lock grant messages to other waiting transactions.

To select a transaction from the list of waiting transactions, the lock manager has to follow some priority technique. Otherwise, it may result in **starvation** of the transactions waiting for an exclusive lock. To understand the concept, consider a situation in which a transaction, say T_i , has a shared lock on a data item Q and another transaction, say T_j , requests an exclusive lock on the data item Q . Clearly, T_j has to wait until the transaction T_i releases the shared lock on Q . Further, suppose in the meanwhile, another transaction T_k requests a shared lock on Q . Since the mode of lock request is compatible with the lock held by T_i , the lock request is granted to T_k . Similarly, there may be a sequence of transactions requesting shared lock on Q . In that case, T_j never gets exclusive lock on Q , and is said to be **starved**, that is, waits indefinitely. T_j does not starve if the request of T_k is queued behind that of T_j , even if the mode of lock request is compatible with the lock held by T_i .

To guarantee freedom from starvation problem, the lock manager maintains a linked list of records for each locked data item in the order in which the requests arrive. Each record of the linked list is used to keep the transaction identifier that made the request and the mode in which the lock is requested. It also records whether the request has been granted. Lock manager uses a hash table known as **lock table**, indexed on the data item identifier, to find the linked list for a data item.

Consider a lock table, shown in Figure 10.2, which contains locks for three different data items, namely, Q_1 , Q_2 , and Q_3 . In addition, two transactions, namely, T_i and T_j , are shown which have been granted locks or are waiting for locks.

Observe that T_i has been granted locks on Q_1 and Q_3 in shared mode. Similarly, T_j has been granted lock on Q_2 and Q_3 in shared mode, and is waiting to acquire a lock on Q_1 in exclusive mode, which has already been locked by T_i .

Learn More

Both the lock and unlock request must be implemented as atomic operations in order to prevent any type of inconsistency. For this, multiple instances of the lock manager code may run simultaneously to receive the requests; however, the access to lock table requires some synchronization mechanism such as semaphore.

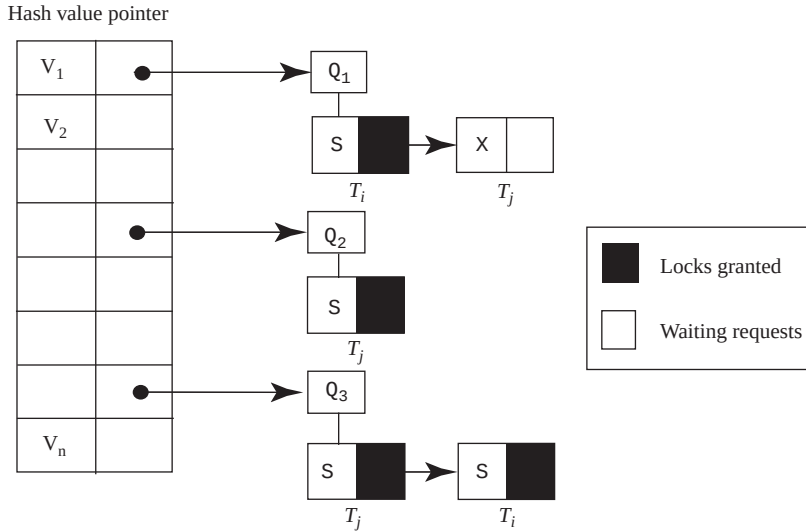


Fig. 10.2 Lock table

The lock manager processes the request by the transaction to lock and unlock a data item in the way given here.

⇒ **Lock Request:** When first request to lock a data item arrives, the lock manager creates a new linked list to record the lock request for the data item. In addition, it immediately grants the lock request of the transaction.

However, if the linked list for the data item already exists, it includes the request at the end of the linked list. Note that the lock request will be granted only if the lock request is compatible with all the existing locks and no other transaction is waiting for acquiring lock on this data item. Otherwise, the transaction has to wait.

⇒ **Unlock Request:** When an unlock request for a data item arrives, the lock manager deletes the record corresponding to that transaction from the linked list for that data item. It then checks whether other waiting requests on that data item can be granted. If the request can be granted, it is granted by the lock manager, and the next record, if any, is processed.

If a transaction aborts, the lock manager deletes all waiting lock requests by the transaction. In addition, the lock manager releases all locks acquired by the transaction and updates the records in the lock table.

10.2 LOCK-BASED TECHNIQUES

A transaction does not release a lock on the data item as long as it uses that data item. It may release the lock immediately after the final accessing of the data item is done. However, releasing the data item immediately is not always desirable. As an illustration, consider transactions T_1 and T_2 , and data items Q and R with the initial value of 500 units and 1000 units, respectively. T_1 wants to deduct 200 units from the data item R and add 200 units to the data item Q . Whereas T_2 wants to add the values of Q and R . The transactions T_1 and T_2 with lock requests are given in Figure 10.3.

T_1	T_2
lock-X(R) read(R) $R := R - 200$ write(R) unlock(R) lock-X(Q) read(Q) $Q := Q + 200$ write(Q) unlock(R)	lock-X(sum) $sum := 0$ lock-S(Q) read(Q) $sum := sum + Q$ unlock(Q) lock-S(R) read(R) $sum := sum + R$ write(sum) unlock(R) unlock(sum)

Fig. 10.3 Transactions T_1 and T_2 with lock requests

If these transactions are executed serially, either T_1 followed by T_2 or vice-versa, T_2 will display the sum 1500. Alternatively, T_1 and T_2 may be executed concurrently. A possible schedule of concurrent execution of T_1 and T_2 is shown in Figure 10.4. This schedule shows the statements issued by the transactions in the interleaved manner. Note that the lock must be granted after the transaction requests the lock but before the transaction executes its next statement. The lock can be granted anywhere in between this interval; however, we are not interested in the exact point of time where the lock is granted. For simplicity, we assume that the lock is granted immediately before the execution of next statement by the transaction.

T_1	T_2
lock-X(R) read(R) $R := R - 200$ write(R) unlock(R)	lock-X(sum) $sum := 0$ lock-S(Q) read(Q) $sum := sum + Q$ unlock(Q) lock-S(R) read(R) $sum := sum + R$ write(sum) unlock(R) unlock(sum)
lock-X(Q) read(Q) $Q := Q + 200$ write(Q) unlock(Q)	

Fig. 10.4 Schedule of T_1 and T_2

In Figure 10.4, the concurrent execution of the transactions results in displaying the sum 1300 units instead of 1500. Observe that the database is in an inconsistent state since 200 units are deducted from data item R but not added to Q. In addition, T_2 is allowed to read the values of Q and R before transaction T_1 is complete. This is because the locks on the data items Q and R are released as soon as possible. Now, suppose that the unlock statements in Figure 10.4 are delayed to the end of the transactions. Then, T_1 unlocks Q and R only after the completion of its actions. So, the database remains in consistent state.

However, sometimes delaying the lock until the end of transaction may lead to an undesirable situation, called **deadlock**. Deadlock is a situation that occurs when all the transactions in a set of two or more transactions are in a simultaneous wait state and each of them is waiting for the release of a data item held by one of the other waiting transaction in the set. None of the transactions can proceed until at least one of the waiting transactions releases lock on the data item. For example, consider the partial schedule of transactions T_3 and T_4 shown in Figure 10.5.

T_3	T_4
lock-X(R)	
...	
	lock-S(Q)
	...
	lock-S(R)
lock-X(Q)	

Fig. 10.5 *Partial schedule*

Observe that T_3 is waiting for T_4 to unlock Q, and T_4 is waiting for T_3 to unlock R. Thus, a situation is arrived where these two transactions can no longer continue with their normal execution. This situation is called deadlock. Now, one of these transactions must be rolled back by the system so that the data items locked by that transaction are released and become available to the other transaction.

From this discussion, it is clear that if locking is not used or if data items are unlocked too early, a database may become inconsistent. On the other hand, if the locks on data items are not released until the end of transaction, deadlock may occur. Out of these two problems, deadlocks are more desirable, since they can be handled by the database system but inconsistent state cannot be handled. Deadlock handling is discussed in detail in Section 10.8.

There is a need that all the transactions in a schedule must follow some set of rules called **locking technique**. These rules indicate when a transaction may lock or unlock any data item. We discuss several locking techniques in subsequent sections.

10.2.1 Two-Phase Locking

Two-phase locking requires that each transaction be divided into two phases. During the first phase, the transaction acquires all the locks; during the second phase, it releases all the locks. The phase during which locks are acquired is *growing* (or *expanding*) *phase*. In this phase, the number of locks held by a transaction increases from zero to maximum. On the other hand, a phase during which locks are released is *shrinking* (or *contracting*) *phase*. In this phase, the number of locks held by a transaction decreases from maximum to zero.

Whenever, a transaction releases a lock on a data item, it enters into the shrinking phase, and from this point, it is not allowed to acquire any lock further. So, until all the required locks on the data items are acquired, the release of the locks must be delayed. Thus, the two-phase locking leads to lower degree of concurrency among transactions. This is because a transaction cannot release any data item (which is no longer required), if it needs to acquire locks on additional data items later on. Alternatively, it must acquire locks on additional data items before it actually performs certain action on those data items, in order to release other data items. Meanwhile, another transaction that needs to access any of these locked data items is forced to wait.



The point in the schedule at which the transaction successfully acquires its last lock is called the **lock point** of the transaction.

For example, the transactions T_1 and T_2 (see Figure 10.3) can be rewritten under two-phase locking as shown in Figure 10.6.

T_1	T_2
lock-X(R) read(R) $R := R - 200$ write(R) lock-X(Q) read(Q) $Q := Q + 200$ write(Q) unlock(R) unlock(Q)	lock-X(sum) $sum := 0$ lock-S(Q) read(Q) $sum := sum + Q$ lock-S(R) read(R) $sum := sum + R$ write(sum) unlock(Q) unlock(R) unlock(sum)

Fig. 10.6 Transactions T_1 and T_2 in two-phase locking

In Figure 10.6, the statements used for releasing the lock are written at the end of the transaction. However, such statements do not always need to appear at the end of the transaction to retain two-phase locking property. For example, the `unlock(R)` statement of T_1 may appear just after the `lock-X(Q)` statement and still maintains the two-phase locking property.

The two-phase locking produces the serializable schedules. To verify this, consider a schedule S for a set of transactions $T_1, T_2, \dots, T_k$ under two-phase locking. Now assume that S is non-serializable, thus, there must exist a cycle $T_i \rightarrow T_j \rightarrow T_m \dots \rightarrow T_n \rightarrow T_i$ in the precedence graph for S . Here, $T_i \rightarrow T_j$ indicates that T_j acquires a lock on a data item released by T_i . Similarly, T_m acquires lock on a data item released by T_j , and finally T_i acquires a lock on a data item released by T_n . It means T_i acquires a lock

on a data item after releasing a lock, and this is a contradiction that T_i is following two-phase locking. Thus, our assumption that S is non-serializable is wrong.

Although, serializability is ensured by two-phase locking, cascading rollback and deadlock is possible under two-phase locking. To avoid cascading rollback, a modification of two-phase locking called **strict two-phase locking** can be used. In the strict two-phase locking, a transaction does not release any of its exclusive-mode locks until it commits or aborts. If all the transactions $\{T_1, T_2, \dots, T_n\}$ participating in a schedule S follow strict two-phase locking, the schedule S is said to be **strict schedule**. Strict schedules ensure that no other transaction can read or write the data item exclusively locked by a transaction T until it commits. It makes strict schedules recoverable. However, strict two-phase locking can also lead to deadlock.

Another more restrictive variation of two-phase locking is known as **rigorous two-phase locking**. In rigorous two-phase locking, a transaction does not release any of its locks (both exclusive and shared) until it commits or aborts.

Lock Conversion

When **lock conversions** are allowed, the transactions can change the mode of locks from one mode to another on a data item on which it already holds a lock. A transaction can request a lock on a data item many times but it can hold only one lock on the data item at any time. Lock conversion helps in achieving more concurrency.

To understand the need of lock conversion, suppose that a transaction T_i has locked a data item Q_1 in shared mode. Further, consider a transaction T_j given in Figure 10.7.

Suppose the transaction T_j follows two-phase locking, then it needs the data items Q_1 , Q_2 , and Q_3 to be locked in exclusive mode. Clearly, T_j has to wait, since T_i has locked the data item Q_1 in shared mode.

However, one can easily observe that T_j needs exclusive lock on Q_1 only when it updates Q_1 , that is, at the end of its execution. Till then, it just needs a shared lock on Q_1 . Thus, if T_j initially acquires shared lock on Q_1 , it may start processing concurrently with T_i . Later, when T_i has done with Q_1 and released its shared lock, then, T_j can easily acquire exclusive lock on Q_1 to update it. This changing of lock mode from shared (less restrictive) to exclusive (more restrictive) is known as **upgrading**. Similarly, changing the lock mode from exclusive to shared is known as **downgrading**. Therefore, to achieve more concurrency, two-phase locking is extended to allow conversion of locks.

Learn More

To achieve more concurrency, a new lock mode referred to as **update** mode can be introduced. This lock mode is compatible with shared mode but not with other lock modes. Initially, each transaction locks the data item in update mode. Later, it can upgrade the lock mode to exclusive if it needs to modify the data item, or downgrade to shared if it does not.

T_j
<pre> read(Q₁) read(Q₂) Q₂ := Q₁ + 200 write(Q₂) read(Q₃) Q₃ := Q₃ + Q₂ write(Q₃) Q₁ := Q₁ + Q₃ write(Q₁) </pre>

Fig. 10.7 Transaction T_j



NOTE The lock on a data item can be upgraded in only growing phase. On the other hand, the lock on a data item can be downgraded in only shrinking phase.

A transaction that already holds a lock of mode m_1 on Q can upgrade it to a more restrictive mode m_2 , if no other transaction holds a lock conflicting with mode m_2 , otherwise it has to wait. On the other hand, a transaction that already holds a lock of mode m_1 on Q can downgrade it to a less restrictive mode m_2 on Q at any time.

10.2.2 Graph-Based Locking

We have discussed two-phase locking, which ensures serializability even when the information about the order in which the data items are accessed is unknown. However, if we know in advance the order in which the data items are accessed, it is possible to develop other techniques to ensure conflict serializability. One such technique is **graph-based locking**.

In this technique, a directed acyclic graph called a **database graph** is formed, which consists of a set $S = \{A, B, \dots, L\}$ of all data items as its nodes, and a set of directed edges. A directed edge from A to B ($A \rightarrow B$) in the database graph denotes that any transaction T_i must access A before B when it needs to access both A and B. For simplicity, we discuss a simple kind of graph-based locking called the **tree-locking**. In this all database graphs are tree-structured, and any transaction T_i can acquire only exclusive locks on data items. A tree-structured database graph for the set S is shown in Figure 10.8.

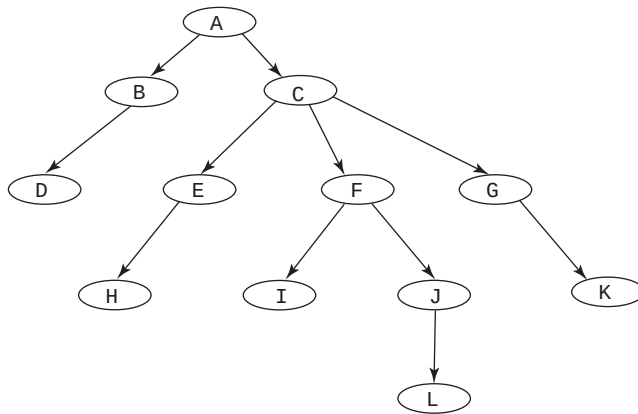


Fig. 10.8 Tree-structured database graph

A transaction T_i can acquire its first lock on any data item. Suppose T_i requests a lock on data item F, it is granted. After that, T_i can lock a data item only if it has already locked the parent of that data item. For example, if transaction T_i needs to access L, it has to lock J before requesting lock on L. Note that locking the parent of any data item does not automatically lock that data item.

Tree locking allows a transaction to unlock a data item at any time. However, once a transaction releases lock on a data item, it cannot relock that data item. Tree-locking not only ensures conflict serializability but also ensures freedom from deadlock. However, recoverability and cascadelessness are not ensured.

To ensure recoverability and cascadelessness, the tree-locking technique can be modified. The modified technique does not allow any transaction to release its exclusive locks until it commits or aborts. Clearly, it reduces the concurrency. Alternatively, we have a way that helps in improving concurrency but ensures only recoverability. In this, for each data item, we record the uncommitted transaction T_i that issued the last write instruction to that data item. Further, whenever T_j reads any such data item, we ensure that T_j cannot commit before T_i . To ensure this, we record the commit dependency of T_j on all the transactions that issued the last write instruction to the data items that T_j reads. If any one of them rolls back, T_j must also be rolled back.

The advantages of the tree-locking are given here.

- ⇒ It ensures freedom from deadlock.
- ⇒ Unlike two-phase locking, transactions can unlock data items earlier.
- ⇒ It ensures shorter waiting times and greater amount of concurrency.

The disadvantage of the tree-locking is that in order to access a data item, a transaction has to lock other data items even if it does not need to access them. For example, in order to lock data items A and L in Figure 10.8, a transaction must lock not only A and L, but also data items C, F, and J. Thus, the number of locks and associated locking overhead including possibility of additional waiting time is increased.

10.3 SPECIALIZED LOCKING TECHNIQUES

A static view of a database has been considered for locking as discussed so far. In reality, a database is dynamic since the size of database changes over time. To deal with the dynamic nature of database, we need some specialized locking techniques, which are discussed in this section.

10.3.1 Handling the Phantom Problem

Due to the dynamic nature of database, the **phantom problem** may arise. Consider the **BOOK** relation of *Online Book* database that stores information about books including their price. Now, suppose that the **PUBLISHER** relation is modified to store information about average price of books that are published by corresponding publishers in the attribute **Avg_price**. Consider a transaction T_1 that verifies whether the average price of books in **PUBLISHER** relation for the publisher *P001* is consistent with the information about the individual books recorded in **BOOK** relation that are published by *P001*. T_1 first locks all the tuples of books that are published by *P001* in **BOOK** relation and thereafter locks the tuple in **PUBLISHER** relation referring to *P001*. Meanwhile, another transaction T_2 inserts a new tuple for a book published by *P001* into **BOOK** relation, and then, before T_1 locks the tuple in **PUBLISHER** relation referring to *P001*, T_2 locks this tuple and updates it with the new value. In this case, average information of T_1 will be inconsistent even though both transactions follow two-phase locking, since new book tuple is not taken into account. The new book tuple inserted into **BOOK** relation by T_2 is called a **phantom tuple**. This is because T_1 assumes that the relation it has locked includes all information of books published by *P001*, and this assumption is violated when T_2 inserted the new book tuple into **BOOK** relation.

Here, the problem is that T_1 did not lock what it logically required to lock. Instead of locking existing tuples of publisher *P001*, it also needed to restrict the insertion of new tuples having $P_ID = "P001"$. This can be done by using a general technique known as **predicate locking**. In this technique, all the tuples (whether existing or new tuples that are to be inserted) that satisfy an *arbitrary predicate* are locked. In our example, since the attribute that refers to the publisher who has published the book is P_ID , the predicate which locks all the books of *P001* publisher is $P_ID = "P001"$. However, the predicate locking is an expensive technique, so most of the systems do not support predicate locking. These systems yet manage to prevent the phantoms problem by using another technique, called **index locking**. In this technique, any transaction that tries to insert a tuple with predicate $P_ID = "P001"$ must insert a data entry pointing to the new tuple into the index and is blocked until T_1 releases the lock. These indexes must be locked in order to eliminate the phantom problem. Here, we take only B^+ -tree index to maintain simplicity.

In order to access a relation, one or more indexes are used. In our example, assume that we have an index on **BOOK** relation for P_ID . Then, the entry in P_ID will be locked for *P001*. In this way,

the creation of phantoms will be prevented since the creation requires the index to be updated and it also requires an exclusive lock to be acquired on that index.

10.3.2 Concurrency Control in Tree-Structured Indexes

Like other database objects, indexes also need to be accessed concurrently by the transactions. Further, the changes in index structure between two accesses by a transaction are acceptable as long as the index structure directs the transaction to the correct set of tuples.

We can apply two-phase locking to tree-structured indexes like B⁺-trees; however, it results in low degree of concurrency. This is because searching an index always starts at the root, and if any transaction locks the root node, then all other transactions with conflicting lock requests have to wait. Thus, we require other techniques for managing concurrent access to indexes. In this section, we present two techniques for concurrency control in tree-structured indexes.

The first technique is **crabbing**, which proceeds in the similar manner as a crab walks, that is, releasing lock on a parent node and acquiring lock on a child node and so on alternately. To understand this technique, consider a transaction T_i which wants to insert a tuple in a relation on which a B⁺-tree index exists. Clearly, T_i also needs to insert a key-value (or corresponding entry) in the appropriate node. For this, the transaction T_i proceeds as follows:

- ⇒ T_i first locks the root node of tree in shared mode. Then, it acquires a shared lock on the child node, which is to be traversed next, and releases the lock on the parent node. This process is repeated till we traverse down to the appropriate leaf node.
- ⇒ T_i then upgrades its shared lock on leaf node to exclusive lock, and inserts the key-value there, assuming that the leaf node has space for the new entry. Otherwise, the node needs to be split.
- ⇒ In case of split, T_i acquires exclusive-mode lock on its parent, performs the splitting operation (as explained in Chapter 07), and releases its lock on the parent node.

Note that in case of split, T_i needs to acquire exclusive locks on the nodes while moving up the tree. Thus, there is a possibility of deadlock of T_i with any other transaction T_j that is trying to traverse down the tree. However, such deadlocks can be handled by restarting T_j .

An alternative technique uses a variant of the B⁺-tree called **B-link tree**. In B-link tree, every node of the tree (not just leaf nodes) maintains a pointer that points to its right sibling. This technique differs from crabbing as it allows the transactions to release locks on nodes even before acquiring locks on the child nodes. Thus, allows more transactions to operate concurrently.

Now, consider a transaction T_i that inserts a key-value in a leaf node and a split occurs. The split causes the redistribution of key-values in the sibling nodes as well as in the parent node. Suppose at that point of time, another transaction T_j follows the same path for searching a key-value. Since, the key-values of the node are already redistributed with the right siblings by T_i , T_j may not find the required key-value in the desired leaf node. However, T_j can still complete its searching process by following the pointer to the right sibling. This is the reason for the nodes at every level of the tree to have pointers to right siblings. Note that the search and insertion operation cannot result in deadlock.

Performing the deletion operation is very simple one. The transaction traverses down the tree to search the required leaf node and removes the key-value from the leaf node. If there are few key-values left in the node after the deletion operation, it needs to be coalesced (or combined). In case of coalescing, the sibling nodes must be locked in exclusive mode. After coalescing on the same level, an exclusive lock is requested on the parent node to remove the deleted node. At this point, the locks on the nodes that are combined are released. Note that if there are few key-values left in the parent node, it can also be coalesced with its siblings. Once the transaction has combined the two parent nodes, its lock is released.

In case of coalescing of nodes during deletion operation, a concurrent search operation may find a pointer to a deleted node, if the parent node is not updated. In such situations, search operation has to be restarted. To avoid such inconsistencies, nodes can be left uncoalesced. If this is done, the B⁺-tree will contain the nodes with few values, which violates its properties.

10.3.3 Multiple-Granularity Locking

Before discussing multiple-granularity locking, one must first understand the concept of locking granularity. **Locking granularity** is the size of the data item that the lock protects. It is important for the performance of a database system. **Coarse granularity** refers to large data item sizes such as an entire relation or a database, whereas **fine granularity** refers to a small data item sizes such as either a tuple or an attribute. If the larger data item is locked, it is easier for the lock manager to manage the lock. The overhead associated with locking the large data item is low since there are fewer locks to manage. In this case, the degree of concurrency is low because the transaction is holding a lock on a large data item, even if it is accessing only a small portion of the data item. Observe that the transaction completes its operations successfully, but at the expense of forcing many transactions to wait. On the other hand, the overhead associated with locking the small data item is considerably high since there are many locks to be managed. However, the degree of concurrency in this case is high, since any number of transactions can acquire any number of locks, which will be managed by the lock manager.

A transaction requires lock granularity on the basis of the operation being performed. An attempt to modify a particular tuple requires only that tuple to be locked. At the same time, an attempt to modify or delete multiple tuples requires an entire relation to be locked. Since different transactions have dissimilar requests, it is desirable that the database system provides a range of locking granules called **multiple-granularity locking**. The efficiency of the locking mechanism can be improved by considering the lock granularity to be applied. However, the overheads associated with multiple-granularity locking can outweigh the performance gains of the transactions.

Multiple-granularity locking permits each transaction to use levels of locking that are most suitable for its operations. This implies that long transactions use coarse granularity and short transactions use fine granularity. In this way, long transactions can save time by using few locks on a large data item and short transactions do not block the large data item, when its requirement can be satisfied by the small data item.

Note that the size of the data items to be locked influences the level of concurrency. While choosing locking granularity, trade-off between concurrency and overheads should be considered. In other words, the choice is between the loss of concurrency due to coarse granularity and increased overheads of maintaining fine granularity. This choice depends upon the type of the applications being executed and how those applications utilize the database. Since the choice depends upon the type of the applications, it is appropriate for the database system to support multiple-granularity locking.

As shown in Figure 10.9, the hierarchy of data items of various sizes can be represented in the form of a tree in which small data items are nested within larger data items. The hierarchy in this figure consists of database DB with three files, namely, F_1 , F_2 , and F_3 . Each file consists of several records as its child nodes. Here, notice the difference between the multiple-granularity tree and tree-locking. In the multiple-granularity tree, each non-leaf node represents data associated with its descendents whereas each node in tree-locking is an independent data item.

Learn More

To predict an appropriate locking granularity for a given transaction, a technique called **lock escalation** may be used. In this technique, initially, a transaction starts locking items of fine granularity. But when the transaction has exceeded a certain number of locks at that granularity, it starts obtaining locks at the next higher level of granularity.

Whenever a transaction acquires shared-mode or exclusive-mode lock on a node in multiple-granularity tree, the descendants of that node are implicitly locked in the same mode. Consider the scenario given here to clear this point: Suppose T_i acquires an exclusive lock on file F_1 in Figure 10.9. In that case, it has an exclusive lock on all the records, that is, $R_{11}, \dots, R_{1n}$ of that file. Observe that, in this way, T_i has to acquire only a single lock on F_1 instead of acquiring locks on individual records of F_1 . Now, assume that T_j requests a shared lock on R_{11} of F_1 . Now, T_j must traverse from the root of the tree to record R_{11} to determine whether this request can be granted. If any node in that path is locked in an incompatible mode, the lock request for R_{11} is denied and T_j must wait. Further, assume that another transaction T_k wants to lock the entire database. This can be done by locking the root of the tree. However, this request should not be granted since T_i is holding a lock on file F_1 . In order to determine whether the root node can be locked, the tree has to be traversed to examine every node to see whether any of them is currently locked by any other transaction. This would be undesirable and would defeat the purpose of multiple-granularity locking technique.

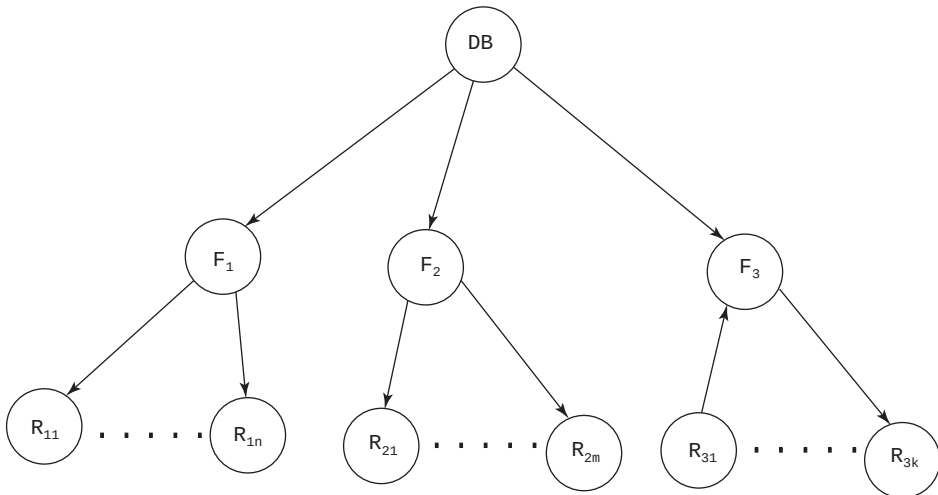


Fig. 10.9 Multiple-granularity tree

Instead, we introduce another type of lock mode, called **intention lock**, in which a transaction intends to explicitly lock a lower level of the tree. While traversing from the root node to the desired node, all the nodes along the path are locked in intention-mode. In simpler terms, no transaction is allowed to acquire a lock on a node before acquiring an intention-mode lock on all its ancestor nodes. For example, to lock a node R_{11} in Figure 10.9, a transaction needs to lock all the nodes along the path from root node to node R_{11} in an intention-mode before acquiring lock on node R_{11} .

To provide higher degree of concurrency, the intention-mode is associated with shared-mode and exclusive-mode. When intention-mode is associated with shared-mode, it is called **intention-shared (IS) mode**, and when it is associated with exclusive-mode, it is called **intention-exclusive (IX) mode**. To lock a node in shared mode, all its ancestor nodes must be locked in intention-shared (IS) mode. Similarly, to lock a node in exclusive mode, all its ancestor nodes must be locked in intention-exclusive (IX) mode.



NOTE *Intention-shared (IS) lock is incompatible only with exclusive lock whereas intention-exclusive (IX) lock is incompatible with both shared and exclusive locks.*

It is clear from the discussion that the lower level of the tree has to be explicitly locked in the mode requested by the transaction. However, it is undesirable if a transaction needs to access only a small portion of the tree. Thus, another type of lock mode, called **shared and intention-exclusive (SIX) mode** is introduced. The shared and intention-exclusive mode explicitly locks the sub tree rooted by a node in the shared mode, and the lower level of that sub tree is explicitly locked in exclusive-mode. This mode provides the higher degree of concurrency than exclusive mode because it allows other transactions to access the part of the sub tree that is not locked in the exclusive mode.

Relative Privilege of Locking Modes

Now, consider Figure 10.10, which shows the relative privilege of the locking modes. Among all the locking modes, the highest privilege is given to the exclusive mode. This is because it does not allow other transactions to acquire any lock on the portion of the tree, which is rooted at the node locked in exclusive mode. In addition, the lower level of the sub tree, rooted by that node, is implicitly locked in the exclusive mode. On the other hand, the lowest privilege is given to the intention-shared mode. The shared mode and the intention-exclusive mode are given same privilege.

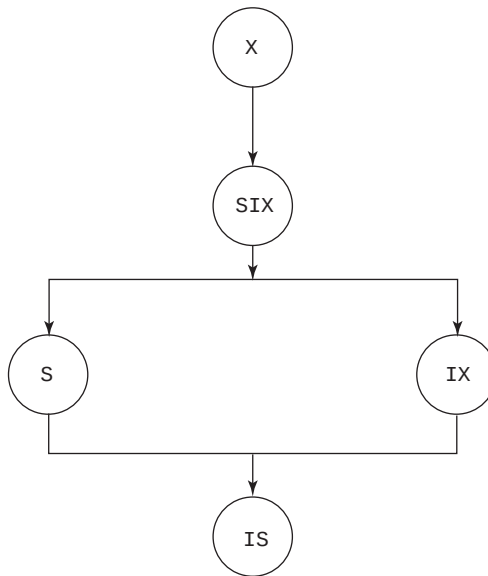


Fig. 10.10 Relative privilege of the locking modes

Consider two lock modes, namely, m_1 and m_2 , such that m_1 is given higher privilege than m_2 . Now, if a lock request of mode m_2 is denied on a data item, then a lock request of mode m_1 on the same data item will also be denied definitely. It implies that if there is “NO” in column of m_2 in the compatibility matrix (see Figure 10.11) for a particular lock request, there will be “NO” in the column of m_1 .

Requested mode	Current lock on the node				
	X	SIX	IX	S	IS
X	NO	NO	NO	NO	NO
SIX	NO	NO	NO	NO	YES
IX	NO	NO	YES	NO	YES
S	NO	NO	NO	YES	YES
IS	NO	YES	YES	YES	YES

Fig. 10.11 Compatibility matrix for different access modes

Locking Rules

Each transaction T_i can acquire a lock on node N in any locking mode (see Figure 10.11) by following certain rules, which ensure serializability. These rules are given here.

- ⇒ T_i must adhere to the compatibility matrix given in Figure 10.11.
- ⇒ T_i must acquire any lock on the root of the tree before acquiring any lock on N.
- ⇒ T_i can acquire S or IS lock on N only if it has successfully acquired either IX or IS lock on the parent of N.
- ⇒ T_i can acquire X, SIX, or IX lock on N only if it has successfully acquired either IX or SIX lock on the parent of N.
- ⇒ T_i can acquire additional locks if it has not released any lock. Observe that this is requirement of two-phase locking.
- ⇒ T_i must release the locks in bottom-up order (that is, leaf-to-root). Therefore, T_i can release its lock on N only if it has released all its locks on the lower level of the sub tree rooted by N.

10.4 PERFORMANCE OF LOCKING

Normally, two factors govern the performance of locking, namely, *resource contention* and *data contention*. **Resource contention** refers to the contention over memory space, computing time and other resources. It determines the rate at which a transaction executes between its lock requests. On the other hand, **data contention** refers to the contention over data. It determines the number of currently executing transactions.

Now, assume that the concurrency control is turned off; in that case the transactions suffer from resource contention. For high loads, the system may thrash, that is, the throughput of the system first increases and then decreases. Initially, the throughput increases since only few transactions request the resources. Later, with the increase in the number of transactions, the throughput decreases. If the system has enough resources (memory space, computing power, etc.) that make the contention over resources negligible, the transactions only suffer from data contention. For high loads, the system may thrash due to *aborting* (or rollback) and *blocking*. Both the mechanisms degrade the performance.

Clearly, aborting the transaction degrades the performance as the work done so far by the transaction is wasted. However, performance degradation due to blocking is more subtle as it prevents other transactions to access the data item which is held by blocked transaction. An instance of blocking is deadlock in which two more transactions are in a simultaneous wait state, and are waiting for some data item, which is locked by one of the other transactions. Hence, the transactions are blocked till the time one of the deadlocked transactions is aborted. Practically, it is seen that there are less than 1% of transactions, which are involved in a deadlock and there are comparatively lesser aborts. Thus, the system thrashes mainly due to blocking.

Consider how system thrashes due to blocking. Initially, the first few transactions are unlikely to block each other; as a result, the throughput increases. Transactions begin to block each other when more and more transactions execute concurrently. Thus, chances of occurrence of blocking increase with the increase in the number of active transaction. However, the throughput does not increase with the increase in the number of transactions. In fact, there comes a point when including additional transaction decreases the throughput. This is because additional transaction is blocked and competes with other transactions to acquire locks on the data item, which decreases the throughput. At this point, the system thrashes, which is shown in Figure 10.12.

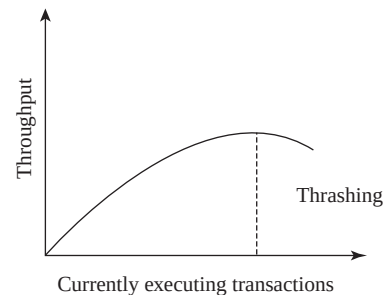


Fig. 10.12 Thrashing

In order to handle this situation, the database administrator should reduce the number of concurrently executing transactions. It is seen that thrashing occurs when 30% of the currently executing transactions are blocked. So, the database administrator should use this fraction of blocked transaction to check whether the system thrashes or not. In addition, the database administrator can increase the throughput in following ways:

- ⇒ By locking the data item with smallest granularity.
- ⇒ By reducing the time for which the transaction can hold lock on a data item.
- ⇒ By reducing **hot spots** which are frequently accessed data items that remain in the buffer for long period of time.

10.5 TIMESTAMP-BASED TECHNIQUE

So far, we have discussed that the locks with the two-phase locking ensures the serializability of schedules. Two-phase locking generates the serializable schedules based on the order in which the transactions acquire the locks on the data items. A transaction requesting a lock on a locked data item may be forced to wait till the data item is unlocked. Serializability of the schedules can also be ensured by another method, which involves ordering the execution of the transactions in advance using timestamps.



Timestamp-based concurrency control is a non-lock concurrency control technique, hence, deadlocks cannot occur.

10.5.1 Timestamps

Timestamp is a unique identifier assigned to transactions in the order they begin. If transaction T_i begins before transaction T_j , T_i is assigned a lower timestamp. The priority of a transaction will be higher if it is assigned lower timestamp. More precisely, the older transaction has the higher priority since it is assigned a lower timestamp. The timestamp of a transaction T_i is denoted by $TS(T_i)$. Timestamp can be considered as *starting time* of the transaction and it is assigned to each transaction T_i in the order in which the transactions are entered in the database system. Suppose a new transaction T_j enters in the system after T_i . In that case, the timestamp of T_i is less than that of T_j , which can be denoted as $TS(T_i) < TS(T_j)$.

The timestamp of each transaction can be generated in two simple ways:

- ⇒ The value of *system clock* can be used as the timestamp. When a transaction enters the system, it must be assigned a timestamp that is equal to the value (that is, current date/time) of the system clock. It must be ensured that no two transactions are assigned the timestamp values at the same tick of the system clock.
- ⇒ A *logical counter* can be used which is incremented each time a transaction is assigned a timestamp. When a transaction enters the system, it must be assigned a value that is equal to the value of the counter. The value of the counter may be numbered as 1, 2, 3, A counter has finite number of values, so the value of the counter must be reset periodically to zero when no transaction is currently executing for some short period of time.

10.5.2 The Timestamp Ordering

In the timestamp ordering, the transactions are ordered on the basis of their timestamps. It means when two concurrently executing transactions, namely, T_i and T_j are assigned the timestamp values $TS(T_i)$ and $TS(T_j)$, respectively, such that $TS(T_i) < TS(T_j)$, the system generates a schedule equivalent to a serial schedule in which the older transaction T_i appears before the younger transaction T_j . This is termed as **timestamp ordering (TO)**.

When the timestamp ordering is enforced, the order in which the data item is accessed by conflicting operations in the schedule does not violate the serializability order. In order to determine this, the following two timestamps values must be associated with each data item Q .

Things to Remember

Although the timestamp ordering is a non-locking technique in the sense that it does not lock a data item from concurrent access for the duration of a transaction. But in actual, at the time of recording timestamps against the data item, it requires a lock on the data item or on its instance for a small duration.

- ⇒ **read_TS(Q)**: The **read timestamp** of Q ; this is the largest timestamp among all the timestamps of transactions that have successfully executed $read(Q)$.
- ⇒ **write_TS(Q)**: The **write timestamp** of Q ; this is the largest timestamp among all the timestamps of transactions that have successfully executed $write(Q)$.



NOTE Whenever new $read(Q)$ or $write(Q)$ operations are executed, $read_TS(Q)$ and $write_TS(Q)$ are updated.

A number of implementation techniques based on the timestamp ordering have been proposed for concurrency control techniques, which ensure that all the conflicting operations of the transactions are executed in timestamp order. Three of them, namely, *basic timestamp ordering*, *strict timestamp ordering*, and *Thomas' write rule* are discussed here.

Basic Timestamp Ordering

The **basic timestamp ordering** ensures that the transaction T_i is executed in the timestamp order whenever T_i requests read or write operation on Q . This is done by comparing the timestamp of T_i with $read_TS(Q)$ and $write_TS(Q)$. If the timestamp order is violated, the system rolls back the transaction T_i and restarts it with a new timestamp. In this situation, a transaction T_j , which may have used a value of the data item written by T_i , must be rolled back. Similarly, any other transaction T_k , which may have used a value of the data item written by T_j , must also be rolled back, and so on. This situation is known as **cascading rollback**, which is one of the problems with basic timestamp ordering.

Basic timestamp ordering operates as follows:

1. Transaction T_i requests a read operation on Q :
 - (a) If $TS(T_i) < write_TS(Q)$, the read operation is rejected and T_i is rolled back. This is because another transaction with higher timestamp (that is, younger transaction) has already written the value of Q , which T_i needs to read. The transaction T_i is too late in performing the required read operation and any other values it has acquired are expected to be inconsistent with the modified value of Q . Thus, it is better to rollback T_i and restart it with a new timestamp.
 - (b) If $TS(T_i) \geq write_TS(Q)$, the read operation is executed, and $read_TS(Q)$ is set to the maximum of $read_TS(Q)$ and $TS(T_i)$. This is because T_i is younger than the transaction

that has written the value of Q , which T_i needs to read. If the timestamp of T_i is more than the current value of read timestamp (that is, $\text{read_TS}(Q)$), the timestamp of T_i becomes the new value of $\text{read_TS}(Q)$.

2. Transaction T_i requests a write operation on Q :

- If $\text{TS}(T_i) < \text{read_TS}(Q)$, the write operation is rejected and T_i is rolled back. This is because another transaction with higher timestamp (that is, younger transaction) has already read the value of Q , which T_i needs to write. So, writing the value of Q by T_i violates the timestamp ordering. Thus, it is better to rollback T_i and restart it with a new timestamp.
- If $\text{TS}(T_i) < \text{write_TS}(Q)$, the write operation is rejected and T_i is rolled back. This is because another transaction with higher timestamp (that is, younger transaction) has already written the value of Q , which T_i needs to write. So, the value that T_i is attempting to write becomes obsolete. Thus, it is better to rollback T_i and restart it with a new timestamp.
- If $\text{TS}(T_i) \geq \text{read_TS}(Q)$ and $\text{TS}(T_i) \geq \text{write_TS}(Q)$, the write operation is executed, and the value of $\text{TS}(T_i)$ becomes the new value of $\text{write_TS}(Q)$. This is because T_i is younger than both the transactions that have last written the value of Q and read the value of Q .

Consider two transactions, namely, T_5 and T_6 such that $\text{TS}(T_5) < \text{TS}(T_6)$. One possible schedule for T_5 and T_6 is shown in Figure 10.13. For maintaining simplicity, only the read and write operations of the transactions are shown in this schedule.

When T_5 issues $\text{read}(R)$ operation, it is observed that $\text{TS}(T_5) < \text{write_TS}(R)$. This is due to the reason that $\text{write_TS}(R) = \text{TS}(T_6)$, hence, the read operation of T_5 is rejected and T_5 is rolled back. This rollback is required in basic timestamp ordering but it is not always necessary.

Basic timestamp ordering executes the conflict operations in timestamp order; hence, it ensures conflict serializability. In addition, it is deadlock-free since no transaction ever waits. However, cyclic restart of a transaction may occur in basic timestamp ordering, if it is repeatedly rolled back and restarted. This cyclic restart of the transaction leads to the starvation of the transaction. If the restarting of a transaction occurs several times, conflicting transactions need to be temporarily blocked which allows the transaction to finish.

T_5	T_6
	$\text{read}(R)$
	$\text{write}(R)$
$\text{read}(R)$	
$\text{read}(Q)$	
	$\text{read}(Q)$
	$\text{write}(Q)$

Fig. 10.13 Schedule for T_5 and T_6

Strict Timestamp Ordering

The **strict timestamp ordering** is a variation of basic timestamp ordering. In addition to the basic timestamp ordering constraints, it follows another constraint when the transaction T_i requests read or write operations on some data item. This constraint ensures a strict schedule, which guarantees recoverability in addition to serializability. Suppose a transaction T_i requests a read or write operation on Q and $\text{TS}(T_i) > \text{write_TS}(Q)$, T_i is delayed until the transaction, say T_j , that wrote the value of Q has committed or aborted.

The strict timestamp ordering is implemented by locking Q that has been written by transaction T_j until it is either committed or aborted. Furthermore, the strict timestamp ordering ensures freedom from deadlock, since T_i waits for T_j only if $\text{TS}(T_i) > \text{TS}(T_j)$.

Thomas' Write Rule

Thomas' write rule is the modification to the basic timestamp ordering, in which the rules for write operations are slightly different from those of basic timestamp ordering. The rules for a transaction T_i that request a write operation on data item Q are given here.

1. If $TS(T_i) < read_TS(Q)$, the write operation is rejected and T_i is rolled back. This is because another transaction with higher timestamp (that is, younger transaction) has already read the value of Q , which T_i needs to write. So, the value of Q that T_i is producing was previously needed, and it had been assumed that the value would never be produced.
2. If $TS(T_i) < write_TS(Q)$, the write operation can be ignored. This is because another transaction with higher timestamp (that is, younger transaction) has immediately overwritten the value of Q , which T_i wrote. So, no transaction can read the value written by T_i and hence, T_i is trying to write an obsolete value of Q .
3. If both the conditions given in points 1 and 2 do not occur, the write operation of T_i is executed and $write_TS(Q)$ is set to $TS(T_i)$.

The main difference between these rules and those of basic timestamp ordering is the second rule. It states that if T_i requests a write operation on Q and $TS(T_i) < write_TS(Q)$, the write operation of T_i is ignored. Whereas, according to basic timestamp ordering, if T_i requests a write operation on Q and $TS(T_i) < write_TS(Q)$, T_i has to be rolled back.

Unlike other techniques discussed so far, Thomas' write rule does not enforce conflict serializability; however, it makes use of view serializability. It is possible to generate serializable schedules using Thomas' write rule that are not possible under other techniques like two-phase locking, tree-locking, etc. To illustrate this, consider the schedule shown in Figure 10.14. In this figure, the write operation of T_{10} succeeds the read operation and precedes the write operation of T_9 . Hence, the schedule is not conflict serializable.

Under Thomas' write rule, the $write(Q)$ operation of T_9 is ignored. Therefore, the schedule shown in Figure 10.15 is view equivalent to the serial schedule of T_9 followed by T_{10} .

T_9	T_{10}
read(Q)	
	write(Q)
	commit
write(Q)	
commit	

Fig. 10.14 A non-conflict serializable schedule

T_9	T_{10}
read(Q)	
	write(Q)
	commit
commit	

Fig. 10.15 A schedule under Thomas' write rule

10.6 OPTIMISTIC (OR VALIDATION) TECHNIQUE

All the concurrency control techniques, discussed so far (locking and timestamp ordering) result either in transaction delay or transaction rollback, thereby named as pessimistic techniques. These techniques require performing a check before executing any read or write operation. For instance, in locking, a check is done to determine whether the data item being accessed is locked. On the other hand, in timestamp ordering, a check is done on the timestamp of the transaction against the read and write timestamps of the data item to determine whether the transaction can access the data item. These checks can be expensive and represent overhead during transaction execution as they slow down the transactions. In addition, these checks are unnecessary overhead when a majority of transactions are read-only transactions. This is because the rate of conflicts among these transactions may be low. Therefore, these transactions can be executed without applying checks and still maintaining the consistency of the system by using an alternative technique, known as **optimistic (or validation) technique**.



NOTE Optimistic technique is named so because the transactions execute optimistically and thereby assumes that few conflicts will occur among the transactions.

In optimistic concurrency control techniques, it is assumed that the transactions do not directly update the data items in the database until they finish their execution. Instead, each transaction maintains local copies of the data items it requires and updates them during execution. All the data items in the database are updated at the end of the transaction execution. In this technique, each transaction T_i proceeds through three phases, namely, *read phase*, *validation phase*, and *write phase*, depending on whether it is a read-only or an update transaction. The execution of transaction T_i begins with read phase, and the three timestamp values are associated with it during its lifetime. The three phases are explained here.

1. **Read phase:** At the start of this phase, transaction T_i is associated with a timestamp $\text{Start}(T_i)$. T_i reads the values of data items from the database and these values are then stored in the temporary local copies of the data items kept in the workspace of T_i . All modifications are performed on these temporary local copies of the data items without updating the actual data items of the database.
2. **Validation phase:** At the start of this phase, transaction T_i is associated with a timestamp $\text{Validation}(T_i)$. The system performs a validation test when T_i decides to commit. This validation test is performed to determine whether the modifications made to the temporary local copies can be copied to the database. In addition, it determines whether there is a possibility of T_i to conflict with any other concurrently executing transaction. In case any conflict exists, T_i is rolled back, its workspace is cleared and T_i is restarted.
3. **Write phase:** In this phase, the system copies the modifications made by T_i in its workspace to the database only if it succeeds in the validation phase. At the end of this phase, T_i is associated with a timestamp $\text{Finish}(T_i)$.

Using the value of the timestamp $\text{Validation}(T_i)$, the serializability order of the transactions can be determined by the timestamp ordering technique. Therefore, the value of timestamp $\text{Validation}(T_i)$ is chosen as the timestamp of T_i instead of $\text{Start}(T_i)$. This is because we expect that it provides faster response time, if conflict rates among transactions are indeed low.



In addition to transaction timestamp, the optimistic technique requires that the `read_set` and `write_set` of the transaction be maintained by the system. The `read_set` of the transaction is the set of data items it reads and the `write_set` of the transaction is the set of data items it writes.

For every pair of transactions T_i and T_j such that $\text{TS}(T_i) < \text{TS}(T_j)$, the generated schedule must be equivalent to a serial schedule in which T_i appears before T_j . In addition, this pair of transactions must hold one of the following validation conditions.

1. **$\text{Finish}(T_i) < \text{Start}(T_j)$:** This condition ensures that the older transaction T_i must complete its all three phases before transaction T_j begins its start phase. Thus, this condition maintains serializability order.
2. **$\text{Finish}(T_i) < \text{Validation}(T_j)$:** This condition ensures that T_i completes its write phase before T_j starts its validation phase. It also ensures that the `write_set` of T_i does not overlap with the `read_set` of T_j . In addition, the older transaction T_i must complete its write phase before younger transaction T_j finishes its read phase and starts its validation phase. This is due to the reason that writes of T_j are not overwritten by writes of T_i . Hence, it maintains the serializability order.

3. **Validation(T_i) < Validation(T_j)**: This condition ensures that T_i completes its read phase before T_j completes its read phase. More precisely, this condition ensures that the `write_set` of T_i does not intersect with the `read_set` or `write_set` of T_j . Since T_i completes its read phase before T_j completes its read phase, T_i cannot affect the read or write phase of T_j , which also maintains serializability.



NOTE The validation conditions ensure that modifications made by the younger transaction, say T_j , are not visible to the older transaction say T_i .

To validate T_j , the first condition is checked for each committed transaction T_i such that $TS(T_i) < TS(T_j)$. Only if the first condition does not hold, the second condition is checked. Further, if the second condition is false, the third condition is checked. If any one of the mentioned validation conditions holds, there is no conflict and T_j is validated successfully. If none of these conditions holds, there is conflict and the validation of T_j fails and it is rolled back and restarted. Observe that the first condition permits T_j to see the changes made to data items by T_i . The second condition permits T_j to n read data items when T_i is writing data items. However, since T_j does not read any data item modified by T_i , there is no conflict. The third condition permits both T_i and T_j to write data items at the same time, but the sets of data items written by the two transactions cannot overlap.

No transaction can be allowed to commit if any transaction is being validated. This is because the transaction, which is being validated, might overlook conflicts with regard to the newly committed transaction. Before validating other transactions, the write phase of a validated transaction must also be completed. So that the changes, if any, made by the validated transaction must be applied to the database.

It can be ensured that at most one transaction is in its validation/write phase at any time by using a synchronization mechanism such as a **critical section**. This mechanism may lead to lower degree of concurrency. So, it is necessary to keep these phases as short as possible. However, if the values of temporary local copies of data items have to be copied to the actual data items of the database, this can make the write phase long. Thus, an alternative approach that uses a level of indirection to shorten the write phase may be followed. This approach uses a logical pointer to access any data item and we simply change the logical pointer to point to the temporary local copies of data items in the write phase instead of copying the data items.

For example, consider the schedule given in Figure 10.16. Suppose that the transaction T_{11} starts before T_{12} , such that $TS(T_{11}) < TS(T_{12})$. Since the pair of transactions satisfies the validation conditions given earlier, the validation phase will be successful. Note that the values of temporary local copies of data items are copied to the actual data items of the

Things to Remember

To choose an optimistic or pessimistic concurrency control technique for a transaction depends on the type of transaction. The transactions for which recovery would be risky in case of a failure can be better managed with pessimistic techniques. While the transactions for which it is better to take the risk of failure rather than compromising the efficiency can be better managed with optimistic techniques.

T_{11}	T_{12}
read(Q)	read(Q) Q:=Q-200 read(R) R:=R+200
read(R) <Validate> display(Q+R)	<validate> write(Q) write(R) display(Q+R)

Fig. 10.16 A schedule under optimistic technique

database only after the validation phase of T_{12} . Furthermore, this schedule maintains serializability order because T_{11} reads the old values of Q and R.

Since the values of temporary local copies of data items are copied to the actual data items of the database only after the validation phase of T_{12} , cascading rollbacks cannot occur. However, starvation of long transactions can be possible due to the conflicts that occur because of short transactions. This conflict results in restarting the long transaction repeatedly. The starvation must be avoided by temporarily blocking the conflicting transactions so that the long transaction can complete its execution.

It is clear that optimistic concurrency control technique has certain overheads like locking technique. Following are the overheads in optimistic technique.

- ⇒ It maintains `read_set` and `write_set` for each transaction.
- ⇒ It checks for conflicts among the transactions.
- ⇒ It copies the values from the workspace of the transaction to the actual database.
- ⇒ It repeatedly restarts the transactions, which leads to wastage of work they have done so far.

10.7 MULTIVERSION TECHNIQUE

So far, we have discussed that serializability must be maintained when one or more of the transactions need to modify the data item. The serializability can be maintained either by delaying an operation or by aborting the requesting transaction. For example, if the appropriate value of the data item has not been written yet, a read operation may be delayed; or the transaction issuing the read operation must be rolled back in case the value that it was supposed to read has already been overwritten. However, these problems can be avoided by another concurrency control technique, which keeps the old version (or value) of each data item in the system when the data item is updated. This concurrency control technique is known as **multiversion technique**.

In this technique, several versions (or values) of a data item are maintained. When a transaction issues a write operation on the data item Q, it creates a new version of Q, the old version of Q is retained. When a transaction issues a read operation on the data item Q, an appropriate version of Q is chosen by concurrency control techniques in such a way that the serializability of the currently executing transactions be maintained.

The main drawback of the multiversion technique is that more memory space is required to keep multiple versions of the data items. However, old versions of the data items have to be maintained for recovery purposes. Out of several proposed multiversion techniques, this section discusses two multiversion techniques, namely, *multiversion technique based on timestamp ordering* and *multiversion two-phase locking*.

10.7.1 Multiversion Technique Based on Timestamp Ordering

For each version of a data item, say Q_i , system maintains the value of the version and associates the following two timestamps.

- ⇒ **read_TS(Q_i)** The **read timestamp** of Q_i ; this is the largest timestamp among all the timestamps of transactions that have successfully read Q_i .
- ⇒ **write_TS(Q_i)** The **write timestamp** of Q_i ; this is the timestamp of the transaction that has written the value of Q_i .

Consider a data item Q with the most recent version Q_i , that is $\text{write_TS}(Q_i)$ is the largest among all the versions. Further, assume a transaction T_i with $\text{write_TS}(Q_i) \leq \text{TS}(T_i)$. Now when T_i issues $\text{read}(Q)$ request, the system returns the value of Q_i and updates the value

of $\text{read_TS}(Q_i)$ with $\text{TS}(T_i)$ if $\text{read_TS}(Q_i) < \text{TS}(T_i)$. On the other hand, when T_i issues a $\text{write}(Q)$ request, the following situations may occur.

- ⇒ $\text{read_TS}(Q_i) > \text{TS}(T_i)$, which means any younger transaction has already read the value of Q_i . In this situation, T_i is rolled back.
- ⇒ $\text{TS}(T_i) = \text{write_TS}(Q_i)$. In this situation, the contents of Q_i are rewritten.
- ⇒ If none of the above situation holds, a new version, say Q_j , of Q is created with $\text{read_TS}(Q_i) = \text{write_TS}(Q_j) = \text{TS}(T_i)$.

The main advantage of the multiversion timestamp ordering technique is that read requests by the transaction are never blocked. Hence, it is important for typical database systems in which read requests are more frequent than write requests. On the other hand, there are two main disadvantages of this technique, which are given here.

- ⇒ Whenever a transaction reads a data item, $\text{read_TS}(Q_i)$ is updated. It results in accessing the disk twice, that is, one for data item and another for updating $\text{read_TS}(Q_i)$.
- ⇒ Whenever two transactions conflict, one of them is rolled back (rather than wait) in order to resolve the conflict, which could result in cascading and hence, can be expensive.



The multiversion based on timestamp ordering technique does not ensure recoverability and cascadelessness.

10.7.2 Multiversion Two-Phase Locking

In addition to read and write lock modes, **multiversion two-phase locking** provides another lock mode, that is, **certify**. In order to determine whether these lock modes are compatible with each other or not, consider Figure 10.17.

Requested mode	Shared	Exclusive	Certify
Shared	YES	YES	NO
Exclusive	YES	NO	NO
Certify	NO	NO	NO

Fig. 10.17 Compatibility matrix for multiversion two-phase locking

The term “YES” indicates that, if a transaction T_i holds the lock on data item Q with the mode specified in column header and another transaction T_j requests to acquire the lock on Q with the mode specified in row header, then the lock can be granted. This is because the requested mode is compatible with the mode of lock held. On the other hand, the term “NO” indicates that the requested mode is not compatible with the mode of lock held, so, the transaction that has requested the lock must wait until the lock is released.

Unlike locking technique, in multiversion two-phase locking, other transactions are allowed to read a data item while a transaction still holds an exclusive lock on the data item. This is done by maintaining two versions for each data item. One version, known as **certified version**, must be written by any committed transaction and second version, known as **uncertified version**, is created when an active transaction acquires an exclusive lock on a data item. The basic idea behind this technique is that transactions can read only the most recently certified version.

Suppose a transaction, T_j , requests a shared lock on a data item Q , on which another transaction T_i holds an exclusive lock. In this situation, T_j is allowed to read the certified version of Q while T_i is writing the value of uncertified version of Q . However, once T_i is ready to commit, it must acquire a certify lock on Q . Since certify lock is not compatible with other locks (see Figure 10.17), T_i must delay its commit until there is no transaction accessing certified version of the data item in order to obtain the certify lock. Once T_i acquires the certify lock on Q , the value of uncertified version becomes the certified version of Q and the uncertified version is deleted.

This technique has an advantage over two-phase locking that many read operations can execute concurrently with a single write operation on a data item, which is not possible under two-phase locking. This technique; however, has an overhead that the transaction may have to delay its commit until the certify locks are acquired on all the data items it has updated. This technique avoids cascading rollbacks because transactions read certified version instead of uncertified version. Whereas, deadlock may occur if a read lock is allowed to upgrade to a write lock, and they must be handled using the some deadlock handling technique.

10.8 DEALING WITH DEADLOCK

As discussed earlier, **deadlock** is a situation that occurs when all the transactions in a set of two or more transactions are in a simultaneous wait state and each of them is waiting for the release of a data item held by one of the other waiting transaction in the set. None of the transactions can proceed until at least one of the waiting transactions releases lock on the data item.

To get rid of this undesirable situation, system has to rollback some of the transactions involved in the deadlock. Various steps the system can take to deal with deadlock are discussed in this section.

Deadlock Prevention

Deadlock prevention ensures that deadlock never happens. Generally, this technique is used when the chances of occurring deadlock are high. Following are the approaches to prevent the deadlocks.

- ⇒ **Conservative 2PL:** In this approach, each transaction locks in advance all the data items that it needs during its life-time.
- ⇒ **Assigning an order to all the data items:** In this approach, an ordering is imposed on all the data items in the database. Each transaction acquires locks on the data items in a sequence consistent with that order.
- ⇒ **Using timestamps along with locking:** In this approach, each transaction is assigned a priority using a unique timestamp, which determines whether a transaction is allowed to wait or is rolled back. A lower timestamp denotes higher priority and vice-versa.

The first two approaches can be easily implemented if all the data items that are to be accessed by the transaction are known at the beginning. However, it is not a practical assumption because it is often difficult to predict what data items are needed by a transaction before it begins execution. In addition, both approaches limit concurrency since a transaction locks many data items that remain unused for a long duration. Thus, the third approach is mainly used for deadlock prevention.

To understand the third approach, consider a situation in which a data item Q is locked by a transaction T_i and another transaction T_j issues an incompatible lock request on Q . In this situation, two deadlock prevention techniques using timestamps have been proposed which are discussed here.

- ⇒ **Wait-die:** If T_i has a lower timestamp (that is, higher priority), T_i is allowed to wait. Otherwise, T_i is rolled back (*dies*).
- ⇒ **Wound-wait:** If T_i has a lower timestamp (that is, higher priority), T_j is rolled back (T_j is *wounded* by T_i); otherwise, T_i waits.



When a transaction is rolled back and restarted, it retains the same timestamp that it was originally assigned.

Both the wait-die and wound-wait techniques have some similarities, which are discussed here.

- ⇒ Starvation is avoided by both the wait-die and wound-wait techniques. In both the techniques, a transaction with the smallest timestamp is not rolled back. In addition, the new transactions are given higher timestamp than the older transactions. Thus, a transaction that is rolled back repeatedly will eventually become the oldest transaction and has the highest priority. Observe that the oldest transaction has the smallest timestamp. Therefore, it will not be rolled back again and will be granted all the locks that it had requested.
- ⇒ The request to acquire a lock on a data item held by another transaction does not necessarily involve a deadlock. Therefore, unnecessary rollbacks may occur in both wait-die and wound-wait techniques.

In spite of these similarities, there are certain important differences between the wait-die and wound-wait techniques, which are given in Table 10.1.

Table 10.1 *Wait-die and wound-wait technique*

Wait-Die Technique	Wound-Wait Technique
In this technique, the waiting that may be required by a transaction with higher priority (that is, older transaction) could be significantly higher.	In this technique, an older transaction gets the greater probability of acquiring a lock on the data item. It never waits for a younger transaction to release the lock on its data item.
In this technique, a transaction may be rolled back many times before acquiring the lock on the requested data item. For example, when a younger transaction T_i requests a data item Q held by an older transaction T_j , then it is rolled back. When it is restarted, it again requests a lock on Q . Now, if Q is still locked by T_j , T_i will be rolled back again. In this way, T_i may be rolled back several times till it acquires lock on Q .	Rollbacks in wound-wait technique are less as compared to wait-die technique. For example, when an older transaction T_i requests a data item Q held by a younger transaction T_j then T_j is rolled back. When it is restarted, it requests Q , which is now locked by T_i . In this situation, T_j waits.

Deadlock Detection and Recovery

If there is a little chance of interference among the transactions and the transactions are short and require few locks, we use deadlock detection and recovery techniques instead of deadlock prevention. To detect the deadlock, the system maintains a wait-for graph, which consists of nodes and directed arcs. The nodes of this graph represent the currently executing transactions, and there exists a directed arc from one node to another, if the transaction is waiting for another transaction to release a lock. If there exists a cycle in the wait-for graph, it indicates the deadlock in the system. To understand the creation of a wait-for graph, consider four transactions T_{13} , T_{14} , T_{15} , and T_{16} , whose partial schedule is given in Figure 10.18.

T_{13}	T_{14}	T_{15}	T_{16}
lock-S(Q)			
	lock-X(Q_1)		
	lock-S(Q_2)		
		lock-S(Q_4)	
			lock-X(Q_3)
		lock-X(Q_1)	
	lock-X(Q_3)		
lock-X(Q_2)			

Fig. 10.18 A partial schedule for T_{13} , T_{14} , T_{15} , and T_{16}

In this schedule, the transaction T_{13} is waiting for transactions T_{14} to release lock on the data item Q_2 . Similarly, T_{14} is waiting for T_{16} , and T_{15} is waiting for T_{14} . For this situation, the wait-for graph is represented in Figure 10.19.

Observe that there exists no cycle in this graph, thus, there is no deadlock. Now assume that the transaction T_{16} requests a lock on the data item Q_4 held by T_{15} , a directed arc is added in the wait-for graph from T_{16} to T_{15} (see Figure 10.20). The creation of this directed arc results in a cycle in the wait-for graph, thus, a deadlock occurs in the system in which the transactions T_{14} , T_{15} , and T_{16} are involved.

The wait-for graph is periodically examined by the system to check the existence of deadlock. Once the deadlock is detected, there is a need to recover from the deadlock. The simplest solution is to abort some of the transactions involved in the deadlock, in order to allow other transactions to proceed. The transactions chosen to be aborted are called **victim transactions**. Some criteria should be followed to choose victim transaction, like the transaction with fewest locks, the transaction that has done the minimum work, and so on.

Another approach for deadlock handling is to specify the time-interval for which a transaction is allowed to wait for acquiring a lock on a data item. If the time-out occurs, the transaction is rolled back and restarted.

Things to Remember

Invoking the deadlock detection algorithm at small intervals will add considerable overhead and if it is invoked at large intervals, the deadlock will not be detected for a long time. A number of factors, such as the frequency of deadlocks, the number of transactions affected by deadlock, waiting time of transactions, etc., may affect the choice of interval.

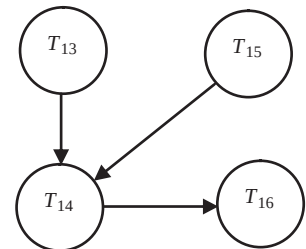


Fig. 10.19 Wait-for graph

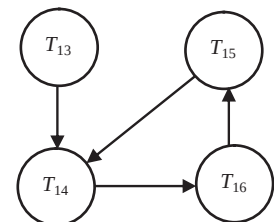


Fig. 10.20 Wait-for graph with cycle

Thus, in case of deadlock one or more transactions will time-out and roll back automatically, thereby, allowing other transactions to proceed. This approach lies somewhere in between deadlock prevention and deadlock detection and recovery. However, this approach has limited use in practical situations, since there is a chance of occurrence of starvation.

● SUMMARY ●

1. Concurrency control techniques are required to control the interaction among concurrent transactions. These techniques ensure that the concurrent transactions maintain the integrity of a database by avoiding the interference among them.
2. Whenever a data item is to be accessed by a transaction, it must not be modified by any other transaction. In order to ensure this, a transaction needs to acquire a lock on the required data items.
3. A lock is a variable associated with each data item that indicates whether read or write operations can be applied to it. Acquiring the lock by modifying its value is called locking.
4. Database systems mainly use two modes of locking. They are exclusive locks and shared locks. Exclusive lock provides a transaction an exclusive control on the data item. Shared lock can be acquired on a data item when a transaction wants to only read a data item and not modify it.
5. The locking or unlocking of the data items is implemented by a subsystem of the database system known as lock manager.
6. The lock manager maintains a linked list of records for each locked data item in the order in which the requests arrive. It uses a hash table known as lock table, which is indexed on the data item identifier.
7. Two-phase locking requires that each transaction be divided into two phases. During the first phase, the transaction acquires all the locks; during the second phase, it releases all the locks. The phase during which locks are acquired is a growing (or expanding) phase. On the other hand, a phase during which locks are released is a shrinking (or contracting) phase.
8. In strict two-phase, a transaction does not release any of its exclusive-mode locks until that transaction commits or aborts.
9. In rigorous two-phase locking, a transaction does not release any of its locks (both exclusive and shared) until that transaction commits or aborts.
10. A transaction is also permitted to change the lock from one mode to another on the data item on which it already holds a lock. This is known as lock conversion.
11. In tree locking, a transaction can acquire only exclusive locks on data items. Transactions are allowed to unlock a data item at any time; however, a data item released once cannot be relocked.
12. In predicate locking technique, all the tuples (whether existing or new tuples that are to be inserted) that satisfy an arbitrary predicate are locked to avoid phantom problem.
13. There are two techniques for concurrency control in tree-structured indexes like B⁺-trees. The first technique is crabbing. The second technique uses a variant of the B⁺-tree called B-link tree, in which every node of the tree maintains a pointer that refers to its right sibling.
14. Locking granularity is the size of the data item that the lock protects. Locking a large data item such as either an entire relation or a database is termed as coarse granularity, whereas, locking a small data item such as either a tuple or an attribute is termed as fine granularity.
15. In intention lock, a transaction intends to explicitly lock a lower level of the tree. When intention-mode is associated with shared-mode, it is called intention-shared (IS) mode, and when it is associated with exclusive-mode, it is called intention-exclusive (IX) mode.
16. Timestamp-based concurrency control is a non-lock concurrency control technique; hence, deadlocks cannot occur. Timestamp is a unique identifier assigned to each transaction in the order it begins.

17. A number of implementation techniques based on the timestamp ordering have been proposed for concurrency control techniques, which ensure that all the conflicting operations of the transactions are executed in timestamp order. Three of them are basic timestamp ordering, strict timestamp ordering, and Thomas' write rule.
18. In optimistic concurrency control techniques, it is assumed that the transactions do not directly update the data items until they finish their execution.
19. In multiversion technique, several versions (or values) of a data item are maintained. When a transaction issues a write operation on the data item Q , it creates a new version of Q , the old version of Q is retained.
20. In multiversion two-phase locking, other transactions are allowed to read a data item while a transaction still holds an exclusive lock on the data item.
21. Deadlock is a situation that occurs when all the transactions in a set of two or more transactions are in a simultaneous wait state and each of them is waiting for the release of a data item held by one of the other waiting transaction in the set.
22. Different approaches to prevent deadlocks are conservative 2PL, assigning an order to data items and using timestamps along with locking.
23. To detect a deadlock, the system maintains a directed graph called wait-for graph. If there exists a cycle in the wait-for graph, it indicates the deadlock in the system.
24. One of the deadlocked transactions chosen to be aborted is termed as the victim transaction.

◀ ● KEY TERMS ● ▶

- | | |
|---|---|
| <ul style="list-style-type: none"> ● Concurrency control techniques ● Lock ● Locking ● Exclusive lock ● Shared lock ● Lock compatibility ● Lock status ● Lock manager ● Starvation ● Lock table ● Lock request ● Unlock request ● Deadlock ● Locking technique ● Two-phase locking ● Growing phase ● Shrinking phase ● Lock point ● Strict two-phase locking ● Rigorous two-phase locking ● Lock conversion ● Upgrading | <ul style="list-style-type: none"> ● Downgrading ● Graph-based locking ● Database graph ● Tree-locking ● Phantom problem ● Predicate locking ● Index locking ● Tree-structured indexes ● Crabbing ● B-linktree ● Multiple-granularity locking ● Locking granularity ● Coarse granularity ● Fine granularity ● Intention lock mode ● Intention-shared (IS) mode ● Intention-exclusive (IX) mode ● Shared and intention-exclusive (SIX) mode ● Resource contention ● Data contention ● Blocking ● Thrashing |
|---|---|

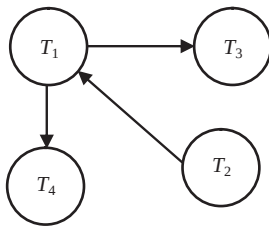
- Hotspots
- Timestamp-based technique
- Timestamp
- System lock
- Logical counter
- Timestamp ordering
- Read timestamp
- Write timestamp
- Basic timestamp ordering
- Cascading rollback
- Strict timestamp ordering
- Recoverability
- Thomas' write rule
- Optimistic or validation technique
- Read phase
- Validation phase
- Validation test
- Write phase
- Critical section
- Multiversion technique
- Certified version
- Uncertified version
- Multiversion timestamp ordering
- Multiversion two-phase locking
- Certify lock mode
- Deadlock prevention
- Conservative 2PL
- Ordering of data items
- Timestamp with locking
- Wait-die technique
- Wound-wait technique
- Deadlock detection and recovery
- Wait-for graph

● EXERCISES ●

A. Multiple Choice Questions

1. Strict two-phase locking does not ensure
 - (a) Cascadelessness
 - (b) Freedom from deadlock
 - (c) Serializability
 - (d) None of these
2. If a transaction T_i holds a lock on some data item Q in shared and intention-exclusive (SIX) mode, then which of the following is true for another transaction T_j ?
 - (a) T_j can lock Q in exclusive (X) mode
 - (b) T_j can lock Q in intention-exclusive (IX) mode
 - (c) T_j can lock Q in intention-shared (IS) mode
 - (d) T_j can lock Q in shared (S) mode
3. Strict timestamp ordering technique ensures
 - (a) Serializability
 - (b) Freedom from deadlock
 - (c) Recoverability
 - (d) All of these
4. Which of the following is not true in case of tree-locking?
 - (a) It does not ensure freedom from deadlock
 - (b) Unlike two-phase locking, transaction can unlock data items earlier
 - (c) It ensures shorter waiting times and greater amount of concurrency
 - (d) All of these
5. The number of modes in which a transaction may request a lock in multiple-granularity locking is
 - (a) Two
 - (b) Three
 - (c) Four
 - (d) Six

6. Which of the following is true for timestamp ordering?
 - (a) Basic timestamp ordering does not ensure conflict serializability
 - (b) Strict timestamp ordering ensures freedom from deadlock
 - (c) Thomas' write rule enforces conflict serializability
 - (d) None of these
7. Which phase is not a part of the optimistic technique?
 - (a) Execution phase
 - (b) Read phase
 - (c) Write phase
 - (d) Validation phase
8. The techniques used to handle the phantom problem are
 - (a) Predicate locking
 - (b) Index locking
 - (c) Timestamping
 - (d) all (a) and (b)
9. Consider the following wait-for graph.



Which of the following statement does not hold true for this wait-for graph?

- (a) Transaction T_1 is waiting for the data item locked by transaction T_4
 - (b) Transaction T_2 is waiting for the data item locked by transaction T_1
 - (c) If transaction T_2 requests for the data item locked by transaction T_3 , deadlock occurs
 - (d) If transaction T_4 requests for the data item locked by transaction T_2 , deadlock occurs
10. Which of the following timestamp is not associated with a transaction T?
 - (a) start (T)
 - (b) end (T)
 - (c) Validation (T)
 - (d) finish (T)

B. Fill in the Blanks

1. A set of rules that all the transactions in a schedule must follow is called _____.
2. A database system mainly uses two modes of locking, namely, _____ and _____.
3. The lock on a data item can be downgraded only in _____ phase.
4. In _____ locking, a transaction does not release any of its locks until it commits or aborts.
5. _____ and _____ techniques are used for concurrency control in tree-structured indexes.
6. _____ permits each transaction to use levels of locking that are most suitable for its operations.
7. In the compatibility matrix for different access modes, the highest privilege is given to _____ and the lowest privilege is given to _____.
8. The frequently accessed data items that remain in buffer for long period of time are known as _____.

9. _____ ensure that modifications made by the younger transaction are not visible to the older transaction.
10. Two deadlock prevention techniques that use timestamps are _____ and _____.

C. Answer the Questions

1. What is concurrency control? How is it implemented in DBMS? Explain.
2. What is deadlock? Explain how do you prevent and detect it. Give suitable examples.
3. Define the following.
 - (a) Lock
 - (b) Coarse granularity and Fine granularity
 - (c) Critical section
 - (d) Certified version
 - (e) Victim transaction
 - (f) Predicate locking
 - (g) Database graph
 - (h) Tree-locking
4. What do you understand by lock compatibility? Discuss in detail with examples.
5. How is locking implemented? What is the role of lock table in implementation? How are requests to lock and unlock a data item handled?
6. Explain the two-phase locking protocol with the help of an example. What are its disadvantages? How can these disadvantages be overcome? What is the benefit of rigorous two-phase locking?
7. How one can change the mode of lock over a data item? What do you understand by lock upgrade and lock downgrade?
8. Discuss the graph-based locking technique. What is the role of database graph?
9. What are the two factors that govern the performance of locking? Discuss how blocking of transactions can degrade the performance.
10. All schedules for a set of transactions under the tree locking are conflict serializable. Justify this statement by taking suitable example.
11. What is phantom problem for concurrency control and how index locking resolves this problem?
12. Discuss two techniques for concurrency control in tree-structured indexes like B⁺-trees.
13. What is multiple-granularity locking? Under what situations it is used?
14. What are intention locks? How does it provide higher degree of concurrency?
15. What is timestamp? How system generates timestamps? What are the values associated with timestamps?
16. What is timestamp ordering protocol for concurrency control? How is strict timestamp ordering protocol different from basic timestamp ordering?
17. How optimistic concurrency control techniques are different from other concurrency control techniques? Why they are also called validation techniques? Discuss the typical phases of an optimistic technique.
18. What is deadlock? Explain how do you prevent and detect it with the help of an example.
19. Discuss “wait-die” and “wound-wait” approaches of deadlock avoidance. Compare these approaches of deadlock prevention with a deadlock prevention approach in which data items are locked in a particular order.
20. How deadlock is detected by wait-for graph? What are the measures for recovery from deadlock? Explain.
21. Discuss multiversion technique, for what purpose is it used? Discuss multiversion timestamp ordering and multiversion two-phase locking. Compare multiversion two-phase locking with basic two-phase locking.

22. Write short note on the following.
- (a) Deadlock prevention protocols
 - (b) Locks and its types
 - (c) Intention lock modes
 - (d) Lock conversion
 - (e) Lock point
 - (f) Thomas' write rule
 - (g) Locking rules for multiple-granularity locking
23. Draw suitable graph for the following requests and find whether the transactions are deadlocked or not.

T_1	T_2	T_3
S_lock A	—	—
—	X_lock B	—
—	S_lock C	—
—	—	X_lock C
—	S_lock A	—
S_lock B	—	—
S_lock A	—	—
—	—	S_lock A
All the locking requests start from here		

DATABASE RECOVERY SYSTEM

After reading this chapter, the reader will understand:

- *Types of failures that can occur in a system*
- *Caching of disk pages containing data items*
- *How system log is maintained during normal execution so that the recovery process can be performed*
- *The process of checkpointing, which helps in reducing the time taken to recover from a crash*
- *The two categories of recovery techniques, namely, log-based recovery techniques and shadow paging*
- *The further classification of log-based techniques into two types, namely, techniques based on deferred update and techniques based on immediate update*
- *Recovery of a system with multiple concurrent transactions*
- *ARIES recovery algorithm*
- *Recovery of a system from catastrophic failures*

In every computer system, there is always a possibility of occurrence of a failure, which may result in loss of information. Therefore, if a failure occurs, then it is the responsibility of database management system (DBMS) to recover the database as quickly, and with as little damaging impact on users, as possible. The main aim of recovery is to restore the database to the most recent consistent state. The **recovery manager** component of DBMS is responsible for performing the recovery operations.

The recovery manager ensures that the two important properties of transactions, namely, *atomicity* and *durability* are preserved. It preserves atomicity by undoing the actions of uncommitted transactions and durability by ensuring that all the actions of committed transactions survive any type of failure. The recovery manager keeps track of all the changes that were applied to the data by various transactions and thus, deals with various states of the database. The recovery manager must also provide the **high availability** to its users, which requires the minimum levels of services and robustness in case of failures.

The recovery depends on the type of failure and the files of the database affected by the failure. This chapter first discusses the various types of failures that can cause abnormal termination of transactions, and then discusses the steps that the system takes during normal execution of transactions for recovery purpose. It then discusses how recovery is performed in case of system crashes. Finally, it discusses how the data is recovered in case of disk failures and natural disasters.

11.1 TYPES OF FAILURES

There are several types of failures that can occur in a system and stop the normal execution of the transactions. Each of them is handled in a different manner. Let us discuss these failures.

- ⇒ **Logical error:** Transactions may fail due to a logical error in the transaction such as incorrect input, integer overflow, division by zero, etc. Certain exceptional conditions that are not programmed correctly may also result in cancellation of the transaction. For example, insufficient balance in a customer account results in the failure of fund withdrawal transaction. If these types of exceptions are checked within the transaction itself, they do not result in transaction failure.
- ⇒ **System error:** Any undesirable state of the system such as deadlock, incorrect synchronization, etc., may also stop the normal execution of the transactions.
- ⇒ **Computer failure (system crash):** Any type of hardware malfunctioning such as RAM failure, error in application software, bug in operating system, and network problem can also bring the transaction processing to a halt. It is assumed that such a failure results in the loss of data in volatile storage and does not affect the contents of the non-volatile storage media such as disks. The assumption that the hardware and software errors bring only the system to a halt and has no effect on the contents of the non-volatile storage media is known as **fail-stop assumption**.
- ⇒ **Disk failure:** Disk failure, also known as **media failure**, refers to the loss of data in some disk blocks because of disk read/write head crash, power disruption or error in data transfer operation. These errors also cause the abnormal termination of the transaction if occurred during a read/write operation of the transaction.
- ⇒ **Physical problems and environment disasters:** The physical problems include theft, fire, sabotage, accidental overwriting of secondary storage media, etc. The environment disasters include floods, earthquakes, tsunami, etc.



NOTE *The transaction failure and system crash are more common types of failures as compared to disk failure and natural disasters.*

In case of a system crash, the information residing in the volatile storage such as main memory and cache memory is lost. Therefore, transaction failure and system crash are non-catastrophic failures as they do not result in the loss of non-volatile storage. On the other hand, disk failure and environment disasters are catastrophic failures as they result in the loss of non-volatile storage.

11.2 CACHING OF DISK PAGES

Whenever a transaction needs to update the database, the disk pages (or disk blocks) containing the data items to be modified are first cached (buffered) by the cache manager into the main memory and then modified in the memory before being written back to the disk. A **cache directory** is maintained to keep track of all the data items present in the buffers.

When an operation needs to be performed on a data item, the cache directory is first searched to determine whether the disk page containing the data item resides in the cache. If it is not present in the cache, the data item is searched on the disk and the appropriate disk page is copied in the cache. Sometimes it may be necessary to replace some of the disk pages to create space for the new pages. Any page-replacement strategy such as least recently used (LRU) or first-in-first-out (FIFO) can be used for replacing the disk page. Each memory buffer has a free bit associated with it which indicates whether the buffer is free (available for allocation) or not. Other associated bits are dirty bit and pin/unpin bit.

The data which is modified in the cache can be copied back to the disk using either of the two strategies, namely, *in-place updating* and *shadow paging*. In **in-place updating** strategy, single copy of the data items are maintained on the disk. The updated buffer is written back to the same original disk location and thus, the old value of any updated data item on the disk is overwritten by the new

value. However, in **shadow paging**, multiple copies of the data items can be maintained on the disk. The updated buffer is written at a different location on the disk and thus, the old value of the data item is not overwritten by the new value.

Standard DBMS recovery terminology includes the terms steal/no-steal and force/no-force, which specify when a modified page in the cache buffer can be written back to the database on disk.

- ⇒ **Steal:** In this approach, an updated cache page may be written to the disk before the transaction commits. It gives more freedom in selecting buffer pages to be replaced by the cache manager.
- ⇒ **No-steal:** In this approach, an updated cache page cannot be written to the disk before the transaction commits. The pin bit is set until the updating transaction commits.
- ⇒ **Force:** In this approach, all the updated cache pages are immediately written (force-written) to disk when the transaction commits.
- ⇒ **No-force:** In this approach, the updated cache pages are not necessarily written to disk immediately when the transaction commits.

Things to Remember

Steal/no-force approach is the most desirable approach in practice as it gives maximum degree of freedom to the cache manager in selecting the victim pages for replacement, and in scheduling the writes to disk.

11.3 RECOVERY RELATED STEPS DURING NORMAL EXECUTION

When a DBMS is restarted after a crash, the control is given to the recovery manager, which is responsible to bring the database to a consistent state. To enable it to perform its task in the event of failure, the recovery manager maintains some information during the normal execution of transactions. It maintains a system log of all the modifications to the database and stores it on the stable storage, which is guaranteed to survive failures. The stable storage is implemented by storing the database on a number of separate non-volatile storage devices such as disks or tapes (perhaps in different locations). Information residing in stable storage is assumed to be *never* lost (in real life *never* cannot be guaranteed).

The amount of work involved during recovery depends on the changes made by the committed transactions that have not been written to the disk at the time of crash. To reduce the time to recover from a crash, the DBMS periodically force-write all the modified buffer pages during normal execution. A process called **checkpointing** also helps in reducing the time taken to recover from a crash.

11.3.1 The System Log

During the normal transaction processing, the system stores relevant information in a log to make sure that enough information is available to recover from failures. A **log** is a sequence of **log records** that contains essential data for each transaction which has updated the database. The various types of log records that are maintained in a log are listed here.

- ⇒ **Start record:** The log record $[T_i, \text{start}]$ is used to indicate that the transaction T_i has started.
- ⇒ **Update log record:** The log record $[T_i, X, V_o, V_n]$ is used to indicate that the transaction T_i has performed an update operation on the data item X , having the old value V_o and new value V_n .
- ⇒ **Read record:** The log record $[T_i, X]$ is used to indicate that the transaction T_i has read the data item X from the database.
- ⇒ **Commit record:** The log record $[T_i, \text{commit}]$ is used to indicate that the transaction T_i has committed.
- ⇒ **Abort record:** The log record $[T_i, \text{abort}]$ is used to indicate that the transaction T_i has aborted, and needs to be rolled back.

Whenever a transaction needs to update a data item, two types of log entry information, namely, *UNDO-type log entry* and *REDO-type log entry* are maintained. The **UNDO-type log entry** includes the **before-image (BFIM)** of the data item which is used to undo the effect of an operation on the database. A **REDO-type log entry**, on the other hand, includes the **after-image (AFIM)** of the data item which is used to redo the effect of an operation on the database, in case the system fails before reflecting the new value of the data item to the database.



A **BFIM** is a copy of the data item prior to modification and an **AFIM** is a copy of the data item after modification.

Before making any changes to the database, it is necessary to force-write all log records to the stable storage. This is known as **write-ahead logging (WAL)**. In general, write-ahead logging protocol states that

1. A transaction is not allowed to update a data item in the database on the disk until all its UNDO-type log records have been force-written to the disk.
2. The transaction is not allowed to commit until all its REDO-type and UNDO-type log records have been force-written to the disk.

If the system fails, we can recover the consistent state of the database by examining the log and using one of the recovery techniques that are discussed in Section 11.4.



WAL is required only for in-place updating. In shadowing, since both BFIM and AFIM of the data item to be modified are kept on the disk, it is not necessary to maintain a log for recovery.

To understand how log records are maintained for a transaction, consider a transaction T_1 that transfers \$100 from account A to account B . Suppose accounts A and B have initial values \$2000 and \$1500, respectively. The transaction T_1 is given here.

```

 $T_1$ : read(A);
      A := A - 100;
      write(A);
      read(B);
      B := B + 100;
      write(B);

```

The log records for the transaction T_1 are shown in Figure 11.1.

For now, we assume that each time a log record for a transaction is created it is immediately written to the stable storage. However, writing each log record individually imposes an extra overhead on the system. To reduce this overhead, multiple log records are first collected in the log buffer in main memory and then copied to the stable storage in a single write operation. This is called **log record buffering**. Thus, a log record resides in the main memory for some time until it is written to the stable storage. The process of writing the buffered log to disk is known as **log force**.

```

[ $T_1$ , start]
[ $T_1$ , A]
[ $T_1$ , A, 2000, 1900]
[ $T_1$ , B]
[ $T_1$ , B, 1500, 1600]
[ $T_1$ , commit]

```

Fig. 11.1 Log records for transaction T_1



The order of log records in the stable storage must be exactly the same as that in the log buffer.

Note that during transaction rollback, only write operations are undone. The read operations are recorded in the log only to determine whether the cascading rollback is required or not. In practice, it is very complex and time-consuming to handle cascading rollback. Therefore, the recovery algorithms are designed in such a way that cascading rollback is never required. Hence, there is no need to record any read operations in the log. From now onwards, we will show the log records without the read operations.

11.3.2 Checkpointing

When the system restarts after a failure, the entire log needs to be scanned to determine the transactions that need to be redone and undone. The main disadvantage of this approach is that scanning of entire log is time-consuming. Moreover, the committed transactions that have already written their updates on the database also need to be redone as there is no way to find out the extent to which the changes of committed transactions have been propagated to the database. This causes recovery to take longer time.

To reduce the time for recovery by avoiding redo of the unnecessary work, checkpoints are used. The checkpoint approach is an additional component of the logging scheme. A **checkpoint** is periodically written into the log. It is typically written at a point when the system writes out all the modified buffers to the database on the disk. As a result, the transactions that have committed prior to the checkpoint will have their $[T_i, \text{commit}]$ record before the $[\text{checkpoint}]$ record. The write operations of these transactions need not be redone in case of a failure as all updates are already recorded in the database on disk. When a checkpoint is taken, the following actions are performed.

1. The execution of the currently running transactions is suspended.
2. All the log records currently residing in the main memory are written to the stable storage.
3. All the modified buffer blocks are force-written to the disk.
4. A $[\text{checkpoint}]$ record is written to the log on the stable storage.
5. The execution of suspended transactions is resumed.

If the number of blocks in the buffer are large, force-writing of all these pages may take long time to finish. The time taken to force-write all modified buffers can result in undesirable delay in the processing of current transactions. For reducing this delay, a technique called fuzzy checkpointing is used in practice. In fuzzy checkpointing, a $[\text{checkpoint}]$ record (known as **fuzzy checkpoint**) is written in the log before writing the modified buffer blocks to the disk. The system is then allowed to resume the execution of other transactions without having to wait for the completion of step 3. However, a system failure may occur while modified buffer blocks are being written to the disk. In that case, the fuzzy checkpoint would no longer be valid. Thus, the system is required to keep track of the previous checkpoint for which all the actions (1 to 5) have been successfully performed. For this, the system maintains a pointer to that checkpoint and it is updated to point to new checkpoint only when all the modified buffer blocks have been written to the disk.



The transactions committed before the checkpoint time need not be considered during recovery process. This reduces the amount of work required to be done during recovery.

11.4 RECOVERY TECHNIQUES

The recovery techniques are categorized mainly into two types, namely, *log-based recovery techniques* and *shadow paging*. The **log-based recovery** techniques maintain transaction logs to keep track of all update operations of the transactions. On the other hand, the **shadow paging** technique does not require the use of a log as both the after image and before image of the data item to be modified are maintained on the disk. This section discusses the recovery techniques for the situations where transactions execute serially one after the other and only one transaction is active at a time. Recovery in situations where multiple transactions execute concurrently is discussed in Section 11.5.

11.4.1 Log-based Recovery

The log-based recovery techniques are classified into two types, namely, techniques based on deferred update and techniques based on immediate update. In **deferred update** technique, the transaction is not allowed to update the database on disk until the transaction enters into the partially committed state. In **immediate update** technique, as soon as a data item is modified in cache, the disk copy is immediately updated. That is, the transaction is allowed to update the database in its active state.

To recover from a failure, basically two operations, namely, *undo* and *redo* are applied with the help of the log on the last consistent state of the database. The undo operation reverses (rolls back) the changes made to the database by an uncommitted transaction and restores the database to the consistent state that existed before the start of transaction.

The redo operation reapplies the changes of a committed transaction and restores the database to the consistent state it would be at the end of the transaction. The redo operation is required when the changes of a committed transaction are not or partially reflected to the database on disk. The redo modifies the database on disk to the new values for the committed transaction.

Sometimes the undo and redo operations may also fail due to any reason, and this type of failure can occur any number of times before the recovery is completely successful. Therefore, the undo and redo operations for a given transaction are required to be **idempotent**, which implies that executing any of these operations several times must be equivalent to executing it once. That is,

undo(operation)=undo(undo(...undo(operation)...))

redo(operation)=redo(redo(...redo(operation)...))

Recovery Techniques Based on Deferred Update

The deferred update technique records all update operations of the transactions in the log, but postpones the execution of all the update operations until the transactions enter into the partially committed state. Once the log records are written to the stable storage under WAL protocol, the database on the disk is updated.

If the transaction fails before reaching its commit point no undo is required as the transaction has not affected the database on the disk. In this case, the information in the log is simply ignored and the failed transaction must be restarted either automatically by the recovery process or manually by the user. However, redo operation is required in case the system fails after the transaction commits but before all the updates are reflected in the database on disk. In such situation, the log is used to redo all the transactions affected by this failure. Thus, this is known as **NO-UNDO/REDO recovery algorithm**. Note that the update log records, in this case, only contain the AFIM of the data item to be modified. The BFIM of the data item is omitted, as no undo operation is required in case of a failure. This technique uses **no-steal/no-force** approach for writing the modified buffers to the database on disk.

Learn More

Undoing the effect of only some (not all) uncommitted transactions is known as **partial undo**. Similarly, redoing the effect of only some committed transactions is known as **partial redo**.

For example, consider the transactions T_1 and T_2 given in Figure 11.2(a). The transaction T_1 transfers \$100 from account A to account B and transaction T_2 deposits \$200 to account C. Assume that the initial values of account A, B, and C are \$2000, \$1500, and \$500, respectively. Suppose that these two transactions are executed serially in the order T_1 followed by T_2 . The corresponding log records (only for write operations) are shown in Figure 11.2(b).

If the system fails before writing the log record $[T_1, \text{commit}]$, no redo is required as no commit record is found in the log. The values of account A and B remain \$2000 and \$1500, respectively. If the system fails after writing the log record $[T_1, \text{commit}]$, but before writing the commit record for the transaction T_2 , only $\text{redo}(T_1)$ operation is performed. If the system fails after writing the log record $[T_2, \text{commit}]$, both T_1 and T_2 need to be redone. In general, for each commit record $[T_i, \text{commit}]$ found in the log, the operation $\text{redo}(T_i)$ is performed by the system.

Recovery Techniques Based on Immediate Update

The immediate update technique allows a transaction to update the database on the disk immediately, even before it enters into partially committed state. That is, a transaction can update the database while it is in the active state. The data modifications made by the active transactions are known as **uncommitted modifications**. Like deferred update technique, immediate update technique also records the update operations in the log (on the stable storage) first so that the database can be recovered after a failure.

If the system fails before the transaction enters into partially committed state, all the changes made to the database by this transaction must be undone. However, if immediate update ensures that all the updates made by a transaction are recorded in the database on the disk before the transaction commits, redo operation is not required. Therefore, this recovery algorithm is known as **UNDO/NO-REDO recovery algorithm**. This technique uses **steal/force** approach for writing the modified buffers to the database on the disk.

The log records in this case contain only before-image of the data item. The after-image can be omitted, since no redo is required. The undo operation is performed by replacing the current value of a data item in the database by its BFIM stored in the log. For example, consider the transaction T_1 and T_2 given in Figure 11.2(a). The corresponding log records (only for write operations) for these transactions are shown in Figure 11.3.

If the system fails before writing the log record $[T_1, \text{commit}]$, the operation $\text{undo}(T_1)$ is performed. If the system fails after writing the log record $[T_1, \text{commit}]$ but before writing the commit record for the transaction T_2 , the operation $\text{undo}(T_2)$ is performed. The operation $\text{redo}(T_1)$ is not required as it is assumed that all updates of transaction T_1 are already reflected to the database on the disk. If the system fails after writing the log record $[T_2, \text{commit}]$, neither undo nor redo operation is required.

There is a possibility that immediate update allows a transaction to commit before all its changes are written to the database. In this case, both undo and redo operations may be required. Therefore, this algorithm

T_1	T_2
read(A)	
A := A - 100	
write(A)	
read(B)	
B := B + 100	
write(B)	
	read(C)
	C := C + 200
	write(C)

(a) Serial execution of transactions T_1 and T_2

```
[T1, start]
[T1, A, 1900]
[T1, B, 1600]
[T1, commit]
[T2, start]
[T2, C, 700]
[T2, commit]
```

(b) Log records for transactions T_1 and T_2 without BFIMs

Fig. 11.2 Transactions and their log records

```
[T1, start]
[T1, A, 2000]
[T1, B, 1500]
[T1, commit]
[T2, start]
[T2, C, 500]
[T2, commit]
```

Fig. 11.3 Log records for transaction T_1 and T_2 without AFIMs

is known as **UNDO/REDO recovery algorithm**. This technique uses **steal/no-force** approach for writing the modified buffers to the database on the disk. The UNDO/REDO algorithm is the most commonly used algorithm. The log records in this case contain both the BFIM and AFIM of the data item to be modified.

For example, consider the transaction T_1 and T_2 given in Figure 11.2(a). The corresponding log records (only for write operations) containing both BFIM and AFIM of the data items to be modified are shown in Figure 11.4.

If the system fails before writing the log record $[T_1, \text{commit}]$, the operation $\text{undo}(T_1)$ is performed. If the system fails after writing the log record $[T_1, \text{commit}]$ but before writing the **commit** record for the transaction T_2 , the operations $\text{redo}(T_1)$ and $\text{undo}(T_2)$ are performed. Here, the transaction T_1 should be redone as all updates of T_1 may not be reflected to the database on the disk. If the system fails after writing the log record $[T_2, \text{commit}]$, the operations $\text{redo}(T_1)$ and $\text{redo}(T_2)$ need to be performed.

```
[T1, start]
[T1, A, 2000, 1900]
[T1, B, 1500, 1600]
[T1, commit]
[T2, start]
[T2, C, 500, 700]
[T2, commit]
```

Fig. 11.4 Log records for transactions T_1 and T_2 with both BFIMs and AFIMs

11.4.2 ShadowPaging

The shadow paging technique is different from the log-based recovery techniques in a way that it does not require the use of log in an environment where only one transaction is active at a time. However, in an environment where multiple concurrent transactions are executing, the log is maintained for the concurrency control method. We will not discuss shadow paging in an environment with multiple concurrent transactions.

Shadow paging is based on making copies (known as **shadow copies**) of the database. This technique assumes that only one transaction is active at a time and thus, can be used as an alternative to log-based recovery technique in case the transactions are executed serially. This recovery technique is simple to implement but not as efficient as log-based recovery techniques.

Shadow paging considers the database to be made up of fixed-size logical units of storage called **pages**. These pages are mapped into physical blocks of storage with the help of **page table** (or **directory**). The physical blocks are of the same size as that of the logical blocks. A page table with n entries is constructed in which the i^{th} entry in the page table points to the i^{th} database page on the disk.

The main idea behind this technique is to maintain two page tables, namely, *current page table* and *shadow page table* during the life of a transaction. The entries in the **current page table** (or **current directory**) points to the most recent database pages on the disk. When a transaction starts, the current page table is copied into a **shadow page table** (or **shadow directory**). The shadow page table is then saved on the disk and the current page table is used by the transaction. The shadow page table is never modified during the execution of the transaction.

Initially, both the tables are identical. Whenever a write operation is to be performed on any database page, a copy of this page is created onto an unused page on the disk. The current page table is then made to point to this copy, and the update is performed on this copy. The shadow page table continues to point to the old unchanged page. The old copy of the page is never overwritten. This implies that for the pages updated by the transactions, two versions are kept. The shadow page table points to the old version and the current page table points to the new version of the page. Figure 11.5 illustrates the concept of shadow and current page table. In this figure, pages 1 and 3 have two versions. The shadow page table references the old versions and current page table references the new versions of page 1 and 3.

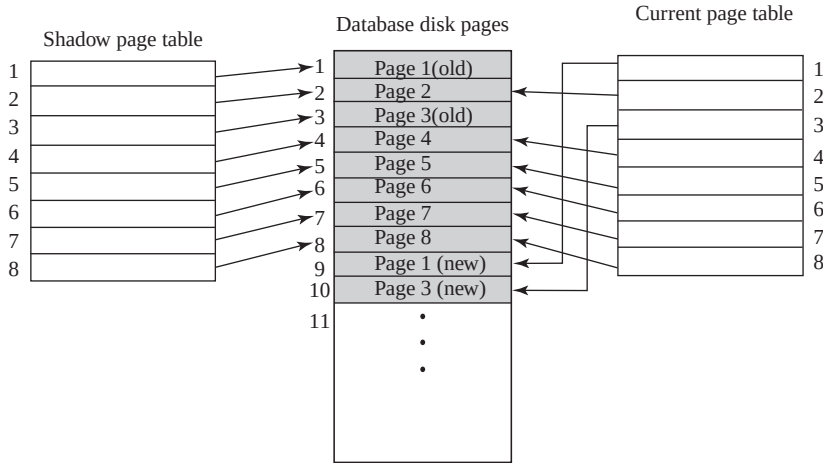


Fig. 11.5 Shadow paging

In case of a failure, the current page table is discarded and the disk blocks containing the modified disk pages are released. The previous state of the database prior to the transaction execution can be recovered from the shadow page table. In case no failure occurs and the transaction is committed, the shadow page table is discarded and the disk blocks with the old data are released. Since recovery in this case requires neither undo nor redo operations, this algorithm is known as **NO-UNDO/NO-REDO recovery algorithm**. This technique uses **no-steal/force** approach for writing the modified buffers to the database on the disk.

The shadow paging technique has several disadvantages. Some of them are discussed here.

- ⇒ **Data fragmentation:** Shadow paging technique causes the updated database pages to change locations on the disk. This makes it difficult to keep the related database pages together on the disk.
- ⇒ **Garbage collection:** Whenever a transaction commits, the database pages containing the old version of data are considered as garbage as they do not contain any usable information. Thus, it is necessary to find all of the garbage pages periodically and add them to the list of free pages. This process of finding garbage pages periodically is known as **garbage collection**. Though this process incurs an extra overhead on the system, but is necessary to improve the system performance.
- ⇒ **Harder to extend:** It is difficult to extend the algorithm to allow transactions to run concurrently.

11.5 RECOVERY FOR CONCURRENT TRANSACTIONS

The recovery techniques discussed so far handle the transactions in an environment where transactions execute serially. This section discusses how the log-based recovery algorithms can be extended to handle multiple transactions running concurrently. Note that regardless of the number of transactions, the system maintains a single log for all the transactions.

In case, where several transactions execute concurrently, the recovery algorithm may be more complex, depending on the concurrency control technique being used. In general, the greater the degree of concurrency, the more time-consuming the task of recovery becomes.

Consider a system in which strict two-phase locking protocol is used for concurrency control. To minimize the work of a recovery manager, checkpoints are also maintained at regular intervals. Two lists, namely, *active list* and *commit list* are maintained by the recovery system. All the active transactions T_A are entered in the **active list** and all the committed transactions T_C since the last checkpoint are entered in the **commit list**.

First, consider the case when UNDO/REDO recovery algorithm is used. During recovery process, the write operations of all the transactions in the commit list are redone in the order in which they were written to the log. The write operations of all the transactions in the active list are undone in the reverse of the order in which they were written to the log.

For example, consider five transactions T_1 , T_2 , T_3 , T_4 , and T_5 executing concurrently as shown in Figure 11.6. Suppose a checkpoint is made at time t_c and the system crash occurs at time t_f . During the recovery process, transactions T_2 and T_3 (present in commit list) need to be redone, as they are committed after the last checkpoint. Since the transaction T_1 is committed before the last checkpoint, its operations need not be redone. The transactions T_4 and T_5 (present in the active list) were not committed at the time of system crash and hence, need to be undone.

If NO-UNDO/REDO algorithm is followed, the transactions in the active list are simply ignored, as these transactions have not affected the database on the disk and hence, need not be undone. Similarly, if UNDO/NO-REDO technique is followed, the transactions in the commit list are ignored as their effects are already reflected in the database on disk.

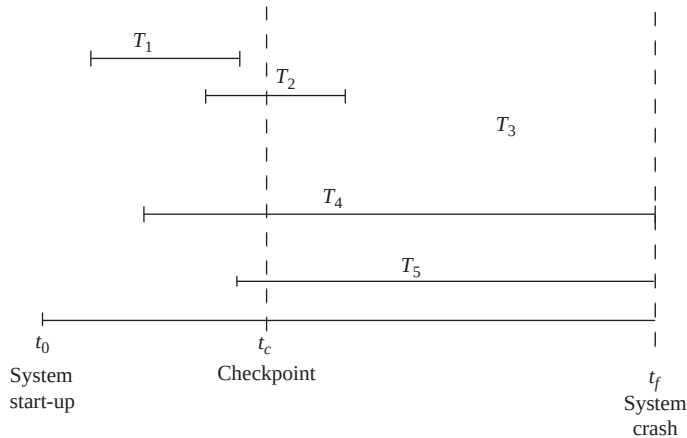


Fig. 11.6 Recovery with concurrent transactions

In case a data item Q is modified by different transactions, all the updates made to Q are overwritten by the last **write** operation. The NO-UNDO/REDO algorithm can be made more efficient by only redoing the last update of Q , rather than redoing all the update operations. In this situation, the log is scanned in the backward direction, starting from the end. As soon as a **write** operation on a data item Q is redone, Q is added in a list of redone items. Now, before applying a redo operation on any data item, it is first searched in the list of redone items. If the item is found in the list, it is not redone again as its last value has already been recovered.

11.6 ARIES RECOVERY ALGORITHM

Algorithm for Recovery and Isolation Exploiting Semantics (ARIES) is an example of recovery algorithm which is widely used in the database systems. It uses steal/no-force approach for writing the modified buffers back to the database on the disk. This implies that ARIES follows UNDO/REDO technique. The ARIES recovery algorithm is based on three main principles which are given here.

- ⇒ **Write-ahead logging:** This principle states that before making any changes to the database, it is necessary to force-write the log records to the stable storage.
- ⇒ **Repeating history during redo:** When the system restarts after a crash, ARIES retraces all the actions of database system prior to the crash to bring the database to the state which existed at

the time of the crash. It then undoes the actions of all the transactions that were not committed at the time of the crash.

- ⇒ **Logging changes during undo:** A separate log is maintained while undoing a transaction to make sure that the undo operation once completed is not repeated in case the failure occurs during the recovery itself, which causes restart of the recovery process.

In ARIES, each log record is assigned a unique id called the **log sequence number (LSN)** in monotonically increasing order. This id indicates the address of the log record on the disk. The ARIES algorithm splits a log into a number of sequential log files, each of which is assigned a file number. When a particular log file grows beyond a certain limit, a new log file is created and new records are added to that file. The new file is assigned a number higher by 1 than the previous log file. In this case, the LSN consists of a file number and an offset within the file. Every data page has a **pageLSN** field that is set to the LSN of the log record corresponding to the last update on the page. During the redo operation, the log records with LSN less than or equal to pageLSN of the page should not be executed as their actions are already reflected in the database.

The set of all log records for a particular transaction are stored in the form of linked list. Every log record includes some common fields such as previous LSN (**prevLSN**), transaction ID, and type of the log record. The **prevLSN** contains the address of the previous log record of the transaction. It is required to traverse the linked list in backward direction. The **transaction ID** field contains the id of the transaction for which the log record is maintained. The **type** field contains the type of the log record. The update log record also contains some additional fields such as **pageID** containing the data item, **length** of the updated item, its **offset** from the beginning of the page, and the **BFIM** and **AFIM** of the data item being modified.

In addition to the log, two tables, namely, *transaction table* and *dirty page table* are also maintained for efficient recovery. The **transaction table** contains a record for each active transaction. The record contains the information such as transaction ID, transaction status (active, committed, or aborted), and the LSN of the most recent log record (called **lastLSN**) for the transaction. The **dirty page table** contains a record for each dirty page in the buffer. Each record consists of a pageID and **recLSN**. The **recLSN** is the LSN of the first log record that has updated the page. This LSN gives the earliest log record that might have to be redone for this page when the system restarts after a crash. During normal operation, the transaction table and the dirty page table are maintained by the transaction manager and buffer manager, respectively.

The most recent portion of the log, which is kept in main memory, is called the **log tail**. The log tail is periodically forced-written to the stable storage. A log record is written for each of these actions.

- ⇒ updating a page (write operation)
- ⇒ committing a transaction
- ⇒ aborting a transaction
- ⇒ undoing an update
- ⇒ ending a transaction

The log records for update operation, and committing and aborting a transaction have already been discussed. For undoing an update operation, a **compensation log record (CLR)** is maintained that records the actions taken during the rollback of a particular update operation. It is appended to the log tail as any other log record. The CLR contains an additional field, called the **UndoNextLSN**, which contains the LSN of the log record that needs to be undone next, when the transaction is being rolled back. Unlike update log records, the CLRs describe the actions that have already been undone and hence, will not be undone again because we never need to undo an action that has already been undone. The number of CLRs written during undo is same as of the number of update log records for transactions that were not committed at the time of crash.

When a transaction is committed or aborted, some additional actions (like removing its entry from the transaction table, etc.) are taken after writing the commit or abort log record. When these actions are completed, the **end** (or **completion**) **log record** containing the transaction id is appended in the log. Thus, this record notes that a particular transaction has been committed or aborted.

A checkpoint is also maintained by writing a `begin_checkpoint` and `end_checkpoint` record in the log. The LSN of the `begin_checkpoint` record is written to a special file which is accessed during recovery to find the last checkpoint information. When an `end_checkpoint` record is written, the contents of both the transaction table and the dirty page table are appended to the end of the log. To reduce the cost of checkpointing, fuzzy checkpointing is used so that DBMS can continue to execute transactions during checkpointing.

The contents of the modified cache buffers need not be flushed to disk during checkpoint. This is because the transaction table and dirty page table, which are already appended to log on the stable storage, contain the information needed for recovery. Note that if the system fails during checkpointing, the special file containing the LSN of `begin_checkpoint` record of the previous checkpoint will be used for recovery.

When a system restarts after a crash, the ARIES recovers from the crash in three phases:

⇒ **Analysis phase:** This phase starts from the `begin_checkpoint` record and proceeds till the end of the log. When the `end_checkpoint` record in the log is encountered, the transaction table and dirty page table are accessed which were written in the log during checkpointing. During analysis, these two tables are reconstructed as follows:

- If an end log record for a transaction T_i is encountered in the transaction table, its entry is deleted from that table.
- If any other type of log record for a transaction T_j is encountered, its entry is inserted into the transaction table (if not present) and the last LSN is modified accordingly.
- If the log record specifying a change in page P is encountered, an entry is made for that page (if not already present) in the dirty page table and associated recLSN field is modified.

At the end of analysis phase, the necessary information for redo and undo phase has been compiled in transaction table and dirty page table.

⇒ **Redo phase:** The redo phase actually reapplies the updates from the log to the database. In ARIES, the redo operation is not applied to only the committed transactions, rather, a starting point from which the redo phase should start is determined first, and from this point the redo phase begins. The smallest LSN, which indicates the log position from where the redo phase needs to be started, is determined from the dirty page table. Before this point, the changes to dirty pages have already been reflected to the database on disk.

⇒ **Undo phase:** This phase rolls back all transactions that were not committed at the time of failure. A backward scan of the log is performed and all the transactions in the undo-list are rolled back. A compensating log record for each undo action is written in log. After undo phase, the recovery process is finished and normal processing can be started again.

To understand how ARIES works, consider three transactions T_1 , T_2 , and T_3 , where T_1 is updating page A, T_2 is updating pages B and C, and T_3 is updating page D. The log records at the point of crash are shown in Figure 11.7(a). The transaction table and dirty page table at the time of checkpoint and after the analysis phase are shown in Figures 11.7(b) and 11.7(c), respectively.

Learn More

IBM DB2, MS SQL Server, and Oracle 8 use Write-ahead logging scheme for recovery from a system crash. Out of these, IBM DB2 uses ARIES, while other use schemes that are more or less similar to ARIES.

The analysis phase starts from the `begin_checkpoint` record, which is at LSN 5 and continues until it reaches the end of the log. The transaction table and dirty page table are reconstructed during analysis phase as given below:

- ⇒ When the log record 7 is encountered, the status of transaction T_2 is changed to 'committed' in the transaction table.
- ⇒ When the log record 8 is encountered, new entry for the transaction T_3 is made in transaction table and an entry for page D is made in dirty page table.

During redo phase, the smallest LSN is determined from the dirty page table, which is 1. Thus, redo phase will start from the log record 1. The LSNs 1, 3, 4, and 8 indicate the change in page A , B , C , and D , respectively. Since these LSNs are not less than the smallest LSN, the updates on these pages need to be reapplied from the log.

During undo phase, the transaction table is scanned to determine the transactions that need to be rolled back. The transaction T_3 is the only transaction that was active at the point of crash. Thus, T_3 needs to be rolled back. The undo phase begins at log record 8 (for transaction T_3) and proceeds backward to undo all the updates performed by transaction T_3 . Since in our example, log record 8 is the only update operation performed by T_3 , it needs to be undone.

Trans_ID	LSN	Last_LSN	Type	Page_ID	Other_info
T_1	1	0	Update	A	...
T_1	2	1	Commit		...
T_2	3	0	Update	B	...
T_2	4	3	Update	C	...
	5	begin_checkpoint			
	6	end_checkpoint			
T_2	7	4	Commit		...
T_3	8	0	Update	D	...

(a) Log at the point of crash

Transaction table

Trans_ID	Last_LSN	Status
T_1	2	Commit
T_2	4	Active

Dirty page table

Page_ID	recLSN
A	1
B	3
C	4

(b) Transaction table and dirty page table at the time of checkpoint

Transaction table

Trans_ID	Last_LSN	Status
T_1	2	Commit
T_2	7	Commit
T_3	8	Active

Dirty page table

Page_ID	recLSN
A	1
B	3
C	4
D	8

(c) Transaction table and dirty page table after the analysis phase

Fig. 11.7 An example of ARIES recovery algorithm

ARIES has many advantages which are given here.

- ⇒ It is simple and flexible to implement.
- ⇒ It can support concurrency control techniques that involve locks of finer granularity. It also provides a variety of optimization techniques to improve concurrency control.
- ⇒ It uses a number of techniques to reduce logging overhead, overheads of checkpoints, and time taken for recovery.

11.7 RECOVERY FROM CATASTROPHIC FAILURES

The recovery techniques discussed so far are applied in case of non-catastrophic failures. The recovery manager should also be able to deal with catastrophic failures. Though, such failures are very rare, still some techniques must be developed to recover from such failures. The main technique used to handle such failures is **database backup** (also known as **dump**). In this technique, the entire contents of the database and the log are copied periodically onto a cheap storage medium such as magnetic tapes.

When a database backup is taken, the following actions are performed.

1. The execution of the currently running transactions is suspended.
2. All the log records currently residing in the main memory are written to the stable storage.
3. All the modified buffer blocks are force-written to the disk.
4. The contents of the database are copied to the stable storage.
5. A [dump] record is written to the log on the stable storage.
6. The execution of suspended transactions is resumed.

Note that all the steps except step 4 are similar to the steps used for checkpoints. Thus, the database backup and checkpointing are similar processes to an extent. Like fuzzy checkpoints, the technique called **fuzzy dump** is also used in practice. In fuzzy dump technique, the transactions are allowed to continue while the dump is in progress.

Since the system log is much smaller than the database itself, the backup of the log records is taken more frequently than the full database backup. The backup of the system log helps in redoing the effect of all the transactions that have been executed since the last backup. In case of disk failure the most recent dump of the database is loaded from the magnetic tapes to the disk. The system log is then used to bring the database to the most recent consistent state by reapplying all the operations since the last backup.

In case of environmental disasters such as flood, earthquakes, etc., the backup of the database taken on the stable storage at a remote site is used to recover the lost data. The copy of the database is made generally by writing each block of stable storage over a computer network. This is called **remote backup**. The site at which the transaction processing is performed is known as **primary site** and the remote site at which the backup is taken is known as **secondary site**. The remote site must be physically separated from the primary site so that any kind of natural disaster does not damage the remote site.

In case of natural disasters when the primary site fails, the remote site takes over the processing. But before that the recovery is performed at the remote site using the most recent backup of the data (may be outdated) and the log records received from the primary site. Once the recovery has been performed, the remote site can start the processing of the transactions.

Note that each time the updates are performed at the primary site; the updates must be propagated to the remote site so that the remote site remains synchronized with the primary site. This can be achieved by sending all the log records from the primary site to the remote site where the backup is taken.

 ● SUMMARY ●

1. The main aim of recovery is to restore the database to the most recent consistent state which existed prior to the failure. The recovery manager component of DBMS is responsible for performing the recovery operations.
2. The recovery manager ensures that the two important properties of transactions, namely, atomicity and durability are preserved.
3. There are several types of failures that can occur in a system and stop the normal execution of the transaction such as logical error, system error, computer failure (system crash), disk failure, physical problems, and environmental disasters.
4. Whenever a transaction needs to update the database, the disk pages containing the data items to be modified are first cached (buffered) by the cache manager into the main memory and then modified in the memory before being written back to the disk.
5. A cache directory is maintained to keep track of all the data items present in the buffers.
6. The data which is modified in the cache can be copied back to the disk using either of the two strategies, namely, in-place updating and shadow paging.
7. Standard DBMS recovery terminology includes the terms steal/no-steal and force/no-force, which specify when a modified page in the cache buffer can be written back to the database disk.
8. The recovery manager maintains a system log of all modifications to the database and stores it on the stable storage.
9. A log is a sequence of log records that contains essential data for each transaction which has updated the database.
10. In case of update operation, the log record contains the before and after images of the portion of the database which have been modified by the transaction.
11. Before making any changes to the database, it is necessary to force-write all log records on the stable storage. This is known as write-ahead logging (WAL).
12. The process in which multiple log records are first collected in the log buffer and then copied to the stable storage in a single write operation is called log record buffering. Writing the buffered log to disk is known as log force.
13. To recover from a failure, basically two operations, namely, undo and redo are applied with the help of the log on the last consistent state of the database.
14. The undo operation undoes (reverses or rollbacks) the changes made to the database by an uncommitted transaction and restores the database to the consistent state that existed before the start of transaction.
15. The redo operation redoes the changes of a committed transaction and restores the database to the consistent state it would be at the end of the transaction. The redo operation is required when the changes of a committed transaction are not or partially reflected to the database on disk.
16. The recovery techniques are categorized mainly into two types, namely log-based recovery techniques and shadow paging.
17. The log-based recovery techniques maintain transaction logs to keep track of transaction operations. The log-based recovery techniques are further classified into two types, namely, techniques based on deferred update and techniques based on immediate update.
18. In deferred update technique, the transaction is not allowed to update the database on disk until the transaction enters into the partially committed state.
19. In immediate update technique, as soon as a data item is modified in cache, the disk copy is immediately updated.
20. To reduce the time to recover from a crash, the DBMS periodically force-write all the modified buffer pages during normal execution using a process known as checkpointing.

21. The shadow paging technique does not require the use of a log as both the after image and the before image of the data item to be modified are maintained on the disk.
22. In case of recovery when concurrent transactions are executing, two lists, namely, active list and commit list are maintained by the recovery system. All the active transactions are entered in the active list and all the committed transactions since the last checkpoint are entered in the commit list.
23. Algorithm for Recovery and Isolation Exploiting Semantics (ARIES) is an example of recovery algorithm which is widely used in the database systems. It uses steal/no-force approach for writing the modified buffers back to the database on the disk.
24. In ARIES, each log record is assigned a unique id called the log sequence number (LSN) in monotonically increasing order. This id indicates the address of the log record on the disk.
25. ARIES is based on three principles including write-ahead logging, repeating history during redo, and logging changes during undo.
26. When a system restarts after a crash, the ARIES recovers from the crash in three phases—the first phase is the analysis phase, second phase is the redo phase, and the last phase is the undo phase.
27. To recover from catastrophic failures which result in loss of data in non-volatile storage, the backup of the database known as dump is used. The most recent dump of the database is loaded from the magnetic tapes to the disk. The system log is then used to bring the database to the most recent consistent state by reapplying all the operations since the last backup.
28. Remote backup systems provide high degree of availability, which allows transaction processing to continue even if the primary site is destroyed by any natural disaster such as fire, flood, or earthquake.

● KEY TERMS ●

- | | |
|---|---|
| <ul style="list-style-type: none"> ● Recovery manager ● Logical error ● System error ● System crash ● Fail-stop assumption ● Disk failure ● Cache directory ● In-place updating ● Shadow paging ● Steal ● No-steal ● Force ● No-force ● Log ● Log records ● Start record ● Update log record ● Read record ● Commit record | <ul style="list-style-type: none"> ● Abort record ● Undo-type log entry ● Before-image (B FIM) ● Redo-type log entry ● After-image (A FIM) ● Write-ahead logging (WAL) ● Log record buffering ● Log force ● Checkpoint ● Log-based recovery ● Deferred update ● Immediate update ● No-undo/redo recovery algorithm ● Uncommitted modifications ● Undo/no-redo recovery algorithm ● Undo/redo recovery algorithm ● Shadow copy ● Pages ● Page table |
|---|---|

- | | |
|---|--|
| <ul style="list-style-type: none"> • Current page table • Shadow page table • No-undo/no-redo recovery algorithm • Data fragmentation • Garbage collection • Active list • Commit list • ARIES recovery algorithm • Log sequence number (LSN) • Transaction table • Dirty page table | <ul style="list-style-type: none"> • Compensation log record (CLR) • End (or completion) log record • Analysis phase • Redo phase • Undo phase • Database backup (dump) • Fuzzy dump • Remote backup • Primary site • Secondary site |
|---|--|

◀ ● EXERCISES ● ▶

A. Multiple Choice Questions

- Which of these is a type of environmental disaster?

(a) Computer failure	(b) Power failure
(c) Logical error	(d) Disk failure
- Which of these strategies can be used for copying modified data in cache to the disk?

(a) In-place updating	(b) Out-place updating
(c) Page caching	(d) Both (a) and (b)
- Which of these terms is not included in DBMS recovery terminology?

(a) Steal	(b) Steal-force
(c) Force	(d) No-force
- Which of these is a type of log record maintained in a log?

(a) Start record	(b) Update log record
(c) Abort record	(d) All of these
- The UNDO/REDO recovery algorithm uses _____ approach for writing the modified buffers to the database on the disk.

(a) Steal/force	(b) No-steal/force
(c) No-steal/no-force	(d) Steal/no-force
- Which of these terms is associated with shadow paging?

(a) Shadow copies	(b) Page directory
(c) Current directory	(d) All of these
- The ARIES algorithm is not based on the principle of _____.

(a) Write-ahead logging	(b) Logging changes during undo
(c) Deferred update	(d) Repeating history during redo

8. Which of these is not a phase of ARIES algorithm?
 - (a) Analysis phase
 - (b) Logging phase
 - (c) Redo phase
 - (d) Undo phase
9. For which of the following actions a log record is written?
 - (a) Committing a transaction
 - (b) Aborting a transaction
 - (c) Undoing an update
 - (d) All of these
10. Which of these actions is not performed while taking database backup?
 - (a) All the modified buffer blocks are force-written to the disk
 - (b) The contents of the database are copied to the stable storage
 - (c) Continue with the execution of currently running transactions
 - (d) A [dump] record is written to the log on the stable storage

B. Fill in the Blanks

1. The assumption that the hardware and software errors bring only the system to a halt and has no effect on the contents of the non-volatile storage media is known as _____.
2. A log is a sequence of _____ that contains essential data for each transaction which has updated the database.
3. The UNDO-type log entry includes the _____ of the data item which is used to undo the effect of an operation on the database.
4. The _____ techniques maintain transaction logs to keep track of all update operations of the transactions.
5. In _____ technique, as soon as a data item is modified in cache, the disk copy is immediately updated.
6. In shadow paging, the entries in the _____ points to the most recent database pages on the disk.
7. In case of recovery when concurrent transactions are executing, the recovery manager maintains an _____ for all the active transactions and _____ for all the committed transactions since the last checkpoint.
8. In ARIES, each log record is assigned a unique id called the _____ in monotonically increasing order.
9. The _____ phase of ARIES algorithm actually reapplies the updates from the log to the database.
10. The site at which the transaction processing is performed is known as _____ and the remote site at which the backup is taken is known as _____.

C. Answer the Questions

1. What is recovery manager? How does it preserve the atomicity and durability of transactions?
2. What actions does a recovery manager perform during normal execution?
3. Discuss the various types of failures that can occur in a system. Differentiate between a system crash and disk crash.

4. Define the following terms:
 - (a) Cache directory
 - (b) Page table
 - (c) BFIM and AFIM
 - (d) Active list and commit list
 - (e) Uncommitted modifications
5. What is the difference between in-place updating and shadow paging?
6. Differentiate between steal/no-steal and force/no-force approaches.
7. What is a system log? What are its contents? How can a log be used for database recovery? Explain with the help of an example.
8. What is write-ahead logging protocol? Discuss the two types of log entry information that are maintained during an update operation of a transaction.
9. What is log record buffering? Why is it used?
10. What are checkpoints? How does fuzzy checkpointing affect the system performance? Explain with the help of an example.
11. If the system fails, can we recover the consistent state of the database? If yes, how? Explain.
12. What are undo and redo operations? Discuss the recovery techniques that use each of the operations. Why do these operations need to be idempotent?
13. What is the difference between deferred update and immediate update techniques?
14. Discuss the recovery techniques based on deferred update. Why is it called NO-UNDO/REDO method?
15. Briefly outline the UNDO/NO-REDO algorithm in an environment where transactions execute serially one after the other.
16. Discuss the UNDO/REDO recovery algorithm in both environments where transactions execute serially and where transactions execute concurrently.
17. Discuss shadow paging. How is it different from log-based recovery techniques? Mention some of its disadvantages.
18. What is ARIES? On what principles it is based? Discuss its three phases. What do you understand by LSN in ARIES?
19. What information do the transaction table and dirty page table in ARIES contain? How are they used in efficient recovery?
20. How is the database recovered from disk failures and natural disasters?
21. Consider the following log records for the transactions T_1 , T_2 , T_3 , T_4 , and T_5 . Describe the recovery process from a system crash if immediate update technique is followed. Also assume that the system maintains the checkpoints. Determine the list of transactions that need to be undone and that need to be redone.

```

[T1, start]
[T1, P, 20, 25]
[T1, Q, 15, 35]
[T1, commit]
[T2, start]
[T2, R, 50, 70]
[T2, P, 25, 16]
[T2, Q, 35, 18]
[T2, commit]
    
```

```
[checkpoint]
[T3, start]
[T3, R, 70, 45]
[T4, start]
[T4, P, 16, 22]
[T4, Q, 18, 25]
[T5, start]
[T5, Q, 25, 32]
[T5, R, 45, 25]
[T5, commit]
[T3, P, 22, 24]
System Crash
```

22. Consider the following log records containing only BFIMs of the data item being modified. Describe the recovery process from a system crash if deferred update technique is followed.

```
[T1, start]
[T1, P, 25]
[T1, Q, 35]
[T1, commit]
[T2, start]
[T2, R, 70]
[T2, P, 16]
[T2, Q, 18]
[T2, commit]
[checkpoint]
[T3, start]
[T3, R, 45]
[T4, start]
[T4, P, 22]
[T4, Q, 25]
[T5, start]
[T5, Q, 32]
[T5, R, 25]
[T5, commit]
[T3, P, 24]
System Crash
```

DATABASE SECURITY

After reading this chapter, the reader will understand:

- Importance of database security for organizations
- Security issues
- Types of threats that could adversely affect the database system
- Role of DBA in database security
- The role of database audit for finding the illegal and unauthorized operations
- Authorization, authorizer, access matrix, and authorization tree
- Various access privileges that a user can have on a relation
- Authentication and various methods to authenticate a user
- Access control and main approaches in DBMS for access control including discretionary access control (DAC), mandatory access control (MAC), and role-based access control (RBAC)
- Granting and revoking privileges to database users
- Implementing multilevel security using Bell–LaPadula model
- Multilevel relations and polyinstantiation
- Creating and destroying roles, and granting and revoking privileges to/from roles
- Access control policies for web applications
- The importance and use of encryption and decryption in communication
- Two encryption techniques, namely, symmetric key encryption and public key encryption
- Digital signatures
- Statistical databases and statistical queries

Database system manages large bodies of information, which is vital to organizations. Any unauthorized access or manipulation of the database, therefore, spells trouble for the organization. Thus, database should be protected from the persons who are not authorized to access either certain parts of the database or the whole database. In other words, protecting database against unauthorized access or manipulation is a major concern.

In this chapter, we discuss various security issues, the types of threats to the database, and the role of DBA in implementing database security. In addition, we discuss how the user is restricted to access only certain portion of the database.

This chapter also discusses how DBMS maintains secrecy of data during transmission. For this, encryption and decryption algorithms are discussed in Section 12.6. Finally, we discuss some aspects of security in specialized database, such as statistical database.

12.1 SECURITY ISSUES

The security of data is the most important, yet, least talked about issue that organizations face while implementing a database system. Database security covers the following issues.

- ⇒ **Privacy:** Only authorized persons should be allowed to access the database. In addition, only the part of the database that is required for the functions they perform should be available to them. In other words, users are allowed to access only the information that is pertinent to their jobs.
- ⇒ **Database integrity:** Database should be protected from improper modifications, either intentional or accidental, to maintain database integrity. Only the type of operations that need to be performed by the user should be allowed to them. For example, an employee who does not belong to accounts department should not be allowed to modify the balance sheet of the organization. The employees of accounts department only should be allowed to do so.
- ⇒ **Database availability:** Security should not restrict the authorized users to perform their actions on the part of the database available to them. For example, an accounts department employee should not be restricted to update the balance sheet.

There are various types of threats that adversely affect the database system. These threats can be classified into two categories, namely, *accidental* and *malicious or intentional threats*.

Accidental

- ⇒ Due to any system error, a user can get access to the portion of the database that should not be accessible to him.
- ⇒ Some people have good intentions and motives; yet do not have full understanding of what they are doing. Assume that an employee shares a password with his co-worker so that he can get something done, and regardless of his intention he runs an update query without proper WHERE clause or no WHERE clause at all. His actions can negatively affect the database.
- ⇒ Improper authorization can be accidentally assigned to a user by the authorizer. Such incidents could lead to security violations.
- ⇒ Concurrent usage of the database could lead to database inconsistency, if proper synchronization mechanism is not implemented.
- ⇒ Failure of the memory protection hardware could result in diversified response from the database.

Malicious or Intentional

- ⇒ Hackers, an ex-employee looking for revenge, or even a regular employee can get into the system parts that they are not authorized to access. They could then intentionally update or even delete the secret and precious information from the database.
- ⇒ Authorized user of an organization could give valuable information to the competitors for personal gain.
- ⇒ Programmers and/or developers of database development team could easily access database files, although not authorized to do so, and can update the data.

Threats, either accidental or intentional, can cause some extent of damage to the database. Impact of threats on the database heavily depends on the type of data affected. In order to protect database against these types of threats, various steps can be taken. Some measures are access control and encryption, which we discuss in this chapter.

Things to Remember

Though, DBMS includes a database security and authorization subsystem for ensuring security of database, some additional security features are also required by operating system. For instance, operating system must restrict any unauthorized user to directly access the files belonging to the database.

12.2 ROLE OF DBA IN DATABASE SECURITY

As discussed in Chapter 01, the database administrator (DBA) is the central authority that is responsible to manage the database system. The DBA is responsible for the overall security of the database system. The two main responsibilities of DBA are managing user accounts and performing database audit.

Managing User Accounts

It includes creation and deletion of accounts as well as granting and revoking privileges to/from the accounts. For this, the DBA has a **system** or **superuser account** in the DBMS, which provides powerful capabilities to control access to the database. He must restrict the user's access to the data so that the user, can perform only the necessary action on the portion of the database to which they are authorized. For this, the DBA deals with the following.

- ⇒ **Creation of user account:** Creates a new account and password for an individual user or a group of users to allow them to access the database. This step is used to control access to the database system as a whole.
- ⇒ **Granting of privileges:** Grants certain privileges to certain accounts.
- ⇒ **Revoking of privileges:** Revokes or cancels the privileges previously granted to certain accounts.
- ⇒ **Classification of user accounts:** Assign user accounts to different security classification levels.

Note that privilege is a kind of permission that allows users to access certain data items in a specified mode (like read, modify, or alter). Granting and revoking of privileges are used to control discretionary authorization, and classification of user accounts is used to control mandatory authorization.

Further, user accounts and passwords have to be kept confidential. The DBMS maintains an encrypted table with two fields, namely, **Account_Number** and **Password** to keep track of database users and their passwords. At the time of creation of a new account, a new entry is inserted into the table and at the time of deletion of an account, the corresponding entry must be deleted from the table.

Database Audit

In order to access the database, the user must log in to the DBMS by entering the account number and password. The DBMS verifies the account number and password and allows the user to access the database if the combination is found valid, otherwise access is denied. When a user logs in successfully, the DBMS records its account number and the terminal ID from where the user logs in. The DBMS keeps track of all the operations applied from that terminal and associate them with the user's account until the user logs off. Thus, in case, the database is tampered with any update operation, the DBA can find out the user who is responsible for that tampering.

To keep track of all the changes (insert/delete/update) applied to the database, system log that includes an entry for each update operation can be used. The entry in the system log can be further modified to include user's account number and terminal ID from where the changes have been applied. In case of database tampering, a review of log is carried out to examine all the operations applied to the database during certain period of time. This review of log is called **database audit**. It helps the DBA to find out the illegal or unauthorized operation and the account number used to perform this operation. A database log that is mainly used for database security is generally called an **audit trail**.

12.3 AUTHORIZATION

Authorization is the granting of privileges to users, which enable them to access certain portion of the database. Since database users perform different roles, they are permitted to access only the part of database that is pertinent to their job. In addition, each user is allowed to perform only the necessary

operation on the database that is required to perform their function. Such type of permission is given to database users by granting different privileges to them. The person who is responsible for granting privileges to database users is called **authorizer**. The DBA assign privileges to different users; however, the owner of relation can also assign privileges, since the owner has all the privileges on his relations.

The authorization information is maintained in a table called **access matrix**, which the database system consults each time an access request is issued in the system. The columns of access matrix represent **objects** and rows represent **subjects**. Here, object is any part of the database that needs to be protected from unauthorized access. For example, a file or a relation and its attribute can be considered as an object. Subject is an active user or account who operates on various objects in the database. An application program can also be considered as a subject. Each subject is given some access rights on some or all objects. The type of access that a subject has with respect to an object is represented by an entry in the access matrix at the intersection of the corresponding row and column. Figure 12.1 shows an example of access matrix with some access rights for listed subjects with respect to various objects.

SUBJECTS	OBJECTS		
	OBJECT 1	OBJECT 2	OBJECT 3
USER 1	Select Update	Select Insert	Select
USER 2	Insert Delete	Select Modify	Select Insert
USER 3	Select Drop Alter	Select Delete	Select Update Propagate
USER 4	Select Insert	Select Delete	Select References

Fig. 12.1 Access matrix

In terms of a relation as an object, a user can have following access privileges on a relation.

- ⇒ **SELECT:** A user having select privilege on relation R is allowed to select or retrieve tuples from R.
- ⇒ **MODIFY:** A user having modify privilege on relation R is allowed to modify the tuples of R. This privilege is further divided into three types, namely, INSERT, UPDATE, and DELETE. User having any of the privilege is allowed to apply the corresponding command to R. For instance, user having INSERT privilege is allowed to insert new tuples in R.
- ⇒ **REFERENCES:** A references privilege on R allows the user to reference the attribute of R while specifying integrity constraints. This privilege can also be restricted to some specific attributes of R.
- ⇒ **DROP:** A drop privilege allows the user to delete existing relation of the database.
- ⇒ **ALTER:** A user having alter privilege on relation R is allowed to add new attributes to the relation R. The user is also allowed to drop or delete the existing attributes of R.
- ⇒ **PROPAGATE ACCESS CONTROL:** A user having propagate right on relation R is allowed to pass all or part of his privileges on R to other users.

Note that, in addition to these privileges, a user may have the privileges to define indexes, execute certain application programs, and so on.

Authorization Tree

Propagate access control allows the users to pass either entire or part of their rights to other users, who may also be allowed to pass their rights as well. Such a series of authorization from one user to another user leads to a tree termed as **authorization tree**. Database users become the nodes in the tree with authorizer as the root node. Edge from one node to another node exists if rights are passed from one user to another user represented by these two nodes. For example, consider an authorization tree shown in Figure 12.2.

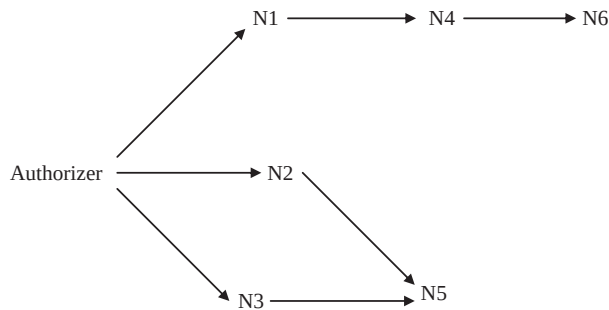


Fig. 12.2 Authorization tree

Here, authorizer assigns some access rights to the users represented by the nodes N1, N2, and N3. N1 assigns access rights to N4, which in turn assigns access rights to N6. Further, N2 and N3 together assign access rights to N5.

A user has some access rights, if there is any path from the root node (that is, authorizer) to the node representing that user. Thus, to revoke access rights from a user, all the incoming edges to that user in authorization tree should be removed. Returning to our example, suppose that authorizer revokes the access rights from N3. In this situation, N5 loses its access rights that are granted by N3; however, it retains those rights that are granted by N2. Further, if the authorizer revokes the access rights from N2 also, then N5 loses all of its access rights. However, if authorizer revokes access rights from N1, then both N4 and N6 lose their authorization, since both N4 and N6 has their access rights from N1.

12.4 AUTHENTICATION

Authorization allows the database users to access certain part of database. However, before accessing the database, users need to identify themselves to the system to confirm their correctness. The process of verifying the identity of a user is termed as **authentication**. Various methods can be used to authenticate a user, such as a secret password, some physical characteristics of the user, a smart card, or a key given to the user.

Password Authentication

It is the simplest and most commonly used authentication scheme. In this scheme, the user is asked to enter the user name and password to log in into the database. The DBMS then verifies the combination of user name and password to authenticate the user, and allows him the access to the database if he is the legitimate user, otherwise access is denied. Generally, password is asked once when a user log in into the database; however, this process can be repeated for each operation when the user is trying to access sensitive data.

Though, the password scheme is widely used by database systems, this scheme has some limitations. In this method, the security of database completely relies on the password. Thus, password itself needs to be secured from unauthorized access. One simple way to secure the password is to store them in an encrypted form. Further, care should be taken to ensure that password would never be displayed on the screen in its decrypted (non-encrypted) form. Encryption and decryption are discussed in detail in the Section 12.6.

Physical Characteristics of User

In this method, the physical characteristics of the users such as fingerprints, voiceprint, length of fingers of hand, face structure, and so on are used for the identification purpose. These characteristics of users are known to be unique, and have a very low probability of duplication. Thus, security of database is relatively high in this scheme as compared to the password scheme. However, this method requires the use of some special hardware and software to identify physical characteristics of the user, which incurs extra cost to the organization.

Smart Card

In this method, a database user is provided with a smart card that is used for identification. The smart card has a key stored on an embedded chip, and the operating system of smart card ensures that the key can never be read. Instead, it allows data to be sent to the card for encryption or decryption using that private key. The smart card is programmed in such a way that it is extremely difficult to extract values from smart card, thus, it is considered as a secure device.

12.5 ACCESS CONTROL

Restricting unauthorized access to the database is the main issue in developing secure DBMS. As discussed earlier, most database users need only a small portion of database to perform their job. Allowing them access to the whole database is undesirable. Thus, an organization should develop effective security policy to enable a group of users to access only a required portion of the database. Security policies of the organizations depend on the type of data maintained by them, hence, they vary from organization to organization. Further, once the security policy is developed, it should be enforced to achieve the level of security required. Two main approaches in DBMS for access control are *discretionary access control* and *mandatory access control*.

12.5.1 Discretionary Access Control

Discretionary access control (DAC) is enforced in a database system by granting and revoking privileges to/from the users. Different users have different access privileges on the object (either a base table or a view) of the database. GRANT and REVOKE commands of data manipulation language corresponds to grant and revoke privileges, respectively.

The syntax to grant some privileges to a user is

```
GRANT <privilege_list> ON <relation or view_name> TO <user_list>
[WITH GRANT OPTION];
```

Suppose DBA decides to allow USER1 to create relations. For this, he issues the following GRANTs statement.

```
GRANT createtab TO USER1;
```

After this, USER1 can create relations. Further, suppose he creates a relation R1 and decides to give some privileges to USER2 and issues the following statement.

```
GRANT SELECT, INSERT, UPDATE ON R1 TO USER2;
```

Since, the user who creates a relation automatically has all privileges on it; hence, the **USER1**—owner of the relation—is allowed to grant privileges to other users on that relation. The preceding statement allows **USER2** to retrieve tuples of relation **R1**. In addition, **USER2** can also insert new tuples into **R1** and can update any existing tuple. However, **USER2** is not allowed to propagate **SELECT**, **INSERT**, and **UPDATE** privileges on **R1** to other users.

Further, **USER1** issues the following statement to allow **USER3** to **SELECT**, **DELETE**, and **UPDATE** any tuple of **R1**. In addition, it enables **USER3** to further propagate its privileges on **R1** to other users (notice the **WITH GRANT OPTION**).

```
GRANT SELECT, DELETE, UPDATE ON R1 TO USER3 WITH GRANT OPTION;
```

It is also possible to restrict a user to access only few attributes or tuples of a relation. Following statement allow **USER4** to **UPDATE** only **A1** and **A2** attributes of relation **R1**.

```
GRANT UPDATE (A1,A2) ON R1 TO USER4;
```

Only **INSERT** or **UPDATE** privilege can be given on a set of attributes of a relation. **SELECT** and **DELETE** privilege on a set of attributes can easily be specified by creating views containing that set from the base relation and granting privilege on that view.

A user who creates a view has only those privileges on the view that he has on each base relations or views which were used to define the view. In addition, the privileges with respect to the view held by its creator changes over time as he gains or loses privileges on the underlying base relations or views. For example, suppose that the creator of a view loses **GRANT** privilege on the base relations of a view. It implies that all the users lose privileges on that view, since all of them were granted privileges from its owner. Similarly, if the creator of a view gains some more privileges on the base relations then he automatically gains additional privilege on the view also.

In order to create a view, the user must have **SELECT** privilege on all the base relations involved in creating the view. In case, the owner of a view loses **SELECT** privilege on the underlying base relations then the view may be dropped.

To cancel or revoke the previously assigned privileges from a user, the **REVOKE** command is used. The syntax to revoke some privilege from the user is

```
REVOKE <privilege_list> ON <relation or view_name> FROM <user_list>;
```

For example, the following statement revokes the privileges of **USER2** on relation **R1**.

```
REVOKE SELECT, INSERT, UPDATE ON R1 FROM USER2;
```

Similarly, the following **REVOKE** statement revokes the privileges of **USER4**.

```
REVOKE UPDATE (A1,A2) ON R1 FROM USER4;
```

Learn More

Techniques have been developed to limit the propagation of privileges, but still not implemented in most of the DBMSs. For example, horizontal propagation of n (an integer number) restricts the user with the **GRANT OPTION** on an object to propagate his privileges to at most n other users.

12.5.2 Mandatory Access Control

Mandatory access control (MAC) enforces multilevel security by classifying **subjects** (for example, database users, programs, etc.) and **objects** (for example, relations, views, etc.) of database system in different security classes. The **Bell–LaPadula** model is commonly used to implement multilevel security. In this model, each object is assigned a **security class** (for example, top secret, secret, confidential, and unclassified) and each subject has some **clearance** (generally the same as security classes). The class assigned to a subject or an object, say **Z**, is denoted as **class(Z)**. The security classes are assumed to form a hierarchy as top secret (TS) > secret (S) > confidential (C) > unclassified (U). It means the class

TS is the highest level and U is the lowest level. In other words, the data of class TS is much more sensitive than class S data and so on. This model imposes two rules on subjects to access objects.

1. Subject *S* is allowed to read object *O* if and only if the clearance level of *S* is greater than or equal to the classification level of *O*, that is $\text{class}(S) \geq \text{class}(O)$. This property is known as **simple security property**.
2. Subject *S* is allowed to write object *O* if and only if the clearance level of *S* is less than or equal to the classification level of *O*, that is $\text{class}(S) \leq \text{class}(O)$. This property is known as **star property**(*-property).

These two rules in combination with discretionary access control, enforce additional security for database. Thus, in order to retrieve or write an object, the subject should have necessary access privileges on the object and in addition, the security classes of subject, and object must satisfy the stated rules. The first rule or simple security property is very obvious, which states that a subject can read an object whose security classification is lower than or equal to the security clearance of subject itself. For example, a subject with clearance level TS can read object of all classes (including TS) but the subject should have required access privileges (through GRANT command) on that object. The second rule or star property restricts a subject from writing an object whose security classification is lower than the security clearance of the subject. For example, a subject with clearance S can write objects of class TS and class S, since object with TS or S classification is at higher or equal classification than that of subject clearance. However, subject with clearance S cannot write objects of class C and class U.

Multilevel Relations and Polyinstantiation

To implement mandatory access control in relational DBMS, attribute values and tuples are considered as objects and therefore, assigned a security class. Each attribute in a relation is associated with a **classification attribute**, say *Y*, and each attribute value is associated with a corresponding security classification. In addition, a **tuple classification attribute**, say *TY*, is added to the relation to classify each tuple as a whole. This situation leads to a **multilevel relation**. A multilevel relation schema *R* with *m* attributes would be represented as

$$R(A_1, Y_1, A_2, Y_2, \dots, A_m, Y_m, TY)$$

where, Y_i represents the classification attribute associated with attribute A_i , and *TY* is tuple classification attribute. The value of tuple classification attribute for each tuple is the highest level of all classification attribute value associated with the attributes within that tuple.

The set of attributes that would have served as the primary key in the regular relation is termed as an **apparent key** in a multilevel relation. For multilevel relations, entity integrity rules ensures that the set of attributes of apparent key must not be *null* and security classification of each attribute of apparent key should be same within each individual tuple. In addition, security classification of all other attributes' values must be greater than or equal to that of the apparent key. Thus, any user who is permitted to see any attribute of tuple is also allowed to see the key value of that tuple.

To understand this concept, consider the modified form of *Online Book* database in which only selected attributes and tuples are taken from PUBLISHER relation (see Figure 12.3) for simplicity. The apparent key of the PUBLISHER relation is *P_ID*, and classification attribute value of each attribute is written next to the attribute value.

P_ID	Pname	State	TY
P001 U	Hills Publication U	Georgia C	C
P002 C	Sunshine Publishers Ltd. S	New Jersey C	S
P003 S	Bright Publications S	Hawaii S	S

Fig. 12.3 An instance of PUBLISHER relation

In response to the query `SELECT * FROM PUBLISHER`, the users with clearance S and TS get all the rows, since all the tuple classifications are less than or equal to S and TS. However, the users with clearance C are not allowed to see the third row. Moreover, they are not allowed to see the value of attribute `Pname` in second row, since it has higher classification in second row. The value for the attribute `Pname` in second row would appear to be *null* [see Figure 12.4(a)]. Similarly, the users with clearance U get only first row, and in that the value of attribute `State` appears to be *null* [see Figure 12.4(b)].

P_ID	Pname	State	TY
P001 U	Hills Publication U	Georgia C	C
P002 C	NULL C	New Jersey C	C

(a)

P_ID	Pname	State	TY
P001 U	Hills Publication U	NULL U	U

(b)

Fig. 12.4 (a) Appearance for clearance level C. (b) Appearance for clearance level U

Further, assume that a user with clearance U is trying to insert a new row `<P002, Wesley Publications, Nevada, U>`. There arises a problem in choosing between the following two alternatives:

1. If the user is allowed to insert that row, the relation will contain two distinct tuples with `P_ID = "P002"`, which violates the primary key constraint.
2. On the other hand, if the user is denied to insert the new row because of violation of primary key constraint, the user with clearance U deduce that there exist a publisher with `P_ID = "P002"`, whose classification is higher than U. Doing so violates the principle that users should not be able to deduce any information about objects of higher classification level. This flow of information from higher classification level to a lower classification level, through indirect means, is called **covert channel**.

This problem can be solved by treating the combination of `P_ID` and `TY` as primary key for the relation, instead of just `P_ID`, and allowing the user to insert the new tuple. The relation instance after insertion is shown in the Figure 12.5.

P_ID	Pname	State	TY
P001 U	Hills Publication U	Georgia C	C
P002 U	Wesley Publication U	Nevada U	U
P002 C	Sunshine Publishers Ltd. S	New Jersey C	S
P003 S	Bright Publications S	Hawaii S	S

Fig. 12.5 Relation *PUBLISHER* after insertion



Note The two tuples having same value for the attribute `P_ID` is revealed to user according to their clearance level.

Multilevel relation has a surprising property that the users of different security classes view a different collection of tuples within the same relation. For example, in the relation instance, shown in the

Figure 12.5, two tuples have same value for apparent key but users with different clearance level view different values in other attributes for `P_ID = "P002"`. This concept is called **polyinstantiation**.

12.5.3 Comparing MAC and DAC

Discretionary access control policies are generally very effective and flexible. Due to their high degree of flexibility, they are well suited for large variety of applications. However, DAC suffers with certain weaknesses. Their vulnerability to malicious attacks, such as Trojan horse, is the major limitation of DAC. **Trojan horse** is a program to breach security of computer system. Using Trojan horse scheme, an unauthorized user can change application program of DBMS to infer sensitive data from authorized user. This is because the discretionary models do not impose any control on flow of information, once it has been accessed by authorized users. In contrast, mandatory access control policies overcome such problems of DAC. MAC control any illegal flow of information using the two rules discussed earlier. Thus, MAC ensures a high degree of protection, which makes them suitable for highly secure applications, such as military applications. However, due to their rigid nature, mandatory policies are applicable to a very few applications.

12.5.4 Role-Based Access Control

Role-based access control (RBAC) emerged as an alternative approach to traditional DAC and MAC. This approach restricts the system access to authorized users only. It appears to be a promising approach for controlling what information computer users can utilize, the programs that they can run, and the modifications that they can make. In RBAC, permissions to perform certain operations are associated with arbitrary roles and these roles are assigned to database users based on their responsibilities in the organization. Only the operations that need to be performed by the members of a role are granted to the role. Further, granting of user membership to roles can be limited. Some roles can only be occupied by a certain number of users at any given point of time. For example, the role of principal can be granted to only one user at a time.

Roles can be created and destroyed using `CREATE ROLE` and `DESTROY ROLE` commands, respectively. The `GRANT` and `REVOKE` commands can then be used to assign and revoke privileges from roles. Since, users are not assigned to the permissions directly; they acquire a set of permissions through their assigned roles. It reduces the amount of administrative effort required to add or delete accounts of database users. In addition, roles can be updated without updating the privileges for every user on an individual basis. Note that roles can be granted to users with the permission to pass on their role to other users. Database users establish sessions during which they are mapped to an activated subset of the set of roles they belong to.

Under RBAC, roles can have overlapping responsibilities and permissions, that is, users belonging to different roles may need to perform common operations. In this situation, it would be inefficient and administrative cumbersome to repeatedly specify those operations for each role. Alternatively, role hierarchies can be established to provide the natural structure of an organization. In the role hierarchy, a role may contain other roles that means, one role implicitly includes the operations that are associated with another role. Usually, roles in the hierarchy include the operations that are associated to the lower level roles.

The RBAC model is policy neutral that means individual RBAC policy can support a mandatory policy, while other can support discretionary policy. In addition, RBAC can also support user-defined or organization-specific policies. Several other desirable features of RBAC such as flexibility, better management, and administrative support make them suitable for implementing security plan for web-based applications.

12.5.5 Access Control Policies for Web Applications

With the rapid growth of web-based applications for commercial purpose, there is a need to develop security standards for them. Usually, the set of authorizations developed by DBA for access control is well suited for traditional database applications. However, such policy is not well applicable for dynamic web applications, since data must also be protected during its transmission over insecure links. For example, transmission of credit card number for an electronic transaction over Internet must be secure.

Various content publishing sites are available over Internet that enables Internet users to share their knowledge, experience, and opinions. Thus, not only data but also their views must be protected in such type of applications. Hence, a prime importance is the support for content-based access control. In **content-based access control**, access to an object partially or entirely depends on its contents in the system. Note that the policy for one object can depend on the contents of other objects in the system. For example, the policy depends on the contents of objects owned by the user requesting access.

Another related requirement is the heterogeneity of the subjects, in which access control policies are based on user's qualifications and characteristics. Suppose, there is an online catalog document that lists available goods sold on the Internet. The access control policy requires that only premium members can view the special discount price information. When a regular member views the document, any such information provided for premium members should be hidden.

It is common to develop web documents using Extensible Markup Language (XML), thus, XML can be very effective in the access control policy for web applications. An XML-based language is developed to specify security policies to be enforced on specific accesses to XML documents. This language is called as **XML access control language (XACL)**. It provides a sophisticated access control mechanism that enables secure browsing and updating of XML documents. In addition, major approach to protect data during transmission is encryption of data before transmission and then decryption of the data to recover original data at the destination.

12.6 ENCRYPTION

Encryption is a means of maintaining security of data in an insecure environment, such as while transmitting data over an insecure communication link. It involves applying an **encryption algorithm** to the data or **plaintext** using a pre-specified **encryption key**. The algorithm produces encrypted version of the plaintext as the output. For example, assume that the encryption algorithm substitutes each character of plaintext with the next third character in the alphabetical sequence. In this algorithm, three is the key for circularly shifted alphabets. Thus, we have the following substitution format.

```
plaintext: a b c d e f g h i j k l m n o p q r s t u v w x y z
ciphertext: d e f g h i j k l m n o p q r s t u v w x y z a b c
```

By this substitution, the text *principle* becomes *sulqflsoh*. It is difficult for an unauthorized user or intruder to understand the meaning of *sulqflsoh*. This encrypted data needs to be decrypted using a **decryption algorithm** to recover the original data. The decryption algorithm requires **decryption key** to decrypt the encrypted data. Note that without correct

Learn More

Over two thousand years ago, Julius Ceasar, a commander of Roman army, used encryption to convert his message into ciphertext by a substitution of three so that he could communicate with his troops securely.

decryption key, the algorithm cannot produce the original data. However, by looking at the frequently occurring pattern and by making guesses at common letters, intruders may be able to guess what substitution is being made. Thus, a good encryption algorithm should have the following features:

- ⇒ Security should not depend only on the secrecy of the algorithm, rather it should also depend on the encryption key.
- ⇒ It should be simple for authorized user to encrypt and decrypt the data.
- ⇒ It should be extremely difficult for an unauthorized user to guess the encryption key.

Most encryption algorithms belong to one of the two categories, namely, *symmetric key encryption*, or *public key encryption*.

12.6.1 SymmetricKey Enc ryption

The type of encryption in which the same key is used for both encryption and decryption of data is called **symmetric key encryption**. Data Encryption Standard (DES) is a well known example of symmetric key encryption. In 1977, the U.S. government developed DES, which was widely adopted by the industry for use in security products. The DES algorithm is parameterized by a 56-bit encryption key. The DES algorithm has a total of 19 distinct stages and encrypts the plaintext in blocks of 64 bits, producing 64 bits of ciphertext. The first stage is independent of key and performs transposition on the 64-bit plaintext. The last stage is exact inverse of first stage transposition. The preceding stage of last one exchanges the first 32 bits with the next 32 bits. The remaining 16 stages perform encryption by using parameterized encryption key. Since, the algorithm is symmetric key encryption; it allows decryption to be done with the same key as encryption. All the steps of algorithm are run in the reverse order to recover the original data.

With the increasing speed of computer, it is estimated that a special-purpose chip can crack DES in under a day by searching 2^{56} possible keys. Thus, after questioning the adequacy of DES, the National Institute of Standards and Technology (NIST) adopted **Rijndael algorithm** (by V. Rijmen and J. Daemen) in 2000 as a new symmetric encryption standard called **advanced encryption standard (AES)**. It supports key lengths of 128, 192, and 256 bits and specifies the block size of 128 bits. Since the key length is 128-bit, there are 2^{128} possible keys. It is estimated that a fast computer that can crack DES in 1 second could take trillion of years to crack 128-bit AES key. The main problem with symmetric algorithm is that the key must be shared among all the authorized users. This increases the chance of key becoming known to an intruder.

12.6.2 PublicKey Enc ryption

In 1976, Diffie and Hellman introduced a new concept of encryption called **public key encryption**. It is based on mathematical functions rather than operations on bit patterns. Unlike DES and AES, it uses two different keys for encryption and decryption. These two keys are referred to as the **public key** (used for encryption) and the **private key** (used for decryption). Each authorized user has a pair of public key and private key. The public key of each user is known to everyone, whereas, the private key is known to its owner only, thus, avoiding the weakness of DES. Assume that E and D represent the public encryption key and the private decryption key, respectively. It must be ensured that deducing D from E should be extremely difficult. In addition, the plaintext that is encrypted using the public key E_i requires the private key D_i to decrypt the data.



NOTE *The keys are referred to as public key and private key instead of secret key (the key used in conventional encryption) to distinguish them from conventional symmetric-key encryption.*

Now suppose that a user A wants to transfer some information to user B securely. The user A encrypts the data by using public key of B and sends the encrypted message to B. On receiving encrypted message, B decrypts it by using his private key. Since decryption process requires private key of user B, which is known only to B, the information is transferred securely. RSA is a well known example of public key encryption.

RSA Algorithm

In 1978, a group at M.I.T. discovered a strong method for public key encryption. It is known as **RSA**, the name derived from the initials of the three discoverers: Ron Rivest, Adi Shamir, and Len Adleman. It is the most widely accepted public key scheme, in fact most of the practically implemented security is based on RSA. The algorithm requires keys of at least 1024 bits for good security. This algorithm is based on some principles from number theory, which states that determining the prime factors of a number is extremely difficult. The algorithm follows the following steps to determine the encryption key and decryption key.

1. Take two large distinct prime numbers, say m and n (about 1024 bits).
2. Calculate $p = m * n$ and $q = (m - 1) * (n - 1)$.
3. Find a number which is relatively prime to q , say D . That number is decryption key.
4. Find encryption key E such that $E * D = 1 \text{ mod } q$.

Using these calculated keys, a block B of plaintext is encrypted as $T_e = B^E \text{ mod } p$. To recover the original data, compute $B = (T_e)^D \text{ mod } p$. Note that E and p are needed to perform encryption, whereas, D and p are needed to perform decryption. Thus, the public key consists of (E, p) , and the private key consists of (D, p) . An important property of RSA algorithm is that the roles of E and D can be interchanged. Note that the number theory suggests that it is very hard to find prime factors of p . Thus, it is extremely difficult for an intruder to determine decryption key D using just E and p , because it requires factoring p which is very hard.

Learn More

The first public key encryption algorithm was the **knapsack algorithm**. Its inventor Ralph Merkle offered a \$100 reward for the one who could break it. Adi Shamir (one of the inventors of RSA algorithm) broke it and collected \$100. Ralph Merkle modified the algorithm and offered a \$1000 reward this time. Unfortunately, Ralph Merkle had to pay \$1000 to Ronald Rivest (again one of the inventors of RSA algorithm) for breaking the knapsack algorithm.

12.6.3 DigitalSignatures

Digital signature is a very useful application of public key encryption technique. Like a physical handwritten signature, **digital signatures** are used to verify the authenticity of electronic document. In other words, digital signatures play the role of physical signatures in verifying electronic documents. The private key of the user is used to digitally sign an electronic document, and the signed document can be made public. Any user can easily verify the authenticity of the document by using

the public key that means it can be easily verified that the data is originated by the person who claims for it. However, no one can sign the document without having the private key.

Digital signatures also ensure **non-repudiation**. It means that no party or person can later deny that he never created such a document which is digitally signed by him. In case, he claims that he did not create that document, it can be easily proved that he must have created the document (unless their private key was not stolen).

12.7 STATISTICAL DATABASE

A **statistical database** contains confidential information about individuals or events. These databases are mainly used to generate statistical information about the stored information. Such databases accept only **statistical queries**, which involve statistical functions such as SUM, AVG, COUNT, MIN, MAX, and so on. However, users are not allowed to retrieve information about a particular individual.

Consider a relation **BankEmp** with the attributes **ECode**, **EName**, **Sex**, **State**, **Salary**, **Branch**, and **Designation**. Using statistical queries, one can retrieve the number of clerks, maximum salary, average salary of clerks, and so on. However, users are not allowed to retrieve the salary of a particular employee.

Implementing security in such a database raises new problems because it is possible to infer the information about specific individuals from a sequence of statistical queries. For example, suppose that the user is interested in retrieving the salary of *John Macmillan* living in *New York*, who is a *Clerk* in *Los Angeles* branch. User issues a statistical query to count the number of clerks in *Los Angeles* branch who are living in *New York*. The query can be defined as

```
SELECT COUNT (*) FROM BankEmp WHERE (State = 'New York' AND Sex = 'M'
AND Designation = 'Clerk' AND Branch = 'Los Angeles');
```

If the result of this query is 1, the next statistical query can be issued with the same condition to find the salary of *John Macmillan*. The query can be defined as

```
SELECT SUM (Salary) FROM BankEmp WHERE (State = 'New York' AND Sex =
'M' AND Designation = 'Clerk' AND Branch = 'Los Angeles');
```

Even if the result of the first query is not 1 but a small value, say 4 or 5, the approximate salary of *John Macmillan* could be inferred by applying MAX, MIN, and AVG functions in statistical queries.

One possible approach to prevent the chances of inferring information about individual is to reject certain statistical queries. For example, if the number of tuples specified by selection condition falls below a particular number, say n , the statistical query can be rejected. Alternatively, by rejecting the sequence of statistical queries from the same user that refers to the same set of tuples, it is possible to prevent retrieval of individual information. Another approach is database partitioning, in which records are classified and stored in small size groups. In this case, any statistical query can refer to complete group or set of groups, but the query referring to the subset of any group is rejected.

● SUMMARY ●

1. Database security is a crucial issue, and it covers the privacy, database integrity, and database availability.
2. Privacy states that only authorized persons should be allowed to access the database.
3. Database integrity states that database should be protected from improper modifications, either intentional or accidental, to maintain database integrity.
4. Availability states that security should not restrict the authorized users to perform their actions on the part of the database available to them.
5. Threats are classified in two categories, namely, accidental and malicious or intentional threats.
6. The DBA plays an important role in the database security. He is responsible for the overall security of the database system. Main responsibilities of the DBA are managing user accounts and database audit.
7. Managing user accounts includes creation of user accounts, granting of privileges, and classification of user accounts.
8. To keep track of all the changes (insert/delete/update) applied to the database, system log—that includes an entry for each update operation—can be used. In case of database tampering, a review of log is carried out to examine all the operations applied to the database during certain period of time. This review of log is called database audit.
9. Authorization is the granting of privileges to users, which enables them to access certain portion of the database. The person who is responsible for granting privileges to database users is called authorizer.
10. The authorization information is maintained in a table called access matrix, which the database system consults each time an access request is issued in the system. The columns of access matrix represent objects and rows represent subjects.
11. Propagation of authorization from one user to another user leads to a tree termed as authorization tree.
12. Before accessing the database, users need to identify themselves to the system to confirm their correctness. The process of verifying the identity of a user is termed as authentication. Various methods can be used to authenticate a user such as secret password, some physical characteristics of a user, smart card, or a key given to the user.
13. Discretionary access control and mandatory access control are the two approaches for access control in DBMS.
14. Discretionary access control (DAC) is enforced in a database system by granting and revoking privileges from the users.
15. Mandatory access control (MAC) enforces multilevel security by classifying subjects (for example, database users, programs, etc.) and objects (for example, relations, views, etc.) of database system in different security classes. Further, it prohibits the flow of information from higher to lower security level.
16. To implement MAC in relational DBMS, tuples and attribute values are considered as objects. Therefore, each attribute in a relation is associated with a classification attribute, and a tuple classification attribute is added to the relation. This situation leads to multilevel relations.

17. An alternative approach to traditional DAC and MAC are role-based access control (RBAC). In RBAC, permissions to perform certain operations are associated with arbitrary roles, and these roles are assigned to database users based on their responsibilities in the organization.
18. With the rapid growth of web-based applications for commercial purpose, there is a need to develop security standards for them. Major approach to protect data during transmission is encryption of data before transmission and then decryption of the data to recover original data at the destination.
19. Encryption involves applying an encryption algorithm to the data or plaintext using a pre-specified encryption key. The algorithm produces encrypted version of the plaintext as the output. Most encryption algorithm belongs to one of the two categories, namely, symmetric key encryption, or public key encryption.
20. Digital signature is a very useful application of public key encryption technique. Like a physical handwritten signature, digital signatures are used to verify the authenticity of electronic document.
21. A statistical database contains confidential information about individuals or events. These databases are mainly used to generate statistical information about the stored information.

● KEY TERMS ●

- | | |
|--|---|
| <ul style="list-style-type: none"> ● Security issues ● Privacy ● Database integrity ● Database availability ● Accidental threats ● Malicious or intentional threats ● System or superuser account ● Database audit ● Auditor role ● Authorization ● Authorizer ● Access matrix ● Object security objects ● Access privileges ● SELECT ● MODIFY ● REFERENCES ● DROP ● ALTER ● Authorization role ● Authentication ● Password authentication | <ul style="list-style-type: none"> ● Smart card ● Access control ● Discretionary access control ● GRANT and REVOKE command ● GRANT OPTION ● Mandatory access control ● Bell-Lapadula model ● Security class ● Clearance ● Simple security property ● Star property ● Classification attribute ● Tuple classification ● Multilevel relation ● Apparent key ● Covert channel ● Polyinstantiation ● Trojan horse ● Role-based access control ● CREATE ROLE ● DESTROY ROLE ● XML access control language (XACL) |
|--|---|

- Encryption
- Encryption algorithm
- Plaintext
- Encryption key
- Decryption algorithm
- Decryption key
- Symmetric encryption
- Data encryption standard (DES)
- Rijndael algorithm
- Advanced encryption standard (AES)
- Public key encryption
- Public key
- Private key
- RSA algorithm
- Digital signatures
- Non-repudiation
- Statistical database
- Statistical queries

● EXERCISES ●

A. Multiple Choice Questions

1. Which of the following issues are covered by database security?

(a) Privacy	(b) Database integrity
(c) Database availability	(d) All of these
2. _____ is responsible for overall security of the database system.

(a) Programmer	(b) Database users
(c) Database administrator	(d) None of these
3. To keep track of database users and their passwords, the DBMS maintains an encrypted table with _____ fields.

(a) One	(b) Two
(c) Three	(d) Four
4. When a user logs in successfully, the DBMS records its

(a) Account number	(b) Terminal ID
(c) Both (a) and (b)	(d) None of these
5. Which of the following is false?

(a) DBA can assign privileges to different users	(b) A user can assign privileges to any user on any relation
(c) A user can assign privileges to any user on its own relations only	(d) All of these
6. _____ is a part of database that needs to be protected from unauthorized access.

(a) Subject	(b) Object
(c) Both (a) and (b)	(d) None of these
7. Which of the following access privileges a user can have on a relation?

(a) SELECT	(b) PROPAGATE ACCESS CONTROL
(c) REFERENCES	(d) Both (a) and (b)

8. Which of the following is incorrect?
- (a) Password itself needs to be secured from unauthorized access
 - (b) Using physical characteristics of user for identification provides relatively high security of database
 - (c) The key stored on embedded chip in the smart card cannot be read
 - (d) None of these
9. Which of the following privileges can be given on a set of attributes of a relation?
- (a) SELECT
 - (b) INSERT
 - (c) UPDATE
 - (d) Both (b) and (c)
10. Which of the following features should not be there in an encryption algorithm?
- (a) Encryption and decryption should be simple for authorized user
 - (b) It should require both keys for encrypting and decrypting data
 - (c) Security should not depend only on the secrecy of the algorithm
 - (d) All of the above

B. Fill in the Blanks

1. _____ account in the DBMS provides powerful capabilities to control access to the database.
2. A database log that is mainly used for database security is generally called an _____.
3. The person who is responsible for _____ to database users is called authorizer.
4. _____ access control allows the users to pass their rights, either entire or part of their rights.
5. The process of verifying the identity of a user is termed as _____.
6. Two main approaches in DBMS for access control are _____ and _____.
7. In order to create a view, the user must have _____ privilege on all the base relations involved in creating the view.
8. In Bell–LaPadula model, each object is assigned a _____ and each subject has _____.
9. _____ is a computing program to breach security of computer system.
10. The type of encryption in which the same key is used for both encryption and decryption of data is called _____.

C. Answer the Questions

1. Define database security. Why is it important? How is it different from database integrity?
2. Define various categories of threats.
3. How DBA manages the overall database security?
4. What is an audit trail? How does it help DBA in finding out unauthorized operations?
5. What is authorization? How does the system maintain the authorization information? Explain with the help of an example.
6. Define authentication. How does it differ from authorization? Discuss various techniques that can be used to authenticate a user.

7. Explain authorization tree with the help of an example.
8. Discuss various access privileges that a user can have on a relation.
9. What is an access control? Discuss the two main approaches for access control in DBMS.
10. How does mandatory access control provide additional security for the database?
11. Access control is the feature of which category of language (DDL, DML, DCL, or DSDL)? At which level of ANSI/SPARC three-level architecture does access control work? Explain the various commands that are used for discretionary access control. Give suitable examples for each of them.
12. Define the following terms.

(a) Subjects	(b) Objects
(c) Authorizer	(d) Classification attribute
(e) Tuple classification	(f) Apparent key
(g) Public key	(h) Private key
(i) Encryption key	(j) Decryption key
13. Define multilevel relations. How does it lead to polyinstantiation? Explain with the help of suitable example.
14. What is RBAC? How is it different from DAC and MAC? Describe the SQL commands that are used for creating and destroying roles.
15. Write a short note on the following.
 - (a) Content based access control
 - (b) Statistical databases
 - (c) RSA algorithm
16. Define encryption and decryption? What are the features of a good encryption algorithm? Discuss the two categories of encryption algorithm.
17. What are digital signatures? How they can be used to verify the authenticity of the electronic documents?

D. Practical Questions

1. Consider the relation **BankEmp** introduced in Section 12.7 and assume yourself as database administrator. Write the SQL statements to grant privileges for the following.
 - (i) Allow **USER1** to retrieve **ECode**, **EName**, and **Salary** only and also allow **USER1** to update the **Salary** of any employee.
 - (ii) Allow **USER2** to act as an authorizer for all privileges on **BankEmp**.
 - (iii) Allow **USER3** to retrieve or modify the relation but he should not be able to propagate privileges to other users.

Create views wherever required.
2. Suppose using any substitution system the plaintext *Strong Security* becomes *Fgebat Frphevgl*. Using the same substitution system, compute the ciphertext for the plaintext *It is really hard* and also list the complete substitution format.
3. Suppose you are given with two small prime numbers 11 and 3. Using RSA algorithm, compute public key and private key.

DATABASE SYSTEM ARCHITECTURES

After reading this chapter, the reader will understand:

- Overview of parallel DBMS that seeks to improve performance by carrying out many operations in parallel
- Distributed DBMS (DDBMS) that permits the management of the distributed database and makes the distribution transparent to the users
- Difference between homogeneous and heterogeneous DDBMS
- Advantages of distributed database
- Techniques (fragmentation and replication) used during the process of distributed database design
- Various types of data transparency provided by DDBMS
- Various factors that must also be taken into account while distributed query processing
- Various strategies used in distributed query processing
- Transactions in distributed system
- Different techniques for concurrency control in distributed database systems
- Various deadlock-detection techniques in distributed database systems
- Commit protocols used for recovery in distributed database systems
- Three-tier client/server architecture that has been used in developing distributed systems, particularly in web applications

Today's computer applications demand high performing machines which process large amounts of data in sophisticated ways. The applications such as geographical information systems, real-time decision making, and computer-aided design demand the capability to manage several hundred gigabytes to terabytes of data. Moreover, with the evolution of Internet, the number of online users as well as size of data has been increased. This demand has been the driving force of the emergence of technologies like parallel processing and data distribution. The database systems based on parallelism and data distribution are called *parallel DBMS* and *distributed DBMS* (DDBMS), respectively.

This chapter begins with an overview of parallel DBMS. The distributed system and various issues involved in its implementation are then discussed. The chapter ends with a discussion about the client/server system (briefly discussed in Chapter 01), which is considered as a special case of distributed system.

13.1 OVERVIEW OF PARALLEL DBMS

To meet the ever-increasing demand for higher performance, the database systems are increasingly required to make use of parallel processing. The simultaneous use of computer resources such as central processing units (CPUs) and disks to perform a specific task is known as **parallel processing**.

In parallel processing, many tasks are performed simultaneously on different CPUs. Parallel DBMS seeks to improve performance by carrying out many operations in parallel, such as loading data, processing queries, and so on.

There are several architectures for building parallel database systems. The three main architectures are as follows.

- ⇒ **Shared-memory:** In shared-memory architecture, all the processors have access to a common memory and storage disks via an interconnection network (bus or switch).
- ⇒ **Shared-disk:** In shared-disk architecture, each processor has a private memory and shares only storage disks via interconnection network.
- ⇒ **Shared-nothing:** In shared-nothing architecture, each processor has private memory and one or more private disk storage. No two processors can access the same storage. The processors communicate through an interconnection network.

All these three architectures are shown in Figure 13.1.

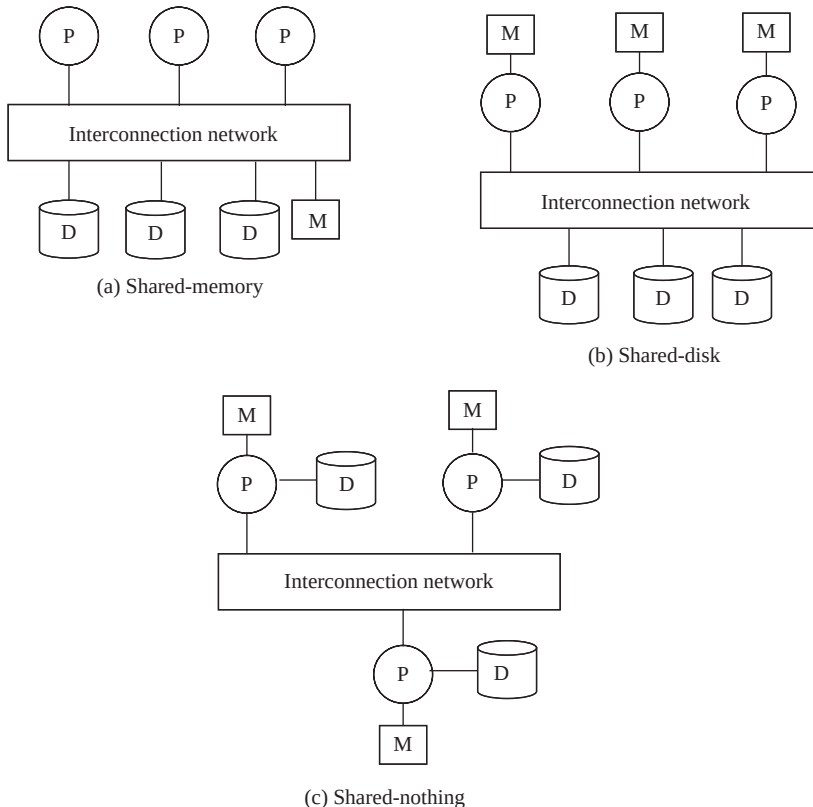


Fig. 13.1 Parallel database architecture

There are two important metrics for measuring the efficiency of a parallel database system, namely, *scaleup* and *speedup*. Running a task in less time by increasing the degree of parallelism is called **speedup**, and handling larger task by increasing the degree of parallelism is called **scaleup**.

13.2 DISTRIBUTED DBMS

In a distributed database system, data is physically stored across several computers. The computers are geographically dispersed and are connected with one another through various communication media (such as high-speed networks or telephone lines). Due to the distribution of data on multiple machines, potential exists for **distributed computing** which means the processing of a single task by several machines in a coordinated manner.

The computers in a distributed system are referred to as **sites**. In a distributed database system, each site is managed by a DBMS that is capable of running independent of other sites. The overall distributed system can be regarded as a kind of partnership among the individual local DBMSs at the individual local sites. The necessary partnership functionality is provided by a new software system at each site. This new software is logically an extension of local DBMS. The combination of this new software together with the existing DBMSs is called **distributed DBMS (DDBMS)**. DDBMS permits the management of the distributed database and makes the distribution transparent to its users.

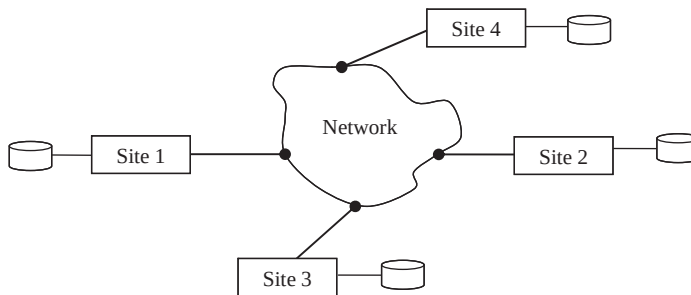


Fig. 13.2 A distributed database system

The shared-nothing parallel database system resembles distributed database system. The main difference is that former is implemented on tightly coupled multiprocessor and comprises a single database system. While in the latter, databases are geographically separated across the sites that share no physical components and are separately administered. The other difference is in their mode of operation. In shared-nothing architecture, there is symmetry and homogeneity of sites. However, in distributed database system, there may be heterogeneity of hardware and operating system at each site.

Learn More

A special purpose distributed operating system is used for the distributed system, which logically works as a single operating system. Thus, all relational database operations used for shared-nothing can be used in distributed DBMS without much modifications.



A system having multiprocessors is itself a distributed system made up of a number of nodes (processors and disks) connected by a fast network within a machine.

Homogeneous and Heterogeneous DDBMS

The distributed database system in which all sites share a common global schema and run the identical DBMS software and is known as **homogeneous distributed database system**. In such

a system, DBMS software on different sites are aware of one another, and agree to work together in processing a user request at any site exactly as if data were stored at the user's own site. In addition, the software must agree in exchanging information about transactions so that a transaction can be processed across multiple sites. Local sites relinquish a certain degree of control in terms of their right to change schemas or database-management system software to some other site.

On the other hand, a distributed database system in which different sites may have different schemas and run different DBMS software essentially autonomously is known as **heterogeneous distributed database system** or **multidatabase system**. In such a system, the sites may not be aware of one another and may provide limited facilities for cooperation in transaction processing. The differences in schemas also causes problem in query processing. In addition, differences in software obstruct the processing of transactions that access multiple sites.

Learn More

Distributed computing is the outcome of using networks to allow computers to communicate efficiently. But it differs from computer networking. The computer networking refers to two or more computers interacting with each other, but not typically sharing the processing of a single program. The World Wide Web is an example of a network but not an example of distributed computing.

13.2.1 Advantages of Distributed Database System

The distributed database system has various advantages ranging from decentralization to autonomy. Some of the advantages are as follows.

- ⇒ **Sharing data:** The primary advantage of DDBMS is the ability to share and access data in an efficient manner. DDBMS provides an environment where users at a given site are able to access data stored at other sites. For example, consider an organization having many branches. Each branch stores data related to that branch; however, a manager of a particular branch can access information of any branch.
- ⇒ **Improved availability and reliability:** Distributed systems provide greater availability and reliability. Availability is the probability that the system is running continuously throughout a specified period whereas reliability is the probability that the system is running (not down) at any given point of time. In distributed database systems, since, data is distributed across several sites, failure of a single site does not halt the entire system. The other sites can continue to operate in case of failure of one site. Only the data that exist at the failed site cannot be accessed. This means, some of the data may be inaccessible but other parts of database may still be accessible to the users. Furthermore, if the data is replicated at one or more sites, further improvement can be achieved.
- ⇒ **Autonomy:** The possibility of local autonomy is the major advantage of distributed database system. Local autonomy implies that all operations at a given site are controlled by that site. It should not depend on any other site. Further, the security, integrity, and representation of local data are under the control of local site.
- ⇒ **Easier expansion:** Distributed systems are more modular, hence, they can be expanded easily as compared to centralized systems, where upgrading a system with changes in hardware and software affects the entire database. In distributed system, the size of the database can be increased, and more processors or sites can be added as needed with little effort.

13.2.2 Distributed Databases eStorage

In distributed DBMS, the relations are stored across several sites. Accessing a relation that is stored at remote site increases the overhead due to exchange of messages. This overhead can be reduced by using two different techniques namely, *fragmentation* and *replication*. In **fragmentation**, a relation is divided into several fragments (or smaller relations), and each fragment can be stored at sites where they are more often accessed. In **replication**, several identical copies or replicas of each relation are maintained and each replica is stored at many distinct sites. Generally, replicas are stored at the sites where they are in high demand.

Note that a relation can be divided into several fragments and there may be several replicas of each fragment. In other words, both the techniques can be combined. These techniques are used during the process of distributed database design.

Fragmentation

Fragmentation of a relation R consists of breaking it into a number of fragments $R_1, R_2, \dots, R_n$ such that it is always possible to reconstruct the original relation R from the fragments $R_1, R_2, \dots, R_n$. The fragmentation can be vertical or horizontal (see Figure 13.3).

TID	ISBN	Book_title	Category	Price	Copyright_date	Year	Page_coun	P_ID
T1	003-456-433-6	Introduction to German Language	Language Book	22	2003	2004	200	P004
T2	003-456-533-8	Learning French Language	Language Book	32	2005	2006	500	P004
T3	001-354-921-1	Ransack	Novel	22	2005	2006	200	P001
T4	002-678-880-2	Call Away	Novel	22	2001	2002	200	P002
T5	001-987-760-9	C++	Textbook	40	2004	2005	800	P001
T6	002-678-980-4	DBMS	Textbook	40	2004	2006	800	P002
T7	004-765-409-5	UNIX	Textbook	26	2006	2007	550	P003
T8	004-765-359-3	Coordinate Geometry	Textbook	35	2006	2006	650	P003
T9	001-987-650-5	Differential Calculus	Textbook	30	2003	2003	450	P001

Fig. 13.3 Horizontal and vertical fragmentation

⇒ **Horizontal fragmentation:** Horizontal fragmentation breaks a relation R by assigning each tuple of R to one or more fragments. In horizontal fragmentation, each fragment is a subset of the tuples in the original relation. Since each tuple of a relation R belongs to at least one of the fragments, original relation can be reconstructed. Generally, a horizontal fragment is defined as a selection on the relation R . That is, the tuples that belong to the horizontal fragment are specified by a condition on one or more attributes of the relation. For instance, fragment R_i can be constructed as shown here.

$$R_i = \sigma_{\text{condition}(i)}(R)$$

The relation R can be reconstructed by applying union on all fragments, that is,

$$R = R_1 \cup R_2 \cup \dots \cup R_n$$

For example, the relation *book* can be divided into several fragments on the basis of attribute *Category* as shown here.

$$\begin{aligned} \text{BOOK1} &= \sigma_{\text{Category} = \text{"Novel"}}(\text{BOOK}) \\ \text{BOOK2} &= \sigma_{\text{Category} = \text{"Textbook"}}(\text{BOOK}) \\ \text{BOOK3} &= \sigma_{\text{Category} = \text{"LanguageBook"}}(\text{BOOK}) \end{aligned}$$

Here, each fragment consists of tuples of *BOOK* relation that belongs to a particular category. Further, the fragments are disjoint. However, by changing the selection condition, a particular tuple of R can appear in more than one fragment.

- ⇒ **Vertical fragmentation:** Vertical fragmentation breaks a relation by decomposing schema of relation R . In vertical fragmentation, each fragment is a subset of the attributes of the original relation. A vertical fragment is defined as a projection on the relation R as shown here.

$$R_i = \pi_{R_i}(R)$$

The relation should be fragmented in such a way that original relation can be reconstructed by applying natural join on the fragments. That is,

$$R = R_1 \bowtie R_2 \bowtie \dots \bowtie R_n$$

To ensure the reconstruction of a relation from the fragments, one of the following options can be used.

- include key attributes of R in each fragment R_i .
 - add a unique tuple id to each tuple in the original relation—this id is attached to each vertical fragment.
- ⇒ **Mixed fragmentation:** Horizontal as well as vertical fragmentation can be applied to a single schema. That is, the fragments obtained by horizontally fragmenting a relation can be further partitioned vertically or vice-versa. This type of fragmentation is called **mixed fragmentation**. The original relation is obtained by the combination of join and union operations.

Replication

Replication means storing a copy (or replica) of a relation or relation fragments in two or more sites. Distribution of an entire relation at all sites is known as **full replication**. When only some fragments of a relation are replicated, it is called **partial replication**. In partial replication, the number of copies of each fragment can range from one to the total number of sites in the system.

Replication is desirable for the following two reasons.

- ⇒ **Increased availability of data:** If a site containing the relation R fails, then the same relation can be found as long as at least one replica is available at some other site. Thus, the system can continue to process queries involving R , despite the failure of one site. Further, if the local copies of remote relations are available, we are less vulnerable to failure of remote site.
- ⇒ **Better performance:** The system can process queries faster by operating on local copy of relation instead of communicating with remote sites. Moreover, if the majority of accesses to relation R require only reading of the relation, then several sites can process query involving R in parallel. In case there are more replicas of a relation, the probability of the needed data at the site where the

transaction is initiated is more. Thus, data replication minimizes the overhead due to the movement of data between sites.

The major disadvantage of replication is that update operations incur greater overhead. The system must ensure that all the replicas of a relation are consistent to avoid erroneous results. Thus, whenever *R* is updated, all the replicas of *R* must be updated. For example, in an airline reservation system where the information regarding a flight is replicated at different sites, it is necessary that the information must agree in all sites.

In general, replication improves the performance of read operations in a distributed system and improves its reliability. However, the requirement that all the replicas must be updated makes the distributed database system more complex. In addition, concurrency control is more complicated.

Transparency

In a distributed system, the users should be able to access the database exactly as if the system were local. Users are not required to know where the data is physically stored and how the data can be accessed at the specific local site. Hiding all such details from the users is referred to as **data transparency**. DDBMS provides following type of transparencies.

- ⇒ **Location transparency** (also known as **location independence**): Users should not know the physical location of the data. Users should be able to access the data as if all the data were stored at their own local site.
- ⇒ **Fragmentation transparency** (also known as **fragmentation independence**): User should not be aware of the data fragmentation.
- ⇒ **Replication transparency** (also known as **replication independence**): User should not be aware of the data replication. Data may be replicated either for better availability or performance. Users should be able to work as if data is not replicated at all.
- ⇒ **Naming transparency**: Data items such as relations, fragments, and replicas must have unique names. This property can be ensured easily in a centralized system. However, in a distributed database, care must be taken to ensure that no two sites have same name for distinct data items. So that once a name is assigned to a data item, that named data item can be accessed unambiguously.

Data transparency is desirable because it simplifies application programs and end-user activities. At any time, in response to changing requirements, data can be changed, the number of copies can vary, and these copies can migrate from site to site. Further, data can be refragmented and fragments can be redistributed. Data transparency allows performing all these operations on data without manipulating programs or activities.

13.2.3 Distributed Query Processing

There are various issues that are involved in processing and optimizing a given query in a centralized database system as discussed in Chapter 08. In a distributed system, several other factors must also be taken into account. One of them is the communication cost. Since in a distributed system, a query may require data from more than one site, this transmission of data entails communication cost. Another factor is the gain in performance, if parts of a query are processed in parallel at different sites.

Things to Remember

The efficiency of the distributed query processing strategies also depends on the type and typology of the network used in the distributed system.

This section discusses the communication cost reduction techniques for one of the most common relational algebra operations, the join. Further, the gain in performance from having several sites process parts of query in parallel is elaborated.

Simple Join Processing

The join operation is one of the most expensive operations to perform. Thus, choosing a strategy for join is the major decision in the selection of a query-processing strategy. Consider the following relational algebra expression.

$$\Pi_{\text{ISBN, Price, P_ID, Pname}} (\text{BOOK} \bowtie_{\text{BOOK.P_ID=PUBLISHER.P_ID}} \text{PUBLISHER})$$

Suppose relations **BOOK** and **PUBLISHER** are stored at sites S_1 and S_2 , respectively, and the result is to be produced at site S_3 . Further, suppose that there are 500 tuples in **BOOK** where each tuple is 100 bytes long and 100 tuples in **PUBLISHER** where each tuple is 150 bytes long. Thus, the size of **BOOK** relation is $500 * 100 = 50,000$ bytes and the size of **PUBLISHER** relation is $100 * 150 = 15,000$ bytes. The result of this query has 500 tuples where each tuple is 75 bytes long (assuming that the attributes **ISBN**, **Price**, **P_ID**, **Pname** are 15, 6, 4, and 50 bytes long). For simplicity, assume that these two relations are neither replicated nor fragmented. The different possible strategies that can be employed for processing this query are following.

1. Transfer replicas of both the relations to site S_3 and process the query at this site to obtain the result. In this strategy, a total of $50,000 + 15,000 = 65,000$ bytes must be transferred.
2. Transfer replica of relation **PUBLISHER** to site S_1 for processing the entire query locally at site S_1 and then ship result to site S_3 . In this strategy, the bytes equivalent to the size of the query result and the size of relation **PUBLISHER** must be transferred. That is, $37,500 + 15,000 = 52,500$ bytes must be transferred.
3. Transfer replica of relation **BOOK** to site S_2 for processing the entire query locally at site S_2 and then ship result to site S_3 . In this case, $50,000 + 37,500 = 87,500$ bytes must be transferred.

If minimizing the amount of data transfer is the criteria for optimization, the strategy 2 must be chosen. However, in addition to amount of data transfer, the optimal choice also depends on the communication cost between a pair of sites and the relative speed of processing at each site. Thus, none of the above strategies is always the best.

Semijoin

Semijoin strategy is used to reduce the communication cost in performing a join operation. The idea behind semijoin strategy is to reduce the size of a relation that needs to be transmitted and hence the communication costs. In this strategy, instead of sending entire relation, joining attribute of one relation is sent to the site where other relation is present. This attribute is then joined with the relation and then the join attributes along with the required attributes are projected and transferred back to the original site. For example, consider once again the following relational algebra expression.

$$\Pi_{\text{ISBN, Price, P_ID, Pname}} (\text{BOOK} \bowtie_{\text{BOOK.P_ID=PUBLISHER.P_ID}} \text{PUBLISHER})$$

where, **BOOK** and **PUBLISHER** are stored at different sites S_1 and S_2 , respectively. Further, suppose that system needs to produce the result at site S_2 . This relational algebra expression can be evaluated using the strategy as follows:

1. Project the join attribute (**P_ID**) of **PUBLISHER** at site S_2 and transfer it to S_1 . A total of $100 * 4 = 400$ bytes will be transferred.

- Join this transferred attribute with the **BOOK** relation and transfer the required attributes from the resultant relation to site S_2 . A total of $500 * 25$ (size of attribute **ISBN** + size of attribute **Price** + size of attribute **P_ID**), that is, 12,500 bytes are required to be transferred.
- Evaluate the expression by joining the transferred attributes with **PUBLISHER** relation at site S_2 .

Here, 12,900 (12,500 + 400) bytes are required to be transferred. This strategy is more advantageous if small fraction of tuples of **BOOK** relation participates in join. It is clear that using this strategy, network cost can be reduced to a greater extent.



Semijoin operation is not commutative.

Parallelism

Consider the evaluation of a query that involves join of four relations as follows:

$$R \bowtie S \bowtie T \bowtie U$$

Suppose the relations R , S , T , and U are stored at sites S_1 , S_2 , S_3 , and S_4 , respectively. Assume that the result is to be obtained at site S_1 . The query can be evaluated in parallel by using the strategy given here.

Relation R is transferred to site S_2 where $R \bowtie S$ can be evaluated. At the same time, relation T is transferred to site S_4 where $T \bowtie U$ can be evaluated. Site S_2 can ship tuples of $R \bowtie S$ to S_1 as they are generated, instead of waiting for the entire join to be computed. Similarly, site S_4 can ship tuples of $T \bowtie U$ to S_1 . Once tuples of $R \bowtie S$ and $T \bowtie U$ reach at S_1 , the computation of $R \bowtie S \bowtie T \bowtie U$ can start. Hence, the computation of the final join result at S_1 can be done in parallel with the computation of $R \bowtie S$ at S_2 and $T \bowtie U$ at S_4 .

13.2.4 Distributed Transactions

In a distributed system, a transaction initiated at one site can access and update data at several other sites too. The transaction that accesses and updates data only at the site where it is initiated is known as **local transaction**. The transaction that accesses and updates data from several sites or the site other than the site at which the transaction is initiated is known as **global transaction**. A transaction that requires data from remote sites is broken into one or more subtransactions. These subtransactions are then distributed to the appropriate sites for execution. Note that such subtransactions are executed independently at the respective sites. The site at which the transaction is initiated is referred to as **coordinator site** and the sites where subtransactions are executed are called **participating sites**.

Like centralized system, each site has its own local transaction manager (TM) that manages the execution of those transactions (subtransactions) that access data stored in that site. The transactions may be either a local transaction or part of a global transaction. In addition, there is a transaction coordinator at each site that is responsible for coordinating the execution of all the transactions initiated at that site. The overall system architecture is shown in Figure 13.4 in which a transaction T_i initiated at coordinator site S_1 requires data from a remote site S_j . As a result, a subtransaction T_{ij} is distributed at the site S_j .

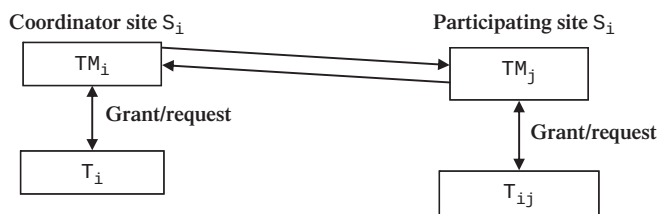


Fig. 13.4 System architecture

As discussed in Chapter 09, transaction must preserve ACID properties to ensure the integrity of the data. The ACID properties for local transactions can be assured by using certain concurrency control and recovery techniques as discussed in Chapter 10 and Chapter 11, respectively. However, for global transactions, ensuring these properties is a complicated task because of data distribution. Thus, both the concurrency control and recovery techniques need some modification to conform the distributed environment.

13.2.5 Concurrency Control in Distributed Databases

In DDBMS, concurrency control has to deal with replicated data items. Variations of the concurrency control techniques of centralized database system are used in distributed environment. Several such possible techniques based on the locking and timestamp are presented in this section.



All the concurrency control techniques require updates to be done on all replicas of a data item. If any site containing a replica of a data item has failed, updates on the data item cannot be performed.

Distributed Locking

The various locking techniques (discussed in Chapter 10) can be applied in distributed systems. However, they need to be modified in such a way that the lock manager can deal with the replicated data.

Single Lock Manager In **single lock manager** technique, the system maintains a single lock manager that resides in any single chosen site and all the requests to lock and unlock the data items are sent to that site. When a transaction needs to acquire lock on a specific data item, it sends request to the site where the lock manager resides. The lock manager checks whether the lock can be granted immediately. If the lock can be granted, the lock manager sends a message to the site from which the request for lock was initiated. Otherwise, the request is delayed until it can be granted. In case of a read lock request, data item at any one of the sites (at which replica of data item is present) is locked in the shared mode and then read. In case of a write lock request, all the replicas of the data item are locked in exclusive mode and then modified.

The advantage of this approach is that it is relatively easy to implement as it is merely a simple extension of the centralized approach. Since all requests to lock and unlock data items are made at one site, deadlock can be detected by applying directed graph (discussed in Chapter 10) directly. This technique has certain disadvantages too. Firstly, sending all locking requests to a single site can overload that site which can cause bottleneck. Secondly, the failure of the site containing lock manager disrupts the entire system as all the locking information is kept at that site.

Distributed Lock Manager In **distributed lock manager** technique, each site maintains a local lock manager which is responsible for handling the lock and unlock request for those data items that are stored in that site. When a transaction needs to acquire lock on a data item (assume that data item is not replicated) that resides at site say S, a message is sent to the lock manager at site S requesting appropriate lock. If the lock request is incompatible with the current state of locking of the requested data item, the request is delayed. Once it has ascertained that the lock can be granted, the lock manager sends a corresponding message to the site at which the request was initiated.

This approach is also simple to implement and to a great extent, it reduces the degree to which the coordinator is a bottleneck. However, in this technique detecting deadlock is more complex. Since

lock requests are directed to a number of different sites, there may be inter-site deadlocks even when there is no deadlock within a single site. Thus, the deadlock-handling algorithm needs to be modified to detect global deadlocks (deadlock involving two or more sites). The modified deadlock-handling algorithm will be discussed later in this section.

Primary Copy This technique deals with the replicated data by designating one copy of each data item (say, Q) as a **primary copy**. The site at which the primary copy of Q resides is called **primary site** of Q . All requests to lock or unlock Q are handled by the lock manager at the primary site, regardless of the number of copies existing for Q . If the lock can be granted, the lock manager sends a message to the site from which the request for lock was initiated. Otherwise, the request is delayed until it can be granted.

Using this technique, concurrency control for replicated data can be handled the same way as for unreplicated data. Furthermore, load of processing lock requests is distributed among various sites by having primary copies of different data items stored at different sites. The failure of one site affects only those transactions that need to acquire locks on data items whose primary copies are stored at the failed site. The other transactions are not affected.

Majority Locking In **majority locking** technique, majority of the copies must be locked in case of both read and write lock request for a data item. Lock request is sent to more than one-half of the sites that contain a copy of the required data item. The local manager at each site determines whether the lock can be granted or not and acts similarly as discussed in earlier techniques. In this technique, a transaction cannot operate on a data item until the lock has been granted on a majority of the replica of the data item. Further, any number of transactions can simultaneously acquire a read lock on a majority of the replicas of a data item, since a read lock is shareable. However, only one transaction can acquire a write lock on a majority of these replicas. No transaction can majority lock a data item in the shared mode if it is already locked in exclusive mode.

This technique is considered as a truly distributed concurrency control method, since most of the sites are involved in decision in locking process. However, this technique is more complicated to implement than earlier discussed techniques. It is because it has higher message traffic among sites. Another problem with this scheme is that in addition to the problem of global deadlocks due to the use of a distributed lock manager, deadlock can occur even if only one data item is being locked. To illustrate this, consider a system with four

sites S_1 , S_2 , S_3 , and S_4 having data replicated at all sites. Suppose that transactions T_1 and T_2 need to lock a data item Q in exclusive mode. Further, suppose that transaction T_1 succeeds in locking Q at sites S_1 and S_4 , while transactions T_2 succeeds in locking Q at sites S_2 and S_3 . Now, to acquire the third lock, each of the transactions must wait. Hence, a deadlock has occurred. However, this type of deadlock can be easily prevented by making all sites to request locks on the replicas of the data item in the predetermined order.

Learn More

A variant of majority locking is **biased locking** in which the request for shared lock on a data item (say, Q) goes to the lock manager only at one site containing a copy of Q , whereas the request for exclusive lock goes to lock manager at all sites containing a copy of Q .

Timestamping

We know that locking technique suffers from two disadvantages: deadlock and low level of concurrency. Timestamping technique therefore can be used as an alternative to locking. Though it is more complex to implement than locking, it prevents deadlock from occurring by avoiding in the first place.

The main idea behind timestamping approach is that each transaction in the system is assigned a unique timestamp to determine its serialization order. Various timestamping methods for a centralized system can also be used in the distributed environment. However, extension in the centralized technique to distributed technique requires a method to be developed for generating unique global timestamps.

For generating unique timestamps in distributed systems, there are two primary methods, namely, *centralized* and *distributed*.

- ⇒ **Centralized scheme:** In this scheme, a single site is responsible for generating the timestamps. The site can use either a logical counter or value of its own clock for this purpose.
- ⇒ **Distributed scheme:** In this scheme, each site generates a unique local timestamp by using either a logical counter or local clock. This unique local timestamp is concatenated with the site identifier (which must also be unique) to get a unique global timestamp (see Figure 13.5). Note that the order of concatenation is important. The site identifier must be at the least significant position so that the global timestamps generated in one site is not always greater than those generated in another site. That is, the transactions are always ordered on the basis of the time of their occurrence not on the basis of the site from which they are initiated.

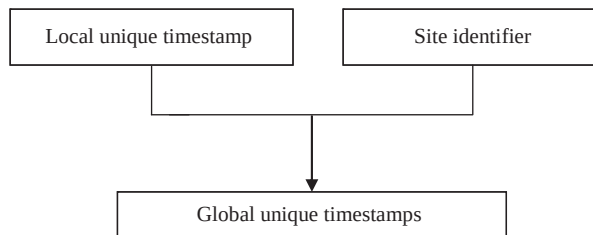


Fig. 13.5 Generation of global unique timestamps

Since the local timestamp is generated using local clock or logical counter, some problems can arise. If the logical counter is used, a relatively faster site would generate timestamps at a faster rate than the slower site. Thus, all the timestamps generated by the fast site will always be larger than those generated by another sites. Similarly, if the local clock is used, the timestamps generated by the site having fast clock will always be larger than those generated by another sites. So, some mechanism is required to keep these local timestamp generating schemes synchronized. It is accomplished by maintaining a **logical clock** at each site. Logical clock is implemented as a counter that is incremented after generating a new timestamp. This timestamp is included in the messages sent between sites. On receiving a message, the site compares its clock with this timestamp. If site finds that the current value of its logical clock is smaller than this message timestamp, it sets it to some value greater than the message timestamp.

Deadlock Handling

As stated earlier, in distributed systems, even when there is no deadlock within a single site, there may be inter-site deadlocks. In other words, deadlock involving two or more sites can occur. To illustrate this, consider a system having two sites S_1 and S_2 with following situation.

- ⇒ At site S_1 , transaction T_1 is waiting to acquire lock on the data item held by transaction T_2 . Transaction T_2 is waiting to acquire lock on the data item held by transaction T_4 . Transaction T_3 is waiting to acquire lock on the data item locked by transaction T_1 and T_2 .
- ⇒ At site S_2 , transaction T_4 is waiting to acquire lock on the data item locked by transaction T_3 .

The common deadlock-detection technique is wait-for graph. This technique requires that each site maintains its own **local wait-for graph**. The local wait-for graph [see Figure 13.6(a)] for both the sites S_1 and S_2 is constructed in the same way as discussed in Chapter 10. The only difference is that the nodes represent all the local as well as non-local transactions that are currently holding or requesting any of the items local to that site. Note that T_3 and T_4 are present in both the graphs indicating that transactions have requested data items at both sites.

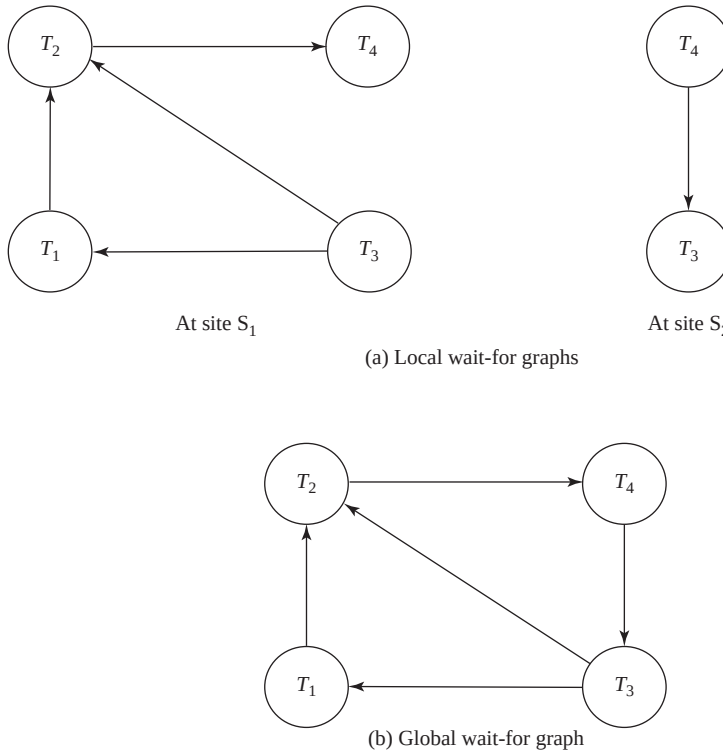


Fig. 13.6 Distributed deadlock

Observe that in Figure 13.6(a), each local wait-for graph is acyclic, indicating that there is no deadlock within the sites. However, absence of cycle in the local wait-for graph does not imply that there is no deadlock. This means, global deadlock cannot be detected solely on the basis of local wait-for graph.

To detect such deadlocks, the wait-for graph technique needs some extended treatment. In a distributed system, this technique requires generating not only local wait-for graph but also global wait-for graph for the entire system. **Global wait-for graph** [see Figure 13.6(b)] is constructed simply by the union of the local wait-for graphs and maintained at a single chosen site. Since this site is responsible for handling global deadlocks, it is known as **deadlock detection coordinator**.

The disadvantage of this technique is that it incurs communication overhead, as it requires sending all local wait-for graphs to the coordinator site. The other disadvantage is that delays in propagating local wait-for graphs may cause deadlock detection algorithms to identify deadlocks that do not really exist. Such deadlocks are known as **phantom deadlocks**. These types of deadlocks may lead to unnecessary rollbacks.

Instead of using one site for deadlock detection, it is possible to detect deadlock in a distributed manner. That is, several sites can take part in detecting deadlocks. In this technique, local wait-for graphs

are sent to all the sites to generate only the concerned portion of global wait-for graph. However, such technique is more complicated to implement and sometimes deadlock may not be detected.

One other simple technique that can be used is time-out based technique. Recall that this technique falls between the deadlock prevention where a deadlock can never happen, and deadlock detection and recovery. In this technique, a transaction is allowed to wait for a fixed interval of time for required data item. If a transaction waits longer than specified time interval, it is aborted. Though this algorithm may cause unnecessary restarts, the overhead of deadlock detection is low. Moreover, if the participating sites do not cooperate to the extent of sharing their waits-for graphs, it may be the only option.



NOTE *Deadlock prevention techniques can also be used for handling deadlocks but they may result in unnecessary delays and rollbacks. Thus, instead of deadlock prevention techniques, deadlock detection techniques are widely used.*

13.2.6 Recovery in Distributed Databases

Recovery in a distributed system is more complicated than in a centralized system. In addition to the same type of failures encountered in a centralized system, a distributed system may suffer from new types of failures. The basic types of failures are loss of messages, failure of the site at which subtransaction is executing, and failure of communication links. The recovery system must ensure atomicity property, that is, either all the subtransactions of a given transaction must commit or abort at all sites. This property must be guaranteed despite any kind of failure. This guarantee is achieved using commit protocol. The most widely used commit protocol is the two-phase commit protocol (2PC).

Two-Phase Commit

In a distributed system, since a transaction is being executed at more than one site, the commit protocol may be divided into two phases, namely, *voting phase* and *decision phase*. To understand the operation of this protocol, consider a transaction T initiated at site S_i where the transaction coordinator is C_i . When T completes its execution, that is, all the sites at which T has executed inform C_i that T has completed, C_i initiates the 2PC protocol. The two phases of this protocol are as follows:

⇒ **Voting phase:** In this phase, the participating sites vote on whether they are ready to commit T or not. During this phase, the following actions are taken.

1. C_i inserts a prepare record [T , prepare] in the log and force-writes the log onto stable storage. After this, C_i sends the prepare T message to all the participating sites.
2. When the transaction manager at a participating site receives the prepare T message, it determines whether the site is ready to commit T or not. If the site is ready to commit T , it inserts a [T , ready] record in the log; otherwise it inserts [T , not-ready] record. It then force-writes the log onto stable storage and sends the ready T or abort T message to C_i accordingly.



NOTE *Any participating site can unilaterally decide to abort T before sending ready T message to the coordinator.*

⇒ **Decision phase:** In this phase, the coordinator site decides whether the transaction T can be committed or it has to be aborted. The decision is based on the responses to prepare T message that C_i receives from all the participating sites. The following responses may be received at the coordinator site.

1. A **ready** T message from all the participating sites. In this case, C_1 decides to commit the transaction and inserts $[T, \text{commit}]$ record to the log. It then force-writes the log onto stable storage and sends **commit** T message to all the participating sites.
2. At least one **not-ready** T message or no response from any participating site within the specified time-out interval. In this case, C_1 decides to abort the transaction and inserts $[T, \text{abort}]$ record to the log. It then force-writes the log onto stable storage and sends **abort** T message to all the participating sites.

Once the participating sites receive either the **commit** T or **abort** T message from C_1 , they insert the corresponding record to the log, and force-write the log onto stable storage. After that, each participating site sends an **acknowledge** T message to C_1 . Upon receiving this message from all the sites, C_1 force-writes the $[T, \text{complete}]$ record to the log.



NOTE As soon as the coordinator receives one **abort** T message, the coordinator decides to abort the transaction without waiting for other sites or time-out period.

Now, there may be a case that any site fails during the execution of the commit protocol. If any participating site fails, there are two possibilities. First, the site fails before sending **ready** T message. In this case, the coordinator does not receive any reply within the specified time interval. Thus, the transaction is aborted. Secondly, if the site fails after sending **ready** T message, the coordinator simply ignores the failure of participating site and proceeds in the normal way.

When the participating site restarts after failure, the recovery process is invoked. During recovery, the site examines its log to find all those transactions that were executing at the time of the failure. To decide the fate of those transactions, the recovery process proceeds as follows:

- ⇒ If the log contains the $[T, \text{commit}]$ or $[T, \text{abort}]$ record, the site simply executes **redo**(T) or **undo**(T), respectively, and sends the **acknowledgement** T message to the coordinator.
- ⇒ If the log contains the $[T, \text{ready}]$ record, the site contacts the coordinator to determine whether to commit or abort T .
- ⇒ If no such record exists in the log, the site decides unilaterally to abort the transaction and execute **undo**(T), since the site could not have voted before its failure.

Now, consider the case of coordinator's failure. In this case, all the participating sites communicate with each other to determine the status of T . If any participating site contains the $[T, \text{commit}]$ or $[T, \text{abort}]$ record in its log, the transaction has to be committed or aborted, respectively. On the other hand, if any participating site does not contain $[T, \text{ready}]$ record in its log, it means that site has not voted for T and without that vote, the coordinator could not have decided to commit T . Thus, in this case, it is preferable to abort T . However, if none of the discussed cases holds, the participating sites have to wait until the coordinator recovers, and all the locks acquired by T are held. This leads to **blocking problem** at the participating sites, since the transactions that require locks on data items locked by T have to wait.

Learn More

In order to continue processing immediately after the coordinator's failure, the system may maintain a **backup coordinator** that resumes the whole responsibility of the coordinator.

When the coordinator restarts after failure, the recovery process is invoked at the coordinator site. The coordinator examines its log to determine the status of T .

- ⇒ If the log contains $[T, \text{commit}]$ or $[T, \text{abort}]$ record, it executes $\text{redo}(T)$ or $\text{undo}(T)$, respectively, and periodically resends the $\text{commit } T$ or $\text{abort } T$ message to the participating sites, until it receives $\text{acknowledgement } T$ message from each participating site.
- ⇒ If the log contains $[T, \text{prepare}]$ record, the coordinator unilaterally decides to abort T and conveys it to all the participating sites.

Three Phase Commit

The commit protocol called **three-phase commit protocol (3PC)**, is an extension of the two-phase commit protocol that avoids blocking even if the coordinator site fails during recovery. To avoid blocking problem, this protocol requires some conditions which are as follows.

- ⇒ There is no network partitioning.
- ⇒ At least there is one available site.
- ⇒ At most K sites fail, where K is some predetermined number.

If these conditions hold, 3PC protocol avoids blocking by introducing third phase between voting phase and the decision phase. In this phase, multiple sites are involved in the decision to commit. The basic idea of 3PC protocol is that when the coordinator sends $\text{prepare } T$ message and receives yes votes from all the participant sites; it sends all participants pre-commit message instead of commit message. After ensuring that at least K other participating sites know about the decision to commit, the coordinator force-writes a commit log record and sends a commit message to all participants. The coordinator effectively postpones the decision to commit until it is sure that enough sites know about the decision to commit.

If the coordinator site fails, the other sites choose a new coordinator. This new coordinator communicates with the remaining other sites to check whether the old coordinator had decided to commit. It can be detected easily if any one of the other K sites that received pre-commit message is up. In this case, new coordinator restarts the third phase. If none of them has received pre-commit message, new coordinator aborts the transaction; otherwise, commits the transaction.

The 3PC protocol incurs an overhead during normal execution and requires that communication link failures do not lead to network partition to ensure freedom from blocking. For these reasons, it is not widely used.



This protocol does not block the participants on failure of sites, except for the case when more than K sites fail.

13.3 CLIENT/SERVER SYSTEMS

As we know client/server refers to the architecture where overall system is divided into two parts, client and server. They can run on two different machines. Thus, there is possibility of distributed computing. In fact, a client/server system can be considered as a special case of distributed system

in which some sites are server sites and some are client sites; data resides at the server sites and all applications execute at the client sites.

In the late 1980s and early to mid of 1990s, a true distributed database system (that implement all the functionality discussed so far) was not a commercially feasible product. Thus, instead of making effort on developing a true distributed DBMS product, the vendors developed the distributed database system based on the client/server architecture. Two-tier client/server architecture can also be used for distributed system. However, it has some limitations. So, nowadays, three-tier client/server architecture has been used in developing distributed systems, particularly in web applications. The three-tier client/server architecture has already been introduced in Chapter 01. Here, all the three layers, namely, *presentation layer*, *application layer*, and *database server* of this architecture are explained in context of web applications. Different technologies related to each layer are given in Figure 13.7.

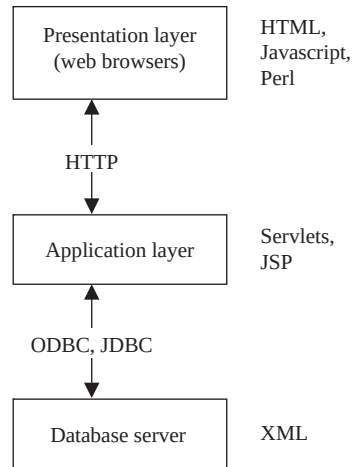


Fig. 13.7 *Three-tier architecture for web applications with respective technologies*

- ⇒ **Presentation layer (client):** This layer provides an interface to handle user input and displays the needed information. At this layer, web interfaces or forms are provided through which the user can issue request, provide input, and the formatted output generated by application layer is displayed. For this purpose, Web browsers are used. HTML is the basic data presentation language that is used to communicate data from the presentation layer to the application layer. The other languages which can be used are Java, JavaScript, etc. This layer communicates with the application layer via HTTP protocol.
- ⇒ **Application layer:** This layer executes the business logic of the application. It controls what data needs to input before an action can be executed, and what action are taken under what conditions. It issues SQL queries to the database server, formats the query result and sends to the client for presentation. The languages that can be used to program business logic are Java servlets, JSP, etc. This layer can communicate with one or more databases using standards such as ODBC, JDBC, etc. Apart from the business logic, it also handles functionality like security checks, identity verification, and so on.
- ⇒ **Database server:** This layer processes the query requested by the application layer and send the results to it. Thus, it is the layer that accesses data from the database system. Generally, SQL is used to access the database if it is relational or object-relational. Stored procedures can also be used for the same purpose. At this level, XML may be used to exchange information regarding the query between the application server and the database server.

There are various approaches that are used to divide the DBMS functionality across client, application server, and database server. The common approach is to include the functionality of centralized database at the database server. The application server formulates SQL queries and connects to the database server when required. The client provides interfaces for user interactions. A number of relational products have adopted this approach.

 ● SUMMARY ●

1. The applications such as geographical information systems, computer-aided design, and real-time decision making demand the capability to manage data size of several hundred gigabytes to terabytes.
2. With the evolution of Internet, the number of online users as well as size of data has been increased. This demand has been the driving force of the emergence of technologies like parallel processing and data distribution.
3. The simultaneous use of computer resources such as CPUs and disks to perform a specific task is known as parallel processing. In parallel processing, many tasks are performed simultaneously on different CPUs.
4. The database systems based on parallelism and data distribution are called parallel DBMS and distributed DBMS.
5. Parallel DBMS seeks to improve performance by carrying out many operations in parallel, such as loading data, processing queries, and so on.
6. The three main architectures for building parallel database systems are shared-memory, shared-disk, and shared-nothing.
7. In distributed database system, data is physically stored across several computers. The computers are geographically dispersed and are connected with one another through various communication media (such as high-speed networks or telephone lines).
8. In distributed database system, each site is managed by a DBMS that is capable of running independent of other sites. The overall distributed system can be regarded as a kind of partnership among the individual local DBMSs at the individual local sites.
9. The necessary partnership functionality is provided by a new software system at each site. The combination of this new software together with the existing DBMSs is called distributed database management system (DDBMS).
10. The distributed database system, in which all sites run the identical DBMS software, is known as homogeneous distributed database system.
11. A distributed database system in which different sites may have different schemas and run different DBMS software, essentially autonomously, is known as heterogeneous distributed database system or multidatabase system.
12. In distributed DBMS, the relations are stored across several sites using two different techniques namely, fragmentation and replication.
13. In fragmentation, relation is divided into several fragments (or smaller relations), and each fragment can be stored at sites where they are more often accessed.
14. The fragmentation can be vertical or horizontal.
15. Horizontal fragmentation breaks a relation R by assigning each tuple of R to one or more fragments. Since each tuple of a relation R belongs to at least one of the fragments, original relation can be reconstructed. Generally, a horizontal fragment is defined as a selection on the relation R .
16. Vertical fragmentation breaks a relation by decomposing schema of relation R . In vertical fragmentation, each fragment is a subset of the attributes of the original relation.

17. The relation should be fragmented in such a way that original relation can be reconstructed by applying natural join on the fragments.
18. Horizontal as well as vertical fragmentation can be applied to a single schema. That is, the fragments obtained by horizontally fragmenting a relation can be further partitioned vertically or vice-versa. This type of fragmentation is called mixed fragmentation.
19. In replication, several identical copies or replicas of each relation are maintained and each replica is stored at many distinct sites. Generally, replicas are stored at the sites where they are in high demand.
20. Replication improves the performance of read operations in a distributed system and improves its reliability. However, updates incur greater overhead and the requirement that all the replicas be updated makes distributed database system more complex.
21. Users are not required to know where the data is physically stored and how the data can be accessed at the specific local site. Hiding all such details from the users is referred to as data transparency.
22. In a distributed system, several other factors must also be taken into account other than the communication cost. Since in a distributed system, a query may require data from more than one site, this transmission of data entails communication cost. Another factor is the gain in performance, if parts of a query are processed in parallel at different sites.
23. Various techniques are used in processing a query. The optimal choice depends on the communication cost between a pair of sites and the relative speed of processing at each site, in addition to the size of the data and the result being transferred.
24. The join operation is one of the most expensive operations to perform. Thus, choosing a strategy for join is the major decision in the selection of a query-processing strategy.
25. Semijoin strategy is used to reduce the communication cost in performing a join operation.
26. A transaction must preserve ACID properties to ensure the integrity of the data. The ACID properties for local transactions can be assured by using certain concurrency control and recovery techniques of centralized database system.
27. For global transactions, ensuring these properties is a complicated task because of data distribution. Thus, both the concurrency control and recovery techniques need some modification to conform the distributed environment.
28. In distributed systems, even when there is no deadlock within a single site, there may be inter-site deadlocks. To detect such deadlocks, the wait-for graph technique requires generating not only local wait-for graph but also global wait-for graph.
29. In addition to the handling new type of failures encountered in a centralized system, the recovery system must ensure atomicity property. This property must be guaranteed despite any kind of failure. This guarantee is achieved using commit protocol. The most widely used commit protocol is the two-phase commit protocol (2PC).
30. The commit protocol called three-phase commit protocol (3PC) is an extension of the two-phase commit protocol that avoids blocking even if the coordinator site fails during recovery.
31. A client-server system can be considered as a special case of distributed system in which some sites are server sites and some are client sites, data resides at the server sites, and all applications execute at the client sites.
32. Nowadays, three-tier client/server architecture has been used in developing distributed systems, particularly in web applications.

KEY TERMS

- Parallel processing
- Parallel database architecture
- Shared-memory architecture
- Shared-disk architecture
- Shared-nothing architecture
- Speedup
- Scaleup
- Distributed computing
- Distributed database management system (DDBMS)
- Homogeneous distributed database system
- Heterogeneous distributed database system or multidatabase system
- Fragmentation
- Horizontal fragmentation
- Vertical fragmentation
- Mixed fragmentation
- Replication
- Full replication
- Partial replication
- Data transparency
- Location transparency
- Fragmentation transparency
- Replication transparency
- Naming transparency
- Distributed query processing
- Semijoin
- Parallelism
- Distributed transactions
- Local transaction
- Global transaction
- Coordinator site
- Participating sites
- Distributed locking
- Single lock manager
- Distributed lock manager
- Primary copy
- Primary site
- Majority locking
- Timestamping
- Deadlock handling
- Local wait-for graph
- Global wait-for graph
- Deadlock detection coordinator
- Phantom deadlocks
- Commit protocol
- Two-phase commit protocol (2PC)
- Voting phase
- Decision phase
- Blocking problem
- Three-phase commit protocol (3PC)
- Three-tier client/server architecture
- Presentation layer
- Application layer
- Databases over

EXERCISES

A. Multiple Choice Questions

1. Which of the following parallel database architectures resembles the distributed database system?
 - (a) Shared-memory
 - (b) Shared-disk
 - (c) Shared-nothing
 - (d) None of these
2. The important metric/s for measuring the efficiency of a parallel database system is
 - (a) Scaleup
 - (b) Speedup
 - (c) Both
 - (d) None of these

3. Which of the following transparencies means that users should be able to access the data as if all the data were stored at their own local site?
 - (a) Replication transparency
 - (b) Naming transparency
 - (c) Fragmentation transparency
 - (d) Location transparency
4. In which fragmentation, each fragment is a subset of the tuples in the original relation?
 - (a) Horizontal
 - (b) Vertical
 - (c) One of seth
 - (d) dth
5. Which of the following is not an advantage of replication?
 - (a) ncreased vailability
 - (b) dReliability
 - (c) Better performance
 - (d) All of these
6. Which of the following conditions are required to avoid blocking problem by three-phase commit protocol (3PC) even if the coordinator site fails during recovery?
 1. There is no network partitioning.
 2. At least there is one available site.
 3. At most K sites fail, where K is some predetermined number.
 - (a) Both 1 and 2
 - (b) Both 2 and 3
 - (c) All of seth
 - (d) dth of seth
7. Which of the following technologies is not related to presentation layer?
 - (a) SP
 - (b) TML
 - (c) rBrowsers
 - (d) avla
8. Which of the following techniques is considered as a truly distributed concurrency control method?
 - (a) Prima opy
 - (b) dity ling
 - (c) Distributed lock manager
 - (d) Timestamping
9. Which of the following are the disadvantages of locking scheme?
 - (a) Low level of concurrency
 - (b) Deadlock
 - (c) Both (a) and (b)
 - (d) None of these
10. Which of these layers can communicate with one or more databases using standards such as ODBC, JDBC etc.?
 - (a) Presentation ylar
 - (b) Application ayler
 - (c) dBase esver
 - (d) dth olase

B. Fill in the Blanks

1. The simultaneous use of computer resources such as CPUs and disks to perform a specific task is known as _____.
2. In _____ architecture, each processor has a private memory and shares only storage disks via interconnection network.
3. _____ strategy is used to reduce the communication cost in performing a join operation.
4. The site at which the transaction is initiated is referred to as _____ and the sites where subtransactions are executed are called _____.

5. Using _____ technique, concurrency control for replicated data can be handled the same way as for unreplicated data.
6. The two phases of two phase commit protocol are _____ and _____.
7. The delays in propagating local wait-for graphs may cause deadlock detection algorithms to identify deadlocks that do not really exist. Such deadlocks are known as _____.
8. The recovery system must ensure _____ property despite any kind of failure.
9. _____ architecture has been used in developing distributed systems, particularly in web applications.
10. _____ layer issues SQL queries to the database server, formats the query result and sends to the client for presentation.

C. Answer the Questions

1. Discuss the factors that motivate parallel and distributed database system.
2. Discuss the architecture of parallel database system.
3. What are distributed databases? How is data distribution performed in DDBMS?
4. What is the need of distributed database system? Give its two disadvantages.
5. Consider a bank that has a collection of sites, each running a database system. The databases interact by the electronic transfer of money between one another. Would such a system characterize as a distributed database? Why?
6. What is difference between shared-nothing parallel database system and distributed database system?
7. Explain the following terms:

i) Distributed locking	(ii) Fragmentation transparency
(iii) Replication transparency	(iv) Location transparency
8. When is it useful to have replication or fragmentation of data? Explain your answer.
9. Illustrate the issues to implement distributed database.
10. Discuss the different techniques for executing an equijoin of two relations located at different sites. What main factors affect the cost of data transfer?
11. Discuss the semijoin method for executing an equijoin of two relations at different sites. Under what conditions is an equijoin strategy efficient?
12. List all possible types of failure in a distributed system. Which of them are also applicable to a centralized system?
13. Suppose that a site gets no response from another site for a long time. Can the first site distinguish whether the connecting link has failed or the other site has failed? How is such a failure handled?
14. What is a commit protocol and why is it required in a distributed database? Describe and compare two phase and three phase commit protocol. What is blocking and how does the three phase protocol prevent it?
15. Explain how 2PC ensures transaction atomicity despite the failure for each possible failure.
16. Give an example of a distributed DBMS with three sites such that no two local waits-for graphs reveal a deadlock, yet there is a global deadlock.
17. Discuss deadlock detection techniques in a distributed database system.
18. What is a phantom deadlock? Give an example.

19. Consider a database of a company where a relation **Employee** (**name**, **address**, **salary**, **branch**) is fragmented horizontally by branch. Assume that each fragment has two replicas: one stored at site A and one stored locally at the branch site. Describe a good strategy for the following queries entered at site B.
 - (i) Find all employees at site C.
 - (ii) Find the average salary of all employees.
 - (iii) Find the lowest-paid employee in the company.
20. A bank has many branches. A customer can open his account in any branch and can operate his account from any branch. The information that is stored about account includes account number, branch, name of customer, address of customer, guarantors of the customer, balance.
 - (i) Suggest a suitable fragmentation scheme for the bank. Give reason in support of your scheme.
 - (ii) Suggest a suitable replication scheme. Justify your suggestion. Make suitable assumptions, if any.
 - (iii) Write at least two advantages and two disadvantages of data replication.

DATA WAREHOUSING, OLAP, AND DATA MINING

After reading this chapter, the reader will understand:

- What are OLTP systems?
- The need of a data warehouse
- Basic characteristics of a data warehouse, which include subject oriented, non-volatile, time varying, and integrated characteristics
- Differences between data warehouse and OLTP systems
- The entire process of getting data into the data warehouse, which is termed as extract, transform, and load (ETL) process
- The importance of metadata repository
- The multidimensional data model for a data warehouse
- The concept of fact and dimension tables
- The two ways of representing multidimensional data in a data warehouse,

which include star schema and snowflake schema

- The online analytical processing (OLAP) technology, that enables analysts, managers, and executives to analyse the complex data derived from the data warehouse
- Various functionalities provided by OLAP system, which include pivoting, slicing and dicing, and rollup and drill down
- The three ways of implementing OLAP
- The data mining technology
- The process of knowledge discovery in database (KDD)
- Different types of information discovered during data mining, which include association rules, classification, and clustering
- Application areas of data mining technology

Over the past few decades, organizations have built large databases by collecting a large amount of data about their day-to-day activities, such as product sales information for the companies or course registration and grade information for universities, etc. The size of these databases may range up to hundreds of gigabytes or even terabytes. The database applications that record and update data in such databases are known as **online transaction processing (OLTP)** applications. The traditional databases such as network, hierarchical, relational, and object-oriented databases support OLTP. The systems that aim to extract high-level information stored in such databases, and to use that information in making a variety of decisions that are important for the organization are known as **decision-support systems (DSS)**. Several tools and techniques are available that provide the correct level of information to help the decision makers in decision-making.

Some of these tools and techniques include data warehousing, online analytical processing (OLAP), and data mining. Data warehouses contain consolidated data from many sources, augmented

with summary information, and covering a long time-period. Online analytical processing refers to the analysis of complex data from the data warehouse, and data mining refers to the mining or discovery of new information in terms of patterns or rules from a vast amount of data. This chapter discusses each of them in detail.

14.1 DATA WAREHOUSING

Large organizations have different databases at different locations. Therefore, any kind of a query on such databases may require access to information scattered over several data sources. This makes querying a time-consuming, cumbersome, and inefficient process. Moreover, these databases usually store only the current data of the organization. Therefore, any kind of a query that requires access to historical data may not be answered from such databases. These problems can be solved by using data warehouses.

A **data warehouse (DW)** is a repository of suitable operational data (data that documents the everyday operations of an organization) gathered from multiple sources, stored under a unified schema, at a single site. It can successfully answer any ad hoc, complex, statistical or analytical queries. The data once gathered can be stored for a longer period allowing access to historical data. The data warehouses provide the user a single consolidated interface to data, which makes decision-support queries easier to write.

Learn More

A **data mart** is a subset of data warehouse that is designed for a particular department of an organization such as sales, marketing, or finance.



NOTE *A query that is run at the spur of the moment, and generally is never saved to run again is known as **ad hoc query**. These queries are generally not predefined, and are built and run as and when required.*

Since data warehouses have been developed in numerous organizations to meet their specific needs, there is no single and standard definition of this term. According to William Inmon, the father of the modern data warehouse, a data warehouse is a *subject-oriented, integrated, time-variant, non-volatile collection of data in support of management's decisions*. Based on this definition, the following four basic characteristics of data warehousing can be visualized:

- ⇒ **Subject oriented:** A data warehouse is organized around a major subject such as customer, products, and sales. That is, data is organized according to a subject instead of application. For example, an insurance company using a data warehouse would organize its data by customer, premium and claim instead of by different policies (auto sweep policy, joint life policy, etc.).
- ⇒ **Nonvolatile:** A data warehouse is always a physically separated store of data. Due to this separation, data warehouse does not require transaction processing, recovery, concurrency control, and so on. The data is not overwritten or deleted once it enters the data warehouse, but is only loaded, refreshed, and accessed for queries. The data in the data warehouse is retained for future reporting.
- ⇒ **Time varying:** Data is stored in a data warehouse to provide a historical perspective. Thus, the data in the data warehouse is time-variant or historical in nature. The data in the warehouse is 5–10 years old, or older, and is used for comparisons, trend analysis, and forecasting. The changes to the data in the data warehouse are tracked and recorded so that reports can be produced showing the changes over time.

⇒ **Integrated:** A data warehouse is usually constructed by integrating multiple, heterogeneous sources such as relational databases and flat files. The database contains data from most or all of an organization's operational applications, and this data is made consistent.

The main advantage of using a data warehouse is that a data analyst can perform complex queries and analyses of the information stored in data warehouse without affecting the OLTP systems. It has some more advantages also which are given here.

- ⇒ It provides historical information that can be used in different forms to perform comparative and competitive analysis.
- ⇒ It increases the quality of the data and tries to make it complete.
- ⇒ With the help of other backup resources, it can also help in recovery from disasters.
- ⇒ It can be used in determining many trends and patterns through the use of data mining.
- ⇒ The users of a data warehouse can generate high-level information from it by analysing the data stored in it.

The data warehouse is more than just data—it is also the process involved in getting these data from various sources to tables and in getting the data from tables to analysts. Data warehouses are different from traditional transaction processing systems that record information about the day-to-day operations of an organization. The key differences between a data warehouse and an OLTP system are listed in Table 14.1.

Table 14.1 *Differences between data warehouse and OLTP system*

Data Warehouse	OLTP System
A data warehouse is mainly designed to handle <i>ad hoc</i> queries.	OLTP systems support only predefined operations like insertion, deletion, updates, and retrieval of data.
A data warehouse is updated on a regular basis (nightly or weekly) using bulk data modification techniques. This modification is done by extraction, transformation and load (ETL) tools (ETL is discussed in next section). The end users are not allowed to directly update the data warehouse.	The database of an OLTP system is always up to date, and reflects the current state of each business transaction. The end users are allowed to directly update the database.
A data warehouse generally uses de-normalized or partially de-normalized schemas to optimize query performance.	OLTP systems use fully normalized schemas to optimize insert, delete, and update performance.
A typical data warehouse query accesses thousands or millions of records.	A typical query in an OLTP system accesses only a few records at a time.
A Data warehouse usually stores data of many months or years to support historical analysis.	OLTP systems usually store data of only a few weeks or months.

14.1.1 Creating and Maintaining a Data Warehouse

Creating and maintaining a data warehouse is a complex task. A data warehouse basically consists of three components, namely, *the data sources*, *the ETL process*, and *the metadata repository*. The architecture of a typical data warehouse is shown in Figure 14.1.

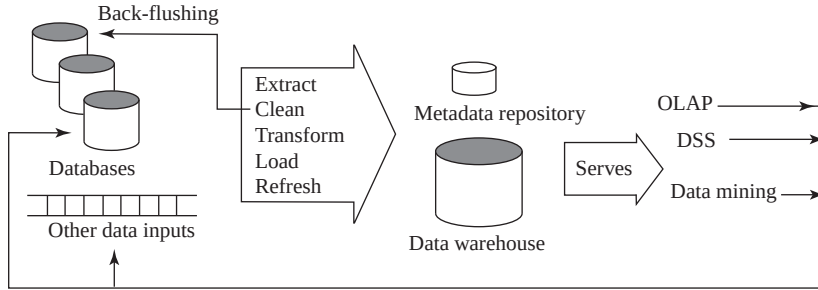


Fig. 14.1 A typical data warehouse architecture

Large companies have various data sources that have been constructed independently by different groups and likely to have different schemas. If the companies want to use such diverse data for making business decisions, they need to gather this data under a unified schema for efficient execution of queries. Moreover, there can be a number of semantic mismatches across these databases such as different currency units, different names for the same attribute, differences in the normalized forms of the tables, etc. These differences must be accommodated before the data is brought into the warehouse. Thus, the main task of data warehousing is to perform schema integration and to convert the data to an integrated schema before it is actually stored.

After the schema is designed, the warehouse must acquire data so that it can fulfil the required objectives. Acquisition of data for the warehouse involves the following steps:

1. The data is **extracted** from multiple, heterogeneous data sources. The data sources include operational databases (databases of the organizations at various sites) and external sources such as web, purchased data, etc.
2. The data sources may contain some minor errors or inconsistencies. For example, the names are often misspelled, and street, area or city names in the addresses are misspelled or zip codes are entered incorrectly. This incorrect data, thus, must be **cleaned** to minimize the errors and fill in the missing information when possible. The task of correcting and preprocessing the data is called **data cleansing**. These errors can be corrected to some reasonable level by looking up a database containing street names and zip codes in each city. The approximate matching of data required for this task is referred to as **fuzzy lookup**. In some cases, the data managers in the organization want to upgrade their data with the cleaned data. This process is known as **back-flushing**. This data is then transformed to accommodate semantic mismatches.
3. The cleaned and transformed data is finally loaded into the warehouse. Data is partitioned, and indexes or other access paths are built for fast and efficient retrieval of data. Loading is a slow process due to the large volume of data. For instance, loading a terabyte of data sequentially can take weeks and a gigabyte can take hours. Thus, parallelism is important for loading warehouses. The raw data generated by transaction processing system may be too large to store in a data warehouse, therefore, some data can be stored in summarized form. Thus, additional preprocessing such as sorting and generation of summarized data is performed at this stage.

This entire process of getting data into the data warehouse is called **extract, transform, and load (ETL)** process. At the backend of the process, OLAP, data mining, and DSS may generate new relevant information such as rules and patterns. This information is sent back to the data warehouse. Once the data is loaded into a warehouse, it must be periodically **refreshed** to reflect the updates on the relations at the data sources and periodically **purge** old data. The

Learn More

Microsoft's SQL Server comes with a free ETL tool called **Data Transforming Service (DTS)**.

updates of a warehouse are handled by the acquisition component of the warehouse that provides all required preprocessing.

The third and the most important component of the data warehouse is the **metadata repository**, which keeps track of currently stored data. It contains the description of the data including its schema definition. The metadata repository includes both technical and business metadata. The **technical metadata** includes the technical details of the warehouse including storage structures, data description, warehouse operations, etc. The **business metadata**, on the other hand, includes the relevant business rules of the organization.

14.1.2 Multidimensional Modeling for a Data Warehouse

The data in a warehouse is usually **multidimensional data** having measure attributes and dimension attributes. The attributes that measure some value and can be aggregated upon are called **measure attributes**. On the other hand, the attributes that define the dimensions on which the measure attributes and their summaries are viewed are called **dimension attributes**.

The relations containing such multidimensional data are called **fact tables**. The fact tables contain the primary information in the data warehouse, and thus are very large. Consider a bookshop selling books of different categories such as textbooks, language books, and novels, and it maintains an *Online Book* database for its customers so that they can buy books online. The bookshop may have several branches in different locations. The SALES relation shown in Figure 14.2 is an example of

BOOK			
bid	Book_title	Category	Price
B1	C++	Textbook	40
B2	Ransack	Novel	22
B3	Learning French Language	Language Book	32

TIME					
tid	Date	Week	Month	Quarter	Year
1	15	3	12	4	2006
2	10	2	3	1	2007
3	15	3	6	2	2007

LOCATION			
lid	City	State	Country
L1	Las Vegas	Nevada	USA
L2	Mumbai	Maharashtra	India
L3	Delhi	Delhi	India

SALES			
bid	tid	lid	Number
B1	1	L1	25
B1	2	L1	18
B1	3	L1	10
B2	1	L1	11
B2	2	L1	12
B2	3	L1	18
B3	1	L1	16
B3	2	L1	10
B3	3	L1	8
B1	1	L2	12
B1	2	L2	10
B1	3	L2	11
B2	1	L2	23
B2	2	L2	9
B2	3	L2	8
B3	1	L2	17
B3	2	L2	19
B3	3	L2	21
B1	1	L3	22
B1	2	L3	19
B1	3	L3	11
B2	1	L3	12
B2	2	L3	17
B2	3	L3	15
B3	1	L3	12
B3	2	L3	14
B3	3	L3	33

Fig. 14.2 Dimension tables and fact tables

a fact table that stores the sales information of various books at different locations in different time periods. The attribute **Number** is the measure attribute, which describes the number of books sold.

Each tuple in the **SALES** relation gives information about which book is sold, which location the book was sold from, and at what time the book was sold. Thus, *book*, *location*, and *time* are the dimensions of the *sales* and for a given *book*, *location*, and *time*, we have at most one associated *sales* value. Each dimension has a set of attributes associated with it. For example, the dimension *time* is identified by the attribute, **tid** in the **SALES** relation. Similarly, the dimensions *book* and *location* are identified by **bid** and **lid**, respectively. These identifiers are kept short in order to minimize the storage requirements.

The dimension attributes **bid**, **tid**, and **lid** in the fact tables are the primary keys in other tables called **dimension tables** (or **lookup tables**), which store additional information about the dimensions. For example, the *book* dimension can have the attributes **Book_title**, **Category**, and **Price**. Similarly, the *location* dimension can have the attributes **City**, **State**, **Country**. The time dimension can have the attributes **Date**, **Week**, **Month**, **Quarter**, and **Year**. This information about *book*, *location*, and *time* is stored in the dimension tables **BOOK**, **LOCATION**, and **TIME**, respectively, as shown in Figure 14.2. The attribute **bid**, **tid**, and **lid** are the primary keys in **BOOK**, **TIME**, and **LOCATION** table, respectively, and these attributes are the foreign keys in fact table **SALES**. The primary key of a fact table is usually a composite key that is made up of all its foreign keys. The dimension tables are much smaller than the fact table.

The set of values associated with each dimension can also be structured as a hierarchy. For example, the dimension *time* can be arranged in a hierarchy as shown in Figure 14.3(a). Dates belong to weeks and months, which are in turn contained in quarters, and quarters are contained in years. Similarly, a city belongs to a state and a state belongs to a country, which led to the *location* hierarchy as shown in Figure 14.3(b). In the same way, different books can belong to different categories, thereby, resulting in *book* hierarchy as shown in Figure 14.3(c).

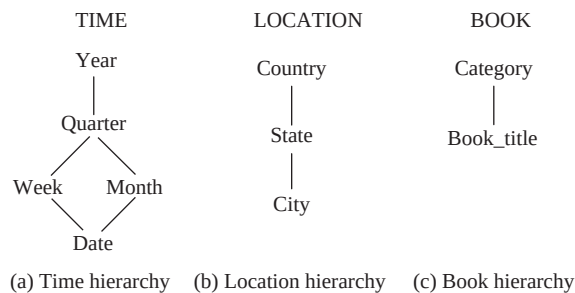
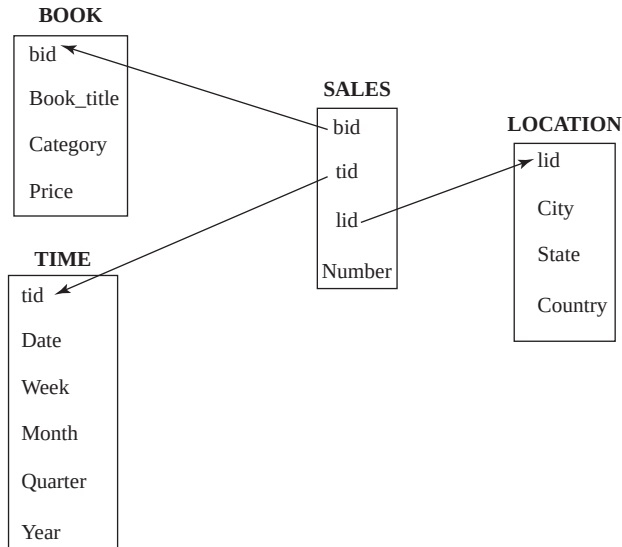


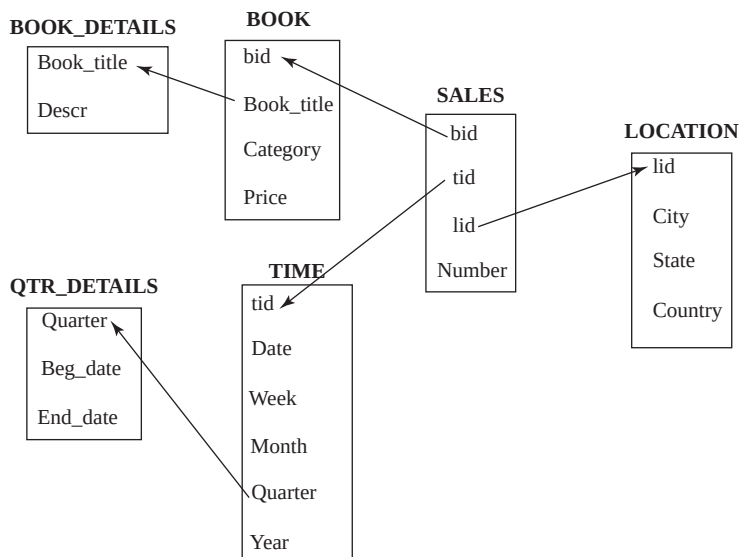
Fig. 14.3 *Dimension hierarchies*

The multidimensional data in a data warehouse can be represented either using a *star schema* or a *snowflake schema*. The **star schema** is the simplest data warehouse schema, which consists of a fact table with a single table for each dimension. The centre of the star schema consists of a large fact table and the points of the star are the dimension tables. The star schema for our running example is shown in Figure 14.4(a).

The **snowflake schema** is a variation of star schema, which may have multiple levels of dimension tables. For example, the attribute **Book_title** in **BOOK** relation can be a foreign key in another relation, say, **BOOK_DETAILS** with an additional attribute **Descr** that gives details of the book. Similarly, the attribute **Quarter** in the **TIME** relation can be a foreign key in another relation, say, **QTR_DETAILS**, with two additional attributes, **Beg_date** and **End_date** that give the starting and ending dates of each quarter, respectively. The snowflake schema for our running example is shown in Figure 14.4(b).



(a) Star schema for a data warehouse



(b) Snowflake schema for a data warehouse

Fig. 14.4 Warehouse schemas

14.2 ONLINE ANALYTICAL PROCESSING (OLAP)

Online Analytical Processing (OLAP) is a category of software technology that enables analysts, managers, and executives to analyze the complex data derived from the data warehouse. The term *online* indicates that the analysts, managers, and executives must be able to request new summaries and get the responses online, within a few seconds. They should not be forced to wait for a long time to see

the result of the query. OLAP enables data analysts to perform ad hoc analysis of data in multiple dimensions, thereby providing the insight and understanding they need for better decision-making.



NOTE *SQL:1999 and SQL:2003 standards contain some additional constructs to support data analysis. The SQL extensions for data analysis are discussed in Appendix B.*

Since the data in the data warehouse is multidimensional in nature, it may be difficult to analyze. Therefore, it can be displayed in the form of cross-tabulation as shown in Figure 14.5. A cross-tabulation (also known as a **cross-tab** or a **pivot table**) can be done on any two dimensions keeping the other dimensions fixed as *all*. In general, a cross-tab is a two-dimensional table in which values for one attribute (say A) form the row headers, and values for another attribute (say B) form the column headers. Each cell can be identified by (a_i, b_j) where a_i is a value for A and b_j is a value for B. If there is single tuple with any value, say (a_i, b_j) , in the fact table, then the value in the cell is derived from that single tuple. However, if there are multiple tuples with (a_i, b_j) value, then the value in the cell is derived by aggregation on the tuples with that value.

lid:	all	tid			
		1	2	3	Total
bid	B1	59	47	32	138
	B2	46	38	41	125
	B3	45	43	62	150
	Total	150	128	135	413

(a) Cross-tabulation of SALES by bid and tid

bid:	all	tid			
		1	2	3	Total
lid	L1	52	40	36	128
	L2	52	38	40	130
	L3	46	50	59	155
	Total	150	128	135	413

(b) Cross-tabulation of SALES by lid and tid

tid:	all	lid			
		L1	L2	L3	Total
bid	B1	53	33	52	138
	B2	41	40	44	125
	B3	34	57	59	150
	Total	128	130	155	413

(c) Cross-tabulation of SALES by bid and lid

Fig. 14.5 Cross-tabulations of SALES relation

In most of the cases, an extra row and an extra column are used for storing the total of the cells in the row/column.

Figure 14.5(a) shows the cross-tabulation of **SALES** relation (shown in Figure 14.2) by **bid** and **tid** for all **lid**. Similarly, Figure 14.5(b) shows the cross-tabulation of **SALES** relation by **lid** and **tid** for all **bid**. Finally, Figure 14.5(c) shows the cross-tabulation of **SALES** relation by **bid** and **lid** for all **tid**.

Unlike relational tables where the number of columns is fixed, the number of columns in a cross-tab may change depending on the data stored in it. New columns can be added, and existing columns can be deleted depending on the data values. Thus, it is desirable to represent a cross-tab in the form of a relation with a fixed number of columns as shown in Figure 14.6. The summary values can be represented by introducing a special value, *all*, that represents sub-totals. The SQL:1999 standard actually uses the *null* value in place of *all*. However, we have used *all* to avoid confusion with the *null* values stored in the database.

In the core of any OLAP system, there is a concept of a **data cube** (also called a **multidimensional cube** or **OLAP cube** or **hypercube**), which is the generalization of a 2D cross-tab to *n* dimensions. A three-dimensional data cube on the **SALES** relation is shown in Figure 14.7. In this figure, *book* dimension is shown on *x*-axis, *time* as *y*-axis, and *location* as *z*-axis. The value for a dimension may be *all*, in which case the cell contains a summary over all values of that dimension like a cross-tabulation.

bid	tid	lid	Number
B1	1	all	59
B1	2	all	47
B1	3	all	32
B1	all	all	138
B2	1	all	46
B2	2	all	38
B2	3	all	41
B2	all	all	125
B3	1	all	45
B3	2	all	43
B3	3	all	62
B3	all	all	150
all	1	all	150
all	2	all	128
all	3	all	135
all	all	all	413

Fig. 14.6 Relational representation of the cross-tab shown in figure 14.5(a)

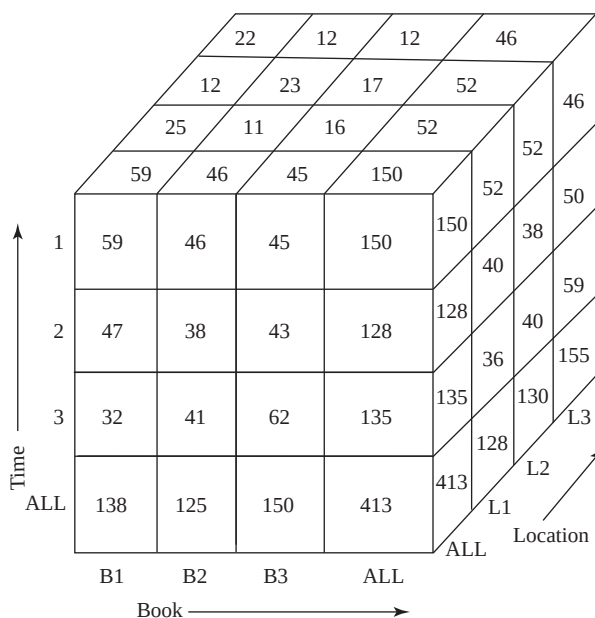


Fig. 14.7 Three-dimensional data cube for **SALES** relation

A data analyst can look at different cross-tabs on the same data by interactively selecting the dimensions in the cross-tab. The technique of changing from one-dimensional orientation to another is known as **pivoting** (or **rotation**). The pivoted version of the data cube in Figure 14.7 is shown in Figure 14.8. In this figure, *book* dimension is shown on x-axis, *location* as y-axis, and *time* as z-axis.

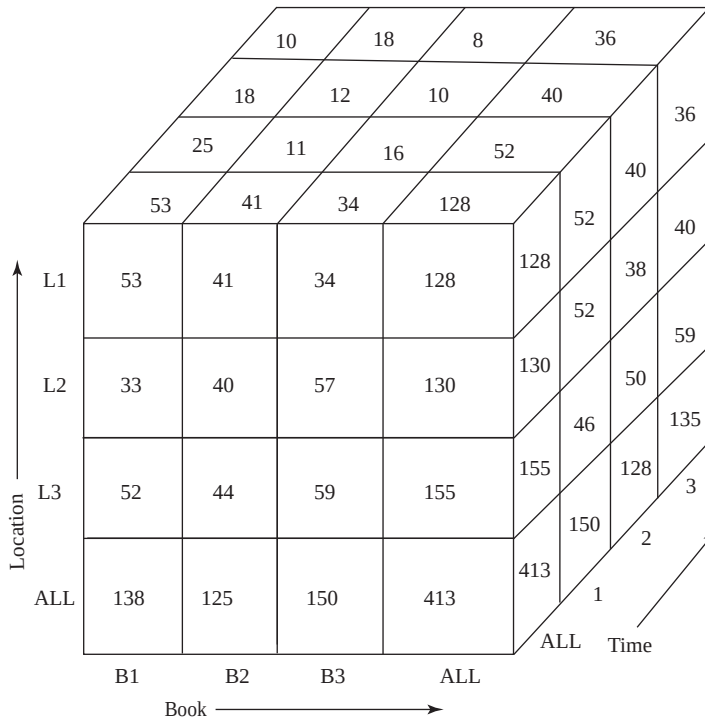


Fig. 14.8 Pivoted version of data cube shown in figure 14.7

In addition to pivoting, an OLAP system also provides some other functionalities, which are given as follows:

- ⇒ **Slice and dice:** The data cube is full of data, and there are many thousands of combinations in which it can be viewed. In case a cross-tabulation is done for a specific value other than *all* for the fixed third dimension, it is called **slice** operation or **slicing**. Slicing can be thought of as viewing a slice of the data cube. For example, an analyst may want to see a cross-tab of **SALES** relation on the attributes **bid** and **lid** for **tid=1**, instead of the sum across all time periods. The resultant cross-tab is shown in Figure 14.9, which can be thought of as a slice orthogonal to **tid** axis. Slicing is sometimes called **dicing** when two or more dimensions are fixed. Slice and dice operations enable users to see the information that is most meaningful to them and examine it from different viewpoints.

Note that instead of specifying *all* for the attributes that are not part of cross-tab, slicing/dicing consists of selecting specific values for these attributes, and displaying them on the top of the cross-tab. For example, in Figure 14.9, **tid=1** is displayed on the top of the cross-tab instead of *all*.

tid: 1		lid			
bid		L1	L2	L3	Total
	B1	25	12	22	59
	B2	11	23	12	46
	B3	16	17	12	45
	Total	52	52	46	150

Fig. 14.9 Slicing the cube for $tid=1$

⇒ **Rollup and drill down:** OLAP system also allows data to be displayed at various levels of granularity. The operation that converts data with a finer-granularity to the coarser-granularity with the help of aggregation is known as **rollup** operation. On the other hand, an operation that converts data with a coarser-granularity to the finer-granularity is known as **drill down** operation. For example, an analyst may be interested in viewing the sales of books category wise (*textbooks*, *language books*, and *novels*) in different countries, instead of looking at individual sales. This is rollup operation. On the other hand, the analyst looking at the level of books categories may drill down the hierarchy to look at individual sales in different states of each country. The resultant cross-tabs after rollup and drill down operations on SALES relation are shown in Figure 14.10. The values are derived from the fact and dimension tables shown in Figure 14.2.

		Country		
		India	USA	Grand total
Book Categories	Textbook	85	53	138
	Language Book	84	41	125
	Novel	116	34	150
	Subtotal	285	128	413

(a) The rollup operation

		India		USA		Grand total
		Maharashtra	Delhi	Naveda		
Book Categories	Textbook	33	52	53		138
	Language Book	40	44	41		125
	Novel	57	59	34		150
	Subtotal	130	155	128		413

(b) The drill down operation

Fig. 14.10 Applying rollup and drill down operations on SALES relation

14.2.1 Implementing OLAP

There are three ways of implementing OLAP system. The traditional OLAP systems used multidimensional arrays in memory to store data cubes and thus, are referred to as **multidimensional OLAP**

(**MOLAP**) systems. The OLAP systems that work directly with relational databases are called **relational OLAP (ROLAP)** systems. In ROLAP systems, the fact tables and dimension tables are stored as relations and new relations are also created to store the aggregated information.

MOLAP systems generally use specialized indexing and storage optimizations and hence, deliver better performance. MOLAP also needs less storage space compared to ROLAP, because the specialized storage typically includes compression techniques. Moreover, ROLAP relies more on the database to perform calculations, thus, it has more limitations in the specialized functions it can use. However, ROLAP is generally more scalable.

The systems that include the best of ROLAP and MOLAP are known as **hybrid OLAP (HOLAP)**. These systems use main memory to hold some summaries, and relational tables to hold base data and other summaries. Therefore, it can generally pre-process quickly, scale well, and offer good function support.

Most of the OLAP systems are implemented as client/server systems. The client systems requests different views of the data from the server, which contains the relational database as well as MOLAP data cubes. The entire data cubes on a relation can be computed by using any standard algorithm for computing aggregate operations, one grouping at a time. However, this algorithm is the naive way to compute the data cube, as it may require scanning a relation large number of times.

A simple way to optimize the algorithm is to compute an aggregate from another aggregate, instead of from the base relation. A further improvement can be achieved by computing multiple groupings on a single scan of the data. Early OLAP implementations pre-computed and stored entire data cubes (groupings on all subsets of dimension attributes). The pre-computation allows OLAP queries to be answered within a few seconds, even if the database contains a large number of tuples.

However, the entire data cube is often larger than the original relation from which the data cube is formed, as there are 2^n groupings for n dimension attributes. The number of groupings further increases with hierarchies on attributes. Thus, in most of the cases it is not feasible to store the entire data cubes. Instead of pre-computing and storing all the possible groupings, some of the groupings can be computed and stored, and others can be computed on demand. For example, consider a query that requires summaries by (*bid*, *tid*) which has not been pre-computed. The result can be computed from the summaries by (*bid*, *tid*, *lid*), if that has been already computed.

14.3 DATA MINING TECHNOLOGY

With rapid computerization in the past two decades, almost all the organizations have collected a vast amount of data in their databases. These organizations need to understand their data and also want to discover useful information in terms of patterns or rules from the existing data. The extraction of hidden and predictive information from such large databases is known as **data mining**. Data mining tools predict future trends and behaviours, which allow organizations to make proactive and knowledge-driven decisions. Identifying the profile of customers with similar buying habits, finding all items that are frequently purchased with some other item, finding all credit card applicants with poor or good credit risks are some of the examples of data mining queries.

Things to Remember

Data mining combines work from areas such as statistics, machine learning, pattern recognition, databases, and more recently high performance computing. The goal of data mining is to discover interesting and previously unknown information in the data sets. Tools for data mining have the ability to analyze enormous amount of data and discover significant patterns and relationships that might otherwise have taken a person thousand of hours to find.

Though data in large operational databases and flat files can be provided as input for the data mining process, data in data warehouses is more suitable for data mining process. Some of the advantages of using data of data warehouses for data mining are listed here.

- ⇒ A data warehouse consists of data from multiple sources. Thus, the data in data warehouse is integrated and subject-oriented.
- ⇒ In a data warehouse, the data is first extracted, cleaned, and transformed before loading, thus, maintaining the consistency of the data.
- ⇒ The data in a data warehouse is in summarized form. Hence, the data mining process can directly use this data without performing any aggregations.
- ⇒ A data warehouse also provides the capability of analysing the data by using OLAP operations.

14.3.1 Knowledge Discovery in Databases

Like knowledge discovery in artificial intelligence (also known as **machine learning**) or exploratory data analysis in statistics, data mining also tries to find useful patterns or rules from the data. However, it differs from machine learning and statistical analysis in a way that it deals with a large volume of data typically stored on disks. That is, data mining deals with **knowledge discovery in databases (KDD)**. Knowledge discovery in databases is the non-trivial extraction of implicit, previously unknown, and potentially useful information from the databases. The process of KDD is divided into six phases that are given as follows:

1. **Data selection or extraction:** The entire raw dataset is examined to identify the target subset of data and the attributes of interest.
2. **Data cleansing or preprocessing:** The data is cleaned to minimize errors and fill in missing information, wherever possible.
3. **Enrichment:** The data is enhanced with additional sources of information.
4. **Data transformation or encoding:** The data is categorized under different groups in terms of categories, ranges, geographical regions, and so on. This may be done to reduce the amount of data stored in the database.
5. **Datamining:** The data mining is applied to discover useful patterns and rules.
6. **Interpretation or evaluation:** The patterns are presented to the users in easily understandable and meaningful formats such as listings, graphical outputs, summary tables or visualizations.

14.3.2 Types of Knowledge Discovered During Data Mining

The knowledge discovered from the database during data mining includes the following:

- ⇒ **Association rules:** Association describes a relationship between a set of items that people tend to buy together. For example, if customers buy two-wheelers, there is a possibility that they also buy some accessories such as seat cover, helmet, gloves, etc. A good salesman, therefore, exploits them to make additional sales. However, our main aim is to automate the process so that the system itself may suggest accessories that tend to be bought along with two-wheelers. The association rules derived during data mining correlate a set of items with another set of items that do not intersect.
- ⇒ **Classification:** The goal of classification is to partition the given data into predefined disjoint groups or classes. For example, an insurance company can define the insurance worthiness level

of a customer as excellent, good, average or bad depending on the income, age, and prior claims experience of the customers.

- ⇒ **Clustering:** The task of clustering is to group the data into set of similar elements so that those similar elements belong to the same class. The principle of clustering is to maximize intra-class similarity and minimize the inter-class similarity.

Association Rules

Discovery of association rules is one of the major tasks involved in data mining. The task of mining association rules is to find interesting relationship among various items in a given data set. Retail shops are generally interested in association between different items that customers buy. A common application is that of *market-basket data* where market-basket describes a set of items a consumer buys in a supermarket during his one visit. Consider some examples of associations given as follows:

- ⇒ A person who buys a mobile is also likely to buy some accessories such as mobile cover, hands free, etc.
- ⇒ A person who buys bread is also likely to buy butter and jam.
- ⇒ Someone who buys eggs is also likely to buy bread.
- ⇒ Someone who bought the book *Data Structure using C* is also likely to buy *Programming in C*.

The association information can be used in several ways. For example, a mobile shop may decide to keep attractive mobile covers to tempt people to buy them. Similarly, if a customer buys a book online, the system may also suggest associated books. A grocery shop may decide to place bread close to eggs, since they are often bought together. This will help the owners of retail shop, bookshop, and grocery shop to increase the sales of associated items.

An association rule has a form $X \Rightarrow Y$, where $X = \{x_1, x_2, \dots, x_n\}$ and $Y = \{y_1, y_2, \dots, y_n\}$ are the disjoint sets of items, that is, $X \cap Y = \emptyset$. It states that if a person buys an item X , he or she is likely to buy an item Y . The set $X \cup Y$ is called an **itemset**—a set of items a customer tends to buy together. The item X is called the **antecedent**, while Y is called the **consequent** of the rule. An example of association rule is

mobile $\Rightarrow$ *mobile cover, hands free*

This rule says that a customer who buys a mobile also tends to buy mobile cover and hands free. An association rule must have an associated **population**, which consists of a set of **instances**. For example, in a bookshop, the population may consist of all customers who made purchases regardless of when they made the purchases. Thus, each customer is an instance. However, in case of grocery shop, the population may consist of all grocery shop purchases irrespective of the number of visits of a single customer. Thus, each purchase (not a customer) is an instance. For an association rule to be of interest to an analyst, the rule should satisfy two interest measures, namely, *support* and *confidence*.

- ⇒ **Support** (also known as **prevalence**): It is the percentage or fraction of a population that satisfies both the antecedent and consequent of the rule. If the support is low, it implies that there is no strong evidence that the items in the itemset $X \cup Y$ are bought together. For example, suppose only 0.002% of the customers buy mobile and chocolates; thus, the support for the rule *mobile* $\Rightarrow$ *chocolates* is low. Companies are generally not interested in such rules since they involve only a few customers. However, if 60% of the purchases involve mobile, mobile cover, and hands free, this implies that its support is high. Companies pay a lot of attention to the rules with high support.

⇒ **Confidence** (also known as **strength**): It is the probability that a customer will buy the items in the set Y if he or she purchases the items in the set X. It is computed as $\text{support}(X \cup Y) / \text{support}(X)$

For example, the association rule *mobile* ⇒ *mobile cover* has the confidence of 80% if 80% of the purchases that include mobile also include mobile cover. Like a rule with low support, a rule with low confidence is also not meaningful for the organizations.

To understand the concept of support and confidence, consider some transactions of a mobile shop shown in Table 14.2. Let us consider two association rules, *mobile* ⇒ *mobile cover* and *mobile* ⇒ *hands free*. The support for *mobile* ⇒ *mobile cover* is 50% and the support for *mobile* ⇒ *hands free* is 75%. The confidence for *mobile* ⇒ *mobile cover* is 66.67%. It implies that out of three transactions, which contain mobile, two contain mobile cover. Further, the confidence for *mobile* ⇒ *hands free* is 100%. It implies that all the three transactions that contain mobile also contain hands free.

Table 14.2 Sample transactions in mobile shop

Transaction_ID	Time	Items Purchased
T1	12:00	Mobile, mobile cover, hands free
T5	2:00	Mobile, hands free
T10	5:00	Mobile, mobile cover, hands free, memory card
T20	8:00	Hands free, memory card

The main goal of mining association rules is to find all the possible rules that exceed some minimum prespecified support and confidence value. This problem of generating a set of possible association rules can be broken down in two sub-problems.

1. Finding all itemsets that have support above the minimum prespecified support. These itemsets are called **large** (or **frequent**) itemsets.
2. Deriving all rules for each large itemset, which have confidence more than the minimum prespecified confidence, and that involve all and only the elements of the set.

If there are n number of items in a company, then the number of distinct itemsets will be 2^n and computing support for all possible itemsets becomes very tedious and time-consuming. To reduce the combinatorial search space, algorithms for finding association rules make use of these properties.

- ⇒ **Downward closure:** This property states that each subset of a large itemset must also be large.
- ⇒ **Anti-monotonicity:** This property states that each superset of a small itemset is also small. Thus, once an itemset is found to have small support, any extensions formed by adding one or more items to the set will also produce a small subset.

There are several algorithms to find the large itemsets. One of them is Apriori algorithm.

Apriori Algorithm Apriori algorithm uses the downward closure property. It takes a database D of t transactions and minimum support, minSup , represented as a fraction of t , as input. Apriori algorithm generates all possible large itemsets $L_1, L_2, \dots, L_k$ as output. The algorithm is shown in Figure 14.11.

The algorithm proceeds iteratively. In the first pass, only the sets with single items are considered for generating large itemsets. This itemset is referred to as large 1-itemset (itemset with one item). In each subsequent pass, large itemsets identified in the previous pass are extended with another item to generate larger itemsets. Therefore, the second pass considers only sets with two items, and so on. Thus, by considering only the itemsets obtained by extending the large itemsets, we reduce the number of candidate large itemsets. The algorithm terminates after k passes, if no large k -itemsets is found.

```

Step 1:  $k=1$ ;
Step 2: Find large itemset  $L_k$  from  $C_k$ ; //  $C_k$  is the set of all
                                           // candidate itemsets
Step 3: Form  $C_{k+1}$  from  $L_k$ ;
Step 4:  $k=k+1$ ;
Step 5: Repeat steps 2, 3, and 4 until  $C_k$  is empty;

```

Fig. 14.11 Apriori algorithm

Step 2 is called the large itemset generation step, and step 3 is called the candidate itemset generation step. The details about steps 2 and 3 are explained in Figure 14.12.

Step 2: Large itemset generation

```

Step 2a: Scan the database  $D$  and count each itemset in  $C_k$ ;
Step 2b: If the count is greater than  $\text{minSup}$  then
        add that itemset to  $L_k$ ;

```

Step 3: Candidate itemset generation

```

Step 3a: For  $k=1$ ,  $C_1$  = all itemsets of length 1;
        For  $k>1$ , generate  $C_k$  from  $L_{k-1}$  as follows:

The join step:
     $C_k$  =  $k-2$  way join of  $L_{k-1}$  with itself;
    If both  $\{I_1, \dots, I_{k-2}, I_{k-1}\}$  and  $\{I_1, \dots, I_{k-2}, I_k\}$  are in  $L_{k-1}$  then
        add  $\{I_1, \dots, I_{k-2}, I_{k-1}, I_k\}$  to  $C_k$ ;

//assuming that the items  $I_1, \dots, I_k$  are always sorted

The prune step:
    Remove  $\{I_1, \dots, I_{k-2}, I_{k-1}, I_k\}$ , if it does not
    contain a large  $(k-1)$  subset;

```

Fig. 14.12 Details of steps 2 and 3 of apriori algorithm

An example of Apriori algorithm is shown in Figure 14.13. Let the value of minSup be 50%, that is, the item in the candidate set should be included in at least two transactions. In the first pass, the database D is scanned to find the candidate itemset C_1 from which large 1-itemset L_1 is produced. Since the support for itemset $\{4\}$ is less than the minSup , it is not included in L_1 . In the second pass, candidate itemset C_2 is generated from L_1 , which consists of $\{1,2\}$, $\{1,3\}$, $\{1,5\}$, $\{2,3\}$, $\{2,5\}$,

and {3,5}. Then the database D is scanned to find the support for these itemsets and produce large 2-itemsets L_2 . Since the support for itemsets {1, 2} and {1, 5} is less than the minSup , they are not included in L_2 . In the third pass, candidate itemset C_3 is generated from L_2 , which includes {2,3,5}. The database D is again scanned to find the support for this itemset and produce large 3-itemsets L_3 , which is the desired large itemset.

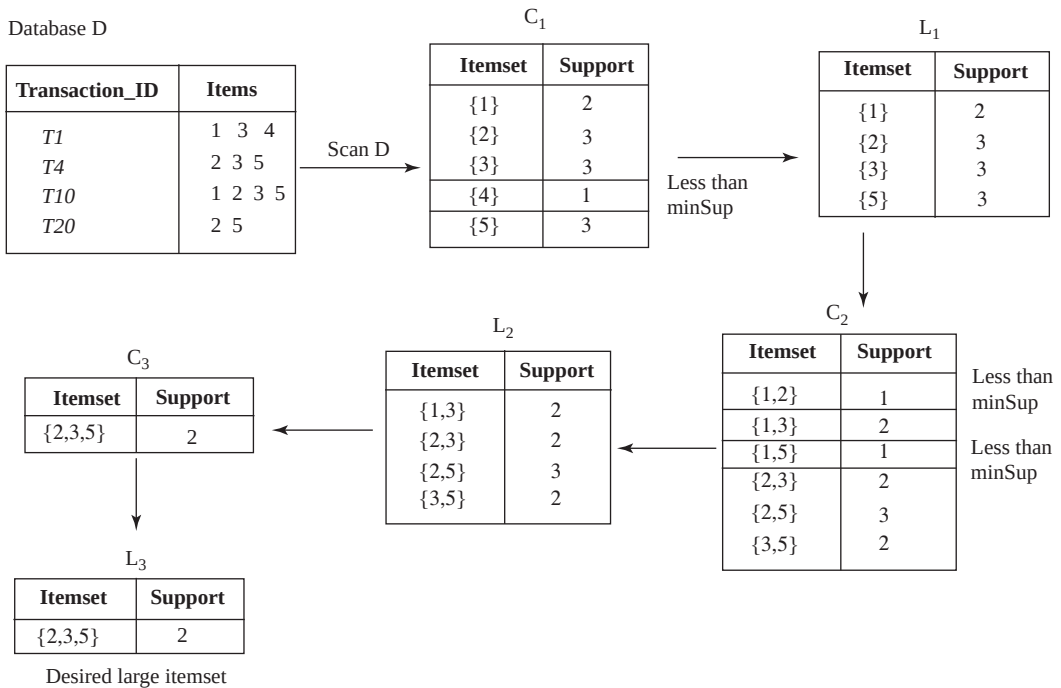


Fig. 14.13 An example of apriori algorithm

Once the large itemsets are identified, the next step is to generate all possible association rules. Association rules can be found from every large itemset L with the help of another algorithm shown in Figure 14.14.

For every non-empty subset S of L

Find $B = L - A$.

$A \Rightarrow B$ is an association rule if

$\text{confidence}(A \Rightarrow B) \geq \text{minConf}$

// $\text{confidence}(A \Rightarrow B) = \text{support}(A \cup B) / \text{support}(A)$

Fig. 14.14 An algorithm for finding association rules

For example, consider the large itemset $L = \{2, 3, 5\}$ generated earlier with support 50%. Proper non-empty subsets of L are: $\{2, 3\}$, $\{2, 5\}$, $\{3, 5\}$, $\{2\}$, $\{3\}$, $\{5\}$ with supports = 50%, 75%, 50%, 75%, 75%, and 75%, respectively. The association rules from these subsets are given in Table 14.3.

Table 14.3 *A set of association rules*

Association Rule $A \Rightarrow B$	Confidence $\text{Support}(A \cup B) / \text{Support}(A)$
$\{2, 3\} \Rightarrow \{5\}$	$50\% / 50\% = 100\%$
$\{2, 5\} \Rightarrow \{3\}$	$50\% / 75\% = 66.67\%$
$\{3, 5\} \Rightarrow \{2\}$	$50\% / 50\% = 100\%$
$\{2\} \Rightarrow \{3, 5\}$	$50\% / 75\% = 66.67\%$
$\{3\} \Rightarrow \{2, 5\}$	$50\% / 75\% = 66.67\%$
$\{5\} \Rightarrow \{2, 3\}$	$50\% / 75\% = 66.67\%$

Classification

Classification can be done by finding rules that partition the given data into predefined disjoint groups or classes. The task of classification is to predict the class of a new item; given that items belong to one of the classes, and given past instances (known as **training instances**) of items along with the classes to which they belong. The process of building a classifier starts by evaluating the **training data** (or **training set**), which is basically the old data that has already been classified by using the domain of the experts' knowledge.

For example, consider an insurance company that wants to decide whether or not to provide insurance facility to a new customer. The company maintains records of its existing customers, which may include name, address, gender, income, age, types of policies purchased, and prior claims experience. Some of this information may be used by the insurance company to define the insurance worthiness level of the new customers. For instance, the company may assign the insurance worthiness level of excellent, good, average, or bad to its customers depending on the prior claims experience.

Note that new customers cannot be classified on the basis of prior claims experience as this information is unavailable for new customers. Therefore, the company attempts to find some rules that classify its current customers on the basis of the attributes other than the prior claim experience. Consider four such rules that are based on two attributes, age and income.

Rule 1: $\forall \text{customer } C, C.\text{age} < 30 \text{ and } C.\text{income} \leq 30,000 \Rightarrow C.\text{insurance} = \text{bad}$

Rule 2: $\forall \text{customer } C, C.\text{age} \geq 30 \text{ and } C.\text{age} < 50 \text{ and } C.\text{income} > 75,000 \Rightarrow C.\text{insurance} = \text{excellent}$

Rule 3: $\forall \text{customer } C, (C.\text{age} \geq 50 \text{ and } C.\text{age} \leq 60) \text{ and } (C.\text{income} \geq 30,000 \text{ and } C.\text{income} \leq 75,000) \Rightarrow C.\text{insurance} = \text{good}$

Rule 4: $\forall \text{customer } C, C.\text{age} > 60 \text{ and } C.\text{income} > 30,000 \Rightarrow C.\text{insurance} = \text{average}$

This type of activity is known as **supervised learning** since the partitions are done on the basis of the training instances that are already partitioned into predefined classes. The actual data, or the population, may consist of all new and the existing customers of the company. There are several techniques used for classification. One of them is decision tree classification.

Decision Tree Classification A **decision tree** (also known as a **classification tree**) is a graphical representation of the classification rules. Each internal node of the decision tree is labelled with an attribute A_i . Each arc is labelled with the predicate (or condition), which can be applied to the attribute at the parent node. Each leaf node is labelled with one of the predefined classes. Therefore, each internal node is associated with a predicate and each leaf node is associated with a class. An example of decision tree is shown in Figure 14.15.

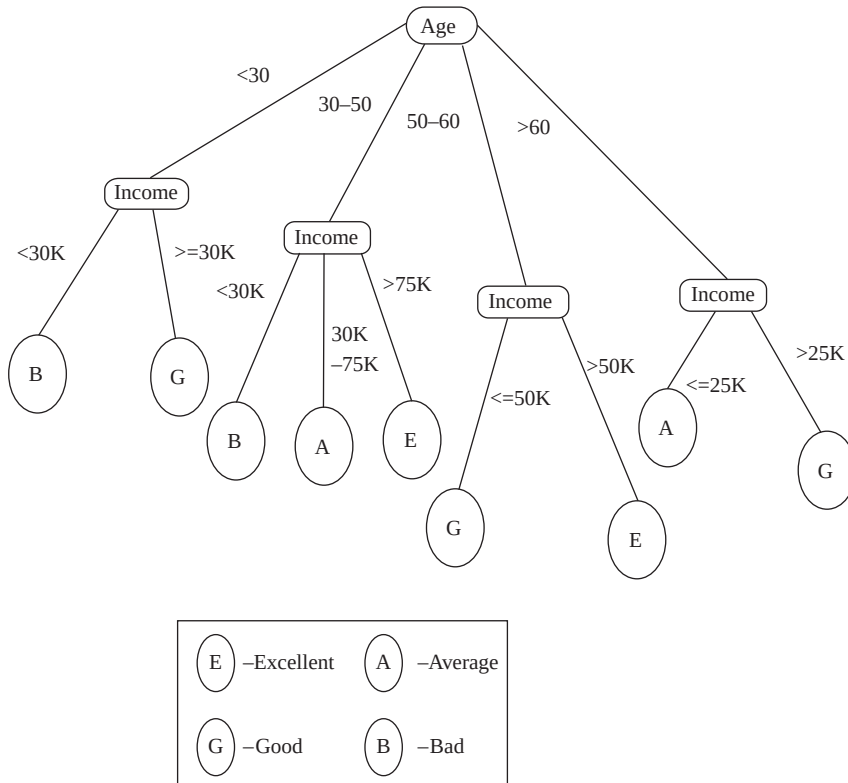


Fig. 14.15 An example of a decision tree

Given a set of training data, the question is how to build a decision tree. There are many algorithms to construct a decision tree. The most commonly used algorithm searches through the attributes of the training set and extracts the attribute that best separates the given instances. This attribute is called the **partitioning attribute**. If the partitioning attribute *A* perfectly classifies the training sets, the algorithm stops, otherwise it recursively selects partitioning attribute and partitioning predicate to create further child nodes. The algorithm uses a **greedy search**, that is, it picks the best attribute and never looks back to reconsider earlier choices.

In Figure 14.15, the attribute *age* is chosen as a partitioning attribute, and four child nodes—one for each partitioning predicate 0–30, 30–50, 50–60, and over 60—are created. For all these child nodes, the attribute *income* is chosen to further partition the training instances belonging to each child node. Depending on the value of the income, a class is associated with customer. For example, if the age of the customer is between 50 and 60, and his income is greater than Rs 75,000, then the class associated with him is ‘excellent’.

Clustering

The process of grouping the records together so that the degree of association is strong between the records of the same group and weak between the records of different groups is known as **clustering**. Unlike classification, clustering is **unsupervised learning** as no training sample is available to guide partitioning. The groups created in this case are also disjoint. Grouping of customers on the basis of similar buying patterns, grouping of students in a class on the basis of grades (A, B, C, D, E or F), etc., are some of the common examples of clustering.

The quality of clustering result depends on the similarity measure. A similarity function based on the distance is typically used if the data is numeric. For example, the Euclidean distance can be used to measure similarity. The Euclidean distance between two n -dimensional records (records with n attributes) r_i and r_j can be computed as follows:

$$D(r_i, r_j) = \sqrt{|r_{i1} - r_{j1}|^2 + |r_{i2} - r_{j2}|^2 + \dots + |r_{in} - r_{jn}|^2}$$

The smaller the distance between two records, the greater is the similarity between them. The most commonly used algorithm used for clustering is **k-means algorithm**, where k is the number of desired clusters, as shown in Figure 14.16.

Input: a set of m records $r_1, \dots, r_m$ and number of desired clusters k

Output: set of k clusters

Process:

- Step 1: Start
- Step 2: Randomly choose k records as the centroid (mean)
for k clusters;
- Step 3: Repeat steps 4 and 5 until no change
- Step 4: For each record r_i , find the distance of the record
from the centroid of each cluster and assign that
record to the cluster from which it has the
minimum distance;
- Step 5: Recompute the centroid for each cluster based
on the current records present in the cluster;
- Step 6: Stop

Fig. 14.16 *k-means algorithm for clustering*

Let us understand the algorithm with the help of an example. Consider a sample of 2D records shown in Table 14.4. Assume that the number of desired clusters $k = 2$. The centroid (mean) of a cluster C_i containing m n -dimensional records can be calculated as follows:

$$\bar{C}_i = \left(\frac{1}{m} \sum_{\forall r_j \in C_i} r_{j1}, \dots, \frac{1}{m} \sum_{\forall r_j \in C_i} r_{jn} \right)$$

Table 14.4 *Sample 2D data for clustering*

Age	Income (,000)
20	10
30	20
30	30
35	35
40	40
50	45

Step 1: Let the algorithm randomly choose record 2 for cluster C_1 and record 5 for cluster C_2 . Thus, the centroid for C_1 is (30, 20) and C_2 is (40, 40).

Step 2: Calculate the distance of the remaining records from the centroid of both C_1 and C_2 , and assign the record to the cluster from which it has the minimum distance as shown in Table 14.5.

Table 14.5 Distance of records from centroids of C_1 and C_2

Record	Distance from C_1	Distance from C_2	Cluster Selected
1	14.14	36.05	C_1
3	18.02	14.14	C_2
4	14.81	7.07	C_2
6	32.01	11.18	C_2

Now, records 1 and 2 are assigned to cluster C_1 and records 3, 4, 5, and 6 are assigned to cluster C_2 .

Step 3: Recompute the centroid of both the clusters. The new centroid for C_1 is (25, 15) and for C_2 is (38.75, 37.5). Again calculate the distance of all six records from the new centroid and assign the records to the appropriate cluster as shown in Table 14.6.

Table 14.6 Distance of all six records from new centroids of C_1 and C_2

Record	Distance from C_1	Distance from C_2	Cluster Selected
1	7.07	33.28	C_1
2	7.07	19.56	C_1
3	14.81	11.52	C_2
4	22.36	4.51	C_2
5	29.15	2.80	C_2
6	39.05	13.52	C_2

After the step 3, the records 1 and 2 are assigned to cluster C_1 and records 3, 4, 5, and 6 are assigned to cluster C_2 . Since after step 3, the records remain in the same cluster as they were in step 2, the algorithm terminates after this step.

14.3.3 Application Areas of Data Mining

Data mining can be used in many areas such as banking, finance, and telecommunications industries. Some of the significant applications of data mining technologies are given here.

- ⇒ **Market-basket analysis:** Companies can analyse the customer behaviour based on their buying patterns, and may launch products according to the customer needs. The companies can also decide their marketing strategies such as advertising, store location, etc. They can also segment their customers, stores or products based on the data discovered after data mining.
- ⇒ **Investment analysis:** Customers can also look at the areas such as stocks, bonds, and mutual funds where they can invest their money to get good returns.
- ⇒ **Fraud detection:** By finding the association between frauds, new frauds can be detected using data mining.

- ⇒ **Manufacturing:** Companies can optimize their resources like machines, manpower, and materials by applying data mining technologies.
- ⇒ **Credit risk analysis:** Given a set of customers and an assessment of their credit worthiness, descriptions for various classes can be developed. These descriptions can be used to classify a new customer into one of the classes.

Other than these significant areas, some other applications of data mining include quality control, process control, medical management, store management, student recruitment and retention, pharmaceutical research, electronic commerce, claims processing, and so on. The full spectrum of application areas is very broad.

● SUMMARY ●

1. The database applications that record and update data about transactions are known as online transaction processing (OLTP) applications.
2. The systems that aim to extract high-level information stored in OLTP databases, and to use that information in making a variety of decisions that are important for the organization are known as decision-support systems (DSS).
3. Several tools and techniques are available that provide the correct level of information to help the decision makers in decision-making. Some of these tools and techniques include data warehousing, online analytical processing (OLAP), and data mining.
4. A data warehouse (DW) is a repository of suitable operational data (data that documents the everyday operations of an organization) gathered from multiple sources, stored under a unified schema, at a single site.
5. According to William Inmon, the father of the modern data warehouse, a data warehouse is a subject-oriented, integrated, time-variant, non-volatile collection of data in support of management's decisions.
6. The main advantage of using a data warehouse is that a data analyst can perform complex queries and analyses on the information stored in data warehouse without affecting the OLTP systems.
7. A data warehouse basically consists of three components, namely, the data sources, the ETL, and the schema of data stored in data warehouse including the metadata.
8. While creating a data warehouse, the data is extracted from multiple, heterogeneous data sources. Then data cleansing is performed to remove any kind of minor errors and inconsistencies. This data is then transformed to accommodate semantic mismatches. The cleaned and transformed data is finally loaded into the warehouse.
9. The entire process of getting data into the data warehouse is called extract, transform, and load (ETL) process.
10. The third and the most important component of the data warehouse is the metadata repository, which keeps track of currently stored data. It contains the description of the data including its schema definition.
11. The data in a warehouse is usually multidimensional data having measure attributes and dimension attributes.
12. The attributes that measure some value and can be aggregated upon are called measure attributes. On the other hand, the attributes that define the dimensions on which the measure attributes and their summaries are viewed are called dimension attributes.
13. The relations containing such multidimensional data are called fact tables. The dimension attributes in the fact tables are the primary keys in other tables called dimension tables, which store additional information about the dimensions.
14. The multidimensional data in a data warehouse can be represented either using a star schema or a snowflake schema.

15. The star schema is the simplest data warehouse schema, which consists of a fact table with a single table for each dimension. The snowflake schema is a variation of star schema, which may have multiple levels of dimension tables.
16. Online Analytical Processing (OLAP) is a category of software technology that enables analysts, managers, and executives to analyse the complex data derived from the data warehouse.
17. Since the data in the data warehouse is multidimensional in nature, it may be difficult to analyse. Therefore, it can be displayed in the form of cross-tabulation.
18. A cross-tab is a two-dimensional table in which values for one attribute (say A) form the row headers, and values for another attribute (say B) form the column headers. Each cell in the cross-tab can be identified by (a_i, b_j) , where a_i is a value for A and b_j is a value for B.
19. A data cube (also called a multidimensional cube or OLAP cube or hypercube) is the generalization of a two-dimensional cross-tab to n dimensions.
20. A data analyst can look at different cross-tabs on the same data by interactively selecting the dimensions in the cross-tab. The technique of changing from one-dimensional orientation to another is known as pivoting (or rotation).
21. In case a cross-tabulation is done for a specific value other than all for the fixed third dimension, it is called slice operation or slicing. Slicing is sometimes called dicing when two or more dimensions are fixed.
22. The operation that converts data with a finer-granularity to the coarser-granularity with the help of aggregation is known as rollup operation. An operation that converts data with a coarser-granularity to the finer-granularity is known as drill down operation.
23. There are three ways of implementing OLAP system. The traditional OLAP systems use multidimensional arrays in memory to store data cubes and thus, are referred to as multidimensional OLAP (MOLAP) systems.
24. The OLAP systems that work directly with relational databases are called relational OLAP (ROLAP) systems.
25. The systems that include the best of ROLAP and MOLAP are known as hybrid OLAP (HOLAP) systems. These systems use main memory to hold some summaries, and relational tables to hold base data and other summaries.
26. The extraction of hidden and predictive information from a vast amount of data is known as data mining.
27. Data mining deals with knowledge discovery in databases (KDD). KDD is the non-trivial extraction of implicit, previously unknown, and potentially useful information from the databases.
28. The knowledge discovered from the database during data mining includes association rules, classification, and clustering.
29. The task of mining association rules is to find interesting relationship among various items in a given data set.
30. An association rule must have an associated population, which consists of a set of instances.
31. For an association rule to be of interest to an analyst, the rule should satisfy two interest measures, namely, support and confidence.
32. The itemsets that have support above the minimum prespecified support are known as large (or frequent) itemsets.
33. The task of classification is to predict the class of a new item given that items belong to one of the classes, and given past instances (known as training instances) of items along with the classes to which they belong.
34. A decision tree (also known as classification tree) is a graphical representation of the classification rules.
35. The process of grouping the records together so that the degree of association is strong between the records of the same group and weak between the records of different groups is known as clustering.

● KEY TERMS ●

- | | |
|--|--|
| <ul style="list-style-type: none"> ● Online transaction processing (OLTP) system ● Decision-supports ystem(D SS) ● Data warehouse ● Data mart ● Data extraction ● Data cleansing ● Fuzzylookup ● Back-flushing ● Extract, transform and load (ETL) process ● Metadataare pository ● Technicalme tadata ● Businessme tadata ● Multidimensionalda ta ● Measurea ttributes ● Dimensiona ttributes ● Factta ble ● Dimension table (lookup table) ● Stars chema ● Snowflake schema ● Online analytical processing (OLAP) ● Pivot table (cross-tab) ● Data cube (OLAP cube) ● Pivoting(rota tion) ● Slicing and dicing ● Rollup and drill down operation ● Multidimensional OLAP (MOLAP) | <ul style="list-style-type: none"> ● Relational OLAP (ROLAP) ● Hybrid OLAP (HOLAP) ● Datamining ● Knowledge discovery in database (KDD) ● Associationr ules ● Itemset ● Antecedent ● Consequent ● Population ● Instances ● Support(pre valence) ● Confidence (strength) ● Large (frequent) itemset ● Downwardc losure ● Anti-monotonicity ● Aprioria lgorithm ● Classification ● Trainings tances ● Training data (training set) ● Supervisedle arning ● Decision tree (classification tree) ● Greedys earch ● Clustering ● Unsupervisedle arning ● k-meansa lgorithm |
|--|--|

● EXERCISES ●

A. Multiple Choice Questions

1. Which of the following is not a basic characteristic of a data warehouse?

(a) u bject oriented	(b) Volatile
(c) Time ay ing	(d) I ntegrated
2. Which of the following statements is incorrect for a data warehouse?

(a) A data warehouse is mainly designed to handle <i>ad hoc</i> queries	(b) Data warehouses generally use de-normalized or partially de-normalized schemas to optimize query performance
---	--

- (c) A data warehouse is always up to date, and reflects the current state of each business transaction
(d) None of these
3. Which of the following is a basic component of a data warehouse?
(a) ~~data~~ sources (b) ~~the~~ process
(c) Metadata repository (d) All of these
4. Which of the following tables contain the primary information in the data warehouse?
(a) Primary table (b) Dimension table
(c) ~~act~~ ~~the~~ (d) Lookup ~~table~~
5. The technique of changing from one-dimensional orientation to another is known as _____.
(a) ~~Plotting~~ (b) ~~Sliding~~
(c) ~~indexing~~ (d) ~~roll~~ ~~down~~
6. Which of the following OLAP systems work directly with relational databases?
(a) ~~OLAP~~ (b) ~~OLAP~~
(c) ~~OLAP~~ (d) ~~one~~ ~~offline~~
7. Which of the following is not a part of the KDD process?
(a) Data selection (b) Data cleansing
(c) ~~Enhancement~~ (d) ~~data~~ ~~warehousing~~
8. The percentage or fraction of the population that satisfies both the antecedent and consequent of the rule is known as _____.
(a) ~~Support~~ (b) ~~population~~
(c) Confidence (d) ~~Itance~~
9. Which of the following techniques is used for classification during data mining?
(a) Apriori algorithm (b) k-means algorithm
(c) Decision tree (d) Any of these
10. Which of the following is an application area of data mining?
(a) Market-basket analysis (b) Fraud detection
(c) Investment analysis (d) All of these

B. Fill in the Blanks

1. A _____ is a repository of suitable operational data (data that documents the every-day operations of an organization) gathered from multiple sources, stored under a unified schema, at a single site.
2. This entire process of getting data into the data warehouse is called _____ process.
3. The _____ contains the description of the data including its schema definition.
4. The multidimensional data in a data warehouse can be represented either using a _____ or a _____.

5. _____ is a category of software technology that enables analysts, managers, and executives to analyse the complex data derived from the data warehouse.
6. A _____ is the generalization of a two-dimensional cross-tab to n dimensions.
7. The operation that converts data with a finer-granularity to the coarser-granularity with the help of aggregation is known as _____ operation.
8. The extraction of hidden and predictive information from such large databases is known as _____.
9. _____ describes a relationship between a set of items that people tend to buy together.
10. The _____ is basically the old data that has already been classified by using the domain of the experts' knowledge.

C. Answer the Questions

1. What are decision-support systems? What are the various tools and techniques that support decision-making activities?
2. What is a data warehouse? How does it differ from an OLTP system?
3. Define the following terms:
 - (a) ~~TEL~~ ~~pross~~ (b) ~~Fny~~ lookup
 - (c) ~~ada~~ ~~ube~~ (d) ~~OL~~AP
 - (e) Frequent itemsets (f) Downward closure and anti-monotonicity
4. Discuss the four basic characteristics of a data warehouse. What are the advantages of using a data warehouse?
5. Describe the steps of building a data warehouse.
6. What is role of the metadata repository in a data warehouse?
7. What are dimension and measure attributes in the multidimensional data model?
8. What is a fact table? How does it differ from a dimension table?
9. Differentiate between a star schema and snowflake schema representation of a data warehouse.
10. What is OLAP? What are the various ways of implementing OLAP? What is the basic difference between MOLAP and ROLAP?
11. What is a pivot table? How does it help in analysing multidimensional data? Explain with the help of an example.
12. Discuss the various functionalities that an OLAP system provides to analyze the data. Explain with the help of suitable examples.
13. What is data mining? How is it different from knowledge discovery in artificial intelligence?
14. 'Though data in large operational databases and flat files can be provided as input for the data mining process, data in data warehouses is more suitable for data mining process.' Why?
15. What is KDD? Discuss the different phases of KDD process.
16. What are association rules in the context of data mining? Describe the terms support and confidence with the help of suitable examples.

17. Discuss the Apriori algorithm for generating large itemsets. Apply this algorithm for generating large itemset on the following dataset:

Transaction_ID	Items Purchased
T101	I_1, I_3, I_4
T200	I_2, I_3, I_5
T550	I_1, I_2, I_3, I_5
T650	I_2, I_5

18. What is classification in context of data mining? Why is it called supervised learning? How is classification different from clustering?
19. Discuss the k-means algorithm for clustering with the help of an example.
20. What are the various applications of data mining?

INFORMATION RETRIEVAL

After reading this chapter, the reader will understand:

- Information retrieval process and information retrieval system
- Differences between database system and information retrieval system
- Vector space model for document representation
- TF/IDF based ranking
- Similarity based retrieval, cosine similarity, and relevance feedback
- Performance measures, recall, and precision
- Inverted index
- Signature files
- Working of web search engines

Any web page, usually designed in HTML or XML, is considered to be a document. The web documents may be multimedia objects such as images or video clips or it may also contain data in the form of text. There are billions of documents on the Web, which is the biggest source of information. However, unlike structured data in relational databases, textual data is unstructured. The management of such a large number of documents in DBMS is a big challenge, and therefore, becomes a point of interest for database researchers.

Retrieving text information from such a gigantic digital library is a difficult task. The process of extracting information from unstructured textual data is referred to as **information retrieval (IR)**. The emphasis in information retrieval system is different from the one in database systems. The main emphasis is on querying based on keywords and ranking documents on the basis of their relevance to the query. This chapter discusses IR systems and its various features.

15.1 INFORMATION RETRIEVAL SYSTEMS

Information retrieval is the process of searching for documents, information within the documents or within the databases. The process of information retrieval consists of locating relevant documents based on user inputs. The users of IR systems request to retrieve a particular set of documents by providing a set of words or **keywords**. Information retrieval system locates and returns the documents whose associated keywords match with the given keywords. For example, the keywords ‘computer memory’ and ‘memory system’ may be used to locate articles/documents describing memory system of the computer.

There is much similarity between database systems and IR systems, such as both have the common objective of supporting searches over collection of data. However, several issues that IR systems are dealing with are not addressed in database systems and vice versa. Some of these issues are listed in Table 15.1.

Table 15.1 *Differences between database system and IR system*

Database System	IR System
Data in database system is rigidly structured and supports search based on very general class of queries.	IR system deals with unstructured data and supports approximate searching based on the keywords.
It supports the notion of transaction and its associated requirements, such as concurrency control and durability.	Notion of transaction is less significant in IR system and it is mainly designed to handle read mostly workload.
Database systems do not order the search result in terms of its relevance to the query.	Ranking of search documents in terms of their relevance to the keyword is the main feature of IR system.

IR systems support search based on query expressions, which are formed using keywords and logical connectives ‘*and*’, ‘*or*’, and ‘*not*’. For example, a user may ask to retrieve documents that contain the keywords ‘mobile technology *or* CDMA’, or documents containing keywords ‘health *and* wealth’, or even the documents containing keywords ‘computer *but not* software’. If a query contains keywords without any of the logical connectives, it is assumed to have ‘*and*’ implicitly connecting the keywords.

Information retrieval based on keywords are not only used for retrieving textual data, but can also be used for retrieving other types of objects such as audio or video objects. Such specialized multimedia objects have descriptive keywords associated with them. For example, a song can be associated with several related keywords such as title, movie name, and actors.

Information retrieval systems also support **full text** retrieval in which each word in the document is considered to be a keyword. Full text retrieval is essential for unstructured data, since it is possible that document does not contain information about words that are keywords. Note that in full text retrieval where each word is a keyword, we can use **term** to refer words in a document.

IR system typically locates and returns all the documents relevant to query. In information retrieval, a query does not uniquely identify a single document. Instead a collection of document may match the given query. More sophisticated IR systems rank the resultant documents in the order of their relevance to the query. The top-ranked documents are then shown to the user as the query result. The relevance of a document to a query is estimated by using information such as a term occurrence. Ranking the document in the order of their relevance to the query is discussed in Section 15.3.

Learn More

Documents are added to IR system frequently and indexes that are used to locate the documents are updated periodically. Therefore, if indexes are not up to date, it might be possible that documents that are highly relevant to a given query are available in the IR system but are not retrieved in the query result.

15.1.1 Document Representation

As we know, Web contains billions of documents, which contain important information and data. We need a simple model to handle them with ease. One of the most widely used models is *vector space model*. In this model, each document is transformed from full text version to a document vector, which describes the contents of the document. Each word that appears in the collection of documents corresponds to a dimension in the vector, and each document is represented as a vector with one entry per word. For instance, if a given word t appears n times in a document d , the vector for document d contains value n in position t . The document vector for any document d contains nonzero value for the word that appears in d .

Consider a set of text documents shown in Figure 15.1. Each document in the set can be represented as a document vector, which is shown in Figure 15.2. In this representation, each row corresponds to a document and each dimension corresponds to a term in the collection of document. Such vector representation of the documents is called **vector space model**.

ID	Document contents
A	computer keyboard mouse memory mobile
B	keyboard language memory device driver
C	mouse CPU keyboard mobile mouse

Fig. 15.1 Text documents with some records

ID	computer	CPU	device	driver	keyboard	language	memory	mobile	mouse
A	1	0	0	0	1	0	1	1	1
B	0	0	1	1	1	1	1	0	0
C	0	1	0	0	1	0	0	1	2

Fig. 15.2 Example of a document vector

This model is widely used for information retrieval, indexing, and relevance ranking. However, it has some limitations as well that are given as follows:

- ⇒ Representation of long documents is poor because of their poor similarity values.
- ⇒ Similar context documents having different term vocabulary cannot be associated.
- ⇒ The order of term appearance in the document is lost in this representation.

15.1.2 TF/IDF-Based Ranking

In document vector, the value for a term is the number of occurrences of that word in the document or simply the **term frequency (TF)**. It gives the impression that higher the TF, more relevant the document is. However, just counting the number of occurrences is not a good idea. It is because some words appear more frequently in the document than other words. Secondly, number of occurrences also depends on the length of the document.

However, TF can be used in measuring the relevance R of a document d to a given term t using the following formula.

$$R(d, t) = \log (1 + a(d, t) / a(d))$$

where $a(d, t)$ represents TF (the number of times term t appears in the document d) and $a(d)$ represents total number of terms in the documents. Relevance of document $R(d, t)$ increases with TF. However, it is not directly proportional to TF because the formula takes the document length into account.



Many systems refine the result by considering the document more relevant if the term occurs in the title, or the heading, or the author list.

Some common words that appear in almost all the documents are not very useful in searches. For instance, all textual documents in English contain words such as ‘a’, ‘an’, ‘the’, ‘and’, and so on. Set of these most common words, called **stop words**, are defined by IR systems, and documents are preprocessed to ignore these words while searching.

Even after ignoring stop words, all the terms used as keywords are not equally significant. This is because some terms appear more frequently than others in the set of documents. Suppose a query contains two terms ‘MBA’ and ‘course’, it is likely to have much more frequency of occurrence of term ‘course’ than ‘MBA’. To get better results, we should give more importance to the term ‘MBA’ than the term ‘course’.

This can be achieved by assigning weights to terms in the document vector by using **inverse document frequency (IDF)**. IDF of a term t is defined as $1/a(t)$, where $a(t)$ represents total number of documents that contains the term t . The relevance of a document d to a query Q is defined as

$$R(d, Q) = \sum_{t \in Q} a(d, t) * IDF(t)$$

This approach of using term frequency and inverse document frequency as a measure of the relevance of a document is called TF/IDF approach.

15.1.3 Similarity-Based Retrieval

Certain IR systems allow **similarity-based retrieval** whereby users can retrieve documents that are similar to a given document d . Similarity between two documents may be defined on the basis of common terms. The set of n terms in the document d with highest values of $a(d, t) * IDF(t)$ are used as a query to find other relevant documents. The document that is most similar to the query is ranked highest.

Consider n terms $t_1, t_2, \dots, t_n$ that appear in the collection of documents. The document vector for the document collection can be visualized as an n -dimensional space, in which each dimension corresponds to a term. Any document d is represented by a point in this space, with the value of j^{th} coordinate of the point being $R(d, t_j)$. For a two-dimensional space, Figure 15.3 shows document vector for two documents d_1 and d_2 , along with a query Q .

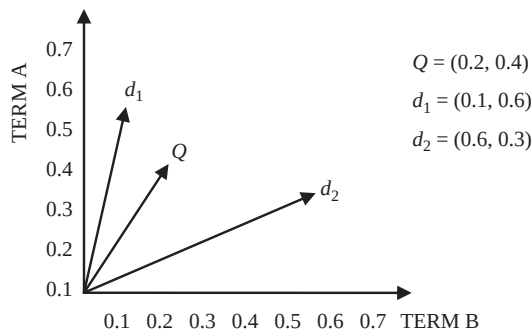


Fig. 15.3 Document similarity

Similarity between two documents d_1 and d_2 is defined by **cosine similarity** metric as

$$\frac{\sum_{j=1}^n R(d_1, t_j) R(d_2, t_j)}{\sqrt{\sum_{j=1}^n R(d_1, t_j)^2} \sqrt{\sum_{j=1}^n R(d_2, t_j)^2}}$$

where, $R(d, t) = A(d, t) * IDF(t)$. Note that the metric derives its name ‘cosine similarity’ from the fact that it computes the cosine of the angle between two vectors, each of which represents a document.

If the system finds a large set of documents that are similar to the query document d , it may present only a few highest ranked documents, and allow the user to select the most relevant few. After this, the system starts searching documents that are similar to d and to the selected documents. The set of documents returned by the system is likely to be what the user is looking for. This scheme of searching is called **relevance feedback**. This scheme can also be used to find relevant documents from a large set of documents that match the given query keywords. In this situation, an alternative approach to the relevance feedback is to allow user to filter the query by adding more keywords.

15.1.4 Performance Measures

In order to determine the accuracy or quality of retrieved result, IR system should be evaluated. The two widely used measures for evaluating the performance of IR systems are *recall* and *precision*. **Precision** is defined as the percentage of retrieved documents that are relevant to the query. **Recall** is defined as the percentage of relevant documents in the database that are retrieved in response to the query.

In information retrieval, a perfect recall of 100% means that every relevant document is retrieved but it does not guarantee that irrelevant documents are not retrieved. On the other hand, a perfect precision of 100% means that every retrieved document is relevant but it does not guarantee that all relevant documents are retrieved.

Often, there is an inverse relationship between recall and precision. It means that it is possible to increase one at the cost of other. Generally, it is relatively easy for an IR system to achieve either high recall or high precision; however, it is challenging to achieve both. Thus, instead of discussing recall and precision in isolation, either values of one measure are compared for a fixed level at the other measure. For instance, 80% recall at a level of 75% precision.

15.2 INDEXING OF TEXT DOCUMENTS

Efficient index structure plays an important role in efficient processing of queries in an IR system. Size of the index depends on the number of terms in the document collection. Thus, documents are preprocessed to eliminate irrelevant terms such as stop words. In addition, IR system may also apply **stemming**, in which all related terms are reduced to the simplest and the most significant form without loss of generality. For example, the terms *index*, *indexing*, and *indexed* stem to the simplest term *index*. This step provides two advantages, firstly, it reduces the number of terms for indexing, secondly, it allows user to retrieve documents that contain variant of exact query term. Two widely used indexing techniques are *inverted index* and *signature file index*.

Inverted Index

Inverted index maps each term t to a list (called **inverted list**) of documents that contain the term t . The entry for a document in the inverted list also provides location within the document where

t occurs. The collection of inverted lists is called **postings file**. Consider the Figure 15.4, which shows the inverted index of our example documents shown in Figure 15.1.

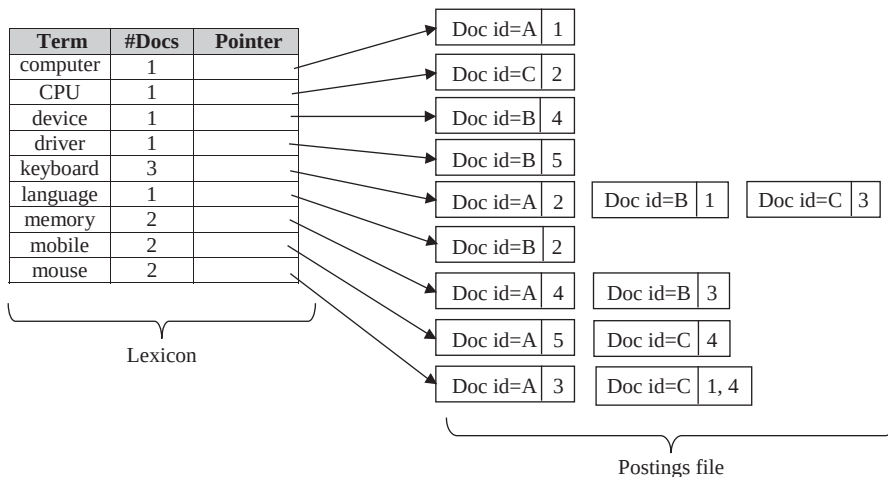


Fig. 15.4 An example of inverted index

Since the term ‘computer’ occurs at location 1 in document A, the entry for document A in the inverted list for term ‘computer’ contains location 1. Similarly, the entry for document C for term ‘mouse’ contains location 1 and 4. Such a data structure not only enables fast retrieval of documents for a given query but also supports relevance ranking. However, inverted index file requires large amount of space. Sometimes the index grows up to several 100 times of the original file, which is a major disadvantage of inverted index.

An inverted list can be very large for large document collections, and thus, must be stored on disk. Mostly, a list spans multiple disk pages and is maintained as a linked list of pages. Alternatively, system might keep it in consecutive pages to minimize the I/O operations required to access the list. Further, to quickly find the inverted list for a given query keyword, a second index structure such as B⁺-tree or hash index is created for all the possible keywords. This index, called the **lexicon**, not only contains the address of inverted lists on disks, but it may also contain some additional information. For instance, in our example, each entry in the lexicon index contains address of inverted list and the number of documents in which the term occurs. Lexicon is much smaller than the postings file, and is kept in the memory. Thus, it enables quick retrieval of the inverted list for a given query term.

A query involving conjunction of several terms is evaluated by retrieving the inverted lists of the query terms one by one and intersecting them. On the other hand, a query involving disjunction of several terms is evaluated by computing union of relevant inverted lists.

Signature Files

Another index that can be created for text documents to support efficient retrieval is **signature file**. In this technique, each document is

Word	Hash value
computer	0001
device	
CPU	0010
driver	
keyboard	0100
language	
mobile	
mouse	1000
memory	

Fig. 15.5 Words with hash value

ID	Document contents	Signature
A	computer keyboard mouse memory mobile	1101
B	keyboard language memory device driver	1111
C	mouse CPU keyboard mobile mouse	1110

Fig. 15.6 An example of signature file

associated with an index record called **signature** of the document. Each signature is represented by a fixed size of bits called **signature width**.

To calculate the signature of a document, first the hash value of each word in the document is calculated by applying a hash function. A hash function may group different words to same buckets. For instance, suppose a hash function is applied on each word in the set of documents shown in Figure 15.1, then the hash values we get for each word is shown in Figure 15.5.

For each word in the document, we set the bits in the signature that are set in the hash value of the word. Since, hash function may map different words to the same bit, the bit could be set multiple times by different words in the signature (until we have a unique bit for each word). Two signatures are said to be same, if at least all those bits that are set in one signature are also set in another. Signature file for the set of documents is shown in Figure 15.6.

A query involving conjunction of several terms is evaluated as follows:

1. First the query signature is generated by applying hash function to each word in the query.
2. Signature file is scanned and all the documents whose signature matches the query signature are retrieved.

Since the signature does not uniquely identify the words contained in a document, all the retrieved documents must be checked to determine whether they actually contain the required words. The retrieved documents that do not contain all the words in the query are called **false positive**.

A query involving disjunction of several terms is evaluated as follows:

1. For each word in the query, a query signature is generated.
2. Signature file is scanned and all the documents whose signature matches signature of any word in the query are retrieved.

Learn More

Since we need to scan the entire signature file for each query, the performance degrades if the signature file cannot fit in main memory at once. Alternatively, the signature file can be vertically partitioned into bit slices; such an index is called **bit-sliced signature file**. Now, for a query with n bits set in the query signature, we need to retrieve only n bit slices.

15.3 WEB SEARCH ENGINES

For a search to be successful, web search engines must find all the relevant documents in the extremely large number of documents. Documents have links (hyperlinks) to other documents, which help in locating relevant documents for a given search. Web search engines **crawl** the Web to locate and gather information found in the documents to a combined index. Since there is extremely large number of

documents on the Web, every search engine covers only a small portion of Web. The crawlers may take several days to perform a single crawl of all the pages they cover. Crawling consists of several processes that run on multiple machines.

Crawling starts at a collection of pages with many hyperlinks. These hyperlinks are followed to find new pages. All the hyperlinks that are to be visited are stored in a database. The new hyperlinks found during a crawl are inserted into the database, and may be assigned to another crawler program, or may be visited later. All the pages found during a crawl are given to an indexing system, which creates inverted lists for the terms that occur in the document. The process is iterated, keeping track of the hyperlinks that have been visited to avoid re-visiting them in a single crawl. Since, the process retrieves enormous amount of pages, creating an index of them is an expensive task. Fortunately, the task is parallelizable, so indexing systems run on multiple machines in parallel.

To keep the index up to date, the pages are re-fetched periodically to obtain the updated information. Pages of new launched sites are added in the index, and the sites that no longer exist are discarded. Adding new pages to the index that is being used to handle queries is not a good technique because it requires concurrency control on the index, which affects the query and update performance. Alternatively, two copies of index are maintained—one is used to handle queries and another one is updated by adding new pages. The two copies are switched over periodically; with old one being updated and new one being used to handle queries.

Search engines must also support a high query rate over such vast indexes. To do so, indexes may be partitioned and kept in main memory across several machines. Each machine maintains inverted index for those terms that are mapped to that machine (for example, by hashing the term). System sends the queries to the machine that contains index for the terms that occurs in the query, thus, balancing the load among multiple machines. Note that a query may have to be sent to multiple machines if the terms it contains are handled by different machines. However, it is not a big problem because web queries rarely contain more than two terms.



Mostly search engines have several thousands of machines involved in the process of crawling, indexing, and handling the queries.

Using Hyperlink Information

As discussed earlier, all the documents retrieved in response to a query should be ranked on the basis of their relevance to the given query. Early search engines use TF/IDF-based ranking (discussed in Section 15.1.2), which uses information within the single page to estimate its relevance to a query. However, most relevant pages may not contain the search terms at all, or may have few occurrences of the query term, thus, would not be ranked higher in the result. For instance, the home page of *The Times of India* may not contain the term ‘newspaper’. However, it is likely that a site containing the term ‘newspaper’ has a link to the home page of *The Times of India*. Thus, researchers exploit the use of information hidden in hyperlinks to get better result.

Just like the way index of a book keeps links to related topics in the book, good web pages are linked too. In fact, the web pages are classified into two types, *authorities* and *hubs*. The **hub** is a page that itself may not contain the actual information on a topic, but stores links to many related pages that contain actual information. On the other hand, **authority** is a page that contains actual information on a topic; however, it may not contain links to other related pages.

The relationship between hubs and authorities influences a link-based search algorithm, the **HITS algorithm**, which retrieves highly relevant pages in response to a given query. The algorithm represents the web as a directed graph with each node corresponding to a web page. If there is a hyperlink from page *X* to page *Y*, there exists an edge in the graph between the two corresponding nodes.

The HITS algorithm proceeds in two steps, namely, *sampling step* and *iteration step*. In the **sampling step**, the algorithm collects the pages that are most relevant to the given query using some traditional method. For instance, all the web pages that contain the occurrence of query terms are collected. The resulting set of pages is called the **root set**. Since some relevant authoritative pages may not contain the query words, the root set may not contain all the relevant pages. However, it may be the case that some pages in the root set contain hyperlinks to those relevant authoritative pages or that some of those authoritative pages have hyperlinks to pages in the root set. Such a page is called a **link page**, if it is linked to some pages in the root set. All link pages are added in the root set to collect all the potentially relevant pages. The resultant set of pages is called the **base set**, which includes all the root pages and all the link pages. A web page in the base set is referred to as **base page**.

Since, the base set can be quite large, each base page is associated with a hub-prestige value and an authority-prestige value. Hub-prestige value indicates the quality of the page as a hub, and the authority-prestige value indicates the quality of the page as an authority. A hub is considered a good hub and assigned higher hub-prestige value if it stores hyperlinks to many pages with high authority-prestige value. On the other hand, an authority is considered a good authority and is assigned higher authority-prestige value if many good hubs have link to it. The pages with good authority-prestige value and hub-prestige value among the base pages are located in the **iteration step**.

Initially we do not know which pages are good hubs and authorities, so all the values are assigned to one. Later, authority-prestige and hub-prestige values are updated iteratively. Rather taking into account the words that occur in the base page, iteration step is concerned with the hyperlinks that the base pages store.

For instance, consider a page b in the base set with authority-prestige value V_a and hub-prestige value V_h . In each iteration, the value V_a is updated to the sum of V_h of all the pages that have link to b . On the other hand, the value V_h is updated to the sum of V_a of all the pages that are pointed to by b .

● SUMMARY ●

1. The process of extracting information from unstructured textual data is referred to as information retrieval (IR).
2. The main emphasis in IR system is on querying based on keywords and ranking documents on the basis of their relevance to the query.
3. The users of IR systems request to retrieve a particular set of documents by providing a set of words or keywords. IR system locates and returns the documents whose associated keywords match with the given keywords.
4. One of the main features of IR system is ranking of search documents in terms of their relevance to the query. The relevance of a document to a query is estimated by using information such as term occurrence. The top-ranked documents are shown to the user as the query result.
5. IR systems also support full text retrieval in which each word in the document is considered to be a keyword. In full text retrieval where each word is a keyword, we can use term to refer words in a document.
6. To handle billions of documents in Web, vector space model is used. In this model, each document is transformed from full text version to a document vector, which describes the contents of the document. Each word that appears in the collection of documents corresponds to a dimension in the vector, and each document is represented as a vector with one entry per word.
7. In document vector, the value for a term is the number of occurrences of that word in the document or simply the term frequency (TF). Term frequency can be used in measuring the relevance of a document to a given term.
8. All the terms that are used as keywords are not equally significant, thus, we should give more importance to significant terms to get better results. This can be achieved by assigning weights to terms in the document vector by using inverse document frequency (IDF).

9. Certain IR systems allow similarity-based retrieval, whereby users can retrieve documents that are similar to a given document d . Similarity between two documents d_1 and d_2 is defined by cosine similarity metric.
10. The two widely used measures for evaluating the performance of IR systems are recall and precision. Generally, it is relatively easy for an IR system to achieve either high recall or high precision; however, it is challenging to achieve both.
11. Efficient index structure plays an important role in efficient processing of queries in an IR system. Two widely used indexing techniques are inverted index and signature file index.
12. Documents have links (hyperlinks) to other documents, which help in locating relevant documents for a given search.
13. Web search engines crawl the Web to locate and gather information found in the documents to a combined index. Crawling consists of several processes that run on multiple machines.
14. To keep the index up to date, the pages are re-fetched periodically to obtain the updated information.
15. Just like the way index of a book keeps links to related topics in the book, good web pages are linked too. In fact, the web pages are classified into two types, authorities and hubs.
16. The hub itself may not contain actual information on a topic, but stores links to many related pages that contain actual information. On the other hand, authority page is one that contains actual information on a topic; however, it may not contain links to other related pages.
17. The relationship between hubs and authorities influences a link-based search algorithm, the HITS algorithm, which retrieves highly relevant pages in response to a given query.
18. The HITS algorithm proceeds in two steps, namely, sampling step and iteration step.
19. In the sampling step, the algorithm collects the pages that are most relevant to the given query using some traditional method. The resultant set of pages is called the base set. A web page in the base set is referred to as base page.
20. Each base page is associated with a hub-prestige value and an authority-prestige value. Hub-prestige value indicates the quality of the page as a hub, and the authority-prestige value indicates the quality of the page as an authority.
21. The pages with good authority-prestige value and hub-prestige value among the base pages are located in the iteration step.

● KEY TERMS ●

- | | |
|--|---|
| <ul style="list-style-type: none"> ● Information retrieval (IR) ● Information retrieval system ● Keywords ● Full text retrieval ● Term ● Vector space model ● Term frequency (TF) ● Stop words ● Inverse document frequency (IDF) ● Similarity-based retrieval ● Relevance feedback ● Recall ● Precision ● Stemming ● Inverted index ● Inverted list | <ul style="list-style-type: none"> ● Postings file ● Lexicon ● Signature file ● Signature ● Signature width ● False positive ● Crawl ● Hub and authority ● HITS algorithm ● Sampling step ● Root set ● Link page ● Base set ● Base page ● Iteration step |
|--|---|

 ● EXERCISES ● **A. Multiple Choice Questions**

1. Which of the following is incorrect?
 - (a) Database system deals with unstructured data
 - (b) Database system supports the notion of transaction
 - (c) Database system orders the search result according to its relevancy to query
 - (d) None of these
2. Which of the following should be ignored while searching documents?
 - (a) Keywords
 - (b) Term
 - (c) Stop words
 - (d) Term frequency
3. A query containing keywords without any of the logical connectives is assumed to have _____ implicitly.
 - (a) or
 - (b) and
 - (c) not
 - (d) Both (a) and (c)
4. Which of the following term is not related with inverted index?
 - (a) Signature
 - (b) Postings file
 - (c) Lexicon
 - (d) Inverted list
5. Which of the following is correct?
 - (a) The new hyperlinks found during crawl cannot be assigned to another crawler program
 - (b) Indexing systems cannot run on multiple machines in parallel
 - (c) While updating the index, the sites that no longer exist cannot be discarded
 - (d) Adding new pages to index in use is not a good technique

B. Fill in the Blanks

1. Unlike structured data in relational databases, textual data is _____.
2. The users of IR system request to retrieve a particular set of documents by providing _____.
3. IR systems also support _____ retrieval in which each word in the document is considered to be a keyword.
4. In vector space model, the representation of _____ documents is poor because of their poor similarity values.
5. The approach of using term frequency and inverse document frequency as a measure of the relevance of a document is called _____.
6. The two widely used measures for evaluating the performance of IR system are _____ and _____.
7. IR system may also apply _____ in which all related terms are reduced to the simplest and most significant form without loss of generality.
8. In signature files, each signature is represented by a fixed size of bits called _____.
9. The retrieved documents that do not contain _____ are called false positive.
10. Web pages are classified into two types: _____ and _____.

C. Answer the Questions

1. Define the term information retrieval and its necessity. Mention the differences between database system and IR system.
2. What are performance measures for IR systems?
3. Define the following terms:
 - (a) Term frequency
 - (b) Stop words
 - (c) Cosine similarity metric
 - (d) Stemming
 - (e) Hub and authority
4. How would you represent a document using vector space model? Explain with the help of suitable example. What are its limitations?
5. How would you measure the relevance of a document to a given term t ?
6. What is similarity-based retrieval?
7. What is the role of indexing in IR system? Describe the various indexing techniques used by IR system.
8. Define search engine. How does a search engine search a page?
9. What is a web page? How are different web pages linked? Discuss link-based search algorithm.

OBJECT-BASED DATABASES

After reading this chapter, the reader will understand:

- *The need for object-based databases*
- *Two streams of object-based databases, that are object-relational and object-oriented databases*
- *Extension of SQL to support complex data types and object-oriented features, such as inheritance*
- *Defining structured data types using row and array type constructors*
- *Defining methods on the structured types*
- *The concept of type inheritance and table inheritance*
- *The concept of object identifier (OID) and reference types in SQL*
- *Characteristics of object-oriented databases including persistence and versioning*
- *The standard model of OODBMS, that is, the ODMG object model*
- *Various concepts and technologies related to object model, such as objects, literals, interface, inheritance, etc.*
- *Representation of the structure of an object-oriented database using object definition language (ODL)*
- *Querying object-oriented databases using object query language (OQL)*
- *Similarities and differences between OODBMS and ORDBMS*

Traditional database applications like airline reservation and employee management consist of data-processing tasks with relatively simpler data types such as integers, dates, and strings, which are well suited to the relational model. However, complex database applications such as Computer Aided Design (CAD), Computer Aided Software Engineering (CASE), and Geographical Information System (GIS) demand the use of complex data types to represent multivalued attributes, composite attributes, and inheritance. Such features can be easily represented in the E-R and Extended E-R model, but are difficult to translate into relational model using simpler SQL data types. Thus, the need of a new database model that allows us to deal with complex data types arose.

The new database model proposed was based on the concept of object, since objects represent complex data types naturally. The concept of object was implemented in the database system in two ways, which resulted in object-relational and object-oriented systems. These object-based database systems provide a richer type system including complex data types and object orientation. This chapter discusses both of them in detail.

16.1 NEED FOR OBJECT-BASED DATABASES

Relational databases systems suffer from certain limitations, which led to the development of object-based database systems. The main disadvantage of relational databases is that they represent entities

and relationships among them in the form of two-dimensional tables. Thus, any kind of complex data such as multivalued and composite attributes are difficult to represent in such databases. For example, consider the multivalued attributes **Phone** and **Email_id** from the E-R model of *Online Book* database. Representation of such attributes in relational model results in the decomposition of relation into several relations (because 1NF states that all the attributes must have atomic values). However, creation of a new relation for each multivalued attribute may be expensive and artificial in some cases.

Similarly, consider an example of a composite attribute **Address**. We can view entire address as an atomic data item of type string. However, answering the queries that need to access the street, city, state, and zip code individually becomes difficult. Alternatively, if we represent the address by breaking it into the components (street, city, state, and zip code), it will make the queries more complicated as the user has to mention each field individually in the query. A better option is to represent these attributes as structured data types, which allow a type **Address** with subparts **street**, **city**, **state**, and **zip_code**.

Another limitation of relational model is that concepts like inheritance are difficult to represent in it. The inheritance hierarchy needs to be represented with a series of tables. In order to ensure data consistency and integrity, referential integrity constraints must also be set up. With complex type systems, E-R model concepts such as multivalued and composite attributes, generalization and specialization can be directly represented without a complex translation to the relational model.

Relational database systems may not be suitable for certain applications, but they have so many advantages. That is why many commercial DBMS products are basically relational but also support object-oriented concepts.

Another major reason for the development of object-based databases is the increasing use of object-oriented programming languages in software development. Traditional databases seem to be difficult to use with software applications that are developed in object-oriented languages such as C++, Smalltalk, or Java. Object-oriented databases are designed so that they can be directly integrated with the applications that are developed using object-oriented programming language.

Learn More

Some of the object-based database systems that were created for experimental purposes include ORION system developed at MCC, ODE system at AT&T Bell Labs, the IRIS system developed at Hewlett-Packard laboratories, and OPE-NOODB at Texas Instruments, etc.

16.2 OBJECT RELATIONAL DATABASE SYSTEMS

Object relational database systems (ORDBMS) are the relational database systems that have been extended to include the features of object-oriented paradigm. The SQL:1999 extends the SQL to support the complex data types and object-oriented features such as inheritance. In this section, we discuss implementation of these features in SQL.

16.2.1 Structured Types

In addition to built-in data types, SQL:1999 allows us to create new types which are called user-defined types in SQL. Structured type is a form of user-defined type that allows representing complex data types. Using structured types, composite attributes as well as multivalued attributes of E-R diagrams are represented efficiently.

Using Type Constructors

Structured data types are defined using type constructors, namely, *row* and *array*. Row type is used to specify the type of a composite attribute of a relation. A row type can be specified using the syntax given here.

```
CREATE TYPE <Type_Name> AS [ROW](<attribute1> <data_type1>,
                                     <attribute2> <data_type2>,
                                     :
                                     <attributen> <data_typen>
                                     );
```



NOTE The keyword *ROW* is optional.

For example, a row type to represent a composite attribute **Address** with component attributes **HouseNo**, **Street**, **City**, **State**, and **Zip** can be specified as follows.

```
CREATE TYPE AddressType AS
    (HouseNo  VARCHAR(30),
     Street   VARCHAR(15),
     City     VARCHAR(12),
     State    VARCHAR(6),
     Zip      INTEGER(6));
```

We can now use the defined type (that is, **AddressType**) as a type for an attribute while creating a relation. For example, a relation **PUBLISHER** can be created by declaring an attribute **Address** of type **AddressType** as shown here.

```
CREATE TABLE PUBLISHER(
    P_ID      VARCHAR(20),
    Pname     VARCHAR(50),
    Address   AddressType);
```

The attribute **Address** in the **PUBLISHER** relation is now a composite attribute having **HouseNo**, **Street**, **City**, **State**, and **Zip** as its components.

The row types discussed so far have names associated with them. SQL provides an alternative way of defining composite attributes by using unnamed row types. To illustrate this, consider the following declaration.

```
CREATE TABLE PUBLISHER(
    P_ID      VARCHAR(20),
    Pname     VARCHAR(50),
    Address   ROW(HouseNo  VARCHAR(30),
                  Street   VARCHAR(15),
                  City     VARCHAR(12),
                  State    VARCHAR(6),
                  Zip      INTEGER(6)));
```

Note that the attribute **Address** has unnamed type and rows of the relation also have an unnamed type.

Now in order to access the components of a composite attribute “dot” notation is used. For example, **Address.City** returns the city component of the **Address** attribute. For example, consider the following query.

```
SELECT Address.HouseNo, Address.City
FROM PUBLISHER;
```

In addition to creating an attribute of user-defined type in a relation, SQL also allows creating a relation whose rows are of a user-defined type. For example, we can define a type `PublisherType`, which can be further used to create a relation `PUBLISHER` as follows.

```
CREATE TYPE PublisherType AS(
    P_ID          VARCHAR(20),
    Pname         VARCHAR(50),
    Address       AddressType);

CREATE TABLE PUBLISHER OF PublisherType;
```

An array type is used to specify an attribute (multivalued attribute) whose value will be a collection. For example, an author can have many phone numbers, thus, the attribute `Phone` can be represented using array type in the type `AuthorType` as follows.

```
CREATE TYPE AuthorType AS(
    Aname        VARCHAR(30)    NOT NULL,
    State        VARCHAR(15),
    City         VARCHAR(15),
    Zip          VARCHAR(10),
    Phone        VARCHAR(20)    ARRAY[5],
    URL          VARCHAR(30));
```

Note that the attribute `Phone` can store at most five values. Now we can create a relation `AUTHOR` of type `AuthorType`, and can insert a row in it using the following statements.

```
CREATE TABLE AUTHOR OF AuthorType;

INSERT INTO AUTHOR VALUES
('James Erin', 'Georgia', 'Atlanta', '31896',
ARRAY['376045', '376123'], 'www.ejames.com');
```

In order to access or update array elements, we use array index. For example, the following query retrieves the name and first phone number of the authors who live in *Georgia*.

```
SELECT Aname, Phone[1]
FROM AUTHOR WHERE State = 'Georgia';
```

In addition to row and array type, other type constructors are set, multiset (bag), and list, but these are not the part of SQL:1999. The types defined using array, set, multiset, and list are referred to as **collection types**.

Defining Methods We can also define methods on the structured types. The methods are declared as part of the type definition as shown here.

```
CREATE TYPE PublisherType AS(
    P_ID          VARCHAR(20),
    Pname         VARCHAR(50),
    Address       VARCHAR(25))
METHOD ShowName (P_ID
    VARCHAR(20))
RETURNS    VARCHAR(50));
```

Learn More

SQL:1999 allows defining constructor functions for the structured types in order to create values for their attributes. The name of constructor function is same as that of structured type and it may or may not have arguments. The default constructor for a structured type has no arguments and it assigns default values to the attributes.

Here a method `ShowName` is declared that takes `P_ID` as parameter and it is supposed to return the name of the publisher. The method body is created separately as shown here.

```
CREATE INSTANCE METHOD ShowName(P_ID VARCHAR(20))
RETURNS VARCHAR(50)
FOR PublisherType
BEGIN
    RETURN SELF.Pname;
END
```

In this declaration, the `FOR` clause indicates that the method is declared for the type `PublisherType`, and the keyword `INSTANCE` indicates that the method executes on an instance of `PublisherType`. Note that the method returns `Pname` by using the variable `SELF`, which refers to the current instance of `PUBLISHER` on which the method is invoked. For example, we can invoke the method `ShowName` as follows:

```
SELECT ShowName('P002')
FROM PUBLISHER;
```



NOTE *Method body can include procedural statements as well as it can update the values of the attributes of the instance on which it is executed.*

16.2.2 Inheritance

Unlike relational system, object-relational database system supports inheritance directly. In object-relational database system, inheritance can be used in two ways: for reusing and refining types (type inheritance) and for creating hierarchies of collection of similar objects (table inheritance).



NOTE *SQL:1999 and SQL:2003 supports only single inheritance. Though it may be possible that future versions of SQL support multiple inheritance also.*

Type Inheritance

Consider the following type definition for `BookType`.

```
CREATE TYPE BookType (
    Book_title      VARCHAR(50),
    Category        VARCHAR(20),
    Price           NUMERIC(6,2),
    Copyright_date   NUMERIC(4),
    Year            NUMERIC(4),
    Page_count      NUMERIC(4),
    P_ID            VARCHAR(4)) NOT FINAL;
```

Further, suppose that we want to store additional information in the database about the books that are textbooks. Instead of creating separate type for textbook, we can inherit `BookType` to define type `TextBookType` as shown here.

```
CREATE TYPE TextBookType UNDER BookType (Subject VARCHAR(30));
```

This statement creates a new type `TextBookType`, which inherits the attributes of `BookType`, plus it has one additional attribute `Subject`. Here, `BookType` is a supertype of `TextBookType`, and `TextBookType` is a subtype of `BookType`.

Note that the keyword `NOT FINAL` states that the subtypes can be created from the given type. In addition, SQL also provides `FINAL` keyword, which states that subtypes cannot be created from the given type.



Subtypes also inherit the methods of supertypes, in addition to their attributes. However, a subtype can override the method of supertypes just by using `OVERRIDING METHOD` instead of `METHOD` in the method declaration.

Table Inheritance

Like type inheritance, SQL also supports table inheritance, and allows us to create subtables. The concept of subtable came into existence to incorporate the implementation of generalization/specialization of an E-R diagram. To understand the table inheritance, consider the following definition of `BOOK` relation.

```
CREATE TABLE BOOK OF BookType;
```

Now we can define `TextBook` as a subtable of `BOOK` using the following statement.

```
CREATE TABLE TextBook OF TextBookType UNDER BOOK;
```

Some important points regarding table inheritance are given here.

- ⇒ The major requirement for the table inheritance is that the type of subtable must be the subtype of the type of the parent table.
- ⇒ Every tuple in the subtable also implicitly exist in the supertable; however, every tuple in the supertable corresponds to at most one tuple in each of its subtable (immediate subtable).
- ⇒ A query on the parent table finds all the tuples from the parent table as well as from its subtables. However, the result of the query would include the attributes of parent table only.
- ⇒ We can restrict the query to find tuples in the parent table only by using the keyword `ONLY` as shown here.

```
SELECT Book_title FROM BOOK ONLY;
```

16.2.3 Object Identity and Reference Types in SQL

In object database system, each object is assigned an object identifier (OID), which uniquely identifies the object over its entire lifetime. Database system is responsible for ensuring the uniqueness of objects by assigning a system generated OID to the objects. The value of OID is used internally by the system for object identification and to manage inter object references. However, its value is invisible to database users. Another important property of OID is immutability, which means, the value of OID of any object should not change. In addition, each OID should be used only once. In other words, even if the object is removed from the database, its OID should not be assigned to another object.

The value of OID can be used to refer to the object from anywhere in the database. In SQL:1999, every tuple in a table—defined in terms of structured type—can be treated as an object, and therefore, can be assigned a unique OID. In such tables, an attribute of a type may be a reference (specified using keyword `REF`) to a tuple of same (or different) table. To understand the concept, consider the type `AuthorBookType` and the table `AuthorBook`, which are shown here.

```
CREATE TYPE AuthorBookType AS(
    Book_id REF(BookType) SCOPE BOOK,
    Author_id REF(AuthorType) SCOPE AUTHOR);
```

```
CREATE TABLE AuthorBook OF AuthorBookType;
```

In the type AuthorBookType, the keyword **SCOPE** specifies that the values in the attributes Book_id and Author_id are references to the rows in the **BOOK** and **AUTHOR** table, respectively. This concept of references is similar to the concept of the foreign keys. Note that the referenced table (**BOOK** and **AUTHOR**) must have an attribute that stores the identifier of the row. This attribute, known as **self-referential attribute**, is specified by using **REF IS** clause while creating the table as shown here.

```
CREATE TABLE AUTHOR OF AuthorType
REF IS A_ID SYSTEM GENERATED;
```

This statement specifies that the values of the attribute A_ID are generated automatically by the database system.

In addition to system generated identifiers, SQL also allows users to generate identifiers. In this case, we must specify the data type of self-referential attribute (using **REF USING** clause) in the type definition of referenced table as shown here.

```
CREATE TYPE BookType (
    Book_title    VARCHAR(50),
    Category      VARCHAR(20),
    Price         NUMERIC(6,2),
    Copyright_date NUMERIC(4),
    Year          NUMERIC(4),
    Page_count    NUMERIC(4),
    P_ID          VARCHAR(4))
REF USING      VARCHAR(50);
```

```
CREATE TABLE BOOK OF BookType
REF IS ISBN USER GENERATED;
```

Note that in case of user generated identifiers, user must specify the value of the self-referential attribute while inserting tuple in the table. For example, the following statement inserts a tuple in the **BOOK** table.

```
INSERT INTO BOOK (ISBN, Book_title, Category)
VALUES ('001-987-760-9', 'C++', 'Textbook');
```

Besides system generated and user generated identifier, we can also use the values of the primary key attribute of a table as identifiers by including the **REF FROM** clause in the type definition of referenced table. The table definition must include the **DERIVED** keyword to specify that the reference is derived as shown here.

```
CREATE TYPE PublisherType AS(
    P_ID          VARCHAR(20) PRIMARY KEY,
    Pname         VARCHAR(50),
    Address       AddressType)
REF FROM      (P_ID);
```

```
CREATE TABLE PUBLISHER OF PublisherType
REF IS Pub_ID DERIVED;
```

Here Pub_ID is the name of self-referential attribute whose values are derived from the primary key P_ID.

16.3 OBJECT-ORIENTED DATABASE MANAGEMENT SYSTEMS

Object-oriented database Management systems (OODBMS) provide a closer integration with an object-oriented programming language such as Python, C#, C++, Java, or Smalltalk. An object-oriented database system extends the concept of object-oriented programming language with persistence, concurrency control, data recovery, security, and other capabilities as shown in Figure 16.1.

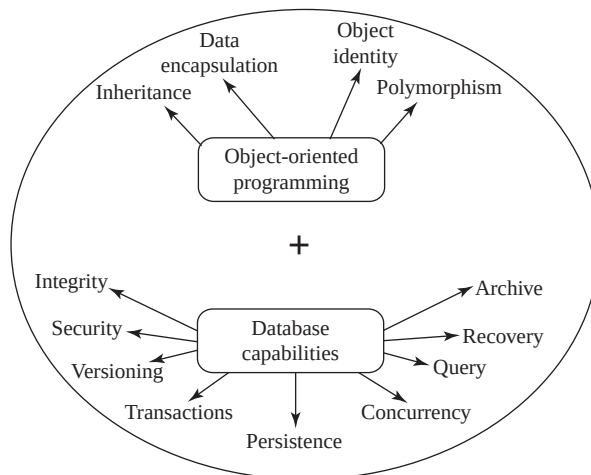


Fig. 16.1 *An object-oriented database management system*

Some object-oriented databases are well suited for available object-oriented programming languages while others have their own programming languages. OODBMSs are built upon the same data model as used by the object-oriented programming languages.

16.3.1 Characteristics of Object-Oriented Databases

Object-oriented databases combine the object-oriented programming concepts and database capabilities to provide an integrated application development system. In addition to basic object-oriented programming concepts such as encapsulation, inheritance, polymorphism, and dynamic binding, object-oriented database also supports persistence and versioning.

Persistence is one of the most important characteristics of object-oriented database systems. In an object-oriented programming language (OOPL), objects are transient in nature, that is, they exist only during program execution and disappear after the program terminates. In order to convert an OOPL into a persistent programming language (or database programming language), the objects need to be made persistent, that is, objects should persist even after the program termination. OO databases store persistent objects permanently on the secondary storage so that they can be retrieved and shared by multiple

Learn More

In C++, the objects are made persistent at the time of their creation by defining them using the overloaded **new** operator. The overloaded new operator accepts some more arguments for specifying the database in which the object should be created. On the other hand, in Java, one or more objects are explicitly declared as persistent and all other objects that can be referred through those persistent objects also become persistent.

programs. The data stored in OO database is accessed directly from the object-oriented programming language using the native type system of the language. Whenever a persistent object is created, the system returns a persistent object identifier. Persistent OID is implemented through a **persistent pointer**, which points to an object in the database and remains valid even after the termination of program.

Another important feature of OODBMS is **versioning**. Versioning allows maintaining multiple versions of an object, and OODBMS provide capabilities for dealing with all the versions of the object. This feature is especially useful for designing and engineering applications in which the older version of the object that contains tested and verified design should be retained until its new version is tested and released.

16.3.2 ODMGObject Model

The object data management group (ODMG), a subgroup of the object management group (OMG), has designed the object model for object-oriented database systems. The OMG is a pool of hundreds of object vendors whose purpose is to set standards for object technology. In 1993, the first release of the ODMG was published which was called **ODMG-93** or **ODMG 1.0** standard. Later, it was revised into ODMG 2.0, which included a common architecture and definitions for an OODBMS, definitions for an object model, an object definition language (ODL), and object query language (OQL). Various concepts and terminologies related to object model are discussed here.

- ⇒ **Objects:** An object is the most basic element of an object model and it consists of two components: state (attributes and relationships) and behavior (operations). An object in the object model is described by four characteristics, namely, *identifier*, *name*, *lifetime*, and *structure*.
 - **Identifier:** Each object is assigned a unique system generated identifier (OID) that identifies it within the database.
 - **Name:** Some objects are also assigned a unique name within a particular database that can be used to refer to that object in the program. We can use the name of an object as an entry point to the database, that is, by locating an object by its unique name we can also locate other objects that are referenced from it.
 - **Lifetime:** The lifetime of an object specifies whether the object is persistent (that is a database object) or transient (a programming language object).
 - **Structure:** The structure of an object specifies how the object is created using a certain type constructor. Various type constructors used in the object model are **atom**, **tuple** (or **row**), **set**, **list**, **bag**, and **array**. We have already discussed tuple and array type constructors in Section 16.2.1. All the basic atomic values such as integers, real numbers, Booleans, character, strings, etc. are represented using atom constructor. The list type constructor is used to represent a collection of objects in some specific order (ascending or descending), whereas, set and bag are used to represent an unordered collection of objects. Note that all the objects in a set must be distinct while a bag can have duplicate objects. The objects in the object model can be classified as atomic objects (created using atom type constructor) or collection objects (created using set, list, bag, or array type constructor).
- ⇒ **Literal:** A literal is basically a constant value that does not have an object identifier. It can be of three types, namely, *atomic*, *collection*, and *structured*. **Atomic literals** correspond to the values of basic data types of the object model including long, short, and unsigned integers, floating point numbers, Boolean values, character, string, and enumeration type values. **Collection literals** define a value that is a collection of objects or values such as set, array, list, bag, etc., but the collection itself does not have an object identifier. **Structured literals** correspond to the values that are constructed using the tuple type constructor. Structured literals include built-in structures

like `Date`, `Interval`, `Time`, and `TimeStamp`, as well as user-defined structures that can be created using the `struct` type constructor.

- ⇒ **Atomic (user-defined) objects:** A user-defined object that is not a collection object is called an **atomic object**. In order to create object types (or instances) for atomic objects, the `class` keyword is used which specifies the properties (state) and operations (behavior) for the object types. The properties define the state of an object type and are described by attributes and relationships. Attributes specify the features of an object and can be simple, enumerated, structured, or complex type, and relationships specify how one object is related to another. In the object model, only binary relationships are explicitly represented and a pair of inverse references (specified using the keyword `relationship`) is used to represent each binary relationship. In addition to properties, each object type can also have a number of operation signatures, which include name of operation, type of arguments passed, return type, and name of exceptions (optional) that can occur during operation execution.
- ⇒ **Interface:** Unlike a class that specifies both the abstract state and abstract behavior of an object type, an interface specifies only the abstract behavior of an object type. The operations that are to be inherited by the user-defined objects or other interfaces for a specific application are defined in an interface. In addition, an interface cannot be instantiated that means objects cannot be created for an interface. An interface is defined using the keyword `interface`.



NOTE *An interface may also specify properties (along with the operations) of an object type; however, these cannot be inherited from the interface.*

- ⇒ **Inheritance:** The object model employs two types of inheritance relationships. One is **behavior inheritance** or **interface inheritance** in which the supertype is an interface and the subtype is either a class or an interface. The behavior inheritance is specified using the colon (`:`) symbol. Another kind of inheritance relationship is **EXTENDS** relationship, which requires both the supertype and the subtype to be classes. This type of inheritance is specified using the keyword `extends`, and it cannot be used for implementing multiple inheritance. However, multiple inheritance is permitted for behavior inheritance that allows an interface or a class to inherit behavior from more than one interface. Note that if a class is used as a subtype in multiple inheritance then it can inherit state and behavior from at most one other class using `extends` in addition to inheriting from several interfaces via (`:`).
- ⇒ **Extents:** In ODMG object model, an extent can be declared for any object type (defined using class declaration), and it contains all the persistent objects of that class. An extent is given a name and is declared using the keyword `extent`. Whenever extents are declared, the database system automatically enforces set/subset relationship between the extents of a supertype and its subtype. For example, if two classes A and B define extents `all_A` and `all_B`, and B is subtype of class A, then the collection of objects in `all_B` must be a subset of those in `all_A`.

16.3.3 Object Definition Language (ODL)

Object definition language, a part of ODMG 2.0 standard, has been designed to represent the structure of an object-oriented database. ODL serves the same purpose as DDL (part of SQL), and is used to support various constructs specified in the ODMG object model. The database schema is defined independently of any programming language, and then the specific language bindings are used to map ODL constructs to the constructs in specific programming languages like C++, Java, or Smalltalk. The main purpose of ODL is to model object specifications (classes and interfaces) and their

characteristics. Any class in the design process has three characteristics that are attributes, relationships, and methods. The syntax for defining a class in ODL is shown here.

```
class <name>
{
    <list of properties>
};
```

Here, `class` is a keyword and the list of properties may be attributes, relationships, or methods. For example, consider the *Online Book* database and its objects `BOOK`, `AUTHOR`, `PUBLISHER`, and `CUSTOMER`. The object `BOOK` can be defined as shown here (for simplicity we have taken only three attributes).

```
class BOOK
{
    attribute string ISBN;
    attribute string Book_title;
    attribute enum Book_type
                {Novel,Textbook,Languagebook} Category;
};
```

In this example, a field `Category` as an `enum` is defined which can take either *Novel* or *Textbook* or *Languagebook* as its value. Similarly, all other objects can also be defined.



An entity in E-R data model or a tuple in relational data model corresponds to an object in object data model, and an entity set or a relation corresponds to a class.

In order to sale these books to the customers, a relationship needs to be specified between the `BOOK` object and the `CUSTOMER` object. To connect a `CUSTOMER` object with a `BOOK` object, a relationship needs to be defined in the `CUSTOMER` class as shown here.

```
relationship set <BOOK> purchases
```

Here, for each object of the class `CUSTOMER`, there is a reference to `BOOK` object and the set of these references is called `purchases`. This relationship helps us to find the books purchased by a particular customer. However, if we want to access the customers based on the book then the **inverse relationship** should be specified in the `BOOK` class as shown here.

```
relationship set <CUSTOMER> purchasedby
```

Now, this pair of inverse references (or relationships) can be connected by using the keyword `inverse` in each class declaration. For example, the relationship set between book and customer is specified in both `BOOK` and `CUSTOMER` classes as shown here.

```
class BOOK
{
    ...
    ...
    relationship set <CUSTOMER> purchasedby
                inverse CUSTOMER :: purchases;
};
```

```

class CUSTOMER
{
    attribute string Cust_id;
    attribute string Name;
    attribute struct Address //structured literal
        {short H_no,string street,string city,string state,
          string country} address;
    relationship set <BOOK> purchases
    inverse BOOK :: purchasedby;
    ...
    ...
};

```

Similarly, other relationship sets between book and author, book and publisher can also be specified. In addition to relationships, operations (or methods) can also be specified by including their signatures within the class declaration. The parameters specified in the methods must have one of the three different modes, namely, *in*, *out*, or *inout*. The *in* parameters are arguments to the method, the *out* parameters are returned from the method and a value is assigned to each *out* parameter that a user can process. The *inout* parameters combine the features of both *in* and *out*. They contain values to be passed to the method and the method can set their values as return values. For example, a method `find_cust()` that finds the name of the customer residing in a particular city, can be included in the `CUSTOMER` class definition as follows.

```

class CUSTOMER
{
    attribute string Cust_id;
    attribute string Name;
    attribute struct Address //structured literal
        {short H_no,string street,string city,string state,
          string country} address;
    relationship set <BOOK> purchases
        inverse BOOK :: purchasedby;
    void find_cust(in string city; out string cust_name)
        raises(city_not_valid);
};

```

Here, the name of the city is passed as a parameter to `find_cust()` method and the name of the customer belonging to that city is returned. In case, an empty string is passed as a parameter for city name, an exception `city_not_valid` is raised.

Suppose we want to store information about journals in our database. Since a journal is also a book, we can use inheritance to define the `Journal` class using the keyword `extends` as shown in this example.

```

class JOURNAL extends BOOK
{
    attribute string VOLUME;
    attribute string Emailauthor1;
    attribute string Emailauthor2;
};

```

Learn More

Some of the object-oriented database systems also support **selective inheritance**, which allows a subtype to inherit only some of the functions of a supertype. The functions not to be inherited by the subtype are mentioned using the **EXCEPT** clause.

To implement multiple inheritances, consider a class **BOOK_SOLD** that stores information about the books sold and the customers who have purchased those books. This class can be derived from both **BOOK** and **CUSTOMER** classes as shown here.

```
class BOOK_SOLD extends BOOK : CUSTOMER
{
    void purchase(in string ISBN;in string card_no;in short pin)
        raises (ISBN_not_valid,card_not_valid,pin_not_valid);
};
```

In `purchase()` method, the ISBN of the book to be purchased, credit/debit card number, and the pin number of the customer are passed as arguments. If the ISBN of the book, card number, or the pin number is not correct, the exception `ISBN_not_valid`, `card_not_valid`, or `pin_not_valid` is raised, respectively.

Like the relation instance of a relation schema, ODL defines an extent of a class, which contains the current set of the objects of that class. For example, an extent `books` of the class **BOOK** can be defined as shown here.

```
class BOOK (extent books)
{
    ...    //same as in
    ...    //BOOK class
};
```

Similarly, an extent `FirstCustomer` of the class **CUSTOMER** can also be defined as shown here.

```
class CUSTOMER (extent FirstCustomer)
{
    ...    //same as in
    ...    //CUSTOMER class
};
```

A class with an extent can have one or more keys associated with it that uniquely identify each object in the extent. A key for a class can be declared using the keyword `key`. For example, the **CUSTOMER** class definition can be modified to include a key as shown here.

```
class CUSTOMER (extent FirstCustomer key Cust_id)
{
    ...    //same as in
    ...    //CUSTOMER class
};
```

Here, `Cust_id` has been defined as a key for the **CUSTOMER** class, thus, no two objects in the extent `FirstCustomer` can have the same value for `Cust_id`.

In ODL, various schema constructs like class, interface, relationships, and inheritance can be represented graphically. The graphical notations for representing ODL schema constructs are shown in Table 16.1.

Using these notations, a graphical object database schema for *Online Book* database can be represented as shown in Figure 16.2.

Table 16.1 Graphical notations for representing ODL schema constructs

Schema Construct	Graphical Notation
Interface	
Class	
Relationships	 1:1  1:N  M:N
Inheritance	 Interface inheritance via ":"  Class inheritance via "extends"

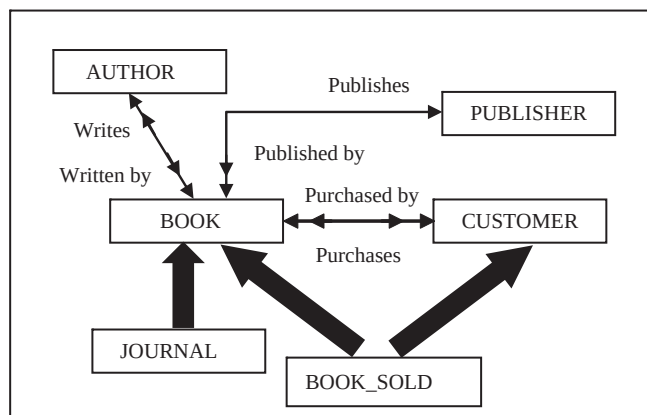


Fig. 16.2 Graphical object database schema for *Online Book* database

16.3.4 Object Query Language (OQL)

Object Query Language (OQL) is a standard query language designed for the ODMG object model. It resembles SQL (used for relational databases) but it also supports object-oriented concepts of the object model, such as object identity, inheritance, relationship sets, operations, etc. An OQL can query object databases either interactively (that is by writing ad hoc queries) or OQL queries can be embedded in object-oriented programming languages like C++, Java, Smalltalk, etc. An embedded OQL query results in objects that match with the type system of that programming language.

Each query in OQL needs an **entry point** to the database for processing. Generally, the name of an extent of a class is used as an entry point; however, any named persistent object (either an atomic object or a collection object) can also be used as a database entry point. Using an extent name as an entry point in a query returns a reference to a persistent collection of objects. In OQL, such a collection is referred to by an **iterator variable** (similar to tuple variable in SQL) that ranges over each object in the collection.

The basic OQL syntax is “SELECT-FROM-WHERE” as for SQL. For example, the query to retrieve the titles of all textbooks can be specified as

```
SELECT b.Book_title
FROM books b
WHERE b.Category = 'Textbook';
```

Here, `books` is an extent of class `BOOK` and `b` is an iterator variable. The query retrieves only the persistent objects that satisfy the condition and the collection of these objects is referred to by `b`. For each textbook object in `b`, the value of `Book_title` is retrieved and displayed. As the type of `Book_title` is a string, the result of the query is a *set* of strings.



NOTE An iterator variable can also be specified using other syntax such as “*b in books*” or “*books as b*”.

In order to access the related attributes and objects of the object under collection, we can specify a **path expression**. It starts at the persistent object name or at the iterator variable, then it is followed by zero or more relationship set names or attribute names connected using the dot notation. To understand the path expression, consider a query that displays the titles of all the books purchased by *Allen*, which is shown here.

```
SELECT b.Book_title
FROM books b
WHERE b.purchasedby.Name = 'Allen';
```

Here, the query retrieves a collection of all the books that are purchased by the customer *Allen* using the path expression `b.purchasedby.Name`. If *Allen* has purchased more than one copy of the same book the query will result in a *bag* of strings. With the use of `DISTINCT` clause the same query will return a *set* of strings as shown here.

```
SELECT DISTINCT b.Book_title
FROM books b
WHERE b.purchasedby.Name = 'Allen';
```

If we want to retrieve a *list* of books purchased by *Allen*, we can use `ORDER BY` clause. For example, the following query results in a *list* of books purchased by *Allen* in the ascending order by `Category` and in the descending order by `Price`.

```
SELECT DISTINCT b.Book_title
FROM books b
WHERE b.purchasedby.Name = 'Allen'
ORDER BY b.Category ASC, b.Price DESC;
```

If the type of result returned from a query is a structure then the keyword `struct` need to be used within the query. For example, a query to retrieve the location of the publisher who publishes the book having ISBN either *003-456-533-8* or *001-354-921-1* in ascending order of name can be written as

```
SELECT struct(name:p.Pname, loc:p.Address, s:p.State, ph:p.Phone)
FROM publishers p
WHERE p.publishes.ISBN='003-456-533-8'
      OR p.publishes.ISBN='001-354-921-1'
ORDER BY name;
```

Here, the type of result returned by the query is a structure that is composed of name, address, state, and phone number of the publisher. Note that `publishers` is an extent of `PUBLISHER` class and the expressions `name:p.Pname`, `loc:p.Address`, `s:p.State`, `ph:p.Phone` represent aliasing of `Pname` as `name`, `Address` as `loc`, `State` as `s`, and `Phone` as `ph`, respectively.

In addition, a number of aggregate operators like `SUM`, `AVG`, `COUNT`, `MAX`, and `MIN` can also be used in OQL, since most of the OQL queries result in a collection. Out of these operators, the `COUNT` operator returns an integer value, while other operators return the same type as the type of the operand collection. The following query returns the average price of books purchased by *Allen*.

```
AVG (SELECT b.Price
      FROM books b
      WHERE b.purchasedby.Name = 'Allen');
```

If we want a query to return only a single element from a singleton collection (that contains only one element), the keyword `ELEMENT` can be used. For example, the following query retrieves the single object reference to the books having price greater than \$34 and less than \$40.

```
ELEMENT(SELECT b
          FROM books b
          WHERE b.Price > 34 AND b.Price < 40);
```

A host language variable can also be used to hold the result of an OQL query. For example, consider the following statement in which a `set<BOOK>` type variable `Costly_Books` stores the *list* of books having price greater than \$30.

```
Costly_Books = SELECT b.Book_title
              FROM books b
              WHERE b.Price > 30
              ORDER BY b.Price DESC;
```

16.4 OODBMS VERSUS ORDBMS

In the earlier sections, we have discussed the important features of both OODBMS and ORDBMS. It is clear from our discussion that the two kinds of object databases are similar in terms of their functionalities. Both the databases support structured types, object identity and reference types, and

inheritance. In addition, both support a query language for accessing and manipulating complex data types, and common DBMS functionality such as concurrency control and recovery. These two databases have certain differences also which are listed in Table 16.2.

Table 16.2 *Differences between OODBMS and ORDBMS*

OODBMS	ORDBMS
It is created on the basis of persistent programming paradigm.	It is built by creating object-oriented extensions of a relational database system.
It supports ODL and OQL for defining and manipulating complex data types.	It supports an extended form of SQL.
It aims to achieve seamless integration with object-oriented programming languages such as C++, Java, or Smalltalk.	Such an integration is not required as SQL:1999 allows us to embed SQL commands in a host language.
Query optimization is difficult to achieve in these databases.	The relational model has a very strong foundation for query optimization, which helps in reducing the time taken to execute a query.
The query facilities of OQL are not supported efficiently in most OODBMS.	The query facilities are the main focus of ORDBMS. The querying in these databases is as simple as in relational database system, even for complex data types and multimedia data.
It is based on object-oriented programming languages; any error of data type made by programmer may affect many users.	It provides good protection against programming errors.

● SUMMARY ●

1. The concept of object was implemented in the database system in two ways, which resulted in object relational and object database systems.
2. SQL:1999 extends the SQL to support the complex data types and object-oriented features.
3. Structured type is a form of user-defined type that allows representing complex data types.
4. Structured data types are defined using type constructors, namely, row and array. Row type is used to specify the type of a composite attribute of a relation.
5. Methods can also be defined on the structured types. The methods are declared as part of the type definition.
6. In object-relational database system, inheritance can be used in two ways: for reusing and refining types (type inheritance) and for creating hierarchies of collection of similar objects (table inheritance).
7. A subtype can override the method of supertypes just by using **OVERRIDING METHOD** instead of **METHOD** in the method declaration.
8. The concept of subtable came into existence to incorporate the implementation of generalization/specialization of an E-R diagram.
9. The value of OID can be used to refer to the object from anywhere in the database.

10. The referenced table must have an attribute that stores the identifier of the row. This attribute, known as self-referential attribute, is specified by using `REF IS` clause.
11. An object-oriented database system extends the concept of object-oriented programming language with persistence, concurrency control, data recovery, security, and other capabilities.
12. Persistence is one of the most important characteristics of object-oriented database systems in which objects persist even after the program termination.
13. OID is implemented through a persistent pointer, which points to an object in the database and remains valid even after the termination of program.
14. Versioning allows maintaining multiple versions of an object, and OODBMS provide capabilities for dealing with all the versions of the object.
15. The object data management group (ODMG), a subgroup of the object management group (OMG), has designed the object model for object-oriented database systems.
16. An object is the most basic element of an object model and it consists of two components: state (attributes and relationships) and behavior (operations).
17. Each object is assigned a unique system generated identifier OID that identifies it within the database.
18. The lifetime of an object specifies whether the object is persistent (that is a database object) or transient (a programming language object).
19. The structure of an object specifies how the object is created using a certain type constructor. Various type constructors used in the object model are atom, tuple (or row), set, list, bag, and array.
20. A literal is basically a constant value that does not have an object identifier.
21. In ODMG object model, an extent can be declared for any object type (defined using class declaration), and it contains all the persistent objects of that class. An extent is given a name and is declared using the keyword `extent`.
22. Object definition language, a part of ODMG 2.0 standard, has been designed to represent the structure of an object-oriented database. The main purpose of ODL is to model object specifications (classes and interfaces) and their characteristics.
23. OQL is a standard query language designed for the ODMG object model. It resembles SQL (used for relational databases) but it also supports object-oriented concepts of the object model, such as object identity, inheritance, relationship sets, operations, etc.

● KEY TERMS ●

- | | |
|---|--|
| <ul style="list-style-type: none"> ● Object relational database systems ● Object-oriented databases systems ● Structured types ● Inheritance ● Type inheritance ● Table inheritance ● Object identifier (OID) ● Self-referential attribute ● Persistence ● Persistent pointer | <ul style="list-style-type: none"> ● Versioning ● ODMG object model ● Object data management group (ODMG) ● Object management group (OMG) ● Objects ● Literal ● Atomic literal ● Collection literal ● Structured literal ● Atomic (user-defined) objects |
|---|--|

- Interface
- Inheritance
- Behavior inheritance or Interface inheritance
- Extents
- Object definition language (ODL)
- Inversere lationship
- Object query language (OQL)
- Entrypoint
- Iteratorv ariable
- Pathe xpression

● EXERCISES ●

A. Multiple Choice Questions

1. Which of these is a complex database application?
 - (a) Computer aided design
 - (b) Computer aided software engineering
 - (c) Geographical information system
 - (d) All of these
2. Which of the following type constructors is not supported by SQL:1999?
 - (a) ~~ov~~
 - (b) Array
 - (c) ~~es~~
 - (d) ~~one~~ of ~~sethe~~
3. Which type of inheritance is supported by SQL:1999?
 - (a) ~~ingle~~-level
 - (b) ~~Multiple~~-level
 - (c) ~~on~~ ~~χ~~and (b)
 - (d) ~~one~~ of ~~sethe~~
4. Which of the following is not the type of an object in object model?
 - (a) ~~ollection~~ ~~objts~~
 - (b) ~~Strctured~~ ~~objts~~
 - (c) Atomic ~~objts~~
 - (d) All of ~~sethe~~
5. Which of these statements is true in reference to OID?
 - (a) OID is only system generated
 - (b) The value of OID can be changed
 - (c) The value of OID is used to manage inter object reference
 - (d) The value of OID is visible to database users
6. Which of these concepts is supported by an object-oriented database?
 - (a) ~~nheritance~~
 - (b) ~~ynamic~~ binding
 - (c) Versioning
 - (d) All of ~~sethe~~
7. An interface specifies _____ of an object type.
 - (a) Abstract ~~tate~~
 - (b) Abstract ~~havior~~
 - (c) ~~on~~ ~~χ~~and (b)
 - (d) ~~one~~ of ~~sethe~~
8. An object in the object model is described by four characteristics, namely, identifier, name, _____ and structure.
 - (a) Attribute
 - (b) ~~consistency~~
 - (c) ~~ifetime~~
 - (d) ~~Size~~

9. OO databases store persistent objects
 - (a) On the secondary storage
 - (b) On the primary storage
 - (c) Either (a) or (b)
 - (d) Both (a) and (b)
10. The OMG is a pool of hundreds of _____ who set standards for object technology.
 - (a) organizations
 - (b) object vendors
 - (c) IEEE members
 - (d) database users

B. Fill in the Blanks

1. A _____ is a form of user-defined type that helps to represent complex data types.
2. SQL supports table inheritance to incorporate implementation of _____ of an E-R diagram.
3. The interface inheritance is specified using the _____ symbol.
4. In OO databases, literals can be of types, namely, _____, _____, and _____.
5. The _____ clause is used to specify a self-referential attribute while creating the table.
6. In object database systems, each object is assigned an _____ that uniquely identifies the object over its entire lifetime.
7. In ODMG object model, an _____ of an object type contains all its persistent objects.
8. To specify that the subtypes cannot be created from the given type, the _____ keyword is used.
9. The objects of an object-oriented programming language are _____, whereas the objects of a database programming language are _____ in nature.
10. Object-oriented databases combine the _____ and _____ to provide an integrated application development system.

C. Answer the Questions

1. Explain how the concept of object was implemented in database system.
2. What were the limitations of relational database system which led to the development of object based systems?
3. Define the following terms:
 - (a) structured type
 - (b) Type inheritance
 - (c) Versioning
 - (d) self-referential attribute
 - (e) Object ID
 - (f) Object table
 - (g) Atomic objects
 - (h) Collection objects
4. What are structured types in SQL? Explain how they represent complex data types.
5. Explain type inheritance with suitable example. How table inheritance is used for creating hierarchies of collections of similar objects?
6. How is OID used to refer to object in the database? Explain with the help of suitable example.
7. Define object-oriented database. What are its characteristics?

8. State similarities and differences between objects and literals in ODMG object model.
9. What does ODL stand for? Describe the purpose of ODL and also describe how multiple inheritance is implemented in ODL.
10. Describe the following OQL concepts.
 - (a) Database entry points
 - (b) Iterator variables
 - (c) Path expressions
 - (d) Aggregate operators
11. What are the similarities and differences between OODBMS and ORDBMS?

XML

After reading this chapter, the reader will understand:

- *Structured, Semi-structured, and Unstructured data*
- *Roles of tags in markup languages like Hyper-Text Markup Language (HTML) and Extensible Markup Language (XML)*
- *Shortcomings of HTML that makes it unsuitable for applications that require exchanging documents*
- *The reason that why XML is a suitable language for data representation as well as data exchange*
- *The structure of XML data and the two main constructs used by XML document, that is, elements and attributes*
- *When an XML document is considered well-formed and valid*
- *Document type definition (DTD) and its limitation*
- *XML Schema and the notion of XML namespace*
- *Querying XML data using XPath and XQuery*
- *The For, Let, Where, Order by & Return (FLWOR) expression*
- *Approaches to store XML data*
- *Uses of XML*

Databases often need to be accessed from the Internet. Most of the major web sites, including the sites for electronic commerce (e-commerce) and other Internet applications, store a large part of their data in the databases and rely on DBMSs to provide fast and reliable responses to user requests. When databases are used in this context, they are often termed as **data sources**.

The most popular and common method of specifying the contents and formatting of web pages is through the use of hypertext documents. To write these documents, various languages are available; **Hyper-Text Markup Language (HTML)** is one of them. Although HTML is suitable and widely used for formatting and structuring contents of web pages, it is not suitable for representing structured data that is extracted from databases. Therefore, a new language, that is, **Extensible Markup Language (XML)** is developed, which can represent database data and many other kinds of structured data. XML not only provides the information about how to structure data in a web page but also the meaning of certain components of data. It plays a major role in structuring data of applications that require exchanging data over Web.

In this chapter, we first discuss the difference between the structured, semi-structured, and unstructured data. Then, we introduce XML, present XML data model, and discuss both the structure of XML documents and the languages for specifying the structure of these documents. Finally, we discuss approaches for storing XML documents and applications.

17.1 STRUCTURED, SEMI-STRUCTURED, AND UNSTRUCTURED DATA

A relation in a relational database, such as the **BOOK** relation of *Online Book* database, contains several records, and each record has a format consistent with other records in that relation. Such type of data that is represented in a strict format is called **structured data**. A well designed database has certain structures and constraints specified in the database schema, and the DBMS ensures that all the data in the database is in accordance with those structures and constraints.

In some applications, data is collected in an ad hoc manner before having information regarding how it will be stored and managed. Although the collected data may have certain structure, but not all the data have an identical structure. Different entities may have different set of attributes that means there is no pre-defined schema. Such type of data is known as **semi-structured data**. In such type of data, the schema information is mixed in with the data values because there may be different attributes associated with each data object that are not known in advance. Thus, semi-structured data is also sometimes referred to as **self-describing data**. Semi-structured data is represented in a model that is often based on using tree or graph data structures. One possible representation of semi-structured data as a directed graph is shown in Figure 17.1.

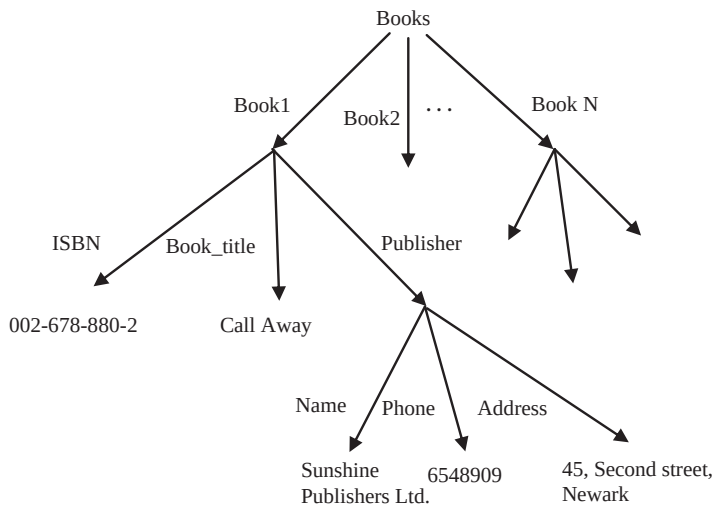


Fig. 17.1 Representing semi-structured data



NOTE The data shown in Figure 17.1 corresponds to some of the structured data shown in **BOOK** relation in Chapter 03.

A third category of data that exists is known as **unstructured data** in which there is very limited indication about the type of data. The data in web pages represented in a markup language, such as HTML, is considered as unstructured data.

HTML augments the text of document within the **tags** enclosed in angled brackets, **<>**. The tags hold a special meaning for a web browser and do not become a part of the printed document. Rather they specify the formatting information for the text to be displayed, such as font style, color, heading level, a numbered or unnumbered list, etc. Tags are used in pairs and there is a **start tag** of the form

<TAG> to indicate the beginning and an **end tag** of the form </TAG> to indicate the end of a portion of the document. HTML prescribes a large number of tags, which are used for formatting text in web documents.

```
<HTML>
<HEAD>
</HEAD>
<BODY>
<H1>Online Book Database</H1>
<H2>PUBLISHERS' information</H2>
<H3>Hills Publications</H3>
<UL>
  <LI>Address: 12, Park street, Atlanta</LI>
  <LI>State: Georgia</LI>
  <LI>Phone: 7134019</LI>
  <LI>Email: h_pub@hills.com</LI>
</UL>

<H3>Sunshine Publishers Ltd.</H3>
<UL>
  <LI>Address: 45, Second street, Newark</LI>
  <LI>State: New Jersey</LI>
  <LI>Phone: 6548909</LI>
</UL>

<H3>Bright Publications</H3>
<UL>
  <LI>Address: 123, Main street, Honolulu</LI>
  <LI>State: Hawaii</LI>
  <LI>Phone: 7678985</LI>
  <LI>Email: bright@bp.com</LI>
</UL>

<H3>Paramount Publishing House</H3>
<UL>
  <LI>Address: 789, Oak street, New York</LI>
  <LI>State: New York</LI>
  <LI>Phone: 9254834</LI>
  <LI>Email: param_house@param.com</LI>
</UL>

<H3>Wesley Publications</H3>
<UL>
  <LI>Address: 456, First street, Las Vegas</LI>
  <LI>State: Nevada</LI>
  <LI>Phone: 5683452</LI>
</UL>
</BODY>
</HTML>
```

Fig. 17.2 HTML representation of publishers' information

For example, consider a part of HTML document shown in Figure 17.2 which lists the publishers' information of *Online Book* database. Observe that a variety of tags are used to represent the document.

- ⇒ The <HTML>...</HTML> tags enclose the entire document to indicate that it is an HTML document.
- ⇒ The <HEAD>...</HEAD> tags enclose the information that specify the various commands to be used elsewhere in the document. For example, it can be used to specify the formatting styles that

can be used in the document. Note that the information enclosed in this tag does not display as part of the document.

- ⇒ The `<BODY>...</BODY>` tags enclose the body of the document, which contains the information of all the five publishers.
- ⇒ The `<H1>...</H1>` tags enclose the name of the database, that is, *Online Book*. The tags `<H1>...</H1>` specify that the enclosing text should be displayed as a level 1 heading. There are many other heading levels in HTML, which includes `<H2>`, `<H3>`, and so on. Each one specifies that the enclosing text should be displayed in a less prominent heading format.
- ⇒ The `<UL>...</UL>` tags enclose an unordered (or bulleted) list. The `<LI>...</LI>` tags enclose an item in an ordered list.

A well designed document in HTML enables users to easily understand the contents of the document, and navigate through the document. However, the source HTML document does not include the schema information about the type of data in the documents. Therefore, it becomes very difficult for a computer program to interpret the source HTML documents automatically. For example, suppose the document shown in Figure 17.2 is sent to another application. The second application can display this document well, but it cannot distinguish the house number, street name, and city name in the address of publishers. This shortcoming of HTML makes it unsuitable for the applications that require exchanging documents.

17.2 OVERVIEW OF XML

The shortcomings of HTML led to the development of XML. Unlike HTML, which defines a set of valid tags and their meanings, XML allows the set of tags to be chosen for each individual application. This feature makes XML a suitable language for data representation as well as data exchange.

Consider a part of XML document shown in Figure 17.3 which lists the publishers' information of the *Online Book* database. Observe the use of `<HNO>`, `<STREET>`, `<CITY>`, `<STATE>`, etc., as tags. These tags enable identifying the semantics of the values. When this document is sent to another application, it can easily distinguish the individual components of the address of the publisher.

Learn More

World Wide Consortium (W3C) developed XML in 1996 by combining two technologies: widely accepted HTML and powerful SGML. **Standard Generalized Markup Language (SGML)** is a meta-language, that is, a language that allows creating other languages (including their grammars and vocabularies). However, SGML is extremely complicated. Thus, one of the major design goals for XML was to keep it relatively simple and still having much of the power of SGML.

17.3 STRUCTURE OF XML DATA

An XML document uses two main constructs: elements and attributes. An **element** is a pair of a start tag and a matching end tag and all the contents enclosed between them. For example, `<PHONE>...</PHONE>` is an element used in our sample document shown in Figure 17.3. XML also allows nesting of elements. It means an element can appear completely within another element. For example, in Figure 17.3, the `HNO`, `STREET`, and `CITY` elements are nested in the `ADDRESS` element, which in turn is nested in the `PUBLISHER` element.

```

<?xml version="1.0" encoding="UTF-8"?>
<PUBLISHERLIST>
<PUBLISHER>
  <NAME>Hills Publications</NAME>
  <ADDRESS>
    <HNO>12</HNO>
    <STREET>Park street</STREET>
    <CITY>Atlanta</CITY>
  </ADDRESS>
  <STATE>Georgia</STATE>
  <PHONE>7134019</PHONE>
  <EMAIL>h_pub@hills.com</EMAIL>
</PUBLISHER>
<PUBLISHER>
  <NAME>Sunshine Publishers Ltd.</NAME>
  <ADDRESS>
    <HNO>45</HNO>
    <STREET>Second street</STREET>
    <CITY>Newark</CITY>
  </ADDRESS>
  <STATE>New Jersey</STATE>
  <PHONE>6548909</PHONE>
</PUBLISHER>
<PUBLISHER>
  <NAME>Bright Publications</NAME>
  <ADDRESS>
    <HNO>123</HNO>
    <STREET>Main street</STREET>
    <CITY>Honolulu</CITY>
  </ADDRESS>
  <STATE>Hawaii</STATE>
  <PHONE>7678985</PHONE>
  <EMAIL>bright@bp.com</EMAIL>
</PUBLISHER>
<PUBLISHER>
  <NAME>Paramount Publishing House</NAME>
  <ADDRESS>
    <HNO>789</HNO>
    <STREET>Oak street</STREET>
    <CITY>New York</CITY>
  </ADDRESS>
  <STATE>New York</STATE>
  <PHONE>9254834</PHONE>
  <EMAIL>param_house@param.com</EMAIL>
</PUBLISHER>
<PUBLISHER>
  <NAME>Wesley Publications</NAME>
  <ADDRESS>
    <HNO>456</HNO>
    <STREET>First street</STREET>
    <CITY>Las Vegas</CITY>
  </ADDRESS>
  <STATE>Nevada</STATE>
  <PHONE>5683452</PHONE>
</PUBLISHER>
</PUBLISHERLIST>

```

Fig. 17.3 XML representation of publishers' information

Elements can have attributes associated with them. An **attribute** provides information about the element and takes the form of `attributename=value` pair. This pair appears inside the start tag of an element just after the name of the element. For example, `P_ID` of publisher may appear as an attribute in the `PUBLISHER` element as shown in Figure 17.4.

```

<PUBLISHER P_ID="P001">
  <NAME>Hills Publications</NAME>
  <ADDRESS>
    <HNO>12</HNO>
    <STREET>Park street</STREET>
    <CITY>Atlanta</CITY>
  </ADDRESS>
  <STATE>Georgia</STATE>
  <PHONE>7134019</PHONE>
  <EMAIL>h_pub@hills.com</EMAIL>
</PUBLISHER>

```

Fig. 17.4 *Use of attributes*

The message in XML documents is easy to understand with the presence of user-defined tags, but sometimes we may want to include notes in an XML document. So XML allows inserting **comments** anywhere in the document (except within other tags). Comments do not become a part of the processed or displayed content, but are useful for the one who is reading the source XML document. Every comment in an XML document begins with `<!--` and ends with `-->`. Everything that appears in between the start and end comment tags are ignored by the XML processor. For example, comments can be used to mark the beginning and end of publisher's information as shown in Figure 17.5.

```

<!-- Beginning of publisher's information -->
<PUBLISHER P_ID="P001">
  <NAME>Hills Publications</NAME>
  <ADDRESS>
    <HNO>12</HNO>
    <STREET>Park street</STREET>
    <CITY>Atlanta</CITY>
  </ADDRESS>
  <STATE>Georgia</STATE>
  <PHONE>7134019</PHONE>
  <EMAIL>h_pub@hills.com</EMAIL>
</PUBLISHER>
<!-- End of publisher's information -->

```

Fig. 17.5 *Use of comments*

An XML document is considered **well-formed** if it follows a few structural guidelines. First, it must start with an XML declaration, which takes the form as the first line in Figure 17.3. This declaration specifies the version of XML for which the document is written and the character-encoding system to describe the non-ASCII characters in the document. Second it must have a single **root** element, which contains all other elements in the document. In our sample document, the **PUBLISHERLIST** element serves the purpose of the root element. Finally, all the elements must be properly nested in the XML document. That is, if an element has its start tag within another element, then it must also have its end tag within the same enclosing element.

Learn More

The angled bracket characters (that is, `<` and `>`) are reserved characters for XML. If these characters are to be included as a part of the text in an XML document, then they must be represented as `<` and `>`, respectively. The three other most commonly used XML reserved characters are ampersand (`&`), apostrophe (`'`), and single quotation mark (`'`). These characters are represented in an XML document as `&`, `'`, and `"`, respectively.

17.4 DTD AND XML SCHEMA

As already discussed, users are free to create and use their own set of elements (or tags) while representing some data in XML document. This freedom is acceptable because it makes the documents self-describing. However, sometimes it limits the possibilities of processing XML documents automatically by a computer program.

Thus, the schema definition languages, including document type definition (DTD) and XML Schema, emerged as part of XML standard. They are used to specify a structure which must be followed by the element names used in the tags.

17.4.1 DTD

A **DTD** is a set of rules that defines a set of elements and attributes that can appear within a document. In addition, it specifies a structure (or pattern) in which elements can be nested. An XML document is considered as a **valid** document if it conforms to the DTD associated with it. To understand the notations used for specifying elements, consider the sample DTD shown in Figure 17.6.

```
<!DOCTYPE PUBLISHERLIST [
  <!ELEMENT PUBLISHERLIST (PUBLISHER)*>
    <!ELEMENT PUBLISHER (NAME, ADDRESS, STATE, PHONE?, EMAIL?)>
      <!ELEMENT NAME (#PCDATA)>
      <!ELEMENT ADDRESS (HNO, STREET, CITY)>
        <!ELEMENT HNO (#PCDATA)>
        <!ELEMENT STREET (#PCDATA)>
        <!ELEMENT CITY (#PCDATA)>
      <!ELEMENT STATE (#PCDATA)>
      <!ELEMENT PHONE (#PCDATA)>
      <!ELEMENT EMAIL (#PCDATA)>
    ]>
```

Fig. 17.6 A sample DTD

Every DTD is in the form of `<!DOCTYPE root [declarations]>`, where `root` is the name of the root element in the document and `declarations` are the rules of the DTD. The DTD shown in Figure 17.6 begins with the root element in our sample document, that is, `PUBLISHERLIST`. Further, various notations are used while specifying the elements and their subelements. The first declaration is:

```
<!ELEMENT PUBLISHERLIST (PUBLISHER)*>
```

It specifies that the `PUBLISHERLIST` element contains zero or more occurrences of the `PUBLISHER` element; the `*` indicates zero or more occurrences. If we were to specify that the `PUBLISHERLIST` element must have at least one occurrence of `PUBLISHER` element, then in place of `*`, we would use `+` (which indicates one or more occurrences) as follows:

```
<!ELEMENT PUBLISHERLIST (PUBLISHER)+>
```

The next declaration specifies that the `PUBLISHER` element can contain `NAME`, `ADDRESS`, `STATE`, `PHONE`, and `EMAIL` elements as its subelements. Note that the `?` with `PHONE` and `EMAIL` elements specifies zero or one occurrence of the element. It means that the `PHONE` and `EMAIL` elements are optional for the `PUBLISHER` element.



NOTE Element that appears without `+`, `*`, or `?` in the DTD must appear exactly once in the document.

In the next declaration, the **NAME** element is declared. This declaration specifies that the **NAME** element does not contain any subelement; instead it contains textual data. It is specified by the special symbol **#PCDATA**, which stands for *parsed character data*. Like **#PCDATA**, the two other special symbols are allowed.

- ⇒ **EMPTY**: It specifies that the element is empty, that is, the element has no contents. Such elements need not have an end tag. They can simply be specified as `<ELEMENT/>`.
- ⇒ **ANY**: It specifies that there is no restriction on the subelements of the element, which means an element may contain any elements, even those that are not specified in the DTD, as its subelements.

The declaration of the **ADDRESS** element specifies that it contains **HNO**, **STREET**, and **CITY** elements as its subelements. Finally, the **HNO**, **STREET**, **CITY**, **STATE**, **PHONE**, and **EMAIL** elements are declared to contain textual data.



NOTE Our sample XML document, which represents publishers' information, conforms to the DTD shown in Figure 17.6.

Although none of the declarations in our sample DTD declares the allowable attributes for the elements, they are also declared in the DTD. The general structure for the declaration of attribute(s) of an element is

```
<!ATTLIST ele_name (att_name att_type default)+>
```

The beginning of an attribute declaration in DTD is indicated by the keyword **ATTLIST**. Following this keyword, the name of the element appears with which the attribute(s) are associated; here the word `ele_name` is used for this purpose. After the name of the element comes the declaration of the attribute(s). The strings `att_name` and `att_type` are the attribute name and type, respectively. Several allowable types for an attribute defined by XML are as follows:

- ⇒ **CDATA**: An attribute of type **CDATA** can contain character data.
- ⇒ **ID**: An attribute of type **ID** is used to provide the element with a unique identifier. It means a value that appears in an **ID** attribute of an element cannot occur in any attribute of any other element in the same document. Further, an element can have at most one attribute of type **ID**.
- ⇒ **IDREF** and **IDREFS**: An attribute of type **IDREF** contains a value that appears in the **ID** attribute of some element in the same document. It means **IDREF** attribute is a reference to some element. The type **IDREFS** is almost similar to type **IDREF** except that the former allows a list of references.

In an attribute declaration, the attribute name and type are followed by the default specification. The default specification can consist of a default value, or **#REQUIRED**, or **#IMPLIED**. If the default value is specified for an attribute of an element, then whenever this element appears without any value for the attribute, the attribute automatically takes the default value. The default specification **#REQUIRED** states that a value for the attribute must be specified whenever the associated element appears. The default specification **#IMPLIED** states that no default value has been specified for the attribute and every appearance of associated element in the document is not bound to provide a value for the attribute.

Consider the following declaration as an example of an attribute declaration.

```
<!ATTLIST PUBLISHER P_ID ID #REQUIRED>
```

This declaration specifies that **PUBLISHER** element has an attribute **P_ID** of type **ID** whose value must be specified whenever the **PUBLISHER** element appears in the document.

Despite the widespread use of DTD, it has several limitations which are as follows:

- ⇒ The allowable types in XML cannot be further typed, which means we cannot constrain an element or attribute to have values in some given range.
- ⇒ It is difficult to specify the elements as an unordered set.
- ⇒ DTD mechanism follows its own syntax, which is different from the XML syntax. This creates the need of specialized processors for processing XML schema description and the XML document.

Due to these limitations of DTD, a replacement of DTD, that is, XML Schema language is developed.

17.4.2 XML Schema

XML Schema defines the structure of XML document by using the same syntax as the XML documents, thereby, allowing the use of same processors for both XML Schema document and the XML document. XML Schema defines various built-in types including **string**, **integer**, **decimal**, **boolean**, **date**, etc., and also allows using the user-defined types. A sample XML Schema document is shown in Figure 17.7.

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="PUBLISHERLIST" type="PublisherType"/>
  <xs:element name="PUBLISHER">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="NAME" type="xs:string"/>
        <xs:element name="ADDRESS">
          <xs:complexType>
            <xs:sequence>
              <xs:element name="HNO" type="xs:string"/>
              <xs:element name="STREET" type="xs:string"/>
              <xs:element name="CITY" type="xs:string"/>
            </xs:sequence>
          </xs:complexType>
        </xs:element>
        <xs:element name="STATE" type="xs:string"/>
        <xs:element name="PHONE" type="xs:string"/>
        <xs:element name="EMAIL" type="xs:string"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
  <xs:complexType name="PublisherType">
    <xs:element ref="PUBLISHER" minOccurs="0"
      maxOccurs="unbounded"/>
  </xs:complexType>
</xs:schema>
```

Fig. 17.7 A sample XML Schema document



NOTE The XML Schema document shown in Figure 17.7 corresponds to the DTD shown in Figure 17.6.

A variety of tags are used in XML Schema and it is required to specify the set of names that can be used as XML Schema tags. For this, the XML Schema uses the notion of **XML namespace**, which defines the set of names that can be used as XML Schema tags. The file <http://www.w3.org/2001/XMLSchema> defines a set of names that are commonly used as XML Schema tags. Every XML Schema tag should be prefixed with this namespace. However, using such a long identifier with every tag decreases the schema readability. Therefore, namespace standard allows assigning identifier to a variable and prefixing each XML Schema tag with that variable. Note that the second line in the sample XML Schema document shown in Figure 17.7 assigns the namespace to a variable `xs` using the attribute `xmlns`. Then every XML Schema tag used is prefixed with `xs`.

The third line in the sample XML Schema document specifies the root element. The name and type attributes of `xs:element` tag are used to specify the name and type of the element. Here, the name of the element is `PUBLISHERLIST` and the type is `PublisherType`. The type `PublisherType` is a user-defined type and is declared later in the XML Schema document itself.

Then, the `PUBLISHER` element is specified to be of a complex type. Complex types in XML Schema can be constructed using `complexType` and `sequence` constructors. The `sequence` constructor is used to specify that the `PUBLISHER` element consists of a sequence of `NAME`, `ADDRESS`, `PHONE`, and `EMAIL` elements. The `NAME` element is specified to be of type `string`, which is a built-in XML Schema type, thus, is prefixed with `xs`. Further, the `ADDRESS` element is specified to be of a complex type. Again, the `sequence` constructor is used to specify that the `ADDRESS` element consists of a sequence of `HNO`, `STREET`, and `CITY` elements. All of these four subelements are specified to be of type `string`. Similarly, the `STATE`, `PHONE`, and `EMAIL` elements are specified to be of type `string`.

After specifying the type for elements, the user-defined type `PublisherType` is defined to consist of `PUBLISHER` element. Note that the `ref` attribute of `xs:element` tag is used to refer the `PUBLISHER` element defined earlier in the XML Schema document. In addition, the `minOccurs` and `maxOccurs` attributes of `xs:element` tag are used to specify the subelement's minimum and maximum number of occurrences.

Our sample XML Schema document covers only a subset of complete set of the features of XML Schema. Other useful features of XML Schema are as follows:

- ⇒ **Attribute declaration:** To specify the attributes of elements in XML Schema, the `xs:attribute` tag is used. The attribute declarations must include a `name` attribute to specify the attribute's name. In addition, it can specify the attribute's type (such as `string`, `decimal`, etc.) using the `type` attribute, its default value using the `default` attribute, and whether the attribute value must appear whenever the associated element appears using the `use` attribute. For example, consider the following attribute declaration added to the declaration of the `PUBLISHER` element.

```
<xs:attribute name = "P_ID" type = "xs:string"
use = "required"/>
```

This declaration specifies an attribute `P_ID` whose type is `string` and is required whenever the `PUBLISHER` element appears. Note that each element that has associated attribute(s) must be specified to be of a complex type, and the attribute(s) must be enclosed within the `complexType` specification.

- ⇒ **Constraints specification:** The unique and primary key constraints of relational database can also be specified in XML Schema. Elements corresponding to the unique attributes are specified using the `xs:unique` tag and elements corresponding to the primary keys are specified using the `xs:key` tag. In addition, the `xs:selector` and `xs:field` tags must be specified. The `xs:selector` tag defines the scope within which the constraint applies and the `xs:field`

tag specifies the elements or attributes that are either unique or form a key. For example, consider the following declaration.

```
<xs:key name="P_ID_KEY">
  <xs:selector xpath="/
    PUBLISHERLIST/PUBLISHER">
  <xs:field xpath="P_ID">
</xs:key>
```

Similarly, the foreign key constraint can be specified using `xs:keyref` tag. The `refer` attribute of `xs:keyref` tag is used to reference the primary key.

Learn More

Another XML Schema definition language is **REgular LAnguage for XML Next Generation (RELAX NG)**, which is designed by OASIS. RELAX NG is a simple, easy-to-use schema definition language and addresses some problems of XML Schema. Originally RELAX NG schemas were written in XML; however, RELAX NG also provides a compact, non-XML syntax.

17.5 QUERYING XML DATA

We need to query XML data to retrieve information of our interest from an XML document, as we need to do with a relation in a relational database. Several languages have been developed to query XML data including XPath and XQuery. These languages not only provide the capabilities of querying XML data but also translating it between different schemas of XML documents.

17.5.1 XPath

The **XPath** language queries XML documents on the basis of path expressions and returns a set of elements that match the patterns specified in the expression. A **path expression** consists of a sequence of element names (or attribute names) in the XML document with each pair of names separated by a **separator**, either a slash (/) or a double slash (//). A / specifies that the name following the slash must be nested directly within the element before the slash. On the other hand, a // specifies that the name following the double slash can be nested anywhere within the element before the double slash. For example, consider the following XPath expression, which returns the names of all the publishers in the sample XML document shown in Figure 17.3.

```
/PUBLISHERLIST/PUBLISHER/NAME
```



NOTE *Evaluation of a path expression proceeds from left to right and initially, the result set is an ordered set of all the elements in the XML document.*

The path expression begins with a /, which indicates the root of the document. Following this is the **PUBLISHERLIST** element, which is nested within the document itself. The result set at this step is an ordered set of all the elements that are nested in the **PUBLISHERLIST** element. This result set becomes the base for evaluation of the next step of the expression. Note that at each step, the result set contains the elements (or attributes) names in the same order as they are in the document itself.

Next in the expression is the **PUBLISHER** element, which must be nested directly within the **PUBLISHERLIST** element as indicated by the / before the **PUBLISHER** element. The result set now contains the elements that are nested in the **PUBLISHER** element. Finally, the evaluation reaches to the **NAME** element, which must be nested directly within the **PUBLISHER** element. The result set after the evaluation of this step reduces to contain the names of all the publishers as shown here.

```

<NAME>Hills Publications</NAME>
<NAME>Sunshine Publishers Ltd.</NAME>
<NAME>Bright Publications</NAME>
<NAME>Paramount Publishing House</NAME>
<NAME>Wesley Publications</NAME>

```

If we know only the names of tags of our interest but do not know the exact path of these tags in the XML document, then using `//` is convenient. For example, consider the following expression, which also returns the names of all the publishers in the sample XML document shown in Figure 17.3.

```
//PUBLISHER/NAME
```

XPath also allows specifying conditions based on which we want to retrieve data from XML documents. The conditions must be specified in square brackets as illustrated in the following expression.

```
//PUBLISHER[NAME="Bright Publications"]/ADDRESS
```

The expression returns the address of the publisher whose name is `Bright Publications`. This expression can also be written as follows when the full path is known to us.

```
/PUBLISHERLIST/PUBLISHER[NAME="Bright Publications"]/ADDRESS
```

17.5.2 XQuery

The XPath language allows us to specify the path expressions to retrieve data from XML documents. It becomes a basic building block for the World Wide Web Consortium (W3C) standard query language **XQuery**. A typical query in XQuery takes the form of a **FLWOR** expression. The letters of FLWOR (pronounced as flower) denote the five main clauses of XQuery language:

- ⇒ **FOR**: It binds a variable to each element returned by XPath expression.
- ⇒ **LET**: It binds a variable to a collection of elements returned by XPath expression.
- ⇒ **WHERE**: It performs some tests to determine whether an element should appear in the result or not.
- ⇒ **ORDER BY**: It allows ordering of result in some sorted order.
- ⇒ **RETURN**: It constructs the result of XQuery in XML.



A query can be constructed using some of the clauses of XQuery language. It means a query is not bound to contain all the clauses always.

Consider the following query in XQuery to retrieve the names of all the publishers in the sample XML document shown in Figure 17.3.

```

FOR $res IN
  /PUBLISHERLIST/PUBLISHER/NAME
RETURN <Result> $res </Result>

```

The **FOR** clause binds the variable `res` to each element returned by the path expression, that is, the names of all the publishers. To mark that `res` is a variable name, it is prefixed with a dollar (\$) sign. The **RETURN** clause then constructs the result of the query, which is the following XML document.

```

<Result><NAME>Hills Publications</NAME></Result>
<Result><NAME>Sunshine Publishers Ltd.</NAME></Result>
<Result><NAME>Bright Publications</NAME></Result>
<Result><NAME>Paramount Publishing House</NAME></Result>
<Result><NAME>Wesley Publications</NAME></Result>

```

Note that each name is enclosed in the `Result` tag. This is because the variable `res` get bound with every individual element and the `RETURN` clause constructs result using `res`. However, if the `FOR` clause in the query is replaced with the `LET` clause, then the variable `res` binds to the collection of elements, and the result of the query is the following XML document.

```

<Result>
  <NAME>Hills Publications</NAME>
  <NAME>Sunshine Publishers Ltd.</NAME>
  <NAME>Bright Publications</NAME>
  <NAME>Paramount Publishing House</NAME>
  <NAME>Wesley Publications</NAME>
</Result>

```

Retrieving data on the basis of some predicates requires using the `WHERE` clause in the query. Use of the `WHERE` clause is illustrated in the following query, which retrieves the address and phone of the publisher whose name is `Bright Publications`.

```

FOR $res IN
  /PUBLISHERLIST/PUBLISHER
WHERE $res/NAME="Bright Publications"
RETURN <Result> $res/ADDRESS, $res/PHONE </Result>

```

As mentioned earlier, XPath returns the result set in which elements appear in the same order as they are in the XML document itself. The XQuery allows sorting of result in some other order using the `ORDER BY` clause. The following query illustrates the use of `ORDER BY` clause and lists the names of all the publishers in the sorted order.

```

FOR $res IN
  /PUBLISHERLIST/PUBLISHER
ORDER BY $res/NAME
RETURN <Result>$res/NAME</Result>

```

17.6 APPROACHES TO STORE XML DATA

For efficient evaluation of a query, the way in which the data is stored plays an important role. There are several approaches to store XML data. One approach is to design and develop a special database system for storing XML data. Such a system must facilitate storing XML data of all types and querying it. In addition, these systems must support other database features such as transaction, security, and many more. Thus, the task of designing and implementing a new system with all these features is much more complex.

Alternatively, facilities to store XML data and query it can be implemented on the top of an existing relational database system that provides basic features of transaction, security, etc. One main issue that must be handled while using this approach is mapping of XML data to relational form. The queries on XML data (which are in either XPath or XQuery) must be converted into SQL, since relational system can handle queries in SQL. In addition, the result of SQL query (which is a relation) must be converted back into XML, since the result of XML query is XML data.

To map XML data into relations, a relation is created for each XML element. The attributes of each relation are defined by the following rules.

- ⇒ Attributes of the element becomes the attributes of type `string` of the relation.
- ⇒ For each subelement of simple type, an attribute is added to the relation. The type of the attribute in the relation is the one that corresponds to the subelement's type in XML document, which is one of the XML Schema built-in types.
- ⇒ For each subelement of complex type, a relation is created using the above rules.

Note that in case, an element has one or more subelements of complex type, then an attribute is added to the relation representing the element and is used as an identifier. In addition, the relation representing the subelement includes an additional attribute to store the identifier of its parent element.

To understand the mapping of XML data to relational database, consider the XML Schema shown in Figure 17.7. We start by creating a relation to represent the `PUBLISHERLIST` element. This element has no associated attributes, but has the `PUBLISHER` element of complex type as its subelement. Thus, an identifier attribute is added to the relation representing the `PUBLISHERLIST` element, and a relation is created to represent the `PUBLISHER` element.

The relation representing the `PUBLISHER` element must contain an attribute to store the identifier of `PUBLISHERLIST` element. In addition, it contains attributes to represent `NAME`, `STATE`, `PHONE`, and `EMAIL` elements, since they are subelements of simple type. The `PUBLISHER` element has the `ADDRESS` element of complex type, thus, an identifier attribute is added to the relation representing the `PUBLISHER` element and another relation is created to represent the `ADDRESS` element.

The relation representing the `ADDRESS` element contains an attribute to store the identifier of `PUBLISHER` element in addition to the attributes representing `HNO`, `STREET`, and `CITY` elements. The following is the resulting relational schema.

```
PUBLISHERLIST(ID: string)
PUBLISHER(ID: string, P_ID: string, NAME: string, STATE: string
          PHONE: string, EMAIL: string)
ADDRESS(P_ID: string, HNO: string, STREET: string CITY: string)
```

Learn More

Recently, relational databases, such as Microsoft SQL Server, started supporting native storage of XML. To represent XML data, a new data type that is **xml** is introduced. In addition, they allow querying XML data using XML query languages such as XPath and XQuery. Some database systems also allow embedding XQuery queries in SQL queries.

17.7 USES OF XML

Some of the many uses of XML are as follows:

- ⇒ XML is a meta-markup language which can be used to create other markup languages. Many new Internet languages are created with XML including XHTML (the latest version of HTML), and chemical markup language (a language for representing information about chemicals, such as their molecular information, boiling and melting points, etc.).
- ⇒ XML simplifies data exchange and sharing because converting data to XML creates data that can be read by different incompatible applications.
- ⇒ XML can play a major role in publishing database data to the Web. The web designers could build a connection between the database and the web server using the web servers' built-in features so that the data could be supplied by the database and they can represent it in a web application using

a style sheet. In this way, the application is just a view of the database data on the Web and any changes in the database will be reflected in the view automatically.

- ⇒ XML makes web searching effective because it defines the type of information contained in a document. For example, a search for books of a given author could search the name of author in the tags corresponding to the author name instead of searching it in the entire document.

● SUMMARY ●

1. A relation in a relational database contains several records, and each record has a format consistent with other records in that relation. Such type of data that is represented in a strict format is called structured data.
2. In some applications, the collected data may have certain structure, but not all the data have an identical structure that means there is no pre-defined schema. Such type of data is known as semi-structured data. Semi-structured data is also sometimes referred to as self-describing data.
3. A third category of data that exists is known as unstructured data in which there is very limited indication about the type of data. The data in web pages represented in a markup language, such as HTML, is considered as unstructured data.
4. HTML prescribes a large number of tags, which are used for formatting text in web documents.
5. The source HTML document does not include the schema information about the type of data in the documents.
6. The shortcomings of HTML led to the development of XML. XML allows the set of tags to be chosen for each individual application, which makes it a suitable language for data representation as well as data exchange.
7. An XML document uses two main constructs: elements and attributes.
8. An element is a pair of a start tag and a matching end tag and all the contents enclosed between them.
9. An attribute provides information about the element and takes the form of `attributename=value` pair.
10. XML allows inserting comments anywhere in the document (except within other tags).
11. The schema definition languages, including document type definition (DTD) and XML Schema, emerged as part of XML standard. They are used to specify a structure which must be followed by the element names used in the tags.
12. A DTD is a set of rules that defines a set of elements and attributes that can appear within a document.
13. An XML document is considered as a valid document if it conforms to the DTD associated with it.
14. Due to the limitations of DTD, a replacement of DTD, that is, XML Schema language is developed.
15. XML Schema defines the structure of XML document by using the same syntax as the XML documents.
16. Several languages have been developed to query XML data including XPath and Xquery.
17. The Xpath language queries XML documents on the basis of path expressions and returns a set of elements that match the patterns specified in the expression.
18. A path expression consists of a sequence of element names (or attribute names) in the XML document with each pair of names separated by a separator, either a slash (/) or a double slash (/).
19. Xpath becomes a basic building block for the World Wide Web Consortium (W3C) standard query language Xquery. A typical query in Xquery takes the form of a FLWOR expression.
20. There are several approaches to store XML data. One approach is to design and develop a special database system for storing XML data. Alternatively, we could build facilities to store XML data and query it on the top of an existing relational database system that provides basic features of transaction, security, etc.

● KEY TERMS ●

- | | |
|--|---|
| <ul style="list-style-type: none"> ● Data sources ● Extensible Markup Language (XML) ● Structured data ● Semi-structured data ● Self-describing data ● Unstructured data ● Tags ● Start tag ● End tag ● Standard Generalized Markup Language (SGML) ● Element ● Attribute ● Comments ● Well-formed ● Root element ● Document type definition (DTD) ● Valid ● #PCDATA | <ul style="list-style-type: none"> ● EMPTY ● ANY ● CDATA ● ID ● IDREF and IDREFS ● XML Schema ● XML Namespace ● XPath ● Regular Language for XML Next Generation (RELAX NG) ● Path expression ● Separator ● XQuery ● FLWOR expression ● FOR ● LET ● WHERE ● ORDER BY ● RETURN |
|--|---|

● EXERCISES ●

A. Multiple Choice Questions

1. A relation in a relational database represents which of the following data:

(a) Structured data	(b) Unstructured data
(c) Semi-structured data	(d) All of these
2. Which of the following constructs is used by an XML document?

(a) Attribute	(b) Element
(c) Both (a) and (b)	(d) None of these
3. Which of the following is a valid comment in an XML document?

(a) <code><!-- Comment --></code>	(b) <code><-- Comment --></code>
(c) <code><!--Comment --></code>	(d) <code><!--Comment --></code>
4. Which of the following is an allowable attribute-type defined by XML?

(a) EMPTY	(b) ANY
(c) CDATA	(d) #PCDATA

5. Which of the following clauses of XQuery binds a variable to each element returned by XPath expression?

- | | |
|-----------|------------|
| (a) WHERE | (b) FOR |
| (c) LET | (d) RETURN |

B. Fill in the Blanks

1. An XML document is considered _____ document if it conforms to the DTD associated with it.
2. An attribute of type _____ is used to provide the element with a unique identifier.
3. The _____ keyword indicates the beginning of attribute declaration.
4. In a path expression, a _____ specifies that the name following it must be nested directly within the element before it.
5. _____ and _____ languages are used to query XML data.

C. Answer the Questions

1. For which purpose, tags are used in an HTML document? Discuss some tags that are used in HTML documents.
2. Compare HTML and XML. Give some disadvantages of HTML which led to the development of XML.
3. Discuss the two main constructs of XML document.
4. When an XML document is considered well-formed?
5. What is a DTD? Why XML Schema is considered better than DTD?
6. Write short notes on the followings.

(a) Path expression	(b) Elements and attributes
(c) IDREF and IDREFS	(d) XQuery
7. Give some uses of XML.

D. Practical Questions

1. Give the XML representation of the AUTHOR relation of *Online Book* database shown in Chapter03 .
2. Create a DTD that corresponds to the XML document created in previous question.
3. Write a query in XPath that retrieves the names of all the authors who live in New York from the XML document created in Question 1.
4. Consider the following DTD.

```
<!DOCTYPE STUDENTLIST [
<!ELEMENT STUDENTLIST (STUDENT)*>
  <!ELEMENT STUDENT (NAME, GENDER, DOB, STREAM?)>
    <!ELEMENT NAME (FName, MName?, LName)>
      <!ELEMENT FName (#PCDATA)>
      <!ELEMENT MName (#PCDATA)>
```

```
<!ELEMENT LName (#PCDATA)>
<!ELEMENT GENDER (#PCDATA)>
<!ELEMENT DOB (Day, Month, Year)>
  <!ELEMENT Day (#PCDATA)>
  <!ELEMENT Month (#PCDATA)>
  <!ELEMENT Year (#PCDATA)>
<!ELEMENT STREAM (#PCDATA)>
```

]>

- (a) Create an XML document with some sample data that corresponds to this DTD.
- (b) Create an XML Schema document that corresponds to this DTD.
- (c) Write a query in XQuery to list the student details sorted by names.
- (d) Write a query in XQuery to list the names of all “MALE” students who have their birthday in “FEB.”

LEADING DATABASE SYSTEMS

After reading this chapter, the reader will understand:

- *History of PostgreSQL*
- *psql, a client application that facilitate user interaction with PostgreSQL server*
- *Some features of PostgreSQL including the query-rewrite system*
- *History of Oracle*
- *Oracle's Interface*
- *Some object-relational features of Oracle*
- *INSTEAD OF trigger and Data Guard feature of Oracle*
- *History of Microsoft SQL Server*
- *SQL Server Management Studio*
- *Establishing connection with SQL Server*
- *Transact-SQL that is used for data access in SQL Server*
- *History of IBM DB2*
- *Managing all the DB2 objects visually through Control Center*
- *Some features of IBM DB2*

In this chapter, we focus on the leading database systems that implement the database features discussed so far in the book. This chapter covers the four leading database systems—PostgreSQL, Oracle, Microsoft SQL Server, and IBM DB2. Out of these, PostgreSQL is the open-source database system whereas others are the three most widely used commercial database systems. In addition to discussing some unique features of each database system, the brief history and the interfaces to interact these databases are also discussed here.

18.1 POSTGRESQL

PostgreSQL is the most advanced and widely used open-source object-relational database system. It comes with a BSD license that allows everyone to use, modify, and distribute its code and documentation for any purpose without any charge. The current version of PostgreSQL (version 8.3) is fully ACID compliant and has full support for foreign keys, joins, triggers, views, stored procedures, multiversion concurrency control, and write-ahead logging. It includes most of the SQL92 and SQL99 data types and allows users to define new data types also. In addition, it has interfaces for many programming languages like C, C++, Java, Perl, and Python, and it runs on almost all the major operating systems including Linux, UNIX, Solaris, Apple Macintosh OS X, and Windows.

PostgreSQL is highly scalable in terms of the number of concurrent users it can handle and the amount of data it can manage. It supports database of virtually any size, and a table in PostgreSQL can contain any number of rows. However, there is an upper limit on the size of a table (32TB), row (1.6TB), and field (1GB).

18.1.1 History of PostgreSQL

PostgreSQL was developed at the University of California at Berkeley (UCB) and is a descendant of Postgres, a database server developed at UCB under the Professor Michael Stonebraker. He started the development of Postgres in 1986 as a follow-up project to Ingres, another database system developed under him at UCB. Version 1 of Postgres was released in 1989, and since then it has undergone many major releases.

Until 1994, Stonebraker and his students continued working on the project and added various useful features to the subsequent versions of Postgres. Later, Illustra Information Technologies developed Postgres into a commercial product. Illustra was later purchased by Informix, which in turn was purchased by IBM in 2001.

In 1995, two students—Andrew Yu and Jolly Chen—at Berkeley added SQL capabilities to Postgres and gave it a new name, Postgres95. The Postgres95 was released to the Web in 1996 and left the development at Berkeley. The system began a new life in the open source world, and a global development team (consisting of a group of dedicated developers from around the world) was formed. These developers devoted themselves to keep the development of the system continued.

Initially, the developers of global development team did not understand the code of system inherited from the Berkeley. Thus, they aimed at fixing bugs in the system reported by its users, instead of adding new capabilities. However, fixing some bugs in the system required extensive research and several reports were duplicated. So, they first cut down the duplicates and aimed to fix the simple bugs. A new version of system was released every three to five months, where each version fixed some problems existing in the previous version. In late 1996, the system was renamed to PostgreSQL to reflect the relationship between the Berkeley's Postgres and the newer versions of Postgres with SQL capabilities.

Slowly, the developers gained the knowledge of the code, which enabled them to fix complicated bugs and add new features thereby improving the system in every area. Thus, every new release of the system not only fixed the bugs in the previous version but also added various new useful features. Today, each new version is a major improvement over the previous one.

18.1.2 PostgreSQL Interface

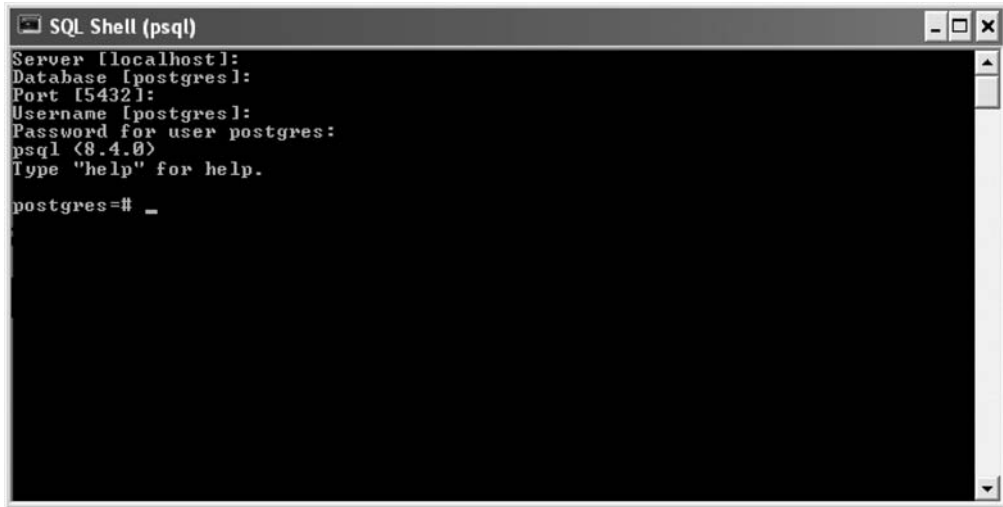
PostgreSQL employs a client/server architecture in which clients send their requests to the server, which processes the requests and returns the result back to the client. To facilitate the user interaction with the PostgreSQL server, several client applications have been developed. One such client application is `psql`, which is an interactive terminal and allows executing SQL commands.

When `psql` application is started, it asks for some information, including the name of the server machine, database name (default is `postgres`) to which to connect, the port (default is 5432) where server is receiving requests, the username (default is `postgres`), and the password. Figure 18.1 shows the `psql` client application. Note that the default information for each field is shown in `[]` (square brackets) and we can simply press the Enter key to let `psql` accept the default information.

PostgreSQL server can manage several databases, and to work with a particular database, we need to connect to it. To create a new database through `psql`, the `CREATE DATABASE` command is used. Once the database is created, we can connect to it using `\c` followed by the database name. Figure 18.2 shows the commands to connect to a database `onlinebook` and create a table `PUBLISHER` in the database.

Learn More

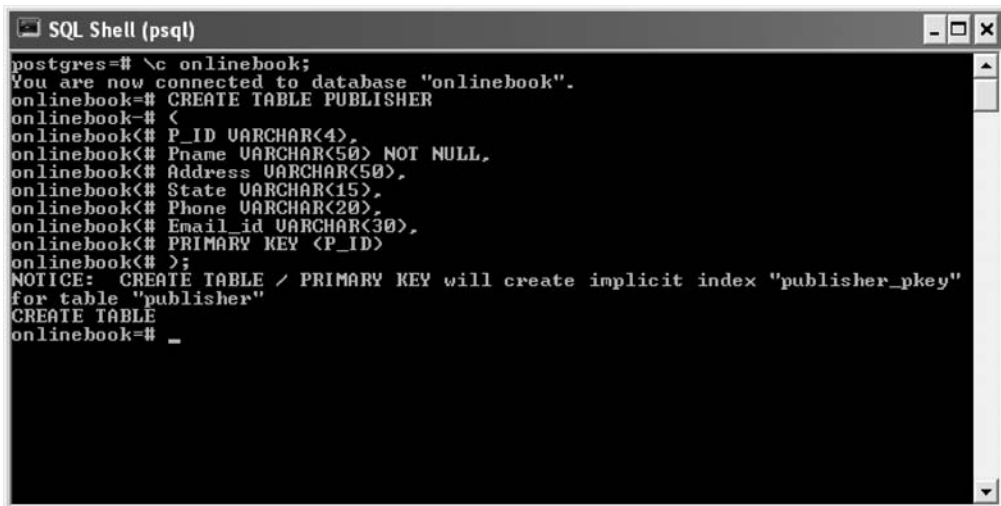
PostgreSQL standard distribution does not come with any graphical interface tool; however, several graphical tools have been developed for database administration as well as database design. `pgAdmin` and `Data Architect` are graphical tools for database administration and database design, respectively.



```
SQL Shell (psql)
Server [localhost]:
Database [postgres]:
Port [5432]:
Username [postgres]:
Password for user postgres:
psql (8.4.0)
Type "help" for help.

postgres=# _
```

Fig. 18.1 *psql client application*



```
SQL Shell (psql)
postgres=# \c onlinebook;
You are now connected to database "onlinebook".
onlinebook=# CREATE TABLE PUBLISHER
onlinebook=# <
onlinebook<# P_ID VARCHAR(4),
onlinebook<# Pname VARCHAR(50) NOT NULL,
onlinebook<# Address VARCHAR(50),
onlinebook<# State VARCHAR(15),
onlinebook<# Phone VARCHAR(20),
onlinebook<# Email_id VARCHAR(30),
onlinebook<# PRIMARY KEY (P_ID)
onlinebook<# >;
NOTICE: CREATE TABLE / PRIMARY KEY will create implicit index "publisher_pkey"
for table "publisher"
CREATE TABLE
onlinebook=# _
```

Fig.18.2 *Creating a database, connecting to it, and creating a table*

The client application psql is widely used for entering SQL commands interactively. However, it is not ideal for writing applications. Fortunately, PostgreSQL provides interfaces for many programming languages including C, C++, Java, Perl, and Python. The language interfaces allow passing queries from applications to PostgreSQL server and receiving the results back. Programming-language interfaces for some popular languages are shown in Table 18.1.



NOTE *pgAdmin is an open source GUI for database administration.*

Table18.1 *Programming language interfaces for some languages*

Programming Language Interfaces	Programming Language
LIBPQ, ECPG, LIBPGEASY	C
LIBPQ++	C++
JDBC	Java
PERL	Perl
PYTHON	Python

18.1.3 Some Features of PostgreSQL

PostgreSQL is rich in features that an organization looks for in a database system. The data types in PostgreSQL include integer, numeric, Boolean, date, varchar, etc., and it also allows defining new types as per the needs of specific applications.

To maintain data integrity, PostgreSQL allows applying SQL constraints such as unique, primary key, foreign key with restricting and cascading updates/deletes, check, and not null. It also supports triggers, which are executed automatically as a result of SQL DML statements. Note that the trigger functions cannot be written in plain SQL; rather they are written in a procedural language such as PL/pgSQL (similar to PL/SQL) or in C.

PostgreSQL supports a rule system (also called **query-rewrite system**) which allows users to define how SQL queries on a given table or view should be modified into alternate queries. The modified queries are then submitted for the execution. Using rules is advantageous in many situations. For example, suppose a new edition of an existing book in the **BOOK** relation of *Online Book* database is published. In this case, it is required to have information about the new edition in the **BOOK** relation instead of having information about the old edition. Further, it is required to keep the information of old edition in some other relation. To perform all this, we can declare a new relation, say **oldEditions**, having the same attributes as that of the **BOOK** relation and define the following rule.

```
CREATE RULE editions AS ON INSERT TO BOOK
WHERE new.Book_title = old.Book_title
DO INSERT INTO oldEditions VALUES ( old.ISBN,
                                     old.Book_title,
                                     old.Category,
                                     old.Price,
                                     old.Copyright_date,
                                     old.Year,
                                     old.Pagecount,
                                     old.P_ID
                                     );
```

Now, a query to insert the information of a new edition of book into the **BOOK** relation automatically transfers the information of old edition of the book into the **oldEditions** relation.

Rules are also used by PostgreSQL to implement views. For instance, consider the following query to define a view on **BOOK** relation of *Online Book* database.

```
CREATE VIEW bookview AS
SELECT ISBN, Book_title, Category
FROM BOOK;
```

PostgreSQL might convert this query into the following alternate queries.

```
CREATE TABLE bookview ( ISBN          VARCHAR(15),
                        Book_title    VARCHAR(50),
                        Category      VARCHAR(20)
                        );
```

```
CREATE RULE QueryView AS ON SELECT TO bookview
DO INSTEAD
SELECT ISBN, Book_title, Category
FROM BOOK;
```

This rule definition redirects all the select queries on `bookview` to the underlying base relation `BOOK`. Similarly, rules can be defined to handle update queries on the `bookview`. Some additional features of PostgreSQL are as follows:

- ⇒ **Extensibility:** Since the source code of PostgreSQL is available (without any cost) under the BSD license, staff of any organization can extend the features of PostgreSQL as well as add new features to it to meet their needs.
- ⇒ **Cross-platform:** PostgreSQL is available for almost all the major operating systems. Though earlier versions were made 'Windows compatible' via the Cygwin framework, version 8.0 and higher have native Windows support.
- ⇒ **GUI support:** PostgreSQL supports graphical user interface (GUI) along with the command-line interface. Several commercial as well as open-source GUI tools are available for database design and administration.

18.2 ORACLE

There are several DBMSs available in the market today, out of which Oracle is the most popular one. It was the first commercial relational database management system. Today, Oracle technology can be found in almost every industry. The latest release of this database is Oracle 11g.

18.2.1 History of Oracle

In 1977, Larry Ellison with his two friends—Bob Miner and Ed Oates— started software development laboratories (SDL). They realized that there was a great potential in relational database model and there were no commercial relational database products in the market. Thus, they committed themselves to build a commercial RDBMS and came out with a version of Oracle in 1979. This version supported only the basic SQL functionalities of queries and joins, and did not support the advanced concepts like transactions.

In 1982, the company was renamed to Oracle Corporation to indicate its close relationship with its product Oracle. Though the Oracle Corporation was initially a much smaller company, it did not confine itself to relational database server only. With time, it started providing query and analysis tools, application software for enterprise resource planning and customer-relationship management, and application server with close integration to the database server.

Today, with the wide range of services and products, Oracle Corporation has become the second largest independent software company in the world. It is serving over three lacs customers in more than 145 countries.

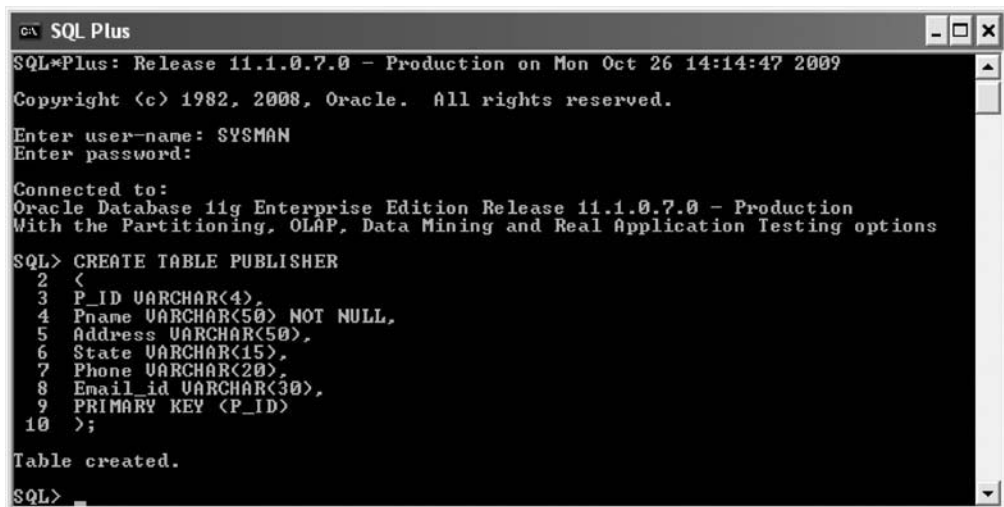
LearnMore

First version of Oracle was written in Assembly language and was never released. The version of Oracle released in 1979 was version 2.0.

18.2.2 Oracle's In terface

There are several tools available to access a database created in Oracle. The most popular and simplest tool is SQL*PLUS, which allows users to interactively query the database using SQL commands as well as executing PL/SQL programs. When SQL*PLUS is started, it asks the users to log on by providing the username and password. Note that Oracle provides some default usernames including SYS, SYSTEM, SYSMAN, and DBSNMP, and during installation, it asks for a password for these default usernames.

When the required information is provided in the dialog box, Oracle performs several checks to determine the authenticity of the user. If the username and password provided are invalid, then the user is not allowed to log on. Otherwise, the SQL plus window appears (see Figure 18.3), which is a command-line interpreter. Note that the version of Oracle along with some additional information is also displayed in the window.



```

SQL Plus
SQL*Plus: Release 11.1.0.7.0 - Production on Mon Oct 26 14:14:47 2009
Copyright (c) 1982, 2008, Oracle. All rights reserved.
Enter user-name: SYSMAN
Enter password:
Connected to:
Oracle Database 11g Enterprise Edition Release 11.1.0.7.0 - Production
With the Partitioning, OLAP, Data Mining and Real Application Testing options
SQL> CREATE TABLE PUBLISHER
2 <
3 P_ID VARCHAR(4),
4 Pname VARCHAR(50) NOT NULL,
5 Address VARCHAR(50),
6 State VARCHAR(15),
7 Phone VARCHAR(20),
8 Email_id VARCHAR(30),
9 PRIMARY KEY (P_ID)
10 >;
Table created.
SQL>

```

Fig. 18.3 SQL Plus window

18.2.3 Some Features of Oracle

Oracle supports all the core features of SQL99 in addition to object-relational features. Some object-relational features that Oracle supports are as follows:

⇒ **Defining types:** New types can be defined as objects in Oracle using the syntax shown here.

```

CREATE TYPE newtype AS OBJECT (
    Declarations of attributes and methods
);
/

```

Note that the use of slash (/) is mandatory at the end of the type definition. It lets Oracle to process the type definition. Once a type is defined, it can be used like other types of SQL. For instance, we may use newtype while defining other object-types.

⇒ **Dropping types:** The new types defined by the users can be dropped by using the syntax shown here.

```

DROP TYPE newtype;

```

Note that before dropping a specific type, we must drop all the types that are using it; otherwise, the query to drop a type fails.

- ⇒ **Defining methods:** Oracle allows declaring methods in `CREATE TYPE` statement using `MEMBER FUNCTION` or `MEMBER PROCEDURE`. These methods are then defined separately in a `CREATE TYPE BODY` statement or in a programming language like Java or C.
- ⇒ **Nested tables:** Oracle supports nested tables, which means in a relation, the value of an attribute in one tuple can be an entire table.
- ⇒ **XML as a data type:** Oracle has XML as its native data type which is used to store XML documents.

Oracle also supports some constructs which are, in syntax and functionality, specific to Oracle. One such construct is OLAP operations. We have already discussed OLAP operations, including roll-up, cube, and mining in Chapter 14.



NOTE Version 6 or higher of Oracle supports PL/SQL and Java as its procedural languages. PL/SQL was the Oracle's original procedural language used for defining stored procedures.

Oracle has extensive support for triggers. **Row level triggers** and **statement level triggers** that Oracle supports are already discussed in Chapter 05. Both types of triggers are defined to execute either before or after some DML operation on a table. Oracle also allows creating triggers for views that are not subject to a DML operation. Recall from Chapter 05 that it is not always possible for Oracle to translate an update query (a DML operation) on a view to its underlying base table. Therefore, Oracle allows users to create **INSTEAD OF** trigger on a view to specify the actions that need to be taken on the base table as a consequence of DML operation on the view. Now, whenever the DML operation for which trigger is defined is performed on the view, Oracle executes the trigger and not the DML operation.

Some additional features of Oracle are as follows:

- ⇒ It provides support for embedding structured query language (SQL) in programming languages. For example, to embed SQL in Java, it supports SQLJ.
- ⇒ It allows referencing external sources (such as flat files) in the **FROM** clause of a query. It is particularly important for ETL process in a data warehouse.
- ⇒ It allows executing queries that need data from tables residing at different sites. It has in-built capability to optimize such queries, that is, it attempts to minimize the amount of data transfer in order to evaluate the query successfully and efficiently.
- ⇒ It provides a standby database feature known as **Data Guard**. A **standby database** is a copy of the regular database maintained on a separate system. In case, a catastrophic failure occurs at the site where the regular database is maintained, the standby database is activated, which then takes over the processing. This feature minimizes the effect of failure and thus, provides high availability of the database.

18.3 MICROSOFT SQL SERVER

SQL Server is a relational model database server developed by Microsoft. It is used to generate databases that can be accessed from desktops or laptops or through handheld devices like PocketPCs and PDAs. The latest release of this software is SQL Server 2008.

18.3.1 History of Microsoft SQL Server

In 1980s, Microsoft and IBM announced a joint development agreement, which was the beginning of OS/2 operating system (a successor to MS-DOS operating system). However, soon after the declaration, IBM announced to launch a special higher-end version of OS/2 which would include an SQL RDBMS called **OS/2 Database Manager**. That announcement aroused a need of database management product for Microsoft; because otherwise no one would buy Microsoft OS/2.

At that time, Sybase had developed (but not yet shipped) a DataServer product for the systems running UNIX operating system. The pre-release version of the DataServer gained a good reputation because of its support for client/server computing and for some new innovative features including stored procedures and triggers.

Microsoft proposed a deal to Sybase that would be beneficial for both the organizations. Sybase was a small organization at that time and the credibility and royalties for its technology that Microsoft could bring was of prime importance for Sybase. Thus, both Microsoft and Sybase signed a deal in 1987 and as a result, Microsoft got exclusive rights to Sybase's DataServer product for all of its operating systems. Microsoft also signed a deal with Ashton-Tate because Ashton-Tate's dBASE product also had a good reputation in PC data products.

In 1989, Ashton-Tate/Microsoft SQL Server version 1.0 was shipped. However, as anticipated, the sales records of the product were not impressive. Therefore, Ashton-Tate started refocusing on its core dBASE desktop product, and Microsoft and Ashton-Tate ended their agreement.



NOTE *Though Sybase was not a part of the product's name, its role in the product was significant.*

In 1990, Microsoft shipped the SQL Server 1.1 as an upgrade to Ashton-Tate/Microsoft SQL Server 1.0. SQL Server 1.1 had support for a new client platform Windows 3.0 (Microsoft's new operating system). Slowly, many Windows-based applications were developed that were supporting SQL Server, and SQL Server became one of the leading database products for Windows-based application.

Initially, Microsoft depended entirely on Sybase for fixing the bugs in the product, since it has no right to access the source code. However, in 1991, Microsoft got rights to fix the bugs, and the developers at Microsoft became experts of SQL Server code. In the same year, Microsoft released SQL Server 1.11, another small upgraded version.

At the same time, developers at Microsoft were involved in the development of a new operating system named Microsoft Windows NT (NT for new technology). SQL Server would, no doubt, be moved to this new operating system. The first version of Windows NT was not expected for two years. In the meantime, engineers at Microsoft and Sybase worked together on new version of SQL Server which was shipped in 1992.

After that, Sybase started working on the new version of its product named System 10, and Microsoft focused on developing SQL Server for Windows NT. Microsoft shipped the beta version of SQL Server for Windows NT in October 1992. It was a major hit and surveys showed that most of the customers upgraded (or planned to upgrade) to SQL Server for Windows NT.

By the late 1993, both Sybase and Microsoft were successful companies in their own rights. Sybase had earned a good position in the database market. Similarly, Microsoft had also earned a

LearnMore

Microsoft decided to focus on developing a new version of Windows since Windows 3.0 was a big success but OS/2 was not up to the expectations. Microsoft and IBM declared the end of their joint development agreement.

good name in the software market. The SQL Server products of Microsoft became a direct competitor of Sybase SQL Server products for UNIX or other platforms. Both organizations decided to develop their own products and declared an end of their joint development in 1994. Since then Microsoft has been shipping the SQL Server products independently.

18.3.2 Microsoft SQL Server's Interface

SQL Server contains SQL Server Management Studio that provides a variety of functionality for managing the server using GUI. SQL Server Management Studio is the tool that allows creating, editing, and deleting database objects like tables, views, procedures, triggers, etc. When it is started, the Connect to Server dialog box (see Figure 18.4) appears.

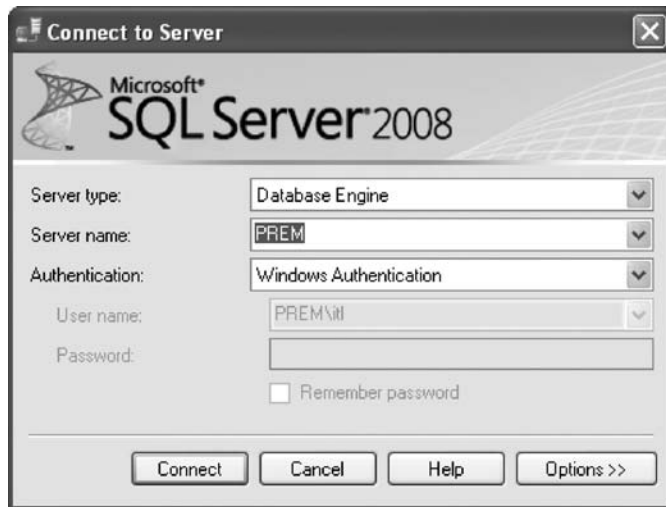


Fig.18. 4 *Connect to Server dialog box*

Here, we need to provide the following information to connect to the SQL Server.

- ⇒ **Server type:** The subsystem of SQL Server into which we have to log in.
- ⇒ **Server name:** The SQL Server into which we have to log in. Note that SQL Server allows multiple instances of SQL Server to run simultaneously and by default, the instance of the server carries the same name as the name of the machine on the network. To get connected to the SQL Server on the same computer from which we are trying to connect, we can use its name.
- ⇒ **Authentication:** The authentication type using which we want to log in. There are two options which are Windows Authentication and SQL Server Authentication. With the Windows Authentication, user name and password are not required. Instead, the user is validated through the Windows domain. On the other hand, with the SQL Server Authentication, the user name and password specific to SQL Server needs to be provided.

After providing the correct information, the interactive sessions with SQL Server are allowed. There is a **New Query** button at the top left corner of the Management Studio by clicking on which a new Query window is opened. In this window, statements are executed using the SQL Server's native language Transact-SQL (or T-SQL). T-SQL provides SELECT, INSERT, UPDATE, and DELETE

statements to manipulate data in addition to other types of commands such as creating a database or table, granting rights to users, and so on.

Figure 18.5 shows the Query window in which a T-SQL command has been executed to create the PUBLISHER relation. Note that after writing a query in the Query window, we click the **Execute** button on the toolbar in order to execute the query.

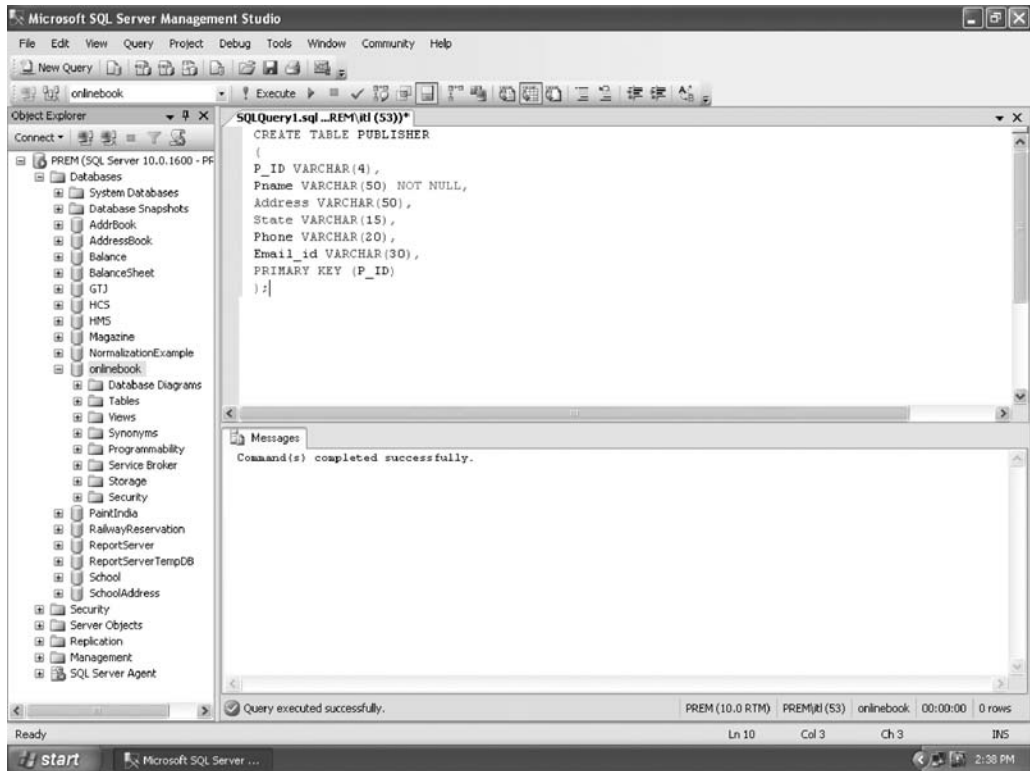


Fig.18.5 *SQL Server Management Studio*

18.3.3 Some Features of Microsoft SQL Server

T-SQL is the core language that is used in SQL Server for data access. It is a common language runtime (CLR) compliant language, which means, a .NET language. SQL Server 2008 introduces several new T-SQL programmability features and enhances some existing ones. Some of its key features are as follows:

- ⇒ It allows initializing variables as a part of the variable declaration statements, whereas in earlier versions, separate **DECLARE** and **SET** statements had to be used.
- ⇒ New compound assignment operators are introduced, which include **+=** (plus equals), **-=** (minus equals), ***=** (multiplication equals), **/=** (division equals), and **%=** (modulo equals). These assignment operators can be used wherever assignment is allowed. These operators help abbreviating the code of assignments.
- ⇒ Four new date and time data types are introduced, namely, **DATE**, **TIME**, **DATETIME2**, and **DATETIMEOFFSET**. These data types provide improved accuracy and support for larger date

and a time zone element. Some new functions that operate on these new types are also introduced and existing ones are enhanced.

- ⇒ SQL Server's new data compression feature helps reducing the size of tables, indexes or a subset of their partitions. Although space savings that can be achieved depends on the schema and data distribution, one can expect 50% to 70% reduction. SQL backups can also be compressed to significantly save space on disk media.
- ⇒ Database mirroring feature that Microsoft introduced in SQL Server 2005 is enhanced in SQL Server 2008. This enhanced feature provides database administrators a low-cost disaster recovery solution.

18.4 IBM DB2 UNIVERSAL DATABASE

DB2 Universal Database (UDB) is one of IBM's families of relational database management system (RDBMS) software products. It is strong enough to meet the needs of large organizations and flexible enough to serve the medium-sized and small business organizations. Initially, DB2 was available on IBM's mainframes only, but later it spanned to a wide variety of platforms including Windows 2000, Windows XP, variants of UNIX like Linux, Solaris, AIX, HP-UX, etc. Today, different editions and versions of DB2 are available that run on devices ranging from handhelds (such as PDAs) to mainframes. In mid-2009, IBM announced version 9.7 of this product on distributed platforms.

Like the free version of Oracle and Microsoft SQL Server, a free version of DB2 called **DB2 Express-C** is also available. This free version has no limit on the number of concurrent users and database size, but the database engine (the software component responsible for storing, processing, and retrieving data) uses only two CPU cores and supports a maximum of 2 GB of RAM.

18.4.1 History of IBM DB2

The idea of DB2 arose in 1970 when E.F. Codd of IBM described the theory of relational databases. He published a paper "A Relational Model of Data for Large Shared Data Banks", which proposed a new architecture for storing and managing the digital data. The new model would free the application developers from the burden of managing the data and allowed them to focus on business logic. Hence, new and powerful application could be developed more quickly and easily.

IBM implemented the new model and came up with the structured english query language (or SEQUEL) after four years. This language became the basis for SQL language standard. In 1983, IBM released its database product on multiple virtual storage (MVS) mainframe platform which was given the name DB2 (formally called **Database 2**). It was the first database product to use SQL.



NOTE IBM overhauled the SEQUEL and renamed as SQL to differentiate it from SEQUEL.

The development of DB2 was continued on the mainframe as well as on the distributed platforms. In 1987, the database manager in OS/2 Extended edition was the first RDBMS on distributed systems. In the next year, SQL/400 for AS/400 server was emerged. Later, new editions of DB2 supported by many other platforms including AIX, HP-UX, Solaris, Windows, and Linux were developed.

In 1996, IBM announced DB2 UDB version 5 for distributed platforms. This version of universal database supported a wide variety of hardware platforms ranging from uni-processor systems and symmetric multiprocessor (SMP) systems to massively parallel processing (MPP) systems.



Note For simplicity, the term “Universal Database” or UDB was dropped from the name in version 9 of DB2, thus, the terms DB2 UDB and DB2 are used interchangeably.

18.4.2 IBM DB2's Interface

DB2 provides Control Center, a GUI tool, to manage all the DB2 objects visually. When we open the Control Center, its main interface (similar to the one shown in Figure 18.6) appears, which allows creating database objects (such as tables, view, triggers, etc.) using simple-to-use wizards.

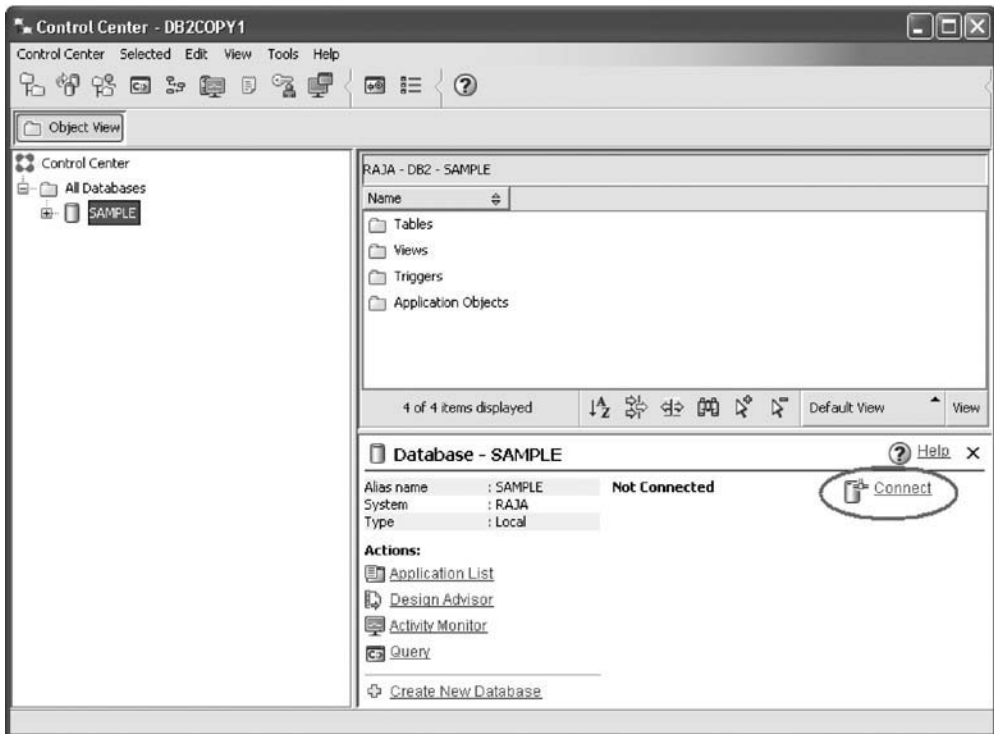


Fig.18.6 Control Center

There are two panels in the Control Center, the left panel and the right panel. The left panel displays the folders and folders' objects in a tree-like structure, and right clicking an item displays a pop-up menu listing all the actions that can be performed on the item. The right panel on the other hand displays the related objects, information, and contents of the selected item in the left panel.

Before creating database objects, we must click on **Connect** (encircled in Figure 18.6). Once we get connected, we can create tables, views, triggers, new databases, etc. For instance, to create a table, we expand the database folder in the left pane to see the **Tables** folder icon, right click on it, select **Create...**, and then follow the **Create Table Wizard**. We can create a new database by clicking on the **Create New Database** (see Figure 18.6) and following the wizards that appear. Figure 18.7 shows the Control Center after creating the *ONBOOK* database and the *PUBLISHER* table in it.

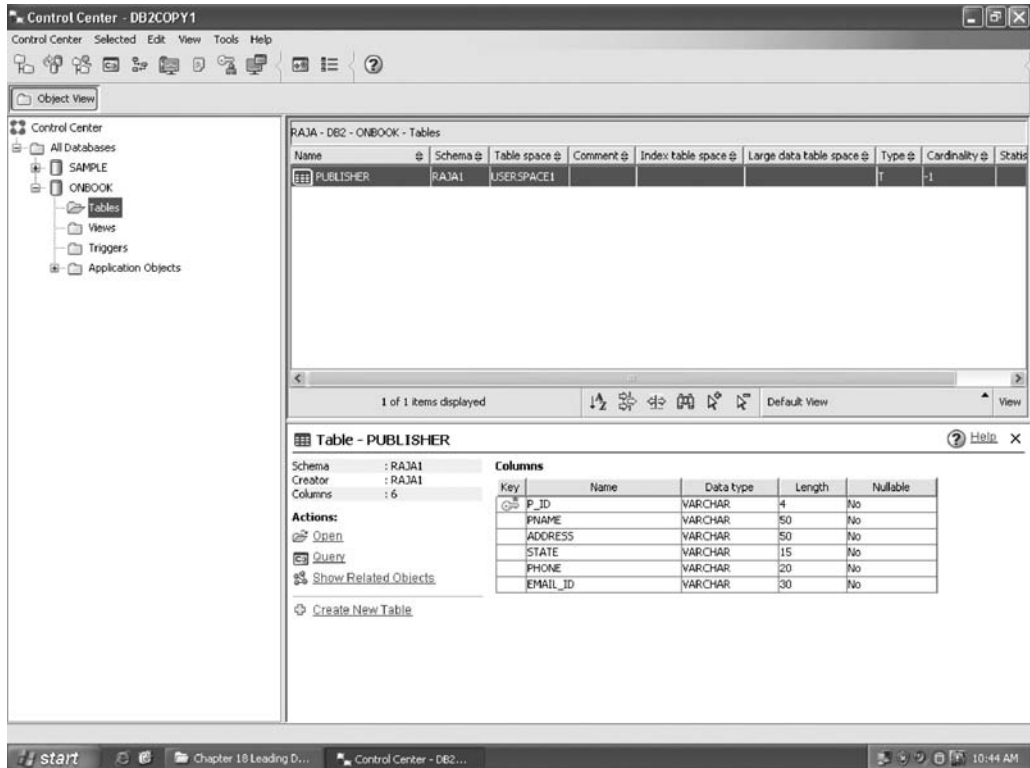


Fig.18.7 Control Center after creation of ONBOOK database and PUBLISHER table

18.4.3 Some Features of IBM DB2

DB2 is a true client/server architecture, which allows applications running on client machines to interact with the databases residing on the server machine. It comes with an expansion of SQL which supports recursive queries, active database features (such as constraints and triggers), etc. It also includes a collection of object-oriented features that allow manipulating large objects such as text, images, video, etc. Some important features of DB2 are as follows.

- ⇒ **Scalable:** DB2 runs on laptops or handheld devices to massive parallel systems with several GBs of data and many concurrent users.
- ⇒ **Business intelligence support:** DB2 supports business intelligence applications such as data warehousing (DW) and online analytical processing (OLAP).
- ⇒ **Open database:** It is an open database and runs on almost all the popular and widely used platforms. It also supports popular development platforms like J2EE and Microsoft .NET and has APIs for .NET CLI, Perl, Python, Java, C, C++, FORTRAN, and many other popular programming languages. This capability enables organizations to reduce cost by utilizing their current investment in hardware, software, and skills.

LearnMore

DB2 also provides a Command Editor which, unlike Control Center, is a command line tool and allows writing and executing DB2 commands and SQL statements interactively. In version 9.7, Command Editor is deprecated and may be removed from future versions.

- ⇒ **Data compression technology:** The data compression technology of DB2 can help reduce the storage need for data, indexes, temporary tables, and other large database objects significantly. Increased storage efficiency along with automated administration, simple deployment of virtual appliances, and improved performance reduce the cost of data management.
- ⇒ **Easy to use and manage:** DB2 can be administered by either command-line interface or GUI. Its GUI contains several wizards that enable novice users to interact with it easily.
- ⇒ **Support for XML data:** DB2 has native support for XML data and contains several XML functions including `xmlelement`, `xmlagg`, and `xmlattributes`.
- ⇒ **Support for user-defined types, functions, and methods:** DB2 supports user-defined types and allows users to define distinct or structured types. **Distinct types** are based on built-in types of DB2 but user can define additional semantics to the new types. These are defined using `CREATE DISTINCT TYPE` statement, and user can use them as a type for fields of tables. **Structured types** are complex types and can contain more than one attribute. These are defined using `CREATE TYPE` statement and can be used to define **typed tables** (a table whose rows are of user-defined type) or nested attributes inside a column of table.

DB2 also enables users to define their own functions and methods. Users can write functions in programming languages (such as C or Java) or scripts (such as Perl). These functions can generate either single attribute or tables as their result. To register the functions, the `CREATE FUNCTION` statement is used and the functions can be included in SQL queries. Like functions, users can also define methods in order to define the behavior of objects. However, methods are tightly encapsulated with particular structured type. To register the methods, the `CREATE METHOD` statement is used.

● SUMMARY ●

1. Four leading database systems are PostgreSQL, Oracle, Microsoft SQL Server, and IBM DB2.
2. PostgreSQL is the most advanced and widely used open-source object-relational database system.
3. The current version of PostgreSQL (version 8.3) is fully ACID compliant and has full support for foreign keys, joins, triggers, views, stored procedures, multiversion concurrency control, and write-ahead logging.
4. PostgreSQL is highly scalable in terms of number of concurrent users it can handle and the amount of data it can manage.
5. PostgreSQL was developed at the University of California at Berkeley (UCB) and is a descendant of Postgres, a database server developed at UCB under the Professor Michael Stonebraker.
6. To facilitate the user interaction with PostgreSQL server, several client applications have been developed. One such client application is `psql`, which is an interactive terminal and allows executing SQL commands.
7. PostgreSQL supports a rule system (also called query-rewrite system) which allows users to define how SQL queries on a given table or view should be modified into alternate queries.
8. Oracle was the first commercial relational database management system. The latest release of this software is Oracle 11g.
9. In 1977, Larry Ellison, Bob Miner, and Ed Oates started software development laboratories (SDL) and committed themselves to build a commercial RDBMS and came out with a version of Oracle two years later.
10. In 1982, the company was renamed to Oracle Corporation to indicate its close relationship with its product Oracle.

11. Today with wide range of services and products, Oracle Corporation has become the second largest independent software company in the world.
12. There are several tools available to access a database created in Oracle. The most popular and simplest tool is SQL*PLUS, which allows users to interactively query the database using SQL commands as well as executing PL/SQL programs.
13. Oracle supports all the core features of SQL99 in addition to object-relational features.
14. SQL Server is a relational model database server developed by Microsoft. The latest release of this software is SQL Server 2008.
15. In 1987, Microsoft signed a deal with Sybase to gain rights for Sybase's DataServer product for all of its operating systems. Microsoft also signed a deal with Ashton-Tate because Ashton-Tate's dBASE product also had a good reputation in PC data products.
16. In 1989, Ashton-Tate/Microsoft SQL Server version 1.0 was shipped. Later, Microsoft's agreement with Sybase and Ashton-Tate terminates and Microsoft has been shipping the SQL Server products independently.
17. SQL Server contains SQL Server Management Studio that provides a variety of functionality for managing the server using GUI.
18. Transact-SQL (T-SQL) is the SQL Server's native language and provides SELECT, INSERT, UPDATE, and DELETE statements to manipulate data in addition to other types of commands, such as creating a database or table, granting rights to users, and so on.
19. SQL Server 2008 introduces several new T-SQL programmability features and enhances some existing ones.
20. DB2 Universal Database (UDB) is one of IBM's families of RDBMS software products.
21. Initially, it was available on IBM's mainframes only, but later it spanned to a wide variety of platforms. In mid-2009, IBM announced version 9.7 of this product on distributed platforms.
22. Like free version of Oracle database and Microsoft SQL Server, a free version of DB2 called DB2 Express-C is also available.
23. The idea of DB2 arose in 1970 when E.F. Codd of IBM described the theory of relational databases. He published a paper which proposed a new architecture for storing and managing the digital data.
24. In 1983, IBM released its database product on multiple virtual storage (MVS) mainframe platform which was given the name DB2 (formally called Database 2).
25. DB2 provides Control Center, a GUI tool, to manage all the DB2 objects visually. When we open the Control Center, its main interface appears which allows creating database objects using simple-to-use wizards.
26. DB2 is a true client/server architecture, which allows applications running on client machines to interact with the database residing on the server machine.

● KEY TERMS ●

- | | |
|--|--|
| <ul style="list-style-type: none"> ● Query-rewrites ystems ● Row level triggers ● Statement level triggers ● INSTEAD OF triggers ● Data Guard | <ul style="list-style-type: none"> ● Standbyda tabase ● DB2Expre ss-C ● Database2 ● Distincttype s ● Structuredtype s |
|--|--|

● EXERCISES ●

A. Multiple Choice Questions

1. Which of the following database systems support query-rewrite system?
 (a) Oracle (b) PostgreSQL
 (c) IBM DB2 (d) Microsoft SQL Server
2. Which of the following is the programming-language interface for C++?
 (a) JDBC (b) ECPG
 (c) LIBPQ++ (d) PERL
3. Which of the following is a default username provided by Oracle 11g?
 (a) SYS (b) DBSNMP
 (c) SYSMAN (d) All of these
4. In which release of SQL Server, Microsoft introduced database mirroring?
 (a) SQL Server 2000 (b) SQL Server 2005
 (c) SQL Server 2008 (d) None of these
5. In which year, IBM announced version 9.7 of DB2 on distributed platforms?
 (a) 2006 (b) 2007
 (c) 2008 (d) 2009

B. Fill in the Blanks

1. In 1995, _____ and _____ added SQL capabilities to Postgres and gave it a new name, Postgres95.
2. The most popular and simplest tool to access database created in Oracle is _____.
3. Sybase had developed a _____ product for the systems running UNIX operating system.
4. _____ is the core language that is used in SQL Server for data access.
5. Like the free version of Oracle and Microsoft SQL Server, a free version of DB2 called _____ is also available.

C. Answer the Questions

1. What information psql application asks for when it is started?
2. Discuss query-rewrite system that PostgreSQL supports.
3. Describe some object-relational features of Oracle.
4. Write short notes on the following.
 (a) Data Guard (b) Instead of trigger
5. Explain the types of authentication that can be used to log in to SQL Server 2008.
6. List the new assignment operators introduced in SQL Server 2008.
7. Discuss the support of IBM DB2 for user-defined types, functions, and methods.
8. Create a rule in PostgreSQL for inserting the old price of book in a new relation whenever the price of a book in the BOOK relation is updated.
9. Write steps to connect to the SQL Server 2008 and then create a database and the BOOK relation (whose schema is given in Chapter 04) under the new created database.

CaseStudy-1

Hospital Management System

AIM

XYZ hospital is a multi speciality hospital that includes a number of departments, rooms, doctors, nurses, compounders, and other staff working in the hospital. Patients having different kinds of ailments come to the hospital and get checkup done from the concerned doctors. If required they are admitted in the hospital and discharged after treatment.

The aim of this case study is to design and develop a database for the hospital to maintain the records of various departments, rooms, and doctors in the hospital. It also maintains records of the regular patients, patients admitted in the hospital, the check up of patients done by the doctors, the patients that have been operated, and patients discharged from the hospital.

DESCRIPTION

In hospital, there are many departments like Orthopedic, Pathology, Emergency, Dental, Gynecology, Anesthetics, I.C.U., Blood Bank, Operation Theater, Laboratory, M.R.I., Neurology, Cardiology, Cancer Department, Corpse, etc. There is an OPD where patients come and get a card (that is, entry card of the patient) for check up from the concerned doctor. After making entry in the card, they go to the concerned doctor's room and the doctor checks up their ailments. According to the ailments, the doctor either prescribes medicine or admits the patient in the concerned department. The patient may choose either private or general room according to his/her need. But before getting admission in the hospital, the patient has to fulfill certain formalities of the hospital like room charges, etc. After the treatment is completed, the doctor discharges the patient. Before discharging from the hospital, the patient again has to complete certain formalities of the hospital like balance charges, test charges, operation charges (if any), blood charges, doctors' charges, etc.

Next we talk about the doctors of the hospital. There are two types of the doctors in the hospital, namely, *regular doctors* and *call on doctors*. Regular doctors are those doctors who come to the hospital daily. Call on doctors are those doctors who are called by the hospital if the concerned doctor is not available.

TABLES DESCRIPTION

Following are the tables along with constraints used in *Hospital Management* database.

1. **DEPARTMENT:** This table consists of details about the various departments in the hospital. The information stored in this table includes department name, department location, and facilities available in that department.

Constraint: Department name will be unique for each department.

2. **ALL_DOCTORS:** This table stores information about all the doctors working for the hospital and the departments they are associated with. Each doctor is given an identity number starting with DR or DC prefixes only.

Constraint: Identity number is unique for each doctor and the corresponding department should exist in DEPARTMENT table.

3. **DOC_REG:** This table stores details of regular doctors working in the hospital. Doctors are referred to by their doctor number. This table also stores personal details of doctors like name, qualification, address, phone number, salary, date of joining, etc.

Constraint: Doctor's number entered should contain DR only as a prefix and must exist in ALL_DOCTORS table.

4. **DOC_ON_CALL:** This table stores details of doctors called by hospital when additional doctors are required. Doctors are referred to by their doctor number. Other personal details like name, qualification, fees per call, payment due, address, phone number, etc., are also stored.

Constraint: Doctor's number entered should contain DC only as a prefix and must exist in ALL_DOCTORS table.

5. **PAT_ENTRY:** The record in this table is created when any patient arrives in the hospital for a check up. When patient arrives, a patient number is generated which acts as a primary key. Other details like name, age, sex, address, city, phone number, entry date, name of the doctor referred to, diagnosis, and department name are also stored. After storing the necessary details patient is sent to the doctor for check up.

Constraint: Patient number should begin with prefix PT. Sex should be M or F only. Doctor's name and department referred must exist.

6. **PAT_CHKUP:** This table stores the details about the patients who get treatment from the doctor referred to. Details like patient number from patient entry table, doctor number, date of check up, diagnosis, and treatment are stored. One more field status is used to indicate whether patient is admitted, referred for operation or is a regular patient to the hospital. If patient is admitted, further details are stored in PAT_ADMIT table. If patient is referred for operation, the further details are stored in PAT_OPR table and if patient is a regular patient to the hospital, the further details are stored in PAT_REG table.

Constraint: Patient number should exist in PAT_ENTRY table and it should be unique.

7. **PAT_ADMIT:** When patient is admitted, his/her related details are stored in this table. Information stored includes patient number, advance payment, mode of payment, room number, department, date of admission, initial condition, diagnosis, treatment, number of the doctor under whom treatment is done, attendant name, etc.

Constraint: Patient number should exist in PAT_ENTRY table. Department, doctor number, room number must be valid.

8. **PAT_DIS:** An entry is made in this table whenever a patient gets discharged from the hospital. Each entry includes details like patient number, treatment given, treatment advice, payment made, mode of payment, date of discharge, etc.

Constraint: Patient number should exist in PAT_ENTRY table.

9. **PAT_REG:** Details of regular patients are stored in this table. Information stored includes date of visit, diagnosis, treatment, medicine recommended, status of treatment, etc.

Constraint: Patient number should exist in patient entry table. There can be multiple entries of one patient as patient might be visiting hospital repeatedly for check up and there will be entry for patient's each visit.

10. **PAT_OPR:** If patient is operated in the hospital, his/her details are stored in this table. Information stored includes patient number, date of admission, date of operation, number of the doctor who conducted the operation, number of the operation theater in which operation was carried out, type of operation, patient's condition before and after operation, treatment advice, etc.

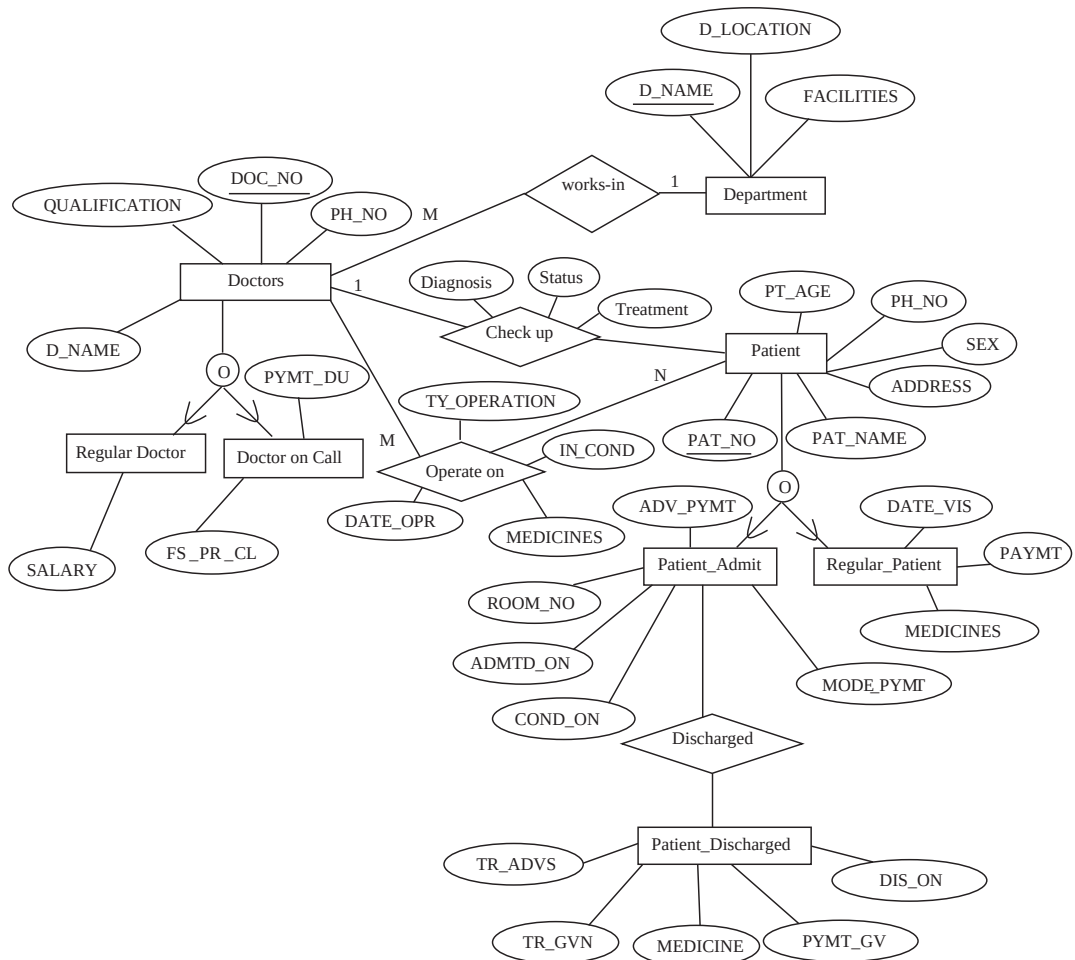
Constraint: Patient number should exist in PAT_ENTRY table. Department, doctor number should exist or should be valid.

11. **ROOM_DETAILS:** It contains details of all rooms in the hospital. The details stored in this table include room number, room type (general or private), status (whether occupied or not), if occupied, then patient number, patient name, charges per day, etc.

Constraint: Room number should be unique. Room type can only be G or P and status can only be Y or N.

E-R DIAGRAM

The E-R diagram of *Hospital Management* database is shown here.



RELATIONAL DATABASE SCHEMA FOR CASE STUDY

The relational database schema for *Hospital Management* database is as follows:

1. DEPARTMENT (D_NAME, D_LOCATION, FACILITIES)
2. ALL_DOCTORS (DOC_NO, DEPARTMENT)
3. DOC_REG(DOC_NO, D_NAME, QUALIFICATION, SALARY, EN_TIME, EX_TIME, ADDRESS, PH_NO, DOJ)
4. DOC_ON_CALL (DOC_NO, D_NAME, QUALIFICATION, FS_PR_CL, PYMT_DU, ADDRESS, PH_NO)
5. PAT_ENTRY (PAT_NO, PAT_NAME, CHKUP_DT, PT_AGE, SEX, RFRG_CSTNT, DIAGNOSIS, RFD, ADDRESS, CITY, PH_NO, DEPARTMENT)
6. PAT_CHKUP (PAT_NO, DOC_NO, DIAGNOSIS, STATUS, TREATMENT)
7. PAT_ADMIT (PAT_NO, ADV_PYMT, MODE_PYMT, ROOM_NO, DEPTNAME, ADMTD_ON, COND_ON, INVSTGTN_DN, TRMT_SDT, ATTDNT_NM)
8. PAT_DIS (PAT_NO, TR_ADVS, TR_GVN, MEDICINES, PYMT_GV, DIS_ON)
9. PAT_REG (PAT_NO, DATE_VIS, CONDITION, TREATMENT, MEDICINES, DOC_NO, PAYMT)
10. PAT_OPR (PAT_NO, DATE_OPR, IN_COND, AFOP_COND, TY_OPERATION, MEDICINES, DOC_NO, OPTH_NO, OTHER_SUG)
11. ROOM_DETAILS (ROOM_NO, TYPE, STATUS, RM_DL_CRG, OTHER_CRG)

IMPLEMENTATION IN SQL SERVER

We have used SQL server to create *Hospital Management* database. In this section, we will discuss how to create tables, insert values in tables, and retrieve data from tables.

Creating Tables

The CREATE TABLE commands for the relations of *Hospital Management* database along with the required constraints are specified here.

1. **CREATE TABLE** DEPARTMENT
 (D_NAME VARCHAR(20) **CONSTRAINT** DNPRKY **PRIMARY KEY**,
 D_LOCATION VARCHAR(35),
 FACILITIES VARCHAR(50)
);
2. **CREATE TABLE** ALL_DOCTORS
 (DOC_NO VARCHAR(8) **CONSTRAINT** DCPRKY **PRIMARY KEY**
CONSTRAINT LIKETHESE **CHECK**(DOC_NO **LIKE** 'DR%' **OR**
 DOC_NO **LIKE** 'HD%' **OR** DOC_NO **LIKE** 'DA%' **OR** DOC_NO
LIKE 'DC%'),
 DEPARTMENT VARCHAR(20) **CONSTRAINT** ADDFRKYDMT
REFERENCES DEPARTMENT
);

3. CREATE TABLE DOC_REG

```
( DOC_NO VARCHAR(8) CONSTRAINT DCRGFRKY REFERENCES
    ALL_DOCTORS CONSTRAINT DRLIKE CHECK(DOC_NO LIKE
        'DR%'),
  D_NAME VARCHAR(25),
  QUALIFICATION VARCHAR(10),
  SALARY NUMERIC(6),
  EN_TIME SMALLDATETIME,
  EX_TIME SMALLDATETIME,
  ADDRESS VARCHAR(50),
  PH_NO VARCHAR(20),
  DOJ SMALLDATETIME
);
```

4. CREATE TABLE DOC_ON_CALL

```
( DOC_NO VARCHAR(8) CONSTRAINT DCCLFRKY REFERENCES
    ALL_DOCTORS CONSTRAINT DCLIKE CHECK(DOC_NO LIKE
        'DC%'),
  D_NAME VARCHAR(25),
  QUALIFICATION VARCHAR(10),
  FS_PR_CL NUMERIC(5),
  PYMT_DU NUMERIC(6),
  ADDRESS VARCHAR(50),
  PH_NO VARCHAR(20)
);
```

5. CREATE TABLE PAT_ENTRY

```
( PAT_NO VARCHAR(8) CONSTRAINT PTPRKY PRIMARY KEY
    CONSTRAINT PTLIKE CHECK(PAT_NO LIKE 'PT%'),
  PAT_NAME VARCHAR(25),
  CHKUP_DT SMALLDATETIME,
  PT_AGE NUMERIC(3),
  SEX VARCHAR(1) CONSTRAINT SEX CHECK(SEX IN
    ('M', 'F')),
  RFRG_CSTNT VARCHAR(25),
  DIAGNOSIS VARCHAR(50),
  RFD VARCHAR(1) CONSTRAINT RFYN CHECK(RFD
    IN('Y', 'N')),
  ADDRESS VARCHAR(50),
  CITY VARCHAR(10),
  PH_NO VARCHAR(10),
  DEPARTMENT VARCHAR(20) CONSTRAINT PTETDPTNFRKYDPT
    REFERENCES DEPARTMENT
);
```

6. CREATE TABLE PAT_CHKUP

```
( PAT_NO VARCHAR(8) CONSTRAINT PRCKRFKY REFERENCES
    PAT_ENTRY,
```



```

DOC_NO VARCHAR(8) CONSTRAINT PTRFDCPRKY REFERENCES
    ALL_DOCTORS,
DIAGNOSIS VARCHAR(50),
STATUS VARCHAR(10),
TREATMENT VARCHAR(35)
);

```

7. **CREATE TABLE ROOM_DETAILS**

```

( ROOM_NO VARCHAR(6) CONSTRAINT RMNOPRKY PRIMARY KEY
    CONSTRAINT RNLIKE CHECK (ROOM_NO LIKE 'RN%'),
TYPE VARCHAR(1) CONSTRAINT WDINGP CHECK(TYPE
    IN('G', 'P')),
STATUS VARCHAR(1) CONSTRAINT INYN CHECK(STATUS
    IN('Y', 'N')),
RM_DL_CRG NUMERIC(4),
OTHER_CRG NUMERIC(5)
);

```

8. **CREATE TABLE PAT_ADMIT**

```

( PAT_NO VARCHAR(8) CONSTRAINT PRADRFKYPE REFERENCES
    PAT_ENTRY CONSTRAINT PRADPRKY PRIMARY KEY,
ADV_PYMT NUMERIC(6) CONSTRAINT ADVPYMT CHECK
    (ADV_PYMT >= 500),
MODE_PYMT VARCHAR(5) CONSTRAINT MDPYMT CHECK
    (MODE_PYMT IN('CHEQUE', 'CASH')),
ROOM_NO VARCHAR(6) CONSTRAINT PARMNOFRKYOCDT
    REFERENCES ROOM_DETAILS,
DEPTNAME VARCHAR(20) CONSTRAINT PAFRKYDP REFERENCES
    DEPARTMENT,
ADMTD_ON SMALLDATETIME,
COND_ON VARCHAR(50),
INVSTGTN_DN VARCHAR(20),
TRMT_SDT SMALLDATETIME,
ATTDNT_NM VARCHAR(20)
);

```

9. **CREATE TABLE PAT_DIS**

```

( PAT_NO VARCHAR(8) CONSTRAINT PADSFRKY REFERENCES
    PAT_ADMIT,
TR_ADV VARCHAR(50),
TR_GVN VARCHAR(50),
MEDICINES VARCHAR(50),
PYMT_GV NUMERIC(7),
DIS_ON SMALLDATETIME
);

```

- ```

10. CREATE TABLE PAT_REG
 (PAT_NO VARCHAR(8) CONSTRAINT PARERFKY REFERENCES
 PAT_ENTRY,
 DATE_VIS SMALLDATETIME,
 CONDITION VARCHAR(30),
 TREATMENT VARCHAR(50),
 MEDICINES VARCHAR(50),
 DOC_NO VARCHAR(8) CONSTRAINT PAREDCPRKY REFERENCES
 ALL_DOCTORS,
 PAYMT NUMERIC(4)
);

11. CREATE TABLE PAT_OPR
 (PAT_NO VARCHAR(8) CONSTRAINT PAOPRFKY REFERENCES
 PAT_ENTRY,
 DATE_OPR SMALLDATETIME,
 IN_COND VARCHAR(35),
 AFOP_COND VARCHAR(35),
 TY_OPERATION VARCHAR(40),
 MEDICINES VARCHAR(5),
 DOC_NO VARCHAR(8) CONSTRAINT PAOPDCPRKY REFERENCES
 ALL_DOCTORS,
 OPTH_NO VARCHAR(4),
 OTHER_SUG VARCHAR(30)
);

```

### Inserting Values in Tables

We have shown insertion of only one record in all the tables. A number of records can be added (as and when required) in the same way.

- INSERT INTO** DEPARTMENT  
**VALUES** ('Cardiology', 'FirstFloor', 'GENERAL AND SPECIALIZED SURGERY');
- INSERT INTO** ALL\_DOCTORS  
**VALUES** ('DR001', 'Cardiology');
- INSERT INTO** DOC\_REG  
**VALUES** ('DR001', 'DR.RAGINI ARAORA', 'MBBS, MD', '25000', '8:00:00 AM', '8:00:00 PM', '230-F, MAYUR VIHAR NEW DELHI', '2568931', '02/02/2009');
- INSERT INTO** DOC\_ON\_CALL  
**VALUES** ('DC001', 'DR.SUSANTA PRADHAN', 'MBBS, MD', '500', '1000', '120F-NOIDA', '2587496');

5. **INSERT INTO PAT\_ENTRY**  
**VALUES**('PT002', 'MR.ALOK', '02/02/2009', '23', 'M',  
 'DR.BRIJESH', 'SWELLING IN JOINTS', 'Y', '330F-NOIDA', 'NEW  
 DELHI', '7584289', 'Cardiology');
6. **INSERT INTO PAT\_CHKUP**  
**VALUES**('PT001', 'DR001', 'CRACK IN ANKLE BONE', 'SERIOUS',  
 'PLASTER');
7. **INSERT INTO PAT\_ADMIT**  
**VALUES** ('PT001', '150000', 'CASH', 'RN001', 'Cardiology', '05/02/2009', 'SERIOUS', 'LEAKAGE IN HEART', '06/02/2009', 'MR. SUDHANSU');
8. **INSERT INTO PAT\_DIS**  
**VALUES** ('PT001', 'REST FOR 6 MNTHS', 'OBSERVED FOR WEEK', 'ANTI ALLERGICS', '275000', '5/12/2009');
9. **INSERT INTO PAT\_REG**  
**VALUES** ('PT001', '8/2/2009', 'STABLE', 'CONTINUE THE MEDICINE', 'PAINKILLER', 'DR001', '100');
10. **INSERT INTO PAT\_OPR**  
**VALUES**('PT001', '6/2/2009', 'CONTROLLED', 'STABLE', 'OPEN HEART SURGERY', 'ANALR', 'DR001', '10', 'UNDER OBSERVATION FOR 24 HOURS');
11. **INSERT INTO ROOM\_DETAILS**  
**VALUES** ('RN001', 'G', 'N', '500', '100');

## SQL Queries

**Query 1:** Retrieve details of the doctor whose doctor number is *DR001*.

```
SELECT *
FROM ALL_DOCTORS
WHERE DOC_NO = 'DR001';
```

**Query 2:** Retrieve details of the regular doctors having salary between 25000 and 30000.

```
SELECT *
FROM DOC_REG
WHERE SALARY BETWEEN 25000 and 30000;
```

**Query 3:** Retrieve names of all the patients with elimination of duplicates.

```
SELECT DISTINCT PAT_NAME
FROM PAT_ENTRY;
```

**Query 4:** Retrieve number, name, qualification, present salary, salary incremented by 30%, address, phone number, and date of joining of all the regular doctors.

```
SELECT DOC_NO, D_NAME, QUALIFICATION, SALARY, SALARY+SALARY*.30,
ADDRESS, PH_NO, DOJ
FROM DOC_REG;
```

**Query 5:** Retrieve details of all the patients admitted in *Cardiology* or *Orthopedic* department.

```
SELECT *
FROM PAT_ADMIT
WHERE DEPTNAME IN ('Cardiology', 'Orthopedic');
```

**Query 6:** Retrieve name and age of all the patients and rename the columns as *PATIENT\_NAME* and *AGE*, respectively in the output.

```
SELECT PAT_NAME AS PATIENT_NAME, PT_AGE AS AGE
FROM PAT_ENTRY;
```

**Query 7:** Retrieve the sum of salaries of the regular doctors.

```
SELECT SUM(SALARY)
FROM DOC_REG;
```

**Query 8:** Delete the tuple from *ALL\_DOCTORS* where *DOC\_NO* is *DR002*.

```
DELETE FROM ALL_DOCTORS
WHERE DOC_NO = 'DR002';
```

**Query 9:** Delete tuples of regular doctors having salary less than the average salary of regular doctors.

```
DELETE FROM DOC_REG
WHERE SALARY < (SELECT AVG(SALARY)
FROM DOC_REG);
```

**Query 10:** Increase the fees per call of all the visiting doctors of the hospital by 2%.

```
UPDATE DOC_ON_CALL
SET FS_PR_CL = FS_PR_CL*1.02;
```

**Query 11:** Modify the location of *Orthopedic* department to *FourthFloor*.

```
UPDATE DEPARTMENT
SET D_LOCATION = 'FourthFloor'
WHERE D_NAME = 'Orthopedic';
```

**Query 12:** Retrieve all the doctor numbers, department name, department location, and facilities provided by the department.

```
SELECT DOC_NO, DEPARTMENT, D_LOCATION, FACILITIES
FROM ALL_DOCTORS, DEPARTMENT;
```

**Query 13:** Retrieve number and name of regular doctors along with the name of department in which they work.

```
SELECT DOC_REG.DOC_NO, D_NAME, DEPARTMENT
FROM ALL_DOCTORS INNER JOIN DOC_REG
ON (ALL_DOCTORS.DOC_NO = DOC_REG.DOC_NO);
```

**Query 14:** Modify **Query 13** to include names of those departments also for which there is no regular doctor.

```
SELECT DOC_REG.DOC_NO, D_NAME, DEPARTMENT
FROM ALL_DOCTORS LEFT OUTER JOIN DOC_REG
ON (ALL_DOCTORS.DOC_NO = DOC_REG.DOC_NO);
```

**Query 15:** Retrieve the maximum, minimum, and average salary of regular doctors.

```
SELECT MAX(SALARY), MIN(SALARY), AVG(SALARY)
FROM DOC_REG;
```

**Query 16:** Find the union of the names of regular doctors and names of call on doctors without eliminating the duplicates.

```
(SELECT D_NAME
FROM DOC_REG)
UNION ALL
(SELECT D_NAME
FROM DOC_ON_CALL);
```

**Query 17:** Find the union of name of regular doctor with doctor number *DR001* and the name of doctor called by hospital with doctor number *DC001*.

```
(SELECT D_NAME
FROM DOC_REG
WHERE DOC_NO='DR001')
UNION
(SELECT D_NAME
FROM DOC_ON_CALL
WHERE DOC_NO='DC001');
```

**Query 18:** Find the union of regular doctors with salary less than 31000 and the regular doctors with salary greater than 20000.

```
(SELECT *
FROM DOC_REG
WHERE SALARY<31000)
UNION
```

```
(SELECT *
FROM DOC_REG
WHERE SALARY>20000);
```

**Query 19:** Retrieve names of those patients who have been operated by the doctor having doctor number *DR002*.

```
SELECT PAT_NAME
FROM PAT_ENTRY
WHERE PAT_NO IN (SELECT PAT_NO
FROM PAT_OPR
WHERE DOC_NO = 'DR002');
```

**Query 20:** Retrieve the qualification and average salary of the regular doctors grouped on the basis of qualification and having minimum salary greater than 20000.

```
SELECT QUALIFICATION, AVG(SALARY)
FROM DOC_REG
GROUP BY QUALIFICATION
HAVING MIN(SALARY)>20000;
```

**Query 21:** Retrieve details of the regular doctors with minimum salary.

```
SELECT *
FROM DOC_REG
WHERE SALARY = (SELECT MIN(SALARY) FROM DOC_REG);
```

**Query 22:** Retrieve number and names of regular patients of the doctor with doctor number *DR002*.

```
SELECT P1.PAT_NO, P2.PAT_NAME
FROM PAT_REG AS P1 INNER JOIN PAT_ENTRY AS P2
ON (P1.PAT_NO = P2.PAT_NO)
WHERE P1.DOC_NO = 'DR002';
```

**Query 23:** Retrieve number and names of all those patients who have been discharged on the same date as the patient having patient number *PT001*.

```
SELECT P1.PAT_NO, P2.PAT_NAME
FROM PAT_DIS AS P1 INNER JOIN PAT_ENTRY AS P2
ON (P1.PAT_NO = P2.PAT_NO)
WHERE P1.DIS_ON = (SELECT DIS_ON
FROM PAT_DIS
WHERE PAT_NO = 'PT001');
```

**Query 24:** Retrieve patient number and room number of all occupied private rooms.

```
SELECT P.PAT_NO, R.ROOM_NO
FROM PAT_ADMIT AS P INNER JOIN ROOM_DETAILS AS R
ON (P.ROOM_NO = R.ROOM_NO)
WHERE R.TYPE = 'P' AND R.STATUS = 'Y';
```

**Query 25:** Create a view containing details of regular doctors having qualification *MBBS* or *MD*.

```
CREATE VIEW DOC_REG_1
AS SELECT *
FROM DOC_REG
WHERE QUALIFICATION IN ('MBBS', 'MD');
```

# CaseStudy-2

## Railway Reservation

### AIM

---

The railway reservation system facilitates the passengers to enquire about the trains available on the basis of source and destination, booking and cancellation of tickets, enquire about the status of the booked ticket, etc.

The aim of case study is to design and develop a database maintaining the records of different trains, train status, and passengers. The record of train includes its number, name, source, destination, and days on which it is available, whereas record of train status includes dates for which tickets can be booked, total number of seats available, and number of seats already booked. The database has been developed and tested on the Oracle.

### DESCRIPTION

---

Passengers can book their tickets for the train in which seats are available. For this, passenger has to provide the desired train number and the date for which ticket is to be booked. Before booking a ticket for a passenger, the validity of train number and booking date is checked. Once the train number and booking date are validated, it is checked whether the seat is available. If yes, the ticket is booked with confirm status and corresponding ticket ID is generated which is stored along with other details of the passenger. After all the available tickets are booked, certain number of tickets are booked with waiting status. If waiting lot is also finished, then tickets are not booked and a message of non-availability of seats is displayed.

The ticket once booked can be cancelled at any time. For this, the passenger has to provide the ticket ID (the unique key). The ticket ID is searched and the corresponding record is deleted. With this, the first ticket with waiting status also gets confirmed.

### LIST OF ASSUMPTIONS

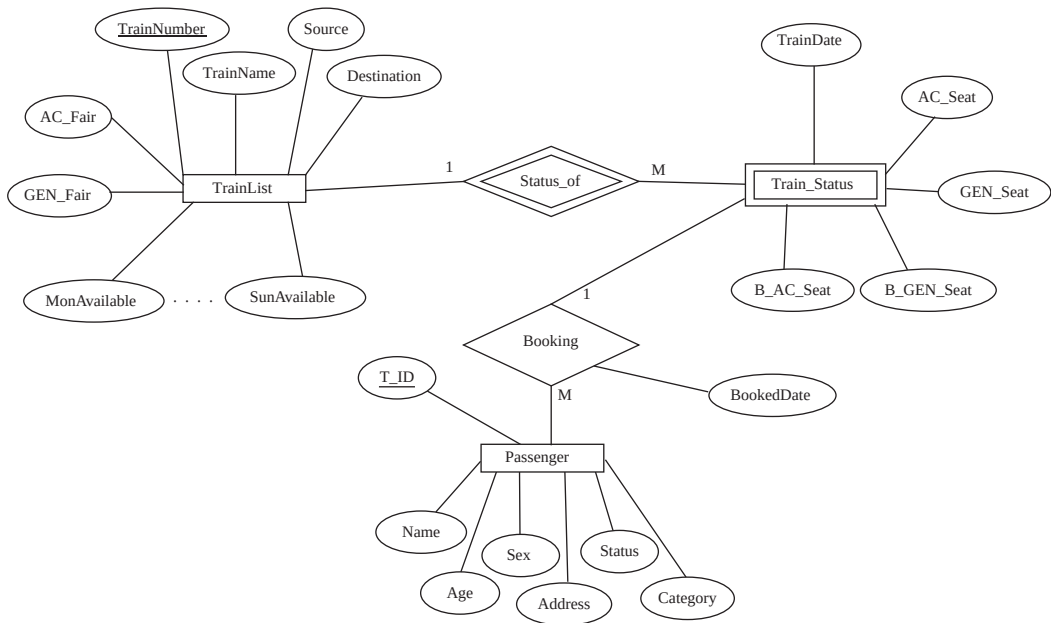
---

Since the reservation system is very large in reality, it is not feasible to develop the case study to that extent and prepare documentation at that level. Therefore, a small sample case study has been created to demonstrate the working of the reservation system. To implement this sample case study, some assumptions have been made, which are as follows:

1. The number of trains has been restricted to 5.
2. The booking is open only for next seven days from the current date.
3. Only two categories of tickets can be booked, namely, *AC* and *General*.
4. The total number of tickets that can be booked in each category (*AC* and *General*) is 10.
5. The total number of tickets that can be given the status of waiting is 2.
6. The in-between stoppage stations and their bookings are not considered.



## E-R DIAGRAM



## DESCRIPTION OF TABLES AND PROCEDURES

Tables and procedures that will be created are as follows:

1. **TrainList:** This table consists of details about all the available trains. The information stored in this table includes train number, train name, source, destination, fair for AC ticket, fair for general ticket, and weekdays on which train is available.  
*Constraint:* The train number is unique.
2. **Train\_Status:** This table consists of details about the dates on which ticket can be booked for a train and the status of the availability of tickets. The information stored in this table includes train number, train date, total number of AC seats, total number of general seats, number of AC seats booked, and number of general seats booked.  
*Constraint:* Train number should exist in **TrainList** table.
3. **Passenger:** This table consists of details about the booked tickets. The information stored in this table includes ticket ID, train number, date for which ticket is booked, name, age, sex and address of the passenger, status of reservation (either confirmed or waiting), and category for which ticket is booked.  
*Constraint:* Ticket ID is unique and the train number should exist in **TrainList** table.
4. **Booking:** In this procedure, the train number, train date, and category is read from the passenger. On the basis of the values provided by the passenger, corresponding record is retrieved from the **Train\_Status** table. If the desired category is AC, then total number of AC seats and number of booked AC seats are compared in order to find whether ticket can be booked or not. Similarly, it can be checked for the general category. If ticket can be booked, then passenger details are read and stored in the **Passenger** table.

5. **Cancel:** In this procedure, ticket ID is read from the passenger and corresponding record is searched in the **Passenger** table. If the record exists, it is deleted from the table. After deleting the record (if it is confirmed), first record with waiting status for the same train and same category are searched from the **Passenger** table and its status is changed to confirm.

## RELATIONAL DATABASE SCHEMA FOR CASE STUDY\_\_\_\_\_

1. **TrainList**(TrainNumber, TrainName, Source, Destination, AC\_Fair, GEN\_Fair, MonAvailable, TueAvailable, WedAvailable, ThuAvailable, FriAvailable, SatAvailable, SunAvailable)
2. **Train\_Status**(TrainNumber, TrainDate, AC\_Seat, GEN\_Seat, B\_AC\_Seat, B\_GEN\_Seat)
3. **Passenger**(T\_ID, TrainNumber, BookedDate, Name, Age, Sex, Address, Status, Category)

### Implementation in Oracle

1. **CREATE TABLE** TrainList

```
(
 TrainNumber NUMERIC(4),
 TrainName VARCHAR(20),
 Source VARCHAR(20),
 Destination VARCHAR(20),
 AC_Fair NUMERIC(10),
 GEN_Fair NUMERIC(10),
 MonAvailable VARCHAR(1),
 TueAvailable VARCHAR(1),
 WedAvailable VARCHAR(1),
 ThuAvailable VARCHAR(1),
 FriAvailable VARCHAR(1),
 SatAvailable VARCHAR(1),
 SunAvailable VARCHAR(1),
 PRIMARY KEY(TrainNumber)
);
```

2. **CREATE TABLE** Train\_Status

```
(
 TrainNumber NUMERIC(4),
 TrainDate DATE,
 AC_Seat NUMERIC(4),
 GEN_Seat NUMERIC(4),
 B_AC_Seat NUMERIC(4),
 B_GEN_Seat NUMERIC(4),
 FOREIGN KEY(TrainNumber) REFERENCES TrainList(TrainNumber)
);
```

3. **CREATE TABLE** Passenger

```
(
 T_ID VARCHAR(15),
 TrainNumber NUMERIC(4),
 BookedDate DATE,
 Name VARCHAR(20),
 Age NUMERIC(3),
 Sex VARCHAR(1),
 Address VARCHAR(35),
 Status VARCHAR(1),
 Category VARCHAR(10),
 PRIMARY KEY(T_ID),
 FOREIGN KEY(TrainNumber) REFERENCES TrainList(TrainNumber)
);
```

## 4. BookingProc edure

**DECLARE**

```
TN NUMERIC(4);
DT DATE;
CAT VARCHAR(10);
ST VARCHAR(1);

CURSOR getTrain IS
SELECT * FROM Train_Status
WHERE TrainNumber=TN AND TrainDate=DT;
trec Train_Status%ROWTYPE;
```

**BEGIN**

```
DBMS_OUTPUT.PUT_LINE('READING VALUES');
TN := &TN;
DT := '&DT';
CAT := '&CAT';

OPEN getTrain;
LOOP
 FETCH getTrain INTO trec;
 EXIT WHEN getTrain%NOTFOUND;
 IF(CAT = 'AC')THEN
 IF(trec.AC_Seat-trec.B_AC_Seat)>0 THEN
 ST := 'C';
 DBMS_OUTPUT.PUT_LINE('Seat Confirmed');
 ELSIF(trec.AC_Seat-trec.B_AC_Seat) > -2 THEN
 ST := 'W';
 DBMS_OUTPUT.PUT_LINE('Seat Waiting');
 ELSE
 DBMS_OUTPUT.PUT_LINE('Seat Not Available');
 EXIT;
 END IF;
 IF ST = 'C' OR ST = 'W' THEN
```

```

 UPDATE Train_Status SET B_AC_Seat = B_AC_Seat + 1 WHERE
 TrainNumber = TN and TrainDate=DT;
 END IF;
ELSIF(CAT = 'GEN') THEN
 IF(trec.GEN_Seat-trec.B_GEN_Seat)>0 THEN
 ST := 'C';
 DBMS_OUTPUT.PUT_LINE('Seat Confirmed');
 ELSIF(trec.GEN_Seat-trec.B_GEN_Seat) > -2 THEN
 ST := 'W';
 DBMS_OUTPUT.PUT_LINE('Seat Waiting');
 ELSE
 DBMS_OUTPUT.PUT_LINE('Seat Not Available');
 EXIT;
 END IF;
 IF ST = 'C' OR ST = 'W' THEN
 UPDATE Train_Status SET B_GEN_Seat = B_GEN_Seat+1
 WHERE TrainNumber = TN and TrainDate=DT;
 END IF;
ELSE
 DBMS_OUTPUT.PUT_LINE('Invalid Category');
END IF;
IF ST = 'C' OR ST = 'W' THEN
 INSERT INTO Passenger VALUES('&T_ID', TN, DT, '&Name', &Age,
 '&Sex', '&Address', ST, CAT);
END IF;
END LOOP;
CLOSE getTrain;
END;
/

```

##### 5. CancelProc edure

###### **DECLARE**

```

 TID VARCHAR(15);
 TN NUMERIC(4);
 DT DATE;
 CAT VARCHAR(10);
 ST VARCHAR(1);
 CURSOR Get_waiting IS
 SELECT * FROM Passenger WHERE (Status = 'W' AND
 TrainNumber = TN AND BookedDate = DT AND Category = CAT) FOR
 UPDATE OF Status;
 prec Passenger%ROWTYPE;

```

###### **BEGIN**

```

 TID := '&TID';
 SELECT TrainNumber, BookedDate, Status, Category INTO TN, DT,
 ST, CAT FROM Passenger WHERE T_ID = TID;
 IF TN IS NOT NULL THEN

```

```

DELETE FROM Passenger WHERE T_ID = TID;
IF CAT = 'AC' THEN
 UPDATE Train_Status SET B_AC_Seat = B_AC_Seat - 1
 WHERE TrainNumber=TN AND TrainDate=DT;
ELSE
 UPDATE Train_Status SET B_GEN_Seat = B_GEN_Seat - 1
 WHERE TrainNumber=TN AND TrainDate=DT;
END IF;
DBMS_OUTPUT.PUT_LINE('TICKET CANCELLED');
ELSE
 DBMS_OUTPUT.PUT_LINE('NO SUCH BOOKING EXISTS');
END IF;
IF (ST = 'C') THEN
 OPEN Get_waiting;
 FETCH Get_waiting INTO prec;
 IF Get_waiting%FOUND THEN
 UPDATE Passenger SET Status = 'C'
 WHERE CURRENT OF Get_waiting;
 END IF;
 CLOSE Get_waiting;
END IF;

END;
/

```

## SampleQueries

- INSERT INTO** TrainList  
VALUES(2444, 'Bhubaneswar Rajdhani', 'New Delhi', 'Bhubaneswar',  
2250, 1850, 'Y', 'Y', 'N', 'Y', 'Y', 'Y', 'N');
- SELECT \***  
**FROM** TrainList;

| TRAINNUMBER | TRAINNAME            | SOURCE        | DESTINATION | AC_FAIR | GEN_FAIR | M | T | W | T | F | S | S |
|-------------|----------------------|---------------|-------------|---------|----------|---|---|---|---|---|---|---|
| 2444        | Bhubaneswar Rajdhani | New Delhi     | Bhubaneswar | 2250    | 1850     | Y | Y | N | Y | Y | Y | N |
| 8508        | Hirakund Express     | Visakhapatnam | Amritsar    | 1150    | 505      | N | N | Y | Y | N | Y | N |
| 8477        | Utkal Express        | Haridwar      | Puri        | 1400    | 495      | Y | Y | Y | Y | N | N | N |
| 2310        | Patna Rajdhani       | New Delhi     | Patna       | 1540    | 1100     | N | N | Y | Y | Y | Y | N |
| 2816        | Puri Express         | New Delhi     | Puri        | 1250    | 510      | Y | N | Y | N | Y | Y | N |

- SELECT** TrainNumber, TrainName, AC\_Fair, GEN\_Fair  
**FROM** TrainList  
**WHERE** Source = 'New Delhi' **AND** Destination = 'Puri';

| TRAINNUMBER | TRAINNAME    | AC_FAIR | GEN_FAIR |
|-------------|--------------|---------|----------|
| 2816        | Puri Express | 1250    | 510      |

4. **INSERT INTO** Train\_Status  
**VALUES**(2444, '02-MAY-2009', 10, 10, 0, 0);
5. **SELECT** \*  
**FROM** Train\_Status  
**WHERE** TrainNumber = 2444;

| TRAINNUMBER | TRAINDATE | AC_SEAT | GEN_SEAT | B_AC_SEAT | B_GEN_SEAT |
|-------------|-----------|---------|----------|-----------|------------|
| 2444        | 01-MAY-09 | 10      | 10       | 0         | 0          |
| 2444        | 02-MAY-09 | 10      | 10       | 0         | 0          |
| 2444        | 04-MAY-09 | 10      | 10       | 0         | 0          |
| 2444        | 05-MAY-09 | 10      | 10       | 0         | 0          |
| 2444        | 07-MAY-09 | 10      | 10       | 0         | 0          |

6. SQL> @e:\RailwayReservation\Booking  
Enter value for tn: 2444  
old 14:           TN := &TN;  
new 14:           TN := 2444;  
Enter value for dt: 02-MAY-2009  
old 15:           DT := '&DT';  
new 15:           DT := '02-MAY-2009';  
Enter value for cat: AC  
old 16:           CAT := '&CAT';  
new 16:           CAT := 'AC';  
Enter value for t\_id: 3  
Enter value for name: PREETI  
Enter value for age: 29  
Enter value for sex: F  
Enter value for address: ROHINI  
old 73: INSERT INTO Passenger VALUES('&T\_ID', TN, DT, '&Name',  
&Age, '&Sex', '&Address', ST, CAT);  
new 73: INSERT INTO Passenger VALUES('3', TN, DT, 'PREETI', 29,  
'F', 'ROHINI', ST, CAT);  
READING VALUES  
Seat Confirmed
7. SQL> @e:\RailwayReservation\Booking  
Enter value for tn: 2444  
old 14:           TN := &TN;  
new 14:           TN := 2444;  
Enter value for dt: 02-MAY-2009  
old 15:           DT := '&DT';  
new 15:           DT := '02-MAY-2009';  
Enter value for cat: AC  
old 16:           CAT := '&CAT';  
new 16:           CAT := 'AC';  
Enter value for t\_id: 4  
Enter value for name: PREMASHIS  
Enter value for age: 26

```

Enter value for sex: M
Enter value for address: MAYURVIHAR
old 73: INSERT INTO Passenger VALUES('&T_ID', TN, DT, '&Name',
&Age, '&Sex', '&Address', ST, CAT);
new 73: INSERT INTO Passenger VALUES('4', TN, DT, 'PREMASHIS',
26, 'M', 'MAYURVIHAR', ST, CAT);
READING VALUES
Seat Confirmed

```

8. SQL> @e:\RailwayReservation\Booking

```

Enter value for tn: 2444
old 14: TN := &TN;
new 14: TN := 2444;
Enter value for dt: 02-MAY-2009
old 15: DT := '&DT';
new 15: DT := '02-MAY-2009';
Enter value for cat: AC
old 16: CAT := '&CAT';
new 16: CAT := 'AC';
Enter value for t_id: 12
Enter value for name: UMA
Enter value for age: 26
Enter value for sex: F
Enter value for address: SUBHASHPLACE
old 73: INSERT INTO Passenger VALUES('&T_ID', TN, DT, '&Name',
&Age, '&Sex', '&Address', ST, CAT);
new 73: INSERT INTO Passenger VALUES('12', TN, DT, 'UMA', 26,
'F', 'SUBHASHPLACE', ST, CAT);
READING VALUES
Seat Waiting

```

9. SQL> @e:\RailwayReservation\Booking

```

Enter value for tn: 2444
old 14: TN := &TN;
new 14: TN := 2444;
Enter value for dt: 02-MAY-2009
old 15: DT := '&DT';
new 15: DT := '02-MAY-2009';
Enter value for cat: AC
old 16: CAT := '&CAT';
new 16: CAT := 'AC';
Enter value for t_id: 13
Enter value for name: MEENU
Enter value for age: 26
Enter value for sex: F
Enter value for address: PITAMPURA
old 73: INSERT INTO Passenger VALUES('&T_ID', TN, DT, '&Name',
&Age, '&Sex', '&Address', ST, CAT);

```

```

new 73: INSERT INTO Passenger VALUES('13', TN, DT, 'MEENU', 26,
'F', 'PITAMPURA', ST, CAT);
READING VALUES
Seat Waiting

```

#### 10. SQL> @e:\RailwayReservation\Booking

```

Enter value for tn: 2444
old 14: TN := &TN;
new 14: TN := 2444;
Enter value for dt: 02-MAY-2009
old 15: DT := '&DT';
new 15: DT := '02-MAY-2009';
Enter value for cat: AC
old 16: CAT := '&CAT';
new 16: CAT := 'AC';
Enter value for t_id: 14
Enter value for name: KHYATI
Enter value for age: 26
Enter value for sex: F
Enter value for address: PITAMPURA
old 73: INSERT INTO Passenger VALUES('&T_ID', TN, DT, '&Name',
&Age, '&Sex', '&Address', ST, CAT);
new 73: INSERT INTO Passenger VALUES('14', TN, DT, 'KHYATI', 26,
'F', 'PITAMPURA', ST, CAT);
READING VALUES
Seat Not Available

```

#### 11. SELECT \*

```

FROM Passenger
WHERE TrainNumber = 2444;

```

| T_ID | TRAINNUMBER | BOOKEDDATE | NAME      | AGE | SEX | ADDRESS      | STATUS | CATEGORY |
|------|-------------|------------|-----------|-----|-----|--------------|--------|----------|
| 1    | 2444        | 02-MAY-09  | SURENDER  | 24  | M   | NEW DELHI    | C      | AC       |
| 2    | 2444        | 02-MAY-09  | SHRUTI    | 25  | F   | PITAMPURA    | C      | AC       |
| 3    | 2444        | 02-MAY-09  | PREETI    | 29  | F   | ROHINI       | C      | AC       |
| 4    | 2444        | 02-MAY-09  | PREMASHIS | 26  | M   | MUYURVIHAR   | C      | AC       |
| 5    | 2444        | 02-MAY-09  | POOJA     | 29  | F   | KOHATENCLAVE | C      | AC       |
| 6    | 2444        | 02-MAY-09  | ROSHINI   | 26  | F   | PITAMPURA    | C      | AC       |
| 7    | 2444        | 02-MAY-09  | ROHIT     | 23  | M   | NSP          | C      | AC       |
| 8    | 2444        | 02-MAY-09  | NEETA     | 35  | F   | PITAMPURA    | C      | AC       |
| 9    | 2444        | 02-MAY-09  | RAJESH    | 35  | M   | SARITAVIHAR  | C      | AC       |
| 10   | 2444        | 02-MAY-09  | NITI      | 26  | F   | ROHINI       | C      | GEN      |
| 11   | 2444        | 02-MAY-09  | RAJA      | 25  | M   | SAHADARA     | C      | AC       |
| 12   | 2444        | 02-MAY-09  | UMA       | 26  | F   | SUBHASHPLACE | C      | AC       |
| 13   | 2444        | 02-MAY-09  | MEENU     | 26  | F   | PITAMPURA    | W      | AC       |

#### 12. SQL> @E:\RailwayReservation\Cancel

```

Enter value for tid: 9
old 11: TID := '&TID';
new 11: TID := '9';
TICKET CANCELLED

```



13. **SELECT \***  
**FROM** Passenger  
**WHERE** TrainNumber = 2444;

| T_ID | TRAINNUMBER | BOOKEDDATE | NAME      | AGE | SEX | ADDRESS      | STATUS | CATEGORY |
|------|-------------|------------|-----------|-----|-----|--------------|--------|----------|
| 1    | 2444        | 02-MAY-09  | SURENDER  | 24  | M   | NEW DELHI    | C      | AC       |
| 2    | 2444        | 02-MAY-09  | SHRUTI    | 25  | F   | PITAMPURA    | C      | AC       |
| 3    | 2444        | 02-MAY-09  | PREETI    | 29  | F   | ROHINI       | C      | AC       |
| 4    | 2444        | 02-MAY-09  | PREMASHIS | 26  | M   | MUYURVIHAR   | C      | AC       |
| 5    | 2444        | 02-MAY-09  | POOJA     | 29  | F   | KOHATENCLAVE | C      | AC       |
| 6    | 2444        | 02-MAY-09  | ROSHINI   | 26  | F   | PITAMPURA    | C      | AC       |
| 7    | 2444        | 02-MAY-09  | ROHIT     | 23  | M   | NSP          | C      | AC       |
| 8    | 2444        | 02-MAY-09  | NEETA     | 35  | F   | PITAMPURA    | C      | AC       |
| 10   | 2444        | 02-MAY-09  | NITI      | 26  | F   | ROHINI       | C      | GEN      |
| 11   | 2444        | 02-MAY-09  | RAJA      | 25  | M   | SAHADARA     | C      | AC       |
| 12   | 2444        | 02-MAY-09  | UMA       | 26  | F   | SUBHASHPLACE | W      | AC       |
| 13   | 2444        | 02-MAY-09  | MEENU     | 26  | F   | PITAMPURA    | W      | AC       |

14. **SELECT \***  
**FROM** Train\_Status  
**WHERE** TrainNumber = 2444;

| TRAINNUMBER | TRAINDATE | AC_SEAT | GEN_SEAT | B_AC_SEAT | B_GEN_SEAT |
|-------------|-----------|---------|----------|-----------|------------|
| 2444        | 01-MAY-09 | 10      | 10       | 0         | 0          |
| 2444        | 02-MAY-09 | 10      | 10       | 11        | 1          |
| 2444        | 04-MAY-09 | 10      | 10       | 0         | 0          |
| 2444        | 05-MAY-09 | 10      | 10       | 0         | 0          |
| 2444        | 07-MAY-09 | 10      | 10       | 0         | 0          |



**NOTE** To display the output messages of the procedures, the command **Set Serveroutput On** is given before executing the procedures.

# Appendix A

## Microsoft Access as Database Management System

Microsoft Office Access, previously known as Microsoft Access is a relational database management system that lets you manage your database efficiently by offering a variety of features. It facilitates creation of database in several ways, manage database structures, export and import databases to/from various data sources and file formats, backup, restore, and perform other operations in easy-to-use graphical user interface. Access can use data stored in Microsoft SQL Server, Oracle, or any ODBC-compliant container including MySQL and PostgreSQL. It supports some object-oriented features but cannot be considered as a fully object-oriented development tool. Access also integrates well with other Office packages, and data transfer between Access and other Office components is relatively easy. Almost all users can use it. Beginners can learn to use the wizards and the easy-to-understand interface while developers can push it to its limits and do some extraordinary things with it. This appendix discusses the basics of designing and creating an efficient and productive database using Access.

### A.1 COMPONENTS OF ACCESS

---

Using an Access database, one can store and manage large quantity of data for a wide variety of business and personal activities. Before learning to work with Access, let us first discuss the components that make up the Access.

- ⇒ **Database engine:** The database engine is the software that stores, retrieves, and indexes data. When you create a standalone database, Access uses the jet engine to manage data.
- ⇒ **Database objects:** Database objects provide the interface to view, enter, and extract information from a database. Access provides many types of objects such as tables, forms, queries, reports, pages, macros, and modules.
- ⇒ **Design tools:** Access includes a set of design tools to create objects. For example, report designer allows sorting data, group by fields, and adding headers and footers to each page as well as the entire report.
- ⇒ **Programming tools:** Access includes a rich set of programming tools to automate repetitive and routine tasks. For example, an object type called macro is used to do tasks that require multiple menu selections and mouse clicks.

Database objects are the basic building blocks of Access database. Access provides many types of objects. The most common database objects are tables, forms, queries, and reports. Learning how to work with these data objects is the main aim of this appendix.

## A.2 STARTING ACCESS

To start Access, click on **Start**, select **Programs**, and then select **Microsoft Access**. The Microsoft Access dialog box appears with the following options:

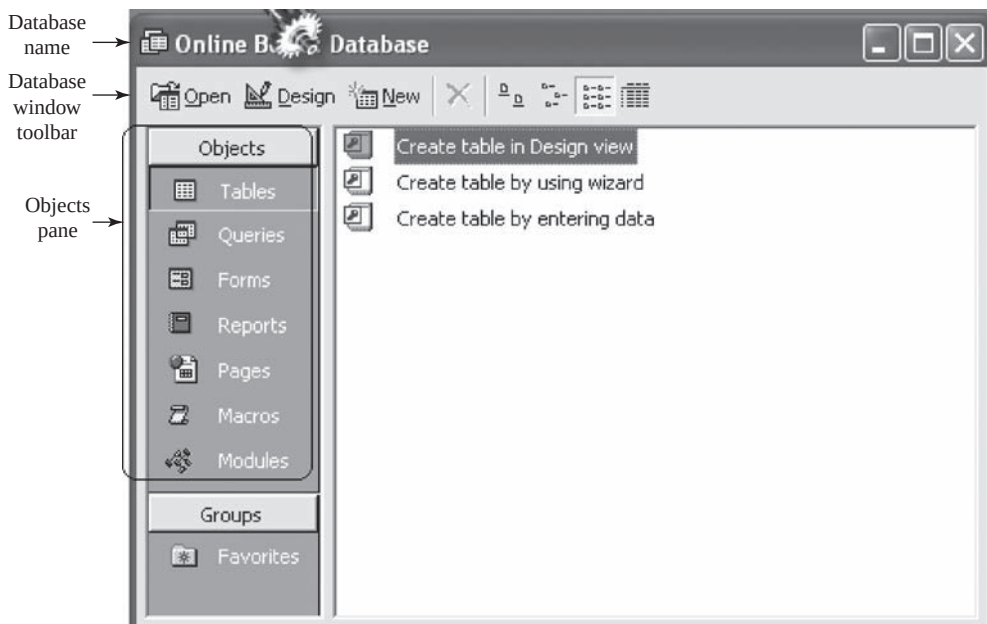
- ⇒ **Blank Access database:** To create a new blank database to which tables, forms, and other objects can be added.
- ⇒ **Access database wizards, pages, and projects:** To create a new database based on wizards you choose or to create a database access page or Microsoft Access project.
- ⇒ **Open an existing database:** To open an already existing database or project.

Choose the **Blank Access database** option to create a new database, and click **OK**. Access will prompt you to specify a location and a name for the database. Enter the desired name of the database in the **Filename** textbox. For instance, we can give the database name as *Online Book*. Then click the **Create** button. This displays the Database window as a sub window within the Microsoft Access Application window.



**NOTE** In Microsoft Access, it is essential to first save the database and then create it. By default, Microsoft Access saves a database with **.mdb** extension.

In the Database window, the name of the database is displayed on the title bar (see Figure A.1). In addition, it contains an **Object pane** that displays different objects that can be created in Microsoft Access.



**Fig. A.1** Database window

## A.3 WORKING WITH TABLES

A **table** is the basic unit for storing and organizing data in an Access database. A table consists of fields, each of which stores a distinct data for a single record. Data within a table is arranged in a basic grid, with each row containing a single record and each column representing a field. A database can contain any number of tables as well as links to tables stored in other location and other formats.

### Choosing a Data Type

The fields that make up a table can be defined in several ways. However, each field definition includes a data type. You can choose from the data types listed in Table A.1 for defining a field.

**Table A.1** Access data types

| Data Type     | Description                                                                                                                                                                                                                                       |
|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Text          | It is used for field values that contain letters, digits, spaces, and special characters. The default size of text field is 50 and it can contain up to 255 characters.                                                                           |
| Memo          | It is used for long text comments and can contain up to 64,000 characters.                                                                                                                                                                        |
| Number        | It is used for numeric values.                                                                                                                                                                                                                    |
| Date/Time     | It is used for date and time.                                                                                                                                                                                                                     |
| Currency      | It is used for currency values. Currency fields are similar to the number data type, except that the decimal places and field size are pre-determined, and calculations performed using the currency data type is not subject to round-off error. |
| AutoNumber    | It is used when you want integers or a value automatically inserted in the field as each new record is created.                                                                                                                                   |
| Yes/No        | It is used for yes/no, true/false, or on/off values.                                                                                                                                                                                              |
| OLE Object    | It is used to embed or provide links to objects like graphics, spreadsheets, sound files, etc.                                                                                                                                                    |
| Hyperlink     | It is used to enter clickable links to web addresses, folders, files, and other objects. Each item in a field of this data type can contain up to 64,000 characters.                                                                              |
| Lookup Wizard | This is not a data type, but this option will create a field that lets you to select a value from another table or from a pre-defined list of values. Once the list is created, Access will then set the data type for you.                       |



**NOTE** By default, every new field you create uses the Text data type, with a maximum length of 50 characters.

Tables form the basis for all objects within an Access database including queries, forms, and reports. In this section, we will discuss how to create tables and establish relationships among them, and to add, modify, and delete records from tables.

### A.3.1 Creating Tables

Access allows creating tables in three different ways: using Design view, using Table Wizard, and using Datasheet view. The easiest approach to create a table is to use the Table Wizard. This wizard

assists in creating tables by suggesting one of the pre-defined tables appropriate for business or personal use. However, it is preferred to create table in Design view. Since, it gives you greater flexibility to design a table that meets your exact requirements.

To create a table in Design view, follow these steps.

1. Click the **Tables** button in the database window on the **Objects** pane.
2. Double-click the **Create table in Design view**. The Design view of a new table will be displayed where you can type the field names, data types, and descriptions.
3. Type in the first field name and press **Tab** or **Enter** to move to the **Data Type**. Note that field names can contain letters, numbers, and/or spaces.
4. Choose the data type from the drop-down list and press **Tab** or **Enter** again to move to the **Description** field.
5. The **Description** field, while optional, is good to use for documentation purposes as well as for providing directions to the person inputting the data.
6. Continue steps 3–5 until all the desired fields are created in the table.
7. While not mandatory, every table should have a primary key, a field whose value uniquely identifies each of the records of the table. For example, in our table [see Figure A. 2(a)] **ISBN** is set as the primary key of the table. To create a primary key, place the cursor in the field row and right-click it. Now select the **Primary Key** option from the pop-up menu. Similarly, multiple fields can be defined as primary key.



**NOTE** *If no primary key is specified by the user, Access creates a primary key of AutoNumber data type automatically under some circumstances.*

The **BOOK** table in Design view is shown in Figure A.2(a). After designing the table, we can save the table with a desired name. For instance, we have saved our example table as **BOOK**. Once, it is saved, it will be displayed in Database window as shown in Figure A.2(b). Similarly, other tables of *Online Book* database can be created.



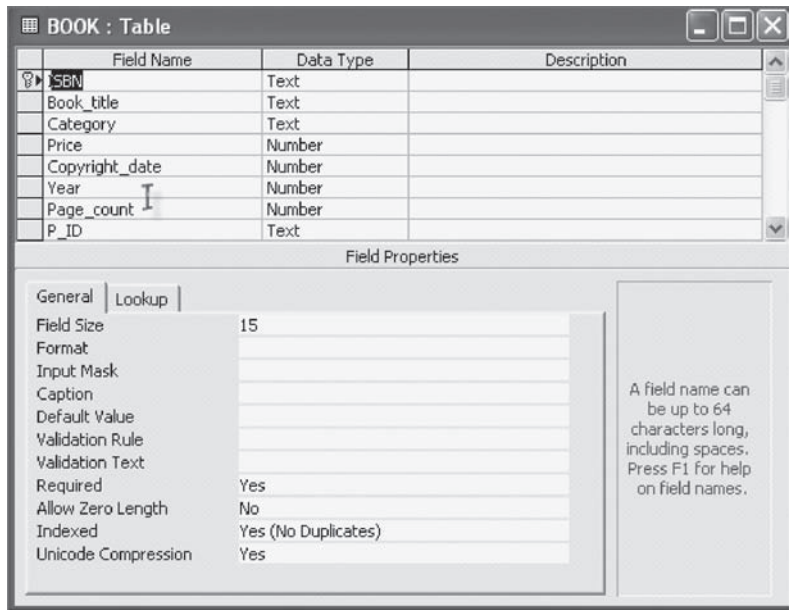
**NOTE** *Creating a new table by entering records directly in Datasheet view is a least used approach.*

Once a table is created, one can change its structure at any time, even if data is added to the table. New fields can be added, and existing fields can be deleted, reordered, or renamed in the Design as well as Datasheet view. However, to change field's data type and properties that validated data entries, one must switch to Design view.

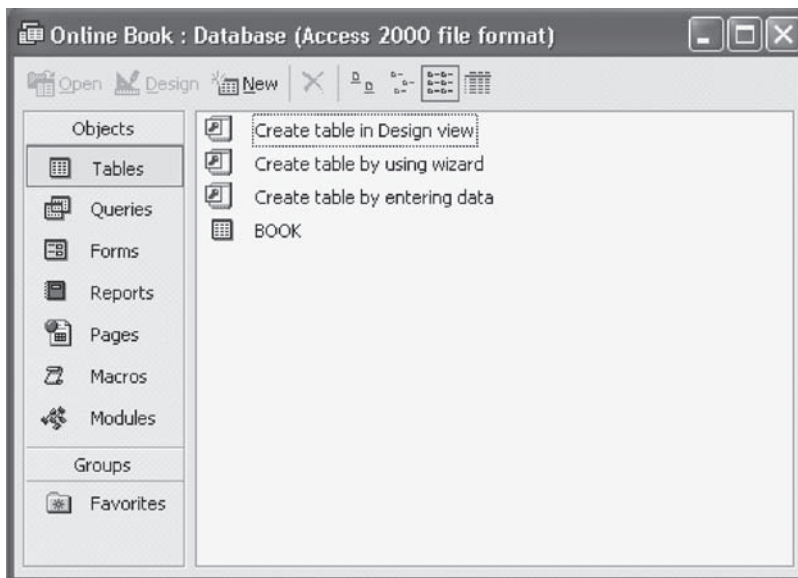


### Using Advanced Options

Access creates some indexes automatically. If naming conventions are followed carefully, it is guaranteed that right fields are indexed. By default, any field name that begins or ends with ID, key, code or num will be indexed. Use field names such as AID and RollNum as the primary key when possible. When same field name is used in related tables where another field is the primary key, Access automatically indexes these fields, making queries that use these fields as fast as possible.



(a) Table BOOK in design view



(b) Table BOOK in Database window

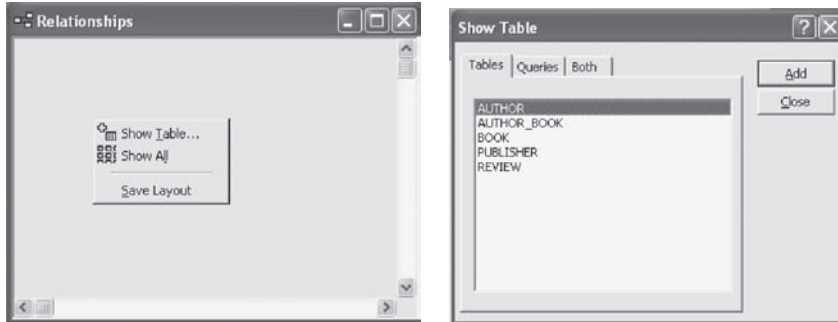
**Fig. A.2** Table BOOK in MS Access

### A.3.2 Creating Relationships

As we know, a well-designed database contains multiple interrelated tables to ensure referential integrity. To work with multiple tables, one has to define relationship between the tables. Establishing a relationship between two tables requires that each table have a field in common with the other.

To create relationships between two or more tables of a database, follow these steps.

1. Click **Tools** menu and then click **Relationships**. A Relationships window appears.
2. Right-click anywhere in the Relationships window. A pop-up menu appears [see Figure A.3(a)].
3. Click **Show Table** option. A Show Table dialog box appears [see Figure A.3(b)].

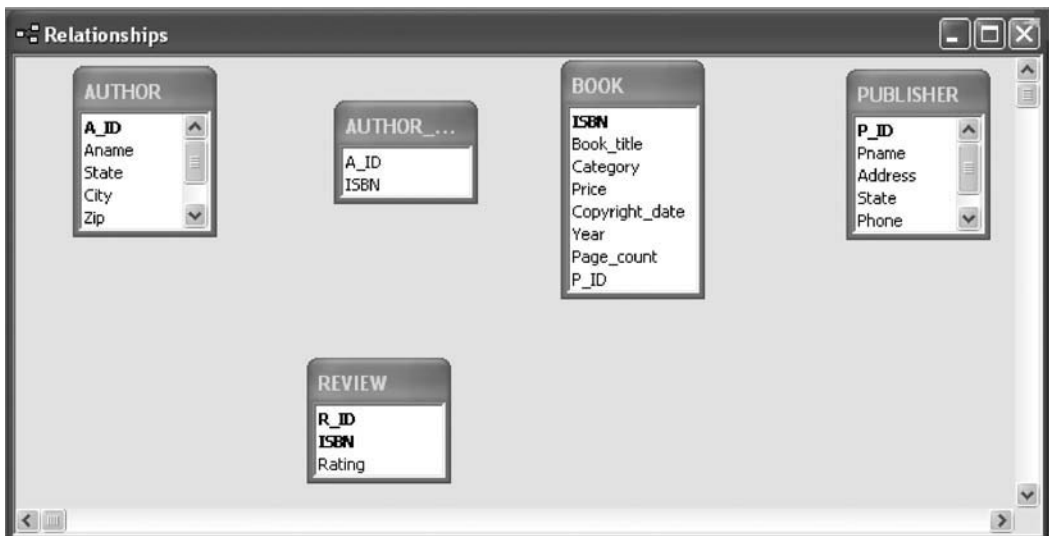


(a) Show Table option

(b) Show Table dialog box

**Fig. A.3** Showing tables in Relationships window

4. Double-click the names of the tables that you want to relate, and then close the **Show Table** dialog box. The tables are added in the Relationships window as shown in Figure A.4.



**Fig. A.4** After adding tables in the Relationships window

5. Drag the field that you want to relate from one table to the related field in the other table. To drag multiple fields, press the **CTRL** key and click each field before dragging them. In most cases, the primary key field (which is displayed in bold text) is dragged from one table to the foreign key in the other table. For example, to create a relationship between the **PUBLISHER** and **BOOK** table, the **P\_ID** attribute in **PUBLISHER** table is dragged to **P\_ID** attribute in the **BOOK** table. An **Edit Relationships** dialog box appears as shown in Figure A.5.

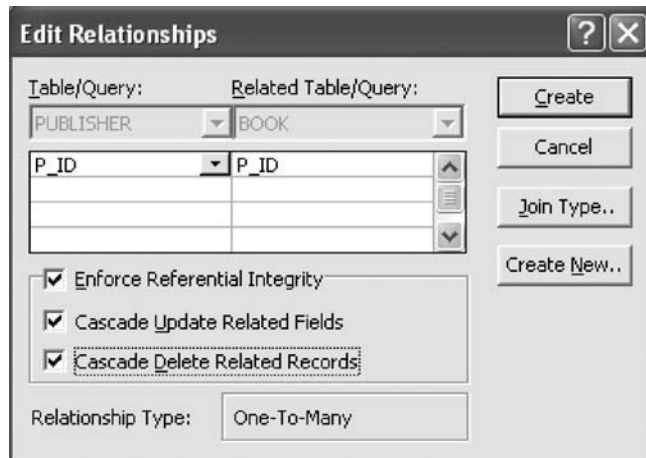


Fig. A.5 *Edit Relationships dialog box*

6. Click the **Enforce Referential Integrity**, **Cascade Update Related Fields**, and **Cascade Delete Related Records** checkboxes. Click **Create** button. A relationship is established between the two tables.
7. Repeat steps 5 and 6 for each pair of tables you want to relate. When you close the Relationships window, Microsoft Access asks if you want to save the layout. Whether you save the layout or not, the relationships you create are saved in the database.

Figure A.6 shows the relationships between all the tables of *Online Book* database.

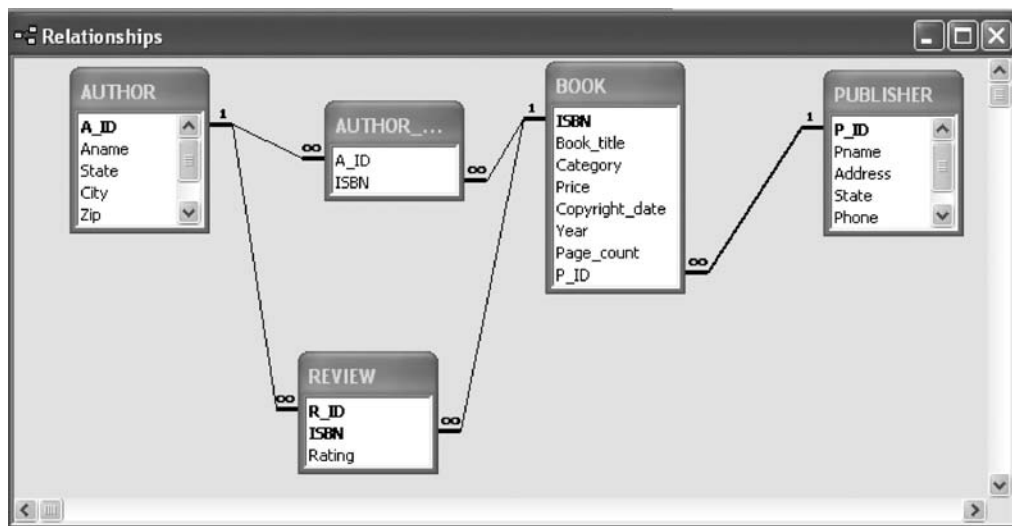


Fig. A.6 *Relationships among various tables of Online Book database*

### A.3.3 Entering and Editing Records

After the structural design of the table is completed, it is required to enter, view, or edit data in the table. Datasheet view offers a quick and easy way to work directly with data. One can add or delete records at



any position within the table. One can also change the look of a particular table in the Datasheet view by changing fonts, hiding columns, and freezing columns so that they remain visible on scroll.

To enter or edit records, open the desired table in the Datasheet view by double-clicking the table from the table list in the Database window.

- ⇒ **Add a new record:** To add a new record to the table in Datasheet view, place the insertion point in the row having asterisk (\*) symbol (marks the new record) at its left and enter data. One can also click the **New Record** button from the Table Datasheet toolbar to add a record.
- ⇒ **Edit an existing record:** To edit record, simply place the cursor in the record that is to be edited and make the required changes.
- ⇒ **Delete a record:** To delete a record, select the entire record, press the **Delete** key. One can also delete record by placing the cursor in any field of the record and select **Delete Record** from the **Edit** menu or **Delete Record** button on the Table Datasheet toolbar.

The BOOK table of *Online Book* database with a sample data is shown in Figure A.7.

|     | ISBN          | Book_title               | Category      | Price | Copyright_date | Year | Page_count | P_ID |
|-----|---------------|--------------------------|---------------|-------|----------------|------|------------|------|
| ▶ + | 001-354-921-1 | Ransack                  | Novel         | 22    | 2005           | 2006 | 200        | P001 |
| +   | 001-987-650-5 | Differential Calculus    | Textbook      | 35    | 2003           | 2003 | 450        | P001 |
| +   | 001-987-760-9 | C++                      | Textbook      | 25    | 2004           | 2005 | 800        | P001 |
| +   | 002-678-880-2 | Call Away                | Novel         | 25    | 2001           | 2002 | 400        | P002 |
| +   | 002-678-980-4 | DBMS                     | Textbook      | 35    | 2004           | 2006 | 860        | P002 |
| +   | 003-456-433-6 | Introduction to German L | Language book | 35    | 2003           | 2004 | 500        | P004 |
| +   | 003-456-533-8 | Learning French Langua   | Language book | 30    | 2005           | 2006 | 500        | P004 |
| +   | 004-765-359-3 | Coordinate Geometry      | Textbook      | 40    | 2006           | 2006 | 650        | P003 |
| +   | 004-765-409-5 | UNIX                     | Textbook      | 26    | 2006           | 2007 | 550        | P003 |
| *   |               |                          |               | 0     | 0              | 0    | 0          |      |

Fig. A.7 Datasheet view of BOOK table with some sample data

## A.4 WORKING WITH QUERIES

Queries are the database objects that enable you to retrieve information from the database. It allows retrieving a subset of data from a single table, from a group of related tables, or from other queries using criteria you define. In addition to fields drawn from the table, a query may also contain calculated fields, which display the results based on the contents of the other fields. The query is saved as a database object and can, therefore, be run at any time. The query will be updated whenever the original tables are updated.



### Using Advanced Options

Filters provide a quick way to create simple queries. Like queries, filters also allow you to work with a subset of records in a database. Using filters, you can limit the display of records in Datasheet or form views. To create a filter in a table, open the table in the Datasheet view, and then follow these steps.

1. Select one instance value (say, *Textbook* in the Category field) you want records to contain in order to be included in the filter's result.
2. Click **Records** menu, then click **Filters** and choose **Filter By Selection** option. This will display the records with the selected value in the particular field. For example, Figure A.8 shows only those records that have *Textbook* value in the Category field.

|                 | ISBN          | Book_title            | Category | Price | Copyright_date | Year | Page_count | P_ID |
|-----------------|---------------|-----------------------|----------|-------|----------------|------|------------|------|
| ▶ +             | 001-987-650-5 | Differential Calculus | Textbook | 35    | 2003           | 2003 | 450        | P001 |
| + 001-987-760-9 |               | C++                   | Textbook | 25    | 2004           | 2005 | 800        | P001 |
| + 002-678-980-4 |               | DBMS                  | Textbook | 70    | 2004           | 2006 | 860        | P002 |
| + 004-765-359-3 |               | Coordinate Geometry   | Textbook | 40    | 2006           | 2006 | 650        | P003 |
| + 004-765-409-5 |               | UNIX                  | Textbook | 26    | 2006           | 2007 | 550        | P003 |
| *               |               |                       |          | 0     | 0              | 0    | 0          |      |

**Fig. A.8** Applying filter to the BOOK table

To remove filter from the table, click the **Records** menu, and then click **Remove Filter/Sort** option.

### A.4.1 QBE in Microsoft Access

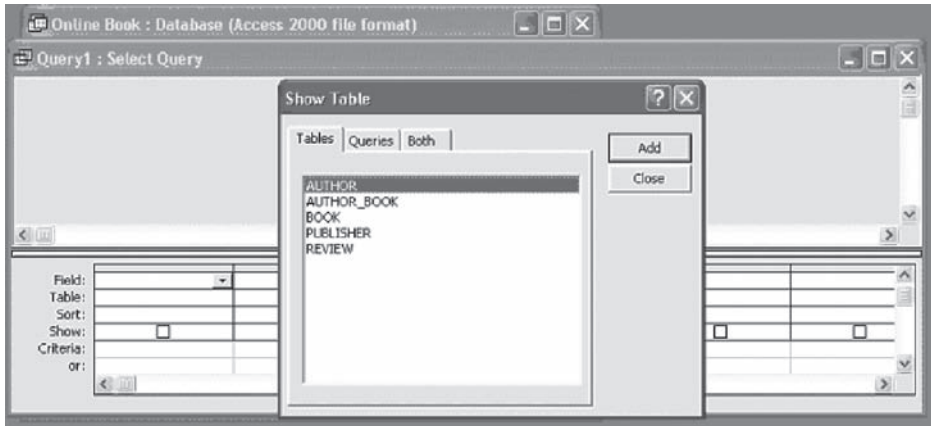
Query By Example (QBE) is a feature that provides a user-friendly interface for running database queries. QBE is included with various database applications. Access also has a QBE interface known as **Query Design View**. A query can be created from scratch in Design view to build one of several specific types of queries. When design view is used to create a new query from the scratch, select query is created by default.

To create a select query in Design view, follow these steps.

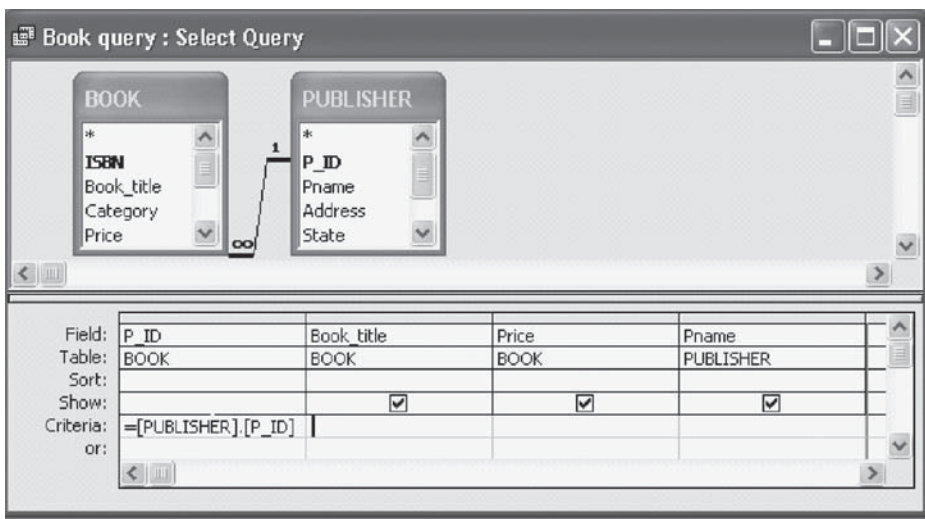
1. Click the **Queries** button in the database window on the **Objects** pane.
2. Double-click the **Create query in Design view**. This will open the query in Design view along with the Show Table dialog box that allows to select the tables and queries on which new query will be based [see Figure A.9(a)].
3. From the Show Table dialog box, select the tables that are to be used to create query and close the dialog box.
4. The Design view of query includes two panes: the top pane contains field lists for each table and query you chose as data source; this pane also shows relationships between the data sources. The lower pane contains a grid with one column for each field that makes up the query. To design the query, select the desired fields from the **Field** drop-down lists in the lower pane, or simply drag and drop the field name from the lists in the upper pane to the **Field** box in the lower pane. Below each field name, there are some rows that explicitly determine the content of the query.

- ⇒ **Table:** This row shows the source of each field. The values in this row are visible by default.
- ⇒ **Total:** This row allows you to specify the operations to be performed on that field. The default selection is Group By. By default, the **Total** option is not visible. To display the **Total** option, click the **View** menu, and then click **Totals** option.
- ⇒ **Sort:** It allows you to sort a particular column.
- ⇒ **Show:** It contains a check for each field that will be displayed as a part of the query results.
- ⇒ **Criteria:** It allows you to specify one or more criterion expressions to determine which records will be included in the query.

Figure A.9(b) shows a select query that displays the detail of a book along with the name of its publisher. You can save this query by any name say, **BOOK query**, so that it can be executed any time.



(a) Design view of query with Show Table dialog box



(b) Query in design view

**Fig. A.9** *Creating query in design view*

Access also allows you to create the queries that can change the data in the tables based on the criteria. These types of queries are known as **action queries**. Access allows creating four types of action queries, namely, **make-table**, **update**, **append**, and **delete** query.

- ⇒ **Make-Table:** A make-table query creates a new table from the result of the query itself. This type of query does not affect the underlying data.
- ⇒ **Update:** An update query replaces the existing data with the new data.
- ⇒ **Append:** An append query adds new records to an existing table from a source query.
- ⇒ **Delete:** A delete query deletes the records that match specified selection criteria for an existing table.

To create make-table query in Design view, follow these steps.

1. Create any query in Design view like retrieve the details of the book having price less than 50.
2. From the **Query** menu, select the **Make-Table Query** option. A dialog box that prompts for the table name appears (see Figure A.10). Type the table name and click **OK** button.

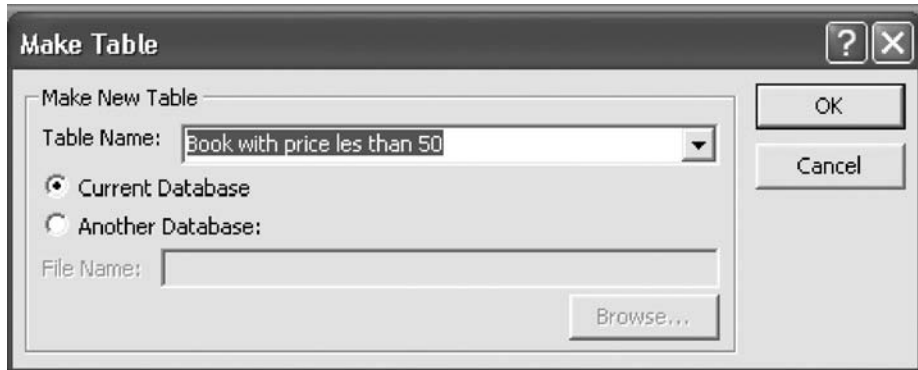


Fig. A.10 Creating a make-table query

To create an update query in Design view, follow these steps.

1. Click the **Queries** button in the Database window on the **Objects** pane.
2. Double-click the **Create query in Design view** (see Figure A.11).
3. From the Show Table dialog box, select the tables that are to be included in the query and close it.
4. From the **Query** menu, select the **Update Query** option. The design of an update query will be displayed.
5. Specify the selection criteria in the **Criteria** box under the desired field name. Also enter the new data in the **Update To** box under the desired field name.

Figure A.11 shows an update query to increase the price of the book by 10. Similarly, you can create, append, and delete query.

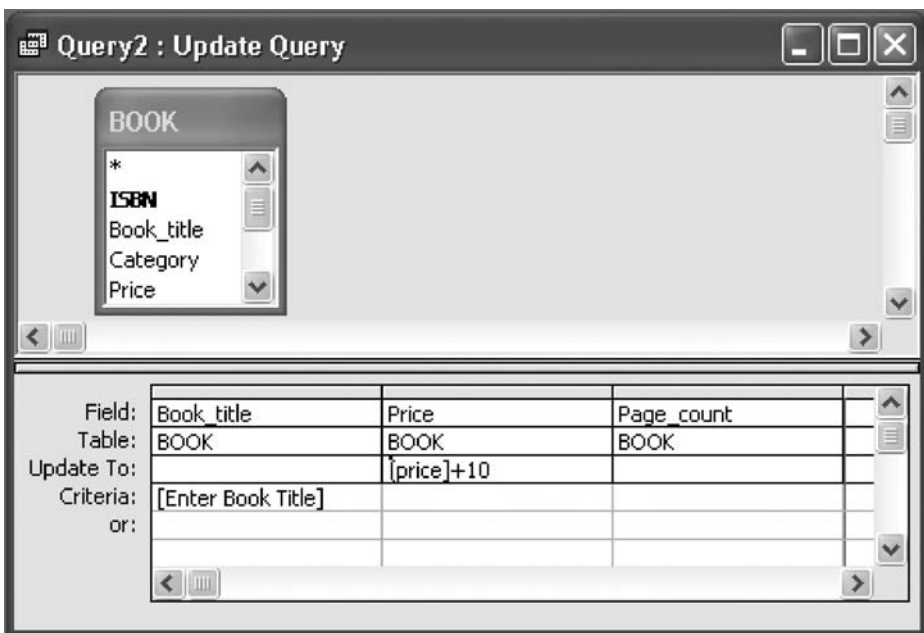
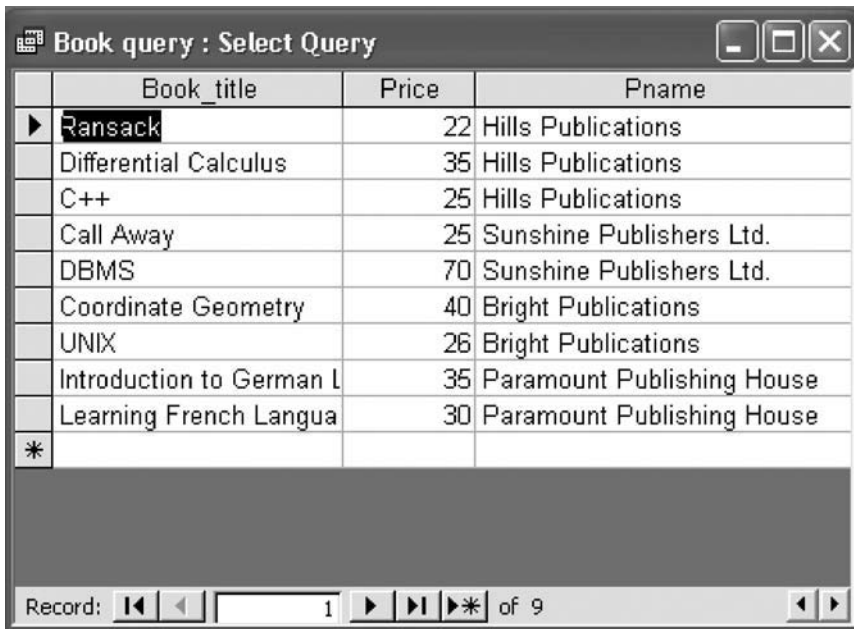


Fig. A.11 Creating an Update Query

### Running a Query

Running a query means viewing the result of the query. Access displays the result of the query in a datasheet view. To run a query, follow these steps.

1. Select **Queries** from the **Objects** pane in the Database window.
2. Double-click the query you want to run say **Book query**. MS Access runs the query and displays the results in Datasheet view as shown in Figure A.12. Alternatively, query can be run by opening it in design view and clicking the **Run** button on Query Design toolbar.



|   | Book_title               | Price | Pname                      |
|---|--------------------------|-------|----------------------------|
| ▶ | Ransack                  | 22    | Hills Publications         |
|   | Differential Calculus    | 35    | Hills Publications         |
|   | C++                      | 25    | Hills Publications         |
|   | Call Away                | 25    | Sunshine Publishers Ltd.   |
|   | DBMS                     | 70    | Sunshine Publishers Ltd.   |
|   | Coordinate Geometry      | 40    | Bright Publications        |
|   | UNIX                     | 26    | Bright Publications        |
|   | Introduction to German L | 35    | Paramount Publishing House |
|   | Learning French Language | 30    | Paramount Publishing House |
| * |                          |       |                            |

Record: 1 of 9

Fig. A.12 Result of Book query



### Using Advanced Options

By default, Access creates an inner join between tables that are added to a query regardless of whether a relationship has been established between the tables. It happens automatically if the two tables contain a common field of the same name and data type, and if this field is the primary key in one of the tables. This autojoin feature can be turned off. To turn off this feature, follow these steps.

1. Click the **Tools** menu on the **Database** toolbar and then click **Options**. The Options dialog box appears.
2. Select **Tables/Queries** tab, remove the check mark from the **Enable Autojoin** option, and click **OK**.

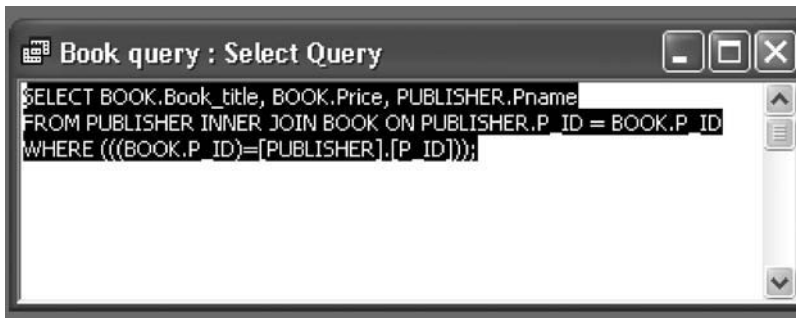
## A.4.2 Using SQL in Microsoft Access

As stated earlier, a query can be created using Wizards or Query Designer. In general, there is no need to deal with SQL. However, each query created in Microsoft Access (created using a wizard or designed in Design view) has an associated SQL statement. You can view the SQL statement behind the existing queries as well as create a new query using SQL. SQL is useful when you want to create a complex query.

### *Viewing and Modifying Existing Queries in SQL*

To view or modify existing queries in SQL, follow these steps.

1. Select the desired query name (say **Book query**) in the query list and click the **Design** button from the Database Window toolbar to open the query in Design view.
2. From the **View** menu, choose **SQL View** to display the query in SQL view. This displays the corresponding SQL statement of the query (see Figure A.13).
3. If you want to make changes, enter the changes into the SQL statement.
4. To return to the Design view, choose **Design View** from the **View** menu.



**Fig. A.13** SQL view of the query

### *Creating SQL Queries*

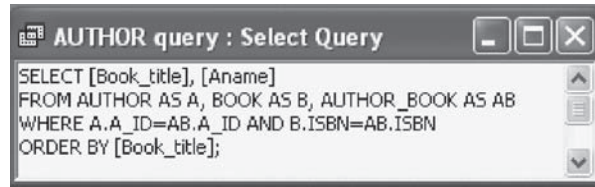
To create an SQL query, follow these steps.

1. Double-click **Create query in Design view** in the Database window. It will display the **Show Table** dialog box.
2. Select the table on which you want to create the SQL query from the table list and click the **Add** button. This will add the table in the Design View. For example, to create a join query on **BOOK**, **AUTHOR**, and **AUTHOR\_BOOK** tables, select these tables.
3. Click the **Close** button to close the **Show Table** dialog box.
4. From the **View** menu, choose **SQL View** to display the query in SQL view.
5. Type the desired query as shown in Figure A.14(a) and save it by clicking the **Save** button on the Query Design toolbar.
6. Type the desired name of the query (say **AUTHOR query**) in the **Query Name** textbox of the **Save As** dialog box and press **OK**.

The result of this query is shown in Figure A.14(b).

## A.5 WORKING WITH FORMS

Forms provide a quick way to insert, update, and delete records in the table. They offer an intuitive, graphical environment easily navigated by anyone. They are generally used to display one record at a time in an onscreen window. A form can be customized by having all or only some of the fields from a table.



(a) SQL query

| Book_title                      | Aname           |
|---------------------------------|-----------------|
| C++                             | James Erin      |
| C++                             | Thomas William  |
| Call Away                       | Jones Martin    |
| Coordinate Geometry             | Suzanne Hedley  |
| DBMS                            | Annie Georgia   |
| DBMS                            | James Erin      |
| Differential Calculus           | John Stephen    |
| Introduction to German Language | Lewis Ian       |
| Learning French Language        | Richard Flaming |
| Ransack                         | Allen Ford      |
| UNIX                            | Annie Georgia   |

Record: 1 of 11

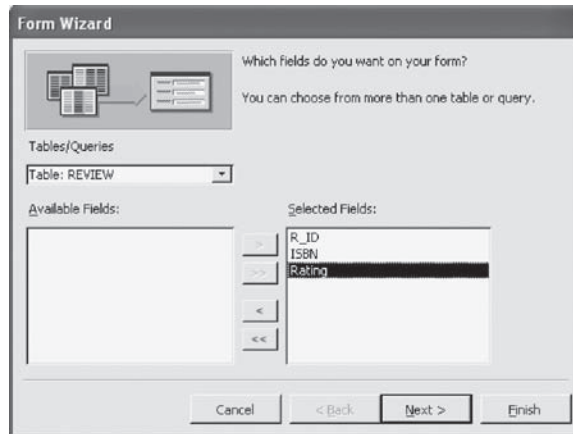
(b) Result of SQL query

**Fig. A.14** *Creating query in SQL view*

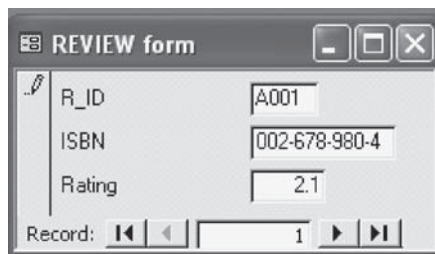
The easiest way to build a form is by using the Form Wizard. The wizard allows the user to choose the layout of records in the form and the background color, and format of the display. The wizard also allows preview of the layout and style options when a form is created. To create a form using the wizard, follow these steps.

1. Choose **Forms** under **Objects** pane and double-click the **Create form by using wizard** option in the Database window. This will initiate the Form Wizard.
2. From the **Tables/Queries** drop-down list, select the desired table or query (say **Rating Query**). Then, select the fields that are to be included in the form by highlighting each one in the **Available Fields** list and clicking the single right arrow (>) button to move the field to the **Selected Fields** list [see Figure A.15(a)]. To move all of the fields to Selected Fields list, click the double right arrow (>>) button. If you make a mistake and would like to remove a field or all of the fields from the Selected Fields list, click the left arrow (<) or left double arrow (<<) button. After the proper fields have been selected, click the **Next >** button to move on to the next screen.
3. On the second screen, select the layout of the form.
  - a. **Columnar:** A single record is displayed at one time with labels and form fields listed side-by-side in columns.
  - b. **Justified:** A single record is displayed with labels and form fields are listed across the screen.
  - c. **Tabular:** Multiple records are listed on the page at a time with fields in columns and records in rows.
  - d. **Datasheet:** Multiple records are displayed in Datasheet view.

4. Select a visual style for the form from the next set of options.
5. On the final screen, name the form in the space provided (say **REVIEW** form). Select **Open the form to view or enter information** to open the form in Form View or **Modify the form's design** to open it in Design view.
6. Click **Finish** to create the form. The form will be displayed as shown in Figure A.15(b). Now you can enter the data in the table through the form.



(a) Form Wizard's first screen



(b) REVIEW form

**Fig. A.15** *Creating form using Form Wizard*

## A.6 WORKING WITH REPORTS

An Access report organizes data in a format ideally suited for printing. Reports can be made by combining data, images, charts, and even audio/video elements. You can add headers, footers, and page numbers, group the information, and change the background colors, among other things. A report provides total control of the presentation of the data. One can add colors, put in summaries and calculations, add in appropriate titles, only show certain fields, and perform many other formatting details.

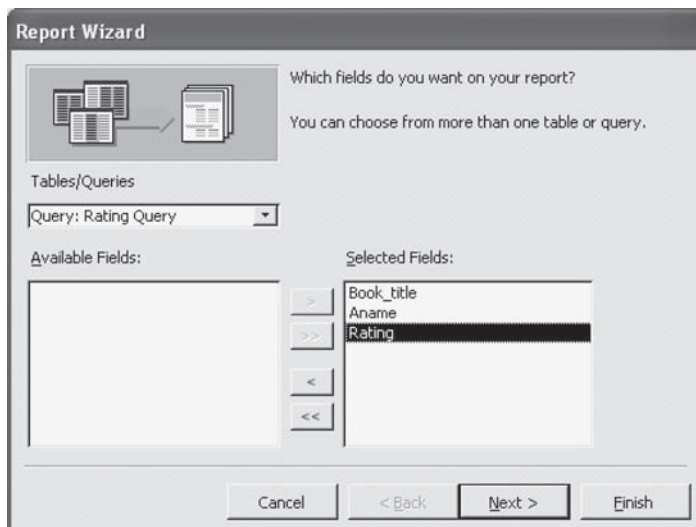
An Access report can be created from the scratch in Design view. However, it is better to use report Wizard to create the basic structure of a report. This is the quickest way of creating a report. The Report Wizard asks you a series of questions to help you design the data exactly as you need. Once



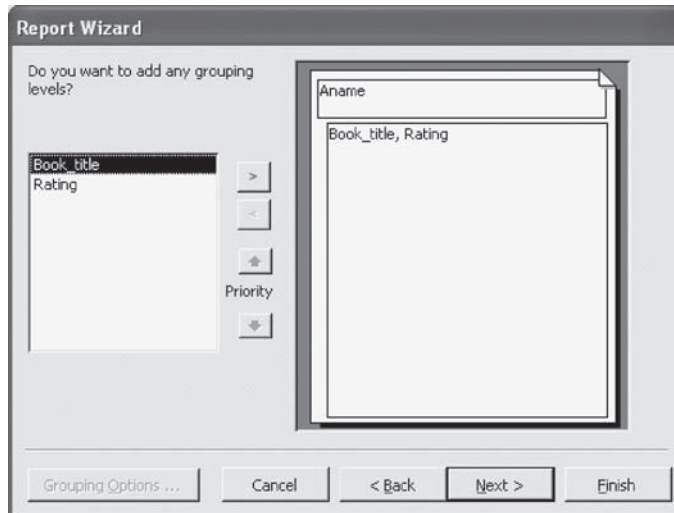
the basic structure of a report is created, it can be opened in Design view to make detailed changes to its contents and appearance.

To create a report using the wizard, follow these steps.

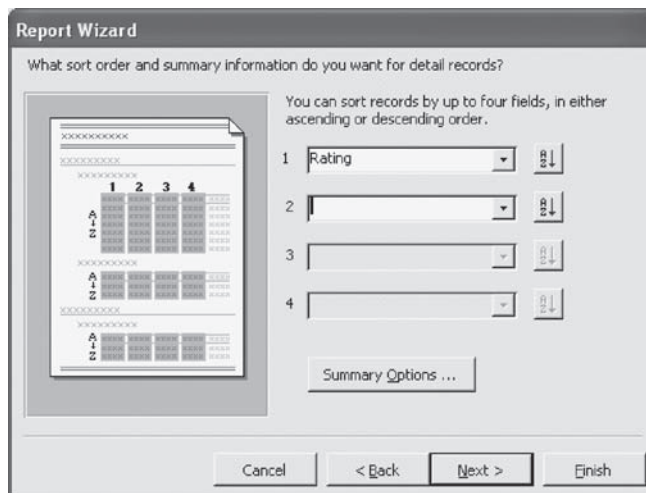
1. Choose **Reports** under **Objects** pane and double-click the **Create report by using wizard** option in the Database window. This will initiate the Report Wizard.
2. From the **Tables/Queries** drop-down list, select the desired table or query (say **Rating Query**). Then, select the fields that will be included on the form by highlighting each one in the **Available Fields** list and clicking the single right arrow (>) button to move the field to the **Selected Fields** list [see Figure A.16(a)]. After the proper fields have been selected, click the **Next >** button to move on to the next screen.
3. Select the fields from the list that the records should be grouped by as shown in Figure A.16(b). Use the Priority buttons to change the order of the grouped fields if more than one field is selected. Then click **Next >** button to move on to the next screen.
4. If the records are to be sorted, identify a sort order. Select the first field that records should be sorted by and click the **Sort** button to choose from ascending or descending order as shown in Figure A.16(c). Click **Next >** button to move on to the next screen.
5. Choose a **Layout** and an **Orientation** for the report.
6. Select a color and graphics style for the report.
7. On the final screen, name the report (say **Rating report**) and select to open it in either print preview or design view mode. Click the **Finish** button to create the report. The report will be displayed as shown in Figure A.16(d).



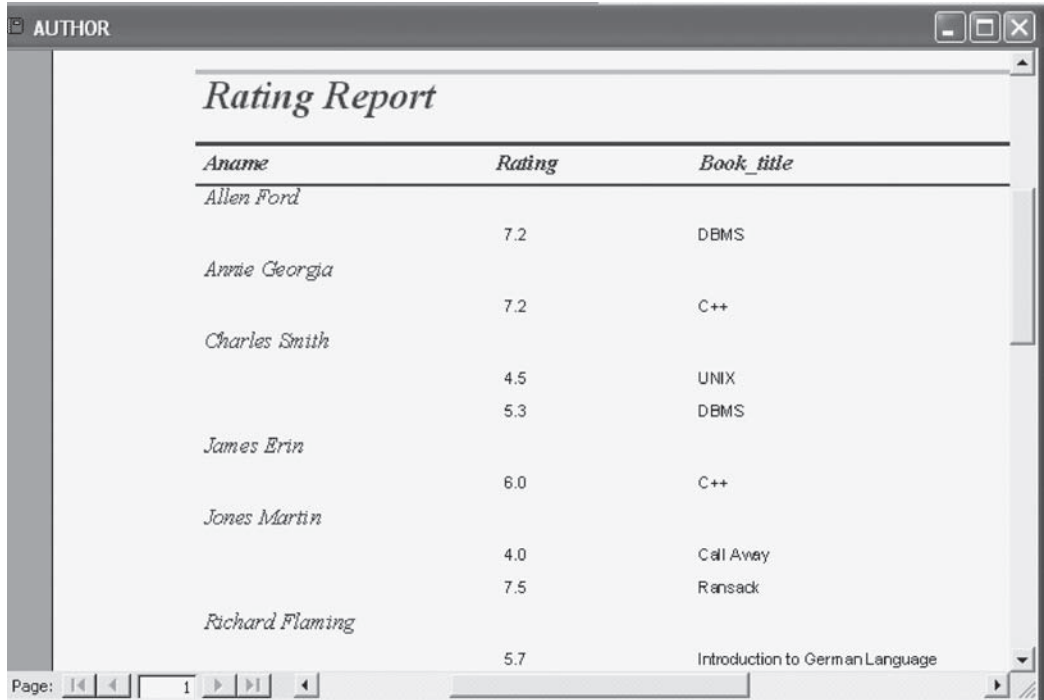
(a) Report Wizard selecting fields to be displayed in the report



(b) Report Wizard selecting a column by which the data to be viewed



(c) Report Wizard selecting a sort order



**AUTHOR**

### Rating Report

| <i>Aname</i>           | <i>Rating</i> | <i>Book_title</i>               |
|------------------------|---------------|---------------------------------|
| <i>Allen Ford</i>      | 7.2           | DBMS                            |
| <i>Annie Georgia</i>   | 7.2           | C++                             |
| <i>Charles Smith</i>   | 4.5           | UNIX                            |
| <i>James Erin</i>      | 5.3           | DBMS                            |
| <i>Jones Martin</i>    | 6.0           | C++                             |
|                        | 4.0           | Call Away                       |
|                        | 7.5           | Ransack                         |
| <i>Richard Flaming</i> | 5.7           | Introduction to German Language |

Page: 1

(d) Displaying the report

**Fig. A.16** *Creating report using Report Wizard*

# Appendix B

## SQL Extensions for Data Analysis

Unlike SQL:92 which provides limited aggregation functionality, SQL:1999 standard defines a rich set of aggregate functions. Most of these functions are supported by Oracle and IBM DB2, and other databases will support these functions in the near future. SQL:1999 provides some extensions to basic **GROUP BY** clause using **ROLLUP** and **CUBE** clauses. It also provides a set of new analytic functions that provide a means by which pivot reports and OLAP queries can be computed easily in nonprocedural SQL.

Prior to the introduction of analytic functions, complex reports could be produced in SQL by complex self-joins and nested queries, which were difficult to write and inefficient to execute. SQL analytic functions and extensions not only make queries easier to code but also make them faster than could be achieved with pure SQL or PL/SQL. This appendix discusses each of them in detail. It also introduces the concept of windowing in SQL:1999.

### B.1 EXTENSIONS TO GROUP BY CLAUSE

SQL:1999 extends the **GROUP BY** clause to provide better support for rollup and cross-tab queries. SQL:1999 supports **ROLLUP** and **CUBE** constructs that are the generalizations of the **GROUP BY** clause. Both constructs are similar in functionality in a way that both of them return additional data in the result of the query when added to the **GROUP BY** clause. The only difference between the two clauses is that **CUBE** returns even more information than **ROLLUP**. To understand the **ROLLUP** and **CUBE** constructs, consider the following SQL query.

```
SELECT bid, tid, SUM(Number) AS Total
FROM SALES
GROUP BY ROLLUP (bid, tid);
```

The **SALES** relation used in this query is shown in Figure 14.2. The result of this query is shown in Figure B.1. The three additional rows with *null* value in the **tid** column provide the total for each value (*B1*, *B2*, and *B3*) in the **bid** column. For example, additional row for *B1* indicates the total books with **bid B1** sold in all time periods. Note that the **tid** column includes a *null* value for these particular rows. Its value cannot be calculated as all three subgroups (from the **tid** column) are already represented. The last row shows that the bookshop has sold 413 books in total.

| bid  | tid  | Total |
|------|------|-------|
| B1   | 1    | 59    |
| B1   | 2    | 47    |
| B1   | 3    | 32    |
| B1   | NULL | 138   |
| B2   | 1    | 46    |
| B2   | 2    | 38    |
| B2   | 3    | 41    |
| B2   | NULL | 125   |
| B3   | 1    | 45    |
| B3   | 2    | 43    |
| B3   | 3    | 62    |
| B3   | NULL | 150   |
| NULL | NULL | 413   |

**Fig. B.1** Result of **GROUP BY ROLLUP** on **SALES**

In addition to attributes from the fact table (such as `bid`, `tid`, and `lid`), attributes from the dimension tables can be also specified in the rollup and cross-tab queries. For example, if the user wants to see the cross-tab on the category of the books and year instead of `bid` and `tid`, following SQL command can be given.

```
SELECT Category, Year, SUM(Number) AS Total
FROM BOOK B, SALES S, TIME T
WHERE B.bid=S.bid AND T.tid=S.tid
GROUP BY ROLLUP (Category, Year);
```

The result of the SQL query is given in Figure B.2. The result of this query provides the information that the bookshop has sold 138 textbooks, 125 language books, and 150 novels. The last row shows that bookshop has sold a total of 413 books.

| Category      | Year | Total |
|---------------|------|-------|
| Textbook      | 2006 | 59    |
| Textbook      | 2007 | 79    |
| Textbook      | NULL | 138   |
| Language Book | 2006 | 46    |
| Language Book | 2007 | 79    |
| Language Book | NULL | 125   |
| Novel         | 2006 | 45    |
| Novel         | 2007 | 105   |
| Novel         | NULL | 150   |
| NULL          | NULL | 413   |

**Fig. B.2** Result of *GROUP BY ROLLUP* on SALES by Category and Year

If we replace the **ROLLUP** clause with the **CUBE** clause in our example SQL query, some more information is provided. For example, consider the following SQL query.

```
SELECT Category, Year, SUM(Number) AS
Total
FROM BOOK B, SALES S, TIME T
WHERE B.bid = S.bid AND T.tid=S.tid
GROUP BY CUBE (Category, Year);
```

The result of this query is shown in Figure B.3. Note that two more rows have been added to the query result—one for each different value in the Year Column. Unlike the **ROLLUP** clause, the **CUBE** clause gives the summarized information for each subgroup. A *null* value in the Category column indicates that all the three categories (*Textbook*, *Language Book*, and *Novel*) are included in each subgroup summary. The **CUBE** clause provides extra information that the bookshop has more sales in the year 2007 than in 2006. This information is not provided by the **ROLLUP** clause.

| Category      | Year | Total |
|---------------|------|-------|
| Textbook      | 2006 | 59    |
| Textbook      | 2007 | 79    |
| Textbook      | NULL | 138   |
| Language Book | 2006 | 46    |
| Language Book | 2007 | 79    |
| Language Book | NULL | 125   |
| Novel         | 2006 | 45    |
| Novel         | 2007 | 105   |
| Novel         | NULL | 150   |
| NULL          | 2006 | 150   |
| NULL          | 2007 | 263   |
| NULL          | NULL | 413   |

**Fig. B.3** Result of *GROUP BY CUBE* on SALES

### Grouping Function

As discussed earlier, SQL:1999 uses *null* to indicate both the *unknown value* as well as *all*. However, *null* values returned by the **ROLLUP** and **CUBE** are not always the stored *null*—instead, a *null* may indicate that the row contains a subtotal. Thus, a problem arises if the query results contain both the stored *null* and *null* created by a **ROLLUP** or **CUBE**. **GROUPING** function is used to distinguish the two *null* values. The function **GROUPING** is an aggregate function that returns 1 when it encounters a *null* value created by a **CUBE** or **ROLLUP** operation. Any other type of value, including a stored *null*, returns 0.

For example, consider the following SQL query.

```
SELECT Category, Year, SUM(Number) AS Total
GROUPING(Category) AS c_flag,
GROUPING(Year) AS y_flag
FROM BOOK B, SALES S, TIME T
WHERE B.bid=S.bid AND T.tid=S.tid
GROUP BY ROLLUP (Category, Year);
```

The result of this query is given in Figure B.4. The result contains two extra columns called *c\_flag* and *y\_flag*. In each tuple, these columns will contain 1 if the corresponding *null* value is generated by ROLLUP or CUBE, otherwise 0 in all other cases. A row with flag values '0 0' indicates that it is a detailed row (row without the summary values). A row with flag values '0 1' indicates that it is a first level subtotal row. The row with flag values '1 1' indicates that it is a grand total row.

| Category      | Year | Total | c_flag | y_flag |
|---------------|------|-------|--------|--------|
| Textbook      | 2006 | 59    | 0      | 0      |
| Textbook      | 2007 | 79    | 0      | 0      |
| Textbook      | NULL | 138   | 0      | 1      |
| Language Book | 2006 | 46    | 0      | 0      |
| Language Book | 2007 | 79    | 0      | 0      |
| Language Book | NULL | 125   | 0      | 1      |
| Novel         | 2006 | 45    | 0      | 0      |
| Novel         | 2007 | 105   | 0      | 0      |
| Novel         | NULL | 150   | 0      | 1      |
| NULL          | NULL | 413   | 1      | 1      |

**Fig. B.4** Result of *GROUPING* function

### **DECODE** Function

The readability of the query result can be further improved by replacing the *null* value by a value of our choice. This can be done by using the *GROUPING* function with the *DECODE* function. For example, consider this SQL query.

```
SELECT Category, Country,
SUM(Number) AS Total
DECODE (GROUPING(Category),1,'All
Categories', Category),
DECODE (GROUPING(Country),1,'All
Countries', Country),
FROM BOOK B, SALES S, LOCATION L
WHERE B.bid=S.bid AND L.lid=S.lid
GROUP BY CUBE (Category, Country);
```

In this query, the *DECODE* function returns the value 'All Categories' if the value of *Category* is a *null* generated by CUBE operator and returns the actual values otherwise. Similarly, it returns the value 'All Countries' if the value of *Country* is a *null* generated by CUBE operator and returns the actual values otherwise. The result is shown in Figure B.5.

| Category       | Country       | Total |
|----------------|---------------|-------|
| Textbook       | India         | 85    |
| Textbook       | USA           | 53    |
| Textbook       | All Countries | 138   |
| Language Book  | India         | 84    |
| Language Book  | USA           | 41    |
| Language Book  | All Countries | 125   |
| Novel          | India         | 116   |
| Novel          | USA           | 34    |
| Novel          | All Countries | 150   |
| All Categories | India         | 285   |
| All Categories | USA           | 128   |
| All Categories | All Countries | 413   |

**Fig. B.5** Result of *DECODE* function

## B.2 NEW ANALYTIC FUNCTIONS

SQL:1999 provides 33 new analytic functions. It supports standard deviation and variance functions (**STDDEV** and **VARIANCE**) which are applied on single attribute. Some of the database systems also support other aggregate functions such as median and mode. SQL:1999 also supports **binary aggregate functions** which can be applied on the pairs of attributes. These functions include correlations and covariance and regression curves (**CORR**, **COVAR\_POP**, **REGR\_COUNT**, etc.). Definitions of these functions may be found in any standard textbook on statistics. Thus, we will not discuss these functions in detail.

Analytic functions enable you to compute aggregate values for a specific group of rows. They differ from aggregate functions in a way that they return multiple rows for each group. Analytic functions are invoked using the **OVER( )** clause which has three components.

- ⇒ **PARTITION BY** clause, which divides the query result set into groups. If **PARTITION BY** clause is missing, the entire result set is treated as a single partition.
- ⇒ **ORDER BY** clause, which sorts the query result set or the partition in ascending or descending order as specified.
- ⇒ **RANGE** or **ROWS** clause, by which the function can be made to include values or rows around the current row in its calculations. These clauses are used in window queries.

This section discusses only the ranking and window aggregate functions provided by SQL:1999.

### *Ranking Functions*

A ranking function computes the rank of a row compared to other rows in the result set. SQL:1999 provides four types of ranking functions.

- ⇒ **RANK** and **DENSE\_RANK**
- ⇒ **CUME\_DIST** and **PERCENT\_RANK**
- ⇒ **ROW\_NUMBER**
- ⇒ **NTILE**

The **BOOK** table shown in Figure B.6 is taken for illustration of these functions.

| Book_title                      | Category      | Price |
|---------------------------------|---------------|-------|
| C++                             | Textbook      | 40    |
| Ransack                         | Novel         | 22    |
| Learning French Language        | Language Book | 32    |
| Differential Calculus           | Textbook      | 32    |
| Call Away                       | Novel         | 22    |
| DBMS                            | Textbook      | 40    |
| Introduction to German Language | Language Book | 22    |
| Coordinate Geometry             | Textbook      | 35    |
| UNIX                            | Textbook      | 26    |

**Fig. B.6** *BOOK table*

**RANK and DENSE\_RANK Functions** The RANK function returns a ranking value for each row in the query result set. Ranking is done in conjunction with an ORDER BY clause. For example, to find the rank of each book in the order of their price (the book with the highest price will get rank 1), following SQL query can be used.

```
SELECT Book_title, Price, RANK() OVER(ORDER BY Price DESC) AS
b_rank FROM BOOK;
```

This query displays the book title, price, and rank assigned to each book according to its price. The output of this query is shown in Figure B.7(a). If there are multiple rows for a given value of the ordering attribute, the RANK function assigns the same rank to all the rows that have the same value of the ordering attribute. For example, if there are two books with the highest price, then both of them will get rank 1. The next row in the sequence will get rank 3 and not 2. Thus, RANK function leaves gaps in the ranking sequence in case of ties as shown in the output.

| Book_title                      | Price | b_rank |
|---------------------------------|-------|--------|
| C++                             | 40    | 1      |
| Ransack                         | 22    | 7      |
| Learning French Language        | 32    | 4      |
| Differential Calculus           | 32    | 4      |
| Call Away                       | 22    | 7      |
| DBMS                            | 40    | 1      |
| Introduction to German Language | 22    | 7      |
| Coordinate Geometry             | 35    | 3      |
| UNIX                            | 26    | 6      |

(a) RANK without partitioning

| Category      | Book_title                      | Price | b_rank |               |
|---------------|---------------------------------|-------|--------|---------------|
| Language Book | Learning French Language        | 32    | 1      | } Partition 1 |
| Language Book | Introduction to German Language | 22    | 2      |               |
| Novel         | Ransack                         | 22    | 1      | } Partition 2 |
| Novel         | Call Away                       | 22    | 1      |               |
| Textbook      | C++                             | 40    | 1      | } Partition 3 |
| Textbook      | DBMS                            | 40    | 1      |               |
| Textbook      | Coordinate Geometry             | 35    | 3      |               |
| Textbook      | Differential Calculus           | 32    | 4      |               |
| Textbook      | UNIX                            | 26    | 5      |               |

(b) RANK with partitioning

**Fig. B.7** Output of query containing RANK function

Note that the rows in the output may not be sorted by rank, thus, an extra ORDER BY clause is required to get them sorted in the order of b\_rank as shown in this SQL query.



```
SELECT Book_title, Price, RANK() OVER(ORDER BY Price DESC) AS
b_rank FROM BOOK ORDER BY b_rank;
```

The `DENSE_RANK` function works similarly as `RANK` function, but it does not leave gaps in the ranking sequence. Thus, two books with highest price will get rank 1 and the next row will get rank 2 instead of 3. To understand the `DENSE_RANK` function, consider the following SQL query.

```
SELECT Book_title, Price, DENSE_RANK() OVER(ORDER BY Price DESC)
AS b_rank FROM BOOK ORDER BY b_rank;
```

Ranking can also be done within the partitions of the result set. The partitions of the query result set can be created using the `PARTITION BY` clause. The `PARTITION BY` clause divides the query result set into partitions within which the `RANK` function is applied. The analytic functions such as `RANK`, `DENSE_RANK`, `NTILE`, `ROW_NUMBER`, etc., are applied to each partition independently. Thus, they are reset for each partition. For example, consider this SQL query.

```
SELECT Category, Book_title, Price, DENSE_RANK() OVER (PARTITION
BY Category ORDER BY Price DESC) AS b_rank FROM BOOK ORDER BY
Category, b_rank;
```

This SQL query first partitions the tuples on the basis of the attribute `Category`. It then assigns a rank to each book within the partition according to its price. The book with the highest price within the partition will get a rank 1. The outer `ORDER BY` clause orders the result tuples first by `Category` and within each category by `b_rank`. The output of this query is shown in Figure B.7(b). Note that within each partition, the rank starts with 1.



If the `PARTITION BY` clause is missing, then ranks are computed over the entire query result set.

If *null* values are present in the table, one *null* value is assumed to be equal to another *null* value while calculating the rank. SQL:1999 allows the user to specify where to put *null* values in the query result set by using the `NULLS FIRST` or `NULLS LAST` clause. For example, consider this SQL query.

```
SELECT Book_title, Price, RANK() OVER(ORDER BY(Price) DESC NULLS
LAST) AS ranking FROM BOOK;
```

**PERCENT\_RANK and CUME\_DIST Functions** The `PERCENT_RANK` function computes the rank of a row within each partition as a fraction. The `PERCENT_RANK` of a row can be calculated as

$$(\text{rank of a row in its partition} - 1) / (\text{number of rows being evaluated} - 1)$$

The number of rows being evaluated can be an entire query result set or a partition. This function will return *null* if there is only one tuple in the partition. For example, consider this SQL query.

```
SELECT Category, Book_title, Price, PERCENT_RANK() OVER (PARTITION
BY Category ORDER BY Price DESC) AS p_rank FROM BOOK ORDER BY
Category, p_rank;
```

It calculates the percent rank of the book's price for each book within the category. The output of this query is given in Figure B.8(a).

| Category      | Book_title                      | Price | p_rank |               |
|---------------|---------------------------------|-------|--------|---------------|
| Language Book | Learning French Language        | 32    | 0      | } Partition 1 |
| Language Book | Introduction to German Language | 22    | 1      |               |
| Novel         | Ransack                         | 22    | 0      | } Partition 2 |
| Novel         | Call Away                       | 22    | 0      |               |
| Textbook      | C++                             | 40    | 0      | } Partition 3 |
| Textbook      | DBMS                            | 40    | 0      |               |
| Textbook      | Coordinate Geometry             | 35    | 0.5    |               |
| Textbook      | Differential Calculus           | 32    | 0.75   |               |
| Textbook      | UNIX                            | 26    | 1      |               |

(a) PERCENT\_RANK function

| Book_title            | Price | c_rank |
|-----------------------|-------|--------|
| UNIX                  | 26    | 0.2    |
| Differential Calculus | 32    | 0.4    |
| Coordinate Geometry   | 35    | 0.6    |
| C++                   | 40    | 1      |
| DBMS                  | 40    | 1      |

(b) CUME\_DIST function

**Fig. B.8** Output of query containing PERCENT\_RANK and CUME\_DIST function

The CUME\_DIST (cumulative distribution) uses the row count (number of rows in the partition with ordering values less than or equal to the ordering value of the tuple) instead of its rank value. The CUME\_DIST can be calculated as

$$(\text{Row count}) / (\text{number of rows being evaluated})$$

For example, consider this SQL query.

```
SELECT Book_title, Price, CUME_DIST() OVER (PARTITION BY Category
ORDER BY Price) AS c_rank FROM BOOK WHERE Category='Textbook'
ORDER BY Price;
```

This query calculates the price percentile of each book in the textbook category. The output of this query is given in Figure B.8(b).

**ROW\_NUMBER Function** The ROW\_NUMBER function sorts the tuples and assigns a unique number to each tuple (sequentially, starting with 1) according to its position in the sort order within the partition or in the entire query result set. For example, consider this SQL query.

```
SELECT Category, Book_title, Price, ROW_NUMBER() OVER (PARTITION
BY Category ORDER BY Price) AS r_no FROM BOOK ORDER BY Category,
Price;
```

This query assigns a number to each tuple in the order of the price within each category. The book with the lowest price will get a number 1, and so on. The output of this query is given in Figure B.9(a).

| Category | Book_title             | Price | r_no |               |
|----------|------------------------|-------|------|---------------|
| Language | Introduction to German | 22    | 1    | } Partition 1 |
| Book     | Language               |       |      |               |
| Language | Learning French        | 32    | 2    | } Partition 2 |
| Book     | Language               |       |      |               |
| Novel    | Ransack                | 22    | 1    | } Partition 3 |
| Novel    | Call Away              | 22    | 2    |               |
| Textbook | UNIX                   | 26    | 1    | } Partition 3 |
| Textbook | Differential Calculus  | 32    | 2    |               |
| Textbook | Coordinate Geometry    | 35    | 3    |               |
| Textbook | C++                    | 40    | 4    |               |
| Textbook | UNIX                   | 40    | 5    |               |

(a) ROW\_NUMBER function

| Book_title                      | Price | bucket_no |
|---------------------------------|-------|-----------|
| Introduction to German Language | 22    | 1         |
| Ransack                         | 22    | 1         |
| Call Away                       | 22    | 1         |
| UNIX                            | 26    | 2         |
| Learning French Language        | 32    | 2         |
| Differential Calculus           | 32    | 2         |
| Coordinate Geometry             | 35    | 3         |
| C++                             | 40    | 3         |
| DBMS                            | 40    | 3         |

(b) NTILE function

**Fig. B.9** Output of the query containing ROW\_NUMBER and NTILE function

**NTILE Function** The NTILE function divides an ordered partition into a specified number of groups called **buckets** and assigns a bucket number (in which the row is placed) to each row in the partition, with bucket number starting with 1. NTILE function is generally useful for constructing histograms based on percentiles.

The buckets are calculated in such a way that each bucket has exactly same number of rows. For example, if a partition contains 100 rows and NTILE function needs to divide these rows in four buckets, then each bucket will contain 25 rows. These buckets are referred to as **equiheight** buckets.

If the number of rows in the partition does not divide evenly into the number of buckets, then the number of rows in each bucket can differ by at most 1. The extra rows are distributed one for each bucket starting from the lowest bucket number. For example, if there are 103 rows in a partition and have to be divided into five buckets, the first 21 rows will be in the first bucket, the next 21 in the second bucket, the next 21 in the third bucket, the next 20 in the fourth bucket, and the final 20 in the fifth bucket.

For example, consider this SQL query that divides the tuples into three buckets. The output of this query is given in Figure B.9(b).

```
SELECT Book_title, Price, NTILE(3) OVER (ORDER BY Price) AS
bucket_no FROM BOOK ORDER BY Price;
```

### B.3 TOP-N QUERIES

The task of retrieving the top or bottom N rows from a table is known as **top-N query**. The simplest and the most common way to solve this problem is by using the Oracle pseudocolumn **ROWNUM**. For each row returned by a query, the **ROWNUM** pseudocolumn returns a number indicating the order in which the Oracle selects the row from a table. The first row selected has a **ROWNUM** of 1, the second has 2, and so on. For example, to retrieve the book title and price of top 5 costly books, the following SQL query can be given.

```
SELECT ROWNUM, Book_title, Price
FROM (SELECT Book_title, Price FROM BOOK
ORDER BY Price DESC NULLS LAST) WHERE ROWNUM<=5;
```

The output of this query is given in Figure B.10. The nested subquery first selects the title and price of all the books in descending order of price with *null* values in the last. The outer query then selects the row number, title, and price of first 5 costly books.

| ROWNUM | Book_title                  | Price |
|--------|-----------------------------|-------|
| 1      | C++                         | 40    |
| 2      | DBMS                        | 40    |
| 3      | Coordinate Geometry         | 35    |
| 4      | Learning French<br>Language | 32    |
| 5      | Differential Calculus       | 32    |

**Fig. B.10** Output of **ROWNUM**

Similarly, to retrieve the bottom three least expensive books, following SQL query can be given.

```
SELECT ROWNUM, Book_title, Price
FROM (SELECT Book_title, Price FROM BOOK
ORDER BY Price ASC NULLS FIRST) WHERE ROWNUM<=3;
```

Another way to compute top-N queries is to use SQL analytic functions such as **ROW\_NUMBER()**, **RANK()**, and **DENSE\_RANK()**. For example, to retrieve the book title and price of top 5 costly books, the following SQL query can be given.

```
SELECT * FROM (SELECT Book_title, Price, ROW_NUMBER()
OVER (ORDER BY Price DESC NULLS LAST) AS top5 FROM BOOK)
WHERE top5<=5;
```

The output of this query is given in Figure B.11. The nested subquery first orders the title and price of all the books in descending order of price and assigns a unique number to each retrieved row. The outer query then selects the top 5 rows from the result set of the nested query.

| Book_title                  | Price | top5 |
|-----------------------------|-------|------|
| C++                         | 40    | 1    |
| DBMS                        | 40    | 2    |
| Coordinate Geometry         | 35    | 3    |
| Learning French<br>Language | 32    | 4    |
| Differential Calculus       | 32    | 5    |

**Fig. B.11** *Top-N query with the help of ROW\_NUMBER function*

## B.4 WINDOWING IN SQL:1999

The windowing feature gives us a way to define a ‘sliding’ window of data on which the analytic function will operate, within a group. The window determines the range of rows used to perform the calculations for the ‘current row’. Window sizes can be based on either a physical number of rows or a logical interval such as time. For example, the average sales over the past  $n$  days, say 5 days, can be associated with every SALES tuple (each of which contains the sales of 1 day). This gives us  $n$ -day (5-day) moving average of sales.

The queries involving such type of computations are harder to express in SQL:92. Therefore, SQL:1999 provides a windowing feature to support such queries. To understand the concept of windowing, consider this SQL query.

```
SELECT L.State, T.Month, Number, SUM(Number) OVER (PARTITION BY
L.State ORDER BY T.Month RANGE BETWEEN INTERVAL '1' MONTH PRE-
CEDING AND INTERVAL '1' MONTH FOLLOWING) AS moving_sum
FROM SALES S, TIME T, LOCATION L
WHERE (S.tid=T.tid AND S.lid=L.lid) AND S.bid='B1';
```

In this query, first the FROM and WHERE clause are processed to generate an intermediate relation, say TEMP. Windows are created over this TEMP relation. There are three steps in creating a window. The first step is to define the partitions of the table using the PARTITION BY clause. In our example query, the partitions are created on the L.State column. The second step is to specify the ordering of rows within a partition using the ORDER BY clause. In our example query, the rows in each partition are ordered by T.Month column. The ordering column has to be numeric type, a datetime type, or an interval type.

The third and final step in defining a window is to frame a window. Defining the boundaries of the window associated with each row in terms of ordering of rows within the partitions is known as **framing a window**. A window can be framed by using either the RANGE clause or the ROWS clause. In our example query, the window is defined using the RANGE clause. The ordering in this case has to be specified over a single column of numeric type, datetime type, or an interval type.

The **RANGE** clause defines a window based on the values of some column (for example, **Month**). In our example, the window of a row includes the row itself plus all the rows whose month value is within a month before or after. Thus, a row with a month value September 2007 has a window containing all rows with month equal to August, September, or October 2007.

The **ROWS** clause, on the other hand, defines a window of row by specifying the number of rows before and after the given row. For example, consider this SQL query which creates a window for each row in the partition by including the row itself plus the previous and the following row.

```
SELECT L.State, T.Month, Number, SUM(Number) OVER (PARTITION BY
L.State ORDER BY T.Month ROWS BETWEEN 1 PRECEDING AND 1 FOLLOW-
ING) AS mvg_sum
FROM SALES S, TIME T, LOCATION L
WHERE (S.tid=T.tid AND S.lid=L.lid) AND S.bid='B1';
```

The output of this query is given in Figure B.12. The **PARTITION BY** clause makes the **SUM(Number)** be computed for each state, independent of other partitions. The **SUM(Number)** is reset as the state changes. The **ORDER BY** clause sorts the data within each state by month. The window clause **ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING** accesses one row prior and one row after the current row in a group in order to compute the **SUM(Number)**. For example, the **mvg\_sum** value for **Month=6** for *Naveda* state is 53, which is the sum of 10 (current row), 18 (row preceding), and 25 (row following).

| State       | Month | Number | mvg sum |
|-------------|-------|--------|---------|
| Naveda      | 3     | 18     | 28      |
| Naveda      | 6     | 10     | 53      |
| Naveda      | 12    | 25     | 35      |
| Maharashtra | 3     | 12     | 30      |
| Maharashtra | 6     | 18     | 41      |
| Maharashtra | 12    | 11     | 29      |
| Delhi       | 3     | 10     | 18      |
| Delhi       | 6     | 8      | 34      |
| Delhi       | 12    | 16     | 24      |

**Fig. B.12** Windowing using **ROWS** clause

Other than **ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING**, some other keywords can also be used which are given here.

- ⇒ **ROWS UNBOUNDED PRECEDING**: Creates a window for each row by including all the rows in the partition that are preceding it.
- ⇒ **BETWEEN ROWS UNBOUNDED PRECEDING AND CURRENT**: Creates a window for each row by including the current row and all the rows in the partition that are preceding it.
- ⇒ **ROWS 10 PRECEDING**: Creates a window for each row by including the previous 10 rows.
- ⇒ **ROWS UNBOUNDED FOLLOWING**: Creates a window for each row by including all the rows in the partition that are following it.

- ⇒ **BETWEEN ROWS CURRENT AND UNBOUNDED FOLLOWING:** Creates a window for each row by including the current row and all the rows in the partition that are following it.
- ⇒ **ROWS 10 FOLLOWING:** Creates a window for each row by including the next 10 rows.



**NOTE** *Unlike the RANGE clause, the ordering can be specified on a composite key in the ROWS clause.*

# AppendixC

## Abbreviations, Acronyms, and Symbols

|            |                                                                                     |
|------------|-------------------------------------------------------------------------------------|
| 1NF        | First Normal Form                                                                   |
| 2NF        | Second Normal Form                                                                  |
| 2PC        | Two-Phase Commit                                                                    |
| 3NF        | Third Normal Form                                                                   |
| 3PC        | Three-Phase Commit                                                                  |
| 4NF        | Fourth Normal Form                                                                  |
| 5NF        | Fifth Normal Form                                                                   |
| ACID       | Atomicity, Consistency, Isolation, and Durability                                   |
| ADT        | Abstract Data Type                                                                  |
| AES        | Advanced Encryption System                                                          |
| AFIM       | After-Image                                                                         |
| ANSI/SPARC | American National Standards Institute/Standards Planning and Requirements Committee |
| API        | Application Programming Interface                                                   |
| ARIES      | Algorithm for Recovery and Isolation Exploiting Semantics                           |
| BCNF       | Boyce-Codd Normal Form                                                              |
| BERM       | Binary Entity Relationship Model                                                    |
| BFIM       | Before-Image                                                                        |
| BLOB       | Binary Large Object                                                                 |
| CD-ROM     | Compact Disk-Read Only Memory                                                       |
| CLI        | Call Level Interface                                                                |
| CLOB       | Character Large Object                                                              |
| CODASYL    | Conference on Data Systems Languages                                                |
| CPU        | Central Processing Unit                                                             |
| DAC        | Discretionary Access Control                                                        |
| DBA        | Database Administrator                                                              |
| DBMS       | Database Management System                                                          |
| DDBMS      | Distributed Database Management System                                              |
| DDL        | Data Definition Language                                                            |
| DES        | Data Encryption Standard                                                            |
| DFD        | Data Flow Diagram                                                                   |
| DK/NF      | Domain-Key Normal Form                                                              |



|        |                                                |
|--------|------------------------------------------------|
| DML    | Data Manipulation Language                     |
| DRAM   | Dynamic Random Access Memory                   |
| DRC    | Domain Relational Calculus                     |
| DSS    | Decision Support System                        |
| DVD    | Digital Video Disk/Digital Versatile Disk      |
| DW     | Data Warehouse                                 |
| ECC    | Error Correcting Code                          |
| EER    | Enhanced Entity Relationship                   |
| E-R    | Entity-Relationship                            |
| ERP    | Enterprise Resource Planning                   |
| ETL    | Extraction, Transformation, and Loading        |
| FD     | Functional Dependency                          |
| FIFO   | First-In-First-Out                             |
| GERM   | General Entity Relationship Model              |
| GUI    | Graphical User Interface                       |
| HOLAP  | Hybrid Online Analytical Processing            |
| HTML   | Hypertext Markup Language                      |
| I/O    | Input/Output                                   |
| IDF    | Inverse Document Frequency                     |
| IS     | Intention Shared (lock)                        |
| ISO    | International Organization for Standardization |
| IT     | Information Technology                         |
| IX     | Intention Exclusive (lock)                     |
| JD     | Join Dependency                                |
| JDBC   | Java Database Connectivity                     |
| KDD    | Knowledge Discovery in Databases               |
| LRU    | Least Recently Used                            |
| LSN    | Log Sequence Number                            |
| MAC    | Mandatory Access Control                       |
| MOLAP  | Multidimensional Online Analytical Processing  |
| MRP    | Management Resource Planning                   |
| MRU    | Most Recently Used                             |
| MTTF   | Mean Time to Failure                           |
| MVD    | Multivalued Dependency                         |
| NAS    | Network Attached Storage                       |
| NV-RAM | Non-volatile Random Access Memory              |
| ODBC   | Open Database Connectivity                     |
| ODL    | Object Data Language                           |

|        |                                              |
|--------|----------------------------------------------|
| ODM    | Object Data Model                            |
| ODMG   | Object Database Management Group             |
| OID    | Object Identifier                            |
| OLAP   | Online Analytical Processing                 |
| OLTP   | Online Transaction Processing                |
| OMG    | Object Management Group                      |
| OOA    | Object-Oriented Analysis                     |
| OODB   | Object-Oriented Database                     |
| OODBMS | Object-Oriented Database Management System   |
| OOPL   | Object-Oriented Programming Language         |
| OQL    | Object Query Language                        |
| ORDBMS | Object-Relational Database Management System |
| PJ/NF  | Project-Join/Normal Form                     |
| PSM    | Persistent Stored Modules                    |
| QBE    | Query-By-Example                             |
| QEP    | Query Evaluation Plan                        |
| RAD    | Rapid Application Development                |
| RAID   | Redundant Arrays of Independent Disks        |
| RAM    | Random Access Memory                         |
| RBAC   | Role-Based Access Control                    |
| RDBMS  | Relational Database Management System        |
| ROLAP  | Relational Online Analytical Processing      |
| RPC    | Remote Procedure Call                        |
| S      | Shared (lock)                                |
| SAN    | Storage Area Network                         |
| SCSI   | Small Computer System Interface              |
| SDL    | Storage Definition Language                  |
| SIX    | Shared and Intention Exclusive (lock)        |
| SQL    | Structured Query Language                    |
| SQLJ   | SQL-Java                                     |
| SRAM   | Static Random Access Memory                  |
| TF     | Term Frequency                               |
| TO     | Timestamp Ordering                           |
| TP     | Transaction Processing                       |
| TRC    | Tuple Relational Calculus                    |
| UML    | Unified Modeling Language                    |
| URL    | Uniform Resource Locator                     |
| VDL    | View Definition Language                     |
| WAL    | Write-Ahead Logging                          |
| WFF    | Well-Formed Formula                          |
| WORM   | Write-Once-Read-Many                         |

|                      |                            |
|----------------------|----------------------------|
| X                    | Exclusive (local)          |
| XML                  | Extensible Markup Language |
| $\wedge$             | Conjunction operator       |
| $\vee$               | Disjunction operator       |
| $\neg$               | Negation operator          |
| $\sigma$             | Select operator            |
| $\pi$                | Project operator           |
| $\rho$               | Rename operator            |
| $\cup$               | Union operator             |
| $\cap$               | Intersection operator      |
| $-$                  | Set difference operator    |
| $\times$             | Cartesian product operator |
| $\bowtie$            | Join operator              |
| $\bowtie_{\text{L}}$ | Left outer join operator   |
| $\bowtie_{\text{R}}$ | Right outer join operator  |
| $\bowtie_{\text{F}}$ | Full outer join operator   |
| $\div$               | Division operator          |
| $\subseteq$          | is a subset of             |
| $\exists$            | Existential quantifier     |
| $\forall$            | Universal quantifier       |
| $\rightarrow$        | Functionally determines    |
| $\twoheadrightarrow$ | Multi-determines           |
| $\Rightarrow$        | Implies                    |

# Appendix D

## E. F. Codd's Rules

The original concept for the relational model was proposed by Dr. E. F. Codd in 1970. Later Dr. Codd clarified his model by defining twelve rules (*Codd's Rules*) that a database management system must meet in order to be considered a relational database. These rules were very useful for evaluating a relational system. The Codd's twelve rules are discussed below preceded by the rule zero.

**Rule zero:** This rule states that all subsequent rules are based on the notion that in order for a database to be considered relational, it must use its relational facilities exclusively to manage the database.

**Rule 1:** The *information rule*: This rule simply requires all information in the database to be represented in one and only one way, namely, by values in column positions within rows of tables. There is no hierarchical ranking of tables and the relationships among tables are logical only.

**Rule 2:** *Guaranteed access rule*: This rule states that each item of data in an RDBMS is guaranteed to be logically accessible by specifying the name of the containing table, the name of the containing column, and the primary key value of the containing row.

**Rule 3:** *Systematic treatment of null value rule*: According to this rule, a field should be allowed to remain empty (except for the primary key). This involves the support of a null value, which is distinct from an empty string or a number with a value of zero. Note that null means no value has been entered, that is, the value is unknown. For example, if an item in a store is not marked, its price is unknown (null) but it is not free.

**Rule 4:** *Dynamic on-line catalog based on the relational model rule*: A relational database must provide access to its structure through the same tools that are used to access the data. This is usually accomplished by storing the structure definition within a special system catalog. In addition to user data, a relational database contains data about itself. There are two types of tables in RDBMS—user tables that contain the 'working' data and system tables that contain metadata. The collection of system tables is also referred to as the *system catalog* or *data dictionary*.

**Rule 5:** *Comprehensive data sub-language rule*: In a relational system, there must be at least one language whose statements are expressible, as character strings and whose ability to support all the following is comprehensible:

- ⇒ Datade finition
- ⇒ Viewde finition
- ⇒ Datama nipulation
- ⇒ Integrityc onstraints
- ⇒ Transactionbounda ries

Nowadays, all commercial relational databases use forms of the standard SQL as their supported comprehensive language.

**Rule 6:** *View updating rule:* This rule specifies that all theoretically possible view updates should be updatable by the system.

**Rule 7:** *High-level insert, update, and delete:* This rule states that insert, update, and delete operations should be supported for any retrievable set rather than just for a single row in a single table.

**Rule 8:** *Physical data independence:* This implies that the user is isolated from the physical method of storing and retrieving information from the database. An application that accesses data in a relational database contains only a basic definition of the data (data type and length); it does not need to know how the data is physically stored or accessed. The changes can be made to the underlying architecture (hardware, disk storage methods, etc.) without affecting the access of the user.

**Rule 9:** *Logical data independence rule:* This rule specifies that the relationships among tables can change without impairing the function of applications and ad hoc queries. The database schema or structure of tables and relationships (logical) can change without having to recreate the database or the applications that use it.

**Rule 10:** *Integrity independence rule:* In order to be considered relational, data integrity must be an internal function of the DBMS, not the application program. Data integrity means the consistency and accuracy of the data in the database. A minimum of the following two integrity constraints must be supported:

- ⇒ **Entity Integrity:** It implies that no component of a primary key is allowed to have a NULL value.
- ⇒ **Referential Integrity:** It implies that if a foreign key is defined in one table, any value in it must exist as a primary key in another table.

**Rule 11:** *Distribution independence rule:* Distribution independence specifies that the user should be unaware of whether or not the database is distributed (whether parts of the database exist in multiple locations).

**Rule 12:** *Non-subversion rule:* This rule implies that there should be no way to modify the database structure other than through the multiple row database language (like SQL).

# Glossary

- Access matrix:** a table containing the authorization information, which the database system consults each time an access request is issued in the system—the columns of access matrix represent objects and rows represent subjects
- Access time:** the period of time that elapses between a request for information from disk or memory, and the information arriving at the requesting device
- ACID properties:** the acronym derived from the first letter of the terms atomicity, consistency, isolation, and durability—the four desirable properties of the transaction
- Activated atabase:** a database having triggers associated with it
- After-image(A FIM):** a copy of the data item after modification
- Aggregate function:** a function that takes a collection of values as input, process them, and returns a single value as the result
- Aggregation:** the process through which one can treat the relationships as higher-level entities
- ANSI/SPARC architecture:** an architecture in which the overall database description can be defined at three levels namely, *internal*, *conceptual*, and *external* levels and thus, named three-level DBMS architecture.
- Apparent key:** the set of attributes in a multilevel relation that would have served as the primary key in the regular relation
- Application layer:** the middle layer of three-tier client/server architecture that executes the business logic of the application
- Algorithm for Recovery and Isolation Exploiting Semantics (ARIES):** an example of recovery algorithm which is widely used in the database systems
- Artificial key:** an attribute introduced to serve as a key when a primary key is not available or inapplicable—also known as a surrogate key
- Atomic literals:** correspond to the values of basic data types of the object model including long, short and unsigned integers, floating point numbers, Boolean values, character, string, and enumeration type values
- Atomicity:** a desirable property of the transaction which implies that either all of the operations that make up a transaction should execute or none of them should occur
- Attribute:** the property of an entity that characterizes and describes it
- Audittr ail:** a database log that is mainly used for database security
- Authentication:** the process of verifying the identity of a user
- Authorization tree:** a tree in which the database users become the nodes with authorizer as the root node—an edge from one node to another node exists if rights are passed from one user to another user represented by these two nodes
- Authorization:** the granting of privileges to users, which enables them to access certain portion of the database
- Authorizer:** the person who is responsible for granting privileges to database users

- Authority:** a type of web page that contains actual information on a topic; however, it may not contain links to other related pages
- Averager esponseti me:** the average time for a transaction to be completed after it has been submitted
- B<sup>+</sup>-tree:** a variation of B-tree—most widely used structure for multilevel indexing
- Backflushing:** the process of upgrading data with the cleaned data
- Before-image(B FIM):** a copy of the data item prior to modification
- Blocking problem:** a situation where the participating sites cannot decide whether to commit or abort the transaction until the coordinator recovers
- Boyce-Codd normal form (BCNF):** a relation schema R is in **BCNF** if and only if every non-trivial FD has a candidate key as its determinant
- B-tree:** a tree data structure with some additional properties and constraints—used for multilevel indexing
- Business metadata:** the metadata repository that includes the relevant business rules of the organization
- Cache directory:** a directory maintained by the cache manager to keep track of all the data items present in the buffers
- Cache memory:** a kind of primary storage which is directly accessed by the CPU, and is basically used to speed up the execution
- Candidatek ey:** a minimal superkey that does not contain any superfluous attribute in it
- Cardinality:** the number of tuples in a relation
- Cartesian product:** a binary operation that returns a third relation containing all possible combinations of the tuples from the two operand relations
- Cascadeless schedule:** the schedule that avoids cascading rollbacks
- Cascading rollback:** a situation in which failure of single transaction leads to a series of transaction rollbacks
- Centralized database systems:** a database system in which the database system, application programs, and user-interface all are executed on a single system and dummy terminals are connected to it
- Certified version:** a version of a data item written by any committed transaction
- Checkpoint:** a record periodically written into the log—the write operations of the transactions that have committed prior to the checkpoint need not be redone in case of a failure as all updates are already recorded in the database on disk
- Checkpointing:** a process in which DBMS periodically force-write all the modified buffer pages during normal execution to reduce the time taken to recover from a crash
- Clock replacement policy:** a variant of LRU policy in which an additional reference bit is associated with each frame, which is set on as soon as `pin_count` of that frame becomes 0
- Clustering:** the process of grouping the records together so that the degree of association is strong between the records of the same group and weak between the records of different groups
- Coarsegr anularity:** large data item sizes such as an entire relation or a database
- Compatibility matrix:** a matrix representing the requested lock modes and their compatibility with the existing lock mode
- Compositek ey:** a primary key formed by the combination of two or more attributes
- Concurrency control techniques:** the techniques required to control the interaction among concurrent transactions—ensure the isolation property of a transaction

- Concurrent execution:** the execution of two or more transactions in an interleaved manner to improve the system performance
- Confidence:** the probability that a customer will buy the items in the set Y if he or she purchases the items in set X
- Constraints:** are required to maintain the integrity of the data, which ensures that the data in database is consistent, correct, and valid
- Coordinators site:** the site at which the transaction is initiated
- Covert channel:** the flow of information from higher classification level to a lower classification level through indirect means
- Cross-tabulation:** a two-dimensional table in which values for one attribute (say A) form the row headers, values for another attribute (say B) form the column headers. Each cell can be identified by  $(a_i, b_j)$  where  $a_i$  is a value for A and  $b_j$  is a value for B—also known as a cross-tab or pivot table
- Cursor:** a mechanism that provides a way to retrieve multiple tuples from a relation and then process each tuple individually in a host program
- Data:** a set of isolated and unrelated raw facts with an implicit meaning
- Data abstraction:** allows the database system to provide an abstract view of the data to its users without giving the physical storage and implementation details
- Data cleansing:** the task of correcting and pre-processing data
- Data cube:** the generalization of a two-dimensional cross-tab to  $n$  dimensions—also called a multi-dimensional cube or OLAP cube or hypercube
- Data definition language (DDL):** commands that are used for defining relation schemas, deleting relations, creating indexes, and modifying relation schemas
- Data dictionary:** a special type of table containing meta-data (data about data)
- Data guard:** a standby database feature provided by Oracle that helps minimizing the effect of failure
- Data independence:** the ability to change the schema at one level of the database system without having to change the schema at the other levels
- Data manipulation language (DML):** commands that are used for retrieving and manipulating data stored in the database
- Data mining:** extraction of hidden and predictive information from large databases
- Data transparency:** hiding all physical details of the database from the users in distributed DBMS
- Data warehouse (DW):** a repository of suitable operational data gathered from multiple sources, stored under a unified schema, at a single site
- Database:** a collection of related data from which users can efficiently retrieve the desired information
- Database2:** IBM's database product on MVS platform
- Database administrator (DBA):** a person who has central control over both data and application programs
- Database audit:** a review of system log to examine all the operations applied to the database during certain period of time
- Database backup:** the entire contents of the database and the log copied periodically onto a cheap storage medium such as magnetic tapes
- Database management system (DBMS):** an integrated set of programs used to create and maintain a database



- Databasemode l:** an abstract model that describes how the data is represented and used
- Databases chema:** the overall design or description of the database
- Database server:** the third layer of three-tier client/server architecture that processes the query requested by the application layer and sends the results to it
- DB2E xpress-C:** free version of DB2
- Deadlock:** a situation that occurs when all the transactions in a set of two or more transactions are in a simultaneous wait state and each of them is waiting for the release of a data item held by one of the other waiting transaction in the set
- Decision phase:** the second phase of two-phase commit protocol in which the coordinator site decides whether the transaction can be committed or it has to be aborted—used in distributed systems
- Decisiontr ee:** a graphical representation of the classification rules—also known as classification tree
- Decision-support system (DSS):** the systems that aim to extract high-level information stored in large databases, and to use that information in making a variety of decisions which are important for the organization
- Decryptionalg orithm:** an algorithm that recovers the original data from the encrypted data
- Deferred update:** a technique in which the transaction is not allowed to update the database on disk until the transaction enters into the partially committed state
- Degree:** the number of attributes in a relation
- Denormalization:** the process of taking a schema from higher normal form to lower normal form to speed up the database access
- Designm ethodology:** the systematic process of designing a database
- Digital signatures:** an application of public key encryption technique used to verify the authenticity of electronic document
- Dimension attributes:** the attributes that define the dimensions on which the measure attributes and their summaries are viewed
- Dimension tables:** the relations which store additional information about the dimensions—also known as lookup tables
- Dirty bit:** a Boolean variable that keeps track of whether the current page in the frame has been modified or not, since it was brought into the frame from disk
- Dirtypa geta ble:** a table containing a record for each dirty page in the buffer
- Discretionary access control (DAC):** an access control approach enforced in a database system by granting and revoking privileges to and from the users
- Disk controller:** interfaces the disk drive to the computer system—handles the transfer of data between memory and disk drive
- Distributed computing:** the processing of a single task by several machines in a coordinated manner
- Distributed Database System:** the database system in which data is physically stored across several computers located at different sites and are connected with one another through various communication media—each site is managed by a DBMS that is capable of running independent of other sites
- Document Type Definition (DTD):** a set of rules that defines a set of elements and attributes that can appear within a document
- Domain integrity:** a type of data integrity that can be specified for each attribute by defining its specific range or domain

**Domain relational calculus:** a form of relational calculus which is based on domain variables—unlike tuple relational calculus, it ranges over the values from the domain of an attribute, rather than values for an entire tuple

**Drilldown:** an operation that converts data with a coarser-granularity to the finer-granularity

**Durability:** a desirable property of a transaction which implies that once a transaction is completed successfully, the changes made by the transaction persist in the database, even if the system fails

**Element:** a pair of start and a matching end tag and all the contents enclosed between them

**Embedded SQL:** an SQL code embedded within an application program written in a host language

**Encryption algorithm:** an algorithm that produces encrypted version of the plaintext as the output

**Entity integrity:** a type of data integrity that assigns restriction on the tuples of a relation and ascertains the accuracy and consistency of the database

**Entity set:** a collection of all instances of a particular entity type in the database at any point of time

**Entity/relationship (E-R) diagram:** a specialized graphical tool that demonstrates the interrelationships among various entities of a database

**Entity:** a distinguishable object that has an independent existence in the real world

**Equi-join:** a type of join query in which tuples are concatenated on the basis of equality condition

**Equivalence rule:** a rule which states that expressions of two forms are equivalent if an expression of one form can be replaced by expression of the other form, and vice versa

**Evaluation engine:** the DBMS component that is responsible for actual execution of the query to get the desired result

**Exclusive(X)lock:** a mode of lock that allows the transaction to read as well as write a data item

**Extents:** contains all the persistent objects of a particular class—an extent is given a name and is declared using the keyword `extent`

**eXtensible Markup Language:** a markup language that can represent database data and many other kinds of structured data

**External sorting:** a sorting technique which requires the use of external memory such as disks or tapes during sorting

**Extract, transform, and load (ETL process):** the entire process of getting data into the data warehouse

**Extraneous attribute:** an attribute of a functional dependency which can be eliminated without changing the closure of the set of functional dependencies

**Facttables:** the relations containing multidimensional data

**Fail-stop assumption:** the assumption that the hardware and software errors bring only the system to a halt and have no effect on the contents of the non-volatile storage media

**Falsepositive:** the retrieved documents that do not contain all the words in the query

**Fifth normal form (5NF):** a relation schema  $R$  is in **fifth normal form (5NF)** with respect to a set  $F$  of functional, multivalued, and join dependencies if, for every non-trivial join dependency  $*$   $(R_1, R_2, \dots, R_n)$  in  $F^+$ , every  $R_1$  is a superkey of  $R$

**File-processing system:** a system in which the data is stored in the form of files and a number of application programs are written by programmers to add, modify, delete, and retrieve data to and from appropriate files

**File scans:** the search algorithms that scan the records of file to locate and retrieve the records satisfying the selection condition

**Finegr anularity:** small data item sizes such as either a tuple or an attribute

**First normal form (1NF):** a relation schema  $R$  is said to be in **first normal form (1NF)** if and only if the domains of all attributes of  $R$  contain atomic (or indivisible) values only

**For, Let, Where, Order By and Return (FLWOR):** form of a typical query in XQuery

**Force:** an approach in which all the updated cache pages are immediately written (force-written) to disk when the transaction commits

**Fourth normal form (4NF):** a relation schema  $R$  is in Fourth normal form (4NF) with respect to a set  $F$  of functional and multivalued dependencies if, for every non-trivial multivalued dependency  $X \twoheadrightarrow Y$  in  $F^+$ , where  $X \subseteq R$  and  $Y \subseteq R$ ,  $X$  is a superkey of  $R$ .

**Fragmentation transparency:** hiding the details of data fragmentation from the users—in distributed DBMS

**Fragmentation:** a technique in which a relation is divided into several fragments (or smaller relations), and each fragment can be stored at sites where they are more often accessed—used in distributed DBMS

**Free bit:** a bit associated with each memory buffer that indicates whether the buffer is free (available for allocation) or not

**Fullr eplication:** the process of distributing an entire relation at all sites

**Functional dependency:** the special form of integrity constraints that generalizes the concept of keys—play a key role in developing a good database schema

**Fuzzy checkpointing:** a technique in which a [checkpoint] record (known as fuzzy checkpoint) is written in the log before writing the modified buffer blocks to the disk

**Fuzzy dump:** a technique in which the transactions are allowed to continue while the dump is in progress

**Global transactions:** the transaction that accesses and updates data from several sites or the site other than the site at which the transaction is initiated

**Granularity:** the size of the smallest unit, which can be independently locked—a database may apply locks at the database level, the relation level, the tuple level, or on the value of a single attribute within a tuple

**Hashing:** enables very fast access to records on the basis of certain search conditions

**Heterogeneous distributed database system:** a distributed database system in which different sites may have different schemas and run different DBMS software—also known as multidatabase system

**Heuristics:** the rules that usually help in reducing the cost of execution of a query

**HITS algorithm:** a link-based search algorithm that retrieves highly relevant pages in response to a given query

**Homogeneous distributed database system:** a distributed database system in which all sites share a common global schema and run the identical DBMS software

**Horizontal fragmentation:** a fragmentation technique which breaks a relation  $R$  by assigning each tuple of  $R$  to one or more fragments

**Host language:** a general purpose programming language used to develop a software program that uses embedded SQL or that calls stored procedures

**Hots pots:** the frequently accessed data items that remain in the buffer for long period of time

**Hub:** a type of web page that itself may not contain the actual information on a topic, but stores links to many related pages that contain actual information

**Hybrid OLAP (HOLAP) systems:** the systems that include the best of ROLAP and MOLAP—these systems use main memory to hold some summaries, and relational tables to hold base data and other summaries

**Hyper-Text Markup Language (HTML):** a markup language that is suitable and widely used for formatting and structuring contents of Web pages

**Immediate update:** a technique in which the transaction is allowed to update the database in its active state

**Indexes can:** the search algorithm that involves the use of indexes

**Index:** a structure used to provide quick response to queries on a relation—most often used with large relations

**Information retrieval (IR):** the process of extracting information from unstructured textual data

**Instance:** a specific occurrence of an entity type

**INSTEAD OF trigger:** trigger on view to specify the actions that need to be taken on the base table as a consequence of DML operation on the view

**Integrity constraints:** a set of rules that governs the entity types and their relationships, and thereby, ensures the integrity of database

**Intention (I) lock:** a type of lock mode in which a transaction intends to explicitly lock a lower level of the tree

**Interface:** specifies only the abstract behaviour of an object type

**Intersection:** a set operation that returns a third relation that contains tuples common to both the operand relations

**Isolation:** a desirable property of a transaction which implies that each transaction should run independent of other concurrently running transactions

**Itemset:** a set of items a customer tends to buy together

**Java database connectivity (JDBC):** provides a standard API that is used to access databases through Java, regardless of the DBMS

**Joinquery:** a query that combines the tuples from two or more relations

**Join:** a cartesian product followed by select and project operations

**Key:** an attribute (or combination of attributes) in a relation that uniquely identifies each tuple of that relation

**Large itemsets:** the itemsets that have support above the minimum pre-specified support—also known as frequent itemsets

**Least recently used (LRU):** a replacement policy that chooses the least recently referenced page for replacement, as and when required

**Left-deep join orders:** the join orders in which the right operand of each join is one of the base relations defined in DBMS catalog

**Lexicon:** an index that not only contains the address of inverted lists on disks, but may also contain some additional information

**Local transaction:** the transaction that accesses and updates data only at the site where it is initiated

- Location transparency:** hiding the details of physical location of the data from the users—in distributed DBMS
- Lock compatibility:** a mechanism that determines whether locks can be acquired on a data item by multiple transactions at the same time
- Lock conversion:** a technique in which a transaction can change the mode of locks from one mode to another on a data item on which it already holds a lock
- Lock manager:** a subsystem of database system that implements the locking or unlocking of the data items
- Lockpo int:** a point in the schedule at which the transaction successfully acquires its last lock
- Lock timeouts:** the approach that prevents the transactions from waiting for a long time for other transaction to release a lock
- Lock:** a variable associated with each data item that indicates whether a read or write operation can be applied to the data item
- Lockinggr anularity:** the size of the data item that the lock protects
- Locking:** the process of acquiring the lock by modifying the value of the lock
- Log sequence number (LSN):** a unique id assigned to each log record in ARIES
- Logtail:** the most recent portion of the log kept in the main memory
- Log:** a sequence of log records that contains essential data for each transaction which has updated the database
- Majority locking:** a locking technique in which majority of the copies must be locked in case of both read and write lock request for a data item—applied in distributed systems
- Mandatory access control (MAC):** an access control approach which enforces multilevel security by classifying subjects and objects of database system in different security classes—each object is assigned a security class and each subject has some clearance
- Materialized evaluation:** a type of query evaluation in which each operation in the expression is evaluated one by one in an appropriate order, and the result of each operation is created in a temporary relation which becomes input for the subsequent operations
- Measureattr ibutes:** the attributes that measure some value and can be aggregated upon
- Mixeddfr agmentation:** a combination of horizontal and vertical fragmentation
- Most recently used (MRU):** a replacement policy that chooses the most recently accessed page for replacement, as and when required.
- Multidimensional data:** the data in a warehouse having measure attributes and dimension attributes
- Multidimensional OLAP (MOLAP) systems:** the traditional OLAP systems that used multidimensional arrays in memory to store data cubes
- Multivalued dependency:** a result of first normal form which disallows an attribute to have multiple values
- Multiversion technique:** a concurrency control technique which keeps the old version (or value) of each data item in the system when the data item is updated
- Multiwayjoin:** the join in which more than two relations are involved
- NAS device:** a server that allows file-sharing—used for high performance storage solution at low cost
- Natural join:** a join obtained after removing one of the two identical attributes from the result of equijoin

- No-force:** an approach in which the updated cache pages are not necessarily written to disk immediately when the transaction commits
- Non-volatile storage:** a type of storage that does not lose its contents when power supply is switched off
- Normal forms:** the rules or constraints that a relation must satisfy to avoid data redundancy and data update anomalies
- Normalization:** the process of decomposing a relation schema into a set of smaller relation schemas so that they conform to certain rules (normal forms)
- No-steal:** an approach in which an updated cache page cannot be written to the disk before the transaction commits
- NULL:** a value that represents a missing or inapplicable value in an attribute of a relation
- Object:** a basic element of an object model which consists of two components, namely, *state (attributes and relationships)* and *behavior (operations)*
- Object definition language (ODL):** a language designed to represent the structure of an object-oriented database—serves the same purpose as DDL
- Object identifier (OID):** a unique system generated identifier assigned to an object that uniquely identifies the object over its entire lifetime
- Observable external writes:** the write operations that are performed on a terminal or a printer
- Off-line storage:** Removable media, such as optical discs and magnetic tapes that can be removed from the drive
- Online analytical processing (OLAP):** a category of software technology that enables analysts, managers, and executives to analyze the complex data derived from the data warehouse
- Online transaction processing (OLTP) system:** the system that supports only predefined operations like insertions, deletions, updates, and retrieval of data
- Open database connectivity (ODBC):** a standard which provides an application programming interface (API) that the application programs can use to establish a connection with the database
- Operational data:** the data that documents everyday operations of an organization
- Optimistic technique:** a concurrency control technique that assumes that the transactions do not directly update the data items until they finish their execution
- Object query language (OQL):** a language designed for the ODMG object model—resembles SQL, also supports object-oriented concepts of the object model, such as object identity, inheritance, relationship sets, operations, etc.
- Outer join:** a join that selects all the tuples satisfying the join condition along with the tuples for which no rows from the other relation satisfy the join condition
- Parallel processing:** the simultaneous use of computer resources such as CPUs and disks to perform a specific task
- Partial replication:** the process of replicating only some fragments of a relation
- Participating sites:** the sites where subtransactions are executed
- Path expression:** consists of sequence of element names (or attribute names) in the XML document with each pair of names separated by a separator
- Persistent pointer:** a pointer pointing to an object in the database and remains valid even after the termination of program
- Pipelined evaluation:** a type of query evaluation in which operations of the query form a queue and results are passed from one operation to another as they are calculated

**Pivoting:** the technique of changing from one dimensional orientation of data cube to another—in this technique, the data cube can be thought of as rotating to show a different orientation of the axes

**Postingsfile:** the collection of inverted lists

**Precedencegraph:** a directed graph consisting of a set of nodes and a set of directed edges

**Precision:** defined as the percentage of retrieved documents that are relevant to the query

**Predicate locking:** a locking technique in which all the tuples (whether existing or new tuples that are to be inserted) that satisfy an arbitrary predicate are locked

**Presentation layer:** the first layer of three-tier client/server architecture that provides an interface to handle user input and display the needed information

**Primarykey:** a candidate key that is chosen to uniquely identify each tuple in a relation

**Primarysite:** the site at which the primary copy of a data item resides

**Primary storage:** provides very fast access to data and is used to keep the data that is currently being accessed by the CPU—volatile in nature and offers limited storage capacity due to its high cost

**Private key:** a key used for decryption in public key encryption technique

**Public key encryption:** the type of encryption which uses two different keys for encryption and decryption—based on mathematical functions rather than operations on bit patterns

**Publickey:** a key used for encryption in public key encryption technique

**Queryblock:** a basic unit that can be translated into relational algebra expression and optimized

**Query-evaluation plan:** a query tree in which each operation is annotated with the instructions that specify the algorithm or the index to be used to evaluate that operation—also known as query-execution plan or simply plan

**Query optimization:** a process in which multiple query-execution plans for satisfying a query are examined and a most efficient query plan is identified for execution

**Query optimizer:** the part of the DBMS that calculates the most efficient way to perform a given query

**Query processing:** a process which includes translation of high-level queries into low-level expressions that can be used at the physical level of the file system, query optimization, and actual execution of the query to get the result

**Query-rewrite system:** allows users to define how SQL queries on a given table or view should be modified into alternate queries

**Query tree:** a tree data structure in which the input relations are represented as the leaf nodes and the relational algebra operations as the internal nodes—also known as operator tree or expression tree

**Read-onlytransaction:** a transaction that does not modify the database but only retrieve the data

**Recall:** defined as the percentage of relevant documents in the database that are retrieved in response to the query

**Recoverymanager:** a component of DBMS which is responsible for performing the recovery operations

**Redo operation:** an operation that reapplies the changes of a committed transaction and restores the database to the consistent state it would be at the end of the transaction

**Redundant arrays of independent disks (RAID):** a technique to improve the performance and reliability of secondary storage—the basic idea behind RAID is to have a large array of small independent disks

**Referencedrelation:** a relation that is being referenced by another relation

**Referencingrelation:** a relation that references another relation

- Referential integrity:** a type of data integrity that ensures that the value of foreign key in the referencing relation must exist in the primary key attribute of the referenced relation, or that the value of foreign key is null
- Regular Language for XML Next Generation (RELAX NG):** an XML schema definition language designed by OASIS
- Relation instance:** a two-dimensional table with a time varying set of tuples—also known as a relation
- Relation schema:** consists of a relation name  $R$  and a set of attributes (or fields)  $A_1, A_2, \dots, A_n$ —it is represented by  $R(A_1, A_2, \dots, A_n)$  and is used to describe a relation  $R$ .
- Relational algebra:** a procedural query language that comprises a set of operations that take one or two relations as input, and result into a new relation as an output
- Relational calculus:** an alternative to relational algebra—a non-procedural or declarative query language as it specifies what is to be retrieved rather than how to retrieve it
- Relational data base:** a collection of relations or two-dimensional tables having distinct names
- Relational OLAP (ROLAP) systems:** the OLAP systems that work directly with relational databases
- Remote backup:** a copy of the database made generally by writing each block of stable storage over a computer network
- Replication transparency:** hiding the details of data replication from the users—in distributed DBMS
- Replication:** a technique in which several identical copies or replicas of each relation are maintained and each replica is stored at many distinct sites—used in distributed DBMS
- Resource contention:** the contention over memory space, computing time, and other resources—determines the rate at which a transaction executes between its lock requests
- Rigorous two-phase locking:** a more restrictive variation of two-phase locking technique in which a transaction does not release any of locks (both exclusive and shared) until it commits or aborts
- Role name :** indicates the role of each participating entity type
- Role-based access control (RBAC):** an approach that restricts the system access to the users based on their role
- Rollup operation:** the operation that converts data with a finer-granularity to the coarser-granularity with the help of aggregation
- Safe expression:** an expression which results in a relation with finite number of tuples
- Scaleup:** handling larger task by increasing the degree of parallelism
- Schedule:** a list of operations (such as reading, writing, aborting, or committing) from a set of transactions
- Second normal form (2NF):** a relation schema  $R$  is said to be in Second Normal Form (2NF) if every non-key attribute  $A$  in  $R$  is fully functionally dependent on the primary key.
- Sector:** the smallest unit of information that can be transferred to/from the disk
- Semantic integrity:** a type of data integrity which ensures that the data in the database is logically consistent and complete with respect to the real world
- Semantic query optimization:** a query optimization technique which uses available semantic knowledge to transform a query into a new query that produces the same answer as the original query in any legal database state; however, requires less system resources to execute
- Semistructured data:** data with no predefined schema—also referred to as self-describing data



- Sequential file organization:** a file organization in which records are sorted based on the value of one of its field
- Serial schedule:** a schedule consisting of a sequence of instructions from various transactions, where the operations of one single transaction appear together in that schedule
- Shadow paging:** a technique in which multiple copies of the data item to be modified are maintained on the disk
- Shared (S) lock:** a lock acquired on a data item that allows the user to only read a data item and not modify it
- Shared and intention-exclusive mode (SIX):** a type of lock mode which explicitly locks the subtree rooted by the node in shared mode and the lower level of that subtree is explicitly locked in exclusive-mode
- Shared disk:** an architecture of parallel database systems in which each processor has a private memory and shares only storage disks via an interconnection network
- Shared memory:** an architecture of parallel database systems in which all the processors have access to a common memory and storage disks via an interconnection network (bus or switch)
- Shared-nothing:** an architecture of parallel database systems in which each processor has private memory and one or more private disk storage—no two processors can access the same storage
- Signature file:** a technique in which each document is associated with an index record called signature of the document—each signature is represented by a fixed size of bits called signature width
- Similarity-based retrieval:** a search whereby users can retrieve documents that are similar to a given document
- Single lock manager:** a locking technique in which the system maintains a single lock manager that resides in any single chosen site—applied in distributed systems
- Snowflake schema:** a variation of star schema which may have multiple levels of dimension tables
- Spanned organization:** a way to organize file records on disk blocks—in this organization a record can span more than one disk block
- Specialization:** the process of defining the subgroups of a given entity type
- Speedup:** running a task in less time by increasing the degree of parallelism
- SQL-Java (SQLJ):** a standard that has been used for embedding SQL statements in Java programming language
- Standard Generalized Markup Language (SGML):** a meta-language, that is, a language that allows creating other languages
- Star schema:** the simplest data warehouse schema which consists of a fact table with a single table for each dimension
- Statistical database:** a database containing confidential information about individuals or events, and allows only statistical queries
- Statistical query:** a query that involves statistical functions such as SUM, AVG, COUNT, MIN, MAX, etc.
- Steal:** an approach in which an updated cache page may be written to the disk before the transaction commits
- Stemming:** a process in which all related terms are reduced to the simplest and the most significant form without loss of generality

- Stop words:** a set of most common words such as ‘a’, ‘an’, ‘the’, and so on defined by IR systems—documents are pre-processed to ignore these words while searching
- Storage area network (SAN):** an architecture used by organizations to manage their storage—in this architecture, many server computers are connected with a large number of disks on a high-speed network
- Storage definition language (SDL):** used to define the internal schema
- Stored procedures:** the procedures or functions that are stored and executed by the DBMS at the database server machine
- Strict two-phase locking:** a modification of two-phase locking technique in which a transaction does not release any of its exclusive-mode locks until it commits or aborts
- Structural diagrams:** a category of UML diagrams used to describe the static or structural relationships among various components of the system
- Structured data:** data that is represented in strict format
- Structured literals:** correspond to the values that are constructed using the tuple type constructor
- Structured query language (SQL):** a high-level language that is used for retrieval and management of data stored in relational database
- Structured type :** a form of user-defined type that allows representing complex data types
- Superkey:** a set of one or more attributes, when taken together, helps in uniquely identifying each entity—can have extraneous attributes
- Support:** the percentage or fraction of the population that satisfies both the antecedent and consequent of the rule—if the support is low, it implies that there is no strong evidence that the items in the itemset are bought together
- Symmetric key encryption:** a type of encryption in which same key is used for both encryption and decryption of data
- Tags:** holds a special meaning for web browser and are enclosed in angled brackets, <>
- Technical metadata:** the metadata repository that includes the technical details of the warehouse including storage structures, data description, warehouse operations, etc.
- Term frequency (TF):** represents the number of occurrences of a word in a document
- Third normal form (3NF):** a relation schema R is in the Third Normal Form (3NF) if and only if it satisfies 2NF and every non-key attribute is non-transitively dependent on the primary key
- Three phase commit:** an extension of the two-phase commit protocol that avoids blocking even if the coordinator site fails during recovery
- Throughput:** the number of transactions executed in a given amount of time
- Timestamp ordering:** a serial schedule generated in the order of the timestamp values of the transactions
- Timestamp:** a unique identifier assigned to transactions in the order they begin
- Topological sorting:** the process of ordering the nodes of an acyclic preceding graph
- Total rollback:** the operation in which the transaction is completely rolled back and restarted
- Training data:** the old data that has already been classified by using the domain of the experts’ knowledge—also known as training instances
- Transact-SQL:** core language that is used in SQL Server for data access

**Transaction table:** a table containing a record for each active transaction

**Transaction:** a collection of operations that forms a single logical unit of work

**Trigger:** a type of stored procedure that is executed automatically when some database related events like, insert, update, delete, etc., occur

**Trivial dependency:** a type of dependency which is satisfied by all relations

**Trojan horse:** a computer program that is used to breach the security of a computer system

**Tuple relational calculus:** a form of relational calculus which is based on tuple variables, and takes tuples of a specific relation as its values

**Two-phase locking:** a locking technique which requires that each transaction be divided into two phases—during the first phase, the transaction acquires all the locks; during the second phase, it releases all the locks

**Two-way join:** a join operation applied on only two relations

**Uncertified version:** a version of a data item created in multiversion two-phase locking when an active transaction acquires an exclusive lock on a data item

**Uncommitted modifications:** the data modifications made by the active transactions

**Undo operation:** an operation that reverses (rolls back) the changes made to the database by an uncommitted transaction and restores the database to the consistent state that existed before the start of transaction

**Unified modeling language (UML):** a standard language which was officially defined by the Object Management Group (OMG)—used as one of the techniques to implement object data modeling

**Unnormalized relation:** a relation that contains non-atomic values

**Unspanned organization:** a way to organize the file records on disk blocks—in this organization the records are not allowed to cross the block boundaries

**Unstructured data:** data in which there is very limited indication about the type of data

**Vector space model:** a vector representation of the documents in which each row corresponds to a document and each dimension corresponds to a term in the collection of documents

**Versioning:** an important feature of OODBMS that allows maintaining multiple versions of an object

**Vertical fragmentation:** a fragmentation technique which breaks a relation by decomposing schema of relation

**Victim transaction:** one of the deadlocked transactions chosen to be aborted

**View definition language (VDL):** used to define the external schema

**View:** a virtual relation whose contents are derived from already existing relations—it does not exist in physical form

**Virtual memory manager:** a part of operating system that maps the virtual address generated by a process to the physical address of the disk block

**Volatile storage:** a type of storage that loses its contents when power supply is switched off

**Voting phase:** the first phase of two-phase commit protocol in which the participating sites vote on whether they are ready to commit or not—used in distributed systems

**Write-ahead logging (WAL):** a protocol which states that before making any changes to the database, it is necessary to force-write all the log records on the stable storage

**XML namespace:** defines the set of names that can be used as XML Schema tags

**XML Schema:** defines the structure of XML document by using the same syntax as the XML documents

**XQuery:** World Wide Web Consortium's standard query language for querying XML data

# Index

- Access control, 367
  - discretionary, 367
  - mandatory, 36
  - policies, 7
  - role-based, 367
- Access matrix, 367
- Access, 1
  - creating tables, 3
  - creating relationships, 3
  - forms, 4
  - QBE in (see also Query by example), 116
  - queries in, 99
  - reports, 6
  - SQL in, 132
- ACID properties, 296
  - atomicity, 295
  - consistency, 2
  - durability, 295
  - isolation, 222
- Advanced Encryption Standard, (AES), 378
- Aggregate functions, 112
  - AVG, 112
  - binary, 27
  - COUNT
  - implementation of, 20
  - MAX, 51
  - MIN, 51
  - queries, 3
  - ranking, 286
  - size estimation of, 278
- Analytic functions, 409
  - order by clause, 149
  - over clause
  - partition by clause, 16
  - range clause, 76
- ANSI/SPARC architecture (see also Three-level DBMS architecture), 23
- Application programming interface (API), 176
- Apriori algorithm, 423
- ARIES, 356
- Armstrong's axioms, 186
  - augmentation rule, 187
  - reflexivity rule, 187
  - transitivity rule, 187
- Assertion, 158
- AT attachment (ATA), 217
- Attributes, 28
  - closure of a set of, 209
  - clustering, 21
  - complex, 2
  - composite, 30
  - derived, 4
  - descriptive, 30
  - dimension, 54
  - identifying, 30
  - indexing, 213
  - inheritance, 45
  - measure, 181
  - multivalued, 26
  - NULL values for, 31
  - simple, 1
  - stored, 1
- Audit trail, 382
- Authentication, 371
  - password, 133
  - smart card, 371
- Authority, 369
- Autonomy, 389
- B<sup>+</sup>-trees, 325
- Backflushing, 412
- Basic timestamp ordering, 331
- Bell-LaPadula model, 367
- Binary operations, 105
  - division, 99
- Binary search, 258
- Bit-interleaved parity organisation, 220
- Blind writes, 311
- B-link tree, 325
- Block-interleaved parity organisation, 220
- Boyce-Codd normal form (BCNF), 195
- Brute force algorithm, 258
- B-trees, 243
  - data pointers, 243
  - tree pointers, 243
- Buffer, 223
- Buffer manager, 223
  - pinning, 224
  - unpinning, 224
- Buffer pool, 249
- Cache, 214
- Cache directory, 362
- Canonical cover, 207
- Cascading rollback, 308
- Catastrophic failure, 360
- Checkpoint, 362
- Checkpointing, 351
- Client/server system, 401
- Coarse granularity, 326
- Codd's rules, 68
- Compensation log record (CLR), 363
- Concurrency control techniques, 341
- Conflict serializability, 311
- Conjunctive selection, 291
- Constraints, 27
  - completeness, 49
  - disjoint, 48
  - overlapping, 46
- Coordinator site, 405
- Cost-based query optimisation, 272
  - equivalence rules, 272
  - estimating statistics, 278
  - join ordering (see also Join), 253
- Covert channel, 382
- Crabbing, 342
- Crawl, 445
- Critical section, 343
- Cursors, 166
- Data, 1
  - Data abstraction, 23
  - Data cleansing, 3
  - Data contention, 342
  - Data cube, 2
  - Data definition language (DDL), 457
    - compiler, 15
  - Data dictionary, 23
  - Data Guard, 501
  - Data inconsistency, 23
  - Data independence, 23
    - logical, 3
    - physical, 4
  - Data integrity, 76
    - domain, 76
    - entity, 12
    - referential, 16
    - semantic, 27
  - Data manipulation language (DML), 15
    - compiler, 15
    - query language, 5
  - Data mining, 420
    - application areas of, 409

- association rules, 409
- classification, 409
- clustering, 21
- knowledge discovery, 409
- Data redundancy (see also Redundancy), 23
- Data security, 4
- Data sharing, 4
- Data source, 484
- Data stripping, 218
  - bit-level, 218
  - block-level, 219
- Data transparency, 392
- Data warehouse, 409
  - architecture, 4
  - components of, 26
  - multidimensional modelling, 413
  - schemas, 8
- Data warehousing, 410
  - characteristics of, 54
- Database, 23
  - availability, 5
  - backup, 3
  - design process, 20
  - instance, 2
  - integrity, 3
  - knowledge discovery in (see also Knowledge discovery in databases), 421
  - models, 5
  - schema (see also Schema), 7
  - standby, 493
  - statistical, 17
- Database administrator (DBA), 7
- Database audit, 382
- Database engine, 497
- Database graph, 342
- Database management system (DBMS), 23
  - advantages of, 21
  - application areas of, 409
  - architecture, 4
  - classification of, 19
  - component modules of, 1
  - distributed (see also Distributed DBMS), 386
  - languages of (see also DBMS languages), 1
  - parallel (see also Parallel DBMS), 386
  - users of, 17
- Database system, 2
  - centralised, 3
  - client/server, 18
  - distributed, 5
  - online transaction processing (see also Online transaction processing), 20
- DBMS catalog, 4
- DBMS languages, 15
  - data definition (see also Data definition language), 132
  - data manipulation (see also Data manipulation language), 15
- Deadlock, 315
  - dealing with, 178
  - detection and recovery, 315
  - handling, 133
  - phantom, 309
  - prevention, 338
- Decision tree, 432
- Decision-support systems (DSS), 409
- Decomposition, 183
  - attribute preservation, 184
  - binary, 34
  - dependency preservation, 181
  - lossless, 181
  - lossy, 197
- Decryption, 383
  - algorithm, 379
  - key, 27
- Deferred update, 362
- Denormalisation, 205
- Digital signatures, 379
- Dimension table, 432
- Dirty data, 299
- Dirty read, 311
- Discretionary access control (DAC), 372
- Disjunctive selection, 291
- Disk (see also Magnetic disks), 216
- Disk controller, 249
  - checksums, 217
- Distributed DBMS (DDBMS), 386
  - distributed computing, 388
  - heterogeneous, 6
  - homogeneous, 19
- Document representation, 438
- Document type definition (DTD), 484
- Domain, 57
  - structured, 10
  - unstructured, 65
- Encryption, 377
  - algorithm, 187
  - key, 27
  - private key, 372
  - public key, 378
  - symmetric key, 378
- Enhanced E-R model, 58
  - aggregation, 49
  - generalisation, 45
  - specialisation, 45
- Entity set, 57
- Entity type, 57
  - instance, 2
  - strong, 32
  - weak, 32
- Entity-relationship (E-R) diagram, 36
  - alternative notations of, 27
  - design choices, 43
  - naming conventions, 37
  - notations, 35
  - representation of, 8
- Entity-relationship (E-R) model, 57
  - attributes (see also Attributes), 8
  - entity, 11
  - relationship (see also Relationships), 8
- Error-correcting code (ECC), 220
- ETL process, 432
- Extensible markup language (XML), 484
  - access control language, 377
  - attribute, 12
  - comments, 474
  - data (see also XML data), 469
  - element, 79
  - namespace, 164
  - overview, 20
  - schema, 7
  - uses of, 482
- Extents, 457
- External sort-merge algorithm, 256
- Fact table, 432
- Fifth normal form, 207
- File organisation, 249
  - hash, 232
  - heap, 213
  - sequential, 167
- File-processing system, 23
- Fine granularity, 342
- First normal form, 190
- Fixed-length records, 225
  - file header, 226
  - free list, 226
- Flash memory, 249
- FLWOR expression, 484
- Fourth normal form, 202
- Fragmentation, 390
  - horizontal, 54
  - mixed, 391
  - vertical, 54
- Functional dependency, 207
  - Armstrong's axioms (see also Armstrong's axioms), 186
  - canonical cover, 188
  - equivalence of sets of, 188
  - full, 111
  - higher normal forms (see also Higher normal forms), 199
  - nontrivial, 186
  - partial, 32

- Functional dependency (*Continued*)
  - transitive, 194
  - trivial, 186
- Fuzzy checkpointing, 351
- Fuzzy dump technique, 360
- Fuzzy lookup, 432
- Garbage collection, 363
- Graph-based locking, 323
- Greedy search, 432
- Grid files, 250
- Hash file organisation, 232
- Hash functions, 249
  - cut key hashing, 232
  - division-remainder hashing, 232
  - folded key hashing, 249
- Hash join, 291
  - hybrid, 265
  - recursive partitioning, 266
  - skewed partitioning, 266
- Hashing, 249
  - buckets, 233
  - chained overflow, 233
  - collision, 232
  - dynamic, 53
  - extendible, 235
  - hash field, 232
  - hash function (see also Hash functions), 232
  - hash key, 232
  - hash table, 232
  - linear, 230
  - multiple, 3
  - open addressing, 233
  - partitioned, 16
  - static, 53
- Heap file organisation, 229
- Heuristic query optimisation, 282
  - heuristics, 282
- Hierarchical data model, 11
- Higher normal forms, 199
  - fifth, 204
  - fourth, 115
- HITS algorithm, 445
- Hot spots, 343
- Hub, 445
- Hybrid OLAP, 432
- Hyper-text markup language (HTML), 469
- IBM DB2, 497
  - Control Center, 498
  - features of, 385
- Immediate update, 353
- Index, 250
  - B\*-trees, 325
  - B-trees (see also B-trees), 243
    - clustering (see also Attributes), 21
    - dense, 238
    - inverted, 440
    - multilevel, 241
    - on multiple keys, 213
    - ordered, 65
    - primary, 32
    - scan, 165
    - secondary, 214
    - signature file, 441
    - single-level, 248
    - sparse, 238
    - tree-structured, 323
  - Indexed sequential access method (ISAM), 245
  - Indexing, 250
    - field, 478
  - Information, 443
  - Information retrieval (IR), 436
  - Inheritance, 452
    - table, 16
    - type, 3
  - In-place updating, 361
  - Integrity constraints, 95
    - check, 80
    - foreign key, 39
    - not null, 80
    - primary key, 32
    - specifying, 16
    - unique, 2
  - Inverse relationship, 466
  - Java database connectivity (JDBC), 170
  - Join, 23
    - block nested-loop, 263
    - equijoin, 108
    - hash (see also Hash join), 265
    - hybrid merge, 265
    - indexed nested-loop, 264
    - left-deep, 285
    - multiway, 262
    - naive, 262
    - natural, 110
    - nested-loop, 262
    - nonadditive, 198
    - non-left-deep, 285
    - ordering, 14
    - outer, 99
    - queries, 3
    - semi, 469
    - simple, 1
    - sort-merge, 256
    - two-way, 256
- Join dependency, 207
- Key, 72
  - alternate, 73
  - apparent, 184
  - candidate, 31
  - composite, 30
  - foreign, 39
  - partial, 20
  - primary, 32
  - superkey, 31
- k-means algorithm, 432
- Linear search, 291
- Lock, 316
  - exclusive, 316
  - intention, 327
  - intention-exclusive (IX), 327
  - intention-shared (IS), 327
  - shared, 3
  - shared and intention-exclusive (SIX), 328
  - table, 16
- Lock-based technique, 318
- Lock compatibility, 316
- Lock conversion, 322
  - downgrading, 322
  - upgrading, 8
- Lock manager, 342
  - distributed, 5
  - single, 11
- Locking, 342
  - distributed, 5
  - granularity, 326
  - graph-based, 323
  - implementation of, 20
  - index, 213
  - lock (see also Lock), 315
  - majority, 333
  - multiple-granularity, 326
  - performance of, 17
  - predicate, 63
  - two-phase (see also Two-phase locking), 321
- Log (see System log), 362
- Log force, 362
- Log record buffering, 362
- Log sequence number, 363
- Lost update, 311
- Magnetic disks, 216
  - access time, 218
  - cylinder, 216
  - data transfer rate, 218
  - head-disk assemblies, 216
  - mean time to failure, 218
  - mean time to repair, 219
  - read-write head, 216
  - rotational delay time, 217

- sector, 216
- seek time, 217
- spindle, 216
- tracks, 216
- Main memory, 214
- Mandatory access control (MAC), 373
- Materialised evaluation expression
  - tree, 270
- Metadata, 23
  - business, 1
  - repository, 410
  - technical, 3
- Microsoft, 495
  - Access (see also Access), 64
  - SQL Server (see also SQL Server), 64
- Multidimensional data, 432
- Multidimensional OLAP, 432
- Multilevel index, 250
- Multiple granularity locking, 342
- Multiversion technique, 336
  - timestamp ordering, 331
  - two-phase locking, 321
- Nested query, 176
- Network data model, 12
- Object based database, 448
  - need of, 2
- Object query language (OQL), 462
  - path expression, 462
- Object relational database, 449
- Object-based data model, 13
  - object-oriented, 5
  - object-relational, 5
- Object-oriented database, 455
  - characteristics of, 54
- ODMG object model, 456
- Online analytical processing (OLAP), 415
  - dicing, 418
  - drill down, 419
  - hybrid, 265
  - multidimensional, 413
  - rollup, 419
  - slicing, 418
- Online transaction processing (OLTP), 432
- Open database connectivity (ODBC), 169
- Oracle, 491
  - features of, 13
  - history, 300
- Page replacement policies, 224
  - clock replacement, 224
  - least recently used (LRU), 224
  - most recently used (MRU), 224
- Page table, 362
  - current, 5
  - dirty, 223
  - shadow, 349
- Parallel DBMS, 387
  - shared disk, 387
  - shared memory, 387
  - shared nothing, 387
- Parallel processing, 405
- Participating site, 405
- Partitioned hashing, 250
- Path expression, 466
- Persistence, 465
- Phantom deadlock, 405
- Phantom problem, 342
- Pipelined evaluation, 271
  - demand-driven pipeline, 290
  - pipeline, 271
  - producer-driven pipeline, 290
- Pivoting, 432
- Polyinstantiation, 382
- PostgreSQL, 487
  - features of, 13
  - history, 300
- Precedence graph, 311
  - acyclic, 304
  - cyclic, 304
- Precision, 445
- Primary site, 363
- Project operation, 101
  - implementation of, 20
- Query, 23
  - action, 21
  - blocks, 219
  - evaluation engine, 17
  - evaluation plan, 255
  - processing (see also Query processing), 2
  - statistical, 17
  - tree (see also Query tree), 11
- Query evaluation engine, 291
- Query evaluation plan, 291
- Query language, 23
- Query processing, 291
  - distributed, 5
  - steps, 20
- Query tree, 291
  - conversion of, 170
  - left-deep join, 285
  - non-left-deep join, 285
- Query-rewrite system, 501
- RAID (see Redundant arrays of independent disks), 249
- Recall, 445
- Recovery manager, 362
- Relation, 95
  - binary, 34
  - characteristics of, 54
  - instance (see also Relation instance), 2
  - multilevel, 241
  - n-ary, 36
  - referenced, 73
  - referencing, 73
  - schema (see also Schema), 7
  - ternary, 34
  - unary, 64
- Relation instance, 95
  - cardinality, 35
  - degree, 4
- Relational algebra, 100
  - operations (see also Unary and Binary operations), 2
- Relational calculus, 116
  - domain relational calculus (DRC), 126
  - tuple relational calculus (TRC), 126
- Relational data model, 12
- Relational database, 95
  - instance, 2
  - schema, 7
- Relational database design, 181
  - features of, 13
- Relational model, 95
  - integrity constraints (see also Integrity constraints), 3
  - manipulative component, 94
  - structural component, 94
- Relationships, 32
  - degree, 4
  - identifying, 30
  - instance, 2
  - recursive, 34
  - type, 3
- Relevance feedback, 445
- Remote backup, 363
- Replication, 391
  - full, 111
  - partial, 20
- Resource contention, 342
- Rijndael algorithm, 383
- RSA algorithm, 379
- Second normal form, 192
- Secondary site, 363
- Security issues, 367
- Select operation, 100
  - implementation of, 20
- Semistructured data model, 13
- Set operations, 105
  - cartesian product, 105



- Set operations (*Continued*)
  - difference, 12
  - implementation of, 20
  - intersection, 105
  - union, 16
- Shadow paging, 354
- Similarity-based retrieval, 439
- Single-level index, 238
- Sorting, 267
  - external, 8
  - internal, 8
- SQL-Java (SQLJ), 172
- Starvation, 342
- Stemming, 445
- Storage area network (SAN), 249
- Storage devices, 248
  - primary, 32
  - secondary, 214
  - tertiary, 214
- Stored procedures, 161
- Strict timestamp ordering, 332
- Structured data, 484
- Structured type, 449
  - type constructor, 450
- System failures, 6
  - type of, 3
- System log, 349
  - log records, 349
- Tags, 484
- TF/IDF based ranking, 438
  - inverse document frequency (IDF), 444
  - term frequency (TF), 444
- Third normal form, 194
- Thomas' write rule, 332
- Thrashing, 342
- Three phase commit, 401
- Three-level DBMS
  - architecture, 23
  - conceptual, 8
  - external, 8
  - internal, 8
- Timestamp, 330
  - ordering, 14
  - read, 37
  - write, 3
- Timestamp-based technique, 330
- Timestamping, 396
- Topological sorting, 311
- Tree-locking, 342
- Triggers, 163
  - instead of, 5
  - row level, 164
  - statement level, 164
- Trojan horse, 382
- Tuple, 95
- Two-phase commit, 399
- Two-phase locking, 320
  - multiversion (see also Multiversion technique), 315
  - rigorous, 322
  - strict, 15
- Unary operations, 100
  - project, 16
  - rename, 104
  - select, 16
- Uncommitted modification, 362
- Unrepeatable read, 311
- Unstructured data, 484
- Variable-length records, 227
  - slotted page structure, 229
- Vector space model, 445
- Versioning, 465
  - View, 23
    - complex, 2
    - creating, 3
    - materialized, 160
- View serializability, 311
- Volatile storage, 248
- Wait-die technique, 343
- Wait-for graph, 343
  - global, 235
  - local, 18
- Web search engines, 442
- Wound-wait technique, 343
- Write-ahead logging, 356
- XML access control language (XACL), 382
- XML Schema, 475
- XPath, 479
- XQuery, 480